ABC 079 解説

writer: namonakiacc

平成 29 年 11 月 18 日

For international readers: English Editorial starts at page 3.

A: Good Integer

N は 4 桁の整数なので、3 つ以上の同じ数字が連続して並ぶ時、前の 3 つの数字か、後の 3 つの数字の少なくともどちらかは必ず等しくなります。

よって、この 2 つの条件の少なくともどちらかを満たしたときに ‘Yes‘、そうでないときに ‘No‘ を出力するとよいです。

```
#include <cstdio>
char s[5];
int main()
{
        scanf("%s",&s);
        if(s[0]==s[1]&&s[1]==s[2])puts("Yes");
        else if(s[1]==s[2]&&s[2]==s[3])puts("Yes");
        else puts("No");
}
```

B: Lucas Number

リュカ数の定義 ($L_0 = 2$, $L_1 = 1$, $L_i = L_{i-1} + L_{i-2} (i \geqq 2)$) に従って計算すると求まります。

ただし、答えが 32 bit に収まらない場合があるので注意してください。

```
#include <cstdio>
typedef long long ll;
int N;
ll L[87];
int main()
{
        scanf("%d",&N);
        L[0]=2LL,L[1]=1LL;
        for(int i=2;i<=N;i++)L[i]=L[i-2]+L[i-1];
        printf("%lld\n",L[N]);
}
```

C: Train Ticket

$op1$,$op2$,$op3$ の決め方は高々 8 通りであるため、これらを全て試して当てはまるものがあれば 1 つ出力するとよいです。

```
#include <cstdio>
char s[5];
int main()
{
        scanf("%s",&s);
        int a=s[0]-'0',b=s[1]-'0',c=s[2]-'0',d=s[3]-'0';
        if(a-b-c-d==7)printf("%d-%d-%d-%d=7\n",a,b,c,d);
        else if(a-b-c+d==7)printf("%d-%d-%d+%d=7\n ",a,b,c,d);
        else if(a-b+c-d==7)printf("%d-%d+%d-%d=7\n",a,b,c,d);
        else if(a-b+c+d==7)printf("%d-%d+%d+%d=7\n",a,b,c,d);
        else if(a+b-c-d==7)printf("%d+%d-%d-%d=7\n",a,b,c,d);
        else if(a+b-c+d==7)printf("%d+%d-%d+%d=7\n",a,b,c,d);
        else if(a+b+c-d==7)printf("%d+%d+%d-%d=7\n",a,b,c,d);
        else if(a+b+c+d==7)printf("%d+%d+%d+%d=7\n",a,b,c,d);
}
```

D: Wall

初めに、すべての数字において、1 に変えるのにかかるコストを計算しておきます。

これには、ワーシャルフロイド法などが使えるでしょう。

その後、全てのマスにおいて、数字が書かれている場合の 1 への変更コストの和を求めるとよいです。

```
#include <cstdio>
#define rep(i,n) for(int (i)=0;(i)<(n);(i)++)
int c[10][10];
int H,W,A,res;
int main()
{
        scanf("%d%d",&H,&W);
        rep(i,10)rep(j,10)scanf("%d",&c[i][j]);
        rep(k,10)rep(i,10)rep(j,10)
                if(c[i][j]>c[i][k]+c[k][j])c[i][j]=c[i][k]+c[k][j];
        rep(i,H)rep(j,W)
        {
                scanf("%d",&A);
                if(A>=0)res+=c[A][1];
        }
        printf("%d\n",res);
}
```

A: Good Integer

N has 4 digits, so first 3 or last 3 digits are same when N is **good**.

So, you can get 100 points by output 'Yes' when at least one of these conditions are met, 'No' when they didn't.

```
#include <cstdio>
char s[5];
int main()
{
        scanf("%s",&s);
        if(s[0]==s[1]&&s[1]==s[2])puts("Yes");
        else if(s[1]==s[2]&&s[2]==s[3])puts("Yes");
        else puts("No");
}
```

B: Lucas Number

You can solve it by calculate with the definition of Lucas Number($L_0 = 2$, $L_1 = 1$, $L_i = L_{i-1} + L_{i-2} (i \geqq 2)$).

Pay attention to the case when the answer can't be expressed in 32 bit.

```
#include <cstdio>
typedef long long ll;
int N;
ll L[87];
int main()
{
        scanf("%d",&N);
        L[0]=2LL,L[1]=1LL;
        for(int i=2;i<=N;i++)L[i]=L[i-2]+L[i-1];
        printf("%lld\n",L[N]);
}
```

C: Train Ticket

There are no more than 8 pattern to decide $op1$,$op2$,$op3$,so you can solve it by try all patterns and output one case which is answer.

```
#include <cstdio>
char s[5];
int main()
{
        scanf("%s",&s);
        int a=s[0]-'0',b=s[1]-'0',c=s[2]-'0',d=s[3]-'0';
        if(a-b-c-d==7)printf("%d-%d-%d-%d=7\n",a,b,c,d);
        else if(a-b-c+d==7)printf("%d-%d-%d+%d=7\n ",a,b,c,d);
        else if(a-b+c-d==7)printf("%d-%d+%d-%d=7\n",a,b,c,d);
        else if(a-b+c+d==7)printf("%d-%d+%d+%d=7\n",a,b,c,d);
        else if(a+b-c-d==7)printf("%d+%d-%d-%d=7\n",a,b,c,d);
        else if(a+b-c+d==7)printf("%d+%d-%d+%d=7\n",a,b,c,d);
        else if(a+b+c-d==7)printf("%d+%d+%d-%d=7\n",a,b,c,d);
        else if(a+b+c+d==7)printf("%d+%d+%d+%d=7\n",a,b,c,d);
}
```

D: Wall

First, calculate cost to turn digits to 1 ,in 0,2,3,...,9.

You can do it,for example,by Warshall Floyd algorithm.

Then, you can solve it by calculate the sum of the cost to turn digits to 1,in every square which contain a digit.

```
#include <cstdio>
#define rep(i,n) for(int (i)=0;(i)<(n);(i)++)
int c[10][10];
int H,W,A,res;
int main()
{
        scanf("%d%d",&H,&W);
        rep(i,10)rep(j,10)scanf("%d",&c[i][j]);
        rep(k,10)rep(i,10)rep(j,10)
                if(c[i][j]>c[i][k]+c[k][j])c[i][j]=c[i][k]+c[k][j];
        rep(i,H)rep(j,W)
        {
                scanf("%d",&A);
                if(A>=0)res+=c[A][1];
        }
        printf("%d\n",res);
}
```

ABC 080 解説

writer: namonakiacc

平成 29 年 12 月 3 日

For international readers: English Editorial starts at page 4.

A: Parking

プラン 1 を選んで駐車した場合、駐車料金は $A \times N$ 円となり、プラン 2 を選んで駐車した場合、駐車料金は B 円となるので、$A \times N$ と B の小さいほうを出力するとよいです。

```
#include <cstdio>
int N,A,B;
int main()
{
        scanf("%d%d%d",&N,&A,&B);
        if(N*A<=B)printf("%d\n",N*A);
        else printf("%d\n",B);
}
```

B: Harshad Number

$f(N)$ の計算方法だが、例えば以下の方法が考えられる。

1: $t = N$、$c = 0$ とする。

2: c に、t を 10 で割った余りを加算する。

3: t に、t を 10 で割った商を代入する。

4: $t > 0$ ならば 2 に戻る。そうでないなら終了する。このとき $c = f(N)$ となる。

$f(N)$ を求めた後、N が $f(N)$ で割り切れるかを判定すればよい。

```
#include <cstdio>
int N;
int main()
{
        scanf("%d",&N);
        int t=N,c=0;
        while(t>0)c+=t%10,t/=10;
        if(N%c==0)puts("Yes");
        else puts("No");
}
```

C: Shopping Street

店を営業する時間帯の組み合わせは $2^{10} - 1 = 1023$ 通りなため、これら全ての場合において利益を計算し、最大値を出力するとよいです。

```
#include <cstdio>
int N;
int F[100][14];
int P[100][15];
int main()
{
        scanf("%d",&N);
        for(int i=0;i<N;i++)
        {
                for(int j=0;j<10;j++)scanf("%d",&F[i][j]);
        }
        for(int i=0;i<N;i++)for(int j=0;j<11;j++)scanf("%d",&P[i][j]);
        int res=-(1<<30);
        for(int b=1;b<(1<<10);b++)
        {
                int cc=0;
                for(int i=0;i<N;i++)
                {
                        int c=0;
                        for(int j=0;j<10;j++)if((b>>j&1)&&F[i][j])c++;
                        cc+=P[i][c];
                }
                if(res<cc)res=cc;
        }
        printf("%d\n",res);
}
```

D: Recording

録画機をあるチャンネルの時刻 S から時刻 T までの録画に使用すると、時刻 $S-0.5$ から時刻 T まで他のチャンネルの録画に使用できませんが、このとき時刻 $S-0.5$ から時刻 T まで録画機を使用していると考えることとします。

まず、同じチャンネルの連続した番組は同じ録画機で一緒に録画した方が録画機の使用時間が短くなるので、そのように考えます。すると、個々のチャンネルにおいて、録画機を使用する時間を最小にできます。

その後、各時間において、いくつの録画機が使用中であるかを考えると、その最大値が答えとなります。

```
#include <cstdio>
int N,C;
int s[100000],t[100000],c[100000];
int sm[200002];
int tt[200002];
int main()
{
        scanf("%d%d",&N,&C);
        for(int i=0;i<N;i++)
                scanf("%d%d%d",&s[i],&t[i],&c[i]);
        for(int i=1;i<=C;i++)
        {
                for(int j=0;j<200002;j++)tt[j]=0;
                for(int j=0;j<N;j++)if(c[j]==i)tt[s[j]*2-1]++,tt[t[j]*2]--;
                for(int j=1;j<200002;j++)tt[j]+=tt[j-1];
                for(int j=0;j<200002;j++)if(tt[j]>0)sm[j]++;
        }
        int mx=0;
        for(int j=0;j<200002;j++)if(mx<sm[j])mx=sm[j];
        printf("%d\n",mx);
}
```

A: Parking

When you choose plan 1 ,the fee will be $A \times N$ yen,and when you choose plan 2, the fee will be B yen.

So,it is good to print the smaller of $A \times N$ and B.

```
#include <cstdio>
int N,A,B;
int main()
{
        scanf("%d%d%d",&N,&A,&B);
        if(N*A<=B)printf("%d\n",N*A);
        else printf("%d\n",B);
}
```

B: Harshad Number

For example,$f(N)$ can be calculated as such.

1: Set $t = N$,$c = 0$.

2: Add to c ,the remainder of t divided by 10.

3: Substitute the quotient of t divided by 10 ,for t.

4: If $t > 0$,go 2. If $t = 0$,quit. Then $c = f(N)$.

After that, it is good to check is N can be divided by $f(N)$ or not.

```
#include <cstdio>
int N;
int main()
{
        scanf("%d",&N);
        int t=N,c=0;
        while(t>0)c+=t%10,t/=10;
        if(N%c==0)puts("Yes");
        else puts("No");
}
```

C: Shopping Street

There are $2^{10}-1=1023$ pattern to decide how to open a shop,so it is good to calculate profit in every pattern and print the maximum one.

```
#include <cstdio>
int N;
int F[100][14];
int P[100][15];
int main()
{
        scanf("%d",&N);
        for(int i=0;i<N;i++)
        {
                for(int j=0;j<10;j++)scanf("%d",&F[i][j]);
        }
        for(int i=0;i<N;i++)for(int j=0;j<11;j++)scanf("%d",&P[i][j]);
        int res=-(1<<30);
        for(int b=1;b<(1<<10);b++)
        {
                int cc=0;
                for(int i=0;i<N;i++)
                {
                        int c=0;
                        for(int j=0;j<10;j++)if((b>>j&1)&&F[i][j])c++;
                        cc+=P[i][c];
                }
                if(res<cc)res=cc;
        }
        printf("%d\n",res);
}
```

D: Recording

When you use recorder from time S to time T in a channel,this recorder can not record other channel from time $S-0.5$ to time T ,so it is OK to think as using the recorder from time $S-0.5$ to time T.

First,continuous program from the same channel should be better to record together in the same recorder,so think as such.Then,the time of using the recorder will be shortest in every channel.

After that,considering which number of record is need at every time.Then,the maximum number of that is the answer.

```
#include <cstdio>
int N,C;
int s[100000],t[100000],c[100000];
int sm[200002];
int tt[200002];
int main()
{
        scanf("%d%d",&N,&C);
        for(int i=0;i<N;i++)
                scanf("%d%d%d",&s[i],&t[i],&c[i]);
        for(int i=1;i<=C;i++)
        {
                for(int j=0;j<200002;j++)tt[j]=0;
                for(int j=0;j<N;j++)if(c[j]==i)tt[s[j]*2-1]++,tt[t[j]*2]--;
                for(int j=1;j<200002;j++)tt[j]+=tt[j-1];
                for(int j=0;j<200002;j++)if(tt[j]>0)sm[j]++;
        }
        int mx=0;
        for(int j=0;j<200002;j++)if(mx<sm[j])mx=sm[j];
        printf("%d\n",mx);
}
```

ABC #082 / ARC #087 Editorial

writer : sugim48

2017 年 12 月 16 日

For International Readers: English editorial starts on page 5.

A: Round Up the Mean

- C++ のコード例 : https://beta.atcoder.jp/contests/abc082/submissions/1872729
- Java のコード例 : https://beta.atcoder.jp/contests/abc082/submissions/1872732
- Python 3 のコード例 : https://beta.atcoder.jp/contests/abc082/submissions/1872734

B: Two Anagrams

s の文字を昇順にソートし，t の文字を降順にソートした後，$s < t$ を判定すればよいです．

- C++ のコード例 : https://beta.atcoder.jp/contests/abc082/submissions/1872811
- Java のコード例 : https://beta.atcoder.jp/contests/abc082/submissions/1872812
- Python 3 のコード例 : https://beta.atcoder.jp/contests/abc082/submissions/1872814

C: Good Sequence

それぞれの正の整数 x について，a 中の値 x の個数を n_x とします．$n_x < x$ の場合，a 中の値 x はすべて取り除かなければならないので，n_x 回の操作が必要です．$n_x \geq x$ の場合，a 中の値 x の個数を x にすればよいので，$n_x - x$ 回の操作が必要です．すべての x についてこれらの操作回数を合計したものが答えです．

実際には，すべての x ではなく，a 中の値 x のみ調べればよいです．x の値は最大で 10^9 と大きいので，普通の配列の代わりに連想配列を用いて各 n_x を数えればよいです．

- C++ のコード例 : https://beta.atcoder.jp/contests/abc082/submissions/1872852

D: FT Robot

s の長さを N とし，目標の座標を (x_t, y_t) とします．

まず思いつく解法は，$\mathrm{dp}_{i,x,y,k} :=$ (i 文字目の命令後に座標 (x, y) に向き $k \in \{$ 左, 上, 右, 下 $\}$ でいられるか)

と定義し，DP をする解法です．しかし，この DP の状態数は $\mathrm{O}(N^3)$ で，遷移は $\mathrm{O}(1)$ なので，全体の時間計算量は $\mathrm{O}(N^3)$ となり，TLE してしまいます．

ロボットの動き方をよく観察することで，DP の状態数を減らしましょう．命令列を `T` を区切り文字として分割したとき，i 番目の区間に含まれる `F` の個数を d_i とします．このとき，ロボットの動き方は

- 右に距離 d_1 だけ動く
- 上または下の好きな向きに距離 d_2 だけ動く
- 左または右の好きな向きに距離 d_3 だけ動く
- 上または下の好きな向きに距離 d_4 だけ動く
- 左または右の好きな向きに距離 d_5 だけ動く
- 以下同様

となっています．よって，x 軸方向の移動と y 軸方向の移動を独立に考えることができます．例えば，x 軸方向の移動については，

- 右に距離 d_1 だけ動く
- 左または右の好きな向きに距離 d_3 だけ動く
- 左または右の好きな向きに距離 d_5 だけ動く
- 以下同様

という移動を終えた後，x 座標が x_t になっているようにできるか判定すればよいです．

これは次のような DP で計算できます．

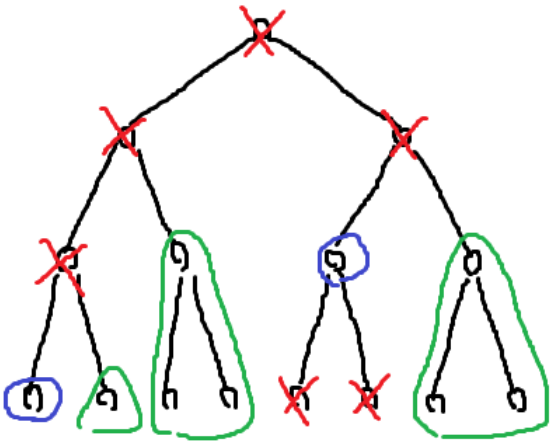
- 定義: $\mathrm{dp}_{i,x} :=$ (i 回目の x 軸方向の移動の後に x 座標が x になっているようにできるか).
- 初期化: 1 回目の x 軸方向の移動距離を d_1 とすると，$\mathrm{dp}_{0,d_1} = \mathrm{true}$.
- 漸化式: i ($i \geq 2$) 回目の x 軸方向の移動距離を d_i とすると，$\mathrm{dp}_{i,x} = \mathrm{dp}_{i-1,x-d_i} \vee \mathrm{dp}_{i-1,x+d_i}$.

この DP の状態数は $\mathrm{O}(N^2)$ で，遷移は $\mathrm{O}(1)$ なので，全体の時間計算量は $\mathrm{O}(N^2)$ となり，十分に高速です．y 軸方向の移動についても，同様の DP で計算できます．

E: Prefix-free Game

長さ L 以下の `0`, `1` のみからなる文字列全体からトライ木を作ると，これは高さ $L+1$ の完全二分木となります．ただし，完全二分木の高さは，根から葉までのパスに含まれる頂点の個数と定義します．この完全二分木を用いると，ゲームは次のように言い換えられます．

> 高さ $L+1$ の完全二分木がある．最初，いくつかの頂点には駒が置かれている．ただし，どの駒どうしも先祖-子孫の関係ではない．Alice と Bob は交互に，駒のない頂点をひとつ選んで駒を置く．ただし，どの駒どうしも先祖-子孫の関係ではないという条件を保つようにする．先に駒を置けなくなった方が負けである．



上図は初期状態の例 ($L = 3$) です．青いマル印は駒を表しており，赤いバツ印は駒を置けない頂点を表しています．このとき，駒を置ける頂点の集合は，いくつかの完全二分木 (緑の三角形) の和になっています．これらの完全二分木は先祖-子孫の関係ではないので，独立に考えることができます．また，完全二分木のある頂点に駒を置くと，その完全二分木はいくつかの完全二分木に分裂します．具体的には，高さ i の完全二分木の深さ j $(0 \leq j \leq i-1)$ の頂点に駒を置くと，高さ $i-1, i-2, ..., i-j$ の完全二分木 1 個ずつに分裂します．

以上より，ゲームはさらに次のように言い換えられます．

> 最初，いくつかのアイテムがある．各アイテムには正整数のレベルが設定されている．Alice と Bob は交互に次の操作を行う．
>
> - アイテムを 1 個選ぶ．選んだアイテムのレベルを i とする．選んだアイテムを取り除き，整数 k $(1 \leq k \leq i)$ を選んでレベル $i-1, i-2, ..., k$ のアイテムを 1 個ずつ追加する．
>
> 先に操作を行えなくなった方が負けである．

このようなゲームの勝敗判定には Grundy 数が有用です．レベル i $(i \geq 1)$ のアイテムの Grundy 数を g_i とします．すると，g_i は

- 0
- g_{i-1}
- $g_{i-1} \oplus g_{i-2}$
- $\vdots$
- $g_{i-1} \oplus g_{i-2} \oplus \cdots \oplus g_1$

に含まれない最小の非負整数です．実験すると，$(g_1, g_2, g_3, g_4, g_5, g_6, g_7, g_8, \ldots) = (1, 2, 1, 4, 1, 2, 1, 8, \ldots)$ となっています．実験結果から分かるように，$g_i =$ (i を割り切る最大の 2 の冪) であることが示せます．各アイテムの Grundy 数の総 XOR が 0 でないならば Alice が勝ち，0 ならば Bob が勝ちます．

元の問題に戻ると，レベル i のアイテムは高さ i の平衡二分木に対応します．よって，初期状態におけるそれぞれの完全二分木の高さを求められればよいことになります．これは，入力文字列から作られるトライ木を辿り，子の個数がちょうど 1 個の頂点を見ていくことで可能です．

F: Squirrel Migration

まず，移動距離の総和を最大化するような移動方法がどのようなものか考えます．そのために，移動距離の総和の最大値がどのような値か考えます．まずは，移動距離の総和の上限を見積もってみましょう．ある移動方法に対して，各辺 e について e を通過したリスの個数を s_e とします．すると，移動距離の総和は $\sum_e s_e$ となります．ここで，辺 $e=(v,w)$ の v 側の頂点数を n_v とし，w 側の頂点数を n_w とすると，$s_e \le 2\min\{n_v, n_w\}$ です．よって，移動距離の総和の上限はこれらの総和となります．実は，この上限を達成するような移動方法が常に存在します．それは次のような移動方法です．

木の重心に注目します．重心は 1 個または 2 個です．重心が 1 個の場合，それを c とします．すると，どのリスの移動経路にも c が含まれるとき，かつそのときに限り，移動距離の総和は上限を達成することが分かります．また，重心が 2 個の場合，それらを c_1, c_2 とします．すると，どのリスの移動経路にも 辺 (c_1, c_2) が含まれるとき，かつそのときに限り，移動距離の総和は上限を達成することが分かります．

以上の移動方法を数え上げます．重心が 2 個の場合の方が簡単なので，先に考えます．重心を c_1, c_2 とします．さらに，c_1 側の部分木を T_1 とし，c_2 側の部分木を T_2 とします．このとき，木の頂点数 N は偶数であり，T_1, T_2 の頂点数はともに $N/2$ です．T_1 の各リスは，T_2 のどの頂点へも移動できます．同様に，T_2 の各リスは，T_1 のどの頂点へも移動できます．よって，移動方法は $(N/2)! \times (N/2)!$ 通りです．

次に，重心が 1 個の場合を考えます．重心を c とします．さらに，c の周りの部分木を $T_1, T_2, ..., T_K$ とします．このとき，c のリスはどの頂点へも移動できます．また，各 i について，T_i のリスは T_i 以外のどの頂点へも移動できます．この移動方法を $\mathrm{O}(N^2)$ 時間で直接数え上げるのは難しいです．

そこで，包除原理を用います．$T := T_1 \cup T_2 \cup \cdots \cup T_K$ とします (すなわち，T は木の頂点集合から c を除いたもの)．すると，答えは $\sum_{S \subseteq T} (-1)^{|S|} f_S$ となります．ただし，$f_S :=$ (「S 中の各リスは自分の部分木の頂点へ移動する」という条件を満たす移動方法の通り数) です．各 $S_1 \subseteq T_1$, ..., $S_K \subseteq T_K$ について，次が成り立ちます．

$$f_{S_1 \cup \cdots \cup S_K} = \left(N - \sum_{k=1}^{K} |S_k|\right)! \prod_{k=1}^{K} \binom{|T_k|}{|S_k|}^2 |S_k|!$$

よって，答えは

$$\sum_{S_1 \subseteq T_1, \ldots, S_K \subseteq T_K} (-1)^{\sum_{k=1}^{K} |S_k|} \left(N - \sum_{k=1}^{K} |S_k|\right)! \prod_{k=1}^{K} \binom{|T_k|}{|S_k|}^2 |S_k|!$$

となります．$\binom{|T_k|}{|S_k|}^2 |S_k|!$ の部分は，各 k ごとに独立に計算できることに注目します．また，$(-1)^{\sum_{k=1}^{K} |S_k|} \left(N - \sum_{k=1}^{K} |S_k|\right)!$ の部分は，$\sum_{k=1}^{K} |S_k|$ さえ分かっていれば計算できることに注目します．

以上の考察をもとに DP を設計します．

- 定義: $\mathrm{dp}_{i,x} := \left(\sum_{k=1}^{i} |S_k| = x \text{ のとき } \prod_{k=1}^{i} \binom{|T_k|}{|S_k|}^2 |S_k|! \text{ の総和}\right)$.
- 初期化: $\mathrm{dp}_{0,0} = 1$.
- 遷移: 各 $0 \le y \le |T_i|$ について，$\mathrm{dp}_{i,x+y}$ に $\mathrm{dp}_{i-1,x} \times \binom{|T_i|}{y}^2 y!$ を足し込む．

x の状態数は $\mathrm{O}(N)$ で，遷移は $1 \le i \le K$ の合計で $\mathrm{O}(N)$ なので，全体の時間計算量は $\mathrm{O}(N^2)$ となり，十分に高速です．最終的に，$\sum_{x=0}^{N-1} (-1)^x (N-x)! \times \mathrm{dp}_{K,x}$ が答えです．

ABC #082 / ARC #087 Editorial

writer : sugim48

December 16, 2017

A: Round Up the Mean

- C++ Code Example : https://beta.atcoder.jp/contests/abc082/submissions/1872729
- Java Code Example : https://beta.atcoder.jp/contests/abc082/submissions/1872732
- Python 3 Code Example : https://beta.atcoder.jp/contests/abc082/submissions/1872734

B: Two Anagrams

Sort s, sort t, reverse t, and check if $s < t$.

- C++ Code Example : https://beta.atcoder.jp/contests/abc082/submissions/1872811
- Java Code Example : https://beta.atcoder.jp/contests/abc082/submissions/1872812
- Python 3 Code Example : https://beta.atcoder.jp/contests/abc082/submissions/1872814

C: Good Sequence

Let n_x be the number of occurrences of x in a. If $n_x < x$, add n_x to the answer.
If $n_x \geq x$, add $n_x - x$ to the answer.

- C++ Code Example : https://beta.atcoder.jp/contests/abc082/submissions/1872852

D: FT Robot

Let N be the length of s and (x_t, y_t) be the coordinates of the goal.

Let's separate the sequence of operations by `T`, and let d_i be the number of `F` in the i-th part. Then, the movement of the robot can be described as follows:

- Move d_1 to right.
- Move d_2 upward or downward.
- Move d_3 to left or right.
- Move d_4 upward or downward.
- Move d_5 to left or right.
- And so on.

Thus, the x-coordinate and the y-coordinate can be handled independently. For example, for the x-coordinate, the robot will perform the following operations:

- Move d_1 to right.
- Move d_3 to left or right.
- Move d_5 to left or right.
- And so on.

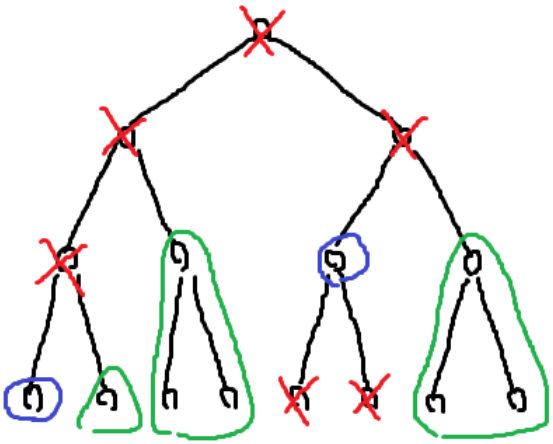
We want to check if x can be x_t after all operations. This can be done by a straightforward DP in $O(N^2)$. The y-coordinate can be handled similarly.

E: Prefix-free Game

Let's construct a trie of all strings of length L. It will be a complete binary tree. On this tree, the game can be described as follows:

> Initially, some vertices contain tokens. Alice and Bob alternately put tokens. Here, no two tokens can be put on two vertices with "ancestor - descendant" relation. The player who fails to put a token loses.

For example, the picture below shows the case with $L = 3$ and two strings 000 and 10.



Blue vertices contain tokens, red vertices are forbidden, and you can put tokens on green vertices. The green vertices will be a disjoint union of complete binary trees, and each part (complete binary tree) can be handled independently. Thus, we can use Grundy Numbers.

Let g_i be the Grundy Number of a complete binary tree of height i (i.e., a complete binary tree with i layers). If you perform an operation on a complete binary tree of height i, it will be splitted into complete binary trees with heights $i-1$, $i-2$, ..., $i-j$ for some j $(0 \leq j \leq i-1)$. Thus, g_i is the smallest integer that doesn't appear in the following:

- 0
- g_{i-1}
- $g_{i-1} \oplus g_{i-2}$
- $\vdots$
- $g_{i-1} \oplus g_{i-2} \oplus \cdots \oplus g_1$

If you perform an experiment, you get $(g_1, g_2, g_3, g_4, g_5, g_6, g_7, g_8, \ldots) = (1, 2, 1, 4, 1, 2, 1, 8, \ldots)$. In general, g_i is the maximum power of two that divides i. (We omit the proof, it is straightforward.) If the XOR of all Grundy Numbers is 0, Bob wins, otherwise Alice wins.

To get the set of green binary trees, you should construct a trie for all input strings, and check all nodes with exactly one child.

F: Squirrel Migration

Consider an edge $e = (v, w)$ in the tree. Let n_v, n_w be the sizes of two subtrees we get when we cut the tree by e. Then, the number of squirrels that pass through this edge (call it s_e) is st most $2\min\{n_v, n_w\}$.

The total distance defined in the statement is the sum of s_e, thus it is at most $\sum 2\min\{n_v, n_w\}$. It turns out that this value is always achievable. There are two different cases:

The tree has two centroids

Call them c_1, c_2. The maximum can be achieved when all squirrels pass through an edge between c_1 and c_2. There are $(N/2)! \times (N/2)!$ such ways.

The tree has one centroid

Call it c. The maximum can be achieved when all squirrels pass through the vertex c.

Let T_1, T_2, ..., T_K be the subtrees we get when we remove c from the tree. We want to count the number of permutations such that for each i, no squirrel in T_i moves to a vertex in T_i.

We use inclusion-exclusion principle. Let $T := T_1 \cup T_2 \cup \cdots \cup T_K$ (i.e., the entire vertex set minus c). The answer is $\sum_{S \subseteq T} (-1)^{|S|} f_S$, where f_S is the number of permutations of squirrels such that each squirrel in S moves to a vertex in the same group.

Now, for each k, we want to compute the sum of f_S for all S such that $|S| = k$.

Let g_k be the number of ways to choose k squirrels and there destinations, such that all chosen squirrels will travel to a vertex in the same group. Formally, g_k is the number of pairs of two k-tuples of vertices $(s_1, \ldots, s_k), (t_1, \ldots, t_k)$ such that:

- $s_1 < s_2 < \cdots$ (the order of chosen squirrels doesn't matter)
- For each i, s_i and t_i are in the same subtree (and they are not c).
- t_i are pairwise distinct.

Since we can freely choose destinations for unchosen squirrels, the sum of f_S for all S such that $|S| = k$ is $g_k(N-k)!$. Therefore, the answer is $\sum_{k=0}^{N-1} g_k(N-k)!(-1)^k$. Since g_k can be easily computed in $O(N^2)$ by a simple DP, we can also get the answer in $O(N^2)$.

ABC 084 解説

writer: namonakiacc

平成 29 年 12 月 30 日

For international readers: English Editorial starts at page 4.

A: New Year

12 月 30 日の M 時から次の年になるまでは、$48 - M$ 時間です。よって、$48 - M$ を出力するとよいです。

```
#include <cstdio>
int M;
int main()
{
        scanf("%d",&M);
        printf("%d\n",48-M);
}
```

B: Postal Code

S の $A+1$ 文字目がハイフンであり、それ以外の全ての文字が 0 以上 9 以下である場合に ‘Yes‘、そうでない場合に ‘No‘ を出力するとよいです。

```
#include <cstdio>
int A,B;
char s[12];
int main()
{
        scanf("%d%d",&A,&B);
        scanf("%s",&s);
        for(int i=0;i<A;i++)if(!('0'<=s[i]&&s[i]<='9'))
        {
                puts("No"); return 0;
        }
        if(s[A]!='-')
        {
                puts("No"); return 0;
        }
        for(int i=A+1;i<=A+B;i++)if(!('0'<=s[i]&&s[i]<='9'))
        {
                puts("No"); return 0;
        }
        puts("Yes");
}
```

C: Special Trains

駅 j に開通式開始 t 秒後に到着したとき、駅 $j+1$ に向かう列車に乗れるのが開通式開始何秒後かを考えると、次のようになります。

・$t < S_j$ ならば開通式開始 S_j 秒後

・そうでなく、$t \,\%\, F_j = 0$ ならば開通式開始 t 秒後

・そうでないなら、開通式開始 $t + F_j - (t \,\%\, F_j)$ 秒後

これに従い、全ての駅 i から出発する場合に対してシミュレーションしてみると、$O(N^2)$ となり、解くことができます。

```
#include <cstdio>
int N,C[500],S[500],F[500];
int main()
{
        scanf("%d",&N);
        for(int i=0;i<N-1;i++)scanf("%d%d%d",&C[i],&S[i],&F[i]);
        for(int i=0;i<N;i++)
        {
                int t=0;
                for(int j=i;j<N-1;j++)
                {
                        if(t<S[j])t=S[j];
                        else if(t%F[j]==0);
                        else t=t+F[j]-t%F[j];
                        t+=C[j];
                }
                printf("%d\n",t);
        }
}
```

D: 2017-like Number

1 が素数ではないことに注意しつつ、2 以上 100000 以下の数に対して、素数であるか否かを求めます。

これは、エラトステネスの篩を用いて実現できます。

具体的には、以下のような手順をとります。

・リストに 2 から 100000 までの整数を入れる。

・リストに入っている、まだ見ていない一番小さい数を x とする。このとき、x は素数なので、x の倍数 (x を除く) をリストから取り除く。

・リストに入っている数に、まだ見ていない数がなくなったら操作を終了する。このとき、2 以上 100000 以下の整数のうち、リストに含まれている数は素数であり、含まれていない数は素数でない。

リストに含まれているか否かをフラグにして保管すると、十分な速さで求めることができます。(M 以下の数に対してエラトステネスの篩を適用した時のオーダーは $O(Mlog(logM))$ となります。)

これにより、1 以上 100000 以下の奇数に対して、**2017 に似た数** か否かが求められるようになりました。

次に、区間内の **2017 に似た数** を高速に求める方法についてです。

これは、累積和を用いることで実現できます。

具体的には、全ての 1 以上 100000 以下の整数 x に対して、1 以上 x 以下の数に **2017 に似た数** がいくつあるかを求めておきます。

$c_i = 1$ 以上 x 以下の **2017 に似た数** の個数　とすると、

・$c_{i+1} = c_i + 1$ ($i+1$ が **2017 に似た数** である場合)

・$c_{i+1} = c_i$ ($i+1$ が **2017 に似た数** でない場合)

となるので、この部分に関しても十分な速さで求めることができます。

このとき、l 以上 r 以下の整数に、**2017 に似た数** は $c_r - c_{l-1}$ 個あります。

これらに基づいて実装すると、解くことができます。

```
#include <cstdio>
bool f[100001];
int c[100002];
int N,L,R;
int main()
{
        for(int i=2;i<=100000;i++)if(!f[i])for(int j=i+i;j<=100000;j+=i)
                f[j]=true;
        for(int i=3;i<=100000;i+=2)if(!f[i]&&!f[(i+1)/2])c[i]++;
        for(int i=3;i<=100000;i++)c[i]+=c[i-1];
        scanf("%d",&N);
        while(N--)
        {
                scanf("%d%d",&L,&R);
                printf("%d\n",c[R]-c[L-1]);
        }
}
```

A: New Year

There are $48 - M$ hours until New Year at M o‘clock on 30th Monday,December.

So you can get 100 point by print $48 - M$.

```
#include <cstdio>
int M;
int main()
{
        scanf("%d",&M);
        printf("%d\n",48-M);
}
```

B: Postal Code

You can get 200 point by print ‘Yes‘ when $(A + 1)$ -th character of S is a hyphen and the other characters are digits from ‘0‘ through ’9’,‘No‘ otherwise.

```
#include <cstdio>
int A,B;
char s[12];
int main()
{
        scanf("%d%d",&A,&B);
        scanf("%s",&s);
        for(int i=0;i<A;i++)if(!('0'<=s[i]&&s[i]<='9'))
        {
                puts("No"); return 0;
        }
        if(s[A]!='-')
        {
                puts("No"); return 0;
        }
        for(int i=A+1;i<=A+B;i++)if(!('0'<=s[i]&&s[i]<='9'))
        {
                puts("No"); return 0;
        }
        puts("Yes");
}
```

C: Special Trains

Fisrt,considering when it is possible to ride a train which goes to station $j+1$,in the situation that arriving station j ,t seconds after the ceremony begin.

- If $t < S_j$, S_j seconds after the ceremony begin.
- If $t \geqq S_j$,but $t \,\%\, F_j = 0$, t seconds after the ceremony begin.
- Othersise, $t + F_j - (t \,\%\, F_j)$ seconds after the ceremony begin.

Considering this,simulate in every case,it would be $O(N^2)$ and you can get 300 points.

```
#include <cstdio>
int N,C[500],S[500],F[500];
int main()
{
        scanf("%d",&N);
        for(int i=0;i<N-1;i++)scanf("%d%d%d",&C[i],&S[i],&F[i]);
        for(int i=0;i<N;i++)
        {
                int t=0;
                for(int j=i;j<N-1;j++)
                {
                        if(t<S[j])t=S[j];
                        else if(t%F[j]==0);
                        else t=t+F[j]-t%F[j];
                        t+=C[j];
                }
                printf("%d\n",t);
        }
}
```

D: 2017-like Number

Pay attention to the fact that 1 is not prime number,calculate if the number is prime number or not ,in the number from 2 to 100000.

It could be done by using Sieve of Eratosthenes.

In more details,do as such.

・put the all number from 2 to 100000 in the list.

・Assume that the minimum number you don't check in the list , as x. x is prime number,so remove the multiples of x (without x) from the list.

・If there are no number you don't check in the list,stop this.After that,numbers from 2 to 100000 in the list is prime number,not in the list is not prime number.

Using flag to note whether or not the number is in the list,it could be done by reasonable time.(If you done Sieve of Eratosthenes in the number from 2 to M , $O(Mlog(logM))$)

Doing so, you can check whether or not x is similar to 2017 ,in x from 1 to 100000.

Next,we should consider about how to solve quickly ,how many number is similar to 2017,in some range.

It can be done by Cumulative sum.

In more detail,solve how many number is similar to 2017 in range from 1 to x ,in all number x from 1 to 100000.

Considering c_i as showing how many number is similar to 2017 in range from 1 to x ,

・$c_{i+1} = c_i + 1$ (when $i+1$ is similar to 2017)

・$c_{i+1} = c_i$ (Otherwise)

So,it could be done in reasonable time.

And, $c_r - c_{l-1}$ numbers is similar to 2017 in range from l to r.

Doing as such,you can get 400 points.

```
#include <cstdio>
bool f[100001];
int c[100002];
int N,L,R;
int main()
{
        for(int i=2;i<=100000;i++)if(!f[i])for(int j=i+i;j<=100000;j+=i)
                f[j]=true;
        for(int i=3;i<=100000;i+=2)if(!f[i]&&!f[(i+1)/2])c[i]++;
        for(int i=3;i<=100000;i++)c[i]+=c[i-1];
        scanf("%d",&N);
        while(N--)
        {
                scanf("%d%d",&L,&R);
                printf("%d\n",c[R]-c[L-1]);
        }
}
```

ABC 085 Editorial (Japanese)

問題・解説: evima

2018 年 1 月 7 日

For International Readers: English editorial starts on page 5.

A: Already 2018

この問題を解くには、以下の手順を踏む必要があります。

0. （言語によっては不要）文字列変数 S を宣言する。
1. 標準入力から S を文字列として受け取る。
2. S の 4 文字目を `'8'` に置き換える（直接的または間接的に）。
3. 得た文字列を標準出力に出力する。

ステップ 2. に関しては、言語によって事情が異なります。C++ のような言語では単に `S[3] = '8';` などとすれば済みますが、Python のような言語では文字列中の文字を直接書き換えることが許されないこともあります。このような場合、文字列を一旦別のデータ構造（リスト）に変換する手もありますが、S の末尾の 2 文字を `"2018/01/"` と連結するなど、別の方針も考えられます。

C++, Python での実装例:

```cpp
#include <iostream>
#include <string>
using namespace std;

int main(){
    string S;
    cin >> S;
    S[3] = '8';
    cout << S << endl;
}
```

```python
print("2018/01/" + input()[-2:])
```

B: Kagami Mochi

問題文は遠回しな言い方をしていますが、要するに「$d_1, d_2, \ldots, d_N$ には何種類の異なる値があるか？」と聞いています。

方針 1: 「自分でやる」

配列 `flag` を用意し、はじめ `flag[1] = flag[2] = ... = flag[100] = 0` とします。d_1 から d_N までのそれぞれの値 `d` に対して `flag[d] = 1` とすると、`flag` の要素の和が答えとなります。

C++ での実装例:

```cpp
#include <iostream>
using namespace std;
int N, flag[101];   // Global variables are initially all 0.
                    // Also note that we need [101], not [100]!
int main(){
    cin >> N;
    for(int i = 0; i < N; ++i){
        int d;
        cin >> d;
        flag[d] = 1;
    }
    int ans = 0;
    for(int i = 0; i < N; ++i){
        ans += flag[i];
    }
    cout << ans << endl;
}
```

方針 2: 「set を使う」

多くの言語には重複を省いた集合を表すデータ構造が実装されており、たいてい set か似たような名前を持ちます。$d_1, d_2, \ldots, d_N$ を入れた set の要素数がそのまま答えとなります。

Python での実装例（少し遊んでいます）:

```python
print(len(set(input() for i in range(int(input())))))
```

C: Otoshidama

袋の中身を「10000 円札 x 枚、5000 円札 y 枚、1000 円札 z 枚」とします。当然ですが重要な点として、x, y, z はどれも 0 以上 N 以下の整数でなければなりません。

もっとも単純な方針は、三重ループで x, y, z の値の決め方をすべて試し、$x+y+z=N, 10000x+5000y+1000z=Y$ が成立するか確かめるものですが、$N \leq 2000$、実行制限時間 2 秒という設定で $\mathrm{O}(N^3)$ 時間の解法は分が悪いです（間に合わせることも不可能ではありません）。しかし、x と y の値が決定されると z の値が $z=N-x-y$ に限られることを利用すると、三重ループではなく二重ループですべての可能性を列挙することができます（ただし、z が負になるケースを取り除く必要があります）。これらのすべての可能性に対して $10000x+5000y+1000z=Y$ が成り立つか調べることで、問題を $\mathrm{O}(N^2)$ 時間で解くことができます。

なお、$\mathrm{O}(N)$ 時間や $\mathrm{O}(1)$ 時間の解法も考えられますが、コンテスト中に追求する必要はないでしょう。

D: Katana Thrower

問題文のルールでは、刀を投げてしまうとその刀を振ることはできなくなります。しかし、すでに投げてしまった刀も振ることができるというルールに変更しても問題の答えは変わりません。これを証明します。以下、問題文のルールを「旧ルール」、ここで定義したルールを「新ルール」と呼びます。

新ルールで T 回攻撃して合計 D ダメージを与えることが可能であると仮定します。そのような攻撃手順の一つで、それぞれの刀 i $(1 \leq i \leq N)$ を x_i 回振って y_i 回投げるとします $(0 \leq x_i, 0 \leq y_i \leq 1)$。このとき、旧ルールでも刀 1 を x_1 回振ってから y_1 回投げ、刀 2 を x_2 回振ってから y_2 回投げ、...、刀 N を x_N 回振ってから y_N 回投げることで、同じく T 回の攻撃で合計 D ダメージを与えることが可能です。よって、新ルールで T 回の攻撃で魔物を退治できるとき、旧ルールでも T 回の攻撃で魔物を退治できます。逆に、旧ルールで T 回の攻撃で魔物を退治できるとき、新ルールでもまったく同じ攻撃手順で T 回の攻撃で魔物を退治できます。以上より、新ルールを採用しても問題の答えは変わりません。

以後、新ルールの下で問題を考えます。新ルールでは、刀を振ることと投げることは直接関連せず、我々は攻撃力 a_i の「振り刀」N 本と攻撃力 b_i の「投げ刀」N 本、合計 $2N$ 本の武器を持っていると考えることができます。(攻撃力 a の振り刀は投げることはできないが何度でも振ることができ、そのたびに a ダメージを与えます。攻撃力 b の投げ刀は振ることはできないが一度だけ投げることができ、投げると b ダメージを与えます。) このときの最適な攻撃手順は以下の通りです。

0. $a_1, \ldots, a_N$ のうち最大の値を $a_{\max}$ とし、攻撃力が $a_{\max}$ 以上の投げ刀を「強い投げ刀」とする。
1. 強い投げ刀を攻撃力が高い順に投げていく。
2. 強い投げ刀を使い果たしたら、攻撃力 $a_{\max}$ の振り刀を振り続ける。

この戦略は、はじめに $b_1, \ldots, b_N$ を降順にソートしておき (言語の標準ライブラリを使うべきです)、ステップ 2. で割り算を使うことで、全体で $\mathrm{O}(N \log(N))$ 時間でシミュレートすることができます。

ABC 085 Editorial (English)

Problems and editorial by: evima

January 7, 2018

A: Already 2018

To solve this problem, you need to go through the procedure below:

0. (Unnecessary for some languages) Declare a string variable S.
1. Receive S as a string from Standard Input.
2. Replace the fourth character in S with `'8'` (directly or indirectly).
3. Print the obtained string to Standard Output.

For step 2., situations are different depending on language. In languages such as C++, writing `S[3] = '8';` or similar code is fine, but in some languages such as Python, rewriting a character in a string is prohibited. In this case, a workaround is to convert the string to another data structure (list) temporarily, but there are also other paths, such as appending the last two characters in S to `"2018/01/"`.

Sample implementations in C++ and Python:

```cpp
#include <iostream>
#include <string>
using namespace std;

int main(){
    string S;
    cin >> S;
    S[3] = '8';
    cout << S << endl;
}
```

```python
print("2018/01/" + input()[-2:])
```

B: Kagami Mochi

The statement is beating around the bush, but in short, the problem is asking you: "how many different values are there among $d_1, d_2, \ldots, d_N$?"

Approach 1: Do it yourself

Prepare an array `flag`, and initially set it to `flag[1] = flag[2] = ... = flag[100] = 0`. Then, for each value `d` from d_1 to d_N, set `flag[d] = 1`. Now, the sum of the elements of `flag` is the answer.

Sample implementation in C++:

```cpp
#include <iostream>
using namespace std;
int N, flag[101];   // Global variables are initially all 0.
                    // Also note that we need [101], not [100]!
int main(){
    cin >> N;
    for(int i = 0; i < N; ++i){
        int d;
        cin >> d;
        flag[d] = 1;
    }
    int ans = 0;
    for(int i = 0; i < N; ++i){
        ans += flag[i];
    }
    cout << ans << endl;
}
```

Approach 2: Use set

Many languages implement data structures representing sets without duplication, usually named "set" or similarly. Insert $d_1, d_2, \ldots, d_N$ into a set, and the number of elements itself is the answer.

Sample implementation in Python (I'm playing a bit):

```python
print(len(set(input() for i in range(int(input())))))
```

C: Otoshidama

Let the contents of the envelope be "x 10000-yen bills, y 5000-yen bills and z 1000-yen bills". Naturally but importantly, x, y and z all have to be integers between 0 and N.

The simplest approach is to try all ways to set the values of x, y and z in three nested loops and check if $x + y + z = N, 10000x + 5000y + 1000z = Y$ hold, but under the constraint $N \leq 2000$ and the execution time limit of 2 seconds, odds are against $\mathrm{O}(N^3)$ time solutions (not impossible to make it in time, though). Notice that, however, when we fix the values of x and y, the value of z will be limited to $z = N - x - y$. Then, we can enumerate all possibilities in two nested loops, not three (but we need to remove the case where z is negative). By checking if $10000x + 5000y + 1000z = Y$ holds for each of these possibilities, we can solve the problem in $\mathrm{O}(N^2)$ time.

There are also $\mathrm{O}(N)$ time and $\mathrm{O}(1)$ time solutions, but no need to find them during the contest.

D: Katana Thrower

Under the rules in the statement, when you throw a katana, you will no longer be able to wield that katana. However, the answer does not change if we change the rule to allow wielding a katana that you have already thrown. We will now prove it. Below, we will call the rules in the statement the "old rules", and the rules defined here the "new rules".

Suppose that we can deal a total of D damage in T attacks under the new rules. Assume that, in one such sequence of attacks, each katana i ($1 \leq i \leq N$) is wielded x_i times and thrown y_i times ($0 \leq x_i, 0 \leq y_i \leq 1$). Then, we can also deal a total of D damage in T attacks under the old rules, by wielding katana 1 x_1 times, then throwing katana 1 y_1 times, then wielding katana 2 x_2 times, then throwing katana 2 y_2 times, ..., then wielding katana N x_N times, then throwing katana N y_N times. Thus, if we can vanish the monster in T attacks under the new rules, we can also vanish the monster in T attacks under the old rules. On the other hand, if we can vanish the monster in T attacks under the old rules, we can also vanish the monster in T attacks under the new rules by performing exactly the same sequence of attacks. Therefore, the answer does not change by adopting the new rules.

We will consider the problem under the new rules from now on. Under the new rules, wielding katana and throwing them is not directly related, and we can assume that we have $2N$ weapons, N "wielding katana" of attack power a_i and N "throwing katana" of attack power b_i. (A wielding katana of attack power a_i cannot be thrown but can be wielded any number of times, and deals a damage each time we wield it. A throwing katana of attack power b_i cannot be wielded but can be thrown only once, and deals b damage when we throw it.) The optimal strategy now is:

0. Let $a_{\max}$ be the maximum value among $a_1, \ldots, a_N$, and "strong throwing katana" be the katana whose attack powers are $a_{\max}$ or above.
1. Throw strong throwing katana one at a time in decreasing order of attack power.
2. When we have no more throwing katana, repeat wielding the wielding katana of attack power $a_{\max}$.

This strategy can be simulated in $\mathrm{O}(N \log(N))$ time in total, by sorting $b_1, \ldots, b_N$ in decreasing order (the standard library in your language should be used) in the beginning and use division in step 2.

Atcoder Beginner Contest 088 解説

問題作成者：E869120, square1001

平成 30 年 2 月 18 日

For International Readers: English editorial starts from page 5.

A: Infinite Coins

500 円硬貨が無限枚あるので，使う 1 円硬貨を最小化することを考えます．

N 円を超えないようにできるだけ多くの 500 円硬貨を使うと 1 円硬貨の枚数は最小化されます．そのとき，500 円硬貨は「N を 500 で割ったときの商」枚使われることになります．

この場合、使用する 1 円硬貨の枚数は N を 500 で割ったあまりになるので、これが持っている N 円硬貨の枚数 A 以下であれば “Yes”，そうでなければ “No” と出力します．

サンプルコード (C++)

- https://beta.atcoder.jp/contests/abc088/submissions/2105041

B: Card Game for Two

それぞれのプレイヤーは，自分のターンのときに，今選べるカードの中で最大の数が書かれているものを取るという戦略をとると，自分の得点を最大化できます．

a の値を大きい順に $a_1, a_2, \ldots, a_N$ となるように並べ替えるとき，$a_1 + a_3 + a_5 + a_7 + \cdots$ が Alice の得点となり，$a_2 + a_4 + a_6 + a_8 + \cdots$ が Bob の得点となります．

数列を小さい順・大きい順に並べ替える操作は「ソート (sorting)」といい，ほとんどのプログラミング言語において数列のソートをする関数はあります．存在しない場合でも，バブルソートなどの方法を使って数列をソートすることもできます．

サンプルコード (C++)：

- https://beta.atcoder.jp/contests/abc088/submissions/2105068

C: Takahashi's Information

高橋君の情報が正しい場合を考えます．$(a_1, a_2, a_3, b_1, b_2, b_3) = (p_1, p_2, p_3, q_1, q_2, q_3)$ の値の組み合わせが条件に沿っているならば，任意の数 x に対して，$(a_1, a_2, a_3, b_1, b_2, b_3) = (p_1 + x, p_2 + x, p_3 + x, q_1 - x, q_2 - x, q_3 - x)$ の値の組み合わせが正しいです．なぜなら，$p_i + q_j = (p_i + x) - (q_j + x)$ が成立するので，$a_i + b_j$ の値はどの (i, j) に対しても変化しないからです．

高橋君の情報が正しい場合，x を適切な値に設定することで，$a_1 = 0$ にすることができます．したがって，高橋君の情報が正しいかどうか判定するとき $a_1 = 0$ を仮定して考えてもよいです．a_1 の値が決まると，b_1, b_2, b_3 の値も決まります．具体的には，$b_i = c_{1,i} - a_1$ とすればよいです．b_1 の値が決まると，$a_i = c_{i,1} - b_1$ を利用して a_2, a_3 の値も決めることができます．

最終的に得られた $a_1, a_2, a_3, b_1, b_2, b_3$ の値に対して，$c_{i,j} = a_i + b_j$ が成立していれば高橋君の情報が正しく，そうでない場合高橋君の情報は間違っています．

D: Grid Repainting

まず，「けぬす君」が移動するルートがあらかじめ決まっている時，すぬけ君は最大で何マス変更できるかについて考えます．

最初に考えるべきは，各マスについてどのような操作ができるかです．「けぬす君」が通るマス・通らないマスそれぞれについて、何の色にすべきかという条件は，以下のようになります。

1. 「けぬす君」が通るマスは、絶対に白でなければならない．
2. 「けぬす君」が通らないマスは，白と黒のどちらでも良い．

まず，マスの色を黒から白に変更することはできないので，1．よりすぬけ君の通るルートに「最初の色」が黒であるマスが存在してはならず，すぬけ君の通るルートに含まれる「最初の色が白のマス」を黒に変えると 1．に反するので，これらのマスは変更することができません．

また，「けぬす君」が通らないマスにおける変更回数の合計の最大値は，2．より最終的な色は何でも良いため，けぬす君の通らない白いマスの色をすべて黒に変更することができるので，（けぬす君が通らない白いマスの個数）になります．

したがって，すぬけ君が変更できるマスの個数の最大値は，(けぬす君が通らない白いマスの個数) = **(マス目全体の白いマスの個数) − (けぬす君が通る白いマスの個数)** となります．

実際の問題では，けぬす君はマス (H, W) に白いマスだけを通ってたどり着くことができればどんなルートをたどっても良いです．しかし，「変更する回数」＝「色を変更できる白いマスの個数」を最大化するので，すなわち**「色を変更できない白いマスの個数」を最小化したい**です．

「色を変更できない白いマスの個数」は，(けぬす君が通る白いマスの個数) と等しいので、「けぬす君は $(1, 1)$ から (H, W) に行くために最低何回の白いマスを通る必要があるか」という問題を解けば，この問題は解けます．言い換えると「けぬす君が白いマスだけを通って移動するとき，最小で何マスを使って $(1, 1)$ から (H, W) に移動できるか」という問題になります．(このような経路が存在しない場合，ゲームクリアすることはできない)

この問題は辺の重みがすべて 1 であるグラフの最短経路問題に帰着することができるので，幅優先探索を用いれば，計算量 $O(HW)$ で解くことができます．

Atcoder Beginner Contest 088 Editorial

Problem Setters : E869120, square1001

平成 30 年 2 月 18 日

A: Infinite Coins

There are infinite number of 500-yen coins. So, to minimize the number of 1-yen coins, he has to maximize the number of 500-yen coins. In this case the number of 500-yen coin is “the quotient when N is divided by 500”.

In this case, the number of 1-yen coins is “the remainder when N is divided by 500”, so you should check if the remainder when N is divided by 500 is greater than A or not.

Sample Code (C++)

- https://beta.atcoder.jp/contests/abc088/submissions/2105041

B: Card Game for Two

If the player takes a strategy which chooses a card with maximum number among all selectable cards in each turn, the final score will be maximized.

Now, let's rearrange the sequence a, that $a_1, a_2, a_3, \ldots, a_N$ will be in decreasing order. Alice's score will be $a_1 + a_3 + a_5 + a_7 + \cdots$, and Bob's score will be $a_2 + a_4 + a_6 + a_8 + \cdots$. Therefore, the difference of Alice's score and Bob's score will be $a_1 - a_2 + a_3 - a_4 + a_5 - a_6 + a_7 - a_8 + \cdots$.

Rearranging sequence in increasing order or decreasing order is called "sorting". Sorting is the basic algorithm in computer science, and most programming language has sorting function. If you don't use programming language that doesn't have sorting function, you can implement sorting algorithm like Bubble Sort.

Sample Code (C++) :

- https://beta.atcoder.jp/contests/abc088/submissions/2105068

C: Takahashi's Information

If Takahashi's information is correct, there exists a combination of numbers $(a_1, a_2, a_3, b_1, b_2, b_3) = (p_1, p_2, p_3, q_1, q_2, q_3)$ that satisfies $a_i + b_j = c_{i,j}$ for all (i, j). And in this case, for any number x, $(a_1, a_2, a_3, b_1, b_2, b_3) = (p_1 + x, p_2 + x, p_3 + x, q_1 - x, q_2 - x, q_3 - x)$ also satisfies $a_i + b_j = c_{i,j}$, because $p_i + q_j = (p_i + x) + (q_j - x)$ holds.

So, if Takahashi's information is correct, you can set $a_1 = 0$ if you choose number x properly. Therefore, if you only want to judge whether Takahashi's information is correct, deciding "$a_1 = 0$" is okay. Then, you can get the values of b_1, b_2, b_3, because $b_i = c_{1,i} - a_1$ therefore you can set $b_i = c_{1,i}$. And then, if you have the value of b_1, you can get the values of a_2, a_3 because $a_i = c_{i,1} - b_i$ holds.

Now you got the values of $a_1, a_2, a_3, b_1, b_2, b_3$. Finally, Takahashi's information is correct if and only if $a_i + b_j = c_{i,j}$ holds for all (i, j) so you can check it easily.

D: Grid Repainting

At first, let's think about the maximum number of cells that Snuke changes, when the route Kenus moves is already decided.

In this case, to achieve the maximum number of cells that Snuke changes, Snuke should change cells as follows:

a. If the cell is belonged to the Kenus's route, the cell must be white. Thus, it is not valid route if at least one of cell is black at first because the color of black cell cannot be changed. And, if it is white, it should remain white so Snuke cannot change the color of cells in this route.
b. If the cell is not belonged to the Kenus's route, any color is OK. So, if the cell is white, Snuke should change the color of the cell (from white to black) to achieve maximum number of cells that he changes.

Therefore, the maximum number of cells that Snuke changes is (The number of white cells that are not belonged to the Kenus's route), and this is equal to **(The number of white cells in the grid) – (The number of white cells that are belonged to the Kenus's route).**

Finally, the first easier problem was solved. But unfortunately, the real problem is: Kenus's route is not decided, and Snuke can decide it.

Snuke want to minimize the number of white cells that are belonged to the Kenus's route. In addition, the number of white cell that are belonged to the Kenus's route is equal to the number of cells that Kenus passes, because all cell that Kenus passes should be white.

So, now the problem is simple. The minimum number of white cells Kenus passes is equal to the result of the problem: "You are given a grid and each cell is black or white. You move from $(1,1)$ to (H,W) but you can't pass black cells. If you can move one cell left, right, up or down, what is the minimum number of cell you pass?".

This is a typical problem of "shortest path problem". You can use Breadth-First Search (BFS) and solvable in $O(HW)$ time complexity.

ABC 089 解説

writer: namonakiacc

平成 30 年 3 月 4 日

For international readers: English Editorial starts at page 4.

A: Grouping 2

3 人以上のグループは最大で $\lfloor N/3 \rfloor$ 個作れます。
よって、 $\lfloor N/3 \rfloor$ を出力するとよいです。

```
#include <cstdio>
int N;
int main()
{
        scanf("%d",&N);
        printf("%d\n",N/3);
}
```

B: Hina Arare

袋には必ず桃色、白色、緑色のひなあられが入っていることが保証されるので、3 種類か 4 種類かの判別は黄色のひなあられの有無で判別できます。
よって、黄色のひなあられがあれば ‘Four‘、なければ ‘Three‘ と出力するとよいです。

```
#include <cstdio>
int N;
char c;
int main()
{
        scanf("%d",&N);
        bool flag=false;
        for(int i=0;i<N;i++)
        {
                scanf("%c",&c);
                if(c=='Y')flag=true;
        }
        if(flag)puts("Four");
        else puts("Three");
}
```

C: March

‘M‘ から始まる名前の人が m 人、‘A‘ から始まる名前の人が a 人、‘R‘ から始まる名前の人が r 人、‘C‘ から始まる名前の人が c 人、‘H‘ から始まる名前の人が h 人いるとします。

例えば、「‘M‘ から始まる名前の人」と「‘A‘ から始まる名前の人」と「‘R‘ から始まる名前の人」の 3 人を選ぶ方法は、$m \times a \times r$ 通りです。

3 人を選ぶとき、3 人の名前の頭文字の組み合わせは高々 10 通りとなります。

よって、m, a, r, c, h を求めた後、10 通りの組み合わせ全てを試せばよいです。

```
#include <cstdio>
#include <iostream>
using namespace std;
typedef long long ll;

string s;
int N;
ll m,a,r,c,h;
ll D[5];
int P[10]={0,0,0,0,0,0,1,1,1,2};
int Q[10]={1,1,1,2,2,3,2,2,3,3};
int R[10]={2,3,4,3,4,4,3,4,4,4};
int main()
{
        scanf("%d",&N);
        for(int i=0;i<N;i++)
        {
                cin>>s;
                if(s[0]=='M')m++;
                if(s[0]=='A')a++;
                if(s[0]=='R')r++;
                if(s[0]=='C')c++;
                if(s[0]=='H')h++;
        }
        D[0]=m,D[1]=a,D[2]=r,D[3]=c,D[4]=h;
        ll res=0;
        for(int d=0;d<10;d++)
                res+=D[P[d]]*D[Q[d]]*D[R[d]];
        printf("%lld\n",res);
}
```

D: Practical Skill Test

初めに、1 以上 $H \times W$ 以下の全ての整数に対して、その整数がどのマスに書かれているかを記録しておきます。

以下、整数 i はマス (px_i, py_i) に書かれているものとします。

次に、D が全てのクエリにおいて共通の値となっているので、D を利用することを考えます。

$R_i - L_i$ が D の倍数であることが保証されているため、D で割った余りを用いて累積和の方式を用いることができます。

- $d_i = 0 \ (1 \leq i \leq D)$
- $d_i = d_{i-D} + |px_i - px_{i-D}| + |py_i - py_{i-D}| \ (D+1 \leq i)$

とすると、実技試験において駒を L から R まで移動するのに消費される魔力は $d_R - d_L$ となります。

```
#include <cstdio>
#define abs(x) ((x>0)?x:(-(x)))
int H,W,D,A;
int Q,L,R;
int px[90001],py[90001];
int d[90001];
int main()
{
        scanf("%d%d%d",&H,&W,&D);
        for(int i=0;i<H;i++)
        {
                for(int j=0;j<W;j++)
                {
                        scanf("%d",&A);
                        px[A]=i,py[A]=j;
                }
        }
        for(int i=D+1;i<=H*W;i++)
                d[i]=d[i-D]+abs(px[i]-px[i-D])+abs(py[i]-py[i-D]);
        scanf("%d",&Q);
        while(Q--)
        {
                scanf("%d%d",&L,&R);
                printf("%d\n",d[R]-d[L]);
        }
}
```

A: Grouping 2

The maximum number of groups consisting of three or more students is $\lfloor N/3 \rfloor$.

So,you can get 100 points by print $\lfloor N/3 \rfloor$.

```
#include <cstdio>
int N;
int main()
{
        scanf("%d",&N);
        printf("%d\n",N/3);
}
```

B: Hina Arare

It is guaranteed that there are pink,white,green arare,so it is possible to distinguish the number of colors by the existence of yellow arare.

So, you can get 200 points by print ‘Four‘ when yellow arare exists, ‘Three‘ otherwise.

```
#include <cstdio>
int N;
char c;
int main()
{
        scanf("%d",&N);
        bool flag=false;
        for(int i=0;i<N;i++)
        {
                scanf("%c",&c);
                if(c=='Y')flag=true;
        }
        if(flag)puts("Four");
        else puts("Three");
}
```

C: March

Considering as m means the number of people the name begins with 'M', a - 'A',r - 'R',c - 'C',h - 'H'.

For example,the way to choose people the name begins with 'M','A','R' is $m \times a \times r$.

When choosing 3 people,there are no more than 10 patterns of the first character of those names.

So,it is OK to calculate m, a, r, c, h and try all 10 patterns.

```
#include <cstdio>
#include <iostream>
using namespace std;
typedef long long ll;

string s;
int N;
ll m,a,r,c,h;
ll D[5];
int P[10]={0,0,0,0,0,0,1,1,1,2};
int Q[10]={1,1,1,2,2,3,2,2,3,3};
int R[10]={2,3,4,3,4,4,3,4,4,4};
int main()
{
        scanf("%d",&N);
        for(int i=0;i<N;i++)
        {
                cin>>s;
                if(s[0]=='M')m++;
                if(s[0]=='A')a++;
                if(s[0]=='R')r++;
                if(s[0]=='C')c++;
                if(s[0]=='H')h++;
        }
        D[0]=m,D[1]=a,D[2]=r,D[3]=c,D[4]=h;
        ll res=0;
        for(int d=0;d<10;d++)
                res+=D[P[d]]*D[Q[d]]*D[R[d]];
        printf("%lld\n",res);
}
```

D: Practical Skill Test

First, memorize where the number is written,in all number in 1 to $H \times W$.

Consider as number i is written in (px_i, py_i).

Next,D is common in every query,so thinking about the way to use D.

$R_i - L_i$ can divided by D,so it is OK to use cumulative sum.

Consider as

- $d_i = 0$ $(1 \leq i \leq D)$
- $d_i = d_{i-D} + |px_i - px_{i-D}| + |py_i - py_{i-D}|$ $(D+1 \leq i)$

,the sum of magic points consumed to move a piece from L to R in test is $d_R - d_L$.

```
#include <cstdio>
#define abs(x) ((x>0)?x:(-(x)))
int H,W,D,A;
int Q,L,R;
int px[90001],py[90001];
int d[90001];
int main()
{
        scanf("%d%d%d",&H,&W,&D);
        for(int i=0;i<H;i++)
        {
                for(int j=0;j<W;j++)
                {
                        scanf("%d",&A);
                        px[A]=i,py[A]=j;
                }
        }
        for(int i=D+1;i<=H*W;i++)
                d[i]=d[i-D]+abs(px[i]-px[i-D])+abs(py[i]-py[i-D]);
        scanf("%d",&Q);
        while(Q--)
        {
                scanf("%d%d",&L,&R);
                printf("%d\n",d[R]-d[L]);
        }
}
```

AtCoder Beginner Contest 096 解説

問題作成者：E869120, square1001

2018 年 5 月 5 日

For international readers: English editorial starts from page 5.

A – Day of Takahashi

1 月 1 日から a 月 b 日までの間には，少なくとも 1 月 1 日，2 月 2 日,…，$a-1$ 月 $a-1$ 日の，合計 $a-1$ 日の高橋があり，これはどのような (a,b) の組に対しても成り立ちます．

また，a 月 a 日も高橋であるので，$a \leq b$ であれば追加で 1 日を足せばよいです．よって，答えの場合分けは以下のようになります．

- $a \leq b$ のとき，高橋は a 日
- そうでないとき，高橋は $a-1$ 日

C++ や Java などのプログラミング言語では，if-else 文を用いて条件分岐をすることで，場合分けをすることができます．

ソースコード（C++）：https://beta.atcoder.jp/contests/abc096/submissions/2456270
ソースコード（Java）：https://beta.atcoder.jp/contests/abc096/submissions/2457128

B – Maximum Sum

A, B, C は正の数なので，「一番大きい数を 2 倍することを K 回繰り返す」という方法が、最終的な $A + B + C$ の値を最大化する方法です．証明は以下のようになります．

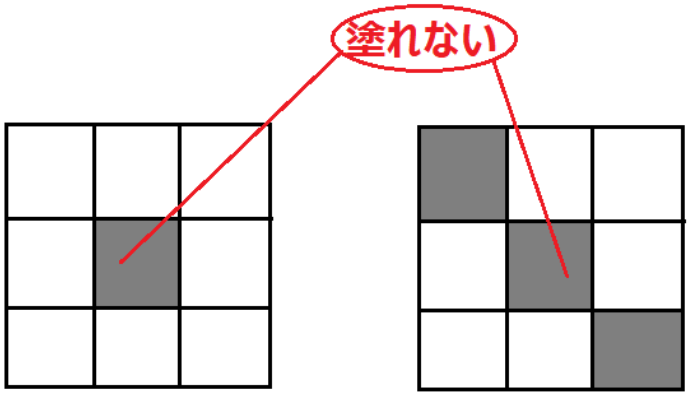
- $K = 1$ の時は，数 p を 2 倍すると数の合計が p 増えるので明らか
- $K = 2$ の時は，一番大きな数を 2 倍してもこの数が 3 つの数の中で一番大きいのに変わりはないから，$K = 1$ の時の状態に持っていける．残り 2 回のときの増分を最大化しながら 残り 1 回の時の増分も最大化できるので，自明にこの方法が最適．
- $K \geq 3$ のときも同様に，帰納的に示せる．

また，よく考えてみると，この方法では操作で選ぶのはすべて 1 つの整数に限られます．これは A, B, C の中で最大の整数なので，$A + B + C + \max(A, B, C) \times (2^K - 1)$ が答えとなります．

ソースコード (C++)：https://beta.atcoder.jp/contests/abc096/submissions/2456285
ソースコード (Java)：https://beta.atcoder.jp/contests/abc096/submissions/2457147

C – Grid Repainting 2

以下の図のような「上下左右に隣り合うマスに黒いマスがない，黒いマス」がある場合を考えましょう.



例 1.　　例 2.

square1001 君は，あるマスを黒で塗る際に隣り合ったマスも一緒に黒で塗らなければならないので，他の隣り合ったマスを塗らずにある特定のマスのみを塗るのは不可能です．ですので，このような場合における答えは必ず “No” となります.

逆に，それ以外の場合，どの黒いマスに対しても「上下左右に隣り合うマスのうち最低 1 つは黒」という条件を満たすことになります．その場合，全ての黒いマス A に対して、「黒いマス A と，A と隣り合う黒いマスのうちの 1 つ B を両方塗る」という操作をすれば，白にすべきマスを黒にすることなく，黒にすべきマスをすべて黒にすることができます．よって，この場合答えは “Yes” となります。

「上下左右に隣り合うマスに黒いマスがない，黒いマス」が存在するかどうか判定すればよいので，個の問題は計算量 $O(HW)$ で解くことができます.

ソースコード（C++）：https://beta.atcoder.jp/contests/abc096/submissions/2447972

D – Five, Five Everywhere

出力すべき数列の条件はかなり厳しく，例えば「小さい順から、条件を満たすように選んでいく」という手法では $N = 9$ までしか答えを出すことはできません（ちなみに，a = [2, 3, 5, 7, 11, 13, 59, 1597, 34019] が作れる）．全探索のような手法でも難しそうなので，何か別の特別な方法を思いつかなければなりません．

想定解法

数列に含まれる数のうちすべてが「5 で割って 1 余る」数であれば，ここから 5 個の整数を選んだ時にこれらの合計は必ず 5 で割り切れます．また数列の値が素数，すなわち 2 以上の値であることから，5 つの整数の合計は必ず 2×5=10 以上になります．5 で割り切れる正の整数の中で素数なものは 5 以外にないので，「5 で割り切れる 10 以上の整数」は，必ず合成数になります．

したがって，「5 で割って 1 余る，55555 以下の素数」を N 個列挙して，これを答えの数列にすればよいです．その場合，a = [11, 31, 41, 61, 71, 101, 131, 151,…, 1381] という数列を作ることができます．これは「すべての数は 55 555 以下」という制約を余裕で（!）満たします．

また，「5 で割って 1 余る」だけではなく，余りの値を 1,2,3,4 のいずれに設定しても条件を満たす数列を作ることができます．

ソースコード（C++）：https://beta.atcoder.jp/contests/abc096/submissions/2444746

AtCoder Beginner Contest 096 Editorial

Problem Setters : E869120, square1001

May 5th, 2018

A: Day of Takahashi

In this editorial, let $p-q$ be "p-th month and q-th day". For example, $5-5$ means May 5th, and $12-31$ means December 31st.

From $1-1$ to $a-b$, there are at least $a-1$ Takahashis: $1-1$, $2-2$, $3-3$, $\cdots$, $(a-1)-(a-1)$.

$a-a$ is also Takahashi, so if $a \leq b$, the number of Takahashi is a. Otherwise, the number of Takahashi is $a-1$.

If you use programming language like C++ or Java, you can use if-else statement to branch.

Code (C++) : https://beta.atcoder.jp/contests/abc096/submissions/2456270
Code (Java) : https://beta.atcoder.jp/contests/abc096/submissions/2457128

Problem B: Maximum Sum

Solution

The optimal solution is, choosing maximum number among three, for each operation. There is K operations in total, so it should be repeated K times.

The proof that this is an optimal way is following. We assume $A \geq B \geq C$, and it will not lose generality because it is achievable with rearranging some numbers.

Proof

- Case $K = 1$: the "increase" of sum of three numbers will be equal to the chosen number. So, it is obvious that you must choose the maximum number, to achieve the maximum sum.
- Case $K = 2$: Considering $K = 1$, the numbers will be $2A, B, C$. Since $A \geq 1$ from constraints of the problem, the maximum number among three will be $2A$. And choosing this also maximizes the "increase" of sum of three numbers, so "choosing A" → "choosing $2A$" will be the optimal choice.
- Case $K \geq 3$: Using induction, we can prove similarly.

Alternative Solution

Seeing this proof, if $A \geq B \geq C$, we will choose only one number: A. So, after doing K operations, the three numbers will be $A \times 2^K, B, C$. So, in this case, the maximum sum will be $A \times 2^K + B + C$.

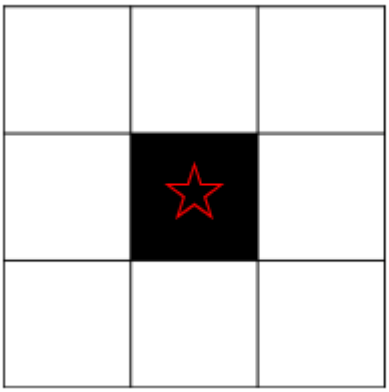
Generally, the maximum sum will be $A + B + C + \max(A, B, C) \times (2^K - 1)$.

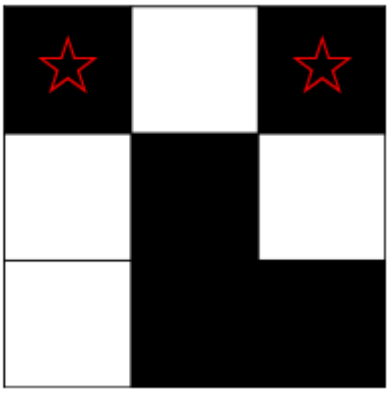
Code (C++) : https://beta.atcoder.jp/contests/abc096/submissions/2456285

Code (Java) : https://beta.atcoder.jp/contests/abc096/submissions/2457147

Problem C: Grid Repainting 2

The following cases are that there are some black cells without adjacent black cells (which is represented as "☆"), in the goal picture.





In these cases, it is impossible to achieve the goal. If you want to paint a "☆" cell black, one of the adjacent cell should be painted black (which should be painted white) because he is not good at drawing pictures.

In other cases, for each black cell X, at least one adjacent cell (let this $f(X)$) is black. Painting "X and $f(X)$" for all X, he can achieve the goal.

Therefore, "there is no '☆' cell in the goal picture" and "he can achieve the goal" is equivalent. You can check whether there are '☆' cells, in straight-forward way, and time complexity will be $O(HW)$.

Code (C++) : https://beta.atcoder.jp/contests/abc096/submissions/2447972

D – Five, Five Everywhere

Since the condition of sequence is very strict, it is difficult to make the sequence in straight-forward way. For example, if you choose from smallest element if the condition is okay, it only passes $N \leq 9$ case, because it constructs $a = [2, 3, 5, 7, 11, 13, 59, 1597, 34019]$. Another possible approach is kind of brute-forcing, but even $N = 15$ is very difficult in this way. So, you should come up with some special way to construct the sequence.

Solution

Suppose that every number in the sequence are "the number which remainder divided by 5 is 1". In this case, if we choose 5 numbers from this sequence, the sum will always be divisible by 5. Also, since each number is at least 2 (because it's a prime number), the sum of five number will be at least 10. Since 5 is the only prime number which is divisible by 5, the sum of five numbers from this sequence will always be a composite number.

Now we should print N distinct prime numbers, which the remainder divided by 5 is 1. We can easily find N $(N \leq 55)$ such numbers less than 55,555.

The sequence when $N = 55$ will be, $a = [11, 31, 41, 61, 71, 101, 131, 151, \ldots, 1381]$. It easily (!) fits in constraints "$a_i \leq 55\,555$".

Code (C++) : https://beta.atcoder.jp/contests/abc096/submissions/2444746

ABC 099 解説

writer: namonakiacc

平成 30 年 6 月 10 日

A: ABD

1 回目から 999 回目のコンテストの名前は ‘ABC‘ から始まり、1000 回目から 1998 回目のコンテストの名前は ‘ABD‘ から始まります。

よって、$N \leq 999$ ならば ‘ABC‘、そうでないならば ‘ABD‘ を出力するとよいです。

```
#include <cstdio>
int N;
int main()
{
        scanf("%d",&N);
        if(N<=999)puts("ABC");
        else puts("ABD");
}
```

B: Stone Monument

高さ $(1+2+3+...+X)$ メートルの塔から、西に 1 メートルの地点に立っている塔の高さは $(1+2+3+...+X-1)$ メートルなので、これら 2 本の塔の高さの差は X メートルとなります。

よって、雪の埋もれていない部分の高さが b メートルの塔の高さは、$(1+2+3+...+b-a)$ メートルであることがわかります。

$(1+2+3+...+X) = X \times (X+1) \div 2$ であることから、雪が $(b-a) \times (b-a+1) \div 2 - b$ メートル積もっていることがわかります。

```
#include <cstdio>
typedef long long ll;
int A,B;
int main()
{
        scanf("%d%d",&A,&B);
        printf("%d\n",(B-A)*(B-A+1)/2-B);
}
```

C: Strange Bank

6^p 円を 6 回引き出す操作は、6^{p+1} 円を 1 回引き出す操作で置き換え可能なため、6^p 円の形で表せる金額それぞれにおいて、 6 回以上引き出すことは考慮に入れなくてよいことがわかります。

9^p の形で表される金額に関しても同様のことが言えます。

そこで、「6^p 円の形で表される金額」のみで A 円を引き出す場合を考えます。同じ金額を 6 回以上引き出すことはないことから、6^i 円は $floor(A \div 6^i)\%6$ 回引き出せばいいことがわかります。

($floor(x)$ は x 以下の最大の整数)

残りの「9^p 円の形で表される金額」についても同様です。

よって、「6^p 円の形であらわされる金額」のみでいくら引き出すかを全通り試すと、正解を得ることができます。

```
#include <cstdio>
int N;
int main()
{
        scanf("%d",&N);
        int res=N;
        for(int i=0;i<=N;i++)
        {
                int cc=0;
                int t=i;
                while(t>0)cc+=t%6,t/=6;
                t=N-i;
                while(t>0)cc+=t%9,t/=9;
                if(res>cc)res=cc;
        }
        printf("%d\n",res);
}
```

D: Good Grid

$(i+j)\%3 = 0,1,2$ の場合の色の組み合わせを全列挙すると、最大で $30 \times 29 \times 28 = 24360$ 通りです。

これら全ての組み合わせにおいて、全てのマスを確認すると時間内に正解を得ることができません。

そこで、$(i+j)\%3 = 0,1,2$ の場合それぞれにおいて、どの色で最初に塗られているマスがいくつあるかをあらかじめ計算しておくと、この部分が $O(C)$ となり、時間内に正解を得ることができます。

```
#include <cstdio>
#define rep(i,n) for(int i=0;i<n;i++)

int N,C,X;
int D[30][39];
int t[3][30];

int main()
{
        scanf("%d%d",&N,&C);
        rep(i,C)rep(j,C)scanf("%d",&D[i][j]);
        rep(i,N)rep(j,N)scanf("%d",&X),t[(i+j)%3][X-1]++;
        int res=1<<30;
        rep(i,C)rep(j,C)if(i!=j)rep(k,C)if(i!=k&&j!=k)
        {
                int tt=0;
                rep(l,C)tt+=D[l][i]*t[0][l];
                rep(l,C)tt+=D[l][j]*t[1][l];
                rep(l,C)tt+=D[l][k]*t[2][l];
                if(tt<res)res=tt;
```

```
        }
        printf("%d\n",res);
}
```

AtCoder Beginner Contest 100 Editorial

by E869120, square1001

June 16th, 2018

For international readers: English editorial starts from page 6.

全体の講評

問題	正解者数	提出数	難易度*	First AC	最速実行	最短コード
A: Happy Birthday!	2321	3656	150	0:40	0ms	19 bytes
B: Ringo's favorite numbers	1806	6710	280	1:40	0ms	22 bytes
C: *3 or /2	1887	2433	270	1:02	1ms	42 bytes
D: Patisserie ABC	574	1636	450	5:48	1ms	138 bytes

※コンテスト中の結果です。また、難易度* はコンテスト中の得点で換算した予想難易度です。

今回は、ABC 100 回目なので全体の講評を書きました。
皆さんコンテストに参加いただきありがとうございました。

今回のコンテストは、ABC 100 回目と記念すべき回なので、誕生日などに多く食べることのできる「ケーキ」をテーマにした問題が多くありました。
例えば、A 問題。これは設定が誕生日でしたが、実は ABC100 回目記念だからケーキにしたという隠された意図がありました。あなたは読み取れましたでしょうか？

また、今回は制約に「100 の倍数」が多く出現しました。例えば、D 問題の制約は $N \leq 10000$ かつ $a_i \leq 10^{10}$ ですね。10 や 100 が多く含まれるように意図的に設定しました。ちなみに B 問題の「100 で割った回数」の 100 も ABC100 由来です。

という訳で、皆さん解説の最初のページをお読みいただきありがとうございました。解説本体もお楽しみくださいませ。

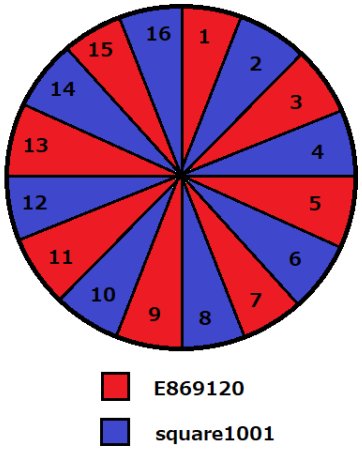
解説自体は、2 ページ目からとなります。

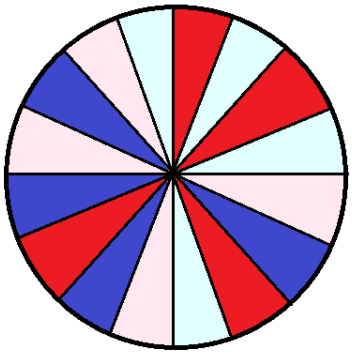
問題 A：Happy Birthday!

答えは、 $X \leq 8$ かつ $Y \leq 8$ のとき答えは “Yay!” となり、それ以外のとき答えは “:(“ となります。

まず、同じ人が取るケーキが隣り合ってはいけないので、16 ÷ 2 = 8 個を超えて置くことはできません。つまり、どちらかが 9 以上のとき食べたい数のケーキを食べることができないことが示せます。

また、$X \leq 8, Y \leq 8$ のとき、E869120 君が時計回りで 1, 3, 5, 7, 9, 11, 13, 15 番目の中から、square1001 君が時計回りで 2, 4, 6, 8, 10, 12, 14, 16 番目の中から選ぶと、食べたい数のケーキを、取り方の条件を満たしながら食べることができます。





[Example: A = 4, B = 4]

これが分かれば、if 文などを用いて条件分岐をするだけです。

サンプルコード： https://beta.atcoder.jp/contests/abc100/submissions/2663851

問題 B：Ringo’s Favorite Numbers

この問題には、たくさんの解法があります。そのうち代表的な 2 通りの解法を説明します。

方針 1：「全探索」

「整数 x が与えられる。そのとき、x は 100 で何回割り切れるか？」という問題を解くことを考えます。これは、x を 100 で割り切れないようになるまで割り続けることによって、解くことができます。

答えが K の場合、$x = 1,2,3,\ldots,K$ について先ほどの問題を解くことになります。先ほどの問題は十分高速に動作し、K の値は高々 1010000（$D = 2, N = 100$ のとき）なので、実行時間制限に余裕で間に合うプログラムを書くことができます。

サンプルコード： https://beta.atcoder.jp/contests/abc100/submissions/2669995

方針 2：「条件に当てはまる数を列挙」

100 でちょうど D 回割り切れるような数は、$a = 100^D$ として次のようになります：

$a, 2a, 3a, 4a, \ldots, 99a, 101a, 102a, 103a, \ldots, 199a, 201a, 202a, 203a, \ldots, 299a, 301a, \ldots$

すなわち、$m \times 100^D$（m は 100 の倍数ではない）と表される数だけが 100 でちょうど D 回割り切れます。これをもとに解法を考えてみましょう。

$q =$（$N - 1$ を 99 で割った商）とします。そのとき、この問題の答えは $(100q + 1) \times 100^D$ 以上 $(100q + 99) \times 100^D$ 以下の値になります。また、$100q \times 100^D$ 以下に「100 でちょうど D 回割り切れるような数」は $99q$ 個あるので、先ほど答えを限定した範囲の中で $N - 99q$ 番目になります。したがって、答えは $(100q + N - 99q) \times 100^D$ になります。

より簡略化すると、答えは $(N - q) \times 100^D = \left(N - \left\lfloor \frac{N-1}{99} \right\rfloor\right) \times 100^D$ になります。

方針 2+α：「ズルい解法：N≤100 までを利用して解く」

「方針 2」の考察で、$a = 100^D$ として、100 でちょうど D 回割り切れるような数の最初 100 個は $a, 2a, 3a, 4a, \ldots, 97a, 98a, 99a, 101a$ になります。

したがって、$N \leq 100$ を仮定すると、$N \leq 99$ のとき $Na = N \times 100^D$ が答えとなり、$N = 100$ のとき $101a = 101 \times 100^D$ が答えになります。

サンプルコード： https://beta.atcoder.jp/contests/abc100/submissions/2663837

問題 C：*3 or /2

$N \leq 10000, a_i \leq 10^9$ なので，操作をたくさん行ったときに、最終的な値が 90000 桁近くなるケースがあります。これは、仮に最適な方法が分かっていたとしても、シミュレーションをするのには桁数が大きすぎるので、何か別の方法を考えなければなりません。

まず、「3 倍する」操作を何回かやっておくことで、「2 で割る」操作の回数が増えたり減ったりするでしょうか？答えは No です。なぜなら、奇数を掛けて 2 で割れる回数が増えることはなく、整数を掛けて 2 で割れる回数が減ることがないからです。

したがって、「3 倍する」操作はその後の操作回数に何ら影響を及ぼしません。すなわち、「3 倍する」は「何もしない」という条件に置き換えることができます。よって、各操作は「N 個すべての数に対して、その数を「2 で割る」または「そのままにする」のどちらかを選ぶが、全部そのままにすることはできない」というように置き換えられます。

次に、各操作について考えてみましょう。「2 で割る」ことによって残り操作回数は減りますが、「そのままにする」は残り操作回数が減らないので、各操作で「2 で割る」個数を最小化、すなわち「2 で割る」個数を 1 個にすることを考えます。したがって、1 回の操作で「N 個の数の中から 1 つ選び、これを 2 で割る」に置き換えることができます。

あとはウィニングランです。答えは (a_1 が 2 で割れる回数) + (a_2 が 2 で割れる回数) + … + (a_N が 2 で割れる回数) となります。x が 2 で割れる回数は単純にシミュレーションしても計算量 $O(log\, x)$ で求めることができるので、全体の計算量は $O(N\, log(max\, a_i))$ になります。

おまけ：計算量 $O(N\, log\, log\, (max\, a_i))$ で求める方法もあります。x が 2 で割れる回数を二分法で求めることによって、この計算量を達成することができます。

サンプルコード： https://beta.atcoder.jp/contests/abc100/submissions/2670458

問題 D：Patisserie ABC

まず、以下の簡単な問題を考えてみましょう。

> N 個のケーキがあり、綺麗さ・おいしさ・人気度が決まっている。
> M 個のケーキを選ぶ。
> （綺麗さの合計） ＋ （おいしさの合計） ＋ （人気度の合計） の最大値を求めよ。

この問題は簡単に解くことができます。ケーキ i の綺麗さを x_i、おいしさを y_i、人気度を z_i とすると、求める値が $(x_i + y_i + z_i)$ の合計になるので、$(x_i + y_i + z_i)$ の値の大きい順に M 個取るという解法が通用します。

しかし、本来の問題は、（綺麗さの合計の**絶対値**）＋（おいしさの合計の**絶対値**）＋（人気度の合計の**絶対値**）を最大化する問題でした。
すなわち、いくつかの要素の合計を負にする、という手も存在します。

ですので、綺麗さ・おいしさ・人気度のそれぞれの要素について、「正の方向に最大化する」「負の方向に最大化する」のどちらを選ぶか、ということを考えると分かりやすいです。要素は 3 つあるので、$2^3 = 8$ 通り全探索します。
例えば、綺麗さ・おいしさを正の方向に最大化、人気度を負の方向に最大化したい場合、$x_i + y_i - z_i$ の大きい順にソートして、上から M 個のケーキを選ぶと良いです。

答えは全探索した 8 通りの中で最も良い解となります。全探索はビット演算などを用いると簡単になります。計算量は $O(N \log N) * 8 = O(N \log N)$ となります。

また、この問題は実は $O(N)$ で解くことができます。長さ N の数列をソートして M 番目に大きい値 A を最悪計算量 $O(N)$ で求めるアルゴリズムは このリンク に書かれており、値 A が求まった後は求めた値より大きい値の合計を計算すれば良いので、この問題の最悪計算量は $O(N)$ となります。

サンプルコード： https://beta.atcoder.jp/contests/abc100/submissions/2670489

Overall Analysis / Review

Problem	Corrects	Submissions	Difficulty*	First AC	Fastest	Shortest
A: Happy Birthday!	2321	3656	150	0:40	0ms	19 bytes
B: Ringo's favorite numbers	1806	6710	280	1:40	0ms	22 bytes
C: *3 or /2	1887	2433	270	1:02	1ms	42 bytes
D: Patisserie ABC	574	1636	450	5:48	1ms	138 bytes

※Expected difficulty is a score in this problem's difficulty. It's only a guess from a formula.

Thank you for everyone to participating the contest or seeing this editorial! This contest is exactly 100th ABC, so we wrote a new thing, "analysis / review".

Since this contest is an anniversary, some problems' theme was "cake", which is commonly eaten on birthdays (at least in Japan).

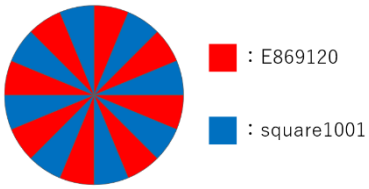
For example, problem A. This was about E869120 and square1001's 16th birthday, but there was a hidden intent to celebrate this ABC 100 anniversary. Or did you make a guess correctly?

And also, in this contest, many "multiple of 100" were appeared. For example, the constraints of problem D was $N \leq 10000 = 100^2$ and $a_i \leq 10^{10} = 100^5$. And the problem B's "the number of times that can be divided by 100" is derived from the ABC "100". Also, there are many 10's because square root of 100 is exactly 10.

Anyway, thanks for reading the first page of English editorial. Please enjoy the main body of this editorial :)

The main body of this editorial starts from the next page.

Problem A : Happy Birthday!



First, consider the case of $X \leq 8$ and $Y \leq 8$ (both wants no more than 8 pieces of cakes). In this case, if you only allow E869120 taking X pieces in red area and square1001 taking Y pieces in blue area, it will satisfy the condition (no two adjacent cakes are for same person). Therefore, the answer for this case is “Yay!”.

Second, if one person wants to take more than 8 cakes, it will definitely not satisfy the condition, so the answer for this case is “:(“.

If you realize this, remaining is only some implementation. The implementation of this problem is as easy as a piece of cake. You can use if-else statement to branch two conditions.

Sample Code (C++) : https://beta.atcoder.jp/contests/abc100/submissions/2663851

Problem B : Ringo's Favorite Numbers

There are many possible solutions in this problem. This time, we will explain two representative solutions.

Approach 1 : Search Everything

Consider this sub-problem: "You are given an integer x. How many times can x be divided by 100?". This can be solved by simulation and it works very fast.

If the answer is K, we should solve this sub-problem for $x = 1,2,3,\ldots,K$. Since K is no more than 1010000 (when $D = 2, N = 100$), you can write a program which is much faster than execution time limit.

Sample Code (C++) : https://beta.atcoder.jp/contests/abc100/submissions/2669995

Approach 2 : "Pencil and Paper" Approach

Actually, you don't have to do brute-forcing (searching all possible answer). Actually, this problem can be solved by pencil and paper (if D, N is fixed).

Let $a = 100^D$. The number which can be divided by 100 exactly D times is following:

$$a, 2a, 3a, 4a, \ldots, 99a, 101a, 102a, 103a, \ldots, 199a, 201a, 202a, 203a, \ldots, 299a, 301a, \ldots$$

This means, only $m \times 100^D$ (m is not a multiple of 100) can be divided by 100 exactly D times. Let's think about the solution based on this property.

Let q the quotient of $N - 1$ divided by 99. The answer of this problem will be between $(100q + 1) \times 100^D$ and $(100q + 99) \times 100^D$. Also, there are $99q$ numbers at most $100q \times 100^D$ and can be divided by 100 exactly D times. This means the answer will be ranked $N - 99q$ from smallest value which we confined the range of the answer a while ago. Therefore, the answer will be $(100q + N - 99q) \times 100^D$.

Simplifying this, the answer will be $(N - q) \times 100^D = \left(N - \left\lfloor \frac{N-1}{99} \right\rfloor\right) \times 100^D$.

Approach 2+α : "Crafty solution" using N≤100

Let $a = 100^D$. Seeing Approach 2, the first 100 numbers which can be divided by 100 exactly D times are $a, 2a, 3a, 4a, \ldots, 97a, 98a, 99a, 101a$. Therefore, assuming $N \leq 100$, the answer will be $Na = N \times 100^D$ if $N \leq 99$ otherwise $101a = 101 \times 100^D$.

Sample Code (C++) : https://beta.atcoder.jp/contests/abc100/submissions/2663837

Problem C: *3 or /2

Let's see the characteristics of "*3" or "/2". You can "multiply a_i by 3" unlimited time, but you cannot "divide a_i by 2", because you cannot choose this operation if a_i is an odd number.

So, the optimal solution seems like as follows:

- For each operation, divide exactly 1 number which is an even number, and multiple other numbers.

Anyway, how many times will you divide?

Let $f(x)$ be "the number of times that x can be divided by 2". For example, $f(8)$ = 3, $f(244)$ = 2 and $f(100)$ = 2.

If you choose "multiple a_i by 3", $f(a_i)$ should always be unchanged. Otherwise (if you choose "divide a_i by 2"), $f(a_i)$ should always decrease by 1.

Finally, if all $f(a_i)$ became zero (This means that all number is odd), you cannot do operation anymore.

So, the maximum number of operation is $f(a_1) + f(a_2) + \cdots + f(a_N)$. Since you can determine $f(x)$ with $O(log\, x)$ complexity, the complexity of this algorithm is $O(N\, log\, max\, a_i)$.

Sample Code: https://beta.atcoder.jp/contests/abc100/submissions/2670458

Bonus: You can solve the problem with complexity $O(N \log\log \max a_i)$. Because, if you use binary-search by the value of $f(x)$, you can determine $f(x)$ with complexity $O(\log\log x)$.

Problem D：Patisserie ABC

First, let's think about this easy sub-problem:

> *There are N cakes, which the value of beauty, tastiness, and popularity.*
> *You will choose M cakes.*
> Find the maximum possible value of (sum of beauty) + (sum of tastiness) + (sum of popularity).

This sub-problem can be solved easily. If cake i's beauty is x_i, tastiness is y_i, and popularity is z_i, the answer will be sum of $(x_k + y_k + z_k)$ when k is the chosen cakes. Therefore, choosing M cakes from higher $(x_i + y_i + z_i)$ is the optimal choice.

But the original problem is to maximize (absolute value of sum of beauty) + (absolute value of sum of tastiness) + (absolute value of sum of popularity). So, there's also a way to make some parameters' sum to negative.

Thus, you can understand easily, if you think about "maximizing force toward positive direction" or "maximizing force toward negative direction" for each parameter: beauty, tastiness, and popularity. Since there are three parameters, you can brute-force $2^3 = 8$ ways.

For example, if you want sum of beauty and sum of tastiness to maximize force toward positive, and want sum of popularity to maximize force toward negative, choosing M cakes from higher $(x_i + y_i - z_i)$ is the optimal choice.

The answer will be the maximum value among $2^3 = 8$ ways. It is possible to use recursion for brute-forcing, but using "brute-force with bit operations" will make easier implementation. Supposing that you sort a length-n array in $O(n \log n)$, the time complexity will be $O(N \log N) \times 8 = O(N \log N)$.

Sample Code (C++)： https://beta.atcoder.jp/contests/abc100/submissions/2670489

Bonus： This problem can be solved in linear time. First, you can calculate m-th biggest value in linear time. The algorithm called "Median of medians" enables calculating median in linear time, and then binary-searching the array, the m-th biggest value can be calculated in linear time. The sum of number from 1st to m-th biggest value will be calculated by, simply, "summing up the value larger than m-th biggest value", plus we need some adjustment.

ABC 103 解説

writer: drafear

2018 年 7 月 21 日

A: Task Scheduling Problem

タスクを完了する順序を全通り調べて最小値を求める方法でも解けますが、実装が若干大変です。実は、A_i の小さいタスクから順に、または A_i の大きいタスクから順に完了したときに最小となり、A_1, A_2, A_3 の最大値と最小値の差が答えになります。

C++ 言語による解答例

```
#include <iostream>
#include <algorithm>

using namespace std;

int main() {
  int A1, A2, A3; cin >> A1 >> A2 >> A3;
  int ans = max({A1, A2, A3}) - min({A1, A2, A3});
  cout << ans << endl;
}
```

B: String Rotation

$|S|$ 回操作を行うと元に戻るので、0 回以上 $|S|-1$ 回以下の回数操作を行った後に T と一致するものがあるか調べれば十分です。

C++ 言語による解答例

```
#include <iostream>

using namespace std;

int main() {
  string S, T; cin >> S >> T;
  bool ans = false;
  for (int i = 0; i < S.size(); ++i) {
    if (S == T) {
      ans = true;
      break;
    }
    S = S.back() + S.substr(0, S.size()-1);
  }
  if (ans) {
    cout << "Yes" << endl;
  }
  else {
    cout << "No" << endl;
  }
}
```

C: Modulo Summation

$m = a_1 \times a_2 \times \cdots \times a_N$ とすると、各 i について $m \ mod \ a_i = 0$ なので、$(m-1) \ mod \ a_i = a_i - 1$ です。したがって $f(m-1) = (a_1 - 1) + (a_2 - 1) + \cdots + (a_n - 1)$ となり、各項が最大値を取っているのでこれが f の最大値です。実装では、m は非常に大きいため、m を計算せずに $f(m-1)$ を直接計算すればよいです。よって、$O(n)$ でこの問題が解けました。

D: Islands War

まず、(a_i, b_i) の組を b_i について昇順にソートします。ソートした順に、以下を行います。

- 既にその要望を満たしている場合、すなわち、既に 1 本以上橋を取り除いていて、直前に取り除いた橋によってその要望が満たされている場合、何もしない。ここで、直前に取り除いた橋によってその要望が満たされている場合とは、西から x 番目の橋を直前に取り除いたとすると、$a_i \leq x$ であることである。
- まだその要望を満たしていない場合、西から $b_i - 1$ 番目の橋、すなわち西から $b_i - 1$ 番目の島と b_i 番目の島を接続する橋を取り除く。

このとき、最適となります。なぜなら、後者の場合について、西から a_i 番目, $a_i + 1$ 番目, ..., $b_i - 1$ 番目の橋のうち 1 つ以上取り除かなければなりませんが、$b_i - 1$ 番目より手前の橋 (西から x 番目の橋とする) を取り除いたとき、後に残る部分問題 (西から $x + 1$ 番目以降の島と橋のみについての問題) を考えれば必ず損するからです。したがって、ソートがボトルネックとなり、$O(nlogn)$ で解けました。
余談: 実は $O(n)$ でも解くことができます。興味があれば考えてみてください。

ABC 104 Editorial

問題・解説: @evima0

2018 年 8 月 5 日

A: Rated for Me

この問題を解くには、以下の手順を踏む必要があります。

0.（言語によっては不要）整数変数 R を宣言する。
1. 標準入力から R を受け取る。
2. R から何らかの方法で答えとなる文字列を得る。
3. 求めた文字列を標準出力に出力する。

手順 0, 1, 3 については、practice contest の問題 A の言語別サンプルコードが参考になる（ほぼそのまま使える）でしょう。手順 2 については、if, else などの構文を用いることになります。C++ による実装例を示します。

```cpp
#include <iostream>
using namespace std;
int main(){
    int R;
    cin >> R;
    string ans;
    if(R < 1200){
        ans = "ABC";
    }else if(R < 2800){
        ans = "ARC";
    }else{
        ans = "AGC";
    }
    cout << ans << endl;
}
```

B: AcCepted

「先頭から 3 文字目から最後から 2 文字目までに C がちょうど 1 個現れ、それ以外の文字はすべて小文字である」という内容を正確に実装することにほぼ終始します（「それ以外の文字」に先頭から 2 文字目と最後の文字も含まれることに注意）。
素直に実装するには、for 文を用いてすべての文字の case（大文字か小文字か）を確かめます。一つ一つの文字が大文字か小文字かを判定するには、言語ごとに用意されたメソッドを使うか、ASCII コードでの文字の並び順を利用します。C++ による実装例を示します。

```cpp
#include <iostream>
using namespace std;
int main(){
    string S;
    cin >> S;
    int L = S.size();
    string ans = "AC";
    if(S[0] != 'A'){
        ans = "WA";
    }
    int cnt = 0;
    for(int i = 1; i < L; ++i){
        if(isupper(S[i])){
            if(i == 1 || i == L - 1 || S[i] != 'C'){
                ans = "WA";
            }
            ++cnt;
        }
    }
    if(cnt != 1){
        ans = "WA";
    }
    cout << ans << endl;
}
```

C: All Green

実行時間制限が許すならば、100 点の問題を何問解き、200 点の問題を何問解き、…という配点ごとの解く問題の数の組み合わせをすべて試して最良のものを採用すればよいです。しかし、最大で 10 種類の配点それぞれに対して問題が最大で 100 問存在するため、このような組み合わせは最大で 101^{10} 通り存在します。これらをすべて試すには 1 万年以上の時間が必要となるでしょう。

しかし、最適解を求める上で 101^{10} 通りすべての組み合わせを試す必要はありません。例えば、100 点の問題と 200 点の問題が 10 問ずつあったとして、最終的にこれらを 3 問ずつ解く、という選択を考慮する必要はありません。なぜなら、その代わりに 200 点問題を 6 問解くことで、解く問題数を増やさずに総合スコアを高くすることができるからです。

以下、「$100i$ 点の問題を p_i 問解く」ことを「$100i$ 点を完全に解く」、「$100i$ 点の問題を 1 問以上 $p_i - 1$ 問以下解く」ことを「$100i$ 点を中途半端に解く」、「$100i$ 点の問題を 0 問解く」ことを「$100i$ 点をまったく解かない」と表現します。上の議論を一般化すると、以下のような選択のみを考慮しても最適解はそれらの選択に含まれます。

- 中途半端に解く配点は 1 種類以下であり、それ以外の配点は完全に解くかまったく解かない。
- 中途半端に解く配点があるなら、それは完全に解く配点以外の配点の中で最も高い配点である。

このような選択をすべて列挙するには、D 種類の配点それぞれに対してその配点を完全に解くか否かを決定し、必要であれば残りの配点のうち最も高いものを中途半端に解く配点とすればよいです（中途半端に解く配点の問題を 1 問除きすべて解いても目標点に達しない場合は、完全に解く配点の選択が不適切であったとして無視します）。

この方針を実装するには、再帰関数を定義するか、0 以上 $2^D - 1$ 以下の整数それぞれの 2 進数表記を D 回の選択を表すリストのようなものとして用いる（下から i 桁目 $(0 \leq i < D)$ が 1 なら $100(i+1)$ 点を完全に解き、0 なら完全には解かない）のが素直でしょう。素直な実装で $O(2^D D)$ 時間の解法が得られます。

参考実装 (提出 #2954675, C++)

D: We Love ABC

表面的なことですが、文字列 S の ABC 数を「S の文字のうち 3 文字に丸をつけ、丸をつけた文字を左から読むと `ABC` となるようにする方法」と捉えると考えやすいかもしれません。求めたいのは、「S に含まれる `?` をそれぞれ `A`, `B`, `C` のいずれかに書き換えつつ、S の文字のうち 3 文字に丸をつけて、丸付きの文字が左から `ABC` となるようにする方法」です。

このように考えると、動的計画法（リンク先は Wikipedia の同名の記事）の適用が見えてきます。値 $dp_{i,j} (0 \leq i \leq |S|, 0 \leq j \leq 3)$ を「S の先頭から $i-1$ 文字目までに対する処理（`?` の置換と丸つけ）をすでに行っていて、これまでに丸を j 個つけている場合の、S の残りの部分に対する処理を行う方法の個数」とすると、以下が成り立ちます（文字列の先頭の文字を 0 文字目とします）。

- $dp_{|S|,3} = 1$
  （処理が正常に終了。これから行うべきことは「何もしない」の 1 通り）
- $j < 3$ のとき、$dp_{|S|,j} = 0$
  （丸を 3 個つける前に最後の文字まで見終わってしまい、手遅れ。これからできることは 0 通り）
- $i < |S|$ のとき、$dp_{i,3} = m \times dp_{i+1,3}$, ここで $S_i =$ `?` なら $m = 3$, そうでなければ $m = 1$
  （丸は 3 個つけ終わったので、あとは `?` を置換するのみ）
- $i < |S|, j < 3$ のとき、$dp_{i,j} = m_1 \times dp_{i+1,j} + m_2 \times dp_{i+1,j+1}$,
  ここで $S_i =$ `?` なら $m_1 = 3$, そうでなければ $m_1 = 1$ であり、
  $S_i =$ `?` または S_i が `ABC` のうち j 番目（0 から数えて）の文字なら $m_2 = 1$, そうでなければ $m_2 = 0$
  （式の前半部分は i 文字目に丸をつけない場合、後半部分は丸をつける場合に対応）

これを元に $dp_{i,j} (0 \leq i \leq |S|, 0 \leq j \leq 3)$ の値を i, j の降順にすべて求めることができ、$O(N)$ 時間の解法が得られます。

参考実装 (提出 #2955456, C++)

ABC 105 解説

drafear, E869120, square1001, DEGwer

2018 年 8 月 11 日

A: AtCoder Crackers

N が K の倍数の時、均等に配ることができるので答えは 0 です。そうでない場合、答えを 0 にはできず、また N を K で割った商とあまりをそれぞれ A, B として、B 人に $A+1$ 枚、$K-B$ 人に A 枚のせんべいを配ればよく、よって答えは 1 です。

C++ では $true, false$ がそれぞれ $0, 1$ に対応することを用いれば、以下のように実装することもできます。

```
#include<stdio.h>
int main()
{
        int a, b;
        scanf("%d%d", &a, &b);
        printf("%d\n", a%b != 0);
}
```

B: Cakes and Donuts

解法 1：全パターン探す

ケーキとドーナツ合計で N 円ということは, ケーキの個数は 0 個以上 $N/4$ 個以下、ドーナツの個数は 0 個以上 $N/7$ 個以下ということが分かります。

この個数の範囲で、ケーキの個数 A ドーナツの個数 B を全探索すればよいです。$4A+7B=N$ となるような (A, B) の組があれば "Yes", なければ "No" を出力すればよいです。

解法 2：数学的に解く

実は、答えが "No" となるのは $N = 1, 2, 3, 5, 6, 9, 10, 13, 17$ の場合のみですので、if 文を用いて場合分けして解くことができます。

証明：まず N が 21 以上の場合に、答えが必ず "Yes" であることを証明します。まず、$N-0, N-7, N-14, N-21$ (それぞれドーナツを $0, 1, 2, 3$ 個買う場合) のうち 4 の倍数であるものは必ず 1 つあります。これを選べば、残りをすべてケーキにすることで適切に買うことができます。N が 20 以下の場合は、「解法 1」の方法で証明することもできますが、実際には表などを使って手計算でやる方が簡単です。

サンプルコード (C++)

解法 1：https://beta.atcoder.jp/contests/abc105/submissions/2990812
解法 2：https://beta.atcoder.jp/contests/abc105/submissions/2991021

C: Base -2 Number

N の -2 進表記の $(-2)^i$ の桁に現れる整数を S_i とします。このとき、S_0 以外の決め方が N を 2 で割ったあまりに影響することはないので、S_0 は N が奇数の時 1 で、そうでないとき 0 にする必要があります。

S_0, S_1 以外の決め方が N を 4 で割ったあまりに影響することはないので、S_0 が決まっている状況下では S_1 も決まります。このようにして、順次各々の S_i の値を決めて行くことができます。また、-2 進 K 桁の数が表すことのできる整数の範囲を考えれば、この繰り返しは高々 $O(\log N)$ 回で終了することも分かり、この問題を解くことができます。

D: Candy Distribution

数列 A に対し、A の先頭 i 項の和を B_i とします。なお、便宜上 $B_0 = 0$ としておきます。

いま、$A_l + ... + A_r = B_r - B_{l-1}$ であり、$A_l + ... + A_r$ が M の倍数であることは B_r, B_{l-1} を M で割ったあまりが等しいことと同値です。よって、数えるべきは、B_i, B_j が $\bmod M$ で等しいような $0 \leq i < j \leq N$ の組の個数です。

これは、各 $x = 0, ..., M-1$ について、B_i を M で割ったあまりが x であるような i の個数を求めておくことで計算可能です。M は大きいので長さ M の配列を持つことはできませんが、平衡二分木 (C++ なら std::map) やハッシュマップ (C++ なら std::unordered_map) を用いることで「B_i を M で割ったあまりが x であるような i が存在するような x」についてのみ値を持っておくことができます。この工夫を用いれば計算量が ($O(N \log N)$ や乱択 $O(N)$ 時間に) 改善でき、この問題を解くことができます。

AtCoder Beginner Contest 106 解説

2018 年 8 月 18 日

E869120, square1001

For international readers: We usually don't have English editorial for ABCs, but this time only!

問題 A: Garden

答えから言うと、$(a-1)(b-1)$ が答えとなります。これを導く方法は、主に 2 通りあります。

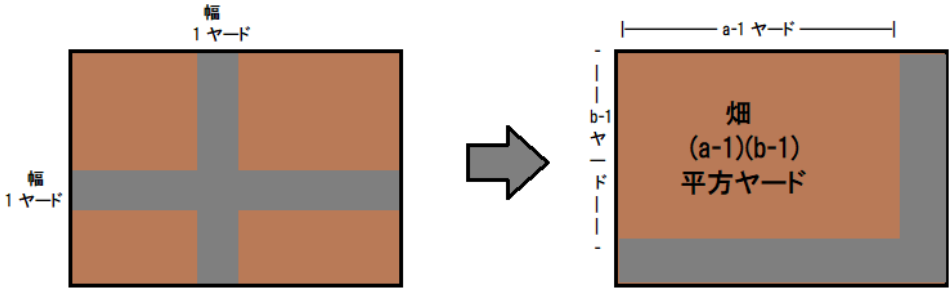
方法 1: 道路の面積から求める

道路の合計面積を求めることを考えます。

縦・横の道路両方重なっている部分が無ければ、$1 \times a + 1 \times b = a + b$ が道路の面積の合計となります。しかし、実際は $1 \times 1 = 1$ 平方ヤード「縦方向・横方向両方の道路が交差している部分」が存在するので、道路となっている面積は $a + b - 1$ 平方ヤードとなります。答えは道路ではない面積なので、$ab - a - b + 1 = (a-1)(b-1)$ となります。

方法 2: 図形をスライドさせる

以下の図のように、縦方向の道路を右端に、横方向の道路を下端に移動させることを考えます。



移動しても道路の幅が変わらないので、自明に求める面積は変わりません。移動した後の畑の面積は、縦 $a-1$ ヤード横 $b-1$ ヤードの長方形と同型となるので $(a-1)(b-1)$ 平方ヤードです。

サンプルコード (C++)

https://beta.atcoder.jp/contests/abc106/submissions/3024930

問題 B: 105

この問題は、ABC 105 に出すつもりで作りましたが、結果的に ABC 106 に出されることになったようです。解法が大きく 2 つあるので、これを書きます。

解法 1: 全部の場合について計算

C の約数の個数は、「C が 1 で割り切れるか？」「C が 2 で割り切れるか？」「C が 3 で割り切れるか？」…「C が C で割り切れるか？」という質問のうち答えが Yes である個数です。

「C が X で割り切れるか？」という質問は、「C を X で割った余りが 0 か？」という質問と同じなので、多くのプログラミング言語では「C % X == 0」かで判定ができます。For 文などを用いて $X = 1, 2, 3, \ldots, C$ までループすると、質問がそれぞれの X に対して実行できます。

それを、$C = 1, 3, 5, 7, \ldots, N$ (あるいは、$C = 1, 3, 5, 7, \ldots, N - 1$) に対して求めて、それぞれに対して約数が 8 個かどうかをチェックします。これにも For 文ループを使うことができます。このように、「2 重ループ」を書けばこの問題の答えを求めることができます。

解法 2: $N \leq 200$ までを利用する，ズルい解法

105 が最も小さい「約数が 8 個ある奇数」なので、200 までに「約数が 8 個ある奇数」が少ないことが想像できるでしょう。実は、約数が 8 個ある奇数は、$105, 135, 165, 189, 195$ の 5 つだけです。したがって、$N \leq 200$ を仮定した場合、答えは次のように場合分けされます。

- $1 \leq N \leq 104$ のとき、答えは 0
- $105 \leq N \leq 134$ のとき、答えは 1
- $135 \leq N \leq 164$ のとき、答えは 2
- $165 \leq N \leq 188$ のとき、答えは 3
- $189 \leq N \leq 194$ のとき、答えは 4
- $195 \leq N \leq 200$ のとき、答えは 5

約数が 8 個ある奇数は、p, q, r を異なる素数として p^7 の形、$p^3 q$ の形、pqr の形以外ありません。これが奇数であるとき、p, q, r はすべて奇数でなければなりません。200 以下に限定すると、pqr の形は $3 \times 5 \times 7 = 105, 3 \times 5 \times 11 = 165, 3 \times 5 \times 13 = 195$ の 3 個あり、$p^3 q$ の形は $3^3 \times 5 = 135, 3^3 \times 7 = 189$ の 2 個あります。

ソースコード (C++)
解法 1: https://beta.atcoder.jp/contests/abc106/submissions/3024975
解法 2: https://beta.atcoder.jp/contests/abc106/submissions/3024982

問題 C: To Infinity

まず、5000 兆回の操作で 1 文字の文字列がどのように変わるかを考えます。

- “1” → “1” → “1” → “1” → … → “1”
- “2” → “22” → “2222” → “22222222” → … → (2 が $2^{5000兆}$ 文字続く文字列)
- “3” → “333” → “333333333” → … → (3 が $3^{5000兆}$ 文字続く文字列)
- “4” → “4444” → “4444444444444444” → (4 が $4^{5000兆}$ 文字続く文字列)
- 5～9 に関しても同じように, 5000 兆回操作を繰り返すと K 文字より明らかに大きくなる

つまり、5000 兆回操作を行った後の文字列は以下のように考えられます。

- “1” はそのまま、変わらない
- “2” 以上の文字 A は全て、A だけから成る無限長の文字列に変化する

よって、以下のことが言えます。

- 文字列 S において、1 文字目から K 文字目まで全て ‘1’ であれば、答えは ‘1’
- そうでなければ、答えは S 中に初めて出現する ‘2’ 以上の文字

実装に関しては、For 文で最初の文字から順に K 文字目まで探し、”1” 以外の文字が出てきたときに打ち切る、とすれば簡単にできます。300 bytes 以内の簡単な実装も容易です。

サンプルコード (C++)

https://beta.atcoder.jp/contests/abc106/submissions/3024955

問題 D: AtCoder Express 2

各列車の走る区間は (L_j, R_j) であり、クエリは (p_i, q_i) であるように、それぞれについて「区間の左端」「区間の右端」の 2 つの要素しかないので 2 次元座標として見做すことを考えましょう。

さて、$x_{l,r}$ を 区間 $[l, r]$ を走る列車の個数、とします。クエリ (p, q) に対して、求めたい値は $x_{p,p} + x_{p,p+1} + \cdots + x_{p,q} + x_{p+1,p} + \cdots + x_{q,q}$ となります。つまり、二次元区間に落とし込むと、求めたいものは、例えば囲まれた区間の合計となります。



これを高速に求めるためには、累積和を使うと上手くいきます。この問題は $\mathrm{N} \leq 500$ と小さいため、累積和を全て記録しても実は間に合います。

$c_{i,j} = x_{i,1} + x_{i,2} + \cdots + x_{i,j}$ とします。そうすると、$x_{i,l} + x_{i,l+1} + \cdots + x_{i,r} = c_{i,r} - c_{i,l-1}$ として表されるので、各クエリにつき $O(N)$ で解けます。$Q \leq 100000, N \leq 500$ より、実行時間制限の 3 秒には余裕で間に合うはずです。計算量は $O(QN + N^2)$ です。

ちなみに、二次元累積和を使うとさらに高速化できます。 それを用いると $O(N^2 + Q)$ が達成できますが、本問題ではここまでは要求されません。

サンプルコード

[1] 解法 – https://beta.atcoder.jp/contests/abc106/submissions/3024990

[2] 二次元累積和を用いる解法 - https://beta.atcoder.jp/contests/abc106/submissions/3024983

A – Garden

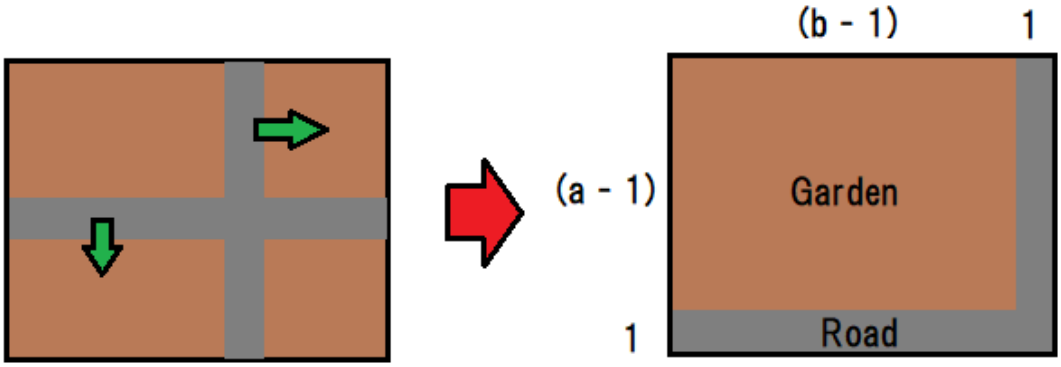
First of all, the answer of this problem is $(a-1)(b-1)$. Here is why this is correct:

[Way 1]

Let's think about the area which covered with road. If there was no area which is covered with both road which are horizonal and vertical, the area should be $a+b$. However, $1*1=1$ square yard is covered with both two roads. So, the answer of this problem is $ab-(a+b-1)=(a-1)(b-1)$.

[Way 2]

Let's think about moving horizonal road to right, and vertical road to left. Obviously, the garden will become a rectangle of $(a-1)*(b-1)$, so the area is obviously $(a-1)*(b-1)$.



Sample Code

- https://beta.atcoder.jp/contests/abc106/submissions/3024930

Problem B: 105

This problem has a background that we made this problem to publish in AtCoder Beginner Contest 105, but it was published in AtCoder Beginner Contest 106.

Solution 1: Compute for all possible cases

First, let's think about counting the number of divisors of C. We can divide C into $1, 2, 3, \ldots, C$, and check the remainder respectively. If the remainder of dividing into X is zero, it means X is a divisor of C. It is convenient if we use loop (for example, for-statement).

Now, we can check for $C = 1, 3, 5, 7, 9, \ldots, N$ (or, $C = 1, 3, 5, 7, 9, \ldots, N-1$ if N is even). We can also use loop like for-statement. It means, one way to implement the solution is, using "double-loop".

Solution 2: Tricky solution using $N \leq 200$

Using "105 is the least odd integer that there are exactly 8 divisors", we can imply that there are only few odd integers below 200, that there are exactly 8 divisors.

Actually, the only such numbers below 200, are 105, 135, 165, 189, 195. Assuming $N \leq 200$, the answer can be divided as follows:

- If $1 \leq N \leq 104$, the answer is 0
- If $105 \leq N \leq 134$, the answer is 1
- If $135 \leq N \leq 164$, the answer is 2
- If $165 \leq N \leq 188$, the answer is 3
- If $189 \leq N \leq 194$, the answer is 4
- If $195 \leq N \leq 200$, the answer is 5

The numbers that there are exactly 8 divisors, can be represented with three different prime numbers p, q, r. There are only three patterns: p^7, p^3q, pqr. There are no p^7 pattern below 200. There are two p^3q patterns, $3^3 \times 5 = 135$ and $3^3 \times 7 = 189$. There are three pqr patterns, $3 \times 5 \times 7 = 105, 3 \times 5 \times 11 = 165, 3 \times 5 \times 13 = 195$.

Source Code (C++)

Solution 1: https://beta.atcoder.jp/contests/abc106/submissions/3024975
Solution 2: https://beta.atcoder.jp/contests/abc106/submissions/3024982

C – To Infinity

Let's think about the state of after $5 * 10^{15}$ days, for **one character.**

a. '1' -> '1' -> ⋯ -> '1'
b. '2' -> '22' -> '2222' -> '22222222' -> ⋯ ($2^{5*10^{15}}$ characters)
c. '3' -> '333' -> '333333333' -> ⋯ ($3^{5*10^{15}}$ characters). And so on.
d. For '2', '3', ⋯, '9', it will become more than K characters obviously.

So, you can assume that character '2' – '9' will be infinite length, because it's more than K characters. You can say as follows:

a. If all characters between 1st to K-th character is '1', the answer will be '1'.
b. Otherwise, the answer will be the character of leftmost index which is not '1'. For example, if $K = 10$ and $S =$ "113123", the answer will be 3.

Implementation is easy. You can write with less than 300 bytes.

Sample Code

https://beta.atcoder.jp/contests/abc106/submissions/3024955

Problem D: AtCoder Express 2

Let's think about placing trains into 2-dimensional plane. Think about placing train i in coordinate (L_i, R_i). Then, for each query, since we have to find the number of i that $L_i \geq p$ and $R_i \leq q$, it means we should count the number of "trains which is right-lower than (p, q)".

We can use dynamic programming (also known as "two-dimensional cumulative sum technique"). Since N is no more than 500, the coordinate is positive and less or equal to 500.

Now, let $C_{i,j}$ the number of trains in (i, j). The answer $a_{i,j}$ when $(p, q) = (i, j)$ is, if you think carefully… It will be $a_{i,j} = C_{i,j} + a_{i,j-1} + a_{i+1,j} - a_{i+1,j-1}$.

Then, if we calculate from the larger x-coordinate and smaller y-coordinate, you can calculate all a where $1 \leq x \leq N, 1 \leq y \leq N$, in $O(N^2)$ time.

Now, the answer of query is stored in $a_{p,q}$. It means you use only constant time for each query. The total time complexity is $O(N^2 + M + Q)$.

Source Code (C++):
https://beta.atcoder.jp/contests/abc106/submissions/3024983

ABC 109 解説

writer: drafear

2018 年 9 月 8 日

A: ABC333

$A \times B$ が奇数なら $C = 1$ とすることで $A \times B \times C$ が奇数になります。一方、$A \times B$ が偶数なら、偶数にどんな自然数をかけても偶数のままです。従って、$A \times B$ が偶数かどうかで場合分けするとスマートです。C++ による実装例を以下に挙げます。

```
#include <iostream>

using namespace std;

int main() {
  int A, B; cin >> A >> B;
  if (A * B % 2 == 1) {
    cout << "Yes" << endl;
  }
  else {
    cout << "No" << endl;
  }
}
```

しかし、この事実に気付かなくとも全ての $C = 1, 2, 3$ に対して $A \times B \times C$ の偶奇を調べる方法でも正答できます。その場合の C++ による実装例は次のようになります。

```
#include <iostream>

using namespace std;

int main() {
  int A, B; cin >> A >> B;
  if (A * B * 1 % 2 == 1 || A * B * 2 % 2 == 1 || A * B * 3 % 2 == 1) {
    cout << "Yes" << endl;
  }
```

```
  else {
    cout << "No" << endl;
  }
}
```

B: Shiritori

問題文の条件を言い換えると、次のようになります。

1. $1 \leq i \leq n-1$ を満たす各 i について W_i の最後の文字と W_{i+1} の最初の文字が等しい
2. $1 \leq i < j \leq n$ を満たす各 i, j について $W_i \neq W_j$

これを C++ で実装した例を以下に示します。

```cpp
#include <iostream>
#include <vector>

using namespace std;

bool solve(const vector<string>& W) {
  const int n = W.size();
  // condition 1
  for (int i = 0; i < n-1; ++i) {
    if (W[i].back() != W[i+1].front()) {
      return false;
    }
  }
  // condition 2
  for (int i = 0; i < n; ++i) {
    for (int j = i+1; j < n; ++j) {
      if (W[i] == W[j]) {
        return false;
      }
    }
  }
  return true;
}
vector<string> input() {
  int n; cin >> n;
  vector<string> W(n);
  for (int i = 0; i < n; ++i) {
    cin >> W[i];
  }
  return W;
}
```

```
int main() {
  const vector<string> W = input();
  if (solve(W)) {
    cout << "Yes" << endl;
  }
  else {
    cout << "No" << endl;
  }
}
```

余談: std::set を用いてより高速に判定することもできますが、今回の場合はこれで十分です。

C: Skip

D が全ての i について $|X - x_i|$ の約数であるとき、全ての都市を訪れることができます。逆に、D が $|X - x_i|$ の約数でないような i が存在するとき、操作によって現在居る座標を D で割った余りが変化しないことから、$|X - x_i|$ を D で割った余りが $|X - x_i|$ または $D - |X - x_i|$ となり、0 になることがないため、座標 x_i を訪れることができません。

したがって、$|X - x_1|, |X - x_2|, ..., |X - x_N|$ 全ての約数であって最大の正整数 D を求めれば良いことになります。このような D は最大公約数と定義され、ユークリッドの互除法により高速に計算できることが知られています。ユークリッドの互除法とは、

$$\mathsf{gcd}(a, b) = \begin{cases} \mathsf{gcd}(b, a \bmod b) & \text{if } b > 0 \\ a & \text{if } b = 0 \end{cases}$$

が成り立ち、これに従って計算するアルゴリズムのことです。ここで、$\mathsf{gcd}(a, b)$ は a と b の最大公約数表し、$a \bmod b$ は a を b で割った余りを表します。ユークリッドの互除法は $a \leq b$ のとき 1 回の反復で $a \bmod b < \frac{a}{2}$ となることから $O(log \max(a, b))$ で求まります。ちなみに、a, b がフィボナッチ数のときに遅くなりますが log で求まることには変わりありません。$\mathsf{gcd}(a, b, c) = \mathsf{gcd}(\mathsf{gcd}(a, b), c)$ なので、答えである $\mathsf{gcd}(|X - x_1|, |X - x_2|, ..., |X - x_N|)$ を $O(Nlog \max\{|X - x_1|, |X - x_2|, ..., |X - x_N|\})$ で求められます。

D: Make Them Even

次の手順で、$\sum a_{ij}$ が偶数なら全て偶数にでき、奇数なら 1 マスを除いて全て偶数にできます。
まず、一筆書きの経路を作ります。例えば、

$$(1,1),(1,2),...,(1,W),(2,W),(2,W-1),...,(2,1),(3,1),(3,2),...$$

です。この経路が通る i 番目のマスを (Y_i, X_i) とします。次に、$i = 1, 2, ..., H \times W - 1$ の順に $a_{Y_iX_i}$ が奇数ならマス (Y_i, X_i) からマス (Y_{i+1}, X_{i+1}) に箱を 1 個移動させる操作をします。この操作により、$a_{Y_iX_i}$ に -1、$a_{Y_{i+1}X_{i+1}}$ に 1 が加算され、$a_{Y_iX_i}$ が偶数になります。一連の操作は valid であり、マス $(Y_{H\times W}, X_{H\times W})$ を除いた全てのマスを偶数にできます。時間計算量は $O(HW)$ になります。

ABC 110 解説

writer: drafear

2018 年 9 月 23 日

A: Maximize the Formula

数式 $XY+Z$ と $Z+XY$ の値は同じなので、$XY+Z$ の形のみを考えることにします。さらに、$XY+Z$ と $XZ+Y$ の値は変わりません。従って、作れる数式の値は $10\times X+Y+Z, X+10\times Y+Z, X+Y+10\times Z$ の 3 通りです。これらの最大値が答えになるので、C++ での実装例は次のようになります。

```
#include <iostream>
#include <algorithm>

using namespace std;

int main() {
  int A, B, C; cin >> A >> B >> C;
  int ans = max<int>({
    10 * A + B + C,
    A + 10 * B + C,
    A + B + 10 * C,
  });
  cout << ans << endl;
}
```

さらに考察すると、X, Y, Z のうち最大のものに 10 をかける場合が最大になります。これを C++ で実装すると、例えば次のようになります。

```
#include <iostream>
#include <algorithm>

using namespace std;

int main() {
  int A, B, C; cin >> A >> B >> C;
  int mx = max({A, B, C});
```

```
    int ans = A + B + C + 9 * mx;
    cout << ans << endl;
}
```

B: 1 Dimensional World's Tale

X と x_i, Y と y_i は同一視できるので、$x_{N+1} = X, y_{M+1} = Y$ とします。すると、Z が満たすべき条件は次のようになります。

- $x_1, x_2, ..., x_{N+1} < Z$
- $y_1, y_2, ..., y_{M+1} \geq Z$

このような Z は存在するなら x_1 (または -100) より大きく、y_1 (または 100) 以下のはずなので、$Z = -100, -99, ..., 100$ と順に条件を満たすものがあるか調べて解くことができます。これを C++ で実装すると、例えば次のようになります。

```
#include <iostream>
#include <vector>

using namespace std;

bool is_war(const vector<int>& x, const vector<int>& y) {
  for (int Z = -100; Z <= 100; ++Z) {
    bool is_ok = true;
    for (int i = 0; i < x.size(); ++i) {
      if (x[i] >= Z) is_ok = false;
    }
    for (int i = 0; i < y.size(); ++i) {
      if (y[i] < Z) is_ok = false;
    }
    if (is_ok) return false;
  }
  return true;
}

int main() {
  int N, M; cin >> N >> M;
  vector<int> x(N+1), y(M+1);
  cin >> x[0] >> y[0];
  for (int i = 0; i < N; ++i) {
    cin >> x[i+1];
  }
  for (int i = 0; i < M; ++i) {
    cin >> y[i+1];
```

```
  }
  if (is_war(x, y)) {
    cout << "War" << endl;
  }
  else {
    cout << "No War" << endl;
  }
}
```

他の考え方もあります。$x_i \geq y_j$ なる i,j が存在すれば、条件を満たす Z は存在しません (存在したとすると $x_i < Z \leq y_j$ となって矛盾します)。逆に、全ての i,j について $x_i < y_j$ が成立すれば、$Z = min\{y_1, y_2, ..., y_{M+1}\}$ とすることで条件を満たします。

すなわち、全ての i,j について $x_i < y_j$ であるかを調べる方法や、$Z = min\{y_1, y_2, ..., y_{M+1}\}$ として条件を満たすかどうかを調べる方法があります。前者については、代わりに $max\{x_1, x_2, ..., x_{N+1}\} < min\{y_1, y_2, ..., y_{M+1}\}$ であるかどうか調べてもよく、上の実装例の is_war 関数をこの方法で実装すると次のようになります。

```
#include <algorithm>
bool is_war(const vector<int>& x, const vector<int>& y) {
  return *max_element(x.begin(), x.end()) >= *min_element(y.begin(), y.end());
}
```

C: String Transformation

S に操作を繰り返してできる文字列は、('a' から 'z' の) アルファベットとアルファベットを **1 対 1** 対応させた置換表 (変換表) に従って S の各文字を置換したもののみです。ここで 1 対 1 とは、異なるアルファベットは異なるアルファベットに置換されることを指します。すなわち、例えば 'a' が 'c' に, 'b' が 'c' にといった様に、同じアルファベットに置換されるものが複数存在しません。従って、S から T に変換される 1 対 1 の置換表が作れるかを調べれば良いことになります。置換表の条件は、アルファベット S_i はアルファベット T_i に置換されるといったものなので、

- $S_i = S_j$ ならば、$T_i = T_j$ である (同じアルファベットは同じアルファベットに置換される)
- $T_i = T_j$ ならば、$S_i = S_j$ である (異なるアルファベットは異なるアルファベットに置換される)

を満たせば、各 S_i を T_i に置換するような置換表を作ることができます。ここで、S に出現しない文字が存在する場合について言及すると、それらを $a_1, ..., a_k$ とすれば T に出現しない文字も k 個存在するはずなのでそれを $b_1, ..., b_k$ とすると例えば a_i を b_i に置換する置換表でうまく置換されることがわかります。

これを愚直に実装すると $O(|S|^2)$ かかりますが、例えば置換表を表す 26 要素の配列 R, R' (初期値-1) を持って、順に

- $R[S_i] >= 0$ ならば $R[S_i]$ が T_i に等しいことを検査する
- $R'[T_i] >= 0$ ならば $R'[T_i]$ が S_i に等しいことを検査する
- $R[S_i] < 0$ ならば $R[S_i] := T_i$ とする
- $R'[T_i] < 0$ ならば $R'[T_i] := S_i$ とする

を行えば $O(|S|)$ でこの問題を解くことができます。

D: Factorization

$p_1, p_2, ..., p_k$ を異なる素数として、

$$M = p_1^{b_1} p_2^{b_2} ... p_k^{b_k}$$

と表せたとします。$a_1 a_2 ... a_N = M$ とすると、これらもまた、ある c_{ij} が存在して

$$a_i = p_1^{c_{i1}} p_2^{c_{i2}} ... p_k^{c_{ik}}$$

と表せるはずです。このとき、a_i' と a_i'' が異なることと、ある j が存在して c_{ij}' と c_{ij}'' が異なることは同値です。従って、この問題の答えは、各 b_j を $c_{1j}, c_{2j}, ..., c_{Nj}$ に $c_{1j} + c_{2j} + ... + c_{Nj} = b_j$ を満たすように振り分ける場合の数と一致します。これは j に関して独立に計算でき、各 j についての場合の数は、組み合わせを $comb$ で表すとすれば、$comb(b_j + N - 1, b_j)$ です (直感的には、b_j 個のボールを $N - 1$ 個の仕切りでグループ分けするイメージで、ボールか仕切りか定まっていない $b_j + N - 1$ 個の連続したオブジェクトから b_j 個の仕切りを選ぶ場合の数です)。よって、答えは

$$comb(b_1 + N - 1, b_1) \times comb(b_2 + N - 1, b_2) \times ... \times comb(b_k + N - 1, b_k)$$

となります。時間計算量は、素因数分解に $O(\sqrt{M})$ かかり、各 b_i は $b_i = O(logM)$ で抑えられるので $comb(b_j + N - 1, b_j)$ の計算は b_j 回かけたり割ったりすれば $O(logM)$ で計算でき、k も高々 $O(logM)$ なので、全体としては $O(\sqrt{M} + (logM)^2)$、すなわち $O(\sqrt{M})$ になります (実は N によりません)。

ABC 112 解説

drafear, square1001, E869120

2018 年 10 月 6 日

A: Programming Education

この問題では、入力形式が 2 通りあります。これにどのように対処すればよいでしょうか。実は、「条件分岐」 で対処することができます。最初に **1 つの整数** N を入力して、もし $N = 1$ であれば 「Hello World」と出力し、そうでないならば ($N = 2$ ならば) 「**残り 2 つの整数** A, B を入力して $A + B$ を出力する」 とすればよいです。入力は、上から順 (同じ行の文字は左から順) に読み込まれるので、$N = 2$ のときは 2 → A → B の順に読み込まれるから、上の方法が通用します。

【擬似コード】

```
input N
if N = 1:
        print "Hello World"
else:
        input A, B
        print A + B
```

【サンプルコード】

サンプルコード (C++) : https://beta.atcoder.jp/contests/abc112/submissions/3340558

サンプルコード (Java) : https://beta.atcoder.jp/contests/abc112/submissions/3340568

サンプルコード (C#) : https://beta.atcoder.jp/contests/abc112/submissions/3340582

サンプルコード (Python 3) : https://beta.atcoder.jp/contests/abc112/submissions/3340592

B: Time Limit Exceeded

$t_i \leq T$ を満たす i のうち c_i の最小値を求めつつ、そのような i が存在するか調べていきます。これを実現するアルゴリズムとして、例えば次のようなアルゴリズムが考えられます。

アルゴリズム 1

1. $ans := \infty$ とする
2. $i = 1, 2, ..., N$ と順に、$t_i \leq T$ かつ $c_i < ans$ なら $ans := c_i$ とする
3. $ans = \infty$ なら条件を満たす i は存在しないので **TLE** と出力し、そうでなければ ans を出力する

アルゴリズム 2

1. $t_i \leq T$ であるような i について c_i を集めた配列を c' とする
2. c' が空なら **TLE** と出力し、そうでなければ c' の最小値を求めて出力する

以下は、これらのアルゴリズムをそれぞれ C++ で実装した例です。

アルゴリズム 1

```cpp
#include <iostream>

using namespace std;

const int inf{int(1e9)};

int main() {
  int N, T; cin >> N >> T;
  int ans{inf};
  for (int i = 0; i < N; ++i) {
    int c, t; cin >> c >> t;
    if (t < T && c < ans) {
      ans = c;
    }
  }
  if (ans == inf) {
    cout << "TLE" << endl;
  }
  else {
    cout << ans << endl;
  }
}
```

アルゴリズム 2

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

int main() {
  int N, T; cin >> N >> T;
  vector<int> c(N), t(N);
  for (int i = 0; i < N; ++i) {
    cin >> c[i] >> t[i];
  }
  vector<int> filtered_c;
  for (int i = 0; i < N; ++i) {
    if (t[i] <= T) {
      filtered_c.push_back(c[i]);
    }
  }
  if (filtered_c.size() == 0) {
    cout << "TLE" << endl;
  }
  else {
    cout << *min_element(filtered_c.begin(), filtered_c.end()) << endl;
  }
}
```

C: Pyramid

さて、問題文の制約をよく見てみましょう。一番重要なのは、以下の二点です。

- 条件を満たすピラミッドは 1 つしか存在しない。
- ピラミッドの中心座標は (C_x, C_y) であり、C_x, C_y は 0 以上 100 以下の整数。

まず、条件を満たすピラミッドがただ 1 つなのであれば、必ず以下のものが 1 個以上存在します。

- 調査で得られた情報のうち、$h_i \geq 1$ を満たす情報 (x_i, y_i, h_i)。

なぜなら、全部 $h_i = 0$ であるとき、調査を行っていない整数座標を中心とする高さ 1 のピラミッドが条件を満たしてしまうので、最低でも $101 \times 101 - N$ 個、$N = 100$ の場合 $10,101$ 個以上の条件を満たすピラミッドが存在することになるからです。そこで、前述の条件を満たす情報のうち 1 個を (x_t, y_t, h_t) とおきます。

C_x, C_y は 0 以上 100 以下の整数なので、中心座標として考えられるのは高々 $101 \times 101 = 10,201$ 通りです。そこで中心座標 (px, py) を全探索することを考えます。

px, py が決まっているとき、ピラミッドの高さは $h_t + |px - x_t| + |py - y_t| (h_t \geq 1)$ となるので、一意に定まります。その後は、その (中心座標, 高さ) の組が調査によって得られた情報に適合するか、を確認するだけです。

計算回数は $101 \times 101 \times N$ 回くらいになります。300 点問題にしては結構実装は重いです。

【サンプルコード (C++)】
`https://beta.atcoder.jp/contests/abc112/submissions/3340382`

D: Partition

$a_1, a_2, ..., a_N$ の最大公約数を D とすると、$a_1, a_2, ..., a_N$ は D で割り切れるので $M = a_1 + a_2 + ... + a_N$ も D で割り切れます。従って、D は M の約数になります。

また、D が M の約数のとき、$a_1, a_2, ..., a_N \geq D$ なので $M \geq N \times D$ です。

実は D が M の約数であって $M \geq N \times D$ を満たすとき、最大公約数が D 以上となるように $a_1, a_2, ..., a_N$ が構成できます。具体的には、$a_1 = a_2 = ... = a_{N-1} = D, a_N = M - (N-1) \times D$ とします。このとき、a_N は正整数かつ D の倍数なので各要素の最大公約数は D 以上[*1]となります。

したがって、M の約数かつ $\frac{M}{N}$ 以下の整数のなかで最大のものが答えとなります。約数は (約数のペアの片方が平方根以下であることを利用して)、$O(\sqrt{M})$ で求められることが知られているため、この問題は $O(\sqrt{M})$ で解くことができます。

[*1] $N = 1$ のとき最大公約数は M となり D より大きくなることがあります

AtCoder Beginner Contest 113 解説

writer: namonakiacc, square1001, E869120

平成 30 年 11 月 4 日

A: Discount Fare

鉄道の運賃が X 円、バスの運賃が Y 円のとき、A 駅から C 駅まで移動するのに $X + Y \div 2$ 円かかります。

よって、$X + Y \div 2$ を出力します。

```
#include <cstdio>
int X,Y;
int main()
{
        scanf("%d%d",&X,&Y);
        printf("%d\n",X+Y/2);
}
```

B: Palace

地点 i の平均気温は $T - H_i \times 0.006$ 度なので、各地点の平均気温を計算し、それらの中から平均気温が A 度に最も近い点を求めるとよいです。

気温は整数ではありませんが、気温を 1000 倍して計算すると浮動小数点型を用いなくても答えを求めることができます。

```
#include <cstdio>
int N,T,A,X;
int main()
{
        scanf("%d%d%d",&N,&T,&A);
        int res=0;
        int cc=1<<30;
        for(int i=1;i<=N;i++)
        {
                scanf("%d",&X);
                int d=(T*1000-X*6)-A*1000;
                if(d<0)d=-d;
                if(cc>d)cc=d,res=i;
        }
        printf("%d\n",res);
}
```

C: ID

県ごとに、その県に属する市を誕生した順にソートして、認識番号を割り振るとよいです。

```
#include <cstdio>
#include <vector>
#include <algorithm>
using namespace std;
typedef long long ll;
#define rep(i,n) for(int i=0;i<n;i++)

int N,M;
int P[100000],Y[100000];
vector<int> yd[100001];

int main()
{
        scanf("%d%d",&N,&M);
        rep(i,M)scanf("%d%d",&P[i],&Y[i]),yd[P[i]].push_back(Y[i]);
        rep(i,N)sort(yd[i+1].begin(),yd[i+1].end());
        rep(i,M)printf("%012lld\n",ll(P[i])*1000000+int(lower_bound(yd[P[i]].
        begin(),yd[P[i]].end(),Y[i])-yd[P[i]].begin())+1);
}
```

D: Number of Amidakuji

$H = 100, W = 8$ のケースで、あみだくじの個数は $34^{100} \fallingdotseq 1.4 * 10^{153}$ 個程度あり、到底全探索できそうにありません。そこで、「動的計画法」 という方法を使ってみることにしましょう。

$A(h, x)$ を、縦から h 列目を通過した (横棒を通った場合も含む) 直後に左から x 本目の縦棒にいる場合の数とします。$h = 0$ のとき、1 列目より前を表すことにます。

$h = 0, 1, 2, 3, \cdots, H$ の順に上から下に降りていく感じで $A(h, x)$ を更新していくことにします。左から 1 本目の縦棒からスタートするので、$A(0, 1) = 1$、$A(0, 2) = A(0, 3) = \cdots = A(0, W) = 0$ になります。

$A(h, x)$ が指しているあみだくじの経路からは、$A(h + 1, x - 1), A(h + 1, x), A(h + 1, x + 1)$ のいずれかの状態に行くことになります。$A(h, x)$ の状態から何通りが $x - 1$ 本目の縦棒に行って、何通りが x 本目の縦棒にとどまって、何通りが $x + 1$ 本目の縦棒に行くかは、$h + 1$ 本目の横棒の配置を全探索することで計算することができます。先ほどの $x - 1, x, x + 1$ 本目に行く場合の数をそれぞれ X, Y, Z とおくと、$A(h + 1, x - 1), A(h + 1, x), A(h + 1, x + 1)$ の結果はそれぞれ $X * A(h, x), Y * A(h, x), Z * A(h, x)$ 足されることになります。

したがって、計算量 $O(HW2^W)$ で解くことができます。$H = 100, W = 8$ のケースでも、十分に間に合わせることができます。

おまけ: 実は先ほどの X,Y,Z の値はフィボナッチ数列 ($F_1 = F_2 = 1, F_N = F_{N-1} + F_{N-2} (N \geq 3)$ を満たす数列) を使って求めることができます。ある 1 列の横棒の配置の場合の数は、フィボナッチ数 F_{W+1} です。これを使えばこの問題を計算量 $O(HW)$ で解くことができます。ぜひ考えてみましょう！

サンプルコード (C++) :
`https://beta.atcoder.jp/contests/abc113/submissions/3540482`

ABC 114 解説

出題・解説: @evima0

2018 年 12 月 2 日

A: 753

正解までの道のりを細かく分割すると次の通りです。

1. 標準入力から整数 X を受け取る。
2. X から何らかの方法で答えとなる文字列 `YES` または `NO` を得る。
3. 得た文字列を標準出力に出す。

手順 1, 3 については practice contest 問題 A の言語別サンプルコードが参考になります。手順 2 については、if 文、“or” に相当する論理演算子と比較演算子を使うのが最も素直ですが、他の手も考えられます。C++ による素直な実装例と、Python3 によるやや気取った実装例を示します。

```cpp
#include <iostream>
using namespace std;
int main(){
    int X;
    cin >> X;
    string ans;
    if(X == 7 || X == 5 || X == 3){
        ans = "YES";
    }else{
        ans = "NO";
    }
    cout << ans << endl;
}
```

```python
print(['NO', 'YES'][[7, 5, 3].count(int(input()))])
```

B: 754

全体的な方針としては、ルンルンの行動の選択肢はそれほど多くないので、すべての選択肢を実際に試して最も 753 に近かった結果を採用すればよいです。以下、この方針を実装することについて書きます。

文字列 S の長さを N とし、S の最初の文字を 0 文字目、最後の文字を $N-1$ 文字目と呼ぶことにします。例えば $N=6$ のとき、ルンルンの行動の選択肢は「S の $0,1,2$ 文字目を取り出す」「$1,2,3$ 文字目を取り出す」「$2,3,4$ 文字目」「$3,4,5$ 文字目」の 4 通りです。一般的には、整数 $i=0,1,2,...,N-3$ のそれぞれに対して「S の $i,i+1,i+2$ 文字目を取り出す」という抜き出し方が存在します。

あとは、「S の $i,i+1,i+2$ 文字目を取り出し」て得られる 3 桁の整数をプログラム上でどのように求めるかが主な問題です。言語によって事情が異なり、Python などでは気軽に文字列から整数への変換を行えますが、C++ などではやや手間です。ASCII コード上で数字 '0', '1', ..., '9' が順番に並んでいるという性質を使って「自前」で変換する手も検討に値します。

そのほかに絶対値や「二つ（以上）の数のうち小さい方」などの部品も必要になりますが、言語の標準ライブラリを使うとスムーズです。以下の C++, Python3 による実装例を参考にしてください。

```cpp
#include <algorithm>    // min
#include <cmath>        // abs
#include <iostream>
using namespace std;
int main(){
    string S;
    cin >> S;
    int N = S.size();
    int ans = 999;
    for(int i = 0; i <= N - 3; ++i){
        int num = (c[i] - '0') * 100 + (c[i+1] - '0') * 10 + c[i+2] - '0';
        ans = min(ans, abs(num - 753));
    }
    cout << ans << endl;
}
```

```python
S = input()
print(min(abs(int(S[i:i+3]) - 753) for i in range(len(S) - 2)))
```

C: 755

N が小さければ、1 から N までの数をすべてチェックして七五三数を直接数えることができます。しかし与えられる N は最大で 999999999 (10 億 - 1) と大きく、2 秒という時間制限でこの方針は厳しいです。[*1]

入出力例 3 ($N = 999999999$, 出力: 26484) でひっそり示されているように、七五三数はそれほど多くありません。ですから、「$357, 375, 573, \ldots$」と七五三数の方を列挙して N 以下にある個数を数えるべきです。

そのためには、まず「$7, 5, 3$ のみを含む数」(333 や 57577 なども該当。**準七五三数**と呼ぶことにします) をすべて列挙し、そのうち「$7, 5, 3$ をすべて含む数」(七五三数) のみをカウントすると単純です。N 以下の準七五三数もそれほど多くない[*2]ので時間切れの心配はありません。

あとは、N 以下の準七五三数をどのように列挙するかが問題です。最も素直な手は再帰関数でしょう[*3]。「空っぽのもの」から始めて、「持っているものの後ろに $7, 5, 3$ を付け足して新たに 3 つのものを得る」ことを繰り返します。

具体的な実装について書きます。Python などの整数と文字列の相互変換が簡単な言語では、数を主に文字列として扱うと実装しやすいでしょう。C++ などで文字列を経由するべきかは悩むところです。経由せず整数を直接取り扱う場合、整数を「10 倍して 5 を足す」(例: $1234 \times 10 + 5 = 12345$. 後ろに 5 が付いた！) などの操作が役立ちます。以下の Python3 による実装例を参考にしてください。

```python
N = int(input())

def dfs(s):  # 文字列 s で始まる七五三数の個数
    if int(s) > N:
        return 0
    ret = 1 if all(s.count(c) > 0 for c in '753') else 0  # s 自体が七五三数なら +1
    for c in '753':
        ret += dfs(s + c)
    return ret

print(dfs('0'))  # 本当は dfs('') と書きたいが 4 行目でのエラーを防ぐため仕方なく
```

[*1] 実はこの方針で C++ などの言語で無理やり間に合わせることも不可能ではありませんが、かえって難しいです。
[*2] 9 桁の準七五三数は $3^9 = 19683$ 個、1〜9 桁の準七五三数をすべて合わせても 29523 個
[*3] 他の手段としては、3 進数を小さい方から列挙して $0 \to 3, 1 \to 5, 2 \to 7$ と変換する手もあります。

D: 756

$N = 100$ のとき $N! = 93326215443944152681...$ (158 桁) です。これを直接取り扱っても不毛で、効率的な枠組みで考える必要があります。

準備体操として、入出力例 2 の説明にある 32400 が確かに約数を 75 個持つことを確認しましょう。32400 を素因数分解 (リンク先は Wikipedia の同名の記事) すると $32400 = 2^4 3^4 5^2$ となります。よって $2^a 3^b 5^c (0 \leq a \leq 4, 0 \leq b \leq 4, 0 \leq c \leq 2)$ という形で表せる数はすべて 32400 の約数で、他にはありません。a の値は $0, 1, 2, 3, 4$ の 5 通り、b の値も 5 通り、c の値は 3 通りで、約数は全部で $5 \times 5 \times 3 = 75$ 個です。

同じように考えると、3 つの異なる素数 p, q, r を使って $p^4 q^4 r^2$ と表せる数はすべて七五数です。ほかに、$p^{14} q^4, p^{24} q^2, p^{74}$ という「パターン」の数も約数を $15 \times 5 = 25 \times 3 = 75$ 個持ちます。$75 = 3 \times 5 \times 5$ を分割することを考えると以上の 4 パターンがすべてで、これらをそれぞれ数えればよいです。

最も複雑な $p^4 q^4 r^2$ のパターンを考えます。まず $1, 2, \ldots, N$ をそれぞれ素因数分解して合算することで $N!$ を素因数分解します[*4]。素数 s に対して「$N!$ の素因数分解での s の指数」[*5]を $e(s)$ と書くと、$e(p) \geq 4, e(q) \geq 4, e(r) \geq 2$ が成立すれば $p^4 q^4 r^2$ は $N!$ の約数です。このような p, q, r の組合せの数を三重ループで数える[*6]か、もう少し考えて数式で求めることもできます。Python3 での後者の例を示します。[*7]

```python
N = int(input())
e = [0] * (N+1)
for i in range(2, N+1):
    cur = i
    for j in range(2, i+1):
        while cur % j == 0:
            e[j] += 1
            cur //= j

def num(m):  # e の要素のうち m-1 以上のものの個数
    return len(list(filter(lambda x: x >= m-1, e)))

print(num(75) + num(25) * (num(3) - 1) + num(15) * (num(5) - 1)
    + num(5) * (num(5) - 1) * (num(3) - 2) // 2)
```

[*4] N 自体は 100 以下と小さいので、愚直に割り算を繰り返せば十分です。エラトステネスの篩などの効率的なアルゴリズムは不要

[*5] 例えば $N = 4$ のとき $N! = 24 = 2^3 3$ なので $e(2) = 3, e(3) = 1$

[*6] ただし、例えば $2^4 3^4 5^2$ と $3^4 2^4 5^2$ が同じ数であることに注意

[*7] 余談: ここでは 75 という具体的な値に注目して解きましたが、「約数を x 個持つ数の個数」を x の小さい順に求めていく解法もあります (動的計画法と呼ばれる手法です)。

ABC 115 解説

出題・解説: @evima0

2018 年 12 月 8 日

A: Christmas Eve Eve Eve

正解までの道のりを細かく分割すると次の通りです。

1. 標準入力から整数 D を受け取る。
2. X から何らかの方法で答えとなる文字列 `Christmas...` を得る。
3. 得た文字列を標準出力に出す。

手順 1, 3 については practice contest 問題 A の言語別サンプルコードが参考になります。手順 2 については if 文を使う、for 文を $25 - D$ 周回す (100 点問題の要求レベルを少し超える)、文字列に + や * といった演算子を使う (あれば) といった手が考えられます。C++, Python3 による実装例を示します。

```cpp
#include <iostream>
using namespace std;
int main(){
    int D;
    cin >> D;
    string ans;
    if(D == 25) ans = "Christmas";
    else if(D == 24) ans = "Christmas Eve";
    else if(D == 23) ans = "Christmas Eve Eve";
    else ans = "Christmas Eve Eve Eve";
    cout << ans << endl;
}
```

```python
print('Christmas' + ' Eve' * (25 - int(input())))
```

B: Christmas Eve Eve

「N 個の商品の定価の合計」を S 円、「N 個の商品のうち最も定価が高いものの定価」を M 円とすると答えは $S - M/2$ で、この S と M を実際に求めてみよという問題です。

この問題が A: Christmas Eve Eve Eve より難しいのは、商品の個数が 2 個や 3 個などと固定されておらず、N 個 $(2 \leq N \leq 10)$ と可変であるためです。これに対応すべく、何らかの「ループを生む機構」を使うことになります。代表例が for 文で、この内部で足し算や大小比較を行うことで S や M が求められます。

S を求めるためのより具体的な手順を述べると、まず S という値を宣言して 0 で初期化します。そして $i = 1, i = 2, \ldots, i = N$ のそれぞれに対して S に p_i を足すと、最終的に S に求めたい値が入ります。M についても同様で、最初に M という値を 0 で初期化し[*1]、「M と p_i を比較して p_i の方が大きければ M を p_i に置き換える」ことを繰り返せば求まります。これを C++ で実装したコードが以下です。

```cpp
#include <algorithm>    // max
#include <iostream>
using namespace std;
int main(){
    int N;
    cin >> N;
    int S = 0, M = 0;
    for(int i = 0; i < N; ++i){  // ここでは商品番号を 0, 1, ..., N-1 とする
        int p;
        cin >> p;
        S += p;
        M = max(M, p);  // M と p のうち大きい方。かなり便利！ ちなみに小さい方は min
    }
    int ans = S - M / 2;
    cout << ans << endl;
}
```

言語によっては、純粋な for 文以外の機構も便利です。以下のコード (Python3) をご覧ください。

```python
N = int(input())
p = [int(input()) for i in range(N)]
print(sum(p) - max(p) // 2)
```

[*1] すべての p_i より小さい値なら 0 でなくても可

C: Christmas Eve

木の本数は最大で 10 万本とかなり多く、K も 2 以上 $N-1$ 以下の範囲で自由な値を取りうるため、例えば $N = 100000, K = 50000$ のとき飾る木の選び方の総パターン数は天文学的な数値になります[*2]。したがって木の飾り方の全パターンを 2 秒で調べることはできず、選択肢の数を減らす必要があります。

やや唐突ですが、仮に N 本の木が低い順に左から一列に並んでいたらどうでしょう？「飾った最も高い木」(=飾った最も左の木) と「飾った最も低い木」(=飾った最も右の木) の高さをできるだけ近づけるには、ずばり K 本の隣り合った木を飾るべきです。もしそうせずに飾った木と飾った木の間に飾っていない木があるなら、「間を詰める」ことにより状況が改善されるか、少なくとも悪くはならない[*3]からです。

よって、仮に N 本の木が低い順に左から一列に並んでいた場合、「左から $1, 2, \ldots, K$ 本目を飾る」「$2, 3, \ldots, K+1$ 本目を飾る」...「$N-K+1, N-K+2, \ldots, N$ 本目を飾る」という $N-K+1$ 通りのパターンをすべて試せば十分です。それぞれのパターンにおける $h_{\max} - h_{\min}$ の値も、最も高い木は最も右、最も低い木は最も左にあるため簡単に求まります。

残る問題は、実際には N 本の木が低い順に入力されるとは限らないことです。これは与えられた木の高さを低い順に並び替えてしまえば (ソートすれば) 解決します。ほとんどの主要なプログラミング言語は標準機能として高速に動作する[*4]ソートを備えているので、これを利用しましょう。[*5]

```
N, K = map(int, input().split())
h = [int(input()) for i in range(N)]
h.sort()
print(min(h[i+K-1] - h[i] for i in range(N-K+1)))
```

[*2] 252060836892200 . . . 通り (30101 桁)
[*3] 同じ高さの木が複数並んでいる場合
[*4] 長さ n の列に対し $O(n \log_2 n)$ 時間で動作する
[*5] ソートを「自前」で適当に実装してしまうと、$N = 100000$ のとき 2 秒以内に実行が終わりません。高速に動作するソートの実装は AtCoder Beginner Contest の範囲を超えた課題で、興味があれば Wikipedia などが参考になるでしょう。

D: Christmas

「レベル i バーガーの厚さ (層の総数)」を a_i、「レベル i バーガーに含まれるパティの総数」を p_i とします $(i \geq 0)$。これらの値は $a_0 = 1, a_i = 2a_{i-1} + 3$ $(i \geq 1)$, $p_0 = 1, p_i = 2p_{i-1} + 1$ $(i \geq 1)$ と順次求められます。

求めるべき数、「レベル N バーガーの下から X 層に含まれるパティの枚数」を $f(N, X)$ と書きます。$N = 0$ の場合は問題ないので、$N \geq 1$ の場合を考えましょう。$N = 0$ の場合が問題ないことを活用して、「レベル $N-1$ バーガーについてなら何を聞かれてもわかる。つまり、どんな整数 Y を指定されても $f(N-1, Y)$ を知れる」と仮定してみると、次のようにレベル N バーガーについても何を聞かれてもわかってしまいます。

1. $X = 1$ のとき: $f(N, X) = 0$ (一番下のバンだけ)
2. $1 < X \leq 1 + a_{i-1}$ のとき: $f(N, X) = f(N-1, X-1)$
   (一番下のバン + その上のレベル $N-1$ バーガーの下から $X-1$ 層)
3. $X = 2 + a_{i-1}$ のとき: $f(N, X) = p_{i-1} + 1$
   (一番下のバン + その上のレベル $N-1$ バーガー + その上のパティ)
4. $2 + a_{i-1} < X \leq 2 + 2a_{i-1}$ のとき: $f(N, X) = p_{i-1} + 1 + f(N-1, X-2-a_{i-1})$
   ((前略) + その上のパティ + その上のレベル $N-1$ バーガーの下から $X-2-a_{i-1}$ 層)
5. $X = 3 + 2a_{i-1}$ のとき: $f(N, X) = 2p_{i-1} + 1$
   ((前略) + その上のパティ + その上のレベル $N-1$ バーガー + その上のバン)

これをそのまま再帰関数として実装すれば、$N+1$ 回以下の計算ステップで答えが求まります。

```python
N, X = map(int, input().split())

a, p = [1], [1]
for i in range(N):
    a.append(a[i] * 2 + 3)
    p.append(p[i] * 2 + 1)

def f(N, X):  # X <= 0 や X > a_N を許容し解説本文から簡略化
    if N == 0:
        return 0 if X <= 0 else 1
    elif X <= 1 + a[N-1]:
        return f(N-1, X-1)
    else:
        return p[N-1] + 1 + f(N-1, X-2-a[N-1])

print(f(N, X))
```

ABC116

出題、解説:yuma000

2019 年 1 月 19 日

A - Right Triangle

「三角形の面積」=「底辺」×「高さ」÷ 2 です。よって、$|AB| \times |BC| \div 2$ を出力すればよいです。

この問題では、$|CA|$ の値は解答に影響しませんでした。今回は $|CA|$ の値は最後に与えられるため、これを入力として受け取らなくても問題に正答できます。ですが、不都合が生じる場合もあるので (複数のテストケースがある, クエリ形式の問題である等) 慣れないうちは一応全て入力として受け取る癖をつけておくことをおすすめします。

以下に C++ での解答例を示します。

```
#include <iostream>
using namespace std;

int main() {

        int ab, bc, ca;

        cin >> ab >> bc >> ca;

        int area = ab * bc / 2;

        cout << area << endl;

        return 0;
}
```

B - Collatz Problem

実際にシミュレーションして a の要素を順次求めていきましょう。出てきた a_i の値を配列や連想データ構造に保存していって、重複する値を検出することで、この問題に正答することができます。

また、この問題は Collatz problem として知られており、a_1 の値によらず、a はかならず $4 \to 2 \to 1 \to 4$ のループに到達すると予想されています。この予想は 5×10^{260} 以下ならば実証さ

れているため、$a_m = 4, 2, 1$ となる最小の整数 m をシミュレーションで求めて、$m+3$ を出力することでも、この問題に正答できます。

C - Grand Garden

長さ N の数列 h に対する「水やり」の組を $w(h)$ とする。$w(h)$ の取りうる要素数の最小値を $f(h)$ とする。

定理 1

$h_a = \{h_1, h_2, ...h_N\}(h_i \geqq 1)$,
$h_b = \{h_1 - 1, h_2 - 1, ...h_N - 1\}$
に対して、$f(h_a) = f(h_b) + 1$ である。

[証明 1]

$h_i \leqq 1$ より、$w(h_a)$ には、$(1, r_1), (l_2, r_2), (l_3, r_3)..., (l_x, N)$ が含まれる。$(r_k \leqq l_{k+1} (1 \leqq k \leqq N-1)$
$(1, r_1), (l_2, r_2), (l_3, r_3)..., (l_x, N)$ は、$(1, N), (l_2, r_1), (l_3, r_2)..., (l_x, r_{x-1})$ で置き換えられる。つまり、最小要素数の $w(h_a)$ の中に、$(1, N)$ が含まれているものが必ず存在する。$w(h_a)$ に $(1, N)$ を加えたものは、$w(h_a)$ となるので、$f(h_a) = f(h_b) + 1$ が示される。

定理 2

$h_a = \{h_1, h_2, ..., h_{x-1}, h_x = 0, h_{x+1}, ...h_N\}$,
$h_b = \{h_1, h_2, ..., h_{x-1}, \underbrace{0, ...0}_{N-x+1}\}$,
$h_c = \{\underbrace{0, ..., 0}_{x}, h_{x+1}, ...h_N\}$
に対して、$f(h_a) = f(h_b) + f(h_c)$ である。

[証明 2]

$h_x = 0$ より、$w(h_a)$ に含まれる要素 (l, r) に対して、$x < l$ もしくは　$r < x$ のどちらか一方が成り立つ。前者からなる集合は、$w(h_b)$ であり、後者からなる集合は $w(h_c)$ である。よって、$f(h_a) = f(h_b) + f(h_c)$

また、明らかに、以下の定理も成り立ちます。

定理 3

$h_a = \{h_1, h_2, ...h_N, 0\}$,
$h_b = \{h_1, h_2, ..., h_N\}$,
$h_c = \{0, h_1, h_2, ..., h_N\}$
に対して、$f(h_a) = f(h_b) = f(h_c)$ である。

これら三つの定理から、

- 伸ばすべき高さが全ての花に対して 1 以上ならば、全ての花に水をやる。
- ある花 x をもはや伸ばす必要がないならば、花 $1 \sim x-1$ と花 $x+1 \sim N$ についての最小手数を別々に求めて、それらの必要手数を合計する。

この操作を繰り返すことで、この問題に正答することができます。

D - Various Sushi

$p(1 \leqq a \leqq N)$ 種類のネタを選んだ時の「おいしさ基礎点」の最大値を $f(p)$ と定義します。(そのように選べないときは、$-inf$ とします。)
また、$g(p) = f(p) + p * p$ と定義します。$(1 \leqq p \leqq N)$ において、$g(p)$ の最大値を求めればいいことになります。

まず、寿司を「おいしさ」で降順にソートします。ここから貪欲に (「おいしさ」が大きい順に)K 個寿司を選び、その集合を S とします。S の「ネタ」の種類数が x の時、S の「おいしさ基礎点」の和が、$f(x)$ となります。

明らかに $f(i) < f(x)(i < x)$ なので、$g(i) < g(x)(i < x)$ となり、$i < x$ の場合は考えなくても良いことが分かります。

次に、$f(a)$ とそれを構成する寿司の集合の一つ S_a が分かっている場合に、$f(a+1)$ とそれを構成する寿司の集合の一つ S_{a+1} を求めることを考えます。$(x \leqq a \leqq N-1)$

S_{a+1} は、S_a に以下の操作を加えることで構成することができます。

1. 集合 S_a から、取り除いても「ネタ」の種類数が減らない寿司の中で最も「おいしさ」の小さいものを取り除く。
2. 集合 S_a に含まれない寿司の中で、集合 S_a にない「ネタ」を持つ、最も「おいしさ」の大きいものを S_a に加える。

この操作を繰り返すことにより、$f(i)(x < i \leqq N)$ を再帰的に求めることができます。$priorityqueue$ 等の、値の挿入と最小値の取り出しを計算量 $O(log|T|)$(|T| は要素数) で行えるデータ構造を用いることで、一回の操作を計算量 $O(logN)$ で行えます。これを最大 N 回繰り返すので、O(NlogN) で $f(p)(x \leqq p \leqq N)$ を求められます。

後は、$g(p)(x \leqq p \leqq N)$ の最大値を求めることで、この問題に正答することができます。

ABC 117 解説

writer: drafear

2019 年 2 月 3 日

A: Entrance Examination

答えを t とすると、問題文より $X \times t = T$ が成り立つので、これを解くと $t = \frac{T}{X}$ が得られます。これを C++ で実装すると次のようになります。小数点以下 3 桁以上出力しなければならないことに注意してください。

```
#include <bits/stdc++.h>

using namespace std;

int main() {
  int T, X; cin >> T >> X;
  double ans = double(T) / X;
  cout << fixed << setprecision(10) << ans << endl;
}
```

B: Polygon

最大の L_i が他の L_i の和未満であれば ‘Yes‘、そうでなければ ‘No‘ を出力すれば良いことが問題文の定理からわかります。したがって、「最大の L_i」と「他の L_i の和」を求められれば判定することができます。これらを求めるアプローチはいくつかあります。

方法 1

L を昇順にソートします。すると、始めの $N-1$ 要素の和が「他の L_i の和」、最後の要素が「最大の L_i」となります。

方法 2

L_j が最大となる j を求めます。これは、仮に $j := 1$ として、$L_2 > L_j$ ならば $j := 2$ と更新し、$L_3 > L_j$ ならば $j := 3$ と更新し、・・・と繰り返すことで求められます。すると、$j \neq k$ なる k について L_k を足し合わせたものが「他の L_i の和」、L_j が 「最大の L_i」となります。

方法 3

まず、最大値を求めます。これは、方法 2 やそれと似た方法でできます。次に $L_1, L_2, ..., L_N$ の合計を求め、S とします。すると、$S - L_j$ が「他の L_i の和」、L_j が 「最大の L_i」となります。
これを C++ で実装すると、次のようになります。

```cpp
#include <bits/stdc++.h>

using namespace std;

int main() {
  int n; cin >> n;
  int sum = 0, maxL = 0;
  for (int i = 0; i < n; ++i) {
    int L; cin >> L;
    sum += L;
    maxL = max(maxL, L);
  }
  if (maxL < sum - maxL) {
    cout << "Yes" << endl;
  }
  else {
    cout << "No" << endl;
  }
}
```

C: Streamline

コマの数が目的地の数以上の場合、すなわち $N \geq M$ の場合、初めに各目的地に 1 つ以上のコマを配置できるので 0 回の移動で目的を達成できます。以降、$N < M$ として考えます。さらに、一般性を失わずに $X_1 < X_2 < ... < X_M$ とソートされていると考えて良いです (実装では、入力された座標をソートします)。

ある 1 つのコマに着目したとき、移動を通して訪れる座標は区間になります。逆に、ある区間の整数座標全てをある 1 つのコマで訪れる場合、区間の一端に最初に配置し、もう一端に向かって移動し続けるのが最適です。また、複数のコマが同じ座標を訪れるのは無駄です。したがって、各コマが担当する区間を並べると、ちょうど $N-1$ 個の開区間 (X_i, X_{i+1}) が訪れられていない形になります。$L_i = X_{i+1} - X_i$ とすると、そのときの合計移動回数は $(X_M - X_1)-$ (訪れられていない開区間の L_i の和) となるため、(訪れられていない開区間の L_i の和) を最大化すれば良いです。これは、L_i を大きい方から $N-1$ 個取るのが最大です。計算量はソートがボトルネックとなり $O(MlogM)$ です。

D: XXOR

$0 \leq X \leq K$ なる X 全てについて $f(X)$ を計算し、最大値を出力する方法では、$O(NK)$ となり計算に 10 年以上かかりそうです。そこで、より高速に求める必要があります。

$X < K+1$ なので $K+1 = (K_{39}K_{38}...K_0)_2, X = (X_{39}X_{38}...X_0)_2$ と 2 進数で表したとき[*1]に、

$$\begin{aligned} X_{39} &= K_{39} \\ &\vdots \\ X_{i+1} &= K_{i+1} \\ X_i &< K_i \end{aligned}$$

を満たす i が存在します。このとき、$X_i = 0, K_i = 1$ です。

逆に、ある i について

$$\begin{aligned} X_{39} &= K_{39} \\ &\vdots \\ X_{i+1} &= K_{i+1} \\ X_i &< K_i \end{aligned}$$

であるような X は、$X_{i-1}, X_{i-2}, ..., X_0$ の値に関わらず $X < K$ が成り立ちます。

そこで、このような $i(0 \leq i < 40)$ を全探索することを考えます。各 i に対する $f(X)$ の最大値を a_i とすれば、解は $\max\{a_0, a_1, ..., a_39\}$ になります。

各 i に対する $f(X)$ の最大値を考えます。$X_{39}, X_{38}, ..., X_i$ は条件より

$$\begin{aligned} X_{39} &= K_{39} \\ &\vdots \\ X_{i+1} &= K_{i+1} \\ X_i &= 0 \end{aligned}$$

です。上でも述べたとおり $X_{i-1}, X_{i-2}, ..., X_0$ を決める際には $X < K$ を無視しても構いません。f はビットごとに考えることができるため、$X_{i-1}, X_{i-2}, ..., X_0$ を独立に決めることができます。具体的には、$X_k(0 \leq k \leq i-1)$ について、$A_1, A_2, ..., A_N$ の中で下から k ビット目が立っているものの数を c_k とすると、$X_k = 0$ とすればこのビットが f に貢献する値は $2^k c_k$ となり、$X_k = 1$ とすれば $2^k(n - c_k)$ となります。したがって、$c_k > n - c_k$ の場合は $X_k = 0$ とし、そうでない場合は $X_k = 1$ とするのが最適です。これを i の小さい順または大きい順に探索することで、X を高々 2 ビットずつ変化させて探索でき、$O(NlogK)$ で求めることができます。

別解として、X の上位ビットから順に決める桁 DP による $O(NlogK)$ の解法もあります。

[*1] 制約 $X, K \leq 10^{12} \leq 2^{40}$ より X, K は 40 ビットで表せます。

ABC 118 解説

writer: drafear

2019 年 2 月 16 日

A: B +/- A

A が B の約数であるとは、B が A で割り切れることなので、B を A で割った余りが 0 であるかを調べれば判定できます。したがって、C++ で実装した場合下のコードで正答することができます。

```
#include <bits/stdc++.h>

using namespace std;

int main() {
  int A, B; cin >> A >> B;
  if (B % A == 0) {
    cout << A + B << endl;
  }
  else {
    cout << B - A << endl;
  }
}
```

B: Foods Loved by Everyone

各食べ物 i についてそれを好きだと答えた人数をカウントし、c_i とします。すると、$c_i = N$ である i の数が答えとなります。これを C++ で実装すると次のようになります。

```
#include <bits/stdc++.h>

using namespace std;

int main() {
  int N, M; cin >> N >> M;
  vector<int> cnt(M, 0);
  for (int i = 0; i < N; ++i) {
    int K; cin >> K;
    for (int j = 0; j < K; ++j) {
      int A; cin >> A; --A;
      ++cnt[A];
    }
  }
  int ans = 0;
  for (int i = 0; i < M; ++i) {
    if (cnt[i] == N) {
      ++ans;
    }
  }
  cout << ans << endl;
}
```

C: Monsters Battle Royale

$A_1, A_2, ..., A_N$ の最大公約数を g とします。すると、以下で示すように g が答えになります。

入力例 1 において、なぜどのように攻撃を行っても体力 1 にならないのでしょうか。これは、どのモンスターも初期体力が偶数なので、攻撃して体力が変化しても偶数にしか変化しないためです。これを一般化すると、体力 a のモンスターが体力 b $(b > a)$ のモンスターに攻撃したときに攻撃されたモンスターの体力は $b - a$ となりますが、a も b も x の倍数であるなら $b - a$ も x の倍数です。したがって、常に x として g を取れるので、生きているモンスターの体力は必ず g の倍数となり、g 未満になることはありません。

逆に、次のように最後に生き残ったモンスターの体力を g にすることができます。体力 a のモンスターと体力 b のモンスターがお互いの体力が等しくなるまで体力の低いほうが高い方を攻撃し続けるとユークリッドの互除法と同じ遷移になるため最終的な体力はお互いに a と b の最大公約数になります。これを 1 番目のモンスターと 2 番目のモンスター, 2 番目のモンスターと 3 番目のモンスター, ... と順に行うと、最終的に N 番目のモンスターの体力が $A_1, A_2, ..., A_N$ の最大公約数 g になります。最後に N 番目のモンスターが他のモンスターを攻撃し続ければ、最後に生き残ったモンスター (N 番目のモンスター) の体力が g になります。

実装は、ユークリッドの互除法により $A_1, A_2, ..., A_N$ の最大公約数を求めて出力すればよく、これは $O(N + logA_1)$ で動作します。

D: Match Matching

桁数が異なる整数同士では、桁数が多い方が大きいため、まずはちょうど N 本のマッチ棒を使って最大何桁作れるかを求めることを考えます。ちょうど N 本使わないといけないため、貪欲に作ることはできず、動的計画法で解くことになります。

具体的には

$$dp(i) := \text{ちょうど } i \text{ 本のマッチ棒を使って、条件を満たす整数を作るときの最大桁数}$$

と定義し、これを求めます。これが求まると、答えとなる整数の桁数は $dp(N)$ です。これの求め方については後述します。

今度は、桁数が同じ整数同士の大小は辞書順となるため、できるだけ大きい数字を上位の桁に使うようにします。すなわち、一般性を失わずに $A_1 > A_2 > ... > A_M$ とすると、A_1 を最上位の桁に使えるかを調べ、使えるなら使う、使えないなら A_2 を最上位の桁に使えるかを調べ、・・・と試していきます。ここで、整数 k を 1 桁作るのに使うマッチ棒の本数を $num(k)$ とすると、A_i を最上位の桁に使えることを調べるには、$dp(N - num(A_i)) = dp(N) - 1$ であるかを調べればよいです。最上位の桁が決まれば、同様に、上から 2 番目、3 番目と上位の桁から順に決めていきます。

さて、残るは $dp(i)$ の求め方です。これは、次のように求められます。ただし、ちょうど i 本のマッチ棒を使って条件を満たすように整数を作れない場合は $dp(i) = -\infty$ とします。

$dp(i)$ を求める際には、ちょうど i 本のマッチ棒を使って条件を満たす整数の最上位の桁 A_j を全探索します。最上位の桁が A_j だとすると、ちょうど $i - num(A_j)$ 本のマッチ棒を使って作った $dp(i - num(A_j))$ 桁の整数の頭に A_j を付け加えて $dp(i - num(A_j)) + 1$ 桁の整数を作ることができます。すなわち、

$$dp(i) = \max_j (dp_{i-num(A_j)} + 1)$$

が成り立ちます。さらに、$dp(0) = 0$ なので、$i = 1, 2, ...$ と小さい順に求めていくことができます。

このようにして解くことができ、出力桁数が $\frac{N}{2}$ 以下であることに注意すれば[*1]、計算量は $O(NM + MlogM)$ になります。

余談: 0 も作れる場合は少し複雑になりますが同じような方法で解くことができます。

[*1] $111\cdots1$ と出力する場合が最大です

ABC 119 解説

出題・解説: @evima0

2019 年 2 月 24 日

A: Still TBD

正解までの道のりを細かく分割すると次の通りです。

1. 標準入力から文字列 s を受け取る。(または、別の何らかの形で入力を受け取る。)
2. s が表す日付が 2019/04/30 以前であるか判定し、答えとなる文字列 Heisei または TBD を得る。
3. 得た文字列を標準出力に出力する。

手順 1, 3 については practice contest 問題 A の言語別サンプルコードが参考になります。手順 2 については、まず if 文などの何らかの条件分岐を行う機構を用いる必要があります。それに加えて、s と文字列 2019/04/30 を辞書式の順序で直接比較するのが一つの手です。文字列を辞書式順序で比較する機能は現代の多くの言語に標準で実装されており、「自前」で実装する必要はありません。Python3 での実装例を示します。

```
S = input()
print('Heisei' if S <= '2019/04/30' else 'TBD')
```

言語によっては、入力をはじめから整数と文字の並びとして受け取り、「月」の部分が 4 以下であるか否かを判定するという方針も大いに考えられます。C++ での実装例を示します。

```
#include <iostream>
using namespace std;
int main(){
    int y, m, d;
    char c1, c2;
    cin >> y >> c1 >> m >> c2 >> d;
    cout << (m <= 4 ? "Heisei" : "TBD") << endl;
}
```

B: Digital Gifts

親戚の人数が 2 人、3 人などと固定されていれば問題 A と大差ありませんが、実際の親戚の人数 N は 2 以上 10 以下と固定されておらず、それに対応するために何らかの「ループ機構」を使うことになります。その代表例が for 文で、この問題でも for 文を使うのが素直でしょう。以下、プログラムで答えを求めるための機械的な手順を述べます。

まず標準入力から N の値を読み込み、ans という値を宣言して 0 で初期化します。そして次に述べる処理を N 回繰り返すと、最終的に ans に求めたい値が入ります。i 回目の処理 $(1 \leq i \leq N)$ は次の通りです:「標準入力から小数 x_i (整数も小数の一種と考えられる) と文字列 u_i を読み込む。$u_i =$ `JPY` なら ans に x_i を足す。$u_i =$ `BTC` なら ans に $x_i \times 380000.0$ を足す」。

これを Python3 で実装したコードが以下です。

```
N = int(input())
rate = 380000.0
ans = 0.0
for i in range(N):
    x, u = input().split()
    x = float(x)
    ans += x * rate if u == 'BTC' else x
print('{:.11f}'.format(ans))
```

なお、小数を出力する際は基本的に桁数を多めに (最低でも 10 桁) 持たせることを勧めます。言語と問題によっては、小数を出力する際の標準の桁数に頼ると精度が足りないことがあるためです。

C: Synthetic Kadomatsu

竹の数は 8 本までと少ないですが、それでも三種類の魔法による行動パターンの総数は極めて多いです。

しかし、足し算の結果は順序を入れ替えても変わらないため、例えば「竹 X に延長魔法→竹 Y に延長魔法→竹 X, Y に合成魔法を使用して竹 XY とする→竹 XY に延長魔法」という一連の行動は、「竹 X, Y に合成魔法を使用して竹 XY とする→竹 XY に延長魔法を 3 回」という一連の行動と同等です。

このように、「以後合成魔法を使用しないような竹に対してのみ延長・縮小魔法を使用する」ような行動パターンのみを考えると、可能な行動パターンを本質的にすべて列挙することが可能になります。

具体的には、竹 $1, 2, \ldots, N$ のそれぞれに対し、その竹の扱いは本質的には次の 4 通りに限られます:「長さ A の竹の “材料” とする」「長さ B の竹の材料とする」「長さ C の竹の材料とする」「使わない」。この合計 4^N 通りの竹の扱い方をすべて試行します (最大で $4^8 = 65536$ 通りで、この数は十分現実的です)。

ただし、“無” に対して延長魔法を使って竹を得ることはできないことに注意します。すなわち、「長さ A の竹の材料」「長さ B の竹の材料」「長さ C の竹の材料」がそれぞれ一本以上必要で、これを満たさない竹の扱い方は棄却します。

上記の条件を満たす竹の扱い方に対しては、その場合の「長さ A の竹の材料」をすべて合成して竹を得て、延長魔法と縮小魔法のうち適切な方を適切な回数だけ使って長さ A の竹を得るためのコストを計算します。長さ B, C の竹についても同様に得るためのコストを計算して合計すれば、その竹の扱い方で達成できる最小のコストを計算できます。以上を 4^N 通りすべての竹の扱い方に対して行って得られた最小値が答えです。

```python
N, A, B, C = map(int, input().split())
l = [int(input()) for i in range(N)]
INF = 10 ** 9

def dfs(cur, a, b, c):
    if cur == N:
        return abs(a - A) + abs(b - B) + abs(c - C) - 30 if min(a, b, c) > 0 else INF
    ret0 = dfs(cur + 1, a, b, c)
    ret1 = dfs(cur + 1, a + l[cur], b, c) + 10
    ret2 = dfs(cur + 1, a, b + l[cur], c) + 10
    ret3 = dfs(cur + 1, a, b, c + l[cur]) + 10
    return min(ret0, ret1, ret2, ret3)

print(dfs(0, 0, 0, 0))
```

D: Lazy Faith

一つの問い x_i について考えましょう。地点 x_i より西にある神社のうち最も東にある神社を s_a、x_i より東にある神社のうち最も西にある神社を s_b とします。もとの設定のままでは神社 s_a, s_b のいずれかが存在しない可能性がありますが、道路のはるか西とはるか東、訪れる価値がないような位置 (例えば、道路の西端から 10^{18} メートル西と 10^{18} メートル東) にも神社があるとすると神社 s_a, s_b はどちらも必ず存在します。

このとき、道路上に s_a, s_b 以外の神社は存在しないとしても答えは変わりません。なぜなら、もしその他の神社 s_o を訪れることがあれば、その過程ですでに s_a, s_b のいずれかを通過しているはずで、s_o をあえて訪れる必要はないためです。同様に寺 t_c, t_d を定義すると、それら以外の寺は存在しないとしても答えは変わりません。よって、s_a, s_b, t_c, t_d のうちどれをどの順に訪れるかの選択 8 通り[*1]をすべて試せば答えが求まります。

あとは、s_a, s_b, t_c, t_d をどのように高速に求めるかが課題です。b が求められれば、$a = b - 1$ であり c, d も同様に求められるため、以下では b を求めることを考えます。

b をより数学的に定義すると「$s_j > x_i$ であるような最小の整数 j」となります ($s_1, \ldots, s_A$ がはじめから昇順に並んでいることに注意)。このような値を高速に求めるアルゴリズムに 二分探索 (リンク先は Wikipedia の同名の記事) があり、現代の多くの言語に標準で実装されています。これを用いると、$\lceil \log_2(A+2) \rceil$ ステップ[*2]の演算で b を求めることができ、10 万個の問いに対しても十分高速です。[*3]

```python
import bisect
A, B, Q = map(int, input().split())
INF = 10 ** 18
s = [-INF] + [int(input()) for i in range(A)] + [INF]
t = [-INF] + [int(input()) for i in range(B)] + [INF]
for q in range(Q):
    x = int(input())
    b, d = bisect.bisect_right(s, x), bisect.bisect_right(t, x)
    res = INF
    for S in [s[b - 1], s[b]]:
        for T in [t[d - 1], t[d]]:
            d1, d2 = abs(S - x) + abs(T - S), abs(T - x) + abs(S - T)
            res = min(res, d1, d2)
    print(res)
```

[*1] $s_a \to t_c, s_a \to t_d, s_b \to t_c, s_b \to t_d$ とこれらの逆順。これらの中には通り過ぎた寺社に戻ってくるような無意味な歩き方もありますが、最適解を求める上で害はありません

[*2] $\lceil x \rceil$ は x 以上の最小の整数です。なお、数式中の “+2” は西の果てと東の果てに足した神社に対応します

[*3] ただし “遅い” 言語ではあまり余裕がなく、上記のコードも最適化が甘くかろうじて間に合う程度です

ABC 120 解説

writer: drafear

2019 年 3 月 3 日

A: Favorite Sound

最大 C 回まで聞けるといった制約が無い場合、答えは $\frac{B}{A}$ の小数点以下切り捨てになります。従って、最大 C 回の制約がある場合、この値と C の小さい方が答えになります。以下に C++ による回答例を示します。

```
#include <bits/stdc++.h>

using namespace std;

int main() {
  int A, B, C; cin >> A >> B >> C;
  int ans = min(B / A, C);
  cout << ans << endl;
}
```

B: K-th Common Divisor

正整数 i が A も B も割り切るなら、$1 \leq i \leq \min(A, B)$ を満たします。したがって、$M = \min(A, B)$ として $i = M, M-1, M-2, ..., 1$ と順に i が A, B を共に割り切るか調べていき、K 番目に見つけた数を出力すれば正答できます。M を $\min(A, B)$ とする代わりに、単に A や B, 100 など M が $\min(A, B)$ 以上であれば正答できます。以下に C++ での実装例を示します。

```
#include <bits/stdc++.h>

using namespace std;

int main() {
  int A, B, K; cin >> A >> B >> K;
  for (int i = min(A, B); i >= 1; --i) {
    if (A % i == 0 && B % i == 0) {
      --K;
      if (K == 0) {
        cout << i << endl;
        return 0;
      }
    }
  }
  assert(false);
}
```

別解として、A も B も割り切る数 (A と B の公約数) は A と B の最大公約数の約数なので、これを列挙して大きい方から K 番目の数を出力する方法もあります。

C: Unification

文字列 S に 0 も 1 も 1 つ以上残っている場合、どこかで 0 と 1 が隣り合っています。従って、与えられた文字列 S に含まれる 0 の個数と 1 の個数を数え、それぞれ C_0, C_1 とすると、どのような順番で取り除いても $\min(C_0, C_1)$ 回取り除く操作ができます。よって、答え (取り除けるキューブの個数の最大値) は $2 \times \min(C_0, C_1)$ になります。このアルゴリズムは $O(|S|)$ で動作します。

D: Decayed Bridges

島を頂点、橋を辺としたグラフを考えます。グラフから辺を削除していくのは難しいことが多いので、逆に辺が 1 つもない状態から追加していくことを考えます。すなわち、後ろから答えを求めて配列に保存し、逆順に出力します。

i 番目に出力すべき答えを $ans(i)$ とします。辺を追加していくことを考えると、初めは全ての辺がないので、

$$ans(M) = \frac{N(N-1)}{2}$$

です。さらに、各 i $(1 \leq i \leq M-1)$ について、$i+1$ 番目の辺 $e_{i+1} = (a_{i+1}, b_{i+1})$ によって

1. 元々頂点 a_{i+1}, b_{i+1} が連結でなかった (互いに行き来可能でなかった) 場合
   辺 e を加える前のグラフにおいて、頂点 a_{i+1} を含む連結成分の大きさを N_1、頂点 b_{i+1} を含む連結成分の大きさを N_2 とすると、
   $$ans(i) = ans(i+1) - N_1 \times N_2$$
   です。
2. 元々連結だった場合
   不便さに変化はなく、$ans(i) = ans(i+1)$ です。

しかしこれを愚直に実装すると $O(M(N+M))$ かかってしまい、間に合いません。ボトルネックになっているところは、N_1, N_2 を求める部分です。これは、以下の操作が行える、Union-Find と呼ばれるデータ構造を用いることで効率よく求めることができます。

unite(u, v): 頂点 u が属するグループと頂点 v が属するグループを併合し、同じグループにする $(O(\alpha(n)))$
find(v): 頂点 v が属するグループ番号を得る $(O(\alpha(n)))$
size(v): 頂点 v が属するグループと同じグループに属する頂点数を得る $(O(1))$

ここで、n は管理する頂点数、α はアッカーマン関数の逆関数ですが、$O(\alpha(n))$ は $O(1)$ とほとんど変わりません。したがって、全体の計算量は $O(M\alpha(N))$ となり十分間に合います。

ABC 121 解説

writer: drafear

2019 年 3 月 9 日

A: Cells

”残る白色のマス目の数は行や列の選び方によらない” ことを信じると、例えば後ろから h 行 w 列を選んで黒色で塗りつぶした場合の白マス数を数えれば良いです。このときの白マス数は $(H-h)\times(W-w)$ です。

他の考え方もあります。初めに h 行選んで黒く塗りつぶし、次に w 列選んで黒く塗りつぶすことを考えます。すると、初めに h 行選んで黒く塗りつぶされる白マス数は $h\times W$ になります。その後に w 列選んで黒く塗りつぶされる白マス数は、1 列選ぶたびに $H-h$ 個の白マスが塗りつぶされるので $(H-h)\times w$ になります。初め、白マスは $H\times W$ 個あったので、答えは $HW-hW-(H-h)w$ になります。

”残る白色のマス目の数は行や列の選び方によらない” ことの直感的な証明を述べます。選ばれなかった行、列の数はそれぞれ $H-h$ 行、$W-w$ 列です。選ばれなかった各行、各列に対して 1 つの白マスが対応するので、答えは $(H-h)\times(W-w)$ になります。

C++ での実装例を以下に挙げます。

```cpp
#include <bits/stdc++.h>

using namespace std;

int main() {
  int H, W; cin >> H >> W;
  int h, w; cin >> h >> w;
  int ans = (H - h) * (W - w);
  cout << ans << endl;
}
```

B: Can you solve this?

各 i について、$sum(i) = A_{i1}B_1 + A_{i2}B_2 + ... + A_{iM}B_M$ を計算し、$sum(i) + C > 0$ であるような i の個数が答えになります。これを実装できれば正答することができますが、実装に慣れていない場合特に入力を受け取る部分の実装が難しいと思います。以下の実装例では、

1. 入力を受け取り、変数に格納する
2. 計算し、答えを求める
3. 答えを出力する

の 3 つのパートに分けて実装しています。他の問題でも、入力・計算・出力 に分けて実装するのが分かりやすいのでおすすめです。今回の場合、変数の型としては、A は整数型の 2 次元配列、B は整数型の 1 次元配列、N, M, C は整数型とするのが良いでしょう。

```cpp
#include <bits/stdc++.h>

using namespace std;

int main() {
  // input
  int N, M, C; cin >> N >> M >> C;
  vector<int> B(M);
  for (int i = 0; i < M; ++i) {
    cin >> B[i];
  }
  vector<vector<int>> A(N, vector<int>(M));
  for (int i = 0; i < N; ++i) {
    for (int j = 0; j < M; ++j) {
      cin >> A[i][j];
    }
  }
  // compute
  int ans = 0;
  for (int i = 0; i < N; ++i) {
    // check whether i-th source code solves this problem
    int sum = 0;
    for (int j = 0; j < M; ++j) {
      sum += A[i][j] * B[j];
    }
    if (sum + C > 0) {
```

```
      ++ans;
    }
  }
  // output
  cout << ans << endl;
}
```

C: Energy Drink Collector

安い店から順に M 本のエナドリを買うのが最善です。したがって、A_i の小さい順にソートして、M 本に達するまで前から順に買えるだけ買うことをシミュレーションすれば答えを求めることができます。ソートの方法はいくつかありますが、例えば C++ では vector<pair<int,int>>に組 (A_i, B_i) を全て入れて sort 関数を呼ぶことでソートすることができます。計算量は、ソート部分がボトルネックで $O(NlogN)$ です。

D: XOR World

x と y の排他的論理和を $x \oplus y$ と書くことにします。排他的論理和の計算順序は入れ替えて良く、また、どの非負整数 n についても $n \oplus n = 0$ なので、$A > 0$ のとき

$$F(A, B) = F(0, A-1) \oplus F(0, B)$$

が成り立ちます。そこで、$f(n) = F(0, n)$ を求めることを考えます。

これは、以下の重要な性質に気づくことができれば、簡単に求めることができます。気づかなかった場合でも、答えを 2 進数で表したときの上位桁から順に決めていく $O(log(B))$ の解法もあります。

重要な性質

任意の偶数 n について $n \oplus (n+1) = 1$

この性質は、偶数 n について n と $n+1$ を 2 進数表記したときに最下位ビットのみが異なることから分かります。例えば、$100 = (1100100)_2, 101 = (1100101)_2$ です。この性質を使えば、例えば

$$\begin{aligned} & 0 \oplus 1 \oplus 2 \oplus 3 \oplus 4 \oplus 5 \oplus 6 \\ = & (0 \oplus 1) \oplus (2 \oplus 3) \oplus (4 \oplus 5) \oplus 6 \\ = & 1 \oplus 1 \oplus 1 \oplus 6 \end{aligned}$$

のように計算できます。$1 \oplus 1 \oplus ... \oplus 1$ は 1 が偶数個なら 0、奇数個なら 1 になります。このように計算することで、答えを $O(1)$ で求めることができます。

ABC 122 解説

出題・解説: @evima0

2019 年 3 月 24 日

A: Double Helix

正解までの道のりを細かく分割すると次の通りです。

1. 標準入力から文字 b を受け取る。
2. 何らかの方法で b と対になる塩基を表す文字 b' を得る。
3. b' を標準出力に出力する。

手順 1, 3 については practice contest 問題 A の言語別サンプルコードが参考になります。手順 2 については、結局のところ $b =$ `A` なら $b' =$ `G`、$b =$ `G` なら $b' =$ `A`、... と 4 つの文字それぞれに相方を指定することになるでしょう。方法は大きく分けて二つ考えられ、if, else if, else や switch にあたる構文で b に応じてプログラムを分岐させるか、「答えの一覧」にあたるデータ構造を連想配列などで作って b に対応する値を参照するかです。C++, Python3 による後者のアプローチの実装例を示します。

```cpp
#include <iostream>
using namespace std;
int main(){
    char b, c[128];
    c['A'] = 'G';
    c['G'] = 'A';
    c['C'] = 'T';
    c['T'] = 'C';
    cin >> b;
    cout << c[(int)b] << endl;
}
```

```python
b = input()
c = {'A': 'T', 'T': 'A', 'C': 'G', 'G': 'C'}
print(c[b])
```

B: ATCoder

文字列 S の長さを N とします。S の部分文字列は、空文字列以外に $N + (N-1) + (N-2) + ... + 1 = N(N+1)/2$ 個存在します (S の 1 文字目から始まるものが N 個、2 文字目から始まるものが $N-1$ 個、...、N 文字目から始まるものが 1 個。なお、中身が同じでも S の異なる位置から取り出された部分文字列は別として数えています)。

この問題では $N \leq 10$ であり、この個数は最大で $10(10+1)/2 = 55$ です。これらをすべて調べて、そのうち最も長い ACGT 文字列の長さを報告してほしいという問題です (より効率的に解くこともできますが不必要です)。Python3 による実装例をもってこの解説の残りとします。

```
S = input()
N = len(S)
ans = 0
for i in range(N):
    for j in range(i, N):
        if all('ACGT'.count(c) == 1 for c in S[i : j + 1]):
            ans = max(ans, j - i + 1)
print(ans)
```

C: GeT AC

文字列 S の長さ N と問いの数 Q さえ小さければ、この問題は前問と同等の「書かれた通りに実装する」問題になります。しかし、実際には N も Q も最大で 10 万と大きく、「書かれた通り」の実装を行うと最大で延べ 100 億文字を走査することになります。これは、C++ のような高速な言語でも 2 秒で処理できる量ではありません。処理を高速化する何らかの工夫が必要です。

範囲内の部分文字列 `AC` を数えよと問われていますが、その代わりに「右隣が `C` であるような `A`」を数えた方が単純です。文字列中のこのような `A` を `a` に置き換えると、問 i は「l_i 文字目から $r_i - 1$ 文字目までの `a` を数えよ」となります (右端が 1 左に動いたことに注意してください。r_i 文字目が `a` であってもその右隣の `C` が範囲からはみ出てしまうためです)。

ここで、$t_i =$「S の 1 文字目から i 文字目までに出現する `a` の数」として数列 $t = \{t_0, t_1, \ldots, t_N\}$ を考えると、問 i の答えは $t_{r_i-1} - t_{l_i-1}$ として求められます。この数列 t を問いの処理を始める前に構築しておいて使い回せば、入力サイズの線形時間ですべての問いを処理できます。

```python
N, Q = map(int, input().split())
S = input()
t = [0] * (N + 1)
for i in range(N):
    t[i + 1] = t[i] + (1 if S[i : i + 2] == 'AC' else 0)
for i in range(Q):
    l, r = map(int, input().split())
    print(t[r-1] - t[l-1])
```

D: We Like AGC

N が小さければ、4^N 通りの ACGT 文字列すべてを候補として列挙した中から数えられますが、N が最大で 100 では絶望的です。しかし、結論から述べると、これと同等のことを短時間で行うことができます。

第三の条件「隣接する 2 文字の入れ替えを 1 回行うことで第二の条件 (部分文字列として AGC を含まない) に違反させることはできない」を言い換えると、「部分文字列として GAC, ACG, A?GC, AG?C のいずれも含まない (? は何らかの 1 文字)」となります。

よって、条件を満たす文字列を 1 文字目から順に作るとき、「i 文字目を何にするか」の選択は、「$i-4$ 文字目以前が何であったか」を気にすることなく、$i-3, i-2, i-1$ 文字目のみを考慮して行うことができます。これを元に、メモ化再帰または動的計画法により N の線形時間で解を求められます (“定数倍” は大きいです)。

```python
N, MOD = int(input()), 10 ** 9 + 7
memo = [{} for i in range(N+1)]

def ok(last4):
    for i in range(4):
        t = list(last4)
        if i >= 1:
            t[i-1], t[i] = t[i], t[i-1]
        if ''.join(t).count('AGC') >= 1:
            return False
    return True

def dfs(cur, last3):
    if last3 in memo[cur]:
        return memo[cur][last3]
    if cur == N:
        return 1
    ret = 0
    for c in 'ACGT':
        if ok(last3 + c):
            ret = (ret + dfs(cur + 1, last3[1:] + c)) % MOD
    memo[cur][last3] = ret
    return ret

print(dfs(0, 'TTT'))
```

AtCoder Beginner Contest 123 解説

April 6th, 2019

Problem Setters : E869120, square1001

Notice for Internatioal Readers: Currently, AtCoder is not preparing editorial for ABC-only contests.

問題 A : Five Antennas

この問題は、問題文の通りに、次の 10 個の条件をすべて満たしているか判定し、すべて満たしているならば “Yay!”、そうでないならば “:(” と出力すると解くことができます。

1. $-k \leq a - b \leq k$
2. $-k \leq a - c \leq k$
3. $-k \leq a - d \leq k$
4. $-k \leq a - e \leq k$
5. $-k \leq b - c \leq k$
6. $-k \leq b - d \leq k$
7. $-k \leq b - e \leq k$
8. $-k \leq c - d \leq k$
9. $-k \leq c - e \leq k$
10. $-k \leq d - e \leq k$

しかし、これをすべて判定していると実装をするのにかなり手間がかかります。

そこで、「$a < b < c < d < e$」という制約に注目します。そこで、最も遠いアンテナは A と E なので、結果的に A と E の座標しか気にしなくてもよいのです！よって、次のたった 1 つの条件を満たしているか判定すればよいです。

1. $e - a \leq k$

よって、簡単にこの問題を正解することができます。入力・出力は、C++ では cin/cout を使う、Python では input、print を使うなどをすると実装することができます。また、条件分岐は、多くのプログラミング言語では if/else を使うと実装することができます。

分からない人は、AC している他の人のソースコードを見ると勉強になるかもしれません。

ソースコード（C++）: https://atcoder.jp/contests/abc123/submissions/4840685

問題 B：Five Dishes

主に、この問題には 2 通りの解法があります。1 つ目の方法は「5 つの料理を注文する順番を $5! = 120$ 通り全探索する」もので、2 つ目の方法は「最後に注文した方が良さそうなものを決める」ものです。そのふたつの解法について説明していきます。

解法 #1: 5 つの料理を注文する順番を全探索する

注文する順番が決まると、「料理が来たときに、次に注文できるタイミングですぐ次の料理を注文する」という方法で 5 つの料理を注文していくのが最適です。

例えば $A = 13, B = 17, C = 21, D = 25, E = 29$ で、「ABC 丼 → ARC カレー → AGC パスタ → APC ラーメン → ATC ハンバーグ」の順に注文するとき、次のようになります。

- 時刻 0 に ABC 丼を注文し、13 に届く。次に注文できるのは時刻 20
- 時刻 20 に ARC カレーを注文し、37 に届く。次に注文できるのは時刻 40
- 時刻 40 に AGC パスタを注文し、61 に届く。次に注文できるのは時刻 70
- 時刻 70 に APC ラーメンを注文し、95 に届く。次に注文できるのは時刻 100
- 時刻 100 に ATC ハンバーグを注文し、129 に届く。(次に注文できるのは時刻 130)

「注文の順番」は $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$ 通りあります。この全てに対して時刻いくつで 5 つの料理すべてが届くかを計算し、そのうち最も速いものが答えになります。

解法 #2: 最後に注文した方が良さそうなものを求める

先ほどの例に着目してみましょう。ABC 丼は時刻 0 に注文され、時刻 13 に届き、次に注文できるのは時刻 20 なので、「注文から次の注文までの時間は 20 分」です。一方、AGC パスタは時刻 40 に注文され、時刻 61 に届き、次に注文できるのは時刻 70 なので、「注文か次の注文までの時間は 30 分」です。

実は、この「注文から次の注文までの時間」は、どのような順番で注文しても変わりません！これは、注文してから次の注文までの時刻が 10 の倍数であるからです。つまり、調理時間 X 分の料理について、注文してから次の注文までの時刻は「X 以上の最小の 10 の倍数分」になります。

つまり、最後に注文する料理以外は「X 以上の最小の 10 の倍数分」かかるとみなしてよいです。最後に注文する料理は、調理時間そのままの時間かかるので、「(X 以上の最小の 10 の倍数) と X の差」が最大になるような料理を選べば、最短の時間になります。

解法 1 のソースコード (C++)： https://atcoder.jp/contests/abc123/submissions/4840052
解法 2 のソースコード (C++)： https://atcoder.jp/contests/abc123/submissions/4840010

問題 C：Five Transportations

◆ ステップ #1：貪欲的な方法でシミュレーション

人を都市 6 に向かって「限界の人数まで進ませられるだけ進ませる」貪欲的な方法を考えます。この問題の入力例 1 では、まさにこの「貪欲的な方法」で人を動かしています。直感的に、これが最適そうな感じがします (そしてこれは実に最適な方法です)。

しかし、これを単純にシミュレーションしていると、移動時間に比例する計算時間がかかります。個の問題の制約を見ると $N, A, B, C, D, E \leq 10^{15}$ となっていますので、最悪ケースで $10^{15} + 4$ 分の移動時間がかかります ($N = 10^{15}, A = 1, B = 1, C = 1, D = 1, E = 1$ の場合)。つまり、貪欲的な方法でシミュレーションをすると「実行時間超過 (TLE)」となり、不正解となります！

もっと高速な方法を考えてみましょう・・・が、しかし、単純にシミュレーションを工夫して高速化するのは難しいので、別の方向からこの問題を考えてみることにします。

◆ ステップ #2：最も「きつい」交通機関を考える

最も通りにくい、つまり容量が最も小さい交通機関を考えます。直感的には、最短時間で移動する場合でも、1 分でたどり着けるのは X 人 (交通機関の容量の最小を X とする) になります。これは、「1 秒に 15 L のペースで水を送れるポンプと、1 秒に 12 L のペースで水を送れるポンプをつなげたときに、1 秒に送れる水の量は小さい方を取って 12 L」になるのと同じ原理です。

つまり、先ほどの貪欲的な方法ではなくて、1 分に X 人ずつ 1 つ先に進めてやっても、直感的にはかかる時間は変わらなさそうです。また、移動時間は「($N \div X$ を切り上げた値) + 4 分」と、簡単に計算できるようになります。

これが最短である証明が気になる人もいるかもしれないので書いておきます。

都市 i と $i+1$ を結ぶ交通機関が最小容量 X であるとき、この交通機関にはスタートから $i-1$ 分後から人が乗り始め、ゴールから $5-i$ 分前には人が全員この交通機関を使い終わっていなければなりません。合計で $(i-1)+(5-i)=4$ 分間は“誰も乗らない時間”があります。また、N 人が容量 X の交通機関を使うので最小でも ($N \div X$ を切り上げた値) 回使うことになるので、最低でも“($N \div X$ を切り上げた値) + 4 分”必要になることが分かります。

よって、$ceil(x)$ を実数 x を切り上げた値として、答えは“$ceil(N \div \min(A, B, C, D, E)) + 4$ 分”と、簡単な式で表すことができました！

ソースコード (C++)： https://atcoder.jp/contests/abc123/submissions/4840678

問題 D : Cake 123

この問題では、一番単純な解法として、XYZ 通りを全探索して、全てのコストの合計をソートして、大きい順から K 番目までを順に出力する、という方法が考えられます。

しかし、N 要素のソートの計算量は $O(N \log N)$ です。[1] そのため、この単純な解法の計算量は $O(XYZ \log XYZ)$ となり、$X, Y, Z = 1000$ の時 3×10^{10} 回程度の計算がかかり、実行時間制限の 2 秒に間に合いません。

そこで、ある工夫をするとこの問題が解けます。解法は色々とあり、今回はこのうちの 4 通りを説明したいと思います。

解法 #1 – 工夫したソートで $O(K^2 \log K)$

1 つ目の解法として、以下のような方法が考えられます。

- まず、最初の 2 種類のケーキ（'1', '2' の形のキャンドルが付いたもの）だけを考えて、XY 通りの中で上位 K 通りを順に求める。そこで、$A_i + B_j$ でソートし、その中の k 番目 $(1 \leq k \leq K)$ を E_k とする。
- 次に、最後のケーキ（'3' の形のキャンドルが付いたもの）を考える。最終的な美味しさの合計が上位 K 番目までに入るようなケーキの選び方は、最初の 2 種類のケーキだけを考えても上位 K 番目までに入らなければならないので、$E_i + C_j$ $(1 \leq \mathrm{i}, \mathrm{j} \leq \mathrm{K})$ でソートして、$1, 2, 3, \ldots, \mathrm{K}$ 番目の値が答えである。

例えば、$A = \{1, 2, 3\}, B = \{1, 5, 7\}, C = \{1, 9, 3\}, K = 5$ のとき、

- $A_i + B_j$ でソートした時の上位 5 番目までの値（$E_1 \sim E_5$）は $\{10, 9, 8, 8, 7\}$ である
  - 上位 9 番目までを見ると、$\{10, 9, 8, 8, 7, 6, 4, 3, 2\}$
- 次に、$E_i + C_j$ でソートした時の上位 5 番目までの値は、$\{19, 18, 17, 17, 16\}$ である。
  - 上位 15 番目までを見ると、$\{19, 18, 17, 17, 16, 13, 12, 11, 11, 11, 10, 10, 9, 9, 8\}$
- よって、XYZ 個の中の上位 5 個は $\{19, 18, 17, 17, 16\}$ である。

この解法を用いると、高々 $O(XY + KZ)$ 個をソートする必要がありますが、$X, Y, Z \leq 1000, K \leq 3000$ より実行時間制限に間に合います。

解法 #2 –「ある条件」を満たす最大 106307 通りを全部ソート

2 つ目の解法として、以下のような定理を使います。

【定理】
以下のような選び方は、上位 K 個には絶対入っていない。

- 高橋君が選んだ‘1’の形のキャンドルが付いたケーキが、それらのケーキの中で a 番目に美味しいとする。$(1 \leq a \leq X)$
- 高橋君が選んだ‘2’の形のキャンドルが付いたケーキが、それらのケーキの中で b 番目に美味しいとする。$(1 \leq b \leq Y)$
- 高橋君が選んだ‘3’の形のキャンドルが付いたケーキが、それらのケーキの中で c 番目に美味しいとする。$(1 \leq c \leq Z)$
- そのとき、$abc > K$ の場合、絶対に上位 K 個には入っていない。具体的には、一番順位が高い場合でも上から abc 番目である。

実際に、上の定理の条件を満たさない（つまり、$a \times b \times c \leq K$ である）ようなケーキの選び方は、$X, Y, Z = 1000, K = 3000$ の場合 106,307 通りあります。そうすると、条件を満たす 106,307 通り全てをソートしても、計算量的には全く問題がありません。

また、一般の K に対して、$a \times b \times c \leq K$ であるケーキの選び方は高々 $O(K \log^2 K)$ 通りです。つまり、この解法の計算量は $O(K \log^3 K)$ です。

以下のようなコードを書くと正解します。

```cpp
#include <bits/stdc++.h>
using namespace std;

long long X, Y, Z, K, A[1009], B[1009], C[1009];
vector<long long>vec;

int main() {
	cin >> X >> Y >> Z >> K;
	for (int i = 0; i < X; i++) cin >> A[i]; sort(A, A + X, greater<long long>());
	for (int i = 0; i < Y; i++) cin >> B[i]; sort(B, B + Y, greater<long long>());
	for (int i = 0; i < Z; i++) cin >> C[i]; sort(C, C + Z, greater<long long>());

	for (int i = 0; i < X; i++) {
		for (int j = 0; j < Y; j++) {
			for (int k = 0; k < Z; k++) {
				if ((i + 1) * (j + 1) * (k + 1) <= K) vec.push_back(A[i] + B[j] + C[k]);
				else break;
			}
		}
	}
	sort(vec.begin(), vec.end(), greater<long long>());
	for (int i = 0; i < K; i++) cout << vec[i] << endl;
	return 0;
}
```

解法 #3 – さらに高速な解法、priority-queue を用いて $O(K \log K)$

この問題は、以下のようなアルゴリズムによって $O(K \log K)$ で解くことができます。

- 数列 A, B, C (各形のキャンドルが付いたケーキ) を降順ソートする。そして、A_i が i 番目に美味しい '1' の形のキャンドルが付いたケーキとなるようにする。B_j, C_k に関しても同様とする。
- Priority Queue (優先度付きキュー) である Q を定義する。
- Q に $(A_1 + B_1 + C_1, 1, 1, 1)$ を追加する。なお、$A_1 + B_1 + C_1$ が 1 番目の美味しさの合計であることが保証されている。
- 以下の操作を K 回繰り返す。
  - Q の中で一番大きい要素を (p, i, j, k) とする。この組が t 番目の美味しさの合計である。ただし、ここで行っている操作が t 回目の操作であるとする。
  - Q の中で一番大きい要素を削除する。
  - $(A_{i+1} + B_j + C_k, i + 1, j, k)$ がまだ追加されたことが無ければ Q に追加する。
  - $(A_i + B_{j+1} + C_k, i, j + 1, k)$ がまだ追加されたことが無ければ Q に追加する。
  - $(A_i + B_j + C_{k+1}, i, j, k + 1)$ がまだ追加されたことが無ければ Q に追加する。

実際に、このアルゴリズムは正しく動作します。

理由は、$f(a, b, c)$ の値を「$A_a + B_b + C_c$ という選び方の美味しさの合計が上から何番目か、という値」とするとき、$f(a, b, c) < f(a + 1, b, c), f(a, b, c) < f(a, b + 1, c), f(a, b, c) < f(a, b, c + 1)$ が成り立つからです。

Priority Queue の挿入・削除・一番大きい値を求めるといった操作は $O(\log K)$ で動作するので、この問題を $O(K \log K)$ と、前 2 つの解法より大幅に高速に解くことができます。

なお、Priority Queue (優先度付きキュー) を知らない人は参考文献[2] を参考にしてください。

解法 #4 – 二分探索で $O(K \log max)$

まず、「上から K 番目の美味しさにランクインするボーダーの値」を求めることを考えます。

前提として、ここでは数列 A, B, C がソートされているものとします。

ボーダーの値は、二分探索によって求めることができます。

さて、「美味しさの合計が P 以上である個数が K 個以上あるかどうかを高速判定する方法」はどのように求めればよいのでしょうか。これは、単純な枝刈り全探索で $O(K^2)$ あるいは $O(K + X + Y + Z)$ で解くことができます。

以下は、枝刈り全探索の疑似コードです。

```
bool solve(int P) {
    int cnt = 0;
    for (int i = 1; i <= X; i++) {
        for (int j = 1; j <= Y; j++) {
            for (int k = 1; k <= Z; k++) {
                if (A[i] + B[j] + C[k] < P) break;
                cnt++;
                if (cnt >= K) return true;
            }
        }
    }
    return false;
}
```

上の太字部分が求まれば、P を二分探索するだけで、K 位以内にランクインするためのボーダーの美味しさの合計値が求まります。

あとは、ボーダーの値を $Border$ とするとき、以下の方法で上位 K 個を求めることができます。

- $Border + 1$ 以上であるものを全部列挙する。なお、列挙する方法は上の疑似コードにおいて、$Border + 1$ 以上であるものすべてについて $A_i + B_j + C_k$ を記録すればよい。
- 列挙した個数が K 個に満たなければ残りは全部美味しさの合計が $Border$ である。
- これらを降順ソートしたものが、上位 K 番目まで、つまり答えである。

計算量は $O(K^2 \log max)$ です。実装を工夫すると、$O((K + X + Y + Z) \log max)$ 程度で解くことができます。

ソースコード（C++, 解法 2）: https://atcoder.jp/contests/abc123/submissions/4841936

参考文献

[1] https://qiita.com/drken/items/18b3b3db5735241465ef の 3-2. 節

[2] https://ja.wikipedia.org/wiki/優先度付きキュー

ABC 124 解説

writer: drafear

2019 年 4 月 14 日

A: Buttons

大きい方のボタンを選んで押すことを 2 回繰り返すのが最適です。基本的には同じボタンを押しますが、$A = B$ のときは両方のボタンを 1 回ずつ押すのが最適なので、注意する必要があります。

実装では、以下の 3 つのパターンについて場合分けすると比較的簡単に書くことができます。

- $A > B$ の場合: $A + (A - 1)$ が答えになります。
- $A < B$ の場合: $B + (B - 1)$ が答えになります。
- $A = B$ の場合: $A + B$ が答えになります。

C++ での実装例を以下に示します。

```
#include <bits/stdc++.h>

using namespace std;

int main() {
  int A, B; cin >> A >> B;
  int ans;
  if (A > B) {
    ans = A + (A-1);
  }
  else if (B > A) {
    ans = B + (B-1);
  }
  else {
    ans = A + B;
  }
  cout << ans << endl;
}
```

B: Great Ocean View

入力を配列で受け取り、各 $i = 1, 2, ..., N$ について全ての $j = 1, 2, ..., i - 1$ に対して $H_i \geq H_j$ かを確認し、そうであった i の数を数えることで答えを求めることができます。入力を配列に保存しておくことで後から自由に何度も使えるので、実装がしやすくなるかもしれません。C++ での実装例を以下に挙げます。

```
#include <bits/stdc++.h>

using namespace std;

int main() {
  // input
  int N; cin >> N;
  vector<int> H(N);
  for (int i = 0; i < N; ++i) {
    cin >> H[i];
  }
  // compute
  int ans = 0;
  for (int i = 0; i < N; ++i) {
    bool is_candidate = true;
    for (int j = 0; j < i; ++j) {
      if (H[j] > H[i]) {
        is_candidate = false;
        break;
      }
    }
    if (is_candidate) {
      ++ans;
    }
  }
  // output
  cout << ans << endl;
}
```

C: Coloring Colorfully

1 箇所色を決めれば他の色が全て決まり、パターンとしては 白黒白黒・・・ または 黒白黒白・・・ しかありません。この 2 つのパターンとそれぞれ何箇所異なっているかを調べ、その最小値が答えになります。時間計算量は $O(|S|)$ です。

D: Handstand

指示を一度行うたびに、人の状態が切り替わる部分を 2 箇所まで潰すことができます。これを踏まえて、連続して 1 を並ばせる左端を i 番目としたときにいくつ 1 を並べることができるかを考えます。

- $S_i = 0$ のとき: i 番目以降で $S_j \neq S_{j+1}$ となる場所を順に最大 $2K-1$ 箇所潰すのが最適です。
- $S_i = 1$ のとき: i 番目以降で $S_j \neq S_{j+1}$ となる場所を順に最大 $2K$ 箇所潰すのが最適です。

1 を並べる左端を i としたときの最大値を X_i とすると、答えは $\max\{X_1, X_2, ..., X_N\}$ となります。しかし、これをナイーブに実装しても $O(N^2)$ となり間に合いません。

そこで、何かしら工夫する必要があります。まず、i (連続して 1 を並ばせる左端) として、元の文字列で 0 または 1 が連続する始点のみを探索すれば十分です。例えば、”11000...” の 4 文字目や 5 文字目の 0 を左端とするよりも 3 文字目の 0 を左端とした方が良いでしょう。このような場所を左から順に $1 = i_1 < i_2 < ... < i_r$ とします。また、便宜上 $k > r$ なる k について、$i_k = N+1$ とします。このとき、$k = 2, 3, ..., r$ について、$S_{i_k} \neq S_{i_k - 1}$ です。よって、$k = 1, 2, ..., r$ について、X_{i_k} は

- $S_{i_k} = 0$ のとき: $X_{i_k} = i_{k+2K} - i_k$
- $S_{i_k} = 1$ のとき: $X_{i_k} = i_{k+2K+1} - i_k$

となり、答えは $\max\{X_{i_1}, X_{i_2}, ..., X_{i_r}\}$ です。これは、$O(N)$ で動作します。

別解として、二分探索を使った $O(NlogN)$ の解法や、しゃくとり法を用いた $O(N)$ の解法もあります。

A: Biscuit Generator

$T+0.5$ 秒後までに生産される回数は、T を A で割った商 $\lfloor \frac{T}{A} \rfloor$ (T を A で割った切り捨て) です。これは、例えば次のような $A=3$ のときの表を書いてみると分かりやすいかもしれません。

T	0	1	2	3	4	5	6	7	8	9	$\cdots$
答え	0	0	0	B	B	B	$2B$	$2B$	$2B$	$3B$	$\cdots$

1 回ごとに B 個生産されるので、答えは $\lfloor \frac{T}{A} \rfloor \times B$ になります。C++ での実装例を示します。

```
#include <bits/stdc++.h>

using namespace std;

int main() {
  int A, B, T; cin >> A >> B >> T;
  int generated_times = T / A;
  int ans = generated_times * B;
  cout << ans << endl;
}
```

B: Resale

i 番目の宝石を選ぶことによって $X-Y$ は V_i-C_i だけ変化します。したがって、コストよりも価値が高いような宝石のみを手に入れるのが最適です。

実装では、はじめに全ての入力を受け取ってから処理する方法をとるのが良いでしょう。なぜなら、V_1 を知ったとしても C_1 を知る前に $V_2, V_3, ..., V_N$ を読み込む必要があるためです。$V_1, V_2, ..., V_N$ は配列に保存しておくと後から値を知ることができます。C++ での実装例を挙げます。

```
#include <bits/stdc++.h>

using namespace std;

int main() {
  // input
  int N; cin >> N;
  vector<int> V(N);
  for (int i = 0; i < N; ++i) {
    cin >> V[i];
  }
  vector<int> C(N);
  for (int i = 0; i < N; ++i) {
    cin >> C[i];
  }
  // compute
  int ans = 0;
  for (int i = 0; i < N; ++i) {
    if (V[i] > C[i]) {
      ans += V[i] - C[i];
    }
  }
  // output
  cout << ans << endl;
}
```

C: GCD on Blackboard

方針としては、A_i を書き換えることにした場合の最大公約数の最大値 M_i を全て求めることを高速に行い、M_i の最大値を出力します。

まず、A_i を書き換える場合、残りの $N-1$ 個の最大公約数に書き換えれば、「整数 A_i を書き換える」操作は「整数 A_i を消す」操作と等価になります。

さらに、整数 X と 整数 Y の最大公約数を $gcd(X,Y)$ と表すことすると、gcd は結合則が成り立ちます。すなわち、任意の整数 X,Y,Z について $gcd(gcd(X,Y),Z) = gcd(X,gcd(Y,Z))$ です。簡単に言えば、gcd には「どこから計算しても結果は変わらない性質」があります[*1](例えば、整数や行列同士の加算や乗算もそうですが、減算は前から計算しなければなりません) このとき、次のテクニックが使えます。$i = 1,2,...,N+1$ について、$L(i), R(i)$ を次のように定義します。

$$L(i) := A_1, A_2, ..., A_{i-1}\text{の最大公約数}$$
$$R(i) := A_i, A_{i+1}, ..., A_N\text{の最大公約数}$$

便宜上、$L(0) = R(N+1) = 0$ とし、全ての整数 X に対して $gcd(0,X) = gcd(X,0) = X$ とすることとします。すると、A_i を書き換える (すなわち、A_i を消す) ことにした場合、残りの $N-1$ 個の整数の最大公約数 M_i は $M_i = gcd(L(i), R(i+1))$ になります。さらに、$L(i), R(i)$ は次の漸化式を計算することで高速に $O(NlogA_1 + NlogA_N)$ で求められます[*2]。

$$L(0) = 0$$
$$L(i+1) = gcd(L(i), A(i))$$
$$R(N+1) = 0$$
$$R(i) = gcd(R(i+1), A(i))$$

$M_1, M_2, ..., M_N$ が求まれば、答えはこれらの最大値 $\max(M_1, M_2, ..., M_N)$ になります。全体として計算量は $O(NlogA_1 + NlogA_N)$ となります。

[*1] gcd については「計算順序を入れ替えて良い」という交換則も成り立ちますが、今回は必要ありません。

[*2] $gcd(A,B)$ はユークリッドの互除法により $O(log \min(A,B))$ で求められます

D: Flipping Signs

かっこいい解法

考察 1(重要). この問題のポイントは、「i を選んで A_i, A_{i+1} をそれぞれ $-A_i, -A_{i+1}$ にする」操作を繰り返すことで、「x, y $(x < y)$ を選んで A_x, A_y をそれぞれ $-A_x, -A_y$ にする」ことがどの x, y についても可能であるということです。実際、$i = x, x+1, ..., y-1$ と選んで $y-x$ 回操作を行えば A_x, A_y の符号のみが反転します。

考察 2. 操作を通して、A_i は A_i または $-A_i$ にしかなり得ません。

考察 3. $A_i = 0$ となる i が存在しない場合、操作によって $A_i < 0$ となる i の数の偶奇は変化しません。

したがって、初期状態で負の数が偶数個の場合、または $A_i = 0$ となる i が存在する場合、全てを非負にできるため、$S = |A_1| + |A_2| + ... + |A_N|$ とすると、答えは S になります。そうでない場合、全てを非負にはできませんが、好きな 1 つを除いて全てを非負にできます。負のままにしておく数は、絶対値が最小のものを選ぶのが最適なので、答えは $S - 2 \times \min\{|A_1|, |A_2|, ..., |A_N|\}$ になります。時間計算量は $O(N)$ なので十分間に合います。

アルゴリズムで殴る解法

実は、このような性質を見つけなくとも動的計画法で殴ることができます。同じ i を選んで反転することは無駄なので、各 i について高々 1 回反転することを左から順に考えていきます。

$i \in \{0, 1, ..., N\}, j \in \{0, 1\}$ に対して $dp(i, j)$ を、$A_1, A_2, ..., A_i$ を確定し、$j = 1$ なら i を選んで反転した場合 (すなわち A_{i+1} の符号が反転している状態) の $A_1 + A_2 + ... + A_i$ の最大値と定義します。すると、次の漸化式を計算することで各 i, j に対する $dp(i, j)$ を求めることができます。

$$
\begin{aligned}
dp(0, 0) &= 0 \\
dp(0, 1) &= -\infty \text{ (invalid)} \\
dp(i+1, 0) &= \max(dp(i, 0) + A_i, dp(i, 1) - A_i) \\
dp(i+1, 1) &= \max(dp(i, 0) - A_i, dp(i, 1) + A_i)
\end{aligned}
$$

N を選んで操作することはできないため、答えは $dp(N, 0)$ です。時間計算量はかっこいい解法と同じく $O(N)$ なので十分間に合います。

ABC 126 解説

writer: tozangezan, gazelle, drafear

2019 年 5 月 19 日

A: Changing a Character

問題文のとおりに文字列の K 番目の文字を小文字に変換すればよいです。

A, B, C の 3 文字のみが入力に含まれるので if 文を 3 個書いてもよいですし、アルファベットの大文字と小文字の文字コードが 32 異なるということを利用して char 型の該当する文字に 32 を加えても良いです。

C++ での実装例は以下の通りです (include 等は省いています)。

```
char in[55];
int main(){
  int a,b;scanf("%d%d",&a,&b);
  scanf("%s",in);
  in[b-1]+=32;
  printf("%s\n",in);
}
```

B: YYMM or MMYY

まず、与えられる入力を長さ 4 の文字列とみなしたとき、前半 2 文字から 2 桁の整数と後半 2 文字から 2 桁の整数に変換してみましょう。例えば前半 2 文字から 2 桁の整数に変換するのは、`(s[0]-'0')*10+s[1]-'0'` などと計算することで得られます。

代わりの手段として、入力が数字列であるため、これを整数とみなし、4 桁の整数から 2 桁の整数を 2 つ取り出すこともできます。この場合では、前半の 2 桁の整数は例えば `a/100` 、後半の 2 桁の整数は `a%100` となります。

その後、各種フォーマットの条件を満たすかを判定します。

YYMM フォーマットであるためには、後半の 2 桁の整数が 1 以上 12 以下である必要があります。

MMYY フォーマットであるためには、前半の 2 桁の整数が 1 以上 12 以下である必要があります。

上の 2 つを判定することで、4 つの答えのうちどれであるかが決まります。

C++ での実装例は以下の通りです。

```
int main(){
  int a;scanf("%d",&a);
  int L=a/100;
  int R=a%100;
  if(1<=L&&L<=12){
    if(1<=R&&R<=12)printf("AMBIGUOUS\n");
    else printf("MMYY\n");
  }else{
    if(1<=R&&R<=12)printf("YYMM\n");
    else printf("NA\n");
  }
}
```

C: Dice and Coin

最初に出たサイコロの目ごとに処理していき、最後にそれぞれの確率を合計します。

それぞれ、何回コインの表を出し続けることで初めて得点が K 以上になるかは、ループで数えることで求めることができます。この回数だけコインが表を出し続けることが必要です。(途中で裏が出てしまったら得点は 0 となるので、勝つことはできません)。

サイコロを 1 回振り i $(1 \leq i \leq N)$ が出る確率は $1/N$ です。そのそれぞれに対し、t 回コインの表を出し続ける必要がある場合、成功率は 0.5^t となります。$1/N \times 0.5^t$ のすべての合計が求める答えになります。

以上の方法で計算すると、計算量は $O(NlogK)$ となり、間に合います。

C++ での実装例は以下の通りです。

```
int main(){
  int a,b;scanf("%d%d",&a,&b);
  double ret=0;
  for(int i=1;i<=a;i++){
    double tmp=1.0/a;
    int now=i;
    while(now<b){
      now*=2;
      tmp/=2;
    }
    ret+=tmp;
  }

  printf("%.12f\n",ret);
}
```

D: Even Relation

以下、与えられたグラフを適当な頂点を根とみなした根付き木として考えます。また根から頂点 i への距離を d_i とします。

任意の 2 頂点 u と v について、その最小共通祖先を w とすると、u と v の距離は $d_u + d_v - 2d_w$ と書くことができます。この式の第 3 項は偶数なので、d_u と d_w の偶奇が等しいときに限り、u と w の距離は偶数になります。よって例えば d_i が偶数の頂点は白に、奇数の頂点は黒に塗ることで条件を満たす塗り分けが可能です。

E: 1 or 2

各条件は

- $Z_i \equiv 0 \pmod 2$ のとき

$$A_{X_i} = A_{Y_i}$$

- $Z_i \equiv 1 \pmod 2$ のとき

$$A_{X_i} \neq A_{Y_i} \ (A_{X_i} = 3 - A_{Y_i})$$

です。すなわち、A_{X_i} または A_{Y_i} の一方が分かればもう一方を知ることができます。

したがって、$(X_1, Y_1), ..., (X_M, Y_M)$ 間にのみ辺が存在する N 頂点のグラフ G を考えると、i 番目のカードをめくると G 上で頂点 i と同じ連結成分の頂点に対応するカードの数字が分かります。よって、G の連結成分数が答えになります。

これは、塗りつぶしの要領で深さ優先探索を行うことや、Union-Find を用いることで $O(N + M)$ または $O((N + M)\alpha(M))$ の計算時間で求めることができます。ここで、α はアッカーマン関数の逆関数ですが、非常に小さいため、ほとんど定数と同じと考えて構いません。

おまけ (Hard バージョン)

問題　入力に矛盾がある場合は -1 を出力せよ。

解説

各条件は

$$A_{X_i} + A_{Y_i} \equiv Z_i \pmod 2$$

と表すことができます。

さらに、$i = 1, 2, ..., N, j = 1, 2$ に対して、$p_{i,j} \in \{\text{true}, \text{false}\}$ を

$$p_{i,j} = \text{true} \Leftrightarrow i \text{ 番目のカードに } j \text{ が書かれている}$$

と定義します。すると、各条件は

- $Z_i \equiv 0 \pmod 2$ のとき

$$p_{X_i,0} = p_{Y_i,0}$$

$$p_{X_i,1} = p_{Y_i,1}$$

- $Z_i \equiv 1 \pmod 2$ のとき

$$p_{X_i,0} = p_{Y_i,1}$$

$$p_{X_i,1} = p_{Y_i,0}$$

です。このとき、$i = 1, 2, ..., N$ について

$$p_{i,0} \neq p_{i,1}$$

であることを確認すれば良いです。各 $p_{i,j}$ に対応する頂点を作った Union-Find を考えます。各 $p_{i_1,j_1} = p_{i_2,j_2}$ に対してこれらに対応する頂点同士を unite します。最後に、$p_{i,0} \neq p_{i,1}$ に対して、これらに対応する頂点同士が unite されていない (同じグループに属していない) かを調べることで矛盾があるか確認することができます。

F: XOR Matching

$K \geq 2^M$ のとき、明らかに解は存在しません。なぜなら 2^M 未満の整数同士の排他的論理和をとっても 2^M 以上になることはないからです。以下、そうでないときを考えます。

$M = 0$ のとき、a は一意であり $K = 0$ の解です。

$M = 1$ のとき、$K = 0$ では解が存在し $K = 1$ では存在しないことが全列挙などで分かります。

$M \geq 2$ のとき、0 以上 2^M 未満の任意の K について解が存在します。このことを具体的に a を構成することで示します。

K 以外の 0 以上 2^M 未満の整数を昇順に並べた数列を b とします。また b の逆順を c とします。このとき $b\ K\ c\ K$ と並べてできる数列を a とすればよいです。

というのも、まず任意の i $(0 \leq i < 2^M, i \neq K)$ について、この数列中の 2 つの i の間には、K が 1 回、それ以外の数が 2 回現れます。よって K 以外の数は打ち消し合って、題意の排他的論理和は K になります。

次に、この数列中の 2 つの K の間には、K 以外の 0 以上 2^M 未満の整数が 1 回ずつ現れます。これらの排他的論理和は、演算の性質を考えると $(0 \oplus 1 \oplus ... \oplus (2^M - 1)) \oplus K$ と書けます（$\oplus$ は排他的論理和演算を表します)。この式の括弧で囲った部分については、排他的論理和が $2^M - 1$ になるペアを偶数個作れるので 0 になります。つまり式の値は K になります。

よって K に関しても題意の排他的論理和は K であり、この数列は与えられた条件を満たすことが分かります。

ABC 127 解説

writer: tozangezan, drafear, satashun, potetisensei

2019 年 5 月 25 日

A: Ferris Wheel

if 文を用いて、高橋君が 13 歳以上なのか、6 歳以上 12 歳以下なのか、5 歳以下なのかに応じて、それぞれ答え (B 円、B/2 円、0 円) を出力しましょう。

C++ での実装例は以下の通りです (include 等は省いています)。

```
int main(){
  int a,b;scanf("%d%d",&a,&b);
  if(a<6)printf("0\n");
  else if(a<13)printf("%d\n",b/2);
  else printf("%d\n",b);
}
```

B: Algae

for ループを回すことによって漸化式の通りにシミュレーションしましょう。配列 `x[i]` を用意して `x[2001]`, ..., `x[2010]` を順に求め、出力してもよいですし、変数 x を随時更新し、更新するごとに出力してもよいです。

C++ での実装例は以下の通りです。

```
int main(){
  int r,K;
  int x;
  scanf("%d%d%d",&r,&K,&x);
  for(int i=0;i<10;i++){
    x=x*r-K;
    printf("%d\n",x);
  }
}
```

C: Prison

考え方 1

各 $i = 1, 2, ..., N$ について、i 番目の ID カードで全てのゲートを通過できるかを考えます。すると、全ての $j = 1, 2, ..., M$ について、 $L_j \leq i \leq R_j$ であるときに限り i 番目の ID カードで全てのゲートを通過できます。このような i の個数を数え上げることで答えを求めることができますが、このままでは時間計算量が $O(NM)$ となり間に合いません。

そこで、各 i について、全ての j について $L_j \leq i \leq R_j$ が成り立つかを高速に判定したいです。この条件は、

$$i \geq L_1, L_2, ..., L_M$$

かつ

$$i \leq R_1, R_2, ..., R_M$$

と表すことができます。すなわち、

$$L' = \max\{L_1, L_2, ..., L_M\}$$
$$R' = \min\{R_1, R_2, ..., R_M\}$$

とすると、各条件は $L' \leq i \leq R'$ と表すことができます。このような i の個数が答えになります。これは $L' \leq R'$ のとき $R' - L' + 1$、そうでないとき 0 です。時間計算量は $O(M)$ です。

考え方 2

全ゲートを通過できる ID カードの番号は区間 (連番) になります。また、全ゲートを通過できる ID カードは、$M - 1$ 番目までの全ゲートに通過できる ID カードのうち M 番目のゲートにも通過できるものです。そこで、i 番目までの全ゲートに通過できる ID カードの番号を $l_i, l_i + 1, ..., r_i$ とします。すると、l_{i+1}, r_{i+1} は l_i, r_i から計算できます。初期値も適当に、$l_0 = 1, r_0 = N$ や $l_1 = L_1, r_1 = R_1$ とすれば l_M, r_M を時間計算量 $O(M)$ で計算することができます。考え方 1 と同様に、$l_M \leq i \leq r_M$ であるような i の個数が答えになります。

D: Integer Cards

書き換える操作は好きな順番で行っても構いません。なぜなら、同じカードに対して 2 回以上書き換えるのは無駄だからです。また、C_i に B_i 枚まで書き換える操作は、C_i に 1 個まで書き換える操作を B_i 回行えると考えます。各 C_i を B_i 個ずつ並べた列を $D_1, D_2, ..., D_{\sum B_i}$ とします。すると、操作は「選んだことのない i を選んで 1 枚まで D_i に書き換える」と考えることができます。同じカードに対して 2 回以上書き換えるのは無駄なので、この操作は合計で N 回しか行いません。したがって、D_i の大きい方から N 個まで取り出した列を新しい D として考えても構いません。

X 枚書き換えるとします。すると、書き換え元は小さい方から X 枚書き換えるのが最適です。書き換え先は D_i の大きいものから順に X 個選ぶのが最適です。X を 1 ずらすことによって書き換え元と書き換え先はそれぞれ 1 つずつしか変化しないため、X を順に捜査することで $O(MlogM + NlogN)$ の計算時間で解くことができます。

実装では、D の大きい方から N 個まで取り出す部分が難しいかもしれません。これは、$(B_1, C_1), ..., (B_M, C_M)$ を C_i の大きい順にソートして[*1]配列 D が N 未満である間 C_i を B_i 個まで 1 個ずつ D に追加していく実装が楽でしょう。

最適な X を求めることもできます。A, D をソートして $A_1 \leq A_2 \leq ... \leq A_N$ かつ $D_1 \geq D_2 \geq ... \geq D_|D|$ とします。すると、前述のアルゴリズムでは X を $i-1$ から i にずらすことによって合計値は $D_i - A_i$ 変化しますが、$D_i - A_i$ は単調非増加なので $D_i - A_i \geq 0$ すなわち $D_i \geq A_i$ である間だけ操作を続けるのが最適です。すなわち、そのような i の最大値 (なければ 0) を X とするのが最適です。

別解

残す整数と書き換え先とする整数を合計で N 個選んで N 個の整数の合計を最大化することを考えます。$(X_1, Y_1), ..., (X_{N+M}, Y_{N+M})$ を $(1, A_1), ..., (1, A_N), (B_1, C_1), ..., (B_M, C_M)$ とすると、問題は「整数 Y_i を X_i 個まで選ぶことができ、合計 N 個の整数を選ぶときその和の最大値を求めてください」となります。これは、Y_i の大きい順に合計 N 個となるように選ぶのが最適です。この方法では $O((N+M)log(N+M))$ の時間計算量で求めることができます。

[*1] C++ 言語では (C_i, B_i) の pair を降順にソートする実装が簡単です。より読みやすいコードを書きたいなら構造体を用いると良いでしょう。

E: Cell Distance

$N \times M$ のマス目のうち K マス選ぶ時のマンハッタン距離の和を求めよという問題です。式は明らかに X と Y について独立なので、X の差の絶対値の和と Y の差の絶対値の和をそれぞれ求めることにします。

以下では X の差について考えます。ある 2 マスの組み合わせを固定したとき、これら以外から $K-2$ マス選ぶ場合全てに 1 度ずつこれらの差が寄与するので、この組を固定して考えると $\binom{N\times M-2}{K-2}$ 通りあります。さらに、X が同じ場合は差が 0 なので、X が異なると仮定すると、X の差の絶対値が d となるように 2 マス選ぶ方法は $(N-d)\times M^2$ 通りあります。これを全ての d に対して足し合わせると X についての答えが求まります。Y についても N と M を入れ替えると同様に解くことができ、時間計算量としては二項係数を求めるパートがボトルネックとなり $\mathcal{O}(NM)$ でこの問題が解けました。

F: Absolute Minima

N 個の関数が与えられており、$f(x) = \sum_{i=1}^{N}(|x - a_i| + b_i)$ となっている時を考えます。まず、$f(x) = \sum_{i=1}^{N}|x - a_i| + \sum_{i=1}^{N} b_i$ と分解でき、項 $\sum_{i=1}^{N} b_i$ は定数であるため、最小値を考える際には影響を与えない事が分かります。よって、$\sum_{i=1}^{N}|x - a_i|$ の最小値を与える x を求め、その最小値に $\sum_{i=1}^{N} b_i$ を足したものを出力すれば良いです。

以下、簡単のため、$\{a_i\}$ が昇順に整列されていると仮定し、$a_0 = -\infty, a_{N+1} = \infty$ とおくことにします。

$\sum_{i=1}^{N}|x - a_i|$ の最小値はどのようにして求めれば良いでしょうか？今、i 番目の絶対値関数 $|x - a_i|$ は $x < a_i$ において傾き -1 の 1 次関数、$x > a_i$ において傾き 1 の 1 次関数になっています。1 次関数同士を足し合わせても 1 次関数になる事を考えると、$i = 0, \cdots, N$ それぞれにおいて、区間 $[a_i, a_{i+1}]$ 内で $f(x)$ は 1 次関数と見做すことができます。

更に、x が小さいほど、N 個の関数の内、傾きが -1 であるような絶対値関数が増え、x が大きいほど傾きが 1 であるような絶対値関数が増えることを考えると、$f(x)$ の各区間 $[a_i, a_{i+1}]$ 内における 1 次関数の傾きは単調に増加します（これは凸関数の和は凸関数ということから考えても良いです）。よって、これは凸関数であり、傾きが 0 であるような区間（実際には 1 点のみからなる区間になっている可能性もあります）を求め、その区間内の点を 1 つ出力すれば良い、ということが分かります。

傾きが 0 であるような区間をどのようにして求めれば良いでしょうか？ここで、絶対値関数 $|x - a_i|$ において、傾きの変化点は $x = a_i$ であり、$x = a_i$ を “またいだ” 時に傾きが 2 だけ増えることに注目します。このことは、複数の絶対値関数を足し合わせても変わることはありません。すなわち、関数 $f(x)$ においても、$x = a_i$ を “またいだ” 際、傾きは 2 だけ増加します（ただし、同じ絶対値関数が複数含まれる時、すなわち $a_i = a_j$ となる i, j が存在するような場合には、含まれる個数分だけ増加することに注意してください）。

そして、区間 $[a_0 = -\infty, a_1]$ において、$f(x)$ の傾きは $-N$ であるため、大雑把に言えば、$\frac{N}{2}$ 個の変化点を “またぐ” と傾きが 0 となることが分かります。すなわち、昇順に並べた全ての変化点の内、ちょうど真ん中にある変化点（N が偶数の場合には真ん中にある 2 つの変化点の間）において $f(x)$ は傾きが 0 となり、最小値を取ります。

この計算方法を実装する際には、変化点をソートした状態で格納してくれるデータ構造（以下セットと呼ぶ）を 2 個用います（例えば C++ では multiset です）。片方のセットには昇順に並べた変化点の内、左半分を、もう片方のセットには右半分を全て管理させます。この時、1 つ目のセットの最右点と 2 つ目のセットの最左点の間の区間において $f(x)$ は傾き 0 となります。

更新クエリ (a_i, b_i) が来た際には、値 a_i を **2 個**、元と同じ条件（左半分を片方のセット、右半分をもう片方のセット）を満たすよう適切にセットに挿入すれば良いです（変化点をあえて 2 個挿入することによって、「1 つの変化点につき傾きが 1 変わる」と思うことが出来、シンプルになります）。

さて、最小値を与える点は分かったので、具体的な最小値を求めます。これは、

- 新しく挿入された点が、元々最小値を与えた区間に含まれるならば 0
- 区間から外れているならば、区間の端点との距離（どちらか短い方）

を元の最小値に足したものになります。

以上から、更新・求値両方を高々定数回のセットの操作のみによって実現する事ができ、$O(Q \log Q)$ で問題を解くことができます。また、凸性に基づいた値自体に対する三分探索や、Binary Indexed Tree を用いた

変化点の個数に関する二分探索を行う解法も存在します。

ABC 128 解説

writer: gazelle, tozangezan, satashun, drafear, potetisensei, yuma000

2019 年 5 月 26 日

A: Apple Pie

見通しをよくするため、あらかじめ林檎はすべて砕いて林檎の欠片にしておきます。こうすることで、合計で $3A+P$ 個の林檎の欠片を用意できます。これらの林檎の欠片を使って、作れるだけアップルパイを作ればよいです。求める最大数は、$3A+P$ が偶数のときは $\frac{3A+P}{2}$ 個、奇数のときは $\frac{3A+P-1}{2}$ 個となります。以下に C++ での実装例を示します。

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    int A, P;
    cin >> A >> P;
    cout << (3 * A + P) / 2 << endl;
    return 0;
}
```

B: Guidebook

多くのプログラミング言語では、string 型の配列を標準ライブラリのソート関数 (例: C++ の `std::sort`) によって辞書順に並べることができます。

問題文に合うようにレストランを並べるには、C++ でいう pair 型を利用すると簡単に実装できます。具体的には、`pair<pair<string,int>,int>` の配列を用意し、レストランの情報を格納します。first.first (1 番目のソート基準) に string 型で市名を入れ、first.second (2 番目のソート基準) に int 型で点数の -1 倍を入れ (点数が高い順に並べたいので)、second (3 番目のソート基準) にレストランの番号を入れます。

この pair 型をソートした後、 i 番目の要素の second には、本で i 番目に紹介するレストランの番号が格納されています。

pair 型がない言語などでは、まだ本で紹介されていないレストランのうち市名が辞書順で最も早く、その中で最も点数が高いレストランを順次ループを回して探していくことでも求めることができます。この場合は計算量は $O(N^2)$ となります。これでも間に合います。

C++ での実装例は以下の通りです (include 等は省いています)。

```
char in[120];
pair<pair<string,int>,int> p[110];
int main(){
  int a;scanf("%d",&a);
  for(int i=0;i<a;i++){
    int t;scanf("%s%d",in,&t);
    string tmp=in;
    p[i]=make_pair(make_pair(in,-t),i);
  }
  std::sort(p,p+a);
  for(int i=0;i<a;i++)printf("%d\n",p[i].second+1);
}
```

C: Switches

まず条件を無視すると、スイッチの on/off の組み合わせは 2^N 通りあります。条件を満たすスイッチの状態を直接数えるのは難しいので、この組み合わせを 1 つ固定して考えてみましょう。すると、それぞれの電球が点灯しているかどうかチェックし、全て点灯しているかどうか判定することで答えが求まります。この時間計算量は $\mathcal{O}(MN * 2^N)$ です。

なおスイッチの状態を固定する方法としては、DFS を用いる方法や、スイッチの状態を N bit の整数にエンコードする方法などがあります。

今回は全列挙することで解ける制約でしたが、mod 2 での連立方程式の解を数える問題と見なせるので、rank 等を計算することでより高速に求めることもできます。

解答例: https://atcoder.jp/contests/abc128/submissions/5640467

D: equeue

操作 C, D は最後に行っても構いません。すると、操作 C, D は等価で、「宝石を捨てる」操作になります。

操作 A を A 回、操作 B を B 回行うとします。$A+B \leq \min\{N, K\}$ です。このとき、左から A 個、右から B 個の宝石を手に入れ、捨てる操作は $K-(A+B)$ 回までできます。負の価値の宝石を捨てるとその絶対値分だけ合計価値が上がるので、価値が負でできるだけ絶対値が大きいものから $K-(A+B)$ 個まで捨てるのが最適です。価値が 0 や正の宝石を捨てる必要はありません。

A, B を全探索することで、$R = \min\{N, K\}$ として $O(R^3 logR)$ の時間計算量で解くことができます。

余談ですが、この探索を高速化して $O(R^2 logR)$ で解くこともできます[*1]。

[*1] ヒント: B を 1 ずらすことによって捨てる宝石は高々 2 個しか変わらないことを利用します

E: Roadwork

Q 人の人全てが整数時間に出発するので、時刻 $S_i - 0.5$ から $T_i - 0.5$ までの通行止めは時刻の区間 $[S_i, T_i)$ について通行止めされていると考えて差し支えありません (簡単のためです)。

i 番目の道路工事は、$[S_i, T_i)$ の間、座標 X_i を通行止めにします。したがって、逆算して考えると、この通行止めの影響を受ける可能性があるのは $[S_i - X_i, T_i - X_i)$ の間に座標 0 を出発した人だけです (それらの時刻以外に出発した人は、その通行止めに出くわすことは絶対にありません)。

よって、i 番目の人に対する答えは、その人が影響を受ける可能性がある道路工事を全て求め、それらの道路工事が通行止めにする座標の内、最も小さいものです。

Q 人それぞれについて、N 個の道路工事がその人に影響するかどうかを調べてしまうと、時間計算量が $O(NQ)$ となってしまい間に合いません。そこで、イベントソートを利用します。具体的には、「イベントを入れる配列」および「現時点で通行止めにされている座標を持つセット」を 1 つずつ用意し、以下の 2 種類のイベントを定義しておきます:

1. 追加イベント $(t, 1, x)$ - セットに x を追加する。
2. 削除イベント $(t, -1, x)$ - セットから x を削除する。

そして、N 個の道路工事全てについて、以下のようにしてイベントを配列へ追加します:

i 番目の道路工事については、

1. 追加イベント $(S_i - X_i, 1, X_i)$
2. 削除イベント $(T_i - X_i, -1, X_i)$

を配列に追加する。

配列に追加したイベントは t の値に従ってソートし、順番に処理します。i 番目の人に対して答えを求める際は、t の値が D_i 以下であるようなイベント全てを処理し終わったタイミングで、セットの最小値を調べれば良いです。

こうして、$O(N \log N)$ でイベントをソートし、各イベントについて $O(\log N)$ でセットを処理、各 Q 人について $O(\log N)$ で最小値を調べることができるので、全体で $O((N + Q) \log N)$ でこの問題を解くことができます。

F. Frog Jump

まず明らかに、溺れずに座標 $N-1$(以下ゴール) に辿り着く方が溺れるよりも最終点数が高いので、座標 -1 以下や N 以上に移動するような移動経路は考慮に入れる必要がありません。つまり、$0 < B < A <= N-1$ としてもよいです。

A と B の値を全部試して愚直にシミュレーションするやり方では $O(N^2 logN)$ かかってしまい、TLE してしまいます。色々な高速化の手法が存在すると思いますが、ここでは「最後はゴールに辿り着く」という特性を使った解法を説明します。

ゴールにたどり着くならば、それは奇数回の移動後です。ある A と B の値に対して、$2k+1$(k は非負整数) 回の移動でゴールにたどり着いたとき、その移動経路は、

$$0, A, A-B, 2A-B, 2A-2B, 3A-2B, ..., kA-(k-1)B, kA-kB, (k+1)A-kB$$

のようになります。このままでは分かりにくいので、$C = A - B(> 0)$ とおくと、

$$0, A, C, A+C, 2*C, A+2C, ..., A+(k-1)C, kC, A+kC$$

$A + kC = N-1$ なので、さらに変形すると、

$$0, (N-1)-kC, C, (N-1)-(k-1)C, 2C, (N-1)-(k-2)C, ..., N-1-C, kC, N-1$$

k と C さえ決めれば、経路は一意に定まります。その得点を f(k,C) とおきます。この時、

$$f(k+1, C) = f(k, C) + S_{N-1-kC} + S_{kC} (k >= 0)$$

が成り立つので、動的計画法を用いることにより、それぞれの k, C に対しての $f(k, C)$ は $O(1)$ で求まります。また、$kC < N-1$ より、考えるべき k, C の通り数は $O(NlogN)$ です。よって、$O(NlogN)$ で解けます。途中で溺れないために、同じ座標を二度通ってはいけないということに注意して下さい。

ABC 129 解説

satashun, yuma000, tozangezan, potetisensei, sheyasutaka

2019/06/09

A: Airplane

3 つの空港を順に巡る方法は 6 通りあるので、全て試しその最小値を求めればよいです。ただし、3 つのコストのうち 2 つの和の形をしていることを踏まえると、3 つの和から最大値を引くことでも答えを求めることができます。

実装例をソースコード 1 に示します。

Listing 1　実装例

```
#include <bits/stdc++.h>
using namespace std;

int main() {
  int P, Q, R;
  cin >> P >> Q >> R;
  cout << P + Q + R - max({P, Q, R}) << endl;
  return 0;
}
```

B: Balance

実際に $1 \leq T < N$ を全て試してみましょう。すると、それぞれのグループの和を実際に求めることで最小値を求めることができます。愚直に和を求めると時間計算量は $\mathcal{O}(N^2)$ で、正解することができます。しかし、先頭から連続する和と、全体からその和を引いたものの差の絶対値を求めていけばよいので、和を保持しながら先頭から見ていくことで $\mathcal{O}(N)$ でも解くことができます。

実装例をソースコード 1 に示します。

Listing 1　B 実装例

```
#include <bits/stdc++.h>
using namespace std;

int main() {
  int N;
  cin >> N;
  vector<int> a(N);
  int sum = 0;

  for (int i = 0; i < N; ++i) {
    cin >> a[i];
    sum += a[i];
  }

  int mini = sum;
  int prefix_sum = 0;
  for (int i = 0; i < N; ++i) {
    cin >> a[i];
    prefix_sum += a[i];
    mini = min(mini, abs(prefix_sum - (sum - prefix_sum)));
  }

  cout << mini << endl;
  return 0;
}
```

C. Typical Stairs(writer : yuma000)

まず、壊れた床がない場合 $(M = 0)$ について考察します。x 段目にたどり着くまでの移動方法の数を f_x とおきます。

$x \geq 2$ の時、x 段目にたどり着く方法としては、

- $x - 2$ 段目から一歩で二段上る。
- $x - 1$ 段目から一歩で一段上る。

の二通りがあります。つまり、$f_x = f_{x-1} + f_{x-2}$ が成り立ちます。これはフィボナッチ数の求め方の式と同等です。動的計画法 (DP) を使うことで、$O(N)$ で解くことができます。

ここまで考察できれば、壊れた床がある場合もこれを応用することで解くことができます。具体的には、f_{ai} には遷移しないように DP の遷移式を改造すれば良いです。

```
#include<bits/stdc++.h>
using namespace std;
const long long mod=1e9+7;

int main(){
    int N,M;
    cin>>N>>M;

    vector<int>oks(N+1,true);
    for(int i=0;i<M;++i){
        int a;cin>>a;
        oks[a]=false;
    }

    vector<long long int>dp(N+1);
    dp[0]=1;
    for(int now=0;now<N;++now){
        for(int next=now+1;next<=min(N,now+2);++next){
            if(oks[next]){
                dp[next]+=dp[now];
                dp[next]%=mod;
            }
        }
    }

    cout<<dp[N]<<endl;
    return 0;
}
```

D: Lamp

障害物の無いマス全てに対し、照らされるマスを for ループで数えようとすると、障害物が存在しないときに最悪計算量が $O(HW(H+W))$ となってしまい、これでは時間内に答えを求めることができません。

ただし、照らされるマスを数えるのにいくつかの補助的な前計算を活用することで、それぞれのマスに明かりを設置した時に照らされるマスの個数を定数時間で求められるようになり、こうすれば $O(HW)$ で答えを求めることができます。

まず、前計算として必要なものは、

- $L[i][j]$: i 行目 j 列目に明かりを置いた時にそこから左方向に照らされるマスの個数 (i 行目 j 列目のマスを含む)
- $R[i][j]$: i 行目 j 列目に明かりを置いた時にそこから右方向に照らされるマスの個数 (i 行目 j 列目のマスを含む)
- $D[i][j]$: i 行目 j 列目に明かりを置いた時にそこから下方向に照らされるマスの個数 (i 行目 j 列目のマスを含む)
- $U[i][j]$: i 行目 j 列目に明かりを置いた時にそこから上方向に照らされるマスの個数 (i 行目 j 列目のマスを含む)

の 4 つです。

例えば、$L[i][j]$ の要素は次のように計算できます。

- i 行目 j 列目に障害物がある時は、$L[i][j] = 0$。
- そうでないとき、$j = 1$ ならば、$L[i][j] = 1$。
- そうでないとき、$j > 1$ ならば、$L[i][j] = L[i][j-1] + 1$。

これらを 4 方向に対して計算した後、i 行目 j 列目のマスに明かりを置いた時に照らされるマスの総数は、$L[i][j] + R[i][j] + D[i][j] + U[i][j] - 3$ と求められます。(これら 4 つの値はどれも i 行目 j 列目のマスを含んでいるので、重複を消すために 3 を引いています。)

E: Sum Equals Xor

一般に、$A+B$ は A XOR B 以上になります。これらの値が一致する条件は、A, B を 2 進数表記した際、「どちらの数についても 1 であるような桁」が存在しないことです。逆にそのような桁が存在した時、$A+B$ では繰り上がりが発生し、XOR では発生しないため、$A+B$ の方が真に値が大きくなります (そしてその差を打ち消すことは出来ません)。

この考察を元に桁 DP をすることを考えます。

- $\mathrm{dp1}[k] :=$ A および B の値の上から k 桁目までを確定させ、その時点で既に $A+B$ が L 以下であることが分かっているような値組 (A, B) の個数
- $\mathrm{dp2}[k] :=$ A および B の値の上から k 桁目までを確定させ、その時点ではまだ $A+B$ が L 以下になるかどうか分かっていないような値組 (A, B) の個数

として定義します。この時、$\mathrm{dp*}[k-1]$ から $\mathrm{dp*}[k]$ を計算するには、k 桁目を 0 にするか 1 にするかの 2 通りの遷移を考慮します。k 桁目を 0 にする場合、A および B の k 桁目はどちらも 0 でなければなりません。k 桁目を 1 にする場合には、$(0,1)$ および $(1,0)$ の 2 通りの遷移が考えられます。この時、dp2 に関しては、k 桁目を確定させた際に $A+B$ が L を超えてしまわないよう、慎重に計算を行う必要があります。

以上のように、L を文字列だと見做して、$O(|L|)$ で解くことができます。

F. Takahashi's Basics in Education and Learning

問題を次のように言い換えます．

- 文字列 X と 整数 s を持つ．はじめ，$X = \text{"0"}, s = A$ である．
- 1 回の操作では，X の末尾に s が連結された後，s が B 増える．
- N 回操作した後の $X \bmod M$ はいくらか．

d 桁の要素の個数を C_d とすると，$\underbrace{99\ldots9}_{d}$ 以下の要素の個数から $\underbrace{99\ldots9}_{d-1}$ 以下の要素の個数を引くことで，C_d は容易に求まります．d 桁の要素 C_d 個を X の末尾に連結する操作は，$(X, s) \mapsto (X \times 10^d + s, s + B)$ という操作を C_d 回繰り返すことと同値です．そして，この操作は以下のように行列の積の形に落とし込めます．

$$(X, s, 1) \begin{pmatrix} 10^d & 0 & 0 \\ 1 & 1 & 0 \\ 0 & B & 1 \end{pmatrix} = (X \times 10^d + s, s + B, 1) \tag{1}$$

したがって，この 3×3 行列の C_d 乗が高速に求まればよく，繰り返し二乗法によって $O(\log C_d)$ で結果が求まります．

上の計算を $d = 1, 2, \ldots, 18$ についてこの順に行えば，最終的な $X \bmod M$ の値が高速に求まります．オーバーフローに十分注意してください．

ABC 130 解説

satashun, DEGwer, yuma000, yosupo, gazelle

2019/06/15

For International Readers: English editorial starts on page 8.

A: Rounding

問題文通りに、$X < A$ なら 0, $x >= A$ なら 10 を出力すれば良いです。
実装例をソースコード 1 に示します。

Listing 1　Rounding 実装例

```
#include <bits/stdc++.h>
using namespace std;

int main() {
  int X, A; cin >> X >> A;
  puts(X < A ? "0" : "10");
  return 0;
}
```

B: Bounding

実際に $D_1, D_2, ..., D_{N+1}$ を漸化式に従って生成し、それぞれの要素が X 以下かどうか判定すればよいです。時間計算量と空間計算量は $\mathcal{O}(N)$ です。

実装例をソースコード 2 に示します。

Listing 2　Bounding 実装例

```
#include <bits/stdc++.h>
using namespace std;

int main() {
  int N, X; cin >> N >> X;
  vector<int> D(N + 1);
  D[0] = 0;
  for (int i = 0; i < N; ++i) {
        int x; cin >> x;
    D[i + 1] = D[i] + x;
  }
  int ans = 0;
  for (int i = 0; i <= N; ++i) {
        if (D[i] <= X) {
      ans++;
    }
  }
  cout << ans << endl;
  return 0;
}
```

C: Rectangle Cutting

与えられた点 (x, y) と長方形の中心を両方通るような直線で長方形を切って二つの部分に分けるとき、大きくない方の面積は全体の半分の面積になり、これが最大です。また、中心を通らないような直線で切った場合、中心を含む方の部分の面積の方が大きくなります。

以上の考察から、与えられた点が長方形の中心でない場合は問題文の条件を満たす切り方は一意に定まります。そうでないなら、どの直線に沿って長方形を切っても、できる両方の部分の面積は等しくなるので、以下のようにして答えを求めることができます。

```
#include<stdio.h>
#include<vector>
#include<algorithm>
using namespace std;
typedef long long ll;
int main()
{
        int a, b, x, y;
        scanf("%d%d%d%d", &a, &b, &x, &y);
        printf("%lf %d\n", double(a)*double(b) / 2, x + x == a&&y + y == b);
}
```

D. Enough Arrays(writer : yuma000)

有り得る部分列の左端と右端を列挙して、それぞれの区間和が K 以上かを確かめる方法では、$O(N^2)$ かかってしまいます (累積和を用いることで、区間和は O(1) で求まります)。よって、もっと効率の良い解法を見つける必要があります。

$S(l,r) = \sum_r^l A_k$ としたとき、

- $S(a,b) < S(a,b+1)$
- $S(a,b) > S(a+1,b)$

が成り立ちます。

つまり、ここから

- ある l,r に対して、$S(l,r) >= K$ ならば、全ての $x(x >= r)$ に対して、$S(l,x) >= K$ である。

といえます。つまり、部分列の左端 l を固定したとき、$S(l,r) >= K$ を満たすような最小の r を見つけることができれば、部分列の左端が l の時に、条件を満たすものの個数を求めることができます。(具体的には、$N - r + 1$ です。)

具体的に r を求める方法としては

- 尺取り法 (O(N))
- 二分探索 (O(NlogN))

が挙げられるので、そのどちらかで実装すればよいです (個人的には尺取り法の方が計算量も少なく、実装も軽いと思うので、おすすめです)。

以下が、尺取り法のコードです。

```
#include<bits/stdc++.h>
using namespace std;

int main(){
    int N;long long int K;
    cin>>N>>K;
    vector<long long int>A(N);
    for(int i=0;i<N;++i){
        cin>>A[i];
    }
    long long int answer=0;
    long long int sum=0;

    int r=0;
    for(int l=0;l<N;++l){

        while(sum<K){
            if(r==N)break;
```

```
            else{
                sum+=A[r];
                r++;
            }
        }
        if(sum<K)break;
        answer+=N-r+1;
        sum-=A[l];
    }
    cout<<answer<<endl;
    return 0;
}
```

E: Common Subsequence

ある整数 k に対して、$S_{i_1} = T_{j_1}, ..., S_{i_k} = T_{j_k}$ となるように添字の集合 $1 \leq i_1 < i_2 < ... < i_k \leq N$ と $1 \leq j_1 < j_2 < ... < j_k \leq M$ を選ぶ場合の数を求める問題です。以下ではこの集合を前から決めていくことを考えます。まず、素直な解法として次のような dp が考えられます。dp[i][j] : S の i 文字目までと T の j 文字目までを考慮し、この 2 文字をペアにするときの場合の数とすると、$S_i = T_j$ であるとき dp[i][j] $= (\sum_{k=1}^{i-1} \sum_{l=1}^{j-1} dp$[k][l]$) + 1$ 、そうでないとき 0 として計算できますが、この時間計算量は $\mathcal{O}(N^2 * M^2)$ です。和を取る部分に着目すると、2 次元累積和の考え方を適応することができ、$\mathcal{O}(NM)$ に改善できます。具体的には、sum[i][j] $= \sum_{k=1}^{i} \sum_{l=1}^{j} dp$[k][l] と置いて、$sum$[i][j] $= sum$[i-1][j] + sum[i][j-1] − sum[i-1][j-1] + dp[i][j] として更新していくことができます。

F: Minimum Bounding Box

以下 $(x_{max} - x_{min}) \times (y_{max} - y_{min})$ を bounding box と呼ぶことにします。

点の移動開始後、x_{max} の変化量は減少、不変、増加と推移していきます（各区間が 0 に潰れる場合もあります)。変化量が変わるタイミングは、単調性を利用した二分探索などで求めることができます。同様のことは x_{min} や y_{max} や y_{min} でも成り立ちます。

この性質を利用すると、点の移動開始後の時間軸を各最大最小値の変化量が不変ないくつかの区間に分割できます。そして実は bounding box が最小値をとるタイミングは、そのような区間の端点のいずれかです。

このことを示します。

まず $dx = x_{max} - x_{min}$, $dy = y_{max} - y_{min}$ のように定めます。dx および dy が広義単調増加するような区間では、bounding box は増加します。よってそのような区間では、区間の開始地点で bounding box は最小値をとります。同様に dx および dy が広義単調減少するような区間では、区間の終了地点で bounding box は最小値をとります。

次に dx が狭義増加 dy が狭義減少である場合を考えます（逆の場合も同様です)。このときの bounding box の変化量は dx と dy の比率に左右されます。dx が dy と比べて一定以上小さい間は bounding box は増加します。dx が増加を続けると（あるいは最初から）減少の影響力のほうが強くなり、bounding box は減少します。結局このような区間では bounding box の値は上に 凸 になり、区間端点のいずれかで最小値をとります。

以上より、先述の事実が示せました。

ABC 130

satashun, DEGwer, yuma000, yosupo, gazelle

06/15/2019

A: Rounding

For this problem, implement the instructions in the problem statement. If $X < A$, print 0. Otherwise, print 10. An implementation is below.

Listing 1: Rounding

```
#include <bits/stdc++.h>
using namespace std;

int main() {
  int X, A; cin >> X >> A;
  puts(X < A ? "0" : "10");
  return 0;
}
```

B: Bounding

For this problem, we care about the location of the ball at each bounce. The location of the ball at the first bounce is always 0. The location of the ball at for subsequent bouce i is the location of the ball at the previous bounce plus $L[i]$, the length traveled from the previous bounce. This is the same thing as taking prefix sums. Now that we know the location of all the bounces, we can check each location and count the number of locations that are $<= X$. The runtime is $O(N)$.

Listing 2: Bounding

```
#include <bits/stdc++.h>
using namespace std;

int main() {
  int N, X; cin >> N >> X;
  vector<int> D(N + 1);
  D[0] = 0;
  for (int i = 0; i < N; ++i) {
        int x; cin >> x;
    D[i + 1] = D[i] + x;
  }
  int ans = 0;
  for (int i = 0; i <= N; ++i) {
        if (D[i] <= X) {
      ans++;
    }
  }
  cout << ans << endl;
  return 0;
}
```

C: Rectangle Cutting

A rectangle can be cut into two equal parts by any line that passes through the center of the rectangle. The points (x, y) and the center can form a line that cuts through the rectangle, so it is always possible to cut the rectangle exactly in half.

A line that doesn't pass through the center of the rectangle cannot split the rectangle in half. If (x, y) is the center of the rectangle, there are multiple ways to cut the rectangle in half passing through (x, y) because any line passing through the center works. If (x, y) is not the center, there is exacly one way to cut the rectangle in half because there is exactly one line that passes through both (x, y) and the center.

Listing 3: Rectangle Cutting

```
#include<stdio.h>
#include<vector>
#include<algorithm>
using namespace std;
typedef long long ll;
int main()
{
        int a, b, x, y;
        scanf("%d%d%d%d", &a, &b, &x, &y);
        printf("%lf %d\n", double(a)*double(b) / 2, x + x == a
            &&y + y == b);
}
```

D. Enough Arrays (writer: yuma000)

If the sum of elements in the range form l to r is $>= K$, then the sum of elements in the range from l to $r+1$ is also $>= K$ because all the elements are positive. This means that for each index l, if we can find the first index r such that $sum(a[l...r]) >= K$, then we can add $N - r + 1$ to the answer. $N - r + 1$ is the number of ranges that start with l. The ranges can end at any location from $r...N$.

There are two ways to find the first value r such that the range $sum(a[l...r]) >= K$ for each l.

The first way is binary search. First, we take the prefix sums of the array A. With prefix sums, we can query the sum of a range in $O(1)$. For every starting index l, we can binary search to find r. The runtime for this method is $O(NlogN)$.

The second method is two pointers. This is the method shown in the code below. If we know that r is the first index such that $sum(a[l...r]) >= K$ (this means $sum(a[l+1...r]) < K$), then the range from $l+1$ ro $r-1$ will not work (this means $sum(a[l+1...r-1]) < K$). The brute force solution is to try each possible r for every l. This runs in $O(N^2)$. We can speed this up by only trying a possible value of r if it is greater than or equal to the value of r for the previous l. This speeds the runtime up to $O(N)$ because we only check each possible value of r once total while going through all possible values of l.

Listing 4: Enough Arrays

```
#include<bits/stdc++.h>
using namespace std;

int main(){
    int N;long long int K;
    cin>>N>>K;
    vector<long long int>A(N);
    for(int i=0;i<N;++i){
        cin>>A[i];
    }
    long long int answer=0;
    long long int sum=0;

    int r=0;
    for(int l=0;l<N;++l){

```

```
17          while(sum<K){
18              if(r==N)break;
19              else{
20                  sum+=A[r];
21                  r++;
22              }
23          }
24          if(sum<K)break;
25          answer+=N-r+1;
26          sum-=A[l];
27      }
28      cout<<answer<<endl;
29      return 0;
30  }
```

E: Common Subsequence

Let dp[i][j] be the number of ways to create subsequences only using the first i elements of S and the first j elements of T such that the subsequences are the same and the ith element of S and the jth element of T are part of the subsequences. Basically, dp[i][j] is the answer to the problem if we only consider the first i elements of S and the first j elements of T and we have to use the ith element of S and the jth element of T. If S_i != T_j, then dp[i][j] = 0 because no subsequence will end by using the ith element of S and the jth element of T. If $S_i = T_j$ then dp[i][j] = $(\sum_{k=1}^{i-1} \sum_{l=1}^{j-1} dp[k][l]) + 1$ because the previous index of S can be any index <= i and the previous index of T can be any index <= j. As a base case, we can say dp[0][0] = 1. This represents the case were we don't take any elements. The runtime of this is $\mathcal{O}(N^2 * M^2)$, but we can speed this up by precomputing the sums. Let sum[i][j] = $\sum_{k=1}^{i} \sum_{l=1}^{j} dp[k][l]$. sum[i][j] is a 2D prefix sum of the dp array. sum[i][j] = sum[i-1][j]+sum[i][j-1]−sum[i-1][j-1]+dp[i][j]. With sum[i][j], we can know calculate each state, dp[i][j], in $\mathcal{O}(1)$. Since there are NM states, the runtime is $\mathcal{O}(NM)$.

F: Minimum Bounding Box

Let's call $(x_{max} - x_{min}) \times (y_{max} - y_{min})$ the bounding box. We want to minimize the area of the bounding box.

Consider the value of x_{max} as the points start moving. x_{max} will decrease for some amount of time (which may be 0), then it will stay the same for some amount of time (which may be 0), and then it will increase for some amount of time (which may be 0). We can binary search to determine when x_{max} stops decreasing and starting staying the same and when x_{max} stops staying the same and start increasing. We can repeat this process for x_{min}, y_{max}, and y_{min}.

We care about the interval of time when the values (for example x_{max}) stay the same because those will lead to the smallest bounding box. Consider time as a number line and place the intervals when the values are the same on the numberline. We have now divided the numberline into several sections. For each endpoint of a section, let $dx = x_{max} - x_{min}$, $dy = y_{max} - y_{min}$. Note that all 4 values may not be constant in every section.

In the sections where dx and dy are increasing, the bounding box will also increasing. In the sections where dx and dy are decreasing, the bounding box will also decrease. In both these cases, the area of the bounding box is minimized at an endpoint of the section.

We now have to consider the sections where dx is decreasing and dy is increasing. Consider the ratio that dx changes by and the ratio that dy changes by. If the ratio that dx decreases by is more than the ratio dy increases by, then the area of the bounding box decreases. The area of the bounding box is convex in these sections, so the minimum area will be at the endpoint of one of these sections. We can symetrically consider the sections where dx is increasing and dy is decreasing.

ABC 131 解説

sheyasutaka, yuma000, DEGwer, gazelle, satashun

2019 年 6 月 22 日

For International Readers: English editorial starts on page 9.

A: Security

正解までの道のりを細かく分割すると次の通りです．

1. 標準入力から文字列を受け取る．
2. 文字列の隣り合う文字同士を比較することで，文字列が「入力しずらい」かを判定する．
3. 判定結果に対応する文字列 (“Good” か “Bad”) を標準出力に出力する．

手順 2 (判定) については，正規表現を用いて簡潔に書くことも可能です．実装例を以下に示します．

Listing 1　C++ での実装例

```
#include <iostream>
#include <string>
using std::cin;
using std::cout;
using std::endl;
using std::string;

int main(void){
        string s;

        // 入力
        cin >> s;

        // 隣り合う文字を比較して判定
        bool isgood = true;
        if (s[0] == s[1]) isgood = false;
        if (s[1] == s[2]) isgood = false;
        if (s[2] == s[3]) isgood = false;

        // 出力
        if (isgood) {
                cout << "Good" << endl;
```

```
        } else {
                cout << "Bad" << endl;
        }

        return 0;
}
```

Listing 2　Ruby での実装例

```
# 入力
s = gets().chomp()

# 正規表現を使って判定・出力
if s =~ /(.)\1/ then
        puts "Bad"
else
        puts "Good"
end
```

Listing 3　Brainf*ck での実装例

```
>+++>>,<+++[->>,[-<->>+<]<[[-]<<->>]>>[-<<+>>]<<<]+<[[-]>++++++++++[-<++++++>]<.
[--<+++>]<--.+++>]>[++++++++[-<+++++++++>]<-.+++[-->+++<]>..[-<<+>>]<<----------
>>]]<<.
```

B. Bite Eating(writer : yuma000)

N 個のりんごのおいしさは、L,$L+1$,$L+2$,...,$L+N-1$ です。また、この中から、最もおいしさの絶対値の小さいりんごを食べるのが、最適なので、そのりんごを除いた全てのりんごのおいしさの和を出力すればいいことが分かります。

以下が、C++ のサンプルコードです。

```
#include<iostream>
using namespace std;

int main() {

    int N,A;
    cin>>N>>A;

    int L=A;
    int R=A+N-1;

    int eat;
    if(R<=0)eat=R;
    else if(L>=0)eat=L;
    else eat=0;

    int answer=(R+L)*(R-L+1)/2-eat;

    cout<<answer<<endl;

    return 0;

}
```

C: Anti-Division

条件を満たす B 以下の整数の個数から条件を満たす $A-1$ 以下の整数の個数を引いたものが答えです。以下、条件を満たす B 以下の整数の個数を求めることを考えましょう。同様にして、条件を満たす $A-1$ 以下の整数の個数も求めることができます。

B 以下の整数の個数 (B 個) から、B 以下の C の倍数の個数と B 以下の D の倍数の個数を引き、重複して引いたものを足し戻せばよいです。B 以下の C, D の倍数の個数はそれぞれ $B/C, B/D$ の小数点以下を切り捨てた数です。重複して引いた数は、C, D の最小公倍数を l として、B/l の小数点以下を切り捨てた数です。l は、C, D の最大公約数を g として CD/g と求められます。g は Euclid の互除法などを用いて求められるため、この問題を解くことができました。

D: Megalomania

直感的には，〆切の差し迫っている仕事から片付けるのがよさそうです．実際にこの直感は正しく，仕事を B_i の小さい順に順番に行うのが最適です．つまり，以下のことが言えます．

仕事 N 件の並べ替えであって，この順に行えば全ての仕事が〆切に間に合うようなものを「いい順序付け」と呼ぶことにすると，

答えが Yes となる条件　B_i の小さい順に仕事を行えば，それが「いい順序付け」となる．

証明　十分性は明らかなので, 必要性を示します．つまり,「いい順序付け」$(A_1, B_1), (A_2, B_2), \ldots, (A_N, B_N)$ があるとき，B_i の昇順に並べ替えてもまた「いい順序付け」となることを示します．

k 番目の仕事が〆切に間に合う条件は，それまでの仕事にかかる時間の合計が〆切の時刻を上回らないことです (つまり $A_1 + A_2 + \cdots, A_k \leq B_k$)．$B_k > B_{k+1}$ のとき，$(A_k, B_k), (A_{k+1}, B_{k+1})$ の順序を入れ替えても「いい順序付け」のままです．したがって，バブルソートの要領で B_i の昇順にソートしてもまた「いい順序付け」となります．

C++ や Python3 などの多くのプログラミング言語では，標準で用意されている関数などを用いることでソートをできます．実装例を以下に示します．

Listing 4　C++ での実装例

```
// include, using 略

int n;
vector<pair<int, int> > tasks;

int main(void){
        cin >> n;
        for (int i = 0; i < n; i++) {
                int a, b;
                cin >> a >> b;
                tasks.emplace_back(b, a); // 〆切(, 所要時間)
        }

        sort(tasks.begin(), tasks.end());
        int sum = 0;
        bool isyes = true;
        for (auto v : tasks) {
                sum += v.second;

                if (sum > v.first) {
                        isyes = false;
                        break;
                }
        }

```

```
26          cout << (isyes ? "Yes" : "No") << endl;
27
28          return 0;
29  }
```

E: Friendships

N 頂点の連結グラフには少なくとも $N-1$ 本の辺が存在します。これらの辺によって結ばれた頂点間の最短距離は 1 になるため、$K > \frac{N(N-1)}{2} - (N-1) = \frac{(N-1)(N-2)}{2}$ のとき解が存在しないのは明らかです。

逆に $K \leq \frac{(N-1)(N-2)}{2}$ のとき解が存在することを、条件を満たすグラフを実際に構成することで示します。

頂点 1 とそれ以外の任意の頂点とを結ぶ $N-1$ 本の辺だけが存在するグラフを考えます（このようなグラフをスターまたはうにと言います)。このグラフでは頂点 1 を除く任意の頂点間の最短距離は 2 であり、$K = \frac{(N-1)(N-2)}{2}$ のときの解になっています。ここで、このグラフの最短距離が 2 である頂点同士を結ぶ辺を追加すると、他の頂点間の最短距離に影響を与えずに、その最短距離を 1 にすることができます。

よって、この操作を $\frac{(N-1)(N-2)}{2} - K$ 回くり返すことで条件を満たすグラフを作ることができます。

F: Must Be Rectangular!

まず、このままでは考えにくいので問題を次のようにグラフの言葉で言い換えます。

頂点 $X_1, X_2, ..., X_{10^5}$ と 頂点 $Y_1, Y_2, ..., Y_{10^5}$ がある。点 (x, y) が存在するとき、頂点 X_x と Y_y の間に辺を張る。辺 $a-b, a-d, c-b, c-d$ のうちちょうど 3 辺が存在するような a, b, c, d を選び残りの 1 辺を追加する、という操作を何回行うことができるか。

初めに存在する点に対してグラフを構築すると、異なる連結成分を結ぶような辺は追加しようがないので、連結成分ごとに答えを求めて和を取ることで元の問題の答えが求まります。

ここで、1 つの連結成分については、2 部グラフの両側 (X の頂点と Y の頂点) の全ての組について (元から存在した辺を除き) 辺が追加されることが示せます。概略ですが、両側の頂点数が 2 以上の時は全域木を 1 つ取り、葉 (v) に繋がっているようなある辺 $v-u$ を選び、u 側の連結成分について全ての辺を張った後それらの辺を利用して v と結んでいき、どちらかの頂点数が 1 ならばその時点で連結なことにより全て結ばれています。

よって DFS などを用いて各連結成分について X と Y が何個含まれるか計算することでこの問題が解けました。

ABC 131

sheyasutaka, yuma000, DEGwer, gazelle, satashun

06/22/2019

A: Security

Here is a step-by-step instruction how to solve the problem:

1. Receive the string form the standard input.
2. Judge if the string is "hard to enter" by checking the adjacent characters.
3. Output a string corresponding to the judge result ("Good" or "Bad") to the stdout.

The second procedure (judgement) can be written concisely with regular expressions. The following are implementation examples.

Listing 1: An implementation example in C++

```
#include <iostream>
#include <string>
using std::cin;
using std::cout;
using std::endl;
using std::string;

int main(void){
        string s;

        // Input
        cin >> s;

        // Compare the adjacent characters and judge
        bool isgood = true;
```

```
        if (s[0] == s[1]) isgood = false;
        if (s[1] == s[2]) isgood = false;
        if (s[2] == s[3]) isgood = false;

        // Output
        if (isgood) {
                cout << "Good" << endl;
        } else {
                cout << "Bad" << endl;
        }

        return 0;
}
```

Listing 2: An Implementation example in Ruby

```
# Input
s = gets().chomp()

# Compare the adjacent characters and judge
if s =~ /(.)\1/ then
        puts "Bad"
else
        puts "Good"
end
```

Listing 3: An implementation example in Brainf*ck

```
>+++>>,<+++[->>,[-<->>+<]<[[-]<<->>]>>[-<<+>>]<<<]+<[[-]>++++++++++[-<++++++>]<

[--<+++>]<--.+++>]>[++++++++[-<++++++++>]<-.+++[-->+++<]>..[-<<+>>]<<----------

>>]]<<.
```

B. Bite Eating(writer : yuma000)

The flavor of the N apples are L,$L+1$,$L+2$,...,$L+N-1$. It is optimal to eat the apple whose absolute value of taste is minimum, so it appears that you have to print the sum of all the apples except for that apple.

The following is a sample code in C++.

```
#include<iostream>
using namespace std;

int main() {

    int N,A;
    cin>>N>>A;

    int L=A;
    int R=A+N-1;

    int eat;
    if(R<=0)eat=R;
    else if(L>=0)eat=L;
    else eat=0;

    int answer=(R+L)*(R-L+1)/2-eat;

    cout<<answer<<endl;

    return 0;

}
```

C: Anti-Division

The answer is the number of integers less than or equal to B that satisfies the condition, subtracted by the number of integers less than or equal to $A-1$ that satisfies the condition. Let's first think about how to find the number of integers less than or equal to B that satisfies the number; you can similarly get the number of integers less than or equal to $A-1$ that satisfies the condition.

The number is equal to the number of the integers no less than B ($= B$), subtracted by the number of multiples of C no less than B and the number of multiples of C no less than C, then added by the duplicate of them. The number of multiples of C, D is equal to B/C and B/D, rounded down, respectively. The number of duplicates are B/l rounded down, where l is the least common multiplier of C and D. l is equal to CD/g where g is the greatest common divisor of C, D. You can get g with Euclidean Algorithm, so here the problem could be solved.

D: Megalomania

Intuitively, it seems to be good to finish the job with earlier deadline formerly. Actually this intuition is right; it is optimal to finish the job in the increasing order of B_i. In other words,

Let "good permutation" be the permutation of N jobs, where you can complete all the jobs within the deadlines.

The condition where the answer is Yes If you do the work in the ascending order of B_i, that will be the "good permutation."

Proof The sufficiency is obvious. Now let's show the necessity: that is, if there exists a "good permutation" $(A_1, B_1), (A_2, B_2), \ldots, (A_N, B_N)$, after the permutation is sorted by the order of B_i, it still stays a "good order." You can finish the k-th job by the deadline if the time to take the k jobs so far is no less than the deadline of the job (that is, $A_1 + A_2 + \cdots, A_k \leq B_k$). If $B_k > B_{k+1}$, the permutation stays "good" if you swapped $(A_k, B_k), (A_{k+1}, B_{k+1})$. Therefore, you can sort the permutation in the manner of bubble sort so that the B_i will be a ascending order.

In most programming language like C++ and Python3, you can make use of standard library function for sorting. The implementation is below.

Listing 4: Implementation Example in C++

```
1 // include, using omitted
2
3 int n;
4 vector<pair<int, int> > tasks;
5
6 int main(void){
7         cin >> n;
8         for (int i = 0; i < n; i++) {
9                 int a, b;
10                cin >> a >> b;
11                tasks.emplace_back(b, a); // (deadline, time
                      required)
12        }
13
14        sort(tasks.begin(), tasks.end());
15        int sum = 0;
16        bool isyes = true;
```

```
17          for (auto v : tasks) {
18                  sum += v.second;
19
20                  if (sum > v.first) {
21                          isyes = false;
22                          break;
23                  }
24          }
25
26          cout << (isyes ? "Yes" : "No") << endl;
27
28          return 0;
29  }
```

E: Friendships

A N-vertices connected graph has at least $N-1$ edges. The shortest distance of these each pair is equal to 1, so obviously, it is clear that there doesn't exist a solution if $K > \frac{N(N-1)}{2} - (N-1) = \frac{(N-1)(N-2)}{2}$.

Conversely, now we're showing that there exist a solution if $K \leq \frac{(N-1)(N-2)}{2}$ by constructing a graph that satisfies the condition.

First, let's consider the graph where vertex 1 and the all other vertices are connected, respectively, with $N-1$ edges in total (such graph is called "star", or in Japanese, "sea urchin"). In this graph, the shortest distance of an arbitrary pair of vertices without vertex 1 is equal to 2, so this can be the answer for $K = \frac{(N-1)(N-2)}{2}$. Here, if you add an edge to the pair of vertices where the shortest distance is 2, it will change to 1, while the shortest distances of other pairs remain unchanged.

Therefore, you can obtain the satisfying graph by repeating this operation for $\frac{(N-1)(N-2)}{2} - K$ times.

F: Must Be Rectangular!

First, for simplification, let's rephrase the problem with graph theory terms.

There are vertices $X_1, X_2, ..., X_{10^5}$ and vertices $Y_1, Y_2, ..., Y_{10^5}$. If there exist a point (x, y), add an edge between vertex X_x and Y_y. How many times can you perform a operation to select a, b, c, d where there exist 3 edges out of 4 edges $a - b, a - d, c - b, c - d$ and add the rest to the graph?

Considering the graph is constructed from the initial points, you can never add edge between different connected components, so you can find the answer by calculating the answer for each connected component and then summing them up.

It can be proved that for each connected component, the edges will be added between all the pairs of vertices between two sides of the bipartite graph (between vertices X and vertices Y), except for the edges that originally existed. In a nutshell, if the number of vertices of both sides are more than or equals to 2, you can find a spanning tree, select an edge $v - u$ where v is a leaf, add all the edges for each vertex of the side containing u, then add edges to v using those edges; if the number of vertices of either side is 1, it is connected already.

Therefore, the problem could be solved by counting how many vertices X and Y there are for each connected component by using DFS or something.

ABC 132 解説

DEGwer, gazelle, potetisensei, tozangezan, yuma000

2019 年 6 月 29 日

For International Readers: English editorial starts on page 7.

A: Fifty-Fifty

以下のように、3 通りに場合分けして答えを求めればよいです。

```
#include<stdio.h>
#include<algorithm>
using namespace std;
int main()
{
        char s[10];
        scanf("%s", s);
        if(s[0] == s[1] && s[1] != s[2] && s[2] == s[3])printf("Yes\n");
        else if(s[0] == s[2] && s[2] != s[1] && s[1] == s[3])printf("Yes\n");
        else if(s[0] == s[3] && s[3] != s[1] && s[1] == s[2])printf("Yes\n");
        else printf("No\n");
}
```

また、以下のように文字列を並び替えてから判定を行っても良いです。

```
#include<stdio.h>
#include<algorithm>
using namespace std;
int main()
{
        char s[10];
        scanf("%s", s);
        sort(s, s + 4);
        printf((s[0] == s[1] && s[1] != s[2] && s[2] == s[3]) ? "Yes\n" : "No\n");
}
```

B: Ordinary Number

p_i が条件を満たすことと、$p_{i-1} < p_i < p_{i+1}$ または $p_{i-1} > p_i > p_{i+1}$ のいずれかの条件が成り立つことは同値です。よって、for ループを回してこの条件を満たす要素を数えていけばよいです。

以下は C++ での実装例です。

```cpp
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<int> p(n);
    for(int i = 0; i < n; i++) cin >> p[i];
    int ans = 0;
    for(int i = 1; i < n - 1; i++) {
        if((p[i - 1] < p[i]) && (p[i] < p[i + 1])) ans++;
        else if((p[i - 1] > p[i]) && (p[i] > p[i + 1])) ans++;
    }
    cout << ans << endl;
    return 0;
}
```

C. Dividing Problems(writer : yuma000)

結論から言うと、

- $N/2$ 番目に難しい問題が「ARC 用の問題」、$N/2-1$ 番目に難しい問題が「ABC 用の問題」となること
- 「ARC 用の問題」の数と「ABC 用の問題」の数が同じになること

は、同値であると言えます。よって、解法は以下のようになります。

1. 問題を難易度順に昇順でソートする。
2. $N/2$ 番目の要素から、$N/2-1$ 番目の要素を引いたものを出力する。

多くの言語にはソート用のライブラリが用意されているので、それを利用するのが良いでしょう。(C++ なら std::sort)

以下が、C++ のサンプルコードです。

```
#include<iostream>
#include<algorithm>
#include<vector>
using namespace std;

int main(){
    int N;cin>>N;
    vector<int>v(N);
    for(int i=0;i<N;++i){
        cin>>v[i];
    }
    sort(v.begin(),v.end());

    int answer=v[v.size()/2]-v[v.size()/2-1];

    cout<<answer<<endl;

    return 0;
}
```

D. Blue and Red Balls

K 個の青いボールを回収するのに高橋君がちょうど i 回操作をする必要があるというのは、K 個の青ボールが赤いボールによって i 箇所に区切られているということを意味します。

そこで、次のように組み合わせを考えていくことにしましょう。

1. まず、赤いボールを $N-K$ 個一列に並べます。
2. この中で赤いボールと赤いボールの間、左端、右端の中から i 箇所を選んでそこに青いボールを K 個置くことを考えます。これらの選び方は、 ${}_{N-K+1}C_i$ 通りあります。
3. それぞれの選び方について、青いボールをそれぞれの隙間に何個割り当てていくかを考えます。それぞれに 1 個以上割り当てる必要があるので、この決め方は ${}_{K-1}C_{i-1}$ 通りあります。(*)

よって、それぞれの i について、答えは ${}_{N-K+1}C_i \times {}_{K-1}C_{i-1}$ となります。コンビネーションの計算は、${}_nC_k = \frac{n!}{k!(n-k)!}$ であることを利用し、階乗、逆元、階乗の逆元を前計算しても良いですが、今回は $N \leq 2000$ なので、パスカルの三角形を上から求めていく要領で $C[i][j] =_i C_j$ を DP で ($C[i][j] = C[i-1][j]+C[i-1][j-1]$) 求めてもよいです。

(*) K 個のボールを並べて、 $K-1$ 個のボールとボールの間から $i-1$ 箇所選び、そこで切り分けると i 箇所に分かれる。各部分は最小 1 個のボールを含んでおり、ボールの総数は K 個になる。

E: Hopscotch Addict

言い換えると、この問題で求めたい値は「有向グラフ $G = (V, E)$ 上の、S から T への（単純パスとは限らない）路であって、路長が 3 の倍数であるようなものの内、最短の長さ（を 3 で割ったもの）」です。

これを計算するためには、以下のようにして構成したグラフ G' 上での最短経路問題を解けば良いです:

1. G' は頂点集合として $V' = \{v_t \mid v \in V, t = 0, 1, 2\}$ を持つ。すなわち、元のグラフ G の状態数を “3 倍化” する。
2. グラフ G が u から v への辺を持つ時、G' 上に 3 辺 $u_0 \to v_1, u_1 \to v_2, u_2 \to v_0$ を張る。

こうして構成された G' 上での S_0 から T_0 への最短経路（すなわち単純パスで良い）の長さは上記の求めたい値に一致します。したがって、結局これは重みなし有向グラフ上の最短経路問題に帰着され、G' の頂点数や辺数は G の高々定数倍であるため、BFS や DFS によって $O(N + M)$ で解くことが可能です。

F: Small Products

まず、DP$[i][x]$: 最後の整数が x であるような i 個の整数を並べて条件を満たすようにする場合の数 とした $O(NK)$ 状態の DP を考えます。このままでは計算量が大きいので、この DP を高速化することを考えます。

正整数 x, y が $\lfloor \frac{N}{x} \rfloor = \lfloor \frac{N}{y} \rfloor = t$ を満たすとします。このとき、DP$[i+1][x]$ と DP$[i+1][y]$ は同じ更新式 $\sum_{z=1}^{t}$ DP$[i][z]$ で求まるため、値は等しくなります。よって、このような x, y に対する DP の状態は同一視してよいことがわかります。

上記の規則で同一視できる状態をすべて同一視すると、DP の状態数は $O(\sqrt{N}K)$ になります。これは、$x \leq \sqrt{N}$ のときは x の値が、$x > \sqrt{N}$ のときは $\lfloor \frac{N}{x} \rfloor$ の値が $O(\sqrt{N})$ 個しかないことからわかります。適切に累積和を用いれば、この DP は $O(\sqrt{N}K)$ 時間で動くように実装できるため、この問題を解くことができました。

ABC 132

DEGwer, gazelle, potetisensei, tozangezan, yuma000

06/29/2019

A: Fifty-Fifty

You can solve this problem by splitting into those three cases.

```
#include<stdio.h>
#include<algorithm>
using namespace std;
int main()
{
        char s[10];
        scanf("%s", s);
        if(s[0] == s[1] && s[1] != s[2] && s[2] == s[3])printf
            ("Yes\n");
        else if(s[0] == s[2] && s[2] != s[1] && s[1] == s[3])
            printf("Yes\n");
        else if(s[0] == s[3] && s[3] != s[1] && s[1] == s[2])
            printf("Yes\n");
        else printf("No\n");
}
```

You can also do by sorting the characters.

```
#include<stdio.h>
#include<algorithm>
using namespace std;
int main()
{
        char s[10];
        scanf("%s", s);
        sort(s, s + 4);
        printf((s[0] == s[1] && s[1] != s[2] && s[2] == s[3])
            ? "Yes\n" : "No\n");
}
```

B: Ordinary Number

p_i satisfies the condition if and only if $p_{i-1} < p_i < p_{i+1}$ or $p_{i-1} > p_i > p_{i+1}$. So you can solve the problem by counting the element that satisfies the condition with for-loop.

The following is an implementation example in C++.

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<int> p(n);
    for(int i = 0; i < n; i++) cin >> p[i];
    int ans = 0;
    for(int i = 1; i < n - 1; i++) {
        if((p[i - 1] < p[i]) && (p[i] < p[i + 1])) ans++;
        else if((p[i - 1] > p[i]) && (p[i] > p[i + 1])) ans++;
    }
    cout << ans << endl;
        return 0;
}
```

C. Dividing Problems(writer : yuma000)

To come to the point, the following two conditions are equivalent:

- The $N/2$-th hardest problem is *for ARCs* and the $N/2-1$-th problem is *for ABCs*
- The number of problems for ARCs and the number of problems for ABCs are same

Therefore, you can solve it by following steps:

1. Sort the problems in a increasing order of difficulty.
2. Output the $N/2$-th element subtracted by the $N/2-1$-the element.

You may use sorting library, which is available in many languages. (e.g. std::sort in C++)

The following is an implementation example in C++.

```
#include<iostream>
#include<algorithm>
#include<vector>
using namespace std;

int main(){
    int N;cin>>N;
    vector<int>v(N);
    for(int i=0;i<N;++i){
        cin>>v[i];
    }
    sort(v.begin(),v.end());

    int answer=v[v.size()/2]-v[v.size()/2-1];

    cout<<answer<<endl;

    return 0;
}
```

D. Blue and Red Balls

When Takahashi needs exactly i moves to collect all K blue balls, K blue balls are separated by red balls into i segments.

Then let's think about the combination in the following steps:

1. First, arrange the $N-K$ red balls into a line.
2. Choose i spots out of the places between the red balls or the left or right endpoint, and place the K blue balls to them; there are ${}_{N-K+1}C_i$ ways to do so.
3. For each ways of choosing, decide how many balls to distribute to each spots. You have to put at least 1 ball into each spot, so there are ${}_{K-1}C_{i-1}$ ways to do so. (*)

Therefore, for each i, the answer will be ${}_{N-K+1}C_i \times {}_{K-1}C_{i-1}$. To calculate binomial coefficients, you can make use of the formula ${}_nC_k = \frac{n!}{k!(n-k)!}$ and calculate each factorials, inverse element, and factorials of inverse element; or alternatively, since $N \leq 2000$, you can calculate each $C[i][j] =_i C_j$ with DP ($C[i][j] = C[i-1][j] + C[i-1][j-1]$), in manner of Pascal's triangle.

(*) When you arrange K balls into a line and put bars into $i-1$ spots out of $K-1$ spots between the balls, the balls will be separated into i parts by them. Each segment contains at least 1 ball and the total number of balls is K.

E: Hopscotch Addict

To put the problem statement into other words, you have to find "the shotrtest length of path (divided by 3) among all the (not necessarily simple) path from S to T whose length is a multiple of 3."

This can be calculated by solving a shortest path problem on a graph G' constructed by the following procedure:

1. The vertex set of G' is $V' = \{v_t \mid v \in V, t = 0, 1, 2\}$. Frankly, this graph "triples" the number of states in original graph G.

2. Iff there exist a edge from u to v in G, there exist three edges $u_0 \to v_1, u_1 \to v_2, u_2 \to v_0$.

The length of shortest path from S_0 to T_0 on G' (which is always a simple path) is equal to the length of the path mentioned above.

Therefore, the problem is reduced into a shortest path problem on the unweighted graph, and the number of vertices and edges of G' is at most several times as many as that of original graph, so you can solve it by BFS or DFS in $O(N + M)$.

F: Small Products

Define Let DP$[i][x]$ as the number of ways to satisfy the constraints of the problem where are array is of size i and the last element is x. There are $O(K * N)$ states. Since there are too many states, we need to reduce the number of states by removing states with the same dp values.

Consider two positive integers x and y that satisfy $\lfloor \frac{N}{x} \rfloor = \lfloor \frac{N}{y} \rfloor = t$. Consider the formula for the values of DP$[i][x]$ and DP$[i][y]$. Both the formulas are $\sum_{z=1}^{t}$ DP$[i][z]$ so the two values are equal.

We can now change our DP state to DP$[i][t]$ and since there are $\sqrt{N}$ values of t, the number of states has reduced to $O(\sqrt{N}K)$. Note that while there are only $\sqrt{N}$ values of t, the values are not consecutive. Using prefix sums, we can calculate each state in $O(1)$. This makes the runtime $O(\sqrt{N}K)$.

ABC 133 解説

drafear, yosupo, potetisensei, yuma000, camypaper, evima

2019 年 7 月 7 日

For International Readers: English editorial starts on page 8.

A: T or T

電車を使った場合は $N \times A$ 円、タクシーを使った場合は B 円かかります。よって、$N \times A$ と B の小さい方を出力すれば良いことになります。これを C++ で実装した例を以下に示します。

```
#include <bits/stdc++.h>

using namespace std;

int main() {
  int N, A, B; cin >> N >> A >> B;
  int ans = min(N * A, B);
  cout << ans << endl;
}
```

B: Good Distance

各 i, j $(i < j)$ について $Y_{ij} = |X_{i1} - X_{j1}|^2 + ... + |X_{iD} - X_{jD}|^2$ を計算します。すると、$k^2 = Y_{ij}$ を満たす整数 k が存在するかを調べれば $\sqrt{Y_{ij}}$ が整数であることを判定できます。$k \leq Y_{ij}$ なので、これは、各 $k = 0, 1, 2, ..., Y_{ij}$ について $k^2 = Y_{ij}$ であるか調べれば十分です[*1]。C++ 言語での実装例を以下に示します。

```
#include <bits/stdc++.h>

using namespace std;

int main() {
int N, D; cin >> N >> D;
vector<vector<int>> X(N, vector<int>(D));
for (int i = 0; i < N; ++i) {
for (int j = 0; j < D; ++j) {
  cin >> X[i][j];
}
}
int ans = 0;
for (int i = 0; i < N; ++i) {
for (int j = i+1; j < N; ++j) {
  int norm = 0;
  for (int k = 0; k < D; ++k) {
    int diff = abs(X[i][k] - X[j][k]);
    norm += diff * diff;
  }
  // check whether norm = k for some k
  bool flag = false;
  for (int k = 0; k <= norm; ++k) {
    if (k * k == norm) {
      flag = true;
    }
  }
  if (flag) ++ans;
}
}
cout << ans << endl;
}
```

[*1] $k^2 \leq Y_{ij}$ であることを使えばより高速に判定できます

C: Remainder Minimization 2019

$R-L$ が非常に大きい時、全ての候補 (i, j) を調べてしまうと、$O((R-L)^2)$ の計算量がかかってしまい、間に合いません。よく考えてみると、2019 で割った余りが同じであるような 2 数 a, b について、$(a \times c) \bmod 2019$ と $(b \times c) \bmod 2019$ は必ず同じ値になります。よって、(i, j) として調べるべき候補は高々 $2019^2 = 4076361$ 通り程度しか存在せず、全て調べても十分間に合います。

また、「$O((R-L)^2)$ の全探索の途中で、解が 0 になった時点で探索を打ち切る」という方法でも解くことが出来ます。これは、R-L が 2019 以上の時、必ず割った余りが 0 となるような値が区間中に存在するため、2019^2 個以上の値を見ることはないからです。

D: Rain flows into dams

i 番目の山に降った雨の量 (答え) を X_i とします。すると、

$$A_1 + ... + A_N = X_1 + ... + X_N$$

が成り立ちます。この和を S とおきます。

また、便宜上 $X_{N+1} = X_1$ とすると

$$\frac{X_i + X_{i+1}}{2} = A_i$$

より

$$X_i + X_{i+1} = 2A_i$$

です。

したがって、

$$X_1 = S - (X_2 + ... + X_N) = S - 2(A_2 + A_4 + ... + A_{N-1})$$

と X_1 を求めることができます。同様に $X_2, X_3, ..., X_N$ を求めることができますが、$O(N^2)$ の時間はかけられないため、実装が少し大変です。

そこで、$X_i + X_{i+1} = 2A_i$ を利用し、$X_{i+1} = 2A_i - X_i$ の漸化式に従って $X_2, X_3, ..., X_N$ を前から順に計算していきます。

このアルゴリズムは時間計算量 $O(N)$ で動作し、十分間に合います。

E. Virus Tree 2(writer : yuma000)

今回の条件は以下のように言い換えられます。

- 全ての頂点に対して、以下の条件が成り立つ。
  - 頂点の色と、それと直接辺で繋がっている全ての頂点の色は、それぞれ異なる。

頂点 0 を根とする根付き木として考えます。すると、以下が成り立つことが分かります。

- ある頂点 x とその親が既に塗られているとき、x の子の数を c_x とすると、x の子の塗り分け方は ${}_{K-2}P_{c_x}$ 通りである。

よって、頂点 0 から DFS 的に色を塗っていくことで、木の塗り分け方の総数を求めることができます。頂点 0 には親がないことに注意して下さい。

以下が、サンプルコードです。

```
#include<iostream>
#include<vector>
using namespace std;

const long long int mod=1e9+7;


long long int dfs(int K,const vector<vector<int>>&graph, int now,int from) {

  int can_use_color_num;
  if (from==-1) {
    can_use_color_num=K-1;
  }
  else {
    can_use_color_num=K-2;
  }
  if (K<graph[now].size()) {
    return 0;
  }
  else {
    long long int case_num=1;
    for (auto e : graph[now]) {
      if(e==from)continue;

      case_num*=can_use_color_num;
      can_use_color_num--;
      case_num%=mod;
    }
    for (auto e : graph[now]) {
      if(e==from)continue;
```

```
31
32        case_num *=dfs(K,graph,e,now);
33        case_num %= mod;
34      }
35      return case_num;
36    }
37  }
38
39
40
41  int main() {
42    int N,K;cin>>N>>K;
43
44    vector<vector<int>>graph(N);
45    for (int i = 0; i < N - 1; ++i) {
46      int a,b;cin>>a>>b;a--;b--;
47
48      graph[a].push_back(b);
49      graph[b].push_back(a);
50    }
51
52    long long int answer=K*dfs(K,graph,0,-1);
53    answer%=mod;
54    cout<<answer<<endl;
55    return 0;
56  }
```

F: Colorful Tree

与えられた木を、頂点 1 を根とする根付き木とみなします。また、任意の頂点 w に対し、根から頂点 w までの距離を $dist_w$ とします。このとき、二頂点 $u.v$ 間の距離は u, v の 最近共通祖先 (英語版 Wikipedia の記事) を a として $dist_u + dist_v - 2dist_a$ と書けます。よって、各問 i に対して次の二つの部分問題を解けばよいことになります。

1. 二頂点 u_i, v_i の最近共通祖先 a_i を求める。
2. 色 x_i のすべての辺の長さが y_i に変更されたときの $dist_{u_i}, dist_{v_i}, dist_{a_i}$ を求める。

部分問題 1 は有名問題です。以下、よく知られている解法を紹介します。任意の頂点 w に対し、$par(w, 0)$ を w、$par(w, 1)$ を w の親、$par(w, 2)$ を w の親の親、... とします (根の親は根とします)。まず、事前にすべての頂点 w と整数 $i = 0, 1, \ldots, \lfloor \log_2 N \rfloor$ に対して $par(w, 2^i)$ を求めておきます。これは、$par(w, 2^{i+1}) = par(par(w, 2^i), 2^i)$ に注意すると $\mathrm{O}(N \log N)$ 時間で行えます。また、すべての頂点 w に対してその深さ $depth_w$ (根までの辺数) も事前に求めておきます。これらを元に、以下のようにして二頂点 u_i, v_i の最近共通祖先をそれぞれ $\mathrm{O}(\log N)$ 時間で求めることができます。

1. $a = u_i, b = v_i$ とする。
2. $depth_a < depth_b$ なら $b = par(b, depth_b - depth_a)$、そうでなければ $a = par(a, depth_a - depth_b)$ とする。
3. $a = b$ なら a を返す。
4. $i = \lfloor \log_2 N \rfloor, \lfloor \log_2 N \rfloor - 1, \ldots, 1, 0$ に対してこの順に次を行う: $par(a, 2^i) \neq par(b, 2^i)$ なら $a = par(a, 2^i), b = par(b, 2^i)$ とし、そうでなければ何もしない。
5. $par(a, 1)$ を返す。

部分問題 2 を解くには、まず事前にすべての頂点 w に対して辺の長さが変更される前の $dist_w$ を求めておきます。色 x_i のすべての辺の長さが y_i に変更されると、$dist_w$ は $num(x_i, w) \times y_i - sum(x_i, w)$ だけ増加します。ここで $num(x_i, w)$ は根と頂点 w の間に存在する色 x_i の辺の数、$sum(x_i, w)$ はそれらの辺の元の長さの総和です。

$num(x_i, w), sum(x_i, w)$ を求めるには、まず木の $N - 1$ 本の辺を DFS を行う際に通過する順に並べたリスト (オイラーツアー) を作ります。例えば、入力例 1 の木に対するオイラーツアーの例は次のようになります: $+e_1, +e_3, -e_3, +e_4, -e_4, -e_1, +e_2, -e_2$。ここで、$i$ 番目の辺を根から遠ざかる方向にたどる際に $+e_i$、根に近づく方向にたどる際に $-e_i$ と表記しています。このリストの e_i を i 番目の辺の元の長さで置き換えた数列を考えると、w と w の親を結ぶ辺が j 番目の辺であるとして、この数列の先頭から $+e_j$ に対応する要素までの和が根と w の間に存在する辺の元の長さの総和と一致します。この数列を順序を保ったまま辺の色ごとに分割し、事前に先頭から各要素までの和を計算しておけば、色 x_i と頂点 w が指定された際に $sum(x_i, w)$ を二分探索により $\mathrm{O}(\log N)$ 時間で求めることができます。$num(x_i, w)$ についても同様です。

以上の処理を行えば、合計 $\mathrm{O}((N + Q) \log N)$ 時間ですべての問いを処理することができます。

ABC 133 Editorial

drafear, yosupo, potetisensei, yuma000, camypaper, evima

July 7, 2019

A: T or T

If you use train it costs $N \times A$ yen, and if you use taxi it costs B yen. So you have to print the smaller value of either $N \times A$ or B. The following is an example implemented in C++.

```
#include <bits/stdc++.h>

using namespace std;

int main() {
  int N, A, B; cin >> N >> A >> B;
  int ans = min(N * A, B);
  cout << ans << endl;
}
```

B: Good Distance

For each i, j $(i < j)$, calculate $Y_{ij} = |X_{i1} - X_{j1}|^2 + ... + |X_{iD} - X_{jD}|^2$. Then you can determine if $\sqrt{Y_{ij}}$ is an integer by checking if there exists an integer k such that $k^2 = Y_{ij}$. Since $k \leq Y_{ij}$, it is sufficient if you judged if $k^2 = Y_{ij}$ for each $k = 0, 1, 2, ..., Y_{ij}$[*1]. The following is an implementation example in C++ language.

```
#include <bits/stdc++.h>

using namespace std;

int main() {
int N, D; cin >> N >> D;
vector<vector<int>> X(N, vector<int>(D));
for (int i = 0; i < N; ++i) {
for (int j = 0; j < D; ++j) {
  cin >> X[i][j];
}
}
int ans = 0;
for (int i = 0; i < N; ++i) {
for (int j = i+1; j < N; ++j) {
  int norm = 0;
  for (int k = 0; k < D; ++k) {
    int diff = abs(X[i][k] - X[j][k]);
    norm += diff * diff;
  }
  // check whether norm = k for some k
  bool flag = false;
  for (int k = 0; k <= norm; ++k) {
    if (k * k == norm) {
      flag = true;
    }
  }
  if (flag) ++ans;
}
}
cout << ans << endl;
}
```

[*1] If you used the fact that $k^2 \leq Y_{ij}$, you can judge more fast

C: Remainder Minimization 2019

When $R-L$ is very big, trying all candidates of (i, j) needs time complexity $O((R-L)^2$ and does not finish in time. In fact, if two integers a and b have same remainder when divided by 2019, $(a \times c)$ mod 2019 and $(b \times c)$ mod 2019 is always the same. Therefore, there are at most $2019^2 = 4076361$ candidats that need to be checked, so it will finish in time if you checked all the candidates.

Also, you can solve the problem with the strategy that "Abort the $O((R-L)^2)$ brute-forcing if one of the solution was 0." This works in time because, if R-L is more than or equal to 2019, there always exist a number whose remainder of 2019 is 0 in the segment so it inspects no more than 2019^2 pairs of values.

D: Rain flows into dams

Let X_i be the amount of rain that the i-th mountain received (the answer). Then it holds that

$$A_1 + ... + A_N = X_1 + ... + X_N$$

. Let S be this sum.

Also, let $X_{N+1} = X_1$ for simplicity, then

$$\frac{X_i + X_{i+1}}{2} = A_i$$

holds, so it holds that

$$X_i + X_{i+1} = 2A_i$$

.

Therefore, you can find X_1 with the equation

$$X_1 = S - (X_2 + ... + X_N) = S - 2(A_2 + A_4 + ... + A_{N-1})$$

. Similarly, you can calculate $X_2, X_3, ..., X_N$, but you cannot spend $O(N^2)$ time, so it's a bit troublesome to implement.

Since $X_i + X_{i+1} = 2A_i$, you can use the recurrence relation $X_{i+1} = 2A_i - X_i$ and calculate $X_2, X_3, ..., X_N$ one by one from the beginning.

This algorithm works in $O(N)$, so it's sufficiently fast.

E. Virus Tree 2(writer : yuma000)

This time, the condition can be rephrased as follows:

- For all vertices, the following condition holds:
  - The color of the vertex and the colors of the vertices directly connected to it are distinct.

Let's regard it as a rooted tree whose root is vertex 0. Then the following property can be found:

- If a vertex x and its parent is already painted, letting c_x be the number of children of x, the way of painting the children of x is ${}_{K-2}P_{c_x}$.

Therefore, by painting the vertices in DFS-like order from vertex 0, you can calculate the number of ways of painting the tree. Note that vertex 0 doesn't have parent.

The following is a sample code.

```
#include<iostream>
#include<vector>
using namespace std;

const long long int mod=1e9+7;


long long int dfs(int K,const vector<vector<int>>&graph, int now,int from) {

  int can_use_color_num;
  if (from==-1) {
    can_use_color_num=K-1;
  }
  else {
    can_use_color_num=K-2;
  }
  if (K<graph[now].size()) {
    return 0;
  }
  else {
    long long int case_num=1;
    for (auto e : graph[now]) {
      if(e==from)continue;

      case_num*=can_use_color_num;
      can_use_color_num--;
      case_num%=mod;
    }
```

```
29      for (auto e : graph[now]) {
30        if(e==from)continue;
31
32        case_num *=dfs(K,graph,e,now);
33        case_num %= mod;
34      }
35      return case_num;
36    }
37  }
38
39
40
41  int main() {
42    int N,K;cin>>N>>K;
43
44    vector<vector<int>>graph(N);
45    for (int i = 0; i < N - 1; ++i) {
46      int a,b;cin>>a>>b;a--;b--;
47
48      graph[a].push_back(b);
49      graph[b].push_back(a);
50    }
51
52    long long int answer=K*dfs(K,graph,0,-1);
53    answer%=mod;
54    cout<<answer<<endl;
55    return 0;
56  }
```

F: Colorful Tree

Let's regard the given tree as a rooted tree whose root is vertex 1. Also, for each vertex w, let $dist_w$ be the distance from the root to vertex w. Here, the distance between two vertices u, v can be written as $dist_u + dist_v - 2dist_a$, where a is the lowest common ancestor of vertices u, v. Therefore, each query i can be solved by solving the following two subproblems:

1. Find the least common ancestor of two vertices u_i, v_i.
2. Find $dist_{u_i}, dist_{v_i}$ and $dist_{a_i}$ when the length of every edge whose color is x_i is changed to y_i.

Subproblem 1 is a famous problem. The following is a well-known solution. For each vertex w, let $par(w, 0)$ be w, $par(w, 1)$ be the parent of w, $par(w, 2)$ be the parent of the parent of w, and so on (here we assume the parent of root to be the root itself). First, for each vertex w and integer $i = 0, 1, \ldots, \lfloor \log_2 N \rfloor$, precalculate $par(w, 2^i)$. By using the fact that $par(w, 2^{i+1}) = par(par(w, 2^i), 2^i)$, this can be calculated in $O(N \log N)$. Also, for each vertex w, precalculate its depth $depth_w$ (the number of edges to the root). By using them, you can find the least common ancestor of two vertices $u_i and v_i$ with the following steps:

1. Let $a = u_i, b = v_i$.
2. Let $b = par(b, depth_b - depth_a)$ if $depth_a < depth_b$, otherwise $a = par(a, depth_a - depth_b)$.
3. Return a if $a = b$.
4. For each $i = \lfloor \log_2 N \rfloor, \lfloor \log_2 N \rfloor - 1, \ldots, 1, 0$, do the following: let $a = par(a, 2^i), b = par(b, 2^i)$ if $par(a, 2^i) \neq par(b, 2^i)$, and otherwise do nothing.
5. Return $par(a, 1)$.

To solve the subproblem 2, first of all, for each vertex w, calculate $dist_w$ before the length of the edges are changed. After the length of every edge whose color is x_i is changed to y_i, $dist_w$ increases by $num(x_i, w) \times y_i - sum(x_i, w)$, where $num(x_i, w)$ is the number of edges whose color is x_i, and $sum(x_i, w)$ is the sum of the length of those edges.

To find $num(x_i, w), sum(x_i, w)$, first make a list of vertices in the order of DFS traversing (Euler tour). For example, an Euler tour of the tree in Sample Input 1 is like: $+e_1, +e_3, -e_3, +e_4, -e_4, -e_1, +e_2, -e_2$. Here, $+e_i$ denotes moving away from the root, and $-e_i$ denotes getting closer to the root, while passing through the i-th edge. If you substitute each e_i in this sequence with the length of the i-th edge, the sum of the elements of subsequence from the first element to the element corresponding to $+e_j$ is equal to the sum of the length of edges between the root and vertex w if the j-th edge connects two vertices, w and the parent of w. If you split this sequence into subsequences without changing the order so that each subsequence correspond to each color, and precalculate the sum from the beginning to each element, for each specified color x_i and vertex w, you can calculate $sum(x_i, w)$ in time complexity $O(\log N)$ by using binary search. Similarly you can also calculate $num(x_i, w)$.

With the operations above, you can answer to all the queries in $O((N + Q) \log N)$.

ABC 134 解説

yuma000, drafear, DEGwer, evima, gazelle, potetisensei

2019 年 7 月 20 日

For International Readers: English editorial starts on page 7.

A. Dodecagon(writer : yuma000)

$3 * r * r$ を出力すればよいです。以下が C++ のサンプルコードです。

```
#include<iostream>
using namespace std;

int main() {
        int R;cin>>R;

        int area=R * R * 3;

        cout<<area<<endl;

        return 0;
}
```

B: Golden Apple

1 人の監視員を配置すると連続した $2D+1$ 本のりんごの木を監視できます。前から順に詰めて配置する (例えば最初の監視員には $1, 2, ..., 2D+1$ 番目の木を監視させる) のが最適なので、答えは N を $2D+1$ で割った切り上げになります。一般的に、整数 A, B に対して $\frac{A}{B}$ の切り上げは $\frac{A+B-1}{B}$ の商と等しいです。したがって、今回の場合は $\frac{N+2D}{2D+1}$ の商が答えになります。C++ での実装例を以下に挙げます。

```cpp
#include <bits/stdc++.h>

using namespace std;

int main() {
  int N, D; cin >> N >> D;
  int ans = (N + D * 2) / (D * 2 + 1);
  cout << ans << endl;
}
```

C: Exception Handling

N が数千以下 (言語によっては数万以下) なら、各問で A_i 以外の $N-1$ 個の要素すべてに対してループを回して最大値を直接求めても実行制限時間の 2 秒に間に合います。しかし実際には N は最大で 20 万であり、この方針では C++ でも間に合う望みはありません。計算時間を削減する方針を以下に二つ示します。

方針 1: 場合分け

ほとんどの場合、問いの答えは N 個すべての要素のうちの最大の値 $A_{\max}$ です。唯一の例外は問いで取り除かれる値 A_i が $A_{\max}$ と等しい場合で、この場合の答えはすべての要素のうち 2 番目に大きい値 A_{second} (数列中に $A_{\max}$ が複数回現れる場合は $A_{\mathrm{second}} = A_{\max}$ とします) です。問いの処理を始める前に $A_{\max}$ と A_{second} をあらかじめ求めておけば、各問を直ちに処理できます。なお、A_{second} を最も簡単な実装で求める方法は、与えられた数列をコピーして言語の標準ライブラリでソートすることでしょう (やや「余計」な計算をすることになりますが十分高速です)。

方針 2: 両端から攻める

$j = 0, 1, \ldots, N-1$ に対し、$A_1, A_2, \ldots, A_j$ のうちの最大の値を left_j とします (ただし $\mathrm{left}_0 = 0$ とします)。$j \geq 1$ のとき $\mathrm{left}_j = \max(\mathrm{left}_{j-1}, A_j)$ [*1] であることに注意すると、これらの値は一周のループですべて求められます。また、$j = 2, \ldots, N+1$ に対し、$A_j, A_{j+1}, \ldots, A_N$ のうちの最大の値を right_j とします (ただし $\mathrm{right}_{N+1} = 0$ とします)。$j \leq N$ のとき $\mathrm{right}_j = \max(\mathrm{right}_{j+1}, A_j)$ であることに注意すると、これらの値も一周のループですべて求められます。問いの処理を始める前にこれらの値をあらかじめ求めておけば、各問 i の答えを $\max(\mathrm{left}_{i-1}, \mathrm{right}_{i+1})$ として直ちに求められます。

[*1] $\max(a, b)$ は a と b のうち大きい方 (より厳密には小さくない方) を表します

D: Preparing Boxes

大きい数が書かれた箱からボールを入れるかを決めていくことにします。こうすると、整数 i が書かれた箱にボールを入れるか決めるとき、i 以外の i の倍数が書かれた箱については、すでにボールを入れるかが決まっています。それらの箱に入ったボールの総和の偶奇が a_i と異なる場合は箱にボールを入れて、そうでないときはボールを入れないことにします。このようにしてボールを入れるかを決めていくと、与えられた条件をすべて満たすようにボールを入れることができます。このアルゴリズムを愚直に実装すると $O(\frac{N}{1} + \frac{N}{2} + ... + \frac{N}{N})$ 程度の計算量がかかります。これは一見 $O(N^2)$ に見えますが、丁寧に解析すると $O(NlogN)$ で抑えることができます。よって十分高速です。

E: Sequence Decomposing

結論から言うと、答えは「与えられた数列の、**広義**単調減少列の長さの最大値 L」です。これを証明します。

数列の後ろの要素から順に、どの色で塗るかを決めていくことにします。最も後ろの要素に関しては、何らかの色で塗る必要があり、かつ他の要素に色はついていないので、適当に 1 色で塗っておきます。

後ろから 2 番目以降の数 A_i に関しては、「各色 c で塗られている要素の内、現在の最小の値 B_c はいくつか」を考えます。まず、$B_c <= A_i$ ならば、A_i を c で塗る事は条件に反するため出来ません。したがって、$B_c > A_i$ であるような色 c を用いるか、新しい色を使って A_i を塗る必要があります。

この時、$B_c > A_i$ であるような色 c の内、B_c が最小であるような色で A_i を塗るのが最適です。理由は以下のようにして分かります: まず、簡単のため、B_c を昇順に並べた列を考えます。ある色 c で A_i を塗る時、B_c の値が A_i に更新されると捉えることが出来ます。この時、B_c が最小であるような色で A_i を塗った際に更新された列 $\{B'_c\}$ と、それ以外の色で塗った際に更新された列 $\{B''_c\}$ を（ソートした状態で）考えると、同じインデックスに並んでいる B'_i および B''_i は値が等しいか、$B'_i > B''_i$ が成立するかの 2 通りしか考えられません。したがって、A_{i-1} 以降を色で塗る際には、B' を利用したほうが B'' を利用するよりも、新しい色を追加しなければならない状況に真に陥りにくいといえます。

結局、ソートされた $\{B_c\}$ を保持し、更新していく行為は $O(N \log N)$ の LIS の計算方法と同等であるため、$O(N \log N)$ で求める事が出来ます。

余談: この問題は Dilworth の定理を用いて考えると、より抽象的に議論・正当性の理解が出来ます。Dilworth の定理は、「DAG 上の最小 path 被覆 (全ての頂点を適当な path に属するようにして分割する時の必要な path の最小本数) は、最大 antichain（頂点の集合であって、集合に属する任意の 2 頂点に関して、それらを結ぶような path が存在しない）の要素数と一致する」という内容の定理です。今、この問題においては、与えられた数列の数を頂点に対応させ、$i < j$ かつ $A_i < A_j$ なる 2 点に有向辺を貼ることにすると、求める答えは最小 path 被覆に一致します。したがって、これは最大 antichain を求めれば良く、このグラフにおける antichain が満たすべき条件は、集合に含まれる任意の 2 要素について、$i >= j$ または $A_i >= A_j$ が成立するというものです。これは結局、数列上の**広義**単調減少列と一致するため、この問題の解法の正当性を示すことが出来ます。

F: Permutation Oddness

簡単のため、問題を以下のように言い換えます。

> うさぎ 1, うさぎ 2, ..., うさぎ N と、かめ 1, かめ 2, ..., かめ N がいる。うさぎとかめのペアを N 組作る方法のうち、ペアになったうさぎの添字とかめの添字の差の和 (これを「奇妙さ」と呼ぶ) が K になるものはいくつあるか。

結論から言うと、以下のように状態を定義することで、解を動的計画法で求めることができます。

- $dp[i][j][k][l]$ = うさぎ 1, ..., うさぎ i, かめ 1, ..., かめ i までを見て、ペアにするのを保留にしている (かめ $i+1$ 以降とペアにする) うさぎの数がj、ペアにするのを保留にしている (うさぎi+1以降とペアにする) かめの数が k、確定している奇妙さが l であるときの場合の数。

ここで確定している奇妙さとは、たとえばうさぎ i をかめ $i+1$ 以降とペアにする場合、少なくとも 1 の奇妙さが確定する、といったことを意味します。ところで、ペアを保留にしているうさぎの数とかめの数は一致します。よって DP テーブルの二次元目と三次元目は 1 つにまとめることができます。このとき DP の遷移は以下のように書けます。

- $dp[i][j][k] = (2*j+1)*dp[i-1][j][k-2*j]+(j+1)*(j+1)*dp[i-1][j+1][k-2*j]+dp[i-1][j-1][k-2*j]$

ここで右辺の第 1 項は、うさぎ i とかめ i をペアにする場合および、うさぎ i とかめ i のいずれかを保留にしておいた動物とペアにする場合を表します (ペアを保留にしている動物の数だけ、確定している奇妙さが増えます)。右辺の第 2 項は、うさぎ i とかめ i の両方を保留にしておいた動物とペアにする場合を表します。そして右辺の第 3 項は、うさぎ i とかめ i のどちらもペアにするのを保留する場合を表しています。

この動的計画法は状態数が $O(N^4)$、遷移が $O(1)$ なので、全体で $O(N^4)$ の計算量で解を求めることができます。

ABC 134 Editorial

yuma000, drafear, DEGwer, evima, gazelle, potetisensei

July 20, 2019

A. Dodecagon(writer : yuma000)

You can print $3 * r * r$. The following is the C++ sample code.

```
#include<iostream>
using namespace std;

int main() {
        int R;cin>>R;

        int area=R * R * 3;

        cout<<area<<endl;

        return 0;
}
```

B: Golden Apple

If you deploy 1 inspector, $2D+1$ apple trees can be inspected. It is optimal to deploy from left to right without space, so the answer is N divided by $2D+1$, rounded up. In general, for any integers A, B, the rounded up $\frac{A}{B}$ is equal to the quotient of $\frac{A+B-1}{B}$. Therefore, the answer for this problem is $\frac{N+2D}{2D+1}$. The following is an implementation example in C++.

```cpp
#include <bits/stdc++.h>

using namespace std;

int main() {
  int N, D; cin >> N >> D;
  int ans = (N + D * 2) / (D * 2 + 1);
  cout << ans << endl;
}
```

C: Exception Handling

If N is less than around thousands (or around tens of thousands, dependening on languages), even if you tried to find the maximum value of $N-1$ elements except for A_i using loops for each query, it will meet the time limit of 2 seconds. But practically N is at most 200 thousand and this is not a good strategy. There are two ways to reduce the execution time:

Strategy 1: Case Splitting

In most case, the answer is the maximum value of all N elements, $A_{\max}$. The only exception is that the excluded value a_i is equal to $A_{\max}$, in which case the second largest value A_{second} (if $A_{\max}$ appears multiples times, let $A_{\text{second}} = A_{\max}$) is the answer. If you precalculate $A_{\max}$ and A_{second} before handling the queries, each queries can be immediately answered. The easiest implementation to find A_{second} would be copying the array and sort it with the language's standard library (it needs some "extra" computation but it's fast enough).

Strategy 2: Calculating from the Both Sides

For all $j = 0, 1, \ldots, N-1$, let left_j be the maximum value among $\text{left}_0 = 0$ (here, let $\text{left}_0 = 0$). Since $\text{left}_j = \max(\text{left}_{j-1}, A_j)$ [*1] when $j \geq 1$, those values can be obtained in a single loop. Also, for all $j = 2, \ldots, N+1$, let right_j be the maximum value among $A_j, A_{j+1}, \ldots, A_N$ (let $\text{right}_{N+1} = 0$). Since $\text{right}_j = \max(\text{right}_{j+1}, A_j)$ when $j \leq N$, these values can also be obtained in a single loop. By precalculating those values before hanlding the queries, you can respond to each query immediately with the answer $\max(\text{left}_{i-1}, \text{right}_{i+1})$.

[*1] $\max(a, b)$ represent the greater value of (more strictly, not less) value

D: Preparing Boxes

Let's decide whether to put the ball in a decreasing order of box's indices. This way, when you consider whether or not to put a ball into box i, the boxes whose index is multiple of i except for i itself are already confirmed whether to put balls. If the parity of the number of balls in such boxes are different than desired a_i, put a ball into the box; otherwise, don't put the box. If you determine the choices in this way, you can find the optimal way of putting balls that satisfies all the given conditions. A naive implementation of this algorithm needs about $O(\frac{N}{1} + \frac{N}{2} + ... + \frac{N}{N})$. At first glance it seems $O(N^2)$, but a careful analysis shows that it's at most $O(NlogN)$. So, it's fast enough.

E: Sequence Decomposing

To come to the point, the answer is "the maximum length L of **non-increasing** sequence of the given sequnce." Here is a proof.

Let's decide which color to paint the sequence, from back to front. The very last element has to be painted with some color and the other elements are not painted yet, so it can be painted with a arbitrary color.

For each of the other elements A_i, consider "among the elements painted with color c, what is the current minimum value B_c?" First, if $B_c <= A_i$, you can't paint A_i with c, since it violates the given conditions. So, you have to use color c such that $B_c > A_i$, or new color, to paint A_i.

In such cases, among all color c such that $B_c > A_i$, when painting A_i it is optimal to choose one such that B_c is minimum. The following is the reason: first, for simplicity, consider a sequence in which B_c are sorted in a increasing order. Here, if you compare an updated sequence $\{B'_c\}$ where A_i was painted with the color with minimum B_c, and another updated sequence $\{B'_c\}$ where it was painted by any other color (both sequence being sorted), the two elements at the same position B'_i and B''_i are either equal or $B'_i > B''_i$. Therefore, when painting A_{i-1} and from then on, it is strictly less likely that you will need to add a new color if you prefer B' to B''.

Therefore, keeping and updating sorted $\{B_c\}$ is equivalent to $O(N \log N)$ LIS finding algorithm, so it can be solved in $O(N \log N)$.

Just for aside: This problem's discussion and validity can be understood in more abstract manner by using Dilworth's theorem. Dilworth's theorem is that "on any DAG, the minimum path cover (which is the size of set of path on the graph so that any vertex on it belongs to exactly one of them) is equal to the size of its maximum antichain (a vertex set such that any pair of vertices in the set does not have a path between them)." For this problem, let the elements of the given sequence be corresponding to vertices, and add directed edge between a pair of vertices such that $i < j$ and $A_i < A_j$, the answer is equal to its minimum path cover. Therefore, you can solve it by finding the maximum antichain, and the condition that the antichain on this graph should hold is that, for any pair of vertices in the set, $i >= j$ or $A_i >= A_j$ holds. This is eventually equal to the **non-increasing** subsequence on the sequence, and here the validity of this solution has been shown.

F: Permutation Oddness

There are Rabbit 1, Rabbit 2, ..., Rabbit N, Turtle 1, Turtle 2, ... and Turtle N. How many ways are there to make N pairs of Rabbits and Turtles, so that the sum of the difference between the index of Rabbit and Turtle of each pair (which we call "oddness") is equal to K?

To come to the point, you can solve this problem using DP by defining the state as follows:

- $dp[i][j][k][l]$ = The way of making pairs, where you have seen Rabbit 1, ..., Rabbit i, Turtle 1, ... and Turtle i, where there are j Rabbits that aren't still paired (so that they will be paired with Turtles later than i), where there k Turtles that aren't still paired (so that they will be paired with Rabbits later than i), and where the confirmed oddness so far is l.

Here, "confirmed oddness so far" means that, for instance, if you decided to make a pair of Rabbit i and Turtle later than i, at least 1 more oddness is additionally confirmed. By the way, the number of Rabbits and Turtles that aren't still paired are the same. So the second and third dimensions can be merged into one. Here, the DP recurrence equation can be written as follows:

- $dp[i][j][k] = (2*j+1)*dp[i-1][j][k-2*j]+(j+1)*(j+1)*dp[i-1][j+1][k-2*j]+dp[i-1][j-1][k-2*j]$

Here, the first term of the rhs (right-hand side) represents that Rabbit i is paired with Turtle i, or either Rabbit i or Turtle i is paired with an animal that aren't still paired (in which case the confirmed oddness increases by the number of animals that aren't still paired). The second term of rhs represents that each of Rabbit i and Turtle i is paired with animal that aren't still paired. The third term of rhs represents that both Rabbit i and Turtle i were withheld from being paired.

This DP has $O(N^4)$ states and needs $O(1)$ for each transition, so this can be solved in time complexity of $O(N^4)$.

ABC 135 解説

drafear, evima, gazelle, sheyasutaka, sigma425

2019 年 7 月 27 日

For International Readers: English editorial starts on page 7.

A: Harmony

A と B の偶奇が異なるとき、どのような整数 K についても $|A-K|$ と $|B-K|$ の偶奇は異なります。したがってこの場合、答えは IMPOSSIBLE です。

次に A と B の偶奇が等しいときを考えます。このとき $K = \frac{A+B}{2}$ とおくと K は整数です。また $|A-K| = |\frac{A-B}{2}|$, $|B-K| = |\frac{B-A}{2}|$ となり、絶対値の定義より両者は等しいことが言えます。よってこれが答えです。

C++ でのコード例を示します。

Listing 1　C++ による実装例

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
  long long A, B;
  cin >> A >> B;
  if(A % 2 != B % 2) {
    cout << "IMPOSSIBLE" << endl;
  } else {
    cout << (A + B) / 2 << endl;
  }
  return 0;
}
```

B: 0 or 1 Swap

$p_i \neq i$ であるような要素 p_i の数を k とします。結論から言うと $k \leq 2$ であるときに限り 1 回以下のスワップで p を昇順にすることが可能です。$k = 0$ のとき、最初から p が昇順であることを意味します。$k = 1$ になることはありません。$k = 2$ のとき、$p_i \neq i$ である 2 つの要素を 1 回スワップすることで p を昇順にできます。1 回のスワップで k の値はたかだか 2 しか減らすことができないため、$k \geq 3$ のとき 1 回以下のスワップで p を昇順にすることはできません。

与えられた順列の k の値は線形時間で求めることができます。よってこの問題を $O(N)$ で解くことができました。ちなみに問題をより一般化した「M $(M \leq N)$ 回以下のスワップで p を昇順にできるか？」という問題も $O(N)$ で解くことができます。

Listing 2　C++ による実装例

```cpp
#include<bits/stdc++.h>
using namespace std;
#define rep(i, n) for(int i = 0; i < (int)n; i++)

int main() {
    int n;
    cin >> n;
    vector<int> p(n);
    rep(i, n) cin >> p[i];
    int k = 0;
    rep(i, n) if(p[i] != i + 1) k++;
    if(k <= 2) cout << "YES" << endl;
    else cout << "NO" << endl;
    return 0;
}
```

C: City Savers

1 番目の街は 1 番目の勇者に救ってもらうしかありません。よって、1 番目の勇者は 1 番目の街のモンスターを全力で $\min(A_1, B_1)$ 体倒し、残った力で 2 番目の街のモンスターを全力で $\min(A_2, B_1 - \min(A_1, B_1))$ 体倒すのが最適です。すると、2 番目の街に残ったモンスターは 2 番目の勇者でしか倒すことができません。これを繰り返すことで答えを求めることができます。

正確に述べると、f_{ij} を勇者 i が街 j のモンスターを倒す数とします。すると、合計で

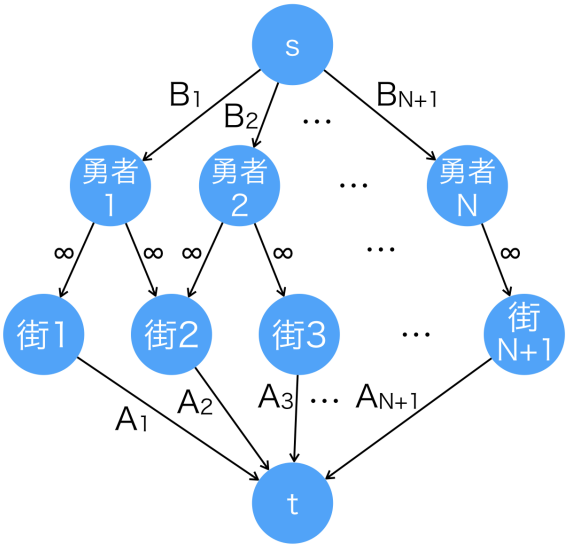
$$\sum_{i=1}^{N}(f_{ii} + f_{i(i+1)})$$

体のモンスターを倒すことになります。これを最大化する方法として上記の方法があり、この方法に従って計算すると、

$$\begin{aligned} f_{11} &= \min(A_1, B_1) \\ f_{i(i+1)} &= \min(A_{i+1}, B_i - f_{ii}) \quad (1 \leq i \leq N) \\ f_{ii} &= \min(A_i - f_{(i-1)i}, B_i) \quad (2 \leq i \leq N) \end{aligned}$$

などによって時間計算量 $O(N)$ で計算できます。

正当性、すなわちこれが最大になることの証明は次のとおりです。まず、この問題は次のような最大流問題に帰着させることができます (s が source、t が sink、辺の重みが容量です)。



勇者 i の頂点から街 j の頂点に流れる流量を上で計算できる f_{ij} とすると、確かに条件を満たしています。さらに、残余グラフにおいて増大道が存在するとすれば作り方に矛盾するため、増大道は存在しません。よって、これが最大です。

ところで、この問題を最大流問題に帰着させて解くことができそうに見えますが、Dinic のアルゴリズムなどでは流す優先順位をうまく設定しないと最悪時間計算量が $O(N^2)$ となり、間に合いません。

D: Digits Parade

次の漸化式を立てます.

$$dp[i][j] := \text{先頭 } i \text{ 文字として考えられるもののうち, 13 で割ったあまりが } j \text{ であるものの数}$$

このとき i の昇順に dp テーブルを見ると, $i-1$ 文字目までを 13 で割ったあまり ($dp[i-1][j]$ の j) と $s[i]$ としてあり得る数字を全て試すことで $dp[i][0] \sim dp[i][12]$ の値がわかります.

以下に解答例を示します.

Listing 3　C++ での実装例

```
static const ull MOD = 1000000007LL;

int n;
char s[100005];
uint64_t dp[100005][13];

uint64_t solve () {
  int i, j, ki;
  uint64_t res = 0;

  scanf("%s", s);
  n = strlen(s);

  dp[0][0] = 1;
  for (i = 0; i < n; i++) {
    int c;
    if (s[i] == '?') c = -1;
    else c = s[i] - '0';

    for (j = 0; j < 10; j++) {
      if (c != -1 && c != j) continue;
      for (ki = 0; ki < 13; ki++) {
        dp[i + 1][(ki * 10 + j) % 13] += dp[i][ki];
      }
    }
    for (j = 0; j < 13; j++) dp[i + 1][j] %= MOD;
  }
  res = dp[n][5];

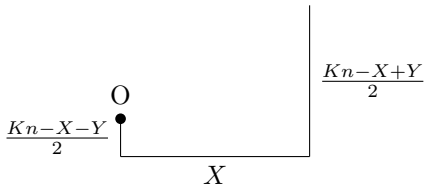
  printf("%llu\n", res);
  return 0;
}
```

E: Golf

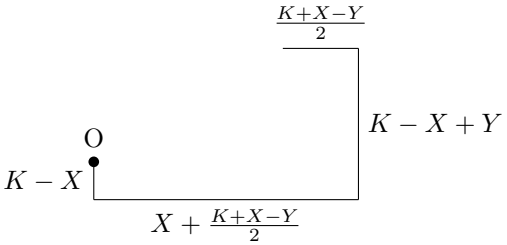
話を簡単にするため，$X \geq Y \geq 0$ としてよいです (適当に変換することで帰着できます)．n をスコアの最小値とします．

K が偶数，$X+Y$ が奇数のときは不可能です．逆に，それ以外のケースでは常に，以下のどちらかの形式で構築できます．

タイプ A:



タイプ B (常に $n=3$):



$c := (K - (X+Y \bmod K)) \bmod K$ とします．$X+Y<K$ かつ $X+Y$ が奇数のときタイプ B が，それ以外のときは次のように n の値を決めることでタイプ A が適用できます．

- $X+Y<K$ かつ $X+Y$ が偶数のとき，$n=2$.
- $X+Y \geq K$ かつ c が偶数のとき，$n = \lceil \frac{X+Y}{K} \rceil$.
- $X+Y \geq K$ かつ c が奇数のとき，$n = \lceil \frac{X+Y}{K} \rceil + 1$.

これらの構築方法が正しいことの証明は容易です．

F: Strings of Eternity

問題文の注記では文字列の先頭の文字を 1 文字目としていますが、ここでは 0 文字目とします。問題の内容を直感的に述べると、s を無限個連結して得られる文字列 s' に t は最大で何連続で現れるか、となります。

ここで、0 以上 $|s|-1$ 以下のすべての整数 i に対して、s' の i 文字目から $i+|t|-1$ 文字目までの部分文字列が t と一致するか否か (この真偽値を match_i とします) が判明しているとします。すると、s' の周期性を利用してシミュレーションを行うことで $\mathrm{O}(|s|)$ 時間で答えを求めることができます。グラフの言葉で表現すると、次のような有向グラフの最長パスの長さが答えとなります。

- $0, 1, \ldots, |s|-1$ の番号がついた $|s|$ 個の頂点を持つ。
- $\mathrm{match}_i = \mathrm{true}$ のとき有向辺 $i \to (i+|t|) \bmod |s|$ が存在する。その他に辺はない。

残るは $\mathrm{match}_0, \ldots, \mathrm{match}_{|s|-1}$ を時間内にすべて求めることです。もちろん、ナイーブに s' と t を比較すると最悪の場合に $\mathrm{O}((|s|+|t|)\times|t|)$ 時間かかってしまい間に合いません。[*1] しかしこれは古典的な問題であり、この問題を高速に解くためのアルゴリズムが多く考案されています。線形時間で $\mathrm{match}_0, \ldots, \mathrm{match}_{|s|-1}$ をすべて求めるアルゴリズムを二つ挙げます。

- KMP 法 (Wikipedia の記事へのリンク)
- Z algorithm (GeeksforGeeks の記事へのリンク)

この他に文字列を非十進表記の数値とみなすローリングハッシュを用いる方法もありますが、剰余の計算を省くために数値を 2^{64} で割った余りを用いると衝突するような入力を用意してありますのでご注意ください。

[*1] 入力がランダムな文字列であればナイーブに比較しても大抵の場合すぐにマッチに失敗するため計算量の期待値は線形ですが、$s = \texttt{aaa...aa}, t = \texttt{aaa...ab}$ のような繰り返しの多い文字列ではなかなかマッチに失敗しません

ABC 136 Editorial

drafear, evima, gazelle, sheyasutaka, sigma425

July 27, 2019

A: Harmony

If the parities of A and B are differemt, For any integer K, the parities of $|A-K|$ and $|B-K|$ are different. In such cases, the answer is IMPOSSIBLE.

Next, let's think about the cases when the parities of A and B are the same. In such cases, $K = \frac{A+B}{2}$, then K is an integer. Also, $|A-K| = |\frac{A-B}{2}|$ and $|B-K| = |\frac{B-A}{2}|$ holds, so these are the equals, according to the definition of absolute value. Therefore, this is the answer.

The following is an example code in C++.

Listing 1 An implementation example in C++

```
#include <bits/stdc++.h>
using namespace std;
int main() {
  long long A, B;
  cin >> A >> B;
  if(A % 2 != B % 2) {
    cout << "IMPOSSIBLE" << endl;
  } else {
    cout << (A + B) / 2 << endl;
  }
  return 0;
}
```

B: 0 or 1 Swap

Let k be the number of elements p_i such that $p_i \neq i$. To come to the point, you can sort p in ascending order by performing at most 1 operation of swap only if $k \leq 2$. If $k = 0$, it means that p is already sorted in ascending order. It is impossible that $k = 1$. If $k = 2$, you can swap the two elements such that $p_i \neq i$ only one time so that p is sorted in ascending order. In one swap, the value of K can be decreased by at most 2, so if $k \leq 3$, you cannot sort p in ascending order by performing at most one swap.

The value of k can be obtained in linear time. Here the problem was solved in $O(N)$. By the way, the problem that "can p be sorted in ascending order by performing at most M ($M \leq N$) swaps?" can also be solved in $O(N)$ time.

Listing 2 An implementation example in C++

```cpp
#include<bits/stdc++.h>
using namespace std;
#define rep(i, n) for(int i = 0; i < (int)n; i++)

int main() {
    int n;
    cin >> n;
    vector<int> p(n);
    rep(i, n) cin >> p[i];
    int k = 0;
    rep(i, n) if(p[i] != i + 1) k++;
    if(k <= 2) cout << "YES" << endl;
    else cout << "NO" << endl;
    return 0;
}
```

C: City Savers

Only the first hero can save the first town. So, it is optimal that the first hero do the best to defeat $\min(A_1, B_1)$ monsters in the first town, and then do the best to defeat $\min(A_2, B_1 - \min(A_1, B_1))$ monsters in the second town. Then, the monsters remaining in the second town can only be defeated by the second hero. You can find the answer repeating this procedure.

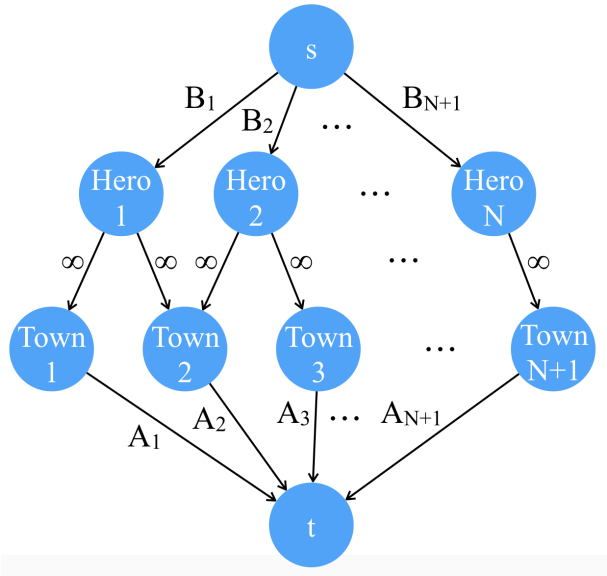
To be precise, let f_{ij} be the number of the monsters in the town j that are defeated by the hero i. Then, in total

$$\sum_{i=1}^{N}(f_{ii} + f_{i(i+1)})$$

monsters would be defeated. One of the way to maximize this is the procedures mentioned above, and with the procedures, it can be calculated in time complexity $O(N)$ with the steps like:

$$\begin{aligned} f_{11} &= \min(A_1, B_1) \\ f_{i(i+1)} &= \min(A_{i+1}, B_i - f_{ii}) \quad (1 \leq i \leq N) \\ f_{ii} &= \min(A_i - f_{(i-1)i}, B_i) \quad (2 \leq i \leq N). \end{aligned}$$

The validity that the sum mentioned above is the maximum can be proved like this: First, this problem can be attributed to the following maximum flow problem (here, s denotes source, t denotes sink and the weight of edges are the capacities).



If the flow from the vertex of hero i to the vertex of town j is f_{ij}, it meets the conditions. Moreover, if there exists a flow augmenting path in the residual network, it contradicts to the way of construction, for there doesn't exist a flow augmenting path. Therefore, this is the maximum.

By the way, it seems that this problem can be solved by attributing to a maximum flow problem, but

if you use Dinic's algorithm etc., the worst time complexity will be $O(N^2)$ and it will not finish in time, if you don't configure a proper priorities.

D: Digits Parade

You can build the following relation:

$$dp[i][j] := \mathrm{The number of all the possible first } i \mathrm{ digits such that remainder of 13 is } j$$

If you see the dp table in an increasing order of i, by trying all the possible reaminder by 13 of the first $i-1$ digits (j of $dp[i-1][j]$) and $s[i]$, you can determine the values of $dp[i][0] \sim dp[i][12]$.

The following is a sample answer.

Listing 3 An implementation example in C++

```
static const ull MOD = 1000000007LL;

int n;
char s[100005];
uint64_t dp[100005][13];

uint64_t solve () {
  int i, j, ki;
  uint64_t res = 0;

  scanf("%s", s);
  n = strlen(s);

  dp[0][0] = 1;
  for (i = 0; i < n; i++) {
    int c;
    if (s[i] == '?') c = -1;
    else c = s[i] - '0';

    for (j = 0; j < 10; j++) {
      if (c != -1 && c != j) continue;
      for (ki = 0; ki < 13; ki++) {
        dp[i + 1][(ki * 10 + j) % 13] += dp[i][ki];
      }
    }
    for (j = 0; j < 13; j++) dp[i + 1][j] %= MOD;
  }
  res = dp[n][5];

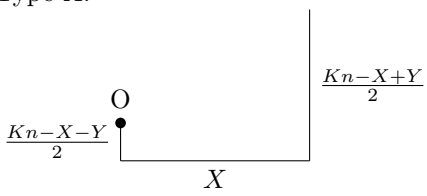
  printf("%llu\n", res);
  return 0;
}
```

E: Golf

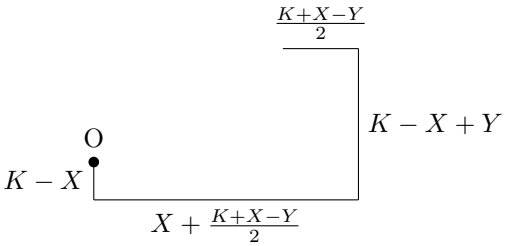
For simplicity, you can assume that $X \geq Y \geq 0$ (you can apply some appropriate transformations to achieve this).

If K is even and $X + Y$ is odd, it's impossible. Otherwise, you can construct the answer by either of the following two ways:

Type A:



Type B (alyaws $n = 3$):



Let $c := (K - (X + Y \bmod K)) \bmod K$. If $X + Y < K$ and $X + Y$ is odd, type B can be applied; otherwise type A can be applied by determining n with the following steps:

- If $X + Y < K$ and $X + Y$ is even, $n = 2$.
- If $X + Y \geq K$ and c is even, $n = \lceil \frac{X+Y}{K} \rceil$.
- If $X + Y \geq K$ and c is odd, $n = \lceil \frac{X+Y}{K} \rceil + 1$.

It is easy to prove that the construction procedures are rightful.

F: Strings of Eternity

In the notes in the problem statement, the first letter of a string is regarded as the first letter, but here let it 0-th letter. An intuitive restatement of the problem statement is: what is the maximum number of consecutive appearances of t in s', where s' is an infinitely repetition of the string s?

Here, for each integer i from 0 through $|s|-1$, suppose that it's already known whether the substring of s' from i-th letter through $i+|t|-1$-th letter is equal to t (let this truth values be match_i). Then you can find the answer in $\mathrm{O}(|s|)$ by simulation, because s' is periodic. In graph-theory terms, the answer is the longest path of the following directed graph such that:

- the graph has $|s|$ vertices, each of which is indexed $0, 1, \ldots, |s|-1$.
- the graph has a directed edge $i \to (i+|t|) \bmod |s|$ if $\mathrm{match}_i = \mathrm{true}$. There are no edges otherwise.

The remaining task is to find all the $\mathrm{match}_0, \ldots, \mathrm{match}_{|s|-1}$ in time. Of course, if you compare s' and t naively, it will need a time of $\mathrm{O}((|s|+|t|) \times |t|)$ at worst, so it won't finish in time. [*1] However, this is a very classic problem and many algorithms are invented to solve it fast. The following two algorithms are some of the ways to find all the $\mathrm{match}_0, \ldots, \mathrm{match}_{|s|-1}$ in a linear time.

- KNP algorithm (a link to Wikipedia article)
- Z algorithm (A link to GeeksforGeeks article)

Otherwise you can also use rolling hash algorithms by regarding the string as non-decimal numbers, but we prepared an input case such that collision would happen if you use mod 2^{64} in order to avoid the modulo operations.

[*1] If the input is random, the match would fail in most case so the expected time complexity is linear, but if the string has many repetition, such as $s = \texttt{aaa...aa}, t = \texttt{aaa...ab}$, it will take long time until the match fails

ABC 136 解説

DEGwer, drafear, evima, satashun

2019 年 8 月 4 日

For International Readers: English editorial starts on page 7.

A: Transfer

容器 1 にはあと $A-B$ ミリリットルまで入れることができます。よって、容器 2 から移す水の量 (ミリリットル) は $A-B$ と C の小さい方になります。したがって、容器 2 に残る水の量は $C-\min(A-B, C)$ ミリリットルになります。C++ 言語での実装例を以下に示します。

```
#include <bits/stdc++.h>

using namespace std;

int main() {
  int A, B, C; cin >> A >> B >> C;
  int cap = A - B;
  int use = min(C, cap);
  int ans = C - use;
  cout << ans;
}
```

B: Uneven Numbers

(原案: DEGwer, 準備・解説: evima)

N は 10 万以下とあまり大きくなく、$1, 2, \ldots, N$ という N 個の数の桁数を直接数えるのが楽です。

そして、それぞれの数の桁数を数える最も素直な方法は、数を文字列に変換してその長さを数えるというものでしょう。最も効率的な方法ではありませんが、今回はこれで十分です。

以下に Python による実装例を二つ掲載します。他の言語でもこれらと似たような実装が可能でしょう。

```python
N = int(input())
ans = 0
for i in range(1, N + 1):
    if len(str(i)) % 2 == 1:
        ans += 1
print(ans)
```

```python
N = int(input())
print(len(list(filter(lambda x: len(str(x)) % 2 == 1, range(1, N + 1)))))
```

C: Build Stairs

右のマスから順に操作を選ぶことにすると、できるだけ高くしておいた方が後で選択肢が増えるので、

- 何もしなくて良いなら何もしない
- そうでなくて、右のマスよりもちょうど 1 高いなら 1 低くする
- そうでないならどう頑張ってもダメなので答えは No

を右から順に行い、うまく構成できれば Yes を出力することで $O(N)$ の計算時間で答えを求めることができます。また、同様の考え方で、左から順に行うこともできます。

他の方法としては、i 番目までの高さの最大値を M_i としたときに、全ての $i = 1, 2, ..., N$ について、$H_i \geq M_i - 1$ が成り立てば Yes、そうでなければ No と判定することもできます。これは帰納法によって示すことができます。

実際に「$\forall i. H_i \geq M_i - 1 \Leftrightarrow$ 答えは Yes」を示してみます。

$\Leftarrow$ は簡単です。対偶を取ると、示すべきは「$\exists i. H_i < M_i - 1 \Rightarrow$ 答えは No」なので、これは自明です[*1]。

$\Rightarrow$ は帰納法で示します。

- $N = 1$ のときは必ず答えは Yes です。
- $N = N' + 1$ のとき、仮定より $i = 1, 2, ..., N'$ について条件を満たすので、帰納法の仮定より $H_1, H_2, ..., H_{N'}$ に対する解 (単調非減少にする操作の仕方) が存在します。この解への操作に上書きして、$H_i = M_N$ となる $i \in \{1, 2, ..., N'\}$ について 1 低くする選択肢を選ぶとしても $H_1, H_2, ..., H_{N'}$ に対する解となっていて、これに加えて H_N については何もしない選択をすると $H_1, H_2, ..., H_N$ に対する解が構成でき、Yes です。

[*1] $H_i < M_i - 1$ となる i を x とし、$H_j = M_x$ となる j を y とします。すると、$y < x$ かつ $H_x \leq H_y - 2$ が成り立ちます。つまりあるマスより右側のマスで、高さが 2 以上低いマスが存在するのでどう頑張ってもこの 2 マスに対する条件を満たすことができません。

D: Gathering Childrens

“RR...RLL...L” の部分に分解すると、他の部分と行き来することはないので独立に考えることができます。例えば、“RRLRL” は “RRL” と “RL” について独立に考えることができます。以降、“RR...RLL...L” の形についてのみ考えます。

十分な回数移動を行うので、境界部分、すなわち “RL” の部分に子供が集まり、他の部分には誰も居ない状態になります。さらに、最初左から偶数番目のマスにいた子供と奇数番目のマスにいた子供が同じマスに集まることはなく、逆に左からの位置の偶奇が等しいマスにいた子供たちは最終的には全員同じマスに集まります。移動回数は 10^{100} 回と偶数回なので、“RL” の “R” には “R” から偶数マス離れた子どもたちが集まり、“RL” の “L” には “L” から偶数マス離れた子どもたちが集まります。

このように計算することで時間計算量 $O(N)$ で答えを求めることができます。

実装では、文字列を連続した文字ごとに分解する (例えば “AABCCC” を (‘A’, 2), (‘B’, 1), (‘C’, 3) のように分解する) ランレングス圧縮のライブラリがあると便利です。

別解として、次のような解法もあります。マス数は高々 10^5 個なので、10^5 回移動した後の状態を求められれば十分です。この状態と 10^{100} 回移動した後の状態は一致します。また、左から i 番目のマスをマス i とし、$dp(i,j)$ を「最初マス j に居た子供が i 回の移動の後に居るマス」と定義します。すると、$dp(1,j)$ は S_j が L なら $j-1$、R なら $j+1$ です。さらに、任意の i_1, i_2, j について $dp(i_1+i_2, j) = dp(i_2, dp(i_1, j))$ なので、ダブリングにより $dp(X,j)$ を $O(NlogX)$ で計算できます。$X = 10^5$ について求めたいので、十分間に合います。ダブリングとは、x^n を $x^1, x^2, x^4, x^8, ...$ を組み合わせて計算する (例えば $x^{10} = x^8 \times x^2$ と計算します) 繰り返し二乗法を応用した手法で、今回の場合だと $dp(10,j) = dp(8, dp(2,j))$ と計算します。ここで、$dp(2^{n+1}, j)$ は $dp(2^n, dp(2^n, j))$ と計算します。この手法を用いれば、例えば問題が 10^{100} 回の移動後の状態ではなく K 回の移動後の状態を求めるものであったり、L,R といった単純な移動でなく右に R_i マス (R_i が負なら左に $-R_i$ マス) 移動といった場合にも答えを求めることができます。

E: Max GCD

まず、K 回の操作で A に対してどのような操作ができるか考えてみましょう。$A_1+d_1, A_2+d_2, \cdots A_N+d_N$ になるとします。

このとき、各操作で和が変化しないことに注目すると、$d_1 + d_2 + \cdots + d_N = 0$ である必要があります。また、d のうち正のものの和が K より大きい場合はありえません。負についても同様であり (和が 0 の時は自動的に満たされますが)、これら全てが満たされる場合はそのように操作可能であることが帰納的に示せます。

答え (d とします) の候補としては、$S = A_1 + A_2 + \cdots + A_N$ としたとき、d が S の約数である必要があります。d を S の約数全て試すことにして固定しましょう。このような d の候補は $O(sqrt(N * max(A)))$ 個あります。

まず、$A_1, \cdots A_N$ を d で割った余りを $r_1, r_2, \cdots r_N$ としましょう。0 であるものはなく、これは小さい順にソートされているとします。このとき、d を正にするものについては最低 $(d - r_i)$ の和の回数の操作が必要です。d を負にするものについては最低 r_i の和の回数の操作が必要です。それぞれ、d の倍数回の操作は余分に行うことができます。よって、r の小さい方から途中までを d を負に設定し、そこからは正になるようにする形のみを考えればよく (r の和が d の倍数であることに注意しましょう)、累積和などを用いて全ての区切り方の d が正のものの回数の和と負のものの回数を求めることができます。

よって、d を固定した場合には $O(NlogN)$ で解けるので、この問題は $O(sqrt(N * max(A)) * NlogN)$ で解けました。

F: Enclosed Points

まず、空でない部分集合は $2^N - 1$ 通りあります。これらについて直接 f の値を考えるのではなく、各点が長方形に含まれるような集合が何個あるかを各点について求めることを考えます。

ある点 P に注目しているとしましょう。P を中心に、P との x, y 座標の大小関係は 4 方向ありますが、このそれぞれに点が存在するかどうかという状態を固定すればこの P が長方形に含まれるかどうかが決まることがわかります。ある領域に K 個の点があるとした時、この領域に点が存在する場合は $2^K - 1$ 通りで、点が存在しない場合は 1 通りです。例えばですが、点がそれぞれに存在するかどうかの 2^4 通りについて、長方形に P が含まれる場合にこれらをかけ合わせて足していけばよいです。(P が集合に含まれるかどうかについても場合分けが必要です)

よって、各点 (x_i, y_i) について $x < x_i, y < y_i$、$x < x_i, y > y_i$、$x > x_i, y < y_i$、$x > x_i, y > y_i$ それぞれの領域についてそれぞれ何点存在するかが求まれば、答えを求めることができます。これはまず点の座標を $1 - N$ に圧縮した後、点を x 座標についてソートし、x の小さい方からと大きい方からで 2 回 segment tree / BIT (fenwick tree) を用いて各 y に存在する点の個数を管理していけば $O(NlogN)$ で計算できます。

以上でこの問題は $O(NlogN)$ で解けました。

解答例

ABC 136 Editorial

DEGwer, drafear, evima, satashun

August 4, 2019

A: Transfer

We can put at most $A - B$ milliliters of water into Bottle 1. Therefore, the amount of water that we will transfer from Bottle 2 would be the smaller of $A - B$ and C(milliliters). Therefore, the amount of water that remains in Bottle 2 would be $C - \min(A - B, C)$. The following is an implementation example in C++.

```
#include <bits/stdc++.h>

using namespace std;

int main() {
  int A, B, C; cin >> A >> B >> C;
  int cap = A - B;
  int use = min(C, cap);
  int ans = C - use;
  cout << ans;
}
```

B: Uneven Numbers

(Draft: DEGwer, Preparation and Editorial: evima)

N is not so large, no more than 100 thousands, so it is easy to count the digits of each of N numbers, $1, 2, \ldots, N$.

And the most natural way to count the digits of each value would be converting the number into string and count its length. It is not the fastest way, but it is enough for now.

The following two example is implementation examples in Python. You would be able to implement in other languages in similar way.

```
N = int(input())
ans = 0
for i in range(1, N + 1):
    if len(str(i)) % 2 == 1:
        ans += 1
print(ans)
```

```
N = int(input())
print(len(list(filter(lambda x: len(str(x)) % 2 == 1, range(1, N + 1)))))
```

C: Build Stairs

Suppose that you choose the operation from right to left; then you will have more choices if you make them as high as possible, so by performing such operations that

- if you don't have to do anything, do nothing
- otherwise, if it's higher than the square by exactly one, decrease it by one
- otherwise, you cannot achieve it no matter what you do, so the answer is No

from left to right, and printing Yes if construction succeeded, the answer can be found in time complexity of $O(N)$. Similarly, you can perform it from left to right.

As another solution, let M_i be the maximum of heights until i-th square, then it can be judged Yes if $H_i \geq M_i - 1$ for all $i = 1, 2, ..., N$, and No otherwise. This can be proved recursively.

Let's prove that "$\forall i. H_i \geq M_i - 1 \Leftrightarrow$ the answer is Yes".

$\Leftarrow$ is easy. Its contraposition is that "$\exists i. H_i < M_i - 1 \Rightarrow$ the answer is No", so it's obvious[*1].

Let's prove $\Rightarrow$ inductively.

- If $N = 1$, the answer is always Yes.
- If $N = N' + 1$, since the proposition is true for $i = 1, 2, ..., N'$, there exist an solution (way of operations to make it non-decreasing) for $H_1, H_2, ..., H_{N'}$. If you overwrite the solution so that you chose to decrease the height by 1 of the square $i \in \{1, 2, ..., N'\}$ such that $H_i = M_N$, it is still solution for $H_1, H_2, ..., H_{N'}$, and in addition, if you choose to do nothing for H_N, you can construct a solution for $H_1, H_2, ..., H_N$, and it's Yes.

[*1] Take one i such that $H_i < M_i - 1$, and let y be such j that $H_j = M_x$ holds. Then $y < x$ and $H_x \leq H_y - 2$ holds. Therefore, there exist a square that the height of some square left to it is at least two units lower than that of the square, so there is no way to achieve the condition between those two squares.

D: Gathering Childrens

If you split it into parts of "RR...RLL...L", the children do not cross the border, so each of them can be considered independently. For example, "RRLRL" can be considered independently for each "RRL" and "RL". Hereafter, only the sequence like "RR...RLL...L" is considered.

Since there are enough times of movement, all the children gather into the border, that is, "RL", and nobody is in elsewhere. Moreover, a child who was initially in a square with even number of index and a child initially in odd index do never gather into same square, and conversely, children in same parity of squares all gather into the same square finally. There are even number, 10^{100} times, of movements, so children whose square's initial index is distant from "R" of "RL" by even number of distance gather into "R", and children whose square's initial index is distant from "L" of "RL" by even number of distance gather into "L".

This way, the answer can be found in time complexity $O(N)$.

It will be useful for implementation if you can use run-length encoding library, which disassembles a string into consecutive same characters (for example, disassembles "AABCCC" into a sequene like ('A', 2), ('B', 1), ('C', 3)).

There is another solution like the following. Since there are at most 10^5 squares, it is sufficient if the state after 10^5 movements can be found. This states corresponds to the state after 10^{100} movements. Let square i be the i-th square counted from left, and $dp(i, j)$ be "the index of square where children who was initially in square i moves into after j movements." Then, $dp(1, j)$ is $j-1$ if S_j is L, and $j+1$ if it's R. Moreover, for any i_1, i_2, j it holds that $dp(i_1 + i_2, j) = dp(i_2, dp(i_1, j))$, so by using doubling you can find $dp(X, j)$ in $O(NlogX)$. You want to find the value of $X = 10^5$, so it's sufficient. The doubling is development of the fast exponentiation algorithm, which is finding x^n combining $x^1, x^2, x^4, x^8, ...$ (for example, in such way that $x^{10} = x^8 \times x^2$), and this time, it is calculated like $dp(10, j) = dp(8, dp(2, j))$. Here, $dp(2^{n+1}, j)$ is $dp(2^n, dp(2^n, j))$. By using this technique, you can also obtain the answer even if the problem asks to find the state after K movements, or the movements are not as simple as L, R but moving R_i sqaures to the left (if R_i is negative, move $-R_i$ squares to the right).

E: Max GCD

First, let's think what kind of modification can be applied to A by performing K operation. Suppose that it is changed into $A_1 + d_1, A_2 + d_2, \cdots A_N + d_N$.

Since each operation does not change the sum of the array, $d_1 + d_2 + \cdots + d_N = 0$ has to be 0. Moreover, the sum of positive elements of d cannot be more than K. So does that of negative elements (if the sum is 0, it is automatically fulfilled, though), and it can be proved inductively that if all of these conditions holds, the operations can be performed in such way.

Let $S = A_1 + A_2 + \cdots + A_N$, then the candidate of answer (let's denote it as d) should be divisor of S. Let's fix some d among the divisor of S. There are $O(sqrt(N * max(A)))$ candidates of such d.

First, let $r_1, r_2, \cdots r_N$ be the remainder of $A_1, \cdots A_N$ divided by d. Suppose that there aren't 0 in it, and they are sorted in increasing order. Then, for the element such that d is positive, the number of operations that shuold be performed is at least sum of $(d - r_i)$, and for the element of negative, it's at least sum of r_i. Therefore, it is optimal that d is set to negative value for the former part of r until certain element, and set to positive for the rest (note that the sum of r is multiple of d), and you can try all the borders and calculate the sum of positive elements of d and that of negative elements by using cumulative sum.

Therefore, when d is fixed it can be solved in $O(NlogN)$, so this problem can be solved in $O(sqrt(N * max(A)) * NlogN)$.

F: Enclosed Points

First, there are $2^N - 1$ non-empty subsets. Instead of counting the value of f for each of them, for each point, let's count how many sets there are such that the point is contained in rectangles.

Let's focus on a point P. Depending on the comparison of the x- and y- coordinates with those of P, the coordinate system is divided into four regions, and if the presence of points in each region is determined, it can also be determined whether this P is contained in the rectangle or not. When a region contains K points, there are $2^K - 1$ subsets of those points such that there exists at least one point in the region, and 1 set such that none of them are in it. For instance, for each of 2^4 combinations of existence of each region, you can calculate the product of them if the rectangle contains P and sum them up. (You also have to check if P is included in the set)

Therefore, you can find the answer if you can count the number of points in each region $x < x_i, y < y_i$, $x < x_i, y > y_i$, $x > x_i, y < y_i$ and $x > x_i, y > y_i$ for each point (x_i, y_i). You can achieve this in $O(NlogN)$ by compressing the coordinates into $1 - N$, sorting the points by x-coordinate, and managing the number of points by their y-coordinates using segment tree / BIT (fenwick tree) twice, in its increasing and decreasing order.

So this problem can be solved in $O(NlogN)$.

Link to sample code

ABC 137 解説

DEGwer, drafear, evima, gazelle, IH19980412, potetisensei, yokozuna57

2019 年 8 月 10 日

For International Readers: English editorial starts on page 9.

A: +-x

指示通り $A+B$, $A-B$, $A\times B$ の中で最大の数を出力すればいいです。C++ による実装例を以下に示します。

Listing 1　C++ による実装例

```
#include<bits/stdc++.h>
using namespace std;

int main() {
    int a, b;
    cin >> a >> b;
    cout << max({a + b, a - b, a * b}) << endl;
    return 0;
}
```

B: One Clue

(原案: DEGwer, 準備・解説: evima)

黒い石のうち最も左にある (座標が最も小さい) ものの座標を L とすると、K 個の黒い石の座標は $L, L+1, \ldots, L+K-1$ と書けます。

座標 X の石が黒であることから、L は $X-K+1, X-K+2, \ldots, X$ のいずれかです。よって、黒い石が置かれている可能性のある座標は $X-K+1, X-K+2, \ldots, X+K-1$ の $2K-1$ 個です。

あとは純粋なプログラミングの課題で、Python での実装例を二つ掲載することをもって解説に代えさせていただきます。[*1] 他の言語でもほぼ同等な実装が可能なはずです。

```
K, X = map(int, input().split())
for i in range(X - K + 1, X + K):
    if i < X + K - 1:
        print(i, end=' ')
    else:
        print(i)
```

```
K, X = map(int, input().split())
print(' '.join(map(str, range(X - K + 1, X + K))))
```

[*1] 実は、一つ目の実装例で 3, 5, 6 行目を削除して、最後の座標の直後でも改行ではなく空白を出力しても正解と判定されます。AtCoder に限らず、競技プログラミングのジャッジは「空白系の文字」の扱いに関して寛容なことが多いです。ただし、ターミナルに出力が表示される際の見やすさを考えてもできるだけ出力の末尾では改行されることを勧めます

C: Green Bin

(原案: yokozuna57, 準備・解説: evima)

まず、二つの文字列 s,t が与えられた際に s が t のアナグラムであるかを判定する方法には次の二つがあります。

1. 26 種のアルファベットそれぞれが s と t にそれぞれ何回出現するか数え、どのアルファベットについても出現回数が同じか確認する。
2. 両方の文字列をソートし、一致するか確認する。

今回の問題では最大で 10 万個の文字列が与えられ、すべてのペア (最大で 50 億個程度) について上の方法をそのまま実行すると 2 秒という時間制限を過ぎてしまうでしょう。しかし、上の方法で文字列から得る「エッセンス」を元に文字列をグループに分けることで計算を効率化できます。入力例 3 を用いて、方法 2 を元に効率化した計算の例を以下に示します。

1. $s_1 =$ `abaaaaaaaa` をソートして `aaaaaaaaab` を得る。
2. $s_2 =$ `oneplustwo` をソートして `elnoopstuw` を得る。
3. $s_3 =$ `aaaaaaaaba` をソートして `aaaaaaaaab` を得る。この結果はすでに 1 回現れているので、答えに 1 を足す。
4. $s_4 =$ `twoplusone` をソートして `elnoopstuw` を得る。この結果はすでに 1 回現れているので、答えに 1 を足す。
5. $s_5 =$ `aaaaaaaaba` をソートして `aaaaaaaaab` を得る。この結果はすでに 2 回現れているので、答えに 2 を足す。

「この結果はすでに 1 回現れているので」などの部分は、ハッシュテーブル (C++ の undered_map, Java の HashMap, Python の dictionary など) を用いて実装すれば線形時間で動作します。他に、 N 個の文字列から得られる N 個の「エッセンス」をソートするという解法も考えられますが、こちらは言語によっては実行が間に合わないかもしれません。

D: Summer Vacation

夏休みにお金を稼いで沖縄旅行に行きたいという気持ちで解くと解きやすいかもしれません。M 日後までに報酬を得る必要があるので M 日後から i $(1 \leq i \leq M)$ 日前には $A_j \leq i$ となる日雇いアルバイトしか選ぶことができません。こういった問題は制約の厳しい方から見ていくと見通しが良くなることが多いです。実際、この問題は後ろ (M 日後から 1 日前) から見ていくと、まず $A_j \leq 1$ となる j の中で B_j が最大のもの (j_1 とする) を選び、次に $A_j \leq 2$ かつ $j \neq j_1$ となる j の中で B_j が最大のものを選び、…としていくのが最適です[*2]。

次に、実装を考えます。したい操作は

- 候補の追加
- 候補の中から最大のものを取り出す

であって、それぞれ $O(N), O(M)$ 回行います。優先度付きキュー (Priority Queue)[*3]を用いると、これらを高速に処理することができます。

これを実装すると、$O(M + NlogN)$ の時間計算量で解くことができます。

別解

少し高度な解き方です。一言で言うと、実行可能解の集合はマトロイドなので、貪欲法で解くことができます。また、最小費用流の動作もこの貪欲法と同じです。

まず、次の 3 つの公理を満たす有限[*4]集合族[*5] I のことをマトロイドといいます。

1. 空集合は I の要素である。
2. 集合 X が I の要素であるとき、X のどんな部分集合 Y も I の要素である。
3. 大きさの異なる集合 X, Y $(|X| < |Y|)$ が I の要素であるとき、集合 Y に属するが集合 X に属さない要素 v であって、集合 X に加えても I の要素となるような v が存在する。

今回の問題に当てはめると、次のようになります。I を実行可能解[*6]の集合と考えます。

1. 空集合 (どの日雇いアルバイトも選ばない) は I の要素 (すなわち、実行可能解) である。
2. 日雇いアルバイト $\{i_1, i_2, ..., i_k\}$ の報酬が間に合うようにうまく割り当てられるとき、ここからいくつかやらないことにした $\{i'_1, i'_2, ..., i'_l\} \subseteq \{i_1, i_2, ..., i_k\}$ についても報酬が間に合うようにうまく割り当てることができる。
3. $\{i_1, i_2, ..., i_k\}$ も $\{j_1, j_2, ..., j_l\}$ も報酬が間に合うようにうまく割り当てられ、$k < l$ であるとき、

[*2] これはこの問題を最小費用流に帰着させて示すことができます。しかし、最小費用流で解こうとすると $O(M(N+M)log(N+M))$ となり間に合いません。

[*3] 優先度付きキューの仕組みの説明はここでは省略しますが、優先度付きキューに追加された要素数を X とすると、$O(logX)$ で要素の追加、最大値の削除を、$O(1)$ で最大値の取得が行えます。C++ では標準 C++ ライブラリ (STL) で提供されています。

[*4] 要素数が有限の集合であるということ。

[*5] 各要素が集合であるような集合。集合の集合。

[*6] 条件を満たすような選び方。最適解とは限らない。今回の場合だと、いつやるかをうまく割り当てると選んだ全ての日雇いアルバイトができる (報酬が間に合う) ような、選ぶ日雇いアルバイトの番号の集合。

$\{i_1, i_2, ..., i_k, j_x\}$ がうまく割り当てられるような x が存在する。

証明は省きますが、この 3 つは満たされるので、今回の問題の実行可能解の集合もマトロイドです。さて、マトロイドであれば何が嬉しいのでしょうか。集合族 I がマトロイドであって、I の要素である集合の要素となりうるもの x に対して重み $w(x)$ が定まっているものとします。今回の場合ですと、各日雇いアルバイトに対して報酬といった重みが定まっています。I の要素 X の重みを、その集合の各要素の重みの和 $\sum_{x \in X} w(x)$ と定義します。どの y についても $w(y) \geq 0$ であるとき、I の要素の重みの最大値を貪欲法で求めることができます。

マトロイド I に対する貪欲法とは次のようなものです。

1. S を空集合とします (後に、S に要素を順番に追加していき、最適解を構成します)。マトロイドの公理 1 より、$S \in I$ です。
2. 重みの大きい順に解の要素として選べるもの (今回の場合だと日雇いアルバイト) をソートし、$x_1, x_2, ..., x_N$ とします。
3. $y = x_1, x_2, ..., x_N$ と順に以下を行う。
   (a) S に y を加えても I の要素である ($S \cup \{y\} \in I$ である) とき、S に y を加える。
4. S を出力する (S が最適解である)。

上のアルゴリズムの中でマトロイドによって異なる部分は「S に y を加えても I の要素である」ことを判定する部分です。「S に y を加えても I の要素である」を今回の問題について言い換えると、「日雇いアルバイト番号の集合 $S \cup \{y\} = \{i_1, i_2, ..., i_k\}$ が実行可能解であるか、すなわちうまく割り当てて全ての報酬を M 日後までに得られるか」です。これを愚直に判定すると全体として $O(N^2)$ となり間に合いません。少し考えてみると、$i_1, i_2, ..., i_k$ をうまく割り当てられることと、好きな順に次のように割り当てられることが同値であることがわかります。

1. 好きな順に割り当てる。今回割り当てるものを x とする。
2. M 日後から x 日前に割り当てようとする。できなければ (すなわち既に他の日雇いアルバイトが割り当てられていれば) M 日後から $x-1$ 日前に割り当てようとする。できなければ M 日後から $x-2$ 日前に割り当てようとする。これを初日まで繰り返しても割り当てられないとき、割り当てに失敗する。

すなわち、上の手順 2 は、M 日後から x 日前よりも以前であって、まだどの日雇いアルバイトも割り当てられていない日のうち最も遅い日が高速に求まれば高速に判定できます。これは、区間の最大値を求められ、一点更新が可能なセグメント木を使えば、S に対する割り当ての情報を保持しながら y に対して $O(logM)$ で判定できるため、ソートも合わせて全体として $O(NlogN + NlogM)$ で解くことができます。

上の判定を区間の最大値を求められるセグメント木の代わりに区間和を求められるセグメント木と二分探索を組み合わせて高速化することや、このような上手い割り当て方を考察せずとも区間加算、区間最小値、一点挿入が高速にできる平衡二分木を使って解くこともできます。

別解の冒頭でも述べましたが、最小費用流に帰着させたときの動作を考えると、マトロイドの解法と同じ動作になっているため、これを高速化する方針でも解くことができます。

E: Coins Respawn

(原案: IH19980412, 準備・解説: evima)

ゲーム終了時に $T \times P$ 枚のコインを支払うのではなく、辺を通るたびに P 枚のコインを支払う (その結果道中でコインの所持枚数が負になることも許容する) ことにすれば、時間の概念は不要になります。求めたいものは、各辺 i の重みを $C_i - P$ としたときの頂点 1 から頂点 N に至るパスの最大の重みです。各辺の重みをさらに -1 倍すると、これは最短路問題そのものになります。

最短路問題を解く有名なアルゴリズムにダイクストラ法 (Wikipedia の記事へのリンク) がありますが、今回は負の重みを持つ辺があるため使えません。ワーシャル–フロイド法 (Wikipedia の記事へのリンク) は負の重みを持つ辺に対応しますが、頂点数を V として $\mathrm{O}(V^3)$ の計算量を要しこれも今回用いるには厳しいでしょう。

今回用いるべきアルゴリズムは ベルマン–フォード法 (Wikipedia の記事へのリンク) です。このアルゴリズムは負の重みを持つ辺に対応し、頂点数を V、辺数を E として計算量 $\mathrm{O}(VE)$ で動作します。これを上で定めたグラフにほぼそのまま適用すれば問題が解けます。ただし、入力例 3 にあるような頂点 1 から頂点 N への移動と無関係な負閉路に反応しないように注意する必要があります。~~その最も簡単な方法は、アルゴリズムにおける「辺の緩和」を追加で V 回行って「頂点 N までの最短距離と思われる距離」が変化しないか確かめることでしょう。~~ (8/11 0:08 JST 修正: この記述は誤りでした)

F: Polynomial Construction

(原案: potetisensei, 準備・解説: evima)

以下、整数 i $(1 \leq i \leq p-1)$ に対し、$ij \equiv 1 \pmod{p}$ であるような整数 j $(1 \leq j \leq p-1)$ を i の逆数と呼びます。p が素数のとき、このような整数は必ず存在し、かつ一つに定まります (詳しくは後述します)。

式 $f(i) \equiv a_i \pmod{p}$ に $i = 0, 1, \ldots, p-1$ を実際に代入することで $b_0, b_1, \ldots, b_{p-1}$ に関する n 本の方程式が得られ、これをあたかも実数の連立一次方程式を解くかのように (ただし割り算を行おうとする際は代わりに割る数の逆数を掛ける) 掃き出し法を用いて解けば $\mathrm{O}(p^3)$ 時間で (唯一の) 解が求まりますが、これでは間に合いません。解をより直接的に構成する必要があります。

天下り的になってしまいますが、鍵を握るのはフェルマーの小定理 (Wikipedia の記事へのリンク): 「a が p の倍数でない整数のとき $a^{p-1} \equiv 1 \pmod{p}$」です。この先を読む前に、この定理を元にして条件を満たす多項式を得る方法を考案されることを勧めます。

(次のページへ続く)

多項式の構成方法を述べます。フェルマーの小定理より、整数 j $(0 \leq j \leq p-1)$ に対して値 $1-(x-j)^{p-1}$ は $x=j$ のときのみ 1、それ以外のとき 0 となります。この値を $a_j = 1$ であるようなすべての j に対して足し合わせれば所望の多項式が得られます。

あとは与えられた j に対して $(x-j)^{p-1}$ の展開を $\mathrm{O}(p)$ 時間で行えれば $\mathrm{O}(p^2)$ 時間で解が求まります。これには二項係数 $\binom{p-1}{i}$ の計算が必要であり、この計算は $0!, 1!, \ldots, (p-1)!$ の逆数を事前に求めておけば $(p-1)!$ に $i!$ の逆数と $(p-1-i)!$ の逆数を掛けることで行えます。

最後に、与えられた整数に対する逆数の求め方と存在性に関して述べます (今回は逆数を全探索しても間に合いますが)。再びフェルマーの小定理を用いると整数 i $(1 \leq i \leq p-1)$ の逆数は i^{p-2} を p で割った余りとして求められ、これは繰り返し二乗法を用いて高速に計算できます。

ABC 137 Editorial

DEGwer, drafear, evima, gazelle, IH19980412, potetisensei, yokozuna57

August 10, 2019

A: +-x

Implement the instructions given in the problem statement, just output the maximum of $A + B$, $A - B$, $A \times B$. A sample implementation in C++ is shown below.

Listing 1 An implementation example in C++

```
#include<bits/stdc++.h>
using namespace std;

int main() {
    int a, b;
    cin >> a >> b;
    cout << max({a + b, a - b, a * b}) << endl;
    return 0;
}
```

B: One Clue

(Original writer: DEGwer, preparation and editorial: evaim)

Let the leftmost black stone (the stone with the least coordinate) be L, then the coordinates of K black stones will be $L, L+1, \ldots, L+K-1$.

Since the stone at coordinate X is black, L is one of $X-K+1, X-K+2, \ldots, X$. Therefore, possible coordinates of black stone are those $2K-1$ values: $X-K+1, X-K+2, \ldots, X+K-1$.

The remaining task is a pure programming task, so we'll show you two implementation examples in Python instead of explanation. [*1] Similar implementation should be possible in other languages.

```python
K, X = map(int, input().split())
for i in range(X - K + 1, X + K):
    if i < X + K - 1:
        print(i, end=' ')
    else:
        print(i)
```

```python
K, X = map(int, input().split())
print(' '.join(map(str, range(X - K + 1, X + K))))
```

[*1] In fact, you can remove the third, fifth and sixth lines from the first implementation, and output a whitespace instead of newline after the last coordinate. Not only AtCoder, but many competitive programming's judge is tolerant to "whitespace-like letters" in many cases. Though, thinking about the visibility shown in terminals, we recommend you print new line at the end of the output

C: Green Bin

(Original writer: yokozuna57, preparation and editorial: evima)

First, given two strings s, t, there are the following two ways to check if s is an anagram of t:

1. For 26 kinds of alphabets, check if how many of them appears in s and t, and check if those alphabets appears same the same time.
2. Sort both strings and check if it corresponds.

This time there are at most 100 million strings, so if you try to check for all pairs (at most about 5 billions), it will not finish in time limit. The following is the optimized version of the second way, using the third input example:

1. Sort $s_1 =$ `abaaaaaaaa` and get `aaaaaaaaab` .
2. Sort $s_2 =$ `oneplustwo` and get `elnoopstuw` .
3. Sort $s_3 =$ `aaaaaaaaba` and get `aaaaaaaaab` . It has been appeared 1 time before, so add 1 to the answer.
4. Sort $s_4 =$ `twoplusone` and get `elnoopstuw` . It has been appeared 1 time before, so add 1 to the answer.
5. Sort $s_5 =$ `aaaaaaaaba` and get `aaaaaaaaab` . It has been appeared 2 time before, so add 2 to the answer.

The step of calculating "it has been appeared 1 times" and so on can be implemented with hash tables (C++'s unordered_map, Java's HashMap, Python's dictionary etc.) so that it works in a linear time. Otherwise, you can sort N "essences" obtained from N strings, but it may not fit the time limit, depending on programming languages.

D: Summer Vacation

If you dream of earning enough money to take a trip for Okinawa, this problem may be solved easily (Okinawa is one of the most famous tourist spot in Japan). You have to receive the rewards no later than M days later, so in the day i ($1 \leq i \leq M$) days before the day M days after today, you can only choose one-off job such that $A_j \leq i$. This kind of problem often becomes clearer if you examine the harder constrains first. Actually, if you check them back to forth (starting from the day 1 day after the day M days after today), you can choose j such that B_j is maximum among the js such that $A_j \leq 1$ (let it be j_1), then choose j such that B_j is maximum among the js such that $A_j \leq 2$ and $j \neq j_1$, ... and so on, you can find the optimal answer[*2].

Next, let's think about the implementation. The operations needed are

- to add a candidate
- pop the maximum value from the candidates

and each of them are done $O(N), O(M)$ times. If you use priority queue[*3], you can perform those operations fast.

By implementing it, you can solve this problem in a total of $O(M + NlogN)$ time.

Another Solution

Here is a little advanced solution. Briefly, the set of possible selection is a matroid, so it can be solved greedily. Also, the behavior of min-cost flow algorithm is the same.

First, matroid is a finite[*4] family of set[*5] I with the following axioms:

1. An empty set is an element of I.
2. When a set X is an element of I, any subset Y of X is an element of I.
3. If a pair of set X, Y with different number of elements ($|X| < |Y|$) are both contained in I, then there exist an element v which is in Y but not in X, such that after v is added to X it is still an element of I.

This time, it's like as follows. Let I be the set of possible selection[*6].

1. An empty set (choosing none of the jobs) is an element of I (in other words, it's a possible selection).

[*2] this can be proved by rephrasing this problem into minimum-cost flow problem. However, solving it as an minimum-cost flow problem takes a total of $O(M(N+M)log(N+M))$ time, so it won't fit in time limit.

[*3] We will not explain the structure of priority queue here. It can operate addition of element and removal of maximum element in $O(logX)$ time, and find maximum value in $O(1)$ time. In C++, it is provided in the standard C++ library (STL).

[*4] That means that the set contains finite number of elements.

[*5] A family of set is a set such that its element is a set, or a set of set.

[*6] A maximum selection is a selection of jobs, possibly not an optimal answer. This time, a set of indices such that there exists an assignment of those job into what day to work, such that all the rewards is paid in time.

2. If a selection of $\{i_1, i_2, ..., i_k\}$-th job can be assigned into workdays properly so that all the rewards is paid in time, if you decided not to do some of them and decided to choose $\{i'_1, i'_2, ..., i'_l\} \subseteq \{i_1, i_2, ..., i_k\}$, there still exists a valid assignment of workdays.
3. If both selection $\{i_1, i_2, ..., i_k\}$ and $\{j_1, j_2, ..., j_l\}$ has a valid assignment and $k < l$, then there exist x such that $\{i_1, i_2, ..., i_k, j_x\}$ has valid assignment.

We will not show the proof here, but these 3 conditions holds. so the set of possible selection for this problem is a matroid. Then what's a good point in matroid? Let's assume that a family of sets I is a matroid, and cost $w(x)$ is determined for any x that can be element of a set in I. This time, for each one-off job, "reward" is determined, that can be treated as its cost. Let define the cost of X in I as the sum of costs of each element of the set, $\sum_{x \in X} w(x)$. If $w(y) \geq 0$ for all y, you can find the maximum cost of elements in I greedil.

A greedy algorithm for a matroid I is as follows:

1. Let S be an empty set (in which elements are to be added one by one later to construct an optimal solution). According to the first axiom of matroid, $S \in I$ holds.
2. Sort the possible elements of solution (that is, the day-off jobs this time) and let it be $x_1, x_2, ..., x_N$.
3. For $y = x_1, x_2, ..., x_N$, do the following:
   (a) If even after y is added to S, it is still an element of I (that is $S \cup \{y\} \in I$), then add y to S.
4. Output S (S is an optimal answer).

In the algorithm above, the steps that depend on matroid is the judgement step of "check if even after y is added to S it's still in I." If you rephrase "check if even after y is added to S it's still in I" for this problem, it is "whether a set of indices of one-day off jobs $S \cup \{y\} = \{i_1, i_2, ..., i_k\}$ is a possible selection or not, in other words, all the rewards can be obtained until M days after today." If you judge this naively, it takes a total of $O(N^2)$ and won't finish in time. Thinking a little more carefully, it appears that the two conditions, whether $i_1, i_2, ..., i_k$ can be assigned to workdays properly, and whether these can be assigned in an arbitrary order as follows, is equivalent.

1. Assign them randomly. Let the current assignment be x.
2. Try to assign it to the day x days after the day M days after today. If not possible (that is, if there exist some other day-off jobs), try to assign it to the day $x - 1$ days after the day M days after today. If not possible (that is, if there exist some other day-off jobs), try to assign it to the day $x - 2$ days after the day M days after today. After checking until the first day, if it was not possible to assign to all the days, then it fails to assign.

Therefore, the second procedure above can be judged rapidly if such value can be found rapidly: the latest day that is before the day x days after the day M days after today, such that none of the jobs is assigned to. If you use a segment tree that provides maximum value of segments and update of one element, it can be judged in $O(logM)$ for y while saving the assignment information of S. So, it can be solved in a total of $O(NlogN + NlogM)$ time.

You can make the judge faster by using a segment tree that provides segment sum combined with

binary search instead of a segment tree that provides minimum of segments, or solve this problem without considering proper assignments, instead using a self-balancing binary tree that provides segment sums, segment minimums, and one-element updates.

As we stated in the beginning of this section, if you consider the behavior when the problem is regarded as a min-cost flow, the behavior is same to that of matroid, so you can solve the problem by making it faster.

E: Coins Respawn

(Original writer: IH19980412, preparation and editorial: evima)

Instead of paying $T \times P$ coins at the end of the game, if you decide to pay P coins every time you traverse an edge (and allow having negative number of coins), you don't have to think about the time order of events. What we want to find is the maximum cost of path from vertex 1 to vertex N, where cost of each edge i is set to $C_i - P$. If you multiply the cost by -1, the problem becomes a shortest-path problem.

One of the famous algorithms to find shortest path is Dijkstra's algorithm (Link to Wikipedia article), but this time there exist edges with negative cost, so it cannot be applied. Floyd – Warshall algorithm (Link to Wikipedia supports edges with negative costs, but it needs a total of $O(V^3)$ time where V is number of vertex, so it is a little bit difficult to finish in time.

The algorithm that should be used this time is Bellman – Ford algorithm (Link to Wikipedia article). This algorithm supports edges with negative costs and works in $O(VE)$ time where V is number of vertices and E is number of edges. If you apply almost directly to the graph mentioned above, you can solve the problem. Note that it should not misdetect negative cycle that is not concerned with the traverse from vertex 1 to vertex N. ~~The easiest way to achieve this is doing "relaxation" V more times and check if "apparent minimum cost to vertex N" does not change.~~ (8/11 0:08 JST corrected: this statement was wrong.)

F: Polynomial Construction

(Original writer: potetisensei, preparation and editorial: evima)

For an integer i ($1 \leq i \leq p-1$), we call an integer j ($1 \leq j \leq p-1$) such that $ij \equiv 1 \pmod{p}$ as "inverse of i." When p is prime, there always exists such an integer, and it's always unique (which will be explained later).

By assigning $i = 0, 1, \ldots, p-1$ to the equations $f(i) \equiv a_i \pmod{p}$, we will obtain n simultaneous equations, and with row reduction just like solving normal linear equations series (except that instead of dividing with some integer, multiplying its inverse), the (unique) solution can be found in a total of $O(p^3)$ time, but it is not enough. You have to find a solution more directly.

It's a bit kind of sudden, but the key is Fermat's little theorem (Link to Wikipedia article: "if a is an integer that is not a multiple of p, then $a^{p-1} \equiv 1 \pmod{p}$." Before reading further, we recommend you consider the ways to obtain a polynomial that meets the conditions.

(Continue to next page)

Here is a way of construction. By Fermat's little theorem, for any integer j $(0 \leq j \leq p-1)$, the value of $1-(x-j)^{p-1}$ is 1 only if $x = j$, and 0 otherwise. If you sum them up for all j, that will be the desired answer.

The remaining task is to expand $(x-j)^{p-1}$ in $\mathrm{O}(p)$ time, in which case the answer can be found in a total of $\mathrm{O}(p^2)$ time. For this task you need to calculate binominal coefficients $\binom{p-1}{i}$, and if you precalculate inverses of $0!, 1!, \ldots, (p-1)!$, those can be calculated by multiplying $(p-1!$ by inverse of $i!$ and inverse of $(p-1-i)!$.

Finally, we will explain how to obtain the inverse of a given number, and its existence (this time though, brute forcing inverses will meet the time). Applying Fermat's little theorem again, the inverse of an integer i $(1 \leq i \leq p-1)$ can be found as a remainder of i^{p-2} divided by p, and this can be calculated by "exponentiation by squaring."

ABC 138 解説

camypaper, evima, sheyasutaka, tozangezan, yosupo

2019 年 8 月 18 日

For International Readers: English editorial will be provided in a few days.

A: Red or Not

(原案: tozangezan, 準備・解説: evima)

practice contest の問題 A で足し算の代わりに条件分岐をさせるような問題です。入出力の行い方はそちらのサンプルコードを参照していただき、あとは if 文や条件演算子 (`?:` など) と比較演算子 (`>=` など) を組み合わせて a と 3200 の大小に応じて出力を変える処理を加えれば完成です。(抽象的な記述で終わってしまい恐縮ですが、個別の言語について述べるにはこの PDF は狭すぎます。Google 検索を活用してください)

以下は Python(3) での実装例です。

```
a = int(input())
s = input()
print(s if a >= 3200 else 'red')
```

B: Resistors in Parallel

(原案: sheyasutaka, 準備・解説: evima)

入出力例の説明では数式の分母 $\frac{1}{A_1} + \frac{1}{A_2} + \ldots + \frac{1}{A_N}$ を分数のまま計算していますが、はじめから浮動小数点型を使って計算するのが簡単でしょう。あとは、for 文やその他の何らかの繰り返し処理を行う機構を用いて実装を行います。諸言語を代表して C++ と Python(3) での実装例を掲載します。

```cpp
#include <iomanip>
#include <iostream>
using namespace std;

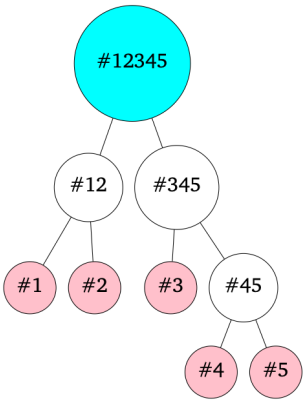
int main(){
    int N;
    double ans = 0.0, A;
    cin >> N;
    for(int i = 1; i <= N; ++i){
        cin >> A;
        ans += 1.0 / A;
    }
    ans = 1.0 / ans;
    cout << setprecision(16) << ans << endl;
}
```

```python
N = int(input())
A = map(int, input().split())
print('{:.16g}'.format(1 / sum(1 / x for x in A)))
```

C: Alchemist

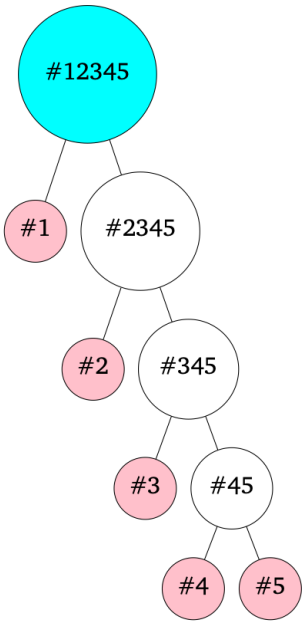
(原案: yosupo, 準備・解説: evima)

以下、具材 $1, \ldots, N$ は価値の大きい順に番号がつけられているとして (はじめに具材をソートしてそのように番号を付け替えます)、合成の過程を下図のように木構造として表すことを考えます (#1 は具材 1、#12 は具材 1, 2 を合成して得られる具材を表します)。



この図で具材 i を表す葉から最終的な具材を表す根までの距離 (間の辺の数) を d_i とすると、最終的な具材の価値は $v_1/2^{d_1} + \ldots + v_N/2^{d_N}$ と書けます。最大の結果を得るには、$d_1 \leq d_2 \leq \ldots \leq d_N$ であるような合成過程のみ考えれば十分です (そうでなければ、具材の番号を付け替えることで最終結果が向上するため)。

さらに考えると、最大の結果を得るには次の図のように $d_1 = 1, d_2 = 2, \ldots, d_{N-1} = d_N = N - 1$ とするべきです。これは、簡潔に述べると、$d_1 \leq d_2 \leq \ldots \leq d_N$ を満たすこれ以外の合成過程 (例えば上図のような) は、もっとも浅い「違反」した「左の子」以下を最も深い「右の子」以下に「移植」することで最終結果が向上するためです。



D: Ki

(原案: tozangezan, 準備・解説: evima)

一度の操作が N 個の頂点すべてに影響する可能性があるため、操作をそのままシミュレートするとカウンターへの加算を NQ 回行うことになり実行時間制限に間に合いません。[*1]

このような木に関する問題では、木の代わりに $1-2-3-\ldots-N$ という直線的なグラフを考えると助けになることがあり、今回はその好例です。この問題でグラフが直線的であるとすると、次のような線形時間の解法が考えられます。

1. $c_1, c_2, \ldots, c_N = 0, 0, \ldots, 0$ とする。
2. 各操作 j に対し、c_{p_j} に x_j を加算する。
3. $i = 2, \ldots, N$ の順に、c_i に c_{i-1} を加算する。
4. $c_1, c_2, \ldots, c_N$ を出力する。

そして、この解法は木に対してもほぼそのまま利用できます。具体的には、手順 3 の c_{i-1} を c_{q_i} (q_i は頂点 i の親) と読み換え~~ればそのまま元の問題への解法となります。~~、この処理を根に近い頂点から順に行えば元の問題への解法となります。(8/19 0:05 JST 修正) [*2] [*3]

[*1] N と Q がともに最大の 20 万の場合、$NQ = 400$ 億回。C++ などの高速な言語でも 2 秒間に処理できる加算の回数は数億回程度で、数百億回となると絶望的です

[*2] ~~ただし、このアルゴリズムは制約 $a_i < b_i$ を利用していることに注意してください。この制約がない場合は、~~ 各 i $(1 \leq i \leq N-1)$ に対して頂点 a_i が b_i の親であるとは限らず、(8/19 0:05 JST 修正) 再帰関数を用いるなどして明示的に根から順に処理を行う必要があります

[*3] このあたりの問題から、言語によっては「正しい」解法でも実行時間制限に間に合うかきわどいことがあるかもしれません。競技プログラミングに真剣に取り組む場合、このあたりで言語の乗り換えを検討し始めることをお勧めします

E: Strings of Impurity

(原案: sheyasutaka, 準備・解説: evima)

一般に、文字列 t が文字列 u の部分列であるかは t を u と貪欲にマッチさせれば判定できます。つまり、t の最初の文字が u に現れる最初の位置を見つけ、その位置より後ろで t の次の文字が u に現れる位置を見つけ、... という要領で t の最後の文字まで u の中に見つけられるかを試せばよく、今回は $u = s'$ (s の無限回の繰り返し) としてこの処理を効率的に行うことが求められています。

s' は無限に長いですが、t の長さは限られている点がポイントで、上の $|t|$ 回の処理の一回一回を高速に行えれば十分です。そのためには、まず `a` から `z` までの各文字種 ch に対して s 内の ch の出現位置をすべて抽出するべきでしょう。あとは周期性を利用し、$i = 0, \ldots, |s| - 1$ のそれぞれに対して $\mathrm{next}_{\mathrm{ch,i}}$:「$s'$ の i 文字目以降で文字種 ch が最初に現れる位置」を O((文字種数) $\times$ |s|) 時間かけて事前にすべて求めておくという方針や、二分探索を用いて ch の次の出現位置を毎回 $O(\log |s|)$ 時間かけて求めるという方針が考えられます。いずれにせよ、s を 2 個連結した文字列を用いると実装が楽になります。

F: Coincidence

(原案: camypaper, 準備・解説: evima)

y の MSB (Most Significant Bit: 最上位ビット) の位置を m とします (つまり、y の 2^m の位の桁は 1 で、任意の整数 n $(n > m)$ に対して x の 2^n の位の桁は 0 です)。ここで x の 2^m の位の桁が 0 であると仮定すると、XOR の定義より x XOR y の 2^m の位の桁は 1 となります。しかし、一般に整数 x, y $(y \geq x)$ に対して y を x で割った余りは y の半分未満であり[*4]、これは y を x で割った余りの MSB の位置が m 未満であることを意味するため、y を x で割った余りは x XOR y と決して一致しません。よって、x の MSB の位置も m である場合のみ考えれば十分です。このとき y を x を割った商は 1 であるため、y を x で割った余りは単に $y - x$ と書けます。以上を整理すると、答えは次の条件を満たす整数の組 (x, y) の個数です。

- $L \leq x \leq y \leq R$
- x と y の MSB の位置が同じである。
- $y - x = x$ XOR y

$y - x = x$ XOR y という条件は、$y - x$ という二進数の引き算において繰り下がりが起こらないことと同値であり、「どの整数 i に対しても、y の 2^i の位の桁が 0 であって x の 2^i の位の桁が 1 であるということはない」と書き換えられます (このとき $x \leq y$ は無条件に満たされ、この不等式は以後考慮する必要がなくなります)。あとはこの条件に加えてさらに「$L \leq x$」「$y \leq R$」「x と y の MSB の位置が同じ」 という三つの条件を満たす二本の 01 列を数える問題となり、これは動的計画法により $O(\log Y)$ 時間で解けます。詳細は実装例[*5]を参照してください。

[*4] y を x で割った商を q, 余りを r とすると $y = xq + r > r + r = 2r$ であるため

[*5] `f(pos, flagX, flagY, flagZ)`: x, y の 2^{pos} 以上の桁がすべて確定しており、そこまでで $L < x$ であることが確定しており (おらず)、$y < R$ であることが確定しており (おらず)、両者の MSB がすでに現れている (いない) 場合の残りの桁の決め方の数

ABC 138 Editorial

camypaper, evima, sheyasutaka, tozangezan, yosupo

August 18, 2019

A: Red or Not

(Original writer: tozangezan, preparation and editorial: evima)

This problem is like problem A of practice contest, but instead of addition, you are asked to write a conditional statement. For the ways of input and output, please refer to the sample code there, and then you can add a process to change the output depending on which of a and 3200 is larger, combining the if statement or conditional operator (e.g. `?:`) with comparison operator (e.g. `>=`), and that's it. (Sorry to end up with abstract explanation, but the margin of PDF is too narrow to explain the ways of each languages. Please make use of Google search)

The following is an implementation example in Python(3).

```
a = int(input())
s = input()
print(s if a >= 3200 else 'red')
```

B: Resistors in Parallel

(Original writer: sheyasutaka, preparation and editorial: evima)

In the explanation in sample input, $\frac{1}{A_1} + \frac{1}{A_2} + \ldots + \frac{1}{A_N}$ is calculated in fractional notation, but it would be easier to calculate using floating point type all the way. Then implementation can be done with looping structure like for-statement or something. On behalf of many languages, here are implementation examples by C++ and Python(3).

```cpp
#include <iomanip>
#include <iostream>
using namespace std;

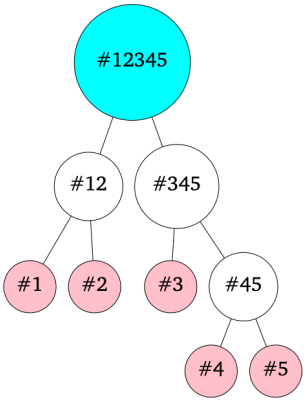
int main(){
    int N;
    double ans = 0.0, A;
    cin >> N;
    for(int i = 1; i <= N; ++i){
        cin >> A;
        ans += 1.0 / A;
    }
    ans = 1.0 / ans;
    cout << setprecision(16) << ans << endl;
}
```

```python
N = int(input())
A = map(int, input().split())
print('{:.16g}'.format(1 / sum(1 / x for x in A)))
```

C: Alchemist

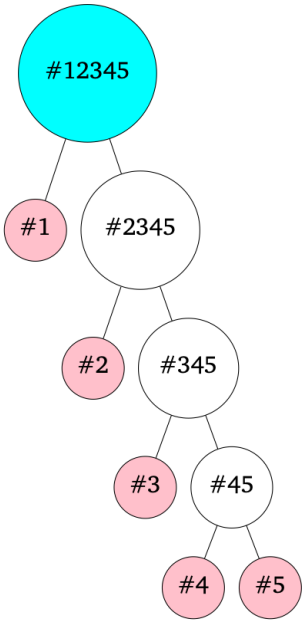
(Original writer: yosupo, preparation and editorial: evima)

Let's assume that ingredients $1, \ldots, N$ are indexed in the decreasing order of value (you can first sort the ingredients and change the indices in such way), and represent the process of composition with tree structures like the following diagram (where #1 denotes ingredient 1, and #12 denotes the ingredient that can be obtained by composing ingredients 1 and 2).



Let d_i be the distance from the leaf that denotes ingredients to the root (the number of edges between them), then the value of the final ingredients will be $v_1/2^{d_1} + \ldots + v_N/2^{d_N}$. To obtain the maximum result, you only have to think about the composition process such that $d_1 \leq d_2 \leq \ldots \leq d_N$ (otherwise, you can swap some indices of ingredients so that the total result will increase).

Moreover, to obtain the maximum result, it should be that $d_1 = 1, d_2 = 2, \ldots, d_{N-1} = d_N = N - 1$. This is because, briefly speaking, if you have a composition process such that $d_1 \leq d_2 \leq \ldots \leq d_N$ other than that (for example, like the diagram above), then you can increase the final result by *transplanting the violating "left-side child" nearest to the root under the "right-side child" farthest from the root.*


#12345
#1
#2345
#2
#345
#3
#45
#4
#5

D: Ki

(Original writer: tozangezan, preparation and editorial: evima)

An operation can affect at most all the N vertices, so if you simulate the operations naively, you will have to apply addition to the counters NQ times in total, and it won't fit in the time limit. [*1]

For the tree-like problem like this, sometimes it is helpful to think about linear tree like $1-2-3-\ldots-N$ instead of general tree, and this problem is a good example. Assuming that the graph is linear, a linear-time solution like the following is available:

1. *Let $c_1, c_2, \ldots, c_N = 0, 0, \ldots, 0$.*
2. *For each operation j, add x_j to c_{p_j}.*
3. *For each $i = 2, \ldots, N$, in this order, add c_{i-1} to c_i.*
4. *Output $c_1, c_2, \ldots, c_N$.*

And this solution can be also applied to trees. Specifically, if you replace the c_{i-1} in the third step with c_{q_i} (where q_i is parent of node i), ~~it will directly be the solution for the original problem.~~ and do the operations from the vertices nere to the root, the original problem can be solved. (8/19 0:05 JST corrected) [*2] [*3]

[*1] If both N and Q are at maximum of 200 thousands, NQ = 40 billion times. Even fast languages like C++ can perform addition at most several hundreds million time, and when it comes to tens of millions of times, it's hopeless

[*2] ~~Note that this algorith, relies on the constraints $a_i < b_i$. Without this constrains,~~ For each i ($1 \leq i \leq N-1$), vertex a_i may not be the parent of b_i, (8/19 0:05 corrected) you have to explicitly perform the operations from the root to children, with recurrence functions or something

[*3] From this problem on, depending on programming languages, even the "correct" solution may not be fast enough to meet the time limit. If you tackle competitive programming seriously, we recommend you think of altering the programming language

E: Strings of Impurity

(Original writer: sheyasutaka, preparation and editorial: evima)

Generally, in order to judge if a string t is a substring of u, you can greedily match t with u. That is, find the first appearance of the first letter of t in u, then find the second letter of t in u after the first match, ... and so on, until checking if the last letter of t appears in u, and this time, it is required to do this operations efficiently under $u = s'$ (an infinite repetition of s).

The key point is that, while s' is infinitely long, the length of t is limited, so it is sufficient to perform each operation of $|t|$ operations mentioned above quickly enough. To do so, for each letter ch from `a` to `z`, you should extract all the appearance of ch in s. Then you can precalculate $\mathrm{next}_{\mathrm{ch,i}}$*: "the first appearance of letter ch after the i-th character of s'" for each $i = 0, \ldots, |s| - 1$ with* $\mathrm{O}((\mathrm{thenumberofkindsofcharacters}) \times |\mathrm{s}|)$ *time, or use binary search to find the position of the next appearance of ch with* $\mathrm{O}(\log |s|)$ *time each. Anyway, if you use 2-time repetition of s, the implementation will be easier.*

F: Coincidence

(Original writer: camypaper, preparation and editorial: evima)

Let y's MSB (Most Significant Bit) be m (in other words, 2^m's place of y is 1, and for any integer n ($n > m$), 2^n's place of x is 0). Here, assuming that 2^m's place of x is 0, then by the definition of XOR, 2^m's place of x XOR y will be 1. However, generally for any integers x, y ($y \geq x$), the remainder of y divided by x is less than half of y[*4], which means that the MSB of remainder of y divided by x is less than m, so the remainder of y divided by x does never corresponds to x XOR y. Therefore, we can always assume that x's MSB is also m. Here, the quotient of y divided by x is 1, so the remainder of y divided by x can simply be denoted as $y - x$. The answer is the number of pairs of integers (x, y) that meets the following conditions.

- $L \leq x \leq y \leq R$
- The MSB position of x and y are the same.
- $y - x = x$ XOR y

The condition $y - x = x$ XOR y is equivalent to the condition that there are no borrowings in binary subtraction of $y - x$, and in other words, "for any integer i, it will never occur that 2^i's place of y is 0 and 2^i's place of x is 1 (in such case, $x \leq y$ always holds, and you don't have to care about it anymore). The remaining task is to count the number of pairs of 01-sequences that meets the three conditions, "$L \leq x$", "$y \leq R$" and "the MSB position of x and y are the same", and this can be solved in a total of $O(\log Y)$ time with DP. For details, please refer to the implementation example[*5].

[*4] Let q be the quotient of y divided by x, and r be the remainder, then $y = xq + r > r + r = 2r$ holds

[*5] `f(pos, flagX, flagY, flagZ)`: The number of ways to determine the remaining digits where the 2^{pos}'s place and higher of x and y is already determined, it's already (still not) confirmed that $L < x$, it's already (still not) confirmed that $y < R$, and their MSB has already (still not) been appeared

ABC 139 解説

drafear, E869120, evima, gazelle, square1001, wo01, yuma000

2019 年 9 月 1 日

For International Readers: English editorial starts on page 9.

A: Tenki

S と T を比較して、一致する文字がいくつあるかを数えればよいです。
この問題では文字列が固定長なので、ループを回さなくても場合分けで正答することができます。

Listing 1　C++ による実装例

```
#include<bits/stdc++.h>
using namespace std;

int main() {
  string s, t;
  cin >> s >> t;
  int ans = 0;
  if(s[0] == t[0]) ans++;
  if(s[1] == t[1]) ans++;
  if(s[2] == t[2]) ans++;
  cout << ans << endl;
  return 0;
}
```

B: Power Socket

B 口以上になるまで、電源タップを 1 つずつ使うシミュレーションを行うことで答えを求めることができます。すなわち、最初差込口を 1 口として、差込口が B 口未満である間、電源タップ 1 つと差込口 1 口を使って差込口を A 口増やすことを続けます。これを C++ 言語で実装した例を以下に示します。

```cpp
#include <bits/stdc++.h>

using namespace std;

int main() {
  int A, B; cin >> A >> B;
  int ans = 0;
  int outlet = 1;
  while (outlet < B) {
    --outlet;
    outlet += A;
    ++ans;
  }
  cout << ans << endl;
}
```

他の解法としては、電源タップを 1 つ使うごとに差込口が $A-1$ 口増え、最終的に初期状態から $B-1$ 口増やしたいと考えれば、答えは $\lceil \frac{B-1}{A-1} \rceil$ ($B-1$ を $A-1$ で割った切り上げ) になります。$\lceil \frac{B-1}{A-1} \rceil = \lfloor \frac{B-1+A-2}{A-1} \rfloor$ なので、切り捨てで計算することもできます。

さらなる別解 (苦肉の策) として、入力の約 400 通りを手計算して埋め込んでも正答することができますが、おすすめはしません。

C: Lower

i 番目のマスに降り立つと x_i マス進めるとします。すると、答えは $\max\{x_1, x_2, ..., x_N\}$ になりますが、愚直に求めようとすると時間計算量が $O(N^2)$ となり間に合いません。この数列 $\{x_i\}$ を眺めると、$\{3, 2, 1, 0, 2, 1, 0, 0, 5, 4, 3, 2, 1, 0\}$ のようになるため、まず x_1 を愚直に求めて、次に x_{x_1+2} を求めて、... としていけば $O(N)$ で求めることができます。数列を眺めなくとも、左端から x_1 マス進めるなら、その途中のマスからスタートしてもより良くはならないため、左端からスタートし、進めるまで進み、今度は次のマスからスタートした場合を考え、... と考えることができます。

他の考え方としては、$x_i \geq x_{i+1}$ なら $i+1$ 番目のマスに降り立つよりも i 番目のマスに降り立ったほうがより良いため、そのようなマスから始めた場合を求めない、といった考え方もあります。

D: ModSum

A を B で割った余りを $A \bmod B$ と表すことにすると

$$(1 \bmod 2) + (2 \bmod 3) + ... + ((N-1) \bmod N) + (N \bmod 1) = 1 + 2 + ... + N - 1 = \frac{N(N-1)}{2}$$

と順列を選ぶのが最適です。以下、これを説明します。

選んだ順列には $1, 2, ..., N$ がそれぞれちょうど 1 回ずつ登場します。$i = 1, 2, ..., N$ について、i が順列の x_i 番目に登場するとします。すると、目的関数 (最大化したいもの) は

$$(x_1 \bmod 1) + (x_2 \bmod 2) + ... + (x_N \bmod N)$$

になります。各項に着目すると、それぞれの項は最大でも $0, 1, 2, ..., N-1$ ですが、$i \geq 2$ について $x_i = i - 1$ とし、余った N を $x_1 = N$ とすると実際に

$$(N \bmod 1) + (1 \bmod 2) + ... + ((N-1) \bmod N) = 0 + 1 + 2 + ... + N - 1$$

とできるため、目的関数の最大値は $0 + 1 + 2 + ... + N - 1 = \frac{N(N-1)}{2}$ です。

E: League

(原案: wo01, 準備・解説: evima)
二つの条件は次のようにまとめられます。

- 各 i, j に対し、選手 i 対 選手 $A_{i,j+1}$ の試合は選手 i 対 選手 $A_{i,j}$ の試合を行った日の翌日以降にしか行えない。

すなわち、選手自体にさほど重要性はなく、単に何個かの試合のペアに対してどちらを先に行うべきかが定められていると考えられます。

以下、$N(N-1)/2$ 試合のそれぞれを頂点に見立て、試合 y が試合 x を行った翌日以降にしか行えないときに辺 $x \to y$ が存在するような有向グラフを考えます。このグラフに閉路が存在する場合は条件を満たせません。閉路が存在しない場合は、毎日、入次数が 0 であるような頂点 (辺が“刺さって”いない頂点) に対応する試合をすべて行ってそれらの頂点とそれらから出る辺をすべて削除すれば、最小の日数で全試合を行えます。(入次数が 0 であるような頂点に対応する試合を翌日以降に繰り越す意義がないため。) この日数はグラフに存在する最長のパスに含まれる頂点の数に等しくなります。

以上から、閉路検出と最大パス長の算出を深さ優先探索により行うことで、頂点と辺の総数に対して線形時間、すなわち $\mathrm{O}(N^2)$ 時間でこの問題を解くことができます。

F: Engines

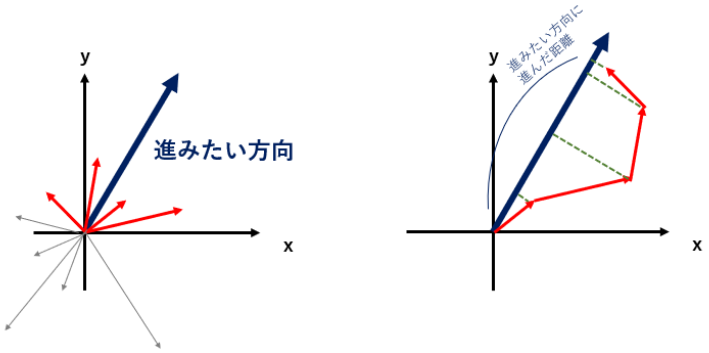
(原案: square1001 + E869120, 準備・解説: square1001 + E869120)

★ 謝辞

高校数学を理解していない人には少し分かりにくい解説となっておりますが、最大限分かりやすく説明した限りですので、ご容赦ください。

★ 全体的な方針

さて、最終的にベクトル (A, B) の方向にできるだけ進むことを考えます。この場合、座標 (X, Y) から座標 $(X + x_i, Y + y_i)$ に移動するエンジンについて、「ベクトル (x_i, y_i) の方向に進むエンジン」とします。その時、ベクトル (A, B) とベクトル (x_i, y_i) のなす角が 90 度未満であるエンジンだけを使えば、最大限 (A, B) の方向に進むことができます。



例えば上の図において、「進みたい方向に進んだ距離」 を最大化するためには、赤のベクトルを持つエンジンだけを選べばよいです。もう少し分かりやすい例を出しましょう。

- X 軸の正の方向に進みたい場合（ベクトル $(1, 0)$ の方向に進みたい場合)、X 方向に進む距離が正であるエンジン（つまり、$x_i > 0$ であるエンジン）だけを選べばよいです。
- Y 軸の負の方向に進みたい場合は、Y 方向に進む距離が負であるエンジン（つまり、$y_i < 0$ であるエンジン）だけを選べばよいです。
- X 軸の正方向から反時計回りに 45 度の方向に進みたい場合（つまりベクトル $(1, 1)$ の方向に進みたい場合)、$x_i + y_i \geq 0$ であるエンジンだけを選べばよいです。

これらは全て、**進みたい方向とエンジンの進む方向の成す角が 90 度以内であるエンジンを選ぶという例です。**

★ まず、どうやって成す角が 90 度未満かを判定するか

(ax, ay) の方向（ベクトル）と、(bx, by) の方向（ベクトル）のなす角が 90 度未満である条件は、$axbx + ayby > 0$、つまりベクトルの内積が正であるという非常に簡単なものです。競プロだけでなく、高校数学とかでも良く使われるので、覚えておきましょう。

そこで、いくつか方針を考えてみることを考えます。

★ 方針 A: 進む方向を全探索する（TLE 解法）

まず、最も簡単な解法として、進む方向（あるいはベクトル）を全探索することを考えます。しかし、制約は以下の通りとなっています。

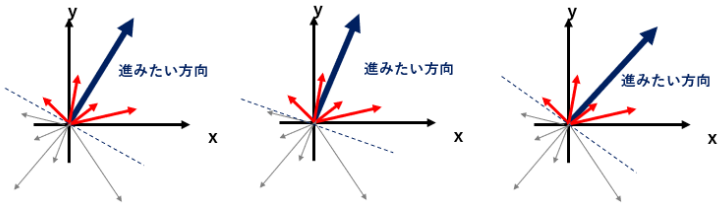
1. $N \leq 100$
2. $-1\ 000\ 000 \leq x_i, y_i \leq 1\ 000\ 000$

ですので、最大で座標 $(100\ 000\ 000, 100\ 000\ 000)$ まで進む可能性があります。進むベクトルの通り数はおおよそ 4×10^{16} 通りになるため、TLE します。

★ 方針 B: 進む方向を絞る

一個重要な考察があります。これは、「必ず進む方向と最終的な位置が一致しなくても、選ぶエンジンが最適であれば問題がない」という考え方です。

例えば、以下の図において、3 つの進む方向（青の矢印）について、全て同じエンジンの集合を選ぶことになります。



ですので、進む方向を絞る事はできないのでしょうか。例えば、進む方向を $1 \leq i \leq N$ に対して、(x_i, y_i)、$(-y_i, x_i)$, $(-x_i, -y_i)$, $(y_i, -x_i)$ だけに絞ると、高々 $O(N)$ 通りだけで済みます。各進む方向について N 回の計算が必要ですので、$O(N^2)$ が達成できます。普通に上の 4 パターンだけを追加すると WA するので、境界を上手く場合分けすることが必要です。次の方針は、もう少し実装や境界判定が簡単なものです。

★ 方針 C: 「角度でソート」を利用する

ここでは、エンジン i が x 軸の正の方向から見て反時計回りに何度回ったかを示す値を r_i とします。例えば、$(1, 2)$ の方向に進むエンジンの場合、r_i の値はおおよそ 64 度くらいになります。

さて、この解説の 2 つの図に注目していただきたいのですが、全て $L \leq r_i \leq R$ を満たすエンジンのみ選ばれています。(ただし、360 度と 0 度の境界を通り過ぎる場合、$L \leq r_i$ または $r_i \leq L$ となります。)

つまり、エンジンの進む方向の角度でエンジンをソートすることにしましょう。(豆知識ですが、角度でソートすることを「偏角ソート」と言い、$O(N \log N)$ でできます。) そうすると、選ぶエンジンの集合は一つの区間になります。

最後に選ぶ区間を全探索すると、$O(N^3)$ 或いは $O(N^2)$ でこの問題を解くことが出来ます。上手く尺取り法を使うと、$O(N \log N)$ で解くことができますが、境界判定や実装が大変です。

★ サンプルコード (C++)

https://atcoder.jp/contests/abc139/submissions/7244437

ABC 139 Editorial

drafear, E869120, evima, gazelle, square1001, wo01, yuma000

September 1, 2019

A: Tenki

You can compare S and T and count the number of corresponding characters.

In this problem the length of string is fixed, so you can get AC by splitting case, without using loop.

Listing 1 Sample Implementation in C++

```
#include<bits/stdc++.h>
using namespace std;

int main() {
  string s, t;
  cin >> s >> t;
  int ans = 0;
  if(s[0] == t[0]) ans++;
  if(s[1] == t[1]) ans++;
  if(s[2] == t[2]) ans++;
  cout << ans << endl;
  return 0;
}
```

B: Power Socket

You can find the answer by the simulation of using power strips one by one, until the number of sockets becomes more than or equal to B. Namely, first assume that there are 1 socket, and while there are less than B empty sockets, repeat using one power strip and one empty socket so as to increase the number of empty socket by A. The following is an implementation example in C++.

```cpp
#include <bits/stdc++.h>

using namespace std;

int main() {
  int A, B; cin >> A >> B;
  int ans = 0;
  int outlet = 1;
  while (outlet < B) {
    --outlet;
    outlet += A;
    ++ans;
  }
  cout << ans << endl;
}
```

Another solution is like this: every time you use one power strip, the number of empty socket increases by $A-1$, and you ultimately want to increase the number of power socket by $B-1$ compared to the beginning, so the answer will be $\lceil \frac{B-1}{A-1} \rceil$ ($B-1$ divided by $A-1$, rounded up). Since $\lceil \frac{B-1}{A-1} \rceil = \lfloor \frac{B-1+A-2}{A-1} \rfloor$ holds, so it can be calculated rounded-down.

As one more solution (or last resort), you can get AC by calculating all the 400 patterns by hand and embed into the source code, but it's not recommended.

C: Lower

Assume that if you land on i-th square you can move x_i times. Then the anwser will be $\max\{x_1, x_2, ..., x_N\}$, but if you try to calculate naively, it will need a total of $O(N^2)$ time and it won't finish in time. If you look at the sequence $\{x_i\}$, it appears that it will be like $\{3, 2, 1, 0, 2, 1, 0, 0, 5, 4, 3, 2, 1, 0\}$, so you can compute x_1 naively, then compute x_{x_1+2} naively, ... and so on, you can find the answer in a total of $O(N)$ time. Or otherwise, without looking at the sequence, you can think like the following: if you can move x_1 times from the leftmost squares, you can't move more times if you start halfway, so think of starting from the next square, .. and so on.

Another idea is that, if $x_i \geq x_{i+1}$, then it is better to land on i-th square instead of $i + 1$-th square, so avoid computing the case of starting from such squares.

D: ModSum

Let A mod B be the remainder of A devided by B, then it is optimal to choose the permutation such that

$$(1 \bmod 2) + (2 \bmod 3) + ... + ((N-1) \bmod N) + (N \bmod 1) = 1 + 2 + ... + N - 1 = \frac{N(N-1)}{2}.$$

The following is the explanation.

In the chosen permutation, each $1, 2, ..., N$ appears exactly once. For $i = 1, 2, ..., N$, assume that i is x_i-the element of the permutation. Then, the objective function (the one we want to maximize) is

$$(x_1 \bmod 1) + (x_2 \bmod 2) + ... + (x_N \bmod N).$$

If you focus on each term, it appears that maximum of each term is $0, 1, 2, ..., N-1$. Let $x_i = i-1$ for $i \geq 2$, and for the remaining N, let $x_1 = N$, then

$$(N \bmod 1) + (1 \bmod 2) + ... + ((N-1) \bmod N) = 0 + 1 + 2 + ... + N - 1$$

actually holds, so the maximum value of the objective function is $0+1+2+...+N-1 = \frac{N(N-1)}{2}$.

E: League

(Original writer: wo01, preparation and editorial: evima)

The two conditions can be simplified as follows:

- For each i, j, the match of Player i vs Player $A_{i,j+1}$ can only be scheduled one or more days after the day of the match of Player A_i and Player $A_{i,j}$.

Namely, the players themselves are not so important, and we can assume that the only constraint is the order of schedule between some pairs of matches.

Let's assume a directed graph consisting of $N(N-1)/2$ vectors, each of which corresponds to each match, such that if match y can only be scheduled one or more days after the match x, there exists an edge $x \to y$. If this graph contains any cylces, it is impossible to meet the conditions. If there are no cycles in it, you can schedule all the matches in minimum days in the following way: for each day, do all the matches whose corresponding vectors' indegree is0 (the vectors without any incoming edges), and remove the vectors and all the outgoing edges from them. (This is because it is meaningless to postpone the matches whose corresponding vectors' indegree is 0.) The minimum number of days is equal to the number of vectors in the longest path in the graph.

Therefore, if you scan cycles and calculate the length of longest path with DFS, you can solve the problem in a total of linear time of the number of vectors and edges, that is, in a total of $\mathrm{O}(N^2)$ time.

F: Engines

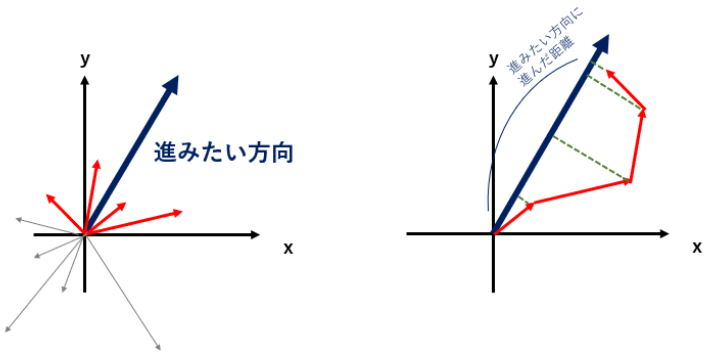
(Original writer: square1001 + E869120, preparation and editorial: square1001 + E869120)

★ Acknowledgment

The editorial may be a little bit difficult because it is explained in mathematical ways, but we tried to explain as easy as possible, so please forgive us.

★ Main idea

Let's try to move towards the direction of vector (A, B) as much as possible. In this case, we call the engine that enables him to move from coordinates (X, Y) to $(X + x_i, Y + y_i)$ "the engine that moves in the direction of vector (x_i, y_i)." Then, if he use the engine such that the angle between vector (A, B) and vector (x_i, y_i) is no more than 90 degrees, he can move towards the direction of (A, B) as much as possible.



For example, in order to maximize the "progressed distance towards the desired direction" ("the desired direction" is drawn with blue arrow in the diagram above), it is sufficient if you only use the red vectors. The following example may be easier to understand:

- If he wants to move towards the positive X direction (towards the direction of vector $(1, 0)$), it is sufficient to choose such engines that enables him to move positively in the X direction (that is, engines such that $x_i > 0$).
- If he wants to move towards the negative Y direction, it is sufficient to choose such engines that enables him to move negatively in the Y direction (that is, engines such that $y_i < 0$).
- If he wants to move towards the direction of X axis rotated by 45 degrees counterclockwise (towards the direction of vector $(1, 1)$), it is sufficient to choose such engines that $x_i + y_i \geq 0$.

All of them are the examples of **choosing the engines such that the angle between its progression direction and the desired direction is no more than 90 degrees**.

★ First, how to judge if the angle between two vectors are less than 90 degrees?

The condition of the angle between the direction (vector) (ax, ay) and the direction (vector) (bx, by) is less than 90 degrees is that $axbx + ayby > 0$, or that inner product of two vectors is positive, which is pretty easy. It is commonly used not only in competitive programming but also other mathematical problems, so it will be good for you to remember.

Let's think about several solutions.

★ Solution A: Brute force all the "desired directions" (TLE solution)

First, the most naive solution is to brute force all the desired directions (or vectors). However, the constraints are the following:

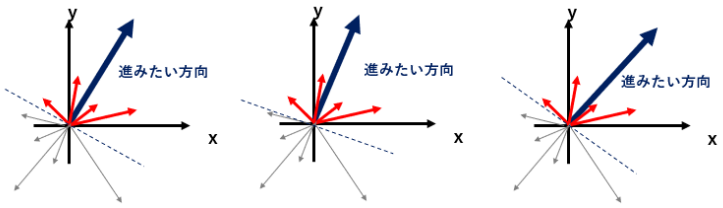
1. $N \leq 100$
2. $-1\ 000\ 000 \leq x_i, y_i \leq 1\ 000\ 000$

It means that there is a possibility of ending up with coordinates $(100\ 000\ 000, 100\ 000\ 000)$ at most. There are about 4×10^{16} possible desired vectors, so it will lead to TLE.

★ Solution B: Limit the number of desired directions

One important point is that "even if the desired direction and the ultimate position does not coincide, it's no problem if the selected engines are optimal."

For example, in the diagram below, no matter which of those three desired directions (the blue arrows) you choose, the selected set of engines are all the same.



Then isn't it possible to limit the number of desired directions? For example, if you limit the desired direction to (x_i, y_i), $(-y_i, x_i)$, $(-x_i, -y_i)$ and $(y_i, -x_i)$ for $1 \leq i \leq N$, the number of desired directions are at most $O(N)$. If you only add those 4 patterns it will end up with WA, so you have to split border cases properly. If you choose the solution described below, the implementation and border judgement will be a little bit easier.

★ Solution C: Use "sort by angles"

Let i be the angle from x-axis to the direction of engine i, rotated counterclockwise. For instance, the r_i value for the engine that enables him to move in the direction of $(1, 2)$ is about 64 degrees.

Pay attention to the two diagrams in the editorial, then you see that only the engines such that $L \leq r_i \leq R$ are selected. (However, if it passes the border of 0 degrees and 360 degrees, it will hold that $L \leq r_i$ or $r_i \leq L$.)

Therefore, let's sort the engines by the angles of directions of them. (Tip: this sort can be achieved in $O(N \log N)$.) Then, the set of engines will be a segment of the sorted sequence.

Finally, if you brute force the segment to choose, you can solve it in a total of $O(N^3)$ or $O(N^2)$ time. If you use sliding window properly, it can be solved in $O(N \log N)$, but border judge and implementation is fairly hard.

★ Sample Code (C++)

https://atcoder.jp/contests/abc139/submissions/7244437

ABC 140 解説

beet, drafear, gazelle, kort0n

2019 年 9 月 7 日

For International Readers: English editorial starts on page 7.

A: Password

各桁ごとに独立して N 通りの数字を設定出来ますから、答えは N^3 です。
N を入力として受け取り、 N^3 を計算して、その値を出力することにより、AC となります。
以下は C++ による実装例です。

```
#include <iostream>
using namespace std;
int main(){
  int N;
  cin >> N;
  cout << N * N * N << endl;
  return 0;
}
```

B: Buffet

次のようにシミュレーションを行うことで答えを求めることができます。まず、A_1 種類目の料理を食べるので、B_{A_1} の満足度を得ます。次に、A_2 種類目の料理を食べるので、B_{A_2} の満足度と、$A_2 = A_1 + 1$ ならば C_{A_1} の満足度を得ます。i $(2 \leq i \leq N)$ 番目には A_i 種類目の料理を食べるので、B_{A_i} の満足度と $A_i = A_{i-1} + 1$ ならば $C_{A_{i-1}}$ の満足度を得ます。これを C++ 言語で実装した例を以下に挙げます。

```cpp
#include <bits/stdc++.h>

using namespace std;

int main() {
  int N; cin >> N;
  // input
  vector<int> A(N);
  for (int i = 0; i < N; ++i) {
    cin >> A[i];
    --A[i];
  }
  vector<int> B(N);
  for (int i = 0; i < N; ++i) {
    cin >> B[i];
  }
  vector<int> C(N-1);
  for (int i = 0; i < N; ++i) {
    cin >> C[i];
  }
  // calc
  int ans = 0;
  for (int i = 0; i < N; ++i) {
    ans += B[A[i]];
    if (i > 0 && A[i] == A[i-1]+1) {
      ans += C[A[i-1]];
    }
  }
  // output
  cout << ans << endl;
}
```

C: Maximal Value

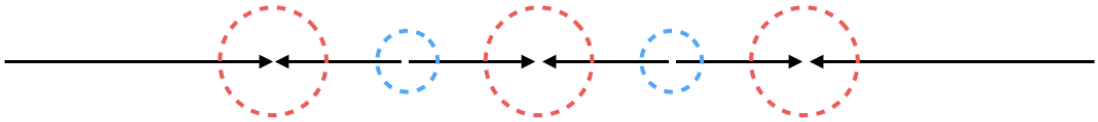
長さ N の数列 C を、

$$
\begin{aligned}
C_1 &= B_1 \\
C_i &= \min\left(B_{i-1}, B_i\right)\ (i = 2, 3, \cdots, N-1) \\
C_N &= B_N
\end{aligned}
$$

で定めます。
このとき、任意の i について $A_i \leq C_i$ が成立しなければなりません。実際、 $i = 2, 3, \cdots, N-1$ について $A_i > C_i$ であったと仮定すると、$A_i > B_{i-1}$ または $A_i > B_i$ が成立しますが、 $A_i > B_{i-1}$ であれば $B_{i-1} \geq \max\left(A_{i-1}, A_i\right)$ に矛盾し、 $A_i > B_i$ であれば $B_i \geq \max\left(A_i, A_{i+1}\right)$ に矛盾します。$i = 1, N$ のときも、矛盾することが分かります。
一方、 $A = C$ とすると、これは問の条件を満たすことが分かります。以上より、 問の条件を満たす数列のうち要素の総和が最大となる数列は、 C です。
実装上は、$B_0 = B_N = \infty$ (十分に大きな値) とすると、場合分けが不要で楽です。

D: Face Produces Unhappiness

人の向きを以下の図のように矢印で表すことにします。この図では、同じ向きの人が連続している部分の矢印を繋げて 1 つの矢印で表しています。また、簡単のため、N 人の左側には右向きの人が無限に並んでいて、右側には左向きの人が無限に並んでいることとします。



幸福である人数を数える代わりに、幸福でない人を数えましょう。すなわち、上図の赤い点線で囲った部分の数をできるだけ減らしたいです。赤い点線の部分 1 箇所につき幸福でない人は基本的には 2 人ですが、この部分が (N 人の) 左端や右端にある場合には 1 人です。

(l, r) を選んで反転する操作を行うと、これらの赤・青の点線の部分はどのように変化するでしょうか。反転した区間の端以外では、増えも減りもせず、赤、青の点線の部分はそれぞれ赤、青の点線のままです。したがって、1 回の操作では選ぶ区間の端として赤い点線部分と青い点線部分を選び、赤い点線部分を 1 箇所を消すことができますが、それ以外の選び方では消すことができません。

また、左には右向きの人が無限に並んでいて、右には左向きの人が無限に並んでいることから、全体として赤または青の点線の部分の数は必ず奇数で、赤、青、赤、青、…、赤と並んでいます。これから分かることとして、赤い点線部分が M 箇所あるとすると、青い点線部分は $M-1$ 箇所であるため、何回操作ができたとしても赤い点線部分は必ず 1 箇所残り、また、$M-1$ 回の操作で $M-1$ 箇所の赤い点線部分を消すことができます。したがって、可能な限り内側の (端でない) 赤い点線部分を消し (1 回あたり幸福な人は 2 人増える)、最後に残った赤い点線部分が内側にあるならそれを端に移動させる (幸福な人は 1 人増える) のが最適です。なぜなら、1 回の操作で幸福な人は高々 2 人しか増えませんが、実際に 2 人ずつ増やすことができるからです。

すなわち、結局は幸福な人 $+2$ 人を X 回でき、$+1$ 人を Y 回でき、K 回までの操作で何人幸福な人を増やせるか、といった問題になります。X, Y は赤い点線部分の数、すなわち S 中に登場する 'RL' の数と、S_1, S_N により定まります。この解法の時間計算量は $O(N)$ です。

別解として、文字列 S を LL...L, RR...R, ..., LL...L のように分解し、奇数番目の RR...R または LL...L の塊について前から順にそれぞれ 1 回の操作で向きを反転させていき、最後に幸福な人数を数える方法もあり、こちらも $O(N)$ で動作します。

E: Second Sum

P の各要素について、$P_i = X_{L,R}$ となるような組 (L, R) の個数を C_i とすると、

$$\sum_{L=1}^{N-1} \sum_{R=L+1}^{N} X_{L,R} = \sum_{i=1}^{N} P_i \times C_i$$

となります。また、$P_i = X_{L,R}$ となる組 (L, R) について、P が順列であることから、$L \leq i \leq R$ を満たします。以降、C_i を求めます。

集合 $S_i = \{j | j < i, P_i < P_j\}, T_i = \{j | j > i, P_i < P_j\}$ を考えます。S_i の中で二番目に大きい要素を w_i、最大の要素を x_i とし、T_i の中で最小の要素を y_i、二番目に小さい要素を z_i とします。S_i, T_i はインデックスの集合であることに注意してください。このとき、$w_i < x_i < i < y_i < z_i$ です。$P_i = X_{L,R}$ となるのは、$w_i < L \leq x_i < i \leq R < y_i < z_i$ のときと、$w_i < x_i \leq L < i < y_i \leq R < z_i$ のときに限られることがわかります。したがって、$C_i = (x_i - w_i) \times (y_i - i) + (i - x_i) \times (z_i - y_i)$ となります。

P の要素を大きい順番に見ていくことにすると、 ある時点までに見た全ての要素は、その時点で見ている要素より大きい要素になっています。したがって、順序集合を扱うデータ構造 (C++ における set, multiset など) を用いてインデックスの集合を管理することで、w_i, x_i, y_i, z_i は 各 i に対し $O(\log N)$ で求めることができます。また番兵として S_i に 0 を二つ、T_i に $N+1$ を二ついれておくと、境界条件を簡潔に実装することができます。全体の計算量は $O(N \log N)$ です。

F: Many Slimes

この問題は以下の問題と等価です。

> 深さ N の完全二分木を考える。
> 葉に集合 S の要素を 1 つずつ書く。また、葉以外の頂点について、子に書かれた数の最大値をボトムアップに書いていく。
> 葉以外の任意の頂点について、子に書かれる数が相違なるように葉への要素の割当を決めることができるか。

S の大きい要素から (同じ数はまとめて) 葉への割当を決めていくことを考えます。

また、要素を割り当てた葉から根までのパス上の頂点を毎回黒く塗るものとします。

ある同じ数を 2 つの葉に割り当てるとき、2 つの葉を結ぶパス上に黒く塗られた頂点がまだなければ、2 つの葉の LCA で条件が満たされなくなるので不適です。見方を変えると、これは黒い頂点を取り除いて木を分割したとき、同じ連結成分の葉に割り当ててしまうことと等価です。逆にこのような葉のペアがなければ、その割当は条件を満たしています。

この事実を考えると、連結成分をできるだけ多くしておきたいので、頂点数の多い連結成分の葉から貪欲に養素を割り当てていくアプローチが有効です。このとき最後まで上のような事態が起こらなければ答えは可能、起これば不可能です。

ABC 140 Editorial

beet, drafear, gazelle, kort0n

Semptember 7, 2019

A: Password

You can independently determine N kinds of number for each digit, so the answer is N^3. You can receive N as input, calculate N^3, and print it to get AC. The following is an implementation example by C++.

```
#include <iostream>
using namespace std;
int main(){
  int N;
  cin >> N;
  cout << N * N * N << endl;
  return 0;
}
```

B: Buffet

You can find the answer by performing simulation as follows: First, he will eat A_1-th dish, so he will gain B_{A_1} satisfaction points. Next, he will eat A_2-th dish, so he will gain B_{A_2} satisfaction points. and if $A_2 = A_1 + 1$, then he will gain C_{A_1} satisfaction points. The i-th dish ($2 \leq i \leq N$) he will eat is A_i-th dish, so he will gain B_{A_i} satisfactory points, and if $A_i = A_{i-1} + 1$, then he will gain $C_{A_{i-1}}$ satisfactory points. The implementation example in C++ language is given as follows.

```cpp
#include <bits/stdc++.h>

using namespace std;

int main() {
  int N; cin >> N;
  // input
  vector<int> A(N);
  for (int i = 0; i < N; ++i) {
    cin >> A[i];
    --A[i];
  }
  vector<int> B(N);
  for (int i = 0; i < N; ++i) {
    cin >> B[i];
  }
  vector<int> C(N-1);
  for (int i = 0; i < N; ++i) {
    cin >> C[i];
  }
  // calc
  int ans = 0;
  for (int i = 0; i < N; ++i) {
    ans += B[A[i]];
    if (i > 0 && A[i] == A[i-1]+1) {
      ans += C[A[i-1]];
    }
  }
  // output
  cout << ans << endl;
}
```

C: Maximal Value

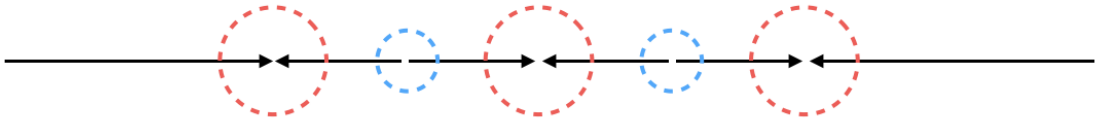
Let's define a sequence C by

$$\begin{aligned} C_1 &= B_1 \\ C_i &= \min(B_{i-1}, B_i) \ \ (i = 2, 3, \cdots, N-1) \\ C_N &= B_N \end{aligned}$$

Then, for each i, $A_i \leq C_i$ should hold. For some $i = 2, 3, \cdots, N-1$, if $A_i > C_i$ holds, then $A_i > B_{i-1}$ or $A_i > B_i$ holds, but if $A_i > B_{i-1}$, then it contradicts to $B_{i-1} \geq \max(A_{i-1}, A_i)$, and if $A_i > B_i$ holds, then it contradicts to $B_i \geq \max(A_i, A_{i+1})$. For $i = 1, N$, it also contradicts.
On the other hand, if you let $A = C$, then it meets the conditions of the problem statement. Therefore, the sequence with maximum sum that meets the conditions of problem is C.
For implementation, if you let $B_0 = B_N = \infty$ (a number big enough), you don't have to split case, so it's easy.

D: Face Produces Unhappiness

Let's represent the directions of the people with arrows like the following diagram. In this diagram, a continuous sequence of people with same directions are represented in one arrow. Also, for simplicity, assume that there are infinite people facing right in the left side of N people, and there are infinite people facing left in the right side.



Instead of counting happy people, let's count the number of unhappy people. That is, you want to decrease the number of places surrounded by red dotted circle in the diagram above. For each place with red dotted circle, there are basically two unhappy people, except when it is at the leftmost or rightmost (of N people), in which case there are one unhappy people.

What will happen to those red and blue dotted places when you perform the selection of choosing (l, r) and reversing? At the places other than the endpoints of the segment, the red and blue dotted part remains the same. Therefore, in one operation you can choose red dotted place and blue dotted place and remove one red dotted part, but otherwise you cannot remove.

Also, there are infinite number of people facing right at the left, and there are infinite number of people facing left at the right, so overall there are always odd number of red or blue dotted place, lined up in an order of red, blue, red blue, $\cdots$, red. From this fact we can see that, assume that there are M red-dotted part, then there are $M-1$ blue-dotted part, so no matter how many operations you can perform, there remains at least one red-dotted part, and you can remove $M-1$ red part in $M-1$ operations. Therefore, the optimal strategy is, first erasing red-dotted part (not at endpoint) as much as possible (so that the number of happy people increases by two), and at last, if there exists a red-dotted part inside, move to the endpoint (so that the number of happy people increases by one). This is because you can increase at most two happy people in each operation, and you can actually increase it by two.

Therefore, the problem can be rephrased by: you can perform an operation of increasing happy people by $+2$ for X times, by $+1$ for Y times, then what is the maximum number of happy people you can increase by performing K operations? X, Y is determined by the number of red-dotted point, that is, the number of 'RL' appears in S, and S_1, S_N. The overall time complexity of this solution is $O(N)$.

As another solution, you can split S like LL...L, RR...R, ..., LL...L, reverse the cluster of RR...R or LL...L at the odd index in one operation each from the beginning, and finally find the number

of happy people. This also works in a total of $O(N)$ time.

E: Second Sum

For each element of P, let C_i be the number of pairs (L, R) such that $P_i = X_{L,R}$, then it holds that

$$\sum_{L=1}^{N-1} \sum_{R=L+1}^{N} X_{L,R} = \sum_{i=1}^{N} P_i \times C_i$$

Also, for any pair (L, R) such that $P_i = X_{L,R}$, it holds that $L \leq i \leq R$, because P is a permutation. Hereinafter, let's find C_i.

Consider a set $S_i = \{j | j < i, P_i < P_j\}, T_i = \{j | j > i, P_i < P_j\}$. Let w_i be the second largest element of S_i, x_i be the largest element of S_i, y_i be the smallest element of T_i, and z_i be the second smallet element. Note that S_i and T_i are both set of indices. Here, $w_i < x_i < i < y_i < z_i$ holds. $P_i = X_{L,R}$ holds only if $w_i < L \leq x_i < i \leq R < y_i < z_i$ or $w_i < x_i \leq L < i < y_i \leq R < z_i$. Therefore, $C_i = (x_i - w_i) \times (y_i - i) + (i - x_i) \times (z_i - y_i)$ holds.

Assume that the elements are processed in a decreasing order, then the elements that has been seen at some time are larger than the element that are currently being processed. Therefore, by managing a set of indices with a data structure that handles with ordered set (set or multiset in C++), you can find w_i, x_i, y_i, z_i in a time of $O(\log N)$ for each i. Also, if you put two 0's to S_i and two $N + 1$'s to T_i as sentinels, you can easily implement the boundary conditions. The time complexity overall is $O(N \log N)$.

F: Many Slimes

The problem is equivalent to the following problem.

> You are given a complete binary tree of depth N.
> You have to write each element of the set S to each leaf. For each vertex other than leaves, you will write the maximum value of the numbers written on its children. Can you determine a mapping of the elements to the leaves, so that for all vertices other than leaves, the numbers written on its children are all distinct?

Let's determine the mapping of the elemnts in the decreasing order (processing the same values altogether).

Also, every time you assign an element to a leaf, let's paint the vertices on the path from the root to the leaf you assigned the number black.

When you assign same numbers to two leaves respectively, if there doesn't still exist a black vertices on the path between the two leaves, the condition above is not satisfied at the LCA of those two leaves, so it's inappropriate. In other words, it is equivalent to assigning it to the two leaves in the same connected component, when the tree is split by removing the black vertices. Conversely, if there doesn't exist such pair of leaves, the mapping satisfies the constraints.

Considering this fact, you want to make as much connected components as possible, so an effective approach is to assign greedily to the connected component with many vertices as possible. If such situation mentioned above does not happen, the answer will be Yes, otherwise No.

ABC 141 解説

beet, drafear, kort0n, sheyasutaka

2019 年 9 月 15 日

For International Readers: English editorial starts on page 9.

A: Weather Prediction

与えられた文字列に対し、対応する文字列を出力する問題です。
条件文を 3 つ列挙することで解くことができます。
以下は C++ における実装例です。

```
#include <iostream>
#include <string>
using namespace std;
int main(){
  string s;
  cin>>s;
  if(s=="Sunny"s) cout<<"Cloudy"<<endl;
  if(s=="Cloudy"s) cout<<"Rainy"<<endl;
  if(s=="Rainy"s) cout<<"Sunny"<<endl;
  return 0;
}
```

B: Tap Dance

条件を「奇数文字目が L でない，偶数文字目が R でない」と言い換えることで単純に書くことができます．以下に実装例を示します．

Listing 1　C での実装例

```c
#include <stdio.h>
#include <string.h>

int main(void){
  char s[100 + 1];

  scanf("%s", s);

  int n = strlen(s); // 文字列の長さを得る s
  for (int i = 0; i < n; i++) {
    if (i % 2 == 0) {
      // 奇数文字目
      if (s[i] == 'L') {
        puts("No");
        return 0;
      }
    } else {
      // 偶数文字目
      if (s[i] == 'R') {
        puts("No");
        return 0;
      }
    }
  }

  puts("Yes");
  return 0;
}
```

Listing 2　Python3 での実装例

```python
import re

s = input()
p = re.compile(r'^([^L][^R])*[^L]?$')
if p.match(s):
```

```
6   print("Yes")
7 else:
8   print("No")
```

C: Attack Survival

次の 2 つの操作は同等です.

- 正解者以外の $N-1$ 人のポイントを 1 減らす.
- N 人全員のポイントを 1 減らしてから，正解者のポイントだけ 1 増やす.

したがって，ポイントの変動は以下のように言い換えられます.

1. あらかじめ全員のポイントを Q 減らしておく (つまり，開始時点のポイントは $K-Q$).
2. Q 回の正解それぞれに対して，正解者のポイントを 1 増やす.

あとはこれをそのまま実装すればよいです.

D: Powerful Discount Tickets

X を 2^Y で割った切り捨て $\lfloor \frac{X}{2^Y} \rfloor$ は

$$\lfloor \frac{X}{2^Y} \rfloor = \lfloor \frac{\lfloor \frac{X}{2^{Y-1}} \rfloor}{2} \rfloor$$

これは、任意の正整数 X, b_1, b_2 について、

$$\lfloor \frac{\lfloor \frac{X}{b_1} \rfloor}{b_2} \rfloor = \lfloor \frac{X}{b_1 b_2} \rfloor$$

であるためです。まず、これを示します。

Proof. X を b_1 で割った商を q_1、余りを r_1 とすると $\lfloor \frac{X}{b_1} \rfloor = q_1$ であり、

$$X = b_1 q_1 + r_1 \quad (0 \leq r_1 \leq b_1 - 1)$$

が成り立ちます。逆にこれが成り立つような非負整数 q_1, r_1 は一意なので、そのような q_1, r_1 に対して $\lfloor \frac{X}{b_1} \rfloor = q_1$ です。次に、$\lfloor \frac{X}{b_1} \rfloor = q_1$ を b_2 で割った商を q_2、余りを r_2 とすると、$\lfloor \frac{\lfloor \frac{X}{b_1} \rfloor}{b_2} \rfloor = q_2$ であり

$$q_1 = b_2 q_2 + r_2 \quad (0 \leq r_2 \leq b_2 - 1)$$

が成り立ちます。したがって、

$$X = b_1 q_1 + r_1 = (b_1 b_2) q_2 + (r_1 + b_1 r_2)$$

であり、

$$0 \leq r_1 + b_1 r_2 \leq (b_1 - 1) + b_1 (b_2 - 1) = b_1 b_2 - 1$$

なので $\lfloor \frac{X}{b_1 b_2} \rfloor = q_2$ です。 □

したがって、品物を順番に購入し、割引券を品物ごとにまとめて使う代わりに、「割引券を 1 枚使うたびに 1 つの品物を選んで値段を半額 (小数点以下切り捨て) にする」として構いません。よって、最も値段が高い品物に割引券を 1 枚使うことを M 回行えば良いです。なぜなら、まだ割引券が余っているのに今最も高いもの (品物 X とする) に割引券を今後使わないとすると、最後に割引券を使ったものの代わりに品物 X に割引券を使ったほうがお得だからです。しかし、これを愚直に実装すると、時間計算量が $O(NM)$ となり間に合いません。そこで優先度付きキュー (Priority Queue) の出番です。優先度付きキューを使えば

- 値の追加
- 値の削除
- 最大値の取得

がそれぞれ $O(\log |Q|), O(\log |Q|), O(1)$ でできます。ここで、$|Q|$ は優先度付きキューに入っている値の数です。これを使うことで、1 回の操作 (最高額の品物を選んで半額にする) を $O(\log N)$ の時間計算量でできるため、全体としては $O(M \log N)$ となり間に合います。

E: Who Says a Pun?

Z-Algorithm と呼ばれるアルゴリズムを使うと、長さ N の文字列 S について、 $i = 2, 3, ..., N$ での S の i 文字目以降から成る文字列と S の最長共通接頭辞の長さを $O(N)$ で求めることが出来ます。
これにより、 $l_1 = 1$ に制限した場合の答えは、 $O(N)$ で求めることが出来ます。
あとは、同様の処理を $S[2:N], S[3:N], ..., S[N:N]$ についても行うことにより、問題の答えを得ることが出来ます。全体での時間計算量は $O\left(N^2\right)$ です。
別解として、ローリングハッシュを用いて文字列比較を $O(1)$ で処理すると、二分法及び平衡二分探索木を用いたハッシュ値の既出判定を行うことにより、$O\left(N(\log N)^2\right)$ で答えを求めることが出来ます。ハッシュ値の衝突には十分に注意してください。

F: Xor Sum 3

$(A_1 \oplus A_2 \oplus \cdots \oplus A_N) \leq (A_1) + (A_2 \oplus \cdots \oplus A_N)$ から，全て同じ色で塗ることを許しても答えに影響しないことがわかります．

赤く塗った数の XOR，青く塗った数の XOR をそれぞれ $X_{\text{あおくない}}, X_{\text{あおい}}$ とおきます．明らかに $X_{\text{あおくない}} \oplus X_{\text{あおい}} = A_1 \oplus \cdots \oplus A_N$ であり，この値は定数です．この値を 2 進法表記したとき，0 となる桁を「興味深いビット」と呼ぶことにします．

$X_{\text{あおくない}}, X_{\text{あおい}}$ の i ビット目は，

- i ビット目が興味深くないとき，$(0,1)$ か $(1,0)$，つまり和は一定
- i ビット目が興味深いとき，$(0,0)$ か $(1,1)$

です．$2^i > 2^{i-1} + 2^{i-2} + \cdots + 2^0$ より，興味深いビットを上位から貪欲に $(1,1)$ としていくのが最適とわかり，次のように問題を言い換えられます．

> A_i の各要素の，興味深くないビットを無視した数列 A'_i を考え，この数列に対する $X'_{\text{あおくない}} + X'_{\text{あおい}}$ を最大化する．興味深くないビットを全て立てた整数をこれに加えれば，元の問題の答えとなる．

上の問題で $X'_{\text{あおくない}} = X'_{\text{あおい}}$ なので，結局 $X'_{\text{あおい}}$ を最大化すればよいです．つまり，以下の問題が解けたらよいです．

> $A'_1, A'_2, \cdots, A'_N$ の中のいくつかを青く塗り，その XOR を最大化する．

これは以下のようにして解けます:

1. 数列 A'_i を $N \times 60$ 行列とみなし，その行標準形を求める．ここで，列の数 60 は A'_i のビット数を表す．
2. 上の行から貪欲に $X'_{\text{あおい}} \leftarrow X'_{\text{あおい}} \oplus (i \text{ 行目})$ とする．

以上よりこの問題は解かれ，ビット演算を利用して簡単に実装できます (C での実装例)．

ABC 141 Editorial

beet, drafear, kort0n, sheyasutaka

September 15, 2019

A: Weather Prediction

This is a problem of outputting the corresponding string to the given string.

You can enumerate three conditions to solve it.

The following is an implementation example in C++.

```cpp
#include <iostream>
#include <string>
using namespace std;
int main(){
  string s;
  cin>>s;
  if(s=="Sunny"s) cout<<"Cloudy"<<endl;
  if(s=="Cloudy"s) cout<<"Rainy"<<endl;
  if(s=="Rainy"s) cout<<"Sunny"<<endl;
  return 0;
}
```

B: Tap Dance

You can rephrase the conditions as "the character of odd index is not L, and the character of even index is not R" so that implementation will be easy. The following is the implementation example.

Listing 1 Implementation example in C

```c
#include <stdio.h>
#include <string.h>

int main(void){
  char s[100 + 1];

  scanf("%s", s);

  int n = strlen(s); // get the length of string s
  for (int i = 0; i < n; i++) {
    if (i % 2 == 0) {
      // character of odd index
      if (s[i] == 'L') {
        puts("No");
        return 0;
      }
    } else {
      // character of even index
      if (s[i] == 'R') {
        puts("No");
        return 0;
      }
    }
  }

  puts("Yes");
  return 0;
}
```

Listing 2 Implementation example in Python

```python
import re

s = input()
p = re.compile(r'^([^L][^R])*[^L]?$')
```

```
5 if p.match(s):
6   print("Yes")
7 else:
8   print("No")
```

C: Attack Survival

The following two operations are equivalent:

- Decrease the score of $N-1$ players, except for the player who gave the correct answer, by 1.
- Decrease the score of all N people by 1, and then increase the score of the player who gave the correct answer by 1.

Therefore, the fluctuation of the scores can be rephrased as follows:

1. Decrease the scores of all the participants by Q beforehand (so the initial scores are all $K-Q$).
2. For each of Q correct answers, increase the score of the player who gave the answer by 1.

You can implement this straightforward.

D: Powerful Discount Tickets

The rounded down quotient of X divided by 2^Y, $\lfloor \frac{X}{2^Y} \rfloor$, is

$$\lfloor \frac{X}{2^Y} \rfloor = \lfloor \frac{\lfloor \frac{X}{2^{Y-1}} \rfloor}{2} \rfloor$$

This is because, for any positive integer X, b_1, b_2,

$$\lfloor \frac{\lfloor \frac{X}{b_1} \rfloor}{b_2} \rfloor = \lfloor \frac{X}{b_1 b_2} \rfloor$$

holds. Here is the proof.

Proof. Let q_1 be the quotient of X divided by b_1, and r_1 be the remainder, then $\lfloor \frac{X}{b_1} \rfloor = q_1$ holds, and

$$X = b_1 q_1 + r_1 \quad (0 \leq r_1 \leq b_1 - 1)$$

holds. Conversely, the pair of non-negative integer q_1, r_1 such that satisfies the equation above is unique, so for such q_1, r_1, $\lfloor \frac{X}{b_1} \rfloor = q_1$ holds. Next, Let q_2 be the quotient of $\lfloor \frac{X}{b_1} \rfloor = q_1$ divided by b_2, and r_2 be the remainder, then it holds that $\lfloor \frac{\lfloor \frac{X}{b_1} \rfloor}{b_2} \rfloor = q_2$ and

$$q_1 = b_2 q_2 + r_2 \quad (0 \leq r_2 \leq b_2 - 1)$$

holds. Therefore,

$$X = b_1 q_1 + r_1 = (b_1 b_2) q_2 + (r_1 + b_1 r_2)$$

holds, and

$$0 \leq r_1 + b_1 r_2 \leq (b_1 - 1) + b_1 (b_2 - 1) = b_1 b_2 - 1$$

holds, so $\lfloor \frac{X}{b_1 b_2} \rfloor = q_2$. □

Therefore, instead of buying items one by one and using discount tickets at once for each item, you can assume that "every time you use a discount ticket, choose one item and halve its price (rounded down to the nearest integer." Therefore, all you have to do is performing operations of choosing the item with the highest price and use a discount ticket to it M times. This is because, assume that you decided not to apply a discount ticket to the current most expensive item (let it be item X), then the total price would be cheaper if you apply the discount ticket to item X rather than to the item you applied it the last time. However, if you naively implement it, it will need a total of $O(NM)$ time, so it won't finish in time. Time for using Priority Queue. If you use priority queue, you can

- add value
- remove value

- get the maximum value

in a $O(\log |Q|), O(\log |Q|), O(1)$ time each, where $|Q|$ is the number of values stored in the priority queue. By making use of it, you can perform each operation (of choosing the most expensive item and halve its price) in a $O(\log N)$ time, so it takes a total of $O(M \log N)$ time, so you can make it in time.

E: Who Says a Pun?

Given a string S of length N, for each $i = 2, 3, ..., N$, you can find the longest common prefix of S and the substring after the i-th character of S in a $O(N)$ time by applying the algorithm called Z-Algorithm.
Therefore, if you fix $l_1 = 1$, you can find the answer in a $O(N)$ time.
All that left is do the same operations for $S[2:N], S[3:N], ..., S[N:N]$ and you can find the answer for the original problem. It needs a total of $O\left(N^2\right)$ time.
As an another solution, if each string comparison are performed in a $O(1)$ time with rolling hash, by performing binary search on self-balancing binary search tree to check existence of hash value, you can find the answer in a total of $O\left(N\left(\log N\right)^2\right)$ time. Be very careful of hash collisions.

F: Xor Sum 3

Since $(A_1 \oplus A_2 \oplus \cdots \oplus A_N) \leq (A_1) + (A_2 \oplus \cdots \oplus A_N)$ holds, it can be assumed that you may paint all integers in the same color.

Let $X_{\text{notblue}}, X_{\text{blue}}$ be the XOR of the integers painted in red and the XOR of the integers painted in blue, respectively. Obviously, $X_{\text{notblue}} \oplus X_{\text{blue}} = A_1 \oplus \cdots \oplus A_N$ holds, and this value is constant. Write the value in binary notation, and let's call its digits with value 0 as "interesting."

The i-th bits of $X_{\text{notblue}}, X_{\text{blue}}$ are

- $(0, 1)$ or $(1, 0)$ if i-th bit is not intersting, in which case the sum is constant, and
- $(0, 0)$ or $(1, 1)$ if i-th bit is interesting.

Since $2^i > 2^{i-1} + 2^{i-2} + \cdots + 2^0$, it appears that it is optimal to greedily letting interesting bit $(1, 1)$ from the higher to the lower, so the problem can be rephrased as follows:

> Let A'_i be a sequence, each element's non-interesting bits being removed, and maximize $X'_{\text{notblue}} + X'_{\text{blue}}$. If you add the integer whose all the non-interesting bits are set to 1 to it, that would be the answer for the original problem.

In the problem above $X'_{\text{notblue}} = X'_{\text{blue}}$ holds, so maximizing X'_{blue} will be sufficient. Eventually, solving the following problem is sufficient:

> Paint some of $A'_1, A'_2, \cdots, A'_N$ in blue, and maximize their XOR.

This can be solved as follows:

1. Regard the sequence A'_i as a $N \times 60$ matrix, and find its row canonical form. Here, 60 denotes the number of bits of A'_i.
2. From the top row to the bottom, let $X'_{\text{blue}} \leftarrow X'_{\text{blue}} \oplus (i\text{−throw})$.

Here the problem was solved, and it can be easily implemented with bit operations (implementation example in C)

ABC 142 解説

beet, drafear, kort0n, ynymxiaolongbao

2019 年 9 月 28 日

For International Readers: English editorial starts on page 7.

A: Odds of Oddness

実数 x について,x を超えない最大の整数を $floor\left(x\right)$ と定義します.N 以下の正の奇数は $N - floor\left(\frac{N}{2}\right)$ 個数存在しますから, 答えは $\left(N - floor\left(\frac{N}{2}\right)\right)/N$ です.
ただし, 実装には注意が必要です. 例えば C++ では,N を int 型の変数とした場合 $floor\left(\frac{N}{2}\right)$ は $N/2$ で計算出来ますが,$(N - N/2)/N$ のように書くと, 二度目の除算も int 型の変数同士の除算となり, 正しい計算結果が得られません.
これを回避する手法はいくつかあります。例えば, 二度目の除算が double 型の変数同士の除算として実行されるように, 型変換を行えば良いです.
より詳しくは, AtCoder Programming Guide for beginners Y-3.01. 数値型 をご参照ください。
C++ による解答例:https://atcoder.jp/contests/abc142/submissions/7731173

B: Roller Coaster

N 個の整数を入力し、そのうち K 以上の整数がいくつあるかを数える問題です。
各整数について K 以上であるかを判定すればいいです。
以下は C++ における実装例です。

```
#include <iostream>
using namespace std;
int main(){
  int N,K;
  cin>>N>>K;

  int cnt=0;
  for(int i=0;i<N;i++){
    int h;
    cin>>h;
    if(h>=K) cnt++;
  }

  cout<<cnt<<endl;
  return 0;
}
```

C: Go to School

与えられた順列に対し、各要素のソート後のインデックスを求める問題です。
$rev[A_i] = i$ となるような配列 rev を求めればいいです。
以下は C++ における実装例です。

```
#include <iostream>
#include <vector>
using namespace std;
int main(){
  int N;
  cin>>N;
  vector<int> A(N);
  for(int i=0;i<N;i++) cin>>A[i];

  vector<int> rev(N);
  for(int i=0;i<N;i++) rev[A[i]-1]=i+1;
  for(int i=0;i<N;i++) cout<<rev[i]<<endl;
  return 0;
}
```

D: Disjoint Set of Common Divisors

x, y が互いに素であるのと、x, y の最大公約数が 1 であることは同値です。さらに、$gcd(x, y)$ を x と y の最大公約数とすれば、$gcd(a, x) \leq gcd(ab, x)$ です。したがって、整数 ab $(a, b > 1)$ を選ぶよりも、a を選んだほうが良いです (注意: ab が公約数ならば a も公約数です)。すなわち、P を最適解とすると、P で選んだ公約数の中に ab $(a, b > 1)$ と表せる整数が存在したとすると、代わりに a を選ぶことができます。

よって、1 または素数のみを選んだ最適解が存在します。さらに、1 または素数である公約数を全て選ぶことができるため、これらを全て選ぶのが最適です。

さて、そのような公約数はどのように探せば良いでしょう。考察も実装も簡単な方法としては、A, B をそれぞれ素因数分解し (時間計算量はそれぞれ $O(\sqrt{A}), O(\sqrt{B})$)、共通の素因数を探す方法があります。素因数の個数はそれぞれ $O(\log A), O(\log B)$ 個なので、共通の素因数を愚直に求めても十分間に合います。1 と共通の素因数を選ぶのが最適なので、共通の素因数の個数に 1 を加えた値が答えになります。

考察をもう少し進めると、x が A と B の公約数であることと、x が $gcd(A, B)$ の約数であることは同値なので、$gcd(A, B)$ を素因数分解し、素因数の個数を数え上げる方法もあります。

いずれの方法でも、時間計算量は $O(\sqrt{A} + \sqrt{B})$ となります。

E: Get Everything

動的計画法により解くことが出来ます.
0 以上 2^N 未満の整数 j が,「1 以上 N 以下の整数 k について, 現在購入している鍵で宝箱 k を開けられることと,j を 2 進数表記した際に $k-1$ 桁目が 1 であることが同値」という状態を表すこととします.
dp テーブルを以下のように定義します.

$$dp_{i,j} = i+1 \text{ 番目以降の鍵は購入せずに状態を } j \text{ にする為に必要な最小費用}$$

1 以上 N 以下の整数 i について,0 以上 2^N 未満の整数 d_i を,「1 以上 N 以下の整数 k について, 鍵 i で宝箱 k を開けられることと,d_i を 2 進数表記した際に $k-1$ 桁目が 1 であることが同値」となるように定めます. このとき, 動的計画法の遷移式は

$$dp_{i,j \text{ or } d_i} = \min\left(dp_{i-1,j \text{ or } d_i}, dp_{i-1,j} + a_i\right)$$

となります. 時間計算量は $O\left(2^N M\right)$ です.
C++ による解答例:https://atcoder.jp/contests/abc142/submissions/7731316

F: Pure

閉路が存在しないとき、条件を満たす誘導部分グラフは存在しません。そうでないとき、以下の手順を実行します。

手順 1 ： 閉路をひとつ探し、閉路に含まれる頂点の誘導部分グラフを残す。

手順 2 ： そのグラフが条件を満たすとき、終了する。

手順 3 ： 閉路に用いられない辺が存在するので、その辺の終点から閉路をたどり、その辺の始点に到達したらやめる。このとき通過した頂点集合の誘導部分グラフを残し、手順 2 に戻る。

手順 3 を行うたび誘導部分グラフの頂点数が必ず減少し、また手順 2, 3 を通して誘導部分グラフは常に閉路を持ちます。閉路を持つ頂点数が 2 のグラフは条件を満たすため、必ず条件を満たす誘導部分グラフが得られます。愚直に実装すれば計算量は $O(N(N+M))$ となり、正答できます。

A: Odds of Oddness

For a real number x, let's define $floor\,(x)$ as a maximum integer that does not exceeds x. Since there are $N - floor\left(\frac{N}{2}\right)$ odd numbers less than or equal to N, so the answer will be $\left(N - floor\left(\frac{N}{2}\right)\right)/N$.

However, you have to be careful of implementation. For example, in C++, if you define N as a variable of type int, you can calculate $floor\left(\frac{N}{2}\right)$ by $N/2$, but if you write such like $(N - N/2)/N$, the second division will be a division of ints, so you cannot obtain a right answer.

There are several ways to avoid this. For example, you can apply a type conversion so that the second division will be a division of doubles.

For more information, please google about integer types in C++.

An example solution in C++:https://atcoder.jp/contests/abc142/submissions/7731173

B: Roller Coaster

In this problem you are asked to input N integers and find the number of integers that is more than or equal to K.

You can judge if each integer is more than or equal to K.

The following is an implementation example in C++.

```
#include <iostream>
using namespace std;
int main(){
  int N,K;
  cin>>N>>K;

  int cnt=0;
  for(int i=0;i<N;i++){
    int h;
    cin>>h;
    if(h>=K) cnt++;
  }

  cout<<cnt<<endl;
  return 0;
}
```

C: Go to School

In this problem you are asked to sort a given permutation and find the indices of the elements after sorted.

You can find an array rev such that $rev[A_i] = i$.

The following is an implementation example in C++.

```
#include <iostream>
#include <vector>
using namespace std;
int main(){
  int N;
  cin>>N;
  vector<int> A(N);
  for(int i=0;i<N;i++) cin>>A[i];

  vector<int> rev(N);
  for(int i=0;i<N;i++) rev[A[i]-1]=i+1;
  for(int i=0;i<N;i++) cout<<rev[i]<<endl;
  return 0;
}
```

D: Disjoint Set of Common Divisors

x, y are coprime if and only if GCD of x, y is 1. Furthermore, let $gcd(x, y)$ be the GCD of x and y, then $gcd(a, x) \leq gcd(ab, x)$ holds. Therefore, instead of choosing an integer ab $(a, b > 1)$, it is better to choose a (Note: if ab is a common divisor, then a is also a common divisor). That is, given an optimal solution P, if an integer ab $(a, b > 1)$ was contained in the common divisors selected in P, then you can choose a instead.

Therefore, there exists an optimal solution only consiting of 1 and primes. Furthermore, you can choose all the common divisors that is 1 or a prime, so it is optimal to choose all of them.

Then how can we find such common divisors? One way that consideration and implementation are both easy is to factorize A, B into prime factors (it will take a total of $O(\sqrt{A}), O(\sqrt{B})$ time each), and find the common prime factors. The number of prime factors are $O(\log A), O(\log B)$ each, so you can find the common prime factors naively in time. It is optimal to choose 1 and common prime factors, so the answer will be the number of common prime factors added by 1.

With a little more consideration, it appears that x is a common divisor of A and B if and only if x is a divisor of $gcd(A, b)$, so you can factorize $gcd(A, B)$ into prime factors and count the number of prime factors.

Anyway, it can be solved in a total of $O(\sqrt{A} + \sqrt{B})$ time.

E: Get Everything

It can be solved by Dynamic Programming.
Assume that an integer j in the range from 0 (inclusive) to 2^N (exclusive) represents the state that "for each integer k from 1 to N, you can open the treasure box k if and only if the $k-1$-th bit of j is 1 when j is binary notated."
Let define a dp table as follows:

$$dp_{i,j} = \text{minimum cost to achieve the state} j \text{without using} i+1\text{-th and later keys}$$

For each integer i from 1 to N, let's define an integer d_i in the range from 0 (inclusive) to 2^N (exclusive) such that "for each integer k from 1 to N, you can open the treasure box k if and only if the $k-1$-th bit of j is 1 when d_i is binary notated." Then the recurrence formula will be

$$dp_{i,j \text{ or } d_i} = \min\left(dp_{i-1,j \text{ or } d_i}, dp_{i-1,j} + a_i\right).$$

This solution costs a total of $o\left(2^n m\right)$ time.
An example solution in C++:https://atcoder.jp/contests/abc142/submissions/7731316

F: Pure

If the graph does not contain a cycle, then induced subgraph that satisfies the conditions does not exist. Otherwise, perform the procedure below.

Step 1 : Find a cycle from the graph, and retain the induced subgraph consisting of the vertices on the cycle.

Step 2 : If the graph satisfies the condition, exit the procedure.

Step 3 : There exists an edge that is not part of the cycle. Walk through the cycle from the ending vertex of the edge, until reaching the starting vertex of the edge. Retain the induced subgraph consisting of the vertices you passed through, and go back to Step 2.

Every time Step 3 is performed the number of vertices decreases, and through the Steps 2 and 3, the induced subgraph always contains a cycle. Since a graph consisting of two vertices that contains a cycle satisfies the conditions, so you can always obtain a induced subgraph that satisfies the conditions. If you implement it naively, the time complexity will be $O(N(N+M))$, so you can get accepted.

ABC 143 解説

gazelle, kort0n, beet

2019 年 10 月 19 日

For International Readers: English editorial starts on page 7.

A: Curtain

答えは，$A > 2B$ のときは $A - 2B$ であり，$A \leq 2B$ のときは 0 です.
A, B を入力として受け取り，答えを計算し，それを出力することにより，AC となります.
尚，答えを計算する際には if 文を用いても構いませんが，max 関数を用いるとより簡潔に書くことが出来ます.
以下は C++ での実装例です.

```
#include <iostream>
using namespace std;
int main(){
    int A, B, ans;
    cin >> A >> B;
    ans = max(0, A - 2 * B);
    cout << ans << endl;
    return 0;
}
```

B: TAKOYAKI FESTIVAL 2019

N が大きくないので、素直な全探索解で正答できます。

つまり、食べるたこ焼き 2 個の選び方を全探索して、それぞれの体力回復量の和をとればいいです。

この問題のように「順番を考慮しない 2 要素の選び方」を全探索する際のポイントとして、下記コードにあるように、内側のループ変数の始点を外側のループ変数 $+1$ から始めるといいです。こうすることで、内側のループで選ぶインデックスが必ず外側のループで選ぶインデックスより大きくなり、同じ選び方を 2 回見てしまうことを回避できます。

以上の全探索解の計算量は $O(N^2)$ ですが、この問題は工夫すると $O(N)$ で解くこともできます。考えてみてください。

Listing 1　C++ による実装例

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<int> d(n);
    for(int i = 0; i < n; i++) cin >> d[i];
    int ans = 0;
    for(int i = 0; i < n; i++) {
        for(int j = i + 1; j < n; j++) {
            ans += d[i] * d[j];
        }
    }
    cout << ans << endl;
    return 0;
}
```

C: Slimes

解法 1

同じ文字が連続する S の極大な連続部分文字列を，それぞれグループと見なします.

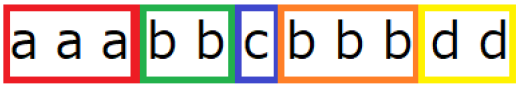


図 1　$S = aaabbcbbbdd$ に対するグループ分けの例

このとき，S の各文字間の境界であって，その両隣の文字が異なるものと，前述したグループの境界は，一対一に対応します.
以上より，前述した S 中の境界に着目することで，解を出力することが出来ます.
C++ による解答例:https://atcoder.jp/contests/abc143/submissions/8034709

解法 2

一部の言語では，文字列中の隣り合う同じ要素を、1 つのみ残して削除する関数があります. 例えば，C++ では以下のようなコードを書けば良いです.
C++ による解答例:https://atcoder.jp/contests/abc143/submissions/8034663

D: Triangles

全ての棒の組について三角形が作れるかをチェックすると，時間計算量が $O\left(N^3\right)$ となり，間に合いません.
三角形を構成する棒のうち，1 番目と 2 番目に長いものを固定します (ただし，同じ長さを持つ棒が複数存在する場合は，予め適当に大小関係を定めておきます).
このとき，3 番目に長い棒として使える棒は，「2 番目の棒より短く，一定以上の長さを持つもの」です.
このような棒の数は，予め棒の長さをソートしておくと，二分探索で効率的に求めることが出来ます.
以上より，1 番目に長い棒と 2 番目に長い棒の選び方について探索すると，時間計算量 $O\left(N^2 \log N\right)$ で答えを求めることが出来ます.
C++ による解答例:https://atcoder.jp/contests/abc143/submissions/8034764

E: Travel by Car

ワーシャルフロイド法というアルゴリズムを用いると、$O\left(N^3\right)$ で全点間の最短距離を求めることが出来ます。
まず、各町を頂点とし、各道を辺とした重み付き無向グラフを作ります。各辺の距離は、問題における道の距離とします。
このグラフにワーシャルフロイド法を適用することで、各町間の最短距離を求めることが出来ます。
次に、もう 1 つグラフを作ります。各町を頂点とし、最短距離が L 以下である町の組の間に、距離 1 の辺を張ります。
このグラフにワーシャルフロイド法を適用することで、各町間を移動する際に必要な燃料補給回数の最小値が求められます。
あとは、この結果に基づいて各クエリに答えれば良いです。時間計算量は $O\left(N^3+Q\right)$ です。
C++ による解答例:https://atcoder.jp/contests/abc143/submissions/8034843

F: Distinct Numbers

与えられた数列から、長さ $K(1 \leq K \leq N)$ の狭義単調増加数列を最大何個取り出すことができるかという問題です（要素の順番を入れ替えてもよい)。

$X(1 \leq X \leq N)$ 個の列を取り出すときの最大の長さを表す関数 $f(X)$ を考えます。

これは、 C_j を $A_i = j$ なる i の個数として、

$$f(X) = \left\lfloor \frac{\sum_{j=1}^{N} \min(C_j, X)}{X} \right\rfloor$$

という式で表せます。また、D_k を $C_j = k$ なる j の個数とすると、

$$f(X) = \left\lfloor \frac{\sum_{j=1}^{N} \min(C_j, X)}{X} \right\rfloor = \left\lfloor \frac{\sum_{k=0}^{N} \min(k, X) \times D_k}{X} \right\rfloor = \left\lfloor \frac{\sum_{k=0}^{X} kD_k + X \sum_{k=X+1}^{N} D_k}{X} \right\rfloor$$

と変形できます。したがって、$\sum_{k=0}^{X} kD_k$ と $\sum_{k=X+1}^{N} D_k$ を累積和を用いて計算することで、全ての $f(X)$ の値を $O(N)$ で求めることができます。その後、各 K について、 $K \leq f(X)$ を満たすような最大の X を求めればいいです（存在しなければ 0)。

A: Curtain

The answer is $A - 2B$ if $A > 2B$, and 0 if $A \leq 2B$.

Receive A, B as input, compute the answer, and output it, and you can get AC.

Here, when calculating the answer you may use if statement, but with max function it will be more simple.

The following is an implementation example in C++.

```cpp
#include <iostream>
using namespace std;
int main(){
    int A, B, ans;
    cin >> A >> B;
    ans = max(0, A - 2 * B);
    cout << ans << endl;
    return 0;
}
```

B: TAKOYAKI FESTIVAL 2019

Since N is not so large, you can got acccepting by simple brute force.

That is, you can perform a brute-force searching such that iterating all the possible pair of takoyaki and sum up the health points restored by them.

In order to iterate through "all the pair chosen from element," it is effectual to let the starting index of the inner loop be the current index of outer loop added by $+1$, just like the listing shown below. This way, the index of the inner loop is always strictly larger than that of outer loop, and you can avoid choosing the same pair twice.

The solution mentioned above has $O(N^2)$ complexity, but with more effort you can also solve this problem in a total of $O(N)$ time. Can you achieve it?

Listing 2 Implementation example in C++

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<int> d(n);
    for(int i = 0; i < n; i++) cin >> d[i];
    int ans = 0;
    for(int i = 0; i < n; i++) {
        for(int j = i + 1; j < n; j++) {
            ans += d[i] * d[j];
        }
    }
    cout << ans << endl;
    return 0;
}
```

C: Slimes

Solution 1

Let's regard each maximum consecutive subsequence of S with the same letter as a group.

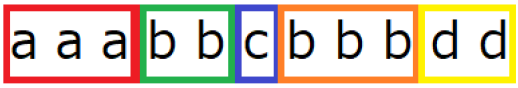


図 2　Grouping example for $S = aaabbcbbbdd$

Then the boundary between two consecutive letters of S such that its adjacent two letters are different and the boundary of the groups mentioned above corresponds one-to-one.
Therefore, by looking up the boundary in S mentioned above, the solution can be obtained.
Sample answer in C++:https://atcoder.jp/contests/abc143/submissions/8034709

Solution 2

Some languages have functions such that removes all but the first element from every consecutive group of given string. For example, in C++ you can write a code like this:
Sample answer in C++:https://atcoder.jp/contests/abc143/submissions/8034663

D: Triangles

If you loop through all the triplets of sticks and check if it is possible to make a triangle, it needs a total of $O\left(N^3\right)$ time and does not fit in time limit.
Let's fix the first- and the second- longest sticks composing a triangle (here, if some sticks have the same length, assume some order between them).
Then, the sticks that can be used as the third longest stick are those whose length is shorter than that of the second stick, but is longer than some extent.
If the sticks are sorted by their lengths beforehand, the number of such sticks can be counted efficiently by using binary search.
Therefore, by iterating through the first- and the second- longest sticks, the solution can be found in a total of $O\left(N^2 \log N\right)$ time.
Sample answer in C++:https://atcoder.jp/contests/abc143/submissions/8034764

E: Travel by Car

With Warshall – Floyd Algorithm, you can find the sortest paths between every pair of vertices in a total of $O\left(N^3\right)$ time.

First, make an undirected weighted graph such that each town corresponds to its vertex and each road corresponds to its edge.

By appying Warshall – Floyd Algorithm to this graph, you can find the shortest path between every pair of towns.

Next, make another graph; each town corresponds to its vertex, and for each pair of town such that minimum distance is less than or equal to L, there exist a vertex of distance 1.

By appying Warshall – Floyd Algorithm to this graph, you can find the minimum number of times he needs to full his tank when travelling between any two towns.

All that left is to answer each query according to the result. The time complexity is $O\left(N^3+Q\right)$.

Sample answer in C++:https://atcoder.jp/contests/abc143/submissions/8034843

F: Distinct Numbers

In this problem you are asked to find how many arrays you can take from the given array, each of which is strictly increasing array of length $K(1 \leq K \leq N)$.

Let $f(X)$ be the maximum length of array when extracting $X(1 \leq X \leq N)$ arrays.

Let C_j be the number of i such that $A_i = j$, then $f(X)$ can be represented like

$$f(X) = \left\lfloor \frac{\sum_{j=1}^{N} \min(C_j, X)}{X} \right\rfloor$$

Moreover, let D_k be the number of j such that $C_j = k$, then it can be deformed like

$$f(X) = \left\lfloor \frac{\sum_{j=1}^{N} \min(C_j, X)}{X} \right\rfloor = \left\lfloor \frac{\sum_{k=0}^{N} \min(k, X) \times D_k}{X} \right\rfloor = \left\lfloor \frac{\sum_{k=0}^{X} kD_k + X \sum_{k=X+1}^{N} D_k}{X} \right\rfloor$$

Therefore, by calculating $\sum_{k=0}^{X} kD_k$ and $\sum_{k=X+1}^{N} D_k$ with cumulative sum, you can calculate all the value of $f(X)$ in a total of $O(N)$ time. Then, for each K, you can find a maximum X such that $K \leq f(X)$ (0 if there doesn't exist).

ABC 144 解説

kort0n, kyopro_friends, sheyasutaka

2019 年 10 月 27 日

For International Readers: English editorial starts on page 7.

A: 9x9

A, B がともに 9 以下であるかどうかで場合分けをすればよいです。複数の条件を満たすかどうかの判断には論理積 (AND) 演算子を用います。以下は C 言語での実装例です。

```
#include<stdio.h>
int main(){
    int a,b;
    scanf("%d%d",&a,&b);
    if(a<=9 && b<=9)printf("%d\n",a*b);
    else printf("-1\n");
}
```

B: 81

$A \times B = N$ となるような 1 以上 9 以下の整数 A, B が存在するかどうか全探索で確かめます。以下は C 言語での実装例です。

```
#include<stdio.h>
int main(){
    int n;
    scanf("%d",&n);
    for(int a=1;a<=9;a++)for(int b=1;b<=9;b++){
        if(a*b==n){
            puts("Yes");
            return 0;
        }
    }
    puts("No");
}
```

C: walk on multiplication table

$(1,1)$ から (a,b) に至るまでの移動回数は $a+b-2$ です。したがって問題は、「$a \times b = N$ となるような (a,b) について、$a+b-2$ の最小値を求めよ」となります。対称性から $a \leq b$ としてよいので、$a \leq \sqrt{N}$ までを全探索することにより $O(\sqrt{N})$ で解くことができます。

D: Water Bottle

解法 1

水筒を θ 傾けた際に水が溢れるか否かには明らかに単調性がありますから、二分法により解を求めることが出来ます。
水筒を θ 傾けた際に水筒の中に存在出来る水量 $f(a,b,\theta)$ を計算し、x と比較することにより、解の範囲を狭めることが出来ます。
ただし、$f(a,b,\theta)$ の計算では、場合分けが必要であることに注意してください。
具体的な計算式については、以下の解答例コードをご確認ください。
C++ による解答例:https://atcoder.jp/contests/abc144/submissions/8133803
立式に際しては、以下の図が役に立ちます。

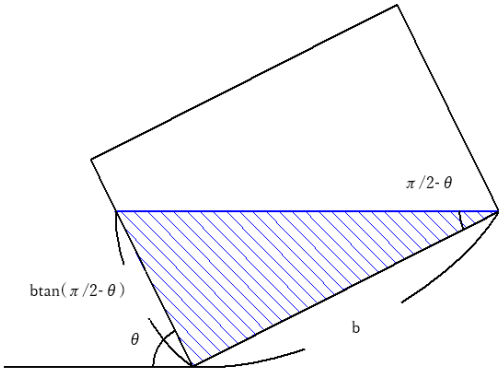


図 1　$b\tan(\pi/2-\theta) \leq a$ のとき

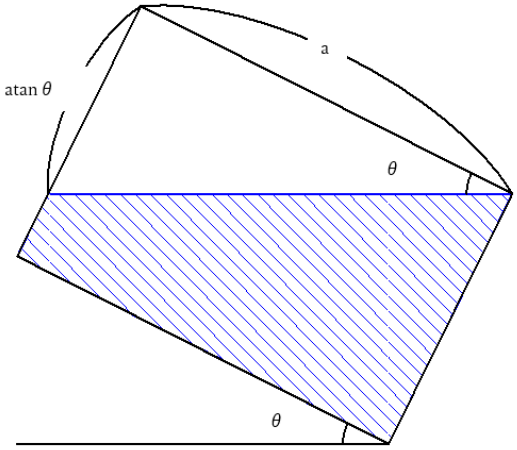


図 2　$b\tan(\pi/2-\theta) > a$ のとき

解法 2

　更に計算を進めると、答えを直接計算することが出来ます。
こちらも a, b, x の値に応じて場合分けが必要であることに注意してください。
具体的な計算式については、以下の解答例コードをご確認ください。
C++ による解答例:https://atcoder.jp/contests/abc144/submissions/8133804

E: Gluttony

メンバーを A_i で昇順に，食べ物を F_i で降順にソートし，i 番目のメンバーに i 番目の食べ物を担当させるのが最適です．以下にそれを示します．

ある 2 人 X, Y の，修行前の消化コスト A_x, A_y および担当する食べ物の食べにくさ F_x, F_y において，$A_x < A_y$ かつ $F_x < F_y$ が成り立つとします．このとき，修行後のコストを A'_x, A'_y とおくと，

- $A'_x < A'_y$ のとき，両名の担当する食べ物のみを入れ替えることで，
- $A'_x \geq A'_y$ のとき，両名の担当する食べ物と修行後のコストを入れ替えることで，

修行の総数も成績も，何も損することなく両名の担当を swap できます．これを繰り返すと有限回で行き詰まり，その結果できた選び方は先述の条件を満たします．

あとは「全てのメンバーについて完食にかかる時間を x 以下にできるか」という問題を考えて，二分探索をすることで解けます．時間計算量は $\Theta(N \log X)$ です (ここで X は答えの値).

F: Fork in the Road

塞ぐ通路を決めた際 (及び、通路を塞がないと決めた際) に部屋 N に到達するまでの移動回数の期待値は、

$$dp_i = \text{部屋 } i \text{ から部屋 } N \text{ へ到達するまでの移動回数の期待値}$$

と dp テーブルを定義し、部屋番号について降順に計算して動的計画法を行うことで、時間計算量 $O(M)$ で計算することが出来ます。しかし、M 個の通路全てについてこの値を計算しようとすると、全体での時間計算量が $O\left(M^2\right)$ となり、間に合いません。

ここで、部屋 i から出ていく通路を塞ぐことを考えます。このとき、塞ぐ通路の選択に依らず、$j = i+1, \cdots, N$ について dp_j の値は変わりません。塞ぐ通路を選択する際は、dp_i の値が最小となるように決めるのが最適です。これは、部屋 i から通路が伸びてる部屋の集合を S_i と定義したとき、部屋 $\arg\max_{k \in S_i} dp_k$ へ進む通路を塞ぐことで達成出来ます。則ち、部屋 i から出ていく通路を塞ぐ際には、これ以外の通路を塞いだ際のことを考慮する必要はありません。

以上より、前述の動的計画法を N 回実行すれば最適解を求めることが出来ます。全体での時間計算量は $O(NM)$ です。

C++ による解答例:https://atcoder.jp/contests/abc144/submissions/8133816

A: 9x9

You can use conditional statement by checking whether both A, B are no more than 9. When checking if multiple conditions holds, you can use logical product (AND) operator. The following is an implementation example in C.

```
#include<stdio.h>
int main(){
    int a,b;
    scanf("%d%d",&a,&b);
    if(a<=9 && b<=9)printf("%d\n",a*b);
    else printf("-1\n");
}
```

B: 81

Check if there exists a pair of integers A, B in a range of 1 to 9 (inclusive) such that $A \times B = N$ by brute forcing. The following is an implementation example in C.

```
#include<stdio.h>
int main(){
    int n;
    scanf("%d",&n);
    for(int a=1;a<=9;a++)for(int b=1;b<=9;b++){
        if(a*b==n){
            puts("Yes");
            return 0;
        }
    }
    puts("No");
}
```

C: walk on multiplication table

The number of moves needed to reach (a, b) from $(1, 1)$ is $a + b - 2$. Therefore, the problem can be rephrased as "find the minimum value of $a + b - 2$ for all (a, b) such that $a \times b = N$." By symmetry you can assume that $a \leq b$, so by performing brute-force searching for all $a \leq \sqrt{N}$, the problem can be solved in a total of $O(\sqrt{N})$ time.

D: Water Bottle

Solution 1

Whether the water will be spilled or not when the bottle is tiled θ degrees is apparently monotonic, so the answer can be found by binary searching.
By calculating $f(a, b, \theta)$, the maximum volume of water that can be inside the bottle when the bottle is tiled θ degrees, and comparing with x, you can narrow the range of solution.
Note that when calculating $f(a, b, \theta)$ you have to divide into cases.
For the specific formula, please refer to the following sample code.
Sample answer in C++:https://atcoder.jp/contests/abc144/submissions/8133803
The following figures help you understand.

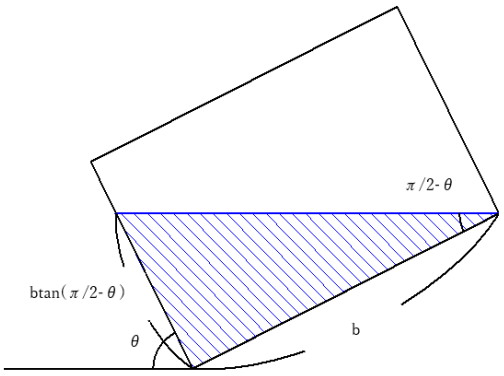


図 3　When $b\tan(\pi/2-\theta) \leq a$

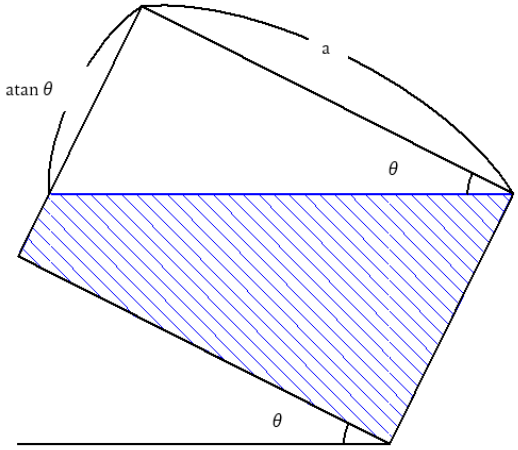


図 4　When $b\tan(\pi/2-\theta) > a$

Solution 2

If you calculate further, you can directly calculate the answer.
Again, note that you need to divide into cases depending on the value of a, b, x.
For the specific formula, please refer to the following sample code.
Sample answer in C++:https://atcoder.jp/contests/abc144/submissions/8133804

E: Gluttony

It is optimal to sort the members in the increasing order of A_i, sort the foods in the decreasing order of F_i, and assign the i-th food to the i-th member. The following is the proof.

For some two members X, Y, assume that their consumption coefficients, A_x, A_y, and the difficulty of foods assigned to them, F_x, F_y, meet such conditions that $A_x < A_y$ and $F_x < F_y$. Let A'_x, A'_y be their consumption coefficient after training, theen they can swap their assigned food without losing the number of total number of training and their score. It can be achieved by performing the following operation:

- If $A'_x < A'_y$, swap the foods assigned to them.
- If $A'_x \geq A'_y$, swap the foods assigned to them, and also swap their consumption coefficients.

This operation can be repetaed finite times, and the resulting assginment meets the condition mentioned above.

All the left is doing binary search by solving problem of "whether or not it is possible to let the longest time of finishing food to be less than x." So this problem can be solved in a total of $\Theta(N \log X)$ time (where X is the answer).

F: Fork in the Road

If you fix any passage to block (or decide not to block any passage), the expected number of passages he takes before he reaches Room N can be calculated by defining the following dp table

$$dp_i = \text{The expected number of passages when he travles from Room}i\text{to Room}N$$

and calculating them in the decreasing order of room index, in which case the whole dp table can be caluculated in a total of $O(M)$ time. However, if you try to calculate this value for each of M passages, the overall time complexity will be $O(M^2)$, so it won't fit in the time limit.
Now let's consider blocking a passage starting from Room i. Then, for each $j = i+1, \cdots, N$, the value of dp_j is independent from the choice of which road to block. When choosing the road to block, the optimal choice is such that dp_i is minimum. Let S_i be the set of rooms such that there exists a road from i to the room, then it can be achieved by blocking the road towards $\arg\max_{k \in S_i} dp_k$. In other words, when you block a road starting from Room i, you don't have to consider the cases when the other roads are blocked.
Therefore, you can find the optimal answer by performing the dynamic programming mentioned above N times. The problem can be solved in a total of $O(NM)$ time.
Sample answer in C++:https://atcoder.jp/contests/abc144/submissions/8133816

ABC 145 解説

kort0n, kyopro_friends, ynymxiaolongbao

2019 年 11 月 16 日

For International Readers: English editorial starts on page 7.

A: Circle

半径 r の円の面積は、円周率を π として πr^2 になります。したがって求める答えは $\pi r^2/\pi 1^2 = r^2$ となります。
以下は C 言語でのコードです。

```
#include<stdio.h>
int main(){
    int r;
    scanf("%d",&r);
    printf("%d\n",r*r);
}
```

B: Echo

N が奇数のときの答えは, 明らかに No です.
N が偶数のとき, 0-indexed で, S の 0 文字目から始まる $N/2$ 文字分の部分文字列と, S の $N/2$ 文字目から始まる $N/2$ 文字分の部分文字列が一致していれば答えは Yes であり, そうでなければ No です.
文字列の部分文字列は, 例えば C++ であれば substr 関数を使うことで容易に取得出来ます.
C++ による解答例:https://atcoder.jp/contests/abc145/submissions/8474441

C: Average Length

解法 1

$N!$ 通りの経路を全探索し, 各経路の長さを計算し, その平均値を出力します.
$N!$ 通りの経路の全探索は, 例えば C++ では next_permutation 関数を使うと容易に実装出来ます.
時間計算量は $O(N!N)$ です.
C++ による解答例:https://atcoder.jp/contests/abc145/submissions/8474526

解法 2

次の問題を考えます.

- N 個の互いに区別出来るボールを 1 列に並べる. 特定の 2 つのボールが隣り合うような並べ方は何通りあるか.

「特定の 2 つのボール」を一纏めにして考えて, この 2 つのボールの並び順も考慮すると, この問題の答えは $2(N-1)!$ 通りであることが分かります.
これより, 元の問題における $N!$ 通りの経路において, 各町のペア間の移動が発生するような経路は, $2(N-1)!$ 個であることが分かります.
これにより, 各町のペア間の距離の総和の $\frac{2(N-1)!}{N!} = \frac{2}{N}$ 倍が答えであることが分かります.
時間計算量は $O\left(N^2\right)$ です.
C++ による解答例:https://atcoder.jp/contests/abc145/submissions/8474573

D: Knight

1 回の移動で x 座標 $+y$ 座標の値は 3 増えます。なので X+Y が 3 の倍数でないとき答えは 0 です。

3 の倍数のとき、(+1,+2) の移動の回数を n、(+2,+1) の移動の回数を m とすると、各座標の値から $n+2m=X, 2n+m=Y$ という連立方程式が得られ、n, m が求まります。$n<0$ または $m<0$ のとき答えは 0 です。

そうでないとき、計 $n+m$ 回の移動のうち、どの n 回で (+1,+2) の移動をするか決めればよいので、答えは ${}_{n+m}C_n$ です。

この値は階乗とその逆元を計算することで $O(n+m+\log \mathrm{mod})$ で求める事ができます。工夫により $O(\min\{n,m\})$ で求めることもできます。

E: All-you-can-eat

解法 1

最後の注文は必ず $T-1$ 分時点で行うとして良いです。最後にどの料理を注文するかを全探索します。i 番目の料理を最後に注文する場合、それ以外の料理は $T-1$ 分時点で完食していなければならないので「i 番目以外の料理で $T-1$ 分以内に完食できる美味しさの合計の最大値 $+$ B_i」が満足度の最大となります。これは $O(NT)$ の DP で求めることができますが、各 i について N 回繰り返していては間に合いません。そこで次のような DP を考えます。

$DP1[i][j] = 1$〜i 番目の料理で j 分以内に完食できる美味しさの合計の最大値

$DP2[i][j] =$ i〜N 番目の料理で j 分以内に完食できる美味しさの合計の最大値

これを用いると、「i 番目以外の料理で $T-1$ 分以内に完食できる美味しさの合計の最大値」は

$$\max_{0 \leq j \leq T-1} DP1[i-1][j] + DP2[i+1][T-1-j]$$

により $O(T)$ で求めることが出来ます。よって全体で $O(NT)$ で求めることができました。

解法 2

最後の注文は必ず $T-1$ 分時点で行うとして良いです。料理を注文する順番は、注文する料理のなかで A_i が最大のものを最後にする場合のみ考えれば良いです (そうでない場合、注文した料理のうち A_i が最大のものと、最後に注文したものの順番を入れ替えても満足度は減りません)。従ってあらかじめ料理を A_i の昇順に並び変えておくことで、解法 1 の $DP1$ のみを用いて答えを計算することができます。計算量は $O(NT)$ です。

F: Laminate

$K = 0$ の場合、最小の操作回数は $\sum_{i=0}^{N-1} max(0, H_{i+1} - H_i)$ (ただし $H_0 = 0$ とする) になります。このことを示します。まず、行ごとに分けて考えたとき、i 列目では白だが $i+1$ 列目では黒で塗られる予定であるような i と同じ数だけ操作が必要になります。そして、列ごとにその列番号が先程の条件を満たす i になる行数を考えると、$max(0, H_{i+1} - H_i)$ であることがわかります。最後に、これらを合計することで元の式が得られます。

$K > 0$ の場合を考えます。列の H の値を変更する際にはいつも変更するものの左隣の値と同じになるようにすることで、実質的にその列が存在しないものとして見なすことができることがわかります。この方法は明らかに最適です。

よって、この問題は以下のように言い換えられます。

> 長さ N の数列 $H_1, H_2, \ldots, H_N$ が与えられるので、K つの項を削除したとき $\sum_{i=0}^{N-K-1} max(0, H_{i+1} - H_i)$ の値が最小でいくつになるか求めてください。ただし、$H_0 = 0$ とします。

この問題は、簡単な動的計画法によって $O(N^3)$ の計算量で解くことができます。$DP[x][y]$ を、削除しない項の集合に対して、最も右の項の番号が x、サイズが y であるときのコストの最小値とします。遷移式を $DP[x][y] = \min_{i=1}^{x-1}\{DP[i][y-1] + max(0, H_x - H_i)\}$ として順次求めることができます。$\min_{i=1}^{N}\{DP[i][N-K]\}$ が答えです。

BIT を用いて動的計画法を高速化することで $O(N^2 logN)$ の計算量で解くこともできます。

A: Circle

The area of a circle of radius r is πr^2. Therefore the answer is $\pi r^2 / \pi 1^2 = r^2$.
The following is a sample code in C.

```
#include<stdio.h>
int main(){
    int r;
    scanf("%d",&r);
    printf("%d\n",r*r);
}
```

B: Echo

If N is odd, then it's obviously No.

When N is even, if the substring of S of length $N/2$ beginning at 0-th letter (0-indexed) and that of length $N/2$ beginning at $N/2$-th letter is the same, the answer is Yes, and otherwise No. The substring of a string can be easily obtained, by using substr function in C++ for example. Sample code in C++:https://atcoder.jp/contests/abc145/submissions/8474441

C: Average Length

Solution 1

Brute force all the $N!$ paths, calculate each legnth, and output their average.
When brute-forcing through $N!$ paths, for example in C++ you can use next_permutation function to implement easily.
The total time complexity is $O(N!N)$.
Sample code in C++:https://atcoder.jp/contests/abc145/submissions/8474526

Solution 2

Let's consider the following question:

- How many ways are there to align N distinguishable balls, so that specific two balls are adjacent to each other?

By regarding "2 specific balls" as one bunch, and also considering the permutations of those two balls, the answer will be $2(N-1)!$.
Therefore, within $N!$ paths in the original problem, the number of paths in which there exists a move between each pair of towns is $2(N-1)!$.
Therefore, the answer will be $\frac{2(N-1)!}{N!} = \frac{2}{N}$ times the sum of distance between each pair of towns.
The total time complexity is $O\left(N^2\right)$.
Sample code in C++:https://atcoder.jp/contests/abc145/submissions/8474573

D: Knight

In one move the value of x coordinate + y coordinate increases by 3. So if X+Y is not a multiple of 3, the answer is 0.

If it's a multiple of three, let n be the number of move (+1,+2), and m be the number of move (+2,+1), then by considering each coordinate we obtain simultaneous equations $n + 2m = X, 2n + m = Y$, so we can find n, m. If $n < 0$ or $m < 0$, the answer is 0.

Otherwise, you have to chose m moves, in which the knight will move (+1, +2), from a total of $n + m$ moves, so the answer will be ${}_{n+m}C_n$.

You can calculate this value by using factorials and its inverse in a total of $O(n + m + \log \mathrm{mod})$ time. With more effort it can be solved in a total of $O(\min\{n, m\})$ time.

E: All-you-can-eat

Solution 1

You can assume that the last order will be done $T-1$ minutes after the first order. We will brute force which dish to order at last. If he will order i-th dish at last, any other dishes should be finished until $T-1$ minutes after the first order, so the maximum happiness will be "(the maximum sum of deliciousness of dishes, except for the i-th dish, that can be eaten within $T-1$ minutes) + B_i." This can be found in an $O(NT)$ DP, but you cannot repeat it N times for each i in time. Now let's consider the following DP.

$DP1[i][j]$ =the maximum sum of deliciousness of dishes, which are chosen from 1st- to i-th dishes, such that he can finish them in j minutes

$DP2[i][j]$=the maximum sum of deliciousness of dishes, which are chosen from i-th- to N-th dishes, such that he can finish them in j minutes

By using them, "the maximum sum of deliciousness of dishes, except for the i-th dish, that can be eaten within $T-1$ minutes" can be calculated in the following formula in a $O(T)$ time:

$\max_{0 \leq j \leq T-1} DP1[i-1][j] + DP2[i+1][T-1-j]$

Therefore this problem could be solved in a total of $O(NT)$ time.

Solution 2

You can assume that the last order will be done $T-1$ minutes after the first order. Assume that you have decided which dishes to order. Among the dishses you will order, consider the dish which its A_i is maximum. Then you can always assume that that dish will be ordered at last (otherwise you can swap the order of orders so that last dish will have the maximum A_i, in which case the hapiness he will gain will not decrease). Therefore, you can sort the dishes beforehand in the increasing order of A_i, so that you will only have to calculate $DP1$ of Solution 1. The total time complexity will be $O(NT)$.

F: Laminate

If $K = 0$, we claim that the minimum number of operations is $\sum_{i=0}^{N-1} max(0, H_{i+1} - H_i)$ (where $H_0 = 0$). Here is a proof. First, for each row, the number of operations you need is equal to the number of i such that i-th column of the row will be painted white and $i + 1$-th onewill be painted black. Then, for each column, consider the number of row such that its index satisfies the condition mentioned above; it is equal to $max(0, H_{i+1} - H_i)$. Finally, we can sum them up so that we obtain the original equation.

Then let's consider $K > 0$. When you change the value H of a column, you can always set the value same to that of the column on its left, so that you can virtually ignore the column. This is obviously optimal.

Therefore, the problem can be rephrased as follows:

> You are given a sequence $H_1, H_2, \ldots, H_N$ of length N. You can remove K elements from the sequence. Find the minimum value of $\sum_{i=0}^{N-K-1} max(0, H_{i+1} - H_i)$ after removing elements. Here, we assume that $H_0 = 0$.

This problem can be solved with a simple DP in a total of $O(N^3)$ time. Let $DP[x][y]$ be the minimum cost when, for a set of elements that are not removed, the index of leftmost element of the set is x and the size of the set is y. Then each value can be calculated by a recurrence relation $DP[x][y] = \min_{i=1}^{x-1}\{DP[i][y-1] + max(0, H_x - H_i)\}$. The answer will be $\min_{i=1}^{N}\{DP[i][N-K]\}$.

You can optimize DP by using BIT so that it will be solved in a total of $O(N^2 logN)$ time.

ABC 146 解説

beet, kort0n, kyopro_friends, tempura0224

2019 年 11 月 24 日

For International Readers: English editorial starts on page 7.

A: Can't wait for holiday

与えられた文字列がどの曜日に対応するものであるか調べて分岐し、答えを出力します。

C++ での実装例は次のとおりです。

```
#include <iostream>
using namespace std;
int main(){
    string s;
    cin >> s;
    int ans;
    if(s == "SUN")ans = 7;
    else if(s == "MON")ans = 6;
    else if(s == "TUE")ans = 5;
    else if(s == "WED")ans = 4;
    else if(s == "THU")ans = 3;
    else if(s == "FRI")ans = 2;
    else if(s == "SAT")ans = 1;
    cout << ans << endl;
}
```

B: ROT N

まず元の文字がアルファベットの何番目の文字かを 0-indexed で求め、それに N を加えて 26 で割った余りを求めると、出力するべき文字が分かります。

```
#include<stdio.h>
#include<string.h>
int main(){
    int n;
    char s[10010];
    scanf("%d␣%s",&n,s);
    int len=strlen(s);
    for(int i=0;i<len;i++){
        int x=s[i]-'A';
        x=(x+n)%26;
        putchar('A'+x);
    }
    putchar('\n');
}
```

C: Buy an Integer

まず、整数 N の値段を計算するためには N の桁数を求める必要がありますが、これは

- $10^k \leq N < 10^{k+1}$ となる整数 k を見つけると、N は $(k+1)$ 桁の整数である。
- 標準の関数等を使って一旦文字列に変換して長さを求める (Python なら len(str(N)) とする、など)。

などの方法で求めることができます。
しかし、1 から 10^9 (10 億) まで順番に値段を計算して購入できるかを判定しているのでは時間制限に間に合わないので工夫が必要です。
今回の問題では値段に単調性 (大きい整数ほど値段も高い) があるので、以下のように二分探索を利用することができます。

0. 10 億を買うことができるなら 10 億が答えです。以下はそうでないと仮定します。
1. まず 5 億が買えるかどうかを調べます。
2. 買えたとします。この場合、買うことのできる最大の整数は 5 億以上で 10 億より小さいということがわかります。
3. 次に 5 億と 10 億の間である 7 億 5000 万が買えるかどうかを調べる。
4. 今度は買えなかったとします。この場合、買うことのできる最大の整数は 5 億以上で 7 億 5000 万より小さいということがわかります。
5. 次は 5 億と 7 億 5000 万の間である 6 億 2500 万が買えるかどうかを調べます。
6. ...。

このようにすることで、1 回ごとに答えの候補の範囲が半分ずつに縮まっていき、30 回程度の計算で答えを見つけることができます。

D: Coloring Edges on Tree

まず、色数の最小値 K がいくつになるかを考えます。任意の頂点 v について、 $\deg(v) \leq K$ を満たす必要があります ($\deg(v)$ は頂点 v の次数を表します)。

実は、 $K = \max_v \deg(v)$ となるような色の塗り分けが存在します。まず、頂点を一つ選び、それを根として G を根付き木にします。根から幅優先探索を行います。各頂点では、その子との間の辺の色を順番に決めていきます。このとき、その頂点を端点に持つ辺の色としてまだ使われていない色の中で最小のものを使うようにすると、その値は K を超えないことがわかります。

構築を終えた後、出力の際には連想配列などを用いることができます。以下は C++ による解答例です。`https://atcoder.jp/contests/abc146/submissions/8600549`

E: Rem of Sum is Num

数列 $\{S_n\}$ を $S_i = A_1 + A_2 + \cdots A_i$ と定めます。$S_0 = 0$ とします。
すると、連続部分列 $A_{i+1}, \ldots, A_j$ がみたすべき条件を式で表すと、

$$(S_j - S_i)\%K = j - i$$

と書けることがわかります。上の式をさらに変形すると、

$$(S_j - j)\%K = (S_i - i)\%K \text{ かつ } j - i < K$$

と同値です。したがって、各 j $(1 \leq j \leq N)$ について、$(S_j - j)\%K = (S_i - i)\%K$ をみたすような $j - K < i < j$ の個数を数えるといいです。
これをそのまま 1 つずつ調べていては間に合いませんが、j に対して調べるべき区間 $(j - K, j)$ と $j + 1$ に対して調べるべき区間 $(j - K + 1, j + 1)$ は左右が 1 つずつ違うだけなので、j 番目について調べたあと $j + 1$ 番目について調べるには、左の要素を 1 つ捨てて、右の要素を 1 つ加えるだけでよいです。
捨てる、加える などの操作は連想配列 (C++ の map や Python の dict など) を用いて $S_i - i$ たちの個数を管理することで、高速に実現することができます。
計算量は全体で $O(N \log K)$ 程度です。

F: Sugoroku

'1' から成る連続した M マスの区間が存在するときは, 答えは明らかに -1 です.
また, そのような区間が存在しないときは, 高橋君がゴールに到達することは可能です.
以下では, そのような区間が存在しない場合のみ考えます.
この双六において, 移動出来るマスの関係をグラフで表すことを考えます. このとき, この双六は $N+1$ 頂点で, $S_i = 0$ となるような頂点 i から, 頂点 $i+1, i+2, ..., \min(i+M, N+1)$ に有向辺が張られたグラフと見なすことが出来ます.
このグラフ上で, 「各頂点からゴールまで最短何手でゴール出来るか」を求めます. これは, 逆辺を張ったグラフ上で頂点 $N+1$ からの最短路問題を解くことに等しいです.
グラフ上の辺が多い為, 有名な最短路問題アルゴリズム (ダイクストラ法等) では実行時間制限に間に合いませんが, グラフの特殊な形状に着目すると, 区間最小値を求めるデータ構造を使えば良いことが分かります.
或いは, このグラフの特殊な形状により, ($S_i = 1$ となる頂点を無視すれば) 前述の値が頂点番号に対して単調減少になることを利用すると, 「頂点 $N+1$ までの最短路の長さが x であるような頂点のうち, 最も頂点番号が小さいもの」を各 x について記録すれば, 頂点番号を降順に走査するだけで簡単に前述の値を求めることが出来ます.
以上の値を求めることが出来れば, あとは頂点 1 から順に, 頂点 $N+1$ までの最短路の長さが減少するような頂点のうち頂点番号が最も小さいものへ進むことを繰り返せば良いです.
最短路の長さが減少しないような頂点へ移動するとゴールまでの手数が最小とはなりません. また, 頂点番号が最小ではない頂点へ移動してもゴールまでの手数は変わりませんが, 出目を並べた数列の辞書式順序が大きくなります.
C++ による解答例:https://atcoder.jp/contests/abc146/submissions/8592598

A: Can't wait for holiday

Find what day of the week does the given string represent, then split into cases and output the answer. The following is a sample code in C++.

```
#include <iostream>
using namespace std;
int main(){
    string s;
    cin >> s;
    int ans;
    if(s == "SUN")ans = 7;
    else if(s == "MON")ans = 6;
    else if(s == "TUE")ans = 5;
    else if(s == "WED")ans = 4;
    else if(s == "THU")ans = 3;
    else if(s == "FRI")ans = 2;
    else if(s == "SAT")ans = 1;
    cout << ans << endl;
}
```

B: ROT N

First find the 0-indexed index of original character in the alphabet table, then add N and get the remainder divided by 26, then you will find which character to output.

```
#include<stdio.h>
#include<string.h>
int main(){
    int n;
    char s[10010];
    scanf("%d␣%s",&n,s);
    int len=strlen(s);
    for(int i=0;i<len;i++){
        int x=s[i]-'A';
        x=(x+n)%26;
        putchar('A'+x);
    }
    putchar('\n');
}
```

C: Buy an Integer

First, in order to calculate the price of integer N you have to find the number of digits of N. This can be found in such ways like:

- Find a integer k such that $10^k \leq N < 10^{k+1}$, then N has $(k+1)$ digits.
- Convert the integer into a string by using standard function etc., and find its length (use len(str(N)) in Python for example).

However, if you calculate prices of integers from 1 to 10^9 (one billion), it won't finish in time, so you have to think out a way. In this problem the problem has monotonicity (the larger integer has the higher price), so you can apply binary search as follows:

0. If he can buy one billion then one billion is the answer. Assume that he can't.
1. First, check if he can buy 500 million.
2. Assume that he can. Then the maximum integer he can buy is greater than or equal to 500 million and less than one billion.
3. Then check if he can buy 750 million, which is between 500 million and one billion.
4. Assume that, this time, he can't. Then the maximum integer he can buy is greater than or equal to 500 million and less than 750 million.
5. Then check if he can buy 750 million, which is between 500 million and 750 million.
6. ...

This way, the span of answer will be halved in every operation, and the answer can be found in about 30 calculations.

D: Coloring Edges on Tree

First, let's think about the minimum number of colors K. For every vertex v, $\deg(v) \leq K$ should meet (where $\deg(v)$ denotes the degree of vertex v).

In fact, there exists coloring such that $K = \max_v \deg(v)$. First, choose a vertex and let it be the root, thus G will be a rooted tree. Perform a breadth first search from the root. For each vertex, determine the colors of edges between its children one by one. For each edge, use the color with the minimum index among those which are not used as colors of edges whose one of endpoints is the current vertex. Then the each index of color does not exceed K.

After construction ended, you can use associative arrays when outputting. The following is a sample code in C++. `https://atcoder.jp/contests/abc146/submissions/8600549`

E: Rem of Sum is Num

Let's define a sequence $\{S_n\}$ such that $S_i = A_1 + A_2 + \cdots A_i$. Let $S_0 = 0$.
Then, the condition that a contiguous subsequence $A_{i+1}, \ldots, A_j$ should meet can be represented like

$$(S_j - S_i)\%K = j - i.$$

This equation can be transformed to the following equivalent conditions:

$$(S_j - j)\%K = (S_i - i)\%K \text{ and } j - i < K$$

Therefore, for each j $(1 \leq j \leq N)$ you should count the number of $j - K < i < j$ such that $(S_j - j)\%K = (S_i - i)\%K$.
If you search them this naively one by one it won't finish in time, but for j the segment needed to be searched is $(j - K, j)$, and for $j + 1$, it is $(j - K + 1, j + 1)$, and these differs only by one elements at the leftmost and rightmost, so in order to search for $j + 1$-th after searching for j-th element, you only have to discard the leftmost element and add the rightmost element.
Operations of discarding or adding can be performed quickly by managing the number of $S_i - i$'s by using associative arrays (such as map in C++ or dict in Python).
The total time complexity is about $O(N \log K)$.

F: Sugoroku

If there exists a segment of cells consisting from '1' of length M, then the answer is obviously -1.

On the other hand, if there does not exist such a segment, then it is possible for Takahashi to reach the goal.

Hereafter we will assume that there does not exist such a segment.

Let's think of representing the available moves in this Sugoroku with a graph. Then we can regard this Sugoroku as a graph of $N+1$ vertices, which has a directed edge from every vertex i such that S_i to vertices $i+1, i+2, ..., \min(i+M, N+1)$.

In this graph, let's find "how many moves are needed to reach the goal from each vertices." This is equivalent to solving shortest path problem on the graph whose vertices are all reversed.

Since there are so many edges, famous shortest-path-finding algorithms won't finish in time, but due to the special pattern of the graph, it appears that you can use a data structure of finding minimum value in the segment.

Or, due to the special pattern of the graph, the values mentioned above is in strictly decreasing order by the indices of the vertices (except for such vertices that $S_i = 1$), so you can find the value mentioned above easily by iterating through the vertices in the decreasing order of their indices, while recording "the vertex having the minimum index among such vertices that the length of the shortest path to the vertex $N+1$ is x" for each x.

Now that you obtained the values mentioned above, all you have to do is move to the vertex with the smallest index among such vertices that length of shortest path to vertex $N+1$ decreases, one after another from vertex 1.

If you move to the vertex such that length of shortest path does not decrease, then the number of moves will not be minimized. If you move to the vertex such that its index is not smallest possible, the number of moves needed does not change, but the sequence of numbers coming up in the roulette will be lexicographically older.

Sample code in C++:https://atcoder.jp/contests/abc146/submissions/8592598

ABC 147 解説

kort0n, kyopro_friends, satashun, sheyasutaka

2019 年 12 月 8 日

For International Readers: English editorial starts on page 7.

A: Blackjack

3 つの整数を受け取り、その合計が 22 以上かそうでないかによって条件分岐します。以下は C 言語での実装例です。

```
#include<stdio.h>
int main(){
    int a,b,c;
    scanf("%d%d%d",&a,&b,&c);
    if(a+b+c>=22)puts("bust");
    else puts("win");
}
```

B: Palindrome-philia

S の長さを N とします。S の長さは変えられないため、S の 0 文字目 (最初の文字) と $N-1$ 文字目 (最後の文字) が一致していなければそのうちの一方をもう一方に合わせ、1 文字目と $N-2$ 文字目が一致していなければそのうちの一方をもう一方に合わせ、...、 文字列の中心までこれを繰り返すのが最適です。for, while などの構文を使って実装するのが一般的でしょう。以下、C 言語での実装例を挙げます。

```c
#include <stdio.h>
#include <string.h>
char s[101];
int main(void) {
  scanf("%s", s);
  int res = 0;
  int n = strlen(s);
  for (int i = 0; i < n; i++) {
    if (s[i] != s[n - 1 - i]) res++;
  }
  printf("%d\n", res / 2);
}
```

C: HonestOrUnkind2

N 人の人がそれぞれ正直者であるか不親切な人であるかを決めれば, その状態が証言と矛盾しないかは, $O\left(N^2\right)$ で容易に確かめることが出来ます.
N 人の人がそれぞれ正直者か否かについては全部で 2^N 通りしかありませんから, これら全ての場合について矛盾が生じないかを調べ, 矛盾が生じないうちでの正直者の最多人数を答えとすれば良いです.
実装上は, 0 以上 2^N 未満の整数 j が「1 以上 N 以下の整数 k について, 人 k が正直者であることと, j を 2 進数表記した際に $k-1$ 桁目が 1 であることが同値」という状態を表すことと定義すると, 容易に全ての場合を試すことが出来ます.
この手法は bit 全探索と呼ばれています.
C++ による解答例:https://atcoder.jp/contests/abc147/submissions/8830089

D: Xor Sum 4

XOR をとる操作で各 bit は互いに干渉しないため、bit ごとに独立に考えることができます。したがって A_i が 0 または 1 である場合が解ければよいです。
A_i XOR A_j は、$A_i = A_j$ のとき 0、$A_i \neq A_j$ のとき 1 なので

$$\begin{aligned}\sum_{i=1}^{N-1}\sum_{j=i+1}^{N}(A_i \text{ XOR } A_j) &= \#\{(i,j) \mid i<j \text{ かつ } A_i \neq A_j\} \\ &= (0 \text{ の個数}) \times (1 \text{ の個数})\end{aligned}$$

として求めることができます。各 bit ごとに 0 のものと 1 のものの個数がわかっていればよいので元の問題は $O(N \log \max A_i)$ で解けました。

E: Balanced path

色の塗り方と移動経路を決める順番は答えに影響しません。そこで、移動しながら塗り方を決めることで、bool 値のテーブルを持つことを考えます。

$$DP[i][j][k] = \text{マス } (i,j) \text{ までの経路の偏りが } k \text{ になることがあるか？}$$

マス (i,j) で偏りが k であるとき、マス $(i+1,j)$ での偏りは、マス $(i+1,j)$ の数の色の塗り方により $k+|A_{i+1,j}-B_{i+1,j}|$ か $|k-|A_{i+1,j}-B_{i+1,j}||$ となります。マス $(i,j+1)$ についても同様です。したがって、i,j が小さい方から順にテーブルを埋めていくことができます。

$M=\max\{|A_{11}-B_{11}|,...,|A_{HW}-B_{HW}|\}$ として、偏りは最大で $O((H+W)M)$ 程度まで考慮する必要があるので、全体で $O(HW(H+W)M)$ で解くことができました。なお、計算の順序を工夫することで空間計算量 $O(\min\{H,W\}(H+W)M)$ で解くことや、bitset を用いた高速化をすることもできます。

F: Sum Difference

S と T の和は一定であるので、$S-T=S-(U-S)=S*2-U$ (U は全ての和) より、 差の代わりに S の和として考えられるものの個数を求めれば良いです。

まず、$D=0$ の場合、X が N 個あります。$X=0$ のとき、和は 0 のみです。$X\neq 0$ のとき、和は $0, X, \cdots N*X$ の $N+1$ 通りです。

以下では $D\neq 0$ の場合を考えます。$D<0$ の場合は、S と T の寄与を入れ替えることを考えると、$X=-X, D=-D$ として解いても同じことがわかるので、$D>0$ とします。

S 側が $k(0\leq k\leq N)$ 個の元から成るとします。数列 $X+i*D(0\leq i<N)$ の和は $k*X+I*D$ (I は i の和) の形をしており、$0+1+\cdots+(k-1)\leq I\leq (N-k)+(N-k+1)\cdots+(N-1)$ を満たしますが、この範囲の全ての整数を取ることができます。

つまり、各 k について $\bmod D$ で同じ整数から成る連続した区間を成しており、これらの和集合が求める答えとなります。これは $k*X \bmod D$ ごとに区間をソートしておくと求めることができます。

以上の時間計算量は $O(NlogN)$ であり、十分高速です。(答えのオーダーは $O(N^3)$ です)

A: Blackjack

Receive 3 integers and switch cases depending on whether their sum is equal to or more than 22 or not. The following is an implementation example in C Language.

```
#include<stdio.h>
int main(){
    int a,b,c;
    scanf("%d%d%d",&a,&b,&c);
    if(a+b+c>=22)puts("bust");
    else puts("win");
}
```

B: Palindrome-philia

Let N be the length of S. Since you cannot change the length of S, it is optimal to set the 0-th (first) letter to be the $N-1$-th (last) letter if it's not same, and to set the 1-st letter to be the $N-2$-th letter if it's not same, ..., until they meets at the middle. The most general way may be using for or while statements. The following is an implementation example in C Language.

```c
#include <stdio.h>
#include <string.h>
char s[101];
int main(void) {
  scanf("%s", s);
  int res = 0;
  int n = strlen(s);
  for (int i = 0; i < n; i++) {
    if (s[i] != s[n - 1 - i]) res++;
  }
  printf("%d\n", res / 2);
}
```

C: HonestOrUnkind2

If you fix whether each of N people is honest or unkind, then you can check easily if it contradicts to any testimonies or not in a total of $O\left(N^2\right)$ time.
There are only 2^N combinations of whether each of N people is honest or unkind in total, so you can check for all the pattern if there are no contradictions, and let the answer be the maximum number of honest person.
When implementation, if you regard an integer j in a range of 0 (inclusive) to 2^N (exclusive) as representing "for each integer k in a range of 1 to N (inclusive), person k is honest if and only if $k-1$-th digit of j when binary-notated is 1," then you can perform brute forcing easily.
This technique is called bitmask brute forcing.
Sample Code in C++:https://atcoder.jp/contests/abc147/submissions/8830089

D: Xor Sum 4

In an operation of taking XOR, each bit is independent from other bits, so you can think each bit independently. Therefore, it is sufficient to solve for A_i is 0 or 1.
A_i XOR A_j is 0 if $A_i = A_j$, and 1 if $A_i \neq A_j$, so it holds that

$$\begin{aligned}\sum_{i=1}^{N-1} \sum_{j=i+1}^{N} (A_i \text{ XOR } A_j) &= \#\{(i,j) \mid i < j \text{ and } A_i \neq A_j\} \\ &= (\text{number of 0s}) \times (\text{number of 1s}).\end{aligned}$$

Now you have to find, for each bit, the number of elements such that the bit is 0 and 1, so the original problem can be solved in a total of $O(N \log \max A_i)$ time.

E: Balanced path

The answer is independent from the order of deciding colors to paint and the path. So let's consider holding a bool table by deciding colors while moving.

$$DP[i][j][k] = \text{Can the unbalancedness of a path to square } (i, j) \text{ be } k?$$

If the unbalancedness at square (i, j) is k, then unbalancedness at square $(i + 1, j)$ is either $k + |A_{i+1,j} - B_{i+1,j}|$ or $|k - |A_{i+1,j} - B_{i+1,j}||$, depending on how to paint the numbers of square (i, j). The same is true for square $(i, j + 1)$. Therefore, you can fill the table in the increasing order of i, j.

Let $M = \max\{|A_{11} - B_{11}|, ..., |A_{HW} - B_{HW}|\}$, then you have to consider a maximum unbalancedness of $O((H + W)M)$, so it can be solved in a total of $O(HW(H + W)M)$ time. If you choose good order of calculation, you can solve in a total of $O(\min\{H, W\}(H + W)M)$ space. You can also accelerate the solution using bitset.

F: Sum Difference

Since the sum of S and T is constant and $S - T = S - (U - S) = S * 2 - U$ (where U is the sum of all the elements), it is sufficient to count the number of possible sums of S instead of differences.

First, if $D = 0$, there are N X's. If $X = 0$, then the only sum is 0. If $X \neq 0$, then there are $N + 1$ possible sums, $0, X, \cdots N * X$.

Hereinafter we assume that $D \neq 0$. If $D < 0$, then the contribution of S and T can be swapped, so we can solve this problem under conditions of $X = -X, D = -D$, so we assume that $D > 0$.

Assume that S consists of $k (0 \leq k \leq N)$ elements. The sum of a sequence $X + i * D (0 \leq i < N)$ can be represented as $k * X + I * D$ (where I is sum of i's), and it holds that $0 + 1 + \cdots + (k - 1) \leq I \leq (N - k) + (N - k + 1) \cdots + (N - 1)$. Actually I can be any integers in this range.

Therefore, for each k it forms a continuous sequence, each element of which has same remainder mod D, and the answer is the union of them. You can find this by sorting the segments for each $k * X \bmod D$.

The total time complexity is $O(NlogN)$, and it's fast enough. (The answer will have an $O(N^3)$ size.)

ABC 148 解説

beet, DEGwer, gazelle, kyopro_friends, sheyasutaka, tozangezan

2019 年 12 月 22 日

For International Readers: English editorial starts on page 8.

A: Round One

$6 - A - B$ を計算し，その値を出力すればよいです．

入力および出力の方法は言語によって異なりますが，たとえば C++ であれば cin, cout を使うことで実現できます．

Listing 1　C での実装例

```c
#include <stdio.h>

int main(void){
        int A, B;
        scanf("%d%d", &A, &B); // 入力からA, B を受け取る

        int result = 6 - A - B; // 6-A-B を計算し，変数result に格納する
        printf("%d\n", result); // 出力

        return 0;
}
```

Listing 2　ferNANDo での実装例

```
r a7 a6 a5 a4 a3 a2 a1 a0
r f e r n a nn d o
r b7 b6 b5 b4 b3 b2 b1 b0
d a7 b7
e a7 d
f b7 d
x7 e f
d a6 b6
e a6 d
```

```
f b6 d
x6 e f
d a5 b5
e a5 d
f b5 d
x5 e f
d a4 b4
e a4 d
f b4 d
x4 e f
d a3 b3
e a3 d
f b3 d
x3 e f
d a2 b2
e a2 d
f b2 d
x2 e f
d a1 b1
e a1 d
f b1 d
x1 e f
d a0 b0
e a0 d
f b0 d
x0 e f
x4 x4 x4
x5 x5 x5
x7 x6 x5 x4 x3 x2 x1 x0
```

B: Strings with the Same Length

1 から N までループを回し、i 回目のループでは S の i 文字目を出力する、その後 T の i 文字目を空白を開けずに出力する、ということを繰り返すと、最終的に目的の新しい文字列が出力されます。

以下は C++ による実装例です (include 等は省略しています)。

```
char S[110];
char T[110];
int main(){
        int a;scanf("%d",&a);
        scanf("%s%s",S,T);
        for(int i=0;i<a;i++)printf("%c%c",S[i],T[i]);printf("\n");
}
```

C: Snack

A の正の倍数を小さい順に見ていき、それが B の倍数であるかを確認すればよいです。このとき、$A \times B$ は明らかに両方の倍数となるので、高々 B 回確認すれば十分なことがわかります。

以下は C++ による解答例です。オーバーフローに注意してください。

`https://atcoder.jp/contests/abc148/submissions/9083476`

余談ですが、このような数は A と B の最小公倍数と呼ばれており、$O(\log(\min(A, B)))$ の計算量で求めることも可能です。

D: Brick Break

以下、「すぬけさんが満足するレンガの砕き方」を「いい砕き方」と言います。

まず 1 が書かれたレンガが存在しない場合、答えは明らかに -1 です。

そうでない場合、1 が書かれたレンガの中で最も左にあるものの番号を i とします。このとき、i 番目のレンガを砕く「いい砕き方」に対して、それと同じ個数のレンガが残るような、i 番目のレンガを砕かない「いい砕き方」が存在します。というのも、元の砕き方で残る 1 が書かれたレンガを砕き、代わりに i 番目のレンガを砕かないとしても変わらず「いい砕き方」になるからです。よって、残すレンガの個数が最大の「いい砕き方」を探す上で、i 番目のレンガは砕かないと仮定しても問題ありません。

同様の考え方をすると、i 番目のレンガより右にあるレンガの中に 2 が書かれたものが存在していれば、そのうち最も左にあるものは砕かずに残すと仮定することが可能です。

この貪欲的なアプローチを繰り返していくことで、残すレンガの個数が最大の「いい砕き方」を見つけることができます。

組合せ最適化問題を解く際に、ある要素を選んで損をすることがないので、その要素を選ぶと仮定して探索空間を小さくする、というアイデアは、競技プログラミングにおける貪欲法で典型的です。

E: Double Factorial

$f(N)$ を 10 進法で表記したときに末尾に続く 0 の個数は、$f(N)$ が 2 で割り切れる回数と $f(N)$ が 5 で割り切れる回数の min です。

以下、N の偶奇に分けて考えます

- N が奇数の場合 $\cdots$ $f(N)$ はいくつかの奇数の積なので、2 で割れることはありません。よって答えは常に 0 です。
- N が偶数の場合 $\cdots$ 明らかに、$f(N)$ が 2 で割り切れる回数は 5 で割り切れる回数よりも多いので、5 で割り切れる回数のみを考えます。

$f(N) = N(N-2)(N-4)\cdots 2$ が 5 で割り切れる回数は、$(2, 4, \cdots, N$ のうち 5 で割り切れるものの個数$) + (2, 4, \cdots, N$ のうち 5^2 で割り切れるものの個数$) + (2, 4, \cdots, N$ のうち 5^3 で割り切れるものの個数$) \cdots$ となります。それぞれ、$2, 4, \cdots, N$ のうち 5 で割り切れるものの個数は $floor(N/10)$、$2, 4, \cdots, N$ のうち 5^2 で割り切れるものの個数は $floor(N/50)$、$2, 4, \cdots, N$ のうち 5^3 で割り切れるものの個数は $floor(N/250)$、$\cdots$ なので、$N/10$ から順に、分母が N 以下の間、分母を 5 倍して商を求めるというのを繰り返し、総和を取ればよいです。

F: Playing tag on tree

高橋君が最初の行動で青木君のいる頂点にしか移動できないとき答えは 0 です。そうでないときを考えます。

青木君が最初にいる頂点を根とします。高橋君と青木君はすれ違うことができないので、ゲームが終了するのは高橋君が”行き止まり”に追い詰められたときです (そうでない場合、高橋君は葉の方へ逃げることでゲームを引き延ばせます)。よって、ゲームが終了する直前は「高橋君が葉 x、青木君が x の親の親にいる」という状況になり、その後 2 人が 1 度ずつ行動し、ともに x の親にいる状況でゲームが終了します。

青木君は常に最短距離で高橋君を追い詰めることができるので、青木君の移動回数は高橋君がどの葉で追い詰められたかのみに依存します。したがって高橋君は追い詰められる場所として、「高橋君が青木君より先に到達することができるような葉のうち、青木君から最も遠いもの」を選択するのが最善となります。これは頂点 u, v からの距離をそれぞれ DFS や BFS などによって計算することで求めることができ、計算量は $O(N)$ です。

A: Round One

It would be enough to calculate $6 - A - B$ and output the value.

The way of input and output depends on languages. For example, in C++, you can implement by using cin and cout.

Listing 3 Implementation example in C

```c
#include <stdio.h>

int main(void){
        int A, B;
        scanf("%d%d", &A, &B); // receive A and B from input

        int result = 6 - A - B; // calculate 6-A-B and store into the variable
            result
        printf("%d\n", result); // output

        return 0;
}
```

Listing 4 Implementation example in ferNANDo

```
r a7 a6 a5 a4 a3 a2 a1 a0
r f e r n a nn d o
r b7 b6 b5 b4 b3 b2 b1 b0
d a7 b7
e a7 d
f b7 d
x7 e f
d a6 b6
e a6 d
f b6 d
x6 e f
d a5 b5
e a5 d
f b5 d
x5 e f
d a4 b4
e a4 d
f b4 d
x4 e f
d a3 b3
```

```
21 e a3 d
22 f b3 d
23 x3 e f
24 d a2 b2
25 e a2 d
26 f b2 d
27 x2 e f
28 d a1 b1
29 e a1 d
30 f b1 d
31 x1 e f
32 d a0 b0
33 e a0 d
34 f b0 d
35 x0 e f
36 x4 x4 x4
37 x5 x5 x5
38 x7 x6 x5 x4 x3 x2 x1 x0
```

B: Strings with the Same Length

Perform loop from 1 to N: in the i-th loop, output i-th letter of S, and then output i-th letter of T without printing whitespace. Then the desired new string is printed in the end.

The following is an implementation example in C++ (include etc. are abbreviated).

```
1 char S[110];
2 char T[110];
3 int main(){
4         int a;scanf("%d",&a);
5         scanf("%s%s",S,T);
6         for(int i=0;i<a;i++)printf("%c%c",S[i],T[i]);printf("\n");
7 }
```

C: Snack

It will be enough to look up each of the multiples of A in the increasing order, and check if it is multiple of B. Since $A \times B$ is obviously multiple of both A and B, it turn out that it is enough to check at most B times.

The following is a sample code by C++. Be careful of overflows.

`https://atcoder.jp/contests/abc148/submissions/9083476`

Just an aside, such integer is called the least common multiple of A and B, and it can be found in a $O(\log(\min(A, B)))$ time.

D: Brick Break

Hereinafter we will call a combination of bricks you will break "good" if Snuke will be satisfied after they are broken.

First, if there are no bricks that 1 is written on it, then the answer is obviously -1.

Otherwise, let i be the index of the leftmost brick with 1 written on it. Then, for any "good combination" including i-th brick, there exists a "good combination" of same size such that i is not included. This is because if you break the brick with 1 written on it which originally remains, and instead leaving i-th brick unbroken, then the resulting combination of bricks is still good. Therefore, when looking for the "good combination" of breaking such that the number of remaining blocks is maximum, you can always assume that you will not break i-th block.

Similarly, if there exists a brick with 2 written on it on the right of i-th brick, then you can assume that you will leave the leftmost brick among them without breaking.

By repeating this greedy approach, you can find a "good combination" of breaking such that the number of remaining blocks is maximum.

When solving combinational optimization problem, the idea of "choosing this element does not lead to disadvantage, so we can narrow down the search space by assuming that we choose it" is very typical in greedy algorithms in competitive programming.

E: Double Factorial

The number of trailing zeros in the decimal notation of $f(N)$ is min of how many times $f(N)$ can be divided by 2 and how many $f(N)$ can be divided by 5.

We will divide into cases depending on parity of N.

- If N is odd $\cdots$ $f(N)$ is product of some odd numbers, so it can never be divided by 2. Therefore, the answer is always 0.
- If N is even $\cdots$ Obviously, the number of times $f(N)$ can be divided by 2 is more than the number of times it can be divided by 5, so we only have to consider how many times it can be divided by 5.

The number of times $f(N) = N(N-2)(N-4)\cdots 2$ can be divided by 5 is equal to (how many integers out of $2, 4, \cdots, N$ can be divided by 5?) + (how many integers out of $2, 4, \cdots, N$ can be divided by 5^2?) + (how many integers out of $2, 4, \cdots, N$ can be divided by 5^3?) $\cdots$. Since $floor(N/10)$ integers out of $2, 4, \cdots, N$ can be divided by 5, $floor(N/50)$ integers out of $2, 4, \cdots, N$ can be divided by 5^2, $floor(N/250)$ integers out of $2, 4, \cdots, N$ can be divided by 5^3, $\cdots$, so it would be enough to, starting from $N/10$, repeat multiplying the denominator by 5 and calculating the quotient while the denominator is no more than N, and find the sum of the quotients.

F: Playing tag on tree

If Takahashi can only move to the vertex where Aoki is standing for the first turn, then the answer is 0. Let's consider the other cases.

Let the vertex where Aoki is initially standing be the root. Since Takahashi and Aoki cannot pass each other, the game ends when Takahashi is cornered into a "dead end" (or otherwise, Takahashi can extend the game by escaping in the direction of leaves). Therefore, right before the game ends, "Takahashi is on a leaf x and Aoki is on the parent of the parent of x," and soon after that they move once each to the parent of x, when the game ends.

Since Aoki can always take the shortest path to corner Takahashi, so the number of moves Aoki performs always depend on which leaf Takahashi will be cornered into. Therefore, the optimal choice of vertex that Takahashi will be cornered into is "the furthest vertex from Aoki which Takahashi can reach before Aoki can." This can be obtained by calculating the distance from u and v by DFS or BFS, and the time complexity is $O(N)$.

ABC 149 解説

writers: beet, DEGwer, IH19980412, kyopro_friends, tozangezan, yuma000

2019 年 12 月 29 日

For International Readers: English editorial starts on page 7.

A: Strings

2 つの文字列 S, T を順に読み込み、T, S の順にスペースを空けずに出力することで目的の文字列を出力したことになります。(T,S の順に文字列を連結することで新しい文字列を作っても良いです。)

Listing 1　C++ での実装例

```
#include <stdio.h>

char S[110];
char T[110];

int main(){
        scanf("%s%s",S,T);
        printf("%s%s\n",T,S);
}
```

B: Greedy Takahashi

K の値によって次のように 3 通りに場合分けすることで解けます。

- $K \leq A$ のとき、高橋君は自分の持っているクッキーを K 枚食べます。
- $A < K \leq A + B$ のとき、高橋君は最初の A 回の行動で自分の持っているクッキーを全て食べ、残りの $K - A$ 回の行動で青木君の持っているクッキーを $K - A$ 枚食べます。
- $A + B < K$ のとき、高橋君は最初の A 回の行動で自分の持っているクッキーを全て食べ、次の B 回の行動で青木君の持っているクッキーを全て食べます。

C++ による実装例は以下の通りです。

```
#include <iostream>
using namespace std;

int main(){
        long long a, b, k;
        cin >> a >> b >> k;

        if(k <= a){
                cout << a - k << '␣' << b <<endl;
        }else if(k <= a+b){
                cout << 0 << '␣' << b - (k-a) <<endl;
        }else{
                cout << 0 << '␣' << 0 <<endl;
        }
}
```

C: Next Prime

X 以上の素数のうち、最小のものを求める問題です。

X 以上の数を小さい順に素数かどうか判定していけばよいです。ある数 A が素数であるかどうかは $O(\sqrt{A})$ で判定できます。100003 は素数なので、答えは 100003 以下になり、十分高速です。

以下は C++ における実装例です。

```
#include<iostream>
using namespace std;

bool is_prime(int x){
  if(x<=1) return false;
  for(int i=2;i*i<=x;i++)
    if(x%i==0) return false;
  return true;
}

signed main(){
  int x;
  cin>>x;

  int p=x;
  while(!is_prime(p)) p++;

  cout<<p<<endl;
  return 0;
}
```

D: Prediction and Restriction(writer : yuma000)

まず前提として、 a 回目にこちらが出す手と b 回目にこちらが出す手は、a mod K != b mod K ならば、独立に選べます (どちらかの選択がもう片方の選択に制限を与えることはない)。

N 回のじゃんけんを、mod K によって K 個のグループに分けることを考えます。この時、各グループに対して、以下の sub 問題を解くことができれば、この問題を解くことができます。

- sub 問題 このグループに属するじゃんけんで、最大何点とれるか？ただし、連続で同じ手を出すことはできない。

以下が、sub 問題への解法です。

解法 1(DP)

前から順に、前回にどんな手を出したかを保持しておきながら DP をします。各 sub 問題に対しての時間計算量は O(N/K) なので、トータルでの時間計算量は O(N) です。

解法 2(貪欲法)

貪欲法でも解けます (こっちの方が実装は楽です)。具体的には前から順に、

- 勝てるならば勝てる手を出す。
- (前回自分が出した手の制限のせいで) 勝てない場合は、次の手の邪魔にならない手を出す。

以上のようにすることで最適な解が求まります。

E: Handshake(writer : yuma000)

もちろん、もっとも上がる幸福度が高い握手から順番に M 回することで、最終的な幸福度を最大にできます。が、愚直に実装すると間に合いません。以下でこれを効率的に求める方法を解説します。

解法 1(二分探索を使った解法)

まず、増える幸福度が、ある定数 X 以上になる握手の方法が何通りあるか、を求めることを考えます。左手で握手する人を先に決めたとき、右手で握手できる人が何人いるかは累積和を用いることで O(1) で求まるのでトータルで O(N) で計算できます。

後は、X を二分探索することで、M 通り以上の握手ができる最小の X が求まるので、もう一度累積和を用いて最終的な幸福度が計算できます。時間計算量は O(NlogN) です。

解法 2(高速フーリエ変換)(おまけ)

$A_i \leq 100,000$ という条件から、高速フーリエ変換 (以下 FFT) でも解けます。FFT の詳細については、以下のページをご覧ください。

`https://atcoder.jp/contests/atc001/tasks/fft_c`

具体的には、

- i 番目の要素にパワー i の人の人数が格納されている配列を二つ用意する。
- それらの配列を畳み込む。(FFT をすることで、時間計算量 $O(max(A_i)log(max(A_i))$ でできます。)

という風にすることで、増える幸福度ごとの通り数が分かります。後はそれを増える幸福度が高い順に加えていくことで、この問題を解くことができます。

F: Number of Surrounded Nodes

解法 1: 辺に注目する方法

求めるものは「S の頂点の個数 − 黒く塗られた頂点の個数」と等しいです。期待値の線形性から、それぞれの項の期待値が求まればよいです。黒く塗られた頂点の個数の期待値は明らかに $N/2$ です。S の頂点の個数の期待値を考えます。

木の頂点の個数は (空グラフの場合を除き) 辺の個数 $+1$ なので、S の頂点の個数のかわりに、S の辺の個数を考えます。期待値の線形性から、各辺が S に含まれる確率を計算し、その合計を求めればよいです。辺 e が S に含まれるための必要十分条件は、T から e を取り除いてできる 2 つの部分木の両方に黒く塗られた頂点が存在することです。2 つの部分木の頂点数を x_e, $N - x_e$ とすると、そのような確率は $\left(1 - \left(\frac{1}{2}\right)^{x_e}\right)\left(1 - \left(\frac{1}{2}\right)^{N-x_e}\right)$ となるので、x_e がわかれば計算することができます。

以上より、各辺 e について x_e が求まればよく、これは DFS などにより $O(N)$ で求めることができます。空グラフの扱いに注意してください。

解法 2: 頂点に注目する方法

期待値の線形性から、各頂点 v について、v が S に属しかつ白く塗られているような確率を計算し、その合計を求めればよいです。頂点 v について、T から v を取り除いてできる部分木たちを $T_{v1}, T_{v2}, ...$ とします。このとき、v が S に含まれるための必要十分条件は、T_{vi} たちのうち少なくとも 2 つが黒く塗られた頂点を含むことです。これは、T_{vi} に属する頂点の個数 x_{vi} がわかれば、余事象を考えることで求めることができます。

以上より、各頂点 v について x_{vi} たちが求まればよく、これは全方位木 DP (ReRooting) などにより $O(N)$ で求めることができます。

また、式を整理することで解法 1 と同じ式を得ることもできます。

A: Strings

Receive two strings S, T in this order, and output them in the order of T, S without printing whitespaces, then you will have outputted the desired string. (You can also make a new string by concatenating the strings in the order of T, S.)

Listing 2 Sample Code in C++

```
#include <stdio.h>

char S[110];
char T[110];

int main(){
        scanf("%s%s",S,T);
        printf("%s%s\n",T,S);
}
```

B: Greedy Takahashi

You can divide into the following three cases depending on the value of K.

- If $K \leq A$, then Takahashi will eat K cookies he owns.
- If $A < K \leq A + B$, then Takahashi will eat all the cookies he own in the first A actions, and then $K - A$ cookies of Aoki's in the $K - A$ actions left.
- If $A + B < K$, then Takahashi will eat all the cookies he owns in the first A actions, and then eat all of Aoki's cookies in the next B actions.

The following is an implementation example in C++.

```
#include <iostream>
using namespace std;

int main(){
        long long a, b, k;
        cin >> a >> b >> k;

        if(k <= a){
                cout << a - k << '␣' << b <<endl;
        }else if(k <= a+b){
                cout << 0 << '␣' << b - (k-a) <<endl;
        }else{
                cout << 0 << '␣' << 0 <<endl;
        }
}
```

C: Next Prime

You are asked to find the minimum prime greater than or equal to X.

It is enough to iterate through the numbers greater than or equal to X in order and check if it is a prime number. You can check if a number A is a prime number or not in a $O(\sqrt{A})$ time. Since 100003 is a prime number, the answer is less than or equal to 100003, so it is fast enough.

The following is an implementation example in C++.

```
#include<iostream>
using namespace std;

bool is_prime(int x){
  if(x<=1) return false;
  for(int i=2;i*i<=x;i++)
    if(x%i==0) return false;
  return true;
}

signed main(){
  int x;
  cin>>x;

  int p=x;
  while(!is_prime(p)) p++;

  cout<<p<<endl;
  return 0;
}
```

D: Prediction and Restriction(writer : yuma000)

First, as a premise, you can choose the a-th hand and the b-th hand independently if a mod K != b mod K (one choice does not restrict another choice).

Consider distributing N rounds of Rock Papers Scissors into K groups depending on mod K. Then, if the following sub problem can be solved for each group, this problem can also be solved.

- sub problem: what is the maximum score earned in the rounds belonging to this group? You cannot use the same hand contiguously.

The following is the solution for the sub problem.

Solution 1(DP)

From the begin to the end, perform DP while retaining the last hand you used. The time complexity for each sub problem is O(N/K), so the total time complexity is O(N).

Solution 2(Greedy Algorithm)

You can also solve with greedy algorithm (the implementation is simpler). Specifically, from the beginning to the end,

- If you can win, use the hand that you can win.
- If you cannot win (because of the hand last you used), use that hand that does not obstruct the next hand.

By doing so, you can find the optimal solution.

E: Handshake(writer : yuma000)

Of course, you can maximize the ultimate happiness M times from the handshake that increases happiness the most to the least. But, if you implement naively it won't finish in time. We will explain the way of finding it efficiently.

Solution 1(Solution using binary search)

First, consider how many ways of handshakes are there by which the happiness increases by more than or equal to a constant X. When a person to shake hand for his left hand is fixed, you can find how many people are there whom he can shake hand with for his right hand in O(1) time using cumulative sums, so it can be calculated in a total of O(N) time.

Then, you can perform binary search for X so as to find the minimum X such that he can perform M ways of handshakes, so you can find the ultimate happiness by using cumulative sums again. The time complexity is O(NlogN).

Solution 2(Fast Fourier Transform)(Bonus)

Since $A_i \leq 100,000$, you can solve this problem by Fast Fourier Transform (hereinafter FFT). For more details about FFT, please look for many resources online. (There are an educational problem on AtCoder, but the problem statement is provided only in Japanese. For your information, the URL is as follows:)

`https://atcoder.jp/contests/atc001/tasks/fft_c`

Specifically,

- Prepare two arrays, each of whose i-th element contains the number of people with power i.
- Find the convolution of those array. (By performing FFT, it can be done in a total of $O(max(A_i)log(max(A_i))$ time.)

so that you can find the number of ways for each increasing happinesses. Lastly count them in the decreasing order of increasing happinesses, and you can solve the problem.

F: Number of Surrounded Nodes

Solution 1: Focusing on edges

What you want to calculate is equal to "the number of vertices of S − the number of vertices painted black." By the linearity of expected values, it is enough to find the expected value of each term. The number of vertices painted black is obviously N_2. Now let's consider the expected number of vertices of S.

The number of vertices of a tree is the number of edges +1 (except for an empty graph), so let's consider the number of edges of S instead of the number of vertices of S. An edge e is contained in S if and only if, when e was removed from T, the resulting two parts both contain vertices painted black. Let x_e, $N - x_e$ be the number of vertices of two subtrees, then such probability is $\left(1 - \left(\frac{1}{2}\right)^{x_e}\right)\left(1 - \left(\frac{1}{2}\right)^{N-x_e}\right)$, so if you found x_e you can calculate them.

Therefore, it is enough to find x_e for each edge e, and this can be found in a $O(N)$ time by DFS etc. Please be careful how to treat empty graphs.

Solution 2: Focusing on vertices

By the linearity of expected values, it is enough to find the probability that, for each vertex v, v is contained in S and also painted white, and to sum them up. For a vertex v, let $T_{v1}, T_{v2}, ...$ be the subtrees when v is removed from T. Then, v is contained in S if and only if at least two subtrees out of T_{vi}'s contain vertices pained black. You can calculate this by considering complementary events by finding x_{vi}, the number of vertices of T_{vi}.

Therefore, it is enough to find x_{vi}'s for each vertex v, and this can be found in a $O(N)$ time by ReRooting etc.

Also, by transforming the equation you can obtain the same equation as Solution 1.

ABC 150 解説

writers: camypaper, DEGwer, kyopro_friends, gazelle, satashun, tozangezan, yuma000

2020 年 1 月 10 日

For International Readers: English editorial starts on page 7.

A: 500 Yen Coins

高橋君が持っているお金の総額は $500 \times K$ 円です。これを X と比較し、$500 \times K$ が X 以上ならば Yes、そうでないならば No です。

Listing 1　C++ での実装例

```
#include <stdio.h>

int main(){
        int K,X;
        scanf("%d%d",&K,&X);
        if(500*K>=X)printf("Yes\n");
        else printf("No\n");
}
```

B: Count ABC

任意の i $(0 \leq i \leq n-3)$ について、S_i 以下 3 文字が ABC であるかを確認すればいいです。C++ による実装例を以下に示します。

Listing 2　C++ による実装例

```
#include<bits/stdc++.h>
using namespace std;

int main() {
    int n;
    string s;
    cin >> n >> s;
    int ans = 0;
    for(int i = 0; i <= n - 3; i++) {
        if(s[i] == 'A' && s[i + 1] == 'B' && s[i + 2] == 'C') ans++;
    }
    cout << ans << endl;
    return 0;
}
```

C: Count Order

順列 X が辞書順で k 番目であることと、X より辞書順で小さい順列が $k-1$ 個あることは同値です。よって、大きさ N の順列をすべて生成して、そのうちに P, Q それぞれより小さいものがいくつあるかを数えることで、この問題を解くことができます。

順列の生成には、深さ優先探索などの探索を行う他に、C++ の next_permutation 関数のような順列生成用の関数を利用することも可能です。

D.SemiCommonMultiple(writer : yuma000)

X が A の半公倍数である条件は、以下のように言い換えられます。

- 任意の $k(1 \leq k \leq N)$ に対して、$X = (a_k/2) * (2p + 1)$ を満たす負でない整数 p が存在する。

X が $a_k/2$ の奇数倍であることから、これは以下のような二つの条件に分割できます。

- 任意の $k(1 \leq k \leq N)$ に対して、X は $a_k/2$ の倍数である。
- 任意の $k(1 \leq k \leq N)$ に対して、X が 2 で割り切れる回数と $a_k/2$ が 2 で割り切れる回数は同じである。

a_i が 2 で割り切れる回数と a_j が 2 で割り切れる回数が異なるような i, j が存在するとき、条件を満たす X はありません。

そのような i, j が存在しないときについて考えます。$a_1/2, a_2/2, ..., a_N/2$ の最小公倍数を T とします。すると、T を奇数倍したものが条件を満たすので、条件を満たす X の個数が求まります。

E: Change a Little Bit

まず、S,T の組を 1 つ固定して、$f(S,T)$ がどうなるかを考えてみましょう。

明らかに S,T で等しい要素を変更する必要はありません。ここで、$p_1,p_2,\cdots,p_k$ が S,T で異なるところの添字であり、この順番に変更するとします。このときのコストは $k*C_{p_1}+(k-1)*C_{p_2}+\cdots+1*C_{p_k}$ であるので、C の小さい方から変更するのが最適です。これは、ある i があって $C_{p_i}>C_{p_{i+1}}$ となるとき、この順番を swap してもコストが悪化しないことからわかります。

以上の考察と対称性より、C は広義単調減少であると仮定します。全ての $S\neq T$ の組について、$S_{i_1}\neq T_{i_1},S_{i_2}\neq T_{i_2}\cdots S_{i_k}\neq T_{i_k}(1\leq i_1<i_2<\cdots<i_k\leq N)$ のとき $\sum_{t=1}^{k}C_{i_t}*t$　を求めたいです。

総和の寄与を分解して、各 C_i が何回足されるかを考えます。これは $j\leq i$ で $S_j\neq T_j$ となる j の個数に依存します。これが k 箇所であるとすると、$\binom{i-1}{k-1}*2^k*2^{i-1-k}*4^{N-i}*C_i*k$ 寄与します。(各要素が異なる場合も等しい場合も、2 通りずつであることに注意してください)

これを k について足し合わせると $C_i*4^{N-i}*2^{i-1}*\sum_{k=1}^{i}\binom{i-1}{k-1}*k$ ですが、総和の中身も各要素が何回カウントされるかと読み替えることで $2^{i-1}*(i+1)$ であることがわかります。

以上より結局のところ答えは $4^{N-1}*\sum_{i=1}^{N}C_i*(i+1)$ と表せます。

ソート部分がボトルネックとなり、 $O(NlogN)$ でこの問題が解けました。

F:XOR Shift

数列の添え字は断りなく $\mathrm{mod} N$ で考えるものとし、XOR を $\oplus$ で表すこととします。

$$(1)\ \text{すべての } i \text{ で } a'_i = b_i$$

となるためには

$$(2)\ \text{すべての } i \text{ で } a'_i \oplus a'_{i+1} = b_i \oplus b_{i+1}$$

が成立していることが必要です。ここで $a'_i \oplus a'_{i+1} = (a_{i+k} \oplus x) \oplus (a_{i+k+1} \oplus x) = a_{i+k} \oplus a_{i+k+1}$ であることから、(2) の条件は k のみによって決まります。この条件を満たす k は、KMP 法などを用いることで、$O(N)$ で列挙することができます。（$c_i = a_i \oplus a_{i+1}$、$d_i = b_i \oplus b_{i+1}$ として、d の中から c を検索します）

逆に (2) を満たすような k に対しては、x が $b_i \oplus a_{i+k}$ であるとき、その時に限り (1) の条件を満たします。したがって、各 k に対してこのような x は定数時間で求めることができます。以上によりこの問題は $O(N)$ で解けました。

XOR になじみがない人は、まず XOR の代わりに足し算で同様のことを考えてみるとわかりやすいかもしれません。その場合 (2) の条件は $a'_i - a'_{i+1} = b_i - b_{i+1}$ となります。

A: 500 Yen Coins

Takahashi has $500 \times K$ yen. Compare this with X, and if $500 \times K$ is greater than or equal to X then the answer is Yes, otherwise No.

Listing 3 Sample code in C++

```
#include <stdio.h>

int main(){
        int K,X;
        scanf("%d%d",&K,&X);
        if(500*K>=X)printf("Yes\n");
        else printf("No\n");
}
```

B: Count ABC

For all i ($0 \leq i \leq n-3$), check if 3 letters starting from S_i is equal to ABC, and it's sufficient. The following is a sample code in C++.

Listing 4 Sample Code in C++

```
#include<bits/stdc++.h>
using namespace std;

int main() {
    int n;
    string s;
    cin >> n >> s;
    int ans = 0;
    for(int i = 0; i <= n - 3; i++) {
        if(s[i] == 'A' && s[i + 1] == 'B' && s[i + 2] == 'C') ans++;
    }
    cout << ans << endl;
    return 0;
}
```

C: Count Order

X is the k-th in a lexicographical order if and only if there exist $k-1$ permutations that are lexicographically smaller than X. Therefore, you can solve this problem by generating all the permutations of length N, and counting how many permutations are smaller than P, Q.

When generating permutations, you can perform searching such as Depth-First Search, or you can also utilize functions generating permutations such as next_permutation function in C++.

D.SemiCommonMultiple(writer : yuma000)

The condition that X is a semi-common multiple of A can be rephrased as follows:

- For all $k(1 \leq k \leq N)$, there exists a non-negative integer p such that $X = (a_k/2) * (2p+1)$.

Since X is an odd-number-times multiple of $a_k/2$, it can be divided into the following two conditions:

- For all $k(1 \leq k \leq N)$, X is a multiple of $a_k/2$.
- For all $k(1 \leq k \leq N)$, the number of times X is divided by 2 is equal to the number of times $a_k/2$ is divided by 2.

If there exist i, j such that the number of times a_i is divided by 2 and the number of times a_j is divided by 2, then there does not exists X such that satisfies the conditions.

Assume that there doesn't exist such i, j. Let T be the least common multiple of T. Then, odd-number-times multiples of T satisfies the conditions, so you can find the number of X that meets the conditions.

E: Change a Little Bit

First, let's fix a pair of S, T and consider $f(S, T)$.

Obviously you don't need to change the same elements between S, T. Here, assume that $p_1, p_2, \cdots, p_k$ are the indices of S and T where they differs each other, and we change them in that order. In such case the total cost is $k * C_{p_1} + (k-1) * C_{p_2} + \cdots + 1 * C_{p_k}$, so it is optimal to change in the ascending order of C. This is followed by the fact that if there exists an i such that $C_{p_i} > C_{p_{i+1}}$, then swapping their order does not worsen the cost.

By the observations above and symmetry, we assume that C is monotonically non-increasing. For all pairs of $S \neq T$, we want to find $\sum_{t=1}^{k} C_{i_t} * t$ where $S_{i_1} \neq T_{i_1}, S_{i_2} \neq T_{i_2} \cdots S_{i_k} \neq T_{i_k} (1 \leq i_1 < i_2 < \cdots < i_k \leq N)$.

Decompose the contributions of sums, and consider how many times C_i are added. This depends on the number of j such that $j \leq i$ and $S_j \neq T_j$. If there are k such positions, then contributions are $\binom{i-1}{k-1} * 2^k * 2^{i-1-k} * 4^{N-i} * C_i * k$. (Note that there are always two patterns no matter each pair of elements is different or the same.)

By summing up this for each k we obtain $C_i * 4^{N-i} * 2^{i-1} * \sum_{k=1}^{i} \binom{i-1}{k-1} * k$. By regarding the contents of summation as how many times each elements are counted, it appears to be $2^{i-1} * (i+1)$.

Therefore, the answer can be represented as $4^{N-1} * \sum_{i=1}^{N} C_i * (i+1)$.

The bottleneck is the sort, and this problem could be solved in a total of $O(NlogN)$ time.

F:XOR Shift

We assume suffixes to be mod$N$, and $\oplus$ denotes XOR.

$$(1)\ a'_i = b_i \text{for all } i$$

implies that

$$(2)\ a'_i \oplus a'_{i+1} = b_i \oplus b_{i+1} \text{for all } i$$

. Here, it holds that $a'_i \oplus a'_{i+1} = (a_{i+k} \oplus x) \oplus (a_{i+k+1} \oplus x) = a_{i+k} \oplus a_{i+k+1}$, so condition (2) depends on only k. Such k can be enumerated by KNP algorithm or any other in a total of $O(N)$ time. (Let $c_i = a_i \oplus a_{i+1}$, $d_i = b_i \oplus b_{i+1}$, and find c in d)

Conversely, for such k that satisfies (2), condition (1) is satisfied if and only if x is $b_i \oplus a_{i+k}$. Therefore, for each k you can find such x in a constant time. As above this problem can be solved in a total of $O(N)$ time.

If you are not familiar with XOR, it may be easier to understand if you first consider the same things with sums instead of XOR. In such case condition (2) will be $a'_i - a'_{i+1} = b_i - b_{i+1}$.

ABC 151 解説

writers: beet, DEGwer, kort0n, kyopro_friends, satashun, tozangezan

2020 年 1 月 12 日

For International Readers: English editorial starts on page 7.

A: Next Alphabet

一つの方法として考えられるのは、25 種類の入力に対して if 文を書き、出力する文字を決めるというものです。

しかし、英小文字は文字コード上で順に並んでいるため、入力に 1 を足すことで正解を得られます。(実は厳密にはどのような環境でも文字コード上で順に並んでいることが保証されているわけではありません。しかし AtCoder を含む現代のパソコン上では、ほとんどの場合このように順に並んでいます。)

Listing 1　C++ での実装例

```
#include <stdio.h>

char in[2];
int main(){
        scanf("%s",in);
        in[0]++;
        printf("%s\n",in);
}
```

B:Achieve the goal

$O(NK)$ 解法

最後のテストの得点を 0 点から K 点まで順に全探索し、初めて平均点が M 点以上になった時の点数が答えです。

$O(N)$ 解法

「N 科目の平均点が M 点以上」というのは、「N 科目の合計点が $N*M$ 点以上」と同じです。よって、最後のテストで は $N*M-(A_1+\ldots+A_{N-1})$ 点以上を取れば目標を達成できます。

C: Welcome to AtCoder

各問題について, 「その問題で既に AC が出たか」及び「その問題で WA が出た回数」を記録しながら, 提出を順に調べて処理します.
WA が出たときは, その問題の WA 回数を 1 増加させます
AC が出たときは, その問題での AC が初めてでなければ, 何もしません. 初めての場合は, 高橋君の正答数が 1 増え, ペナルティが「その問題で WA が出た回数」だけ増加します.
以上の処理は $O(N+M)$ であり, 十分高速です.
C++ での解答例: `https://atcoder.jp/contests/abc151/submissions/9430678`

D:Maze Master

スタート地点を決めたとき、ゴール地点はスタート地点から最も遠い箇所にすべきです。そのような地点は BFS などにより $O(HW)$ で求めることができます。したがって、各マスをスタート地点とした場合についてをそれぞれ計算することで $O((HW)^2)$ で問題が解けました。なお、ワーシャルフロイド法を用いて $O((HW)^3)$ で全点間最短距離を求めることでも解けます。

E:Max-Min Sums

A は予めソートされているとします。また、N までの階乗とその逆元を予め計算しておくことで、二項係数は高速に求められるとして良いです。

max と min について、それぞれ別々に和をとることを考えます。すなわち $\sum f(S)$ ではなく $(\sum \max S) - (\sum \min S)$ を考えます。

簡単のために、A_i は相異なるとします。$\max S$ の値としてありえるのは A の要素のいずれかです。したがって、各 i について $\max S = A_i$ となるような S がいくつあるかがわかれば、$\sum \max S$ を求めることができます。$\max S = A_i$ となる必要十分条件は「S は A_i を含み、かつ、A_i より小さなもの $K-1$ 個を含む」であるので、このようなものは二項係数を用いて直ちに計算することができます。$\sum \min S$ も同様にして求めることができます。

A_i が重複するものを持つ場合、値が等しい A_i たちの間には勝手な大小関係を入れても、上で述べた説明は成立することが確かめられます (例えば (A_i, i) の辞書順による順序を考え、$\max S = (A_i, i)$ となるものの個数を考えます)。したがって、この場合も全く同様に求められます。

F: Enclose All

この問題は以下のような問題と同値です。

問題': N 個の点 (x_i, y_i) が与えられる。それぞれの点を中心とし、半径が r の円のいずれにも含まれる点が存在するための r の最小値を求めよ。

ある半径 R に対し、考えられる N 個の半径 R の円に共通部分が存在するなら、そのどこかに 2 円の交点が存在します。この交点は、もちろん N 個の円どれからも距離 R 以下です。

よって、以下のように求める答えについての二分法をすれば答えが求められます。

- 半径 R を決める。
- N 個の半径 R の円を列挙し、2 つの円の交点を全列挙します。
- それぞれの交点に対し、N 個の点どれからも距離が R 以下かを判定します。
- 上記の条件を満たす交点が 1 つでも存在したら、求める答えは R 以下です。そうでないなら R よりも大きいです。

以上の二分法は、各ステップ $O(N^3)$ であり、50 回程度回せば十分です。(ただし誤差に気をつけてください。候補となっている点が 2 つの円の交点なので、それらの円の中心からはちょうど距離 R なので、そのような点をスキップするか、非常に小さい値を足して判定する必要があります)

半径が等しい 2 つの円の交点は、以下のように求められます。

- 2 つの交点間の距離 h を求める (三平方の定理より、円と円の距離と円の半径から求められます。ただし円と円の距離が遠く、交点が存在しない場合気をつけてください)
- 2 つの円の中心の中点から、2 つの円の中心を結ぶ直線と垂直方向に距離 h の点を取る (2 つあります)
- それら 2 つの点が 2 つの円の交点である。

A: Next Alphabet

One possible way is to write if statements for 25 kinds of input and determine the character to output.

However, the lowercase English letters are aligned consecutively on the character codes, so you can obtain the answer by adding 1 to the input. (Actually, it is not guaranteed that those are aligned consecutively on the character codes in every environment. However, in the modern computers including AtCoder, in most case they are aligned consecutively like this.)

Listing 2 Sample Code in C++

```
#include <stdio.h>

char in[2];
int main(){
        scanf("%s",in);
        in[0]++;
        printf("%s\n",in);
}
```

B:Achieve the goal

$O(NK)$ Solution

Brute force through all the possible scores from 0 points to K points, and when the average score is more than or equal to K points for the first time, the score then is the answer.

$O(N)$Solution

"The average score of K subjects are more than or equal to M" is equivalent to "the sum of scores of K subjects is more than or equal to $N * M$." Therefore, he can achieve the goal if he gets more than or equal to $N * M - (A_1 + \ldots + A_{N-1})$ points for the last test.

C: Welcome to AtCoder

For each problem, record "whether AC has already appeared for the problem or not" and "the number of WAs appeared so far for the problem," while processing the submissions in order.
If the verdict was WA, then increase the number of WAs for the problem by one.
If the verdict was AC, then if it is not the first AC for the problem, then do nothing. If it was the first one, the number of Takahashi's correct answers increases by one, and the penalties increases by "the number of WAs appeared so far for the problem."
The process above is $O(N+M)$ and fast enough.
Sample Code in C++: `https://atcoder.jp/contests/abc151/submissions/9430678`

D:Maze Master

When a starting square is fixed, it is optimal to set the goal square to be the furthest square from the starting square. Such square can be found by BFS or any other algorithms in a total of $O(HW)$ time. Therefore, by calculating for each square as a starting square, this problem could be solved in a total of $O((HW)^2$ time. Note that you can also calculate in a total of $O((HW)^3)$ time by applying Warshall - Floyd Algorithm.

E:Max-Min Sums

Assume that A is sorted beforehand. Also, you can assume that you can calculate binomial coefficients fast by precalculating the factorials till N and their inverses.

Consider calculating the sum separately for min and max. In other words, consider $(\sum \max S) - (\sum \min S)$ instead of $\sum f(S)$.

For simplicity, assume that A_i is distinct from each other. Possible value of $\max S$ is any element of A. Therefore, by counting the number of S such that $\max S = A_i$ for each i, you can find $\sum \max S$. The necessary and sufficient condition of $\max S = A_i$ is "S contains A_i, and also contains $K-1$ elements less than A_i," so such number can be directly calculated by using binomial coefficients. You can calculate $\sum \min S$ similarly.

If A_i contains duplicates, you can prove that the explanation above also holds if you assume arbitrary order between A_is with same value (for example, consider a lexicographical order of $(A_i, i$ and count the number of elements satisfying $\max S = (A_i, i)$. Therefore, you can also process in the same way in this case.

F: Enclose All

This problem is equivalent to the following problem.

Problem': You are given N points, (x_i, y_i). Find minimum r such that there exists a point contained in all the circles of radius r whose centers are the given points.

For a radius r, if there exists an intersection area of N circles of radius R, then it contains an intersection point of some two circles somewhere. Of course, the distance between this intersection point and any center of circle is less than or equal to R.

Therefore, the answer can be found by a binary search for the desired answer.

- Fix a radius R.
- Enumerate the N circles of radius R, and then enumerate all the intersection points of all pair of circles.
- For each intersection, check if the distance from the N points are less than or equal to R or not.
- If any intersection satisfies the condition above, then the answer is less than or equal to R. Otherwise it is more than R.

The binary search above costs $O(N^3)$ time for each step, and it is enough to loop about 50 times. (Be careful of the precision error. The candidate point is an intersection of two circles, so the distance from the centers of those two circles are exactly R, so you have to skip such points or judge by adding very small value.)

The intersection of two circles of same radius can be found by the following steps:

- Find the distance h between the two intersection (by the Pythagorean theorem, this can be found from the distance between the two circles and the radius of the circles. Note that you have to take care of the cases where the circles are far from each other and intersections do not exist)
- Find a vector of length h starting from the midpoint of the centers of two circles and take its endpoint (there are two of them)
- Those two points are the intersections of the two circles.

ABC 152 解説

writers: latte0119, beet, DEGwer, drafear, IH19980412, kyopro_friends, tozangezan, yokozuna57

2020 年 1 月 19 日

For International Readers: English editorial starts on page 7.

A: AC or WA

$N = M$ であるかどうか判定すればよいです。以下は C++ における実装例です。

```
#include<iostream>
using namespace std;

int main(){
        int n,m;
        cin>>n>>m;
        if(n==m)cout<<"Yes"<<endl;
        else cout<<"No"<<endl;
        return 0;
}
```

B: Comparing Strings

考えられる方法の一つは、2 つの文字列を実際に作り、辞書順比較することです。辞書順比較は、例えば C++ ならば string 型同士で比較をすることで可能です。

しかし、a を b 回繰り返した文字列は a で始まり、b を a 回繰り返した文字列は b で始まるため、$a < b$ ならば常に a を b 回繰り返した文字列の方が辞書順で小さいですし、$a > b$ ならば常に b を a 回繰り返した文字列の方が辞書順で小さいです。また、$a = b$ なら二つの文字列が等しくなるのでどちらを出力しても同じになります。

Listing 1　C++ での実装例 1

```
#include <stdio.h>
#include <string>
#include <algorithm>
using namespace std;
char in[110];
int main(){
        int a,b;scanf("%d%d",&a,&b);
        for(int i=0;i<b;i++)in[i]='0'+a;
        string L=in;
        for(int i=0;i<a;i++)in[i]='0'+b;
        in[a]=0;
        string R=in;
        printf("%s\n",min(L,R).c_str());
}
```

Listing 2　C++ での実装例 2

```
#include <stdio.h>

int main(){
        int a,b;scanf("%d%d",&a,&b);
        if(a<b){
                for(int i=0;i<b;i++)printf("%d",a);
        }else{
                for(int i=0;i<a;i++)printf("%d",b);
        }
        printf("\n");
}
```

C: Low Elements

$i = 1, ..., N$ に対して、$M_i := min\{P_j | 1 \leq j \leq i\}$ と定義します。すると、$i(1 \leq i \leq N)$ を固定して考えたときに、 $M_i = P_i$ であることと、 任意の $j(1 \leq j \leq i)$ に対して $P_i \leq P_j$ であることが同値となります。したがって、 M_i をすべて計算することができれば十分です。これは、 i が小さいほうから順に計算していけばよく、 $O(N)$ でこの問題を解くことができました。

D: Handstand 2

整数 $i, j (0 \leq i, j \leq 9)$ に対して、
$c_{i,j} := \#\{1 \leq k \leq N | k$ の先頭の桁の数は i に等しく、末尾の桁の数は j に等しい $\}$
と定義します。これは、$O(N)$ で計算することができます。$c_{i,j}$ を用いると、この問題の答えは
$\sum_{i=0}^{9} \sum_{j=0}^{9} c_{i,j} \times c_{j,i}$
と計算することができます。

E: Flatten

条件を満たすような B_i を決めたとします。このとき $K = A_1B_1$ とおくと、条件より全ての $j > 1$ について $K = A_1B_1 = A_jB_j$ となります。よって K は $A_1, ..., A_N$ の公倍数です。逆に L を $A_1, ..., A_N$ の最小公倍数とおいたとき、$B_i = L/A_i$ と定めると、このような B_i たちは条件を満たします。よって求める答えは、$\sum L/A_i$ です。しかし、L は非常に大きな数となることがあるため、そのまま計算することはできません。そこで、L を素因数分解した形で保持し、計算することを考えます。

p_i を素数とし、$X = \prod p_i^{e_i}$, $Y = \prod p_i^{e'_i}$ と[*1]素因数分解されているとします (e_i, e'_i の一方は 0 でもよい)。このとき、X と Y の最小公倍数は $\prod p_i^{\max(e_i, e'_i)}$ になります。このことを用いると、$A_1, ..., A_N$ の最小公倍数を素因数分解された形で求めることができます。

以上より、この問題は $A = \max A_i$ として、$O(N\sqrt{A})$ で解くことができました。また、適当な前計算により素因数分解を高速化するなどの工夫により $O(A + N \log A)$ で求めることもできます。

[*1] 記号 $\prod$ は、$\sum$ の"掛け算バージョン"です

F: Tree and Constraints

”すべての条件を満たす塗り方”を数える代わりに、” 1 つ以上の条件を満たさない塗り方”を数えることにします。

包除原理を使うと、 M 個の制約から 1 つ以上の制約を選んで、選んだ制約を 1 つも満たさないような塗り方を数える問題になります。

制約の選び方を 1 つ固定します。選ばれた制約を 1 つも満たさないような塗り方の数を効率的に計算することを考えます。選ばれた制約に対応するパス上に存在する辺は、必ず白く塗らなければなりません。そうでない辺は、どちらの色で塗ってもよいです。したがって、選ばれた制約に対応するパスたちに覆われている辺の数を C とおくと、選ばれた制約を 1 つも満たさないような塗り方の数は、 2^{N-1-C} 通りです。

適当に根を選んで根付き木として考え、各 u_i, v_i に対して最近共通祖先 (LCA) を事前に求めておきます。すると、 C の計算の際には、木上で累積和を用いればよく、 $O(N+M)$ で実行できます。

以上より、 $O((N+M)2^M)$ でこの問題を解くことができました。

A: AC or WA

It is enough to check if $N = M$. The following is a sample code in C++.

```
#include<iostream>
using namespace std;

int main(){
        int n,m;
        cin>>n>>m;
        if(n==m)cout<<"Yes"<<endl;
        else cout<<"No"<<endl;
        return 0;
}
```

B: Comparing Strings

One possible way is constructing to strings actually and compare lexicographically. This can be achieved by, for example in C++, comparing between string type.

However, the concatenation of b copies of a starts with a, and the concatenation of a copies of b starts with b, so if $a < b$ then the concatenation of b copies of a is the smaller lexicographically, and if $a < b$ then the concatenation of a copies of b is the smaller lexicographically. Also, if $a = b$, the two strings are the same, so you will obtain the same result by printing either of them.

Listing 3 Sample Code in C++ 1

```
#include <stdio.h>
#include <string>
#include <algorithm>
using namespace std;
char in[110];
int main(){
        int a,b;scanf("%d%d",&a,&b);
        for(int i=0;i<b;i++)in[i]='0'+a;
        string L=in;
        for(int i=0;i<a;i++)in[i]='0'+b;
        in[a]=0;
        string R=in;
        printf("%s\n",min(L,R).c_str());
}
```

Listing 4 Sample Code in C++ 2

```
#include <stdio.h>

int main(){
        int a,b;scanf("%d%d",&a,&b);
        if(a<b){
                for(int i=0;i<b;i++)printf("%d",a);
        }else{
                for(int i=0;i<a;i++)printf("%d",b);
        }
        printf("\n");
}
```

C: Low Elements

For $i = 1, ..., N$ we define $M_i := min\{P_j | 1 \leq j \leq i\}$. Then, when $i (1 \leq i \leq N)$ is fixed, $M_i = P_i$ holds if and only if for all $j (1 \leq j \leq i)$, $P_i \leq P_j$ holds. Therefore, it is enough to calculate all the M_i. These can be calculated in the increasing order of i, so the problem could be solved in a total of $O(N)$ time.

D: Handstand 2

For each pair of integers $i, j (0 \leq i, j \leq 9)$, let us define
$c_{i,j} := \#\{1 \leq k \leq N |$ The first digit of k is equal to i, and the last digit is equal to j $\}$.
Those can be calculated in a total of $O(N)$ time. by using $c_{i,j}$, the answer for the problem can be calculated by
$\sum_{i=0}^{9} \sum_{j=0}^{9} c_{i,j} \times c_{j,i}$.

E: Flatten

Assume that B_i satisfying the given conditions are determined. Then let $K = A_1 B_1$, then by the constraints $K = A_1 B_1 = A_j B_j$ holds for all $j > 1$. Therefore K is common multiple of $A_1, ..., A_N$. Conversely, let L be the least common multiple of $A_1, ..., A_N$, and let $B_i = L/A_i$, then such B_is satisfies the conditions. Therefore, the desired answer is $\sum L/A_i$. However, L can be a very big number, so it can't be calculated directly. Then let's consider calculating, holding L in a factorized form.

Let p_i be primes, and assume that factorizations are given by $X = \prod p_i^{e_i}$, $Y = \prod p_i^{e'_i}$ [*1] (either of e_i, e'_i may be 0). Then the least common multiple of X and Y is given by $\prod p_i^{\max(e_i, e'_i)}$. By using this, the least common multiple of $A_1, ..., A_N$ can be obtained in a factorized form.

Therefore, this problem can be solved in a total of $O(N\sqrt{A})$ time, where $A = \max A_i$. Also, by speeding up the prime factorization with proper precalculations, the answer can also be obtained in a total of $O(A + N \log A)$ time.

[*1] The symbol $\prod$ is a "product version" of $\sum$

F: Tree and Constraints

Instead of counting the number of "ways of painting that satisfy all the conditions," let us count the number of "ways of painting that do not satisfy one or more conditions."

By the inclusion-exclusion principle, it is rephrased to the problem of choosing one or more constraints out of M constraints and count the number of such ways of painting that do not satisfy any of the chosen constraints.

Fix a combination of constraints. Consider how we can efficiently count the such ways of painting that do not satisfy the chosen constraints. The edges on the paths corresponding to the chosen constraints must be painted white. The other edges can be painted with either color. Therefore, let C be the number of edges covered by the paths corresponding to the chosen constraints, then there are 2^{N-1-C} ways of painting such that do not satisfy any of the chosen constraints.

Regard as a rooted tree by choosing an arbitral root, and for each u_i, v_i find their Lowest Common Ancestor (LCA) beforehand. Then, when calculating C you can use cumulative sums, so it can be executed in a total of $O(N + M)$ time.

Therefore, this problem could be solved in a total of $O((N + M)2^M)$ time.

ABC 153 解説

writer: kyopro_friends

2020 年 1 月 26 日

For International Readers: English editorial starts on page 7.

A: Serval vs Monster

求める答えは H/A の切り上げです。ceil 関数などを使ってもよいですし、$(H+A-1)/A$ の切り捨てにより求めることもできます。

C 言語での実装例は次のとおりです。

```
#include<stdio.h>
int main(){
    int h,a;
    scanf("%d%d",&h,&a);
    printf("%d\n",(h+a-1)/a);
}
```

このほか、「H が 0 以下になるまで A を引く」というループ処理で解くこともできます。

B: Common raccoon vs Monster

全ての必殺技を 1 度ずつ使ってモンスターの体力を 0 以下にできれば良いです。つまり、$H - (A_1 + A_2 + \ldots + A_N)$ が 0 以下のとき Yes、そうでないとき No となります。

C 言語での実装例は次のとおりです。

```
int main(){
    int H,N;
    scanf("%d%d",&H,&N);
    for(int i=0;i<N;i++){
        int A;
        scanf("%d",&A);
        H-=A;
    }
    if(H<=0)puts("Yes");
    else puts("No");
}
```

C: Fennec vs Monster

体力の高い方から K 体のモンスターに必殺技を使い、残りのモンスターに攻撃するのが最適です。モンスターを体力の順にソートすることで $O(N \log N)$ で解くことができます。

D: Caracal vs Monster

体力が H のモンスター 1 体に勝つために必要な攻撃回数を $f(H)$ とします。このとき

$$f(H) = \begin{cases} 2 \times f(\lfloor H/2 \rfloor) + 1 & (H > 1) \\ 1 & (H = 1) \end{cases}$$

となります。この定義に従って再帰的に計算することで答えを求めることができます。

E: Crested Ibis vs Monster

次のような DP を考えることで解くことができます。

$$DP[i] = \text{モンスターの体力を } i \text{ 減らすため消耗する魔力の最小値}$$

同じ魔法を複数回使うことができるので、DP 配列の更新順序には気をつけてください。答えは $\min_{H \leq i < H + \max A_i} DP[i]$ になります (モンスターの体力を $H + \max A_i$ 以上減らすには、H 以上減らしてからさらに魔法を使う必要があるので無駄です)。この部分は、「$i \geq H$ であるような部分について予めまとめた状態で DP 配列を持つ」という実装の工夫により簡略化することもできます。計算量は $O(NH)$ です。

なお、答えは最大で $H * \max A_i$ 程度になるので、次のような DP では解けないことに注意してください。

$$DP[i] = \text{魔力を } i \text{ 消耗したときに減らせるモンスターの体力の最大値}$$

F: Silver Fox vs Monster

モンスターたちはあらかじめ座標の昇順にソートされているとしてよいです。座標が小さい方を”左”、大きい方を”右”という向きで呼ぶことにします。

最も左にいるモンスターを倒すためときには、できるだけ右で爆弾を使い、他のモンスターを多く巻き込むのが良いです。そのようにして左側から貪欲に倒していくことが最適になります。

尺取り法や二分探索などを用いて、各モンスター i について「$X_j - X_i \leq 2D$ となる最大の j」を計算しておくことで、モンスター i を左端として攻撃するときにどこまで右側のモンスターを巻き込むことができるかわかります。あとは、累積和などを用いて既に減らした体力量を管理することで答えを求めることができます。

最初に座標順にソートする部分がボトルネックとなり、計算量は $O(N \log N)$ です。

A: Serval vs Monster

The desired answer is H/A rounded up. You can use ceil functions etc., or it can be also calculated by rounding down $(H + A - 1)/A$.

The following is a sample code in C language.

```
#include<stdio.h>
int main(){
    int h,a;
    scanf("%d%d",&h,&a);
    printf("%d\n",(h+a-1)/a);
}
```

Otherwise, this problem can also be solved by loop operations of "subtracting by A until H becomes 0 or below."

B: Common raccoon vs Monster

It is enough if the monster's health become 0 or below by using all the Special Move once each. In other words, the answer is Yes if $H - (A_1 + A_2 + \ldots + A_N)$ is 0 or below, and No otherwise.

The following is a sample code in C language.

```
int main(){
    int H,N;
    scanf("%d%d",&H,&N);
    for(int i=0;i<N;i++){
        int A;
        scanf("%d",&A);
        H-=A;
    }
    if(H<=0)puts("Yes");
    else puts("No");
}
```

C: Fennec vs Monster

It is optimal to use Special Move for K monsters with the largest health, and to Attack to the other monsters. By sorting the monsters by the monsters' healths, it can be solved in a total of $O(N \log N)$ time.

D: Caracal vs Monster

Let $f(H)$ be the number of attacks needed to defeat one monster of health H. Then the following relation holds:

$$f(H) = \begin{cases} 2 \times f(\lfloor H/2 \rfloor) + 1 & (H > 1) \\ 1 & (H = 1) \end{cases}$$

The answer can be found by calculating recursively following the definition above.

E: Crested Ibis vs Monster

This problem cane be solved by considering the following DP.

$DP[i]$ = The minimum total Magic Points that have to be consumed to decrease the monster's health by i

Since the same spell can be cast multiple times, be careful of the order of refreshing the DP array. The answer is $\min_{H \leq i < H + \max A_i} DP[i]$ (in order to decrease the monster's health by $H + \max A_i$ or more, an additional spell has to be cast after it is decreased by H or more, so it's meaningless). This part can be simplified by ingenious implementation of "holding a DP array in which the part of $i \geq H$ are put together. The total time complexity is $O(NH)$.

Note that it cannot be solved by the following DP, since the answer can be $H * \max A_i$ at most.

$DP[i]$ = The maximum total health that can be decreased when the i Magic Points are consumed

F: Silver Fox vs Monster

You can assume that the monsters are sorted in the increasing order of coordinates. We denote the direction of smaller coordinates by being "left," and the larger coordinates by being "right."

In order to defeat the leftmost monster, it is good to use the bomb as in the right as possible and get involved as many monsters as possible. It is optimal to defeat greedily from the left in such a way.

By precalculating "the maximum j such that $X_j - X_i \leq 2D$" for each monster i utilizing sliding windows or binary searching etc., it can be determined how many monsters in the right can be involved when monster i is attacked at the left end. All the left is managing the amount of health already decreased, using cumulative sums etc., then the answer can be found.

The bottleneck is the first step, sorting by the coordinates, so the total time complexity is $O(N \log N)$.

ABC 154 解説

writer: DEGwer, kyopro_friends, latte0119, sheyasutaka, tozangezan

2020 年 2 月 9 日

For International Readers: English editorial starts on page 7.

A: Remaining Balls

$S = U, T = U$ のどちらであるか判定し、場合分けすればよいです。以下は C++ における実装例です。

```
#include<iostream>
using namespace std;

int main(){
        string S,T,U;
        int A,B;
        cin>>S>>T>>A>>B>>U;
        if(S==U)A--;
        else B--;
        cout<<A<<"␣"<<B<<endl;
}
```

B: I miss you...

問題文の通りに実装してもよいですし，S の長さと同じ数だけ `x` を出力すると捉えて簡潔に書くこともできます.

実装例は次のとおりです.

Listing 1　C での実装例

```c
#include <stdio.h>

char s[101];

int main(void){
        scanf("%s", s);

        for (int i = 0; s[i]; i++) {
                s[i] = 'x';
        }

        puts(s);

        return 0;
}
```

Listing 2　Python3 での実装例

```python
s = input().rstrip()
print('x' * len(s))
```

C: Distinct

単純に全ての 2 つの整数の組に関してループで回し等しいか判定する場合、計算量が $O(N^2)$ となり間に合いません。

ところが、与えられた整数列に等しい値がある場合、この整数列をソートすると隣接する場所にこれらの値が集まることになります。

そのため、一度与えられた数列をソートすると、全ての隣接する 2 項を比較し、等しい値がなければ YES で、等しい値があれば NO です。

C++ の std::set のようなデータ構造を用いて異なる要素数を取得しても大丈夫です。

Listing 3　C++ での実装例

```
#include <stdio.h>
#include <algorithm>
using namespace std;
int A[210000];
int main(){
        int N;scanf("%d",&N);
        for(int i=0;i<N;i++)scanf("%d",A+i);
        std::sort(A,A+N);
        for(int i=0;i<N-1;i++){
                if(A[i]==A[i+1]){
                        printf("NO\n");return 0;
                }
        }
        printf("YES\n");
}
```

D: Dice in Line

それぞれのサイコロの出目は独立に決まります。そのため、いくつかのサイコロの出目の合計の期待値は、それぞれのサイコロの出目の期待値の合計に等しいです。

そのため、あらかじめそれぞれのサイコロの出目の期待値を求めておけば、それらを合計することでサイコロの出目の合計の期待値を求めることができます。

1 から P までの P 種類の目が等確率で出るサイコロの出目の期待値は $(1+P)/2$ です。

全ての隣接する K 個のサイコロについて、この値の合計を高速に求める必要があります。これは、累積和を使って求めることができます。具体的には、E_i を i 番目のサイコロの出目の期待値としたとき、$S_i = E_1 + ... + E_i$ となる S_i $(0 \leq i \leq N)$ を計算しておけば、隣接する K 個の E_i の和は $S_{j+K} - S_j$ のように求められます。

計算量は期待値の前計算、累積和の計算、隣接する K 項の和の計算、どれも $O(N)$ となります。

E: Almost everywhere zero

必要なら上の桁を 0 で埋めることで、N と同じ桁数の数だけを考えれば良いです。

桁 DP と呼ばれる考え方を用いて解くことができます。

$$
\begin{aligned}
dp0[i][j] = &\text{上から } i \text{ 桁目まで決めて、0 でない桁が } j \text{ 個あり、} \\
&N \text{ より小さいことが確定している} \\
dp1[i][j] = &\text{上から } i \text{ 桁目まで決めて、0 でない桁が } j \text{ 個あり、} \\
&N \text{ より小さいことが確定していない}
\end{aligned}
$$

として、i が小さい方から順に計算していきます。求める答えは N の桁数を L として、$dp0[L][K] + dp1[L][K]$ です。計算量は $O(LK)$ です (実装によっては基数 10 が定数倍としてつきます)。

F: Many Many Paths

$g(r,c)$ を $0 \leq i \leq r$ かつ $0 \leq j \leq c$ を満たす全ての整数の組 (i,j) に対する $f(i,j)$ の総和とします。すると、求める値は $g(r2,c2) - g(r2,c1-1) - g(r1-1,c2) + g(r1-1,c1-1)$ となります。よって、 $g(r,c)$ が高速に計算できればこの問題が解けます。

唐突ですが、$f(r+1,c) = f(r,0) + f(r,1) + ... + f(r,c)$ が成り立ちます。これは、点 $(0,0)$ から点 $(r+1,c)$ への経路について考えてみると説明できます。点 $(0,0)$ から点 $(r+1,c)$ へのどの経路も、「点 $(0,0)$ から点 $(r,0)$ に至り、そのまま $(r+1,0)$ に移動し、点 $(r+1,c)$ へ移動」「点 $(0,0)$ から点 $(r,1)$ に至り、そのまま $(r+1,1)$ に移動し、点 $(r+1,c)$ へ移動」「点 $(0,0)$ から点 $(r,2)$ に至り、そのまま $(r+1,2)$ に移動し、点 $(r+1,c)$ へ移動」...「点 $(0,0)$ から点 (r,c) に至り、そのまま $(r+1,c)$ に移動し、点 $(r+1,c)$ へ移動」のうちいずれか 1 つに必ず該当し、2 つ以上に該当することはありません。

ところで点 $(0,0)$ から点 $(r+1,c)$ への経路数が $f(r+1,c)$ で、点 $(0,0)$ から点 (r,i) に至り、そのまま $(r+1,i)$ に移動し、点 $(r+1,c)$ へ移動する経路数は、「点 $(0,0)$ から点 (r,i) に至る」のが $f(r,i)$ 通り、そのまま 1 回下に移動、残りはひたすら右に移動を繰り返すことが強制されているので、点 (r,i) から先は 1 通り、すなわち全部で $f(r,i)$ 通りです。以上より式 $f(r+1,c) = f(r,0) + f(r,1) + ... + f(r,c)$ が成り立ちます。

また $f(r,c) =_{r+c} C_c = (r+c)!/(r!c!)$ です。よって、求めたい値は $O(N)$ 個の $f(x,y)$ の和で書き表わせるので、$f(x,y)$ が $O(1)$ で求められれば良いです。これは階乗と階乗の逆元を前計算しておけば良いです。

また、$f(r,c+1) = f(0,c) + f(1,c) + ... + f(r,c)$ をもう一度適用すると、求めたい値は定数個の $f(x,y)$ の和でも書き表すことができます。

A: Remaining Balls

It is enough splitting conditions by judging whether $S = U$ or $T = U$. The following is the sample code in C++.

```
#include<iostream>
using namespace std;

int main(){
        string S,T,U;
        int A,B;
        cin>>S>>T>>A>>B>>U;
        if(S==U)A--;
        else B--;
        cout<<A<<" "<<B<<endl;
}
```

B: I miss you...

You can implement just the same way of the instructions of the problem statement, or you can write concisely by rephrasing as outputting `x`'s for same times of the length of S.

Sample code is as follows.

Listing 4 Sample Code in C

```c
#include <stdio.h>

char s[101];

int main(void){
        scanf("%s", s);

        for (int i = 0; s[i]; i++) {
                s[i] = 'x';
        }

        puts(s);

        return 0;
}
```

Listing 5 Sample Code in Python3

```python
s = input().rstrip()
print('x' * len(s))
```

C: Distinct

If you naively check for all pairs of integers and check if they are the same, it will take a total of $O(N^2)$ time, so it won't finish in time.

However, if the given sequence of integers has duplicates, then after sorting this sequence of integers, such values gather to the consecutive positions.

Therefore, once you sorted the given sequence, you can compare all adjacent pair of terms, and if none of them are the same the answer is YES, otherwise the answer is NO.

It is OK to use data structures such as std::set in C++ to obtain the number of distinct elements.

Listing 6 Sample Code in C++

```
#include <stdio.h>
#include <algorithm>
using namespace std;
int A[210000];
int main(){
        int N;scanf("%d",&N);
        for(int i=0;i<N;i++)scanf("%d",A+i);
        std::sort(A,A+N);
        for(int i=0;i<N-1;i++){
                if(A[i]==A[i+1]){
                        printf("NO\n");return 0;
                }
        }
        printf("YES\n");
}
```

D: Dice in Line

The numbers shown on the dices are independent from each other. Therefore, the expected value of sum of the numbers shown on some dices are the sum of the expected values of the number shown on each dice.

Therefore, by calculating the expected value of the number shown on each dice beforehand, the expected value of sum of the numbers shown on the dices can be calculated by summing them up.

The expected value of the number shown on the dice that shows P numbers from 1 to P is $(1 + P)/2$.

You have to find the sums of the values for all consecutive K dices. This can be calculated using cumulative sums. In specific, let E_i be the expected value of the number shown on i-th dice, then by calculating S_i such that $S_i = E_1 + ... + E_i$ $(0 \leq i \leq N)$ beforehand, the sum of K consecutive values E_i can be calculated as $S_{j+K} - S_j$.

The time complexity is $O(N)$ for the precalculation of expected values, calculation of cumulative sums, and the calculation of sums of consecutive K terms each.

E: Almost everywhere zero

It is enough to consider the integers of N digits, by filling the higher digits with 0 if necessary. This problem can be solved by applying the method called digit DP. Let

$dp0[i][j]$ = The higher i digits has already been decided, and there are j non-zero digits, and it has already been determined that it is less than N,
$dp1[i][j]$ = The higher i digits has already been decided, and there are j non-zero digits, and it has not yet been determined that it is less than N,

and calculate them in the ascending order of i. The desired answer is $dp0[L][K] + dp1[L][K]$, where L is the number of digits of N. The total time complexity is $O(LK)$ (depending on implementation, radix 10 may be multiplied as a constant).

F: Many Many Paths

Let $g(r, c)$ be the sum of $f(i, j)$ for all pairs of integers (i, j) such that $0 \leq i \leq r$ and $0 \leq j \leq c$. Then, the desired answer is $g(r2, c2) - g(r2, c1 - 1) - g(r1 - 1, c2) + g(r1 - 1, c1 - 1)$. Therefore, this problem can be solved if $g(r, c)$ can be calculated fast.

It's a bit kind of sudden, but it holds that $f(r + 1, c) = f(r, 0) + f(r, 1) + ... + f(r, c)$. This can be explained by considering the paths from the point $(0, 0)$ to the point $(r + 1, c)$. Every path from the point $(0, 0)$ to the point $(r + 1, c)$ is either of such paths that "from the point $(0, 0)$, reach the point $(r, 0)$, and move to $(r + 1, 1)$ right away, then move to $(r + 1, c)$", "from the point $(0, 0)$, move to the point $(r, 1)$, and move to $(r + 1, 1)$ right away, then move to $(r + 1, c)$", "from the point $(0, 0)$, move to the point $(r, 2)$, and move to $(r + 1, 2)$ right away, then move to $(r + 1, c)$", ..., or "from the point $(0, 0)$, move to the point (r, c), and move to $(r + 1, c)$ right away, then move to $(r + 1, c)$"; it does not correspond to more than one of them.

By the way, the number of paths from the point $(0, 0)$ to the point $(r + 1, c)$ is $f(r + 1, c)$, and the number of the path such that, from the point $(0, 0)$, move to the point (r, i), and move to $(r + 1, i)$ right away, then move to $(r + 1, c)$ can be calculated as follows: there are $f(r, i)$ paths such that "from the point $(0, 0)$, move to the point (r, i) is $f(r, i)$, then move downwards once, and you are forced to move to the right all the way, so there is a single path from the point (r, i) and beyond, thus the number of paths is $f(r, i)$. Therefore, it holds that $f(r + 1, c) = f(r, 0) + f(r, 1) + ... + f(r, c)$.

Also, it holds that $f(r, c) =_{r+c} C_c = (r + c)!/(r!c!)$. Therefore, the desired answer can be denoted by the sum $f(x, y)$'s, whose number of terms are $O(N)$, so it is enough if $f(x, y)$ can be calculated in $O(1)$ time. To do this, you can precalculate the factorials and the inverses of factorials.

Also, by applying $f(r, c + 1) = f(0, c) + f(1, c) + ... + f(r, c)$ again, the desired answer can be represented by the sum of a constant number of $f(x, y)$'s.

ABC 155 解説

writer: sheyasutaka, rng_58, beet, drafear

2020 年 2 月 16 日

For International Readers: English editorial starts on page 8.

A: Poor

たとえば $A = B$ かつ $B \neq C$ を C 言語における式で表すと `A == B && B != C` となります.
これを踏まえて，過不足なく場合分けして書くと下の実装例のようになります。
C 言語での実装例は次のとおりです.

```
#include <stdio.h>

int main(void){
        int a, b, c;

        scanf("%d%d%d", &a, &b, &c);

        int ispoor = 0;
        if (a == b && b != c) ispoor = 1;
        if (b == c && c != a) ispoor = 1;
        if (c == a && a != b) ispoor = 1;

        if (ispoor) {
                puts("Yes");
        } else {
                puts("No");
        }

        return 0;
}
```

B: Papers, Please

「偶数であるにもかかわらず，3 でも 5 でも割り切れない」ような整数が書類上にあれば DENIED，さもなくば APPROVED とすればよいです．

C 言語での実装例は次のとおりです．

```
#include <stdio.h>

int main (void) {
        int i;

        int a[100];
        for (i = 0; i < n; i++) {
                scanf("%d", &a[i]);
        }

        for (i = 0; i < n; i++) {
                if (a[i] % 2 == 0 && a[i] % 3 != 0 && a[i] % 5 != 0) {
                        puts("DENIED");
                        return 0;
                }
        }
        puts("APPROVED");

        return 0;
}
```

C: Poll

Python や C# などにおける連想配列を使って，各文字列の出現回数を数えてから，最大値をとるものを昇順に出力すればよいです．

std::map の内部ではキーが昇順になるよう要素がソートされているので，イテレータを .begin() から順に見ることで，キーを昇順に見ることができます．

C++ での実装例は次のとおりです．

```
#include <iostream>
using std::cin;
using std::cout;
using std::endl;

#include <map>
using std::map;

#include <string>
using std::string;

map<string, int> memo;

int main (void) {
        int n;

        scanf("%lld", &n);
        for (int i = 0; i < n; i++) {
                string s;
                cin >> s;

                memo[s] += 1;
        }


        int maxv = 0;
        for (const auto& x : memo) {
                int v = x.second;
                if (v > maxv) maxv = v;
        }
        for (auto it = memo.begin(); it != memo.end(); it++) {
                if (it->second == maxv) {
                        cout << it->first << endl;
```

```
34                  }
35          }
36
37          return 0;
38  }
```

D: Pairs

積が負のペア，0 のペア，正のペアの各個数は簡単に求まるので，答えが負・0・正のどれになるかはこれでわかります．

答えが負になる場合は，負の数と正の数を 1 つずつ選んで x 以上になるペアがいくつあるかを数えることが尺取り法によって可能なので，二分探索を用いて答えが求まります．

答えが正になる場合も全く同様ですが，同じ要素を 2 回選ぶこと，それを差し引くと各ペアをちょうど 2 回数えることを考慮しましょう．

E: Payment

各桁について，金額のその桁までを見たときの「ピッタリ払うときの最小」「1 余分に払うときの最小」を上の桁から計算すればよいです．

DP の遷移は桁ごとに定数時間で出来るので，桁数に対して線形時間で解けました．

F: Perils in Parallel

簡単のため，爆弾は座標の昇順に番号づけられているとします (ソートすることで帰着できます).

各コードの挙動は「番号 l_j 未満の爆弾を切り替え，さらに番号 r_j 以下の爆弾を切り替える」と言い換えられます.

爆弾 i と $i+1$ のオン・オフが同時に切り替わらないのは「番号 i 以下の爆弾を切り替える」操作をしたときのみなので，これを用いて各操作の回数の偶奇が何でなければならないかがわかります. 逆に，これを満たせば全ての爆弾がオフになります.

さて，グラフに思いを馳せます．各コードを「2 頂点 l_i, r_i を結ぶ辺」と言い換えます ($l_i = r_i$ となるコードは考えるだけ無駄なので結ばなくてよいです)．そして，「i 以下の…」という操作が奇数となる頂点 i に印をつけます.

このとき，各連結成分について，印の数は偶数でなければなりません (どの辺に対応する 2 操作をしても連結成分内の偶奇は変化しない)．逆に，すべての連結成分について印が偶数個であれば，以下のようにして構成できます.

- 全域木を何でもいいので 1 つとる (明示的でなくてもよく，適当に DFS してとれる経路でよい)．以下これを根付き木として考える.
- 各辺について，その辺よりも下の部分木について印の個数が奇数のとき，またその時に限り，その辺に対応するコードを切る.

以上で構成は完了です.

A: Poor

For example, the condition of $A = B$ and $B \neq C$ can be represented as `A == B && B != C` in C Language.

By splitting cases correctly based on this, it will be like the following implementation example. The following is a sample code in C Language.

```c
#include <stdio.h>

int main(void){
        int a, b, c;

        scanf("%d%d%d", &a, &b, &c);

        int ispoor = 0;
        if (a == b && b != c) ispoor = 1;
        if (b == c && c != a) ispoor = 1;
        if (c == a && a != b) ispoor = 1;

        if (ispoor) {
                puts("Yes");
        } else {
                puts("No");
        }

        return 0;
}
```

B: Papers, Please

If there exists a integer such that "although it is an even number, it is not divisible by 3 nor 5," on the document then the answer is `DENIED`, otherwise the answer is `APPROVED`.

The following is a sample code in C Language.

```
#include <stdio.h>

int main (void) {
        int i;

        int a[100];
        for (i = 0; i < n; i++) {
                scanf("%d", &a[i]);
        }

        for (i = 0; i < n; i++) {
                if (a[i] % 2 == 0 && a[i] % 3 != 0 && a[i] % 5 != 0) {
                        puts("DENIED");
                        return 0;
                }
        }
        puts("APPROVED");

        return 0;
}
```

C: Poll

Using an associative arrays in Python or C#, count the appearances of each string, and output the string taking the maximum value in the increasing order.

Inside std::map, the keys are sorted in the increasing order, so by looking at the iterator in order from .begin(), the keys can be iterated in the increasing order.

The following is an implementation example in C++.

```
#include <iostream>
using std::cin;
using std::cout;
using std::endl;

#include <map>
using std::map;

#include <string>
using std::string;

map<string, int> memo;

int main (void) {
        int n;

        scanf("%lld", &n);
        for (int i = 0; i < n; i++) {
                string s;
                cin >> s;

                memo[s] += 1;
        }


        int maxv = 0;
        for (const auto& x : memo) {
                int v = x.second;
                if (v > maxv) maxv = v;
        }
        for (auto it = memo.begin(); it != memo.end(); it++) {
                if (it->second == maxv) {
                        cout << it->first << endl;
```

```
34                  }
35          }
36
37          return 0;
38  }
```

D: Pairs

The number of pairs such that the products are negative, 0 or positive can be calculated easily, so it is determined whether the answer is negative, 0 or positive.

When the answer is negative, we can count the number of the pairs such that a negative number and a positive number are chosen and the product is more than x by using sliding window, so the answer can be found by binary searching.

The answer can be found likewise when the answer is positive too, but note that you may choose the same element twice, and when they are eliminated each pair is counted twice.

E: Payment

For each digit, calculate "the minimum when paying exactly" and "the minimum when paying one more" when focusing on the prefix of the price until the digit, from the higher to the lower.

Since each DP transition can be performed in a constant time for each digit, the problem has been solved in a linear time of the number of digits.

F: Perils in Parallel

For simplicity, we assume that the bombs are indexed in the increasing order of the coordinates. (It can be achieved by sorting them.)

The behavior of each cord can be reworded to "switch the bombs with the indices less than l_j, and also switch the bombs with the indices less than or equal to r_j."

The activated/deactivated states of bomb i and $i+1$ are switched only when "the bombs with the indices less than or equal to i are switched," so by using the conditions, the parity of the number of each operation is determined. Conversely, if all those parities are satisfied, then all the bombs will be deactivated.

Now, let's think of graphs. Let's regard each code as "an edge connecting between two vertices l_i, r_i" (the cords such that $l_i = r_i$ does not have to be connected because it has no effect). Then, mark vertices i such that the number of operation of "less than or equal to i" is an odd number.

Then, for each connected component, the number of marks has to be even (performing two operations corresponding to any edges does not change the parity in the connected component). Conversely, if the number of marks are even for all the connected components, then the following construction is possible.

- Take an arbitrary spanning tree (not necessarily explicitly; it can be a path of some arbitrary DFS). Hereinafter we will regard this as a rooted tree.
- For each edge, cut the corresponding cord if and only if the number of marks in the subtree under the edge is odd.

Here the construction ends.

ABC 156 解説

writer: gazelle, kort0n, namonakiacc

2020 年 2 月 22 日

For International Readers: English editorial starts on page 10.

A: Beginner

(原案: kort0n, 準備・解説: gazelle)

コンテストの参加回数によって、内部レーティングと表示レーティングの間に成り立つ関係式が異なっています。よって、高橋君のコンテスト参加回数 N が 10 以上かどうかで場合分けをして考えることにします。

N が 10 以上の場合

このとき、内部レーティングと表示レーティングは等しくなるので、入力として受け取った高橋君の表示レーティングの値を、そのまま高橋君の内部レーティングの値として出力すればいいです。

N が 10 未満の場合

このとき、

$$\text{表示レーティング} = \text{内部レーティング} - 100 \times (10 - N)$$

の関係が成り立っています。よって高橋君の内部レーティングの値は、表示レーティングの値に $100 \times (10 - N)$ を足したものに等しいことが分かります。

以上のロジックの C++ での実装例を Listing 1 に示します。

なお Listing 2 のコードのように、min 関数を用いて if 文を使わないコードにすることも可能です。

Listing 1　C++ による実装例

```
#include<bits/stdc++.h>
using namespace std;

int main() {
    int N, R;
    cin >> N >> R;
    if(N >= 10) cout << R << endl;
```

```
8     else cout << R + 100 * (10 - N) << endl;
9     return 0;
10 }
```

Listing 2　C++ による実装例 2

```
1 #include<bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     int N, R;
6     cin >> N >> R;
7     cout << R + 100 * (10 - min(10, N)) << endl;
8     return 0;
9 }
```

B: Digits

(原案・準備・解説: gazelle)

整数 N を K 進数で表したときの桁数が D であるとします。

このとき位取り記数法の定義より、ある整数列 $a_0, a_1, ..., a_{D-1}$ $(0 \leq a_i < K, a_{D-1} \neq 0)$ が存在して、

$$N = a_{D-1}K^{D-1} + a_{D-2}K^{D-2} + ... + a_0K^0$$

と書くことができます。ここで、

$$N = (a_{D-1}K^{D-2} + a_{D-2}K^{D-3} + ... + a_1K^0) \times K + a_0$$

と変形できることから、$0 \leq a_0 < K$ を考えると N を K で割ったときの商が、

$$a_{D-1}K^{D-2} + a_{D-2}K^{D-3} + ... + a_1K^0$$

になることが分かります。$a_{D-1} \neq 0$ なので、これは K 進数で表したときの桁数が $D-1$ になる数です。

この議論を繰り返し適用することで、N を K 進数で表したときの桁数を求めることができます。つまり、N を K で割ったときの商で置き換える、という操作を N が 0 になるまで行ったとき、操作を行った回数が N を K 進数で表したときの桁数になります (厳密には数学的帰納法を使って示します)。

この計算は愚直に実行すると、桁数に比例する時間がかかります。桁数が非常に大きい場合に実行時間制限を超えてしまいそうに見えますが、実はこの問題の入力制約下では N の桁数は多くても数十程度になります。例えば $N = 10^9$、$K = 2$ のとき、$2^{30} = 1073741824 > 10^9$ であることから、N を 2 進数で表したときの桁数は高々 30 です。N が小さかったり K が大きい場合には桁数はこれより少なくなります。一般に整数 N を K 進数で表したときの桁数は、N の対数に比例します。これは N と比べて非常に小さい値です。

C++ による解法の実装例を Listing 3 に示します。

Listing 3　C++ による実装例

```
#include<bits/stdc++.h>
using namespace std;

int main() {
    int N, K;
    cin >> N >> K;
    int ans = 0;
    while(N > 0) {
        N /= K; // N をK で割った商に置き換える
        ans++;
```

```
11      }
12      cout << ans << endl;
13      return 0;
14  }
```

C: Rally

(原案: namonakiacc, 準備・解説: gazelle)

人が住んでいる座標の中で、最も小さいものを L、最も大きいものを R とします。このとき、$x < L$ または $R < x$ となるような座標 x で集会を開くケースを考える必要はありません。というのも、そのようなケースでは、座標 L や R で集会を開くときと比べて参加するために消費する体力が小さくなることはないからです。

厳密には、例えば $x = R + d$ $(d > 0)$ と書ける座標 x で集会を開く場合、i 番目の人が集会に参加するために消費する体力は、

$$(X_i - (R + d))^2 = ((R + d) - X_i)^2 = (R - X_i)^2 + d(R - X_i) + d^2$$

となり、最右辺の第 2 項は非負、第 3 項は正であることから、座標 R で集会を開くときに比べて、真に消費する体力が増えます。$x < L$ となるような座標 x で集会を開く場合も同様です。

このアイデアを元に、集会開催場所の候補として調べる必要のある座標を高々 100 個まで減らすことができます。集会の開催場所を 1 つ固定したとき、N 人が消費する体力の総和は O(N) 回の演算で計算することが可能です。よって、残った開催場所の候補全てを調べて、最も消費する体力の総和が小さいものを見つけるアルゴリズムが、十分高速に動作します。

Listing 4 に C++ による解法の実装例を示します。

なお入力制約を考慮すると、入力によっては多少実行時間が遅くなる可能性はありますが、Listing 5 のように少しシンプルに書くことが可能です。

Listing 4　C++ による実装例

```
#include<bits/stdc++.h>
using namespace std;
#define rep(i, c) for(int i = 0; i < (int)c; i++)
const int inf = 1000000000; // 10^9

int main() {
    int N;
    cin >> N;
    vector<int> X(N);
    rep(i, N) cin >> X[i];
    int L = X[0], R = X[0];
    rep(i, N) {
        L = min(L, X[i]);
        R = max(R, X[i]);
    }
    int ans = inf;
    for(int i = L; i <= R; i++) {
```

```
        int cost = 0;
        rep(j, N) cost += (X[j] - i) * (X[j] - i);
        ans = min(ans, cost);
    }
    cout << ans << endl;
    return 0;
}
```

Listing 5　C++ による実装例 2

```
#include<bits/stdc++.h>
using namespace std;
#define rep(i, c) for(int i = 0; i < (int)c; i++)
const int inf = 1000000000; // 10^9

int main() {
    int N;
    cin >> N;
    vector<int> X(N);
    rep(i, N) cin >> X[i];
    int ans = inf;
    for(int i = 1; i <= 100; i++) { // L やR の計算が不要
        int cost = 0;
        rep(j, N) cost += (X[j] - i) * (X[j] - i);
        ans = min(ans, cost);
    }
    cout << ans << endl;
    return 0;
}
```

D: Bouquet

(原案・準備・解説: gazelle)

まず「a, b 本の花からなる花束は作ることができない」という制約を無視して問題を解いてみます。このとき答えは $2^n - 1$ になります。これは、種類 1 の花を選ぶ / 選ばない、種類 2 の花を選ぶ / 選ばない、...、種類 n の花を選ぶ / 選ばないという枝分かれを考えると分かります (1 本も花を使わない花束を作ることはできないので、1 を引きます)。$2^n \bmod (10^9 + 7)$ は、繰り返し二乗法と呼ばれるテクニックを用いて高速に計算可能です。

ここから a 本の花からなる花束、および b 本の花からなる花束の種類数を引くことで、元の問題を解くことが可能です。

a 本の花からなる花束の種類数 $\bmod (10^9 + 7)$ は、

$$ {}_n\mathrm{C}_a \bmod (10^9 + 7) = \frac{n \times (n-1) \times ... \times (n-a+1)}{a \times (a-1) \times ... \times 1} \bmod (10^9 + 7) $$

として計算できます。右辺の分子の値を X、分母の値を Y と置きます。

$$ \frac{X}{Y} \bmod (10^9 + 7) $$

をどのように計算すればいいでしょうか？ Y で割る、具体的には Y に掛けると $\bmod (10^9 + 7)$ の世界で 1 になるような数 Y^{-1} が何であるかを探してみます。ここで $10^9 + 7$ が素数であることから、フェルマーの小定理より、

$$ Y \times Y^{(10^9+7)-2} \bmod (10^9 + 7) = 1 $$

が成り立ちます。両辺に Y^{-1} を掛けると、

$$ Y^{(10^9+7)-2} \bmod (10^9 + 7) = Y^{-1} $$

になります。つまり $\bmod (10^9 + 7)$ の世界では、Y で割ることと $Y^{(10^9+7)-2}$ を掛けることが等価になっています (この部分の正確な議論については「競技プログラミング 剰余」などで検索してみてください)。よって、

$$ \frac{X}{Y} \bmod (10^9 + 7) = X \times Y^{(10^9+7)-2} \bmod (10^9 + 7) $$

です。$Y^{(10^9+7)-2}$ の値は、2^n と同様に繰り返し二乗法を用いて求めることができるので、この値は高速に計算可能です。

同様の方法で b 本の花からなる花束の種類数 $\bmod (10^9 + 7)$ も分かるので、これらを $2^n - 1$ から引くことで問題を解くことができました。

E: Roaming

(原案・準備・解説: gazelle)

現在、部屋 i にいる人の数を c_i として、非負整数列 $c_1, c_2, ..., c_n$ が満たすべき条件を考えます。

まず、明らかに $\sum_{i=1}^{n} c_i = n$ である必要があります。また、$c_i = 0$ である i の個数は k 以下でなくてはなりません。というのも、1 回の人の移動で、そのような i の個数は高々 1 しか増えないからです。

実は逆に、この条件を満たしている数列 $c_1, c_2, ..., c_n$ に対して、その結果を与えるような人の移動の列が存在します。以下にこのことを示します。

$c_i = 0$ である i が 0 個の場合

まず、最初に部屋 1 にいた人が部屋 2 に移動して、そのあと $k-2$ 回、部屋 2 と 3 を行ったり来たりして、最後に部屋 1 に戻ればいいです。

$c_i = 0$ である i が 1 個の場合

一般性を失わずに $c_1 = 0,\ c_2 = 2$ であると仮定します。このとき、最初に部屋 1 にいた人が $k-1$ 回、部屋 1 と 3 を行ったり来たりして、最後に部屋 2 に行けばいいです。

$c_i = 0$ である i が 2 個以上の場合

$c_i = 0$ である i の数が m $(2 \leq m < n)$ 個であるとして、一般性を失わずに $c_1 = 0,\ c_2 = 0,\ ...,\ c_m = 0$ であると仮定します。このとき、最初に部屋 1 にいた人が $k-m$ 回、部屋 1 と 2 を行ったり来たりして、そのあとで最初に部屋 1 から m までにいた人が 1 人ずつ、条件を満たすように番号 $m+1$ 以上の部屋に移動していけばいいです。

以上より、この問題は先述の条件を満たす非負整数列 $c_1, c_2, ..., c_n$ の数え上げに帰着されました。次に、そのような数列を数え上げる方法を考えます。

$c_i = 0$ である個数が m となるような数列の数は、

$$ {}_n\mathrm{C}_m \times {}_{n-m}\mathrm{H}_m $$

と表せます。ここで ${}_x\mathrm{H}_y$ は、x 種類のものから重複を許して y 個を選ぶ場合の数 (重複組合せ) を意味します。一般に ${}_x\mathrm{H}_y = {}_{x+y-1}C_{x-1}$ が成り立ちます。これは、$x-1$ 個の仕切りと y 個のボールを一列に並べて、一番左の仕切りより左側にあるボールの数だけ種類 1 のものを選び、一番左の仕切りと二番目に左の仕切りとの間にあるボールの数だけ種類 2 のものを選び、……、と対応付けることが可能なことからも分かります。

適切な前処理を行っておけば、この式の計算に必要な二項係数は全て $O(1)$ で計算することができます。従って $m = 0, 1, ..., n-1$ の各場合について式の値を求めて、その総和を計算するアルゴリズムが十分高速に動作します。

F: Modularness

(原案・準備・解説: gazelle)

1 つのクエリに $O(k)$ で答えることを目指します。i 番目のクエリに答えるものとして、$n = n_i,\ x = x_i,\ m = m_i$ と書くことにします。あらかじめ d の要素それぞれを m で割った余りで置き換えておいても、クエリに対する答えは変わりません。以下そのように置き換えたものとして議論を進めます。

$(a_i \bmod m) < (a_{i+1} \bmod m)$ とならない、つまり $(a_i \bmod m) = (a_{i+1} \bmod m)$ もしくは $(a_i \bmod m) > (a_{i+1} \bmod m)$ となるような i $(0 \leq i < n-1)$ がいくつあるかを考えます。

$(a_i \bmod m) = (a_{i+1} \bmod m)$ となる場合

このとき、$d_{i \bmod k} = 0$ になっています。そのような i の個数は、d の 0 である要素がそれぞれ数列 a に何回足されるかを考えることで $O(k)$ で計算可能です。

$(a_i \bmod m) > (a_{i+1} \bmod m)$ となる場合

このとき、d の要素が非負であること、およびあらかじめ行った前処理を考えると、a_{i+1} を m で割った商が a_i を m で割った商よりちょうど 1 大きくなっています。また商が増えるのは、そのような場合に限ることも分かります。そのような i の個数は、a_{n-1} を m で割った商から a_0 を m で割った商を引くことで求めることができます。a_{n-1} の値は、d の要素がそれぞれ数列 a に何回足されるかを考えることで $O(k)$ で計算可能です。

以上より、$(a_i \bmod m) < (a_{i+1} \bmod m)$ とならない i の個数を $O(k)$ で求めることができます。これらを $n-1$ から引くことで、クエリに $O(k)$ で答えることができました。

A: Beginner

(draft: kort0n, preparation, editorial: gazelle)

Depending on the number of participation in the contest, the relational expression between the Inner Rating and Displayed Rating is different. Therefore, we will split into case depending on whether the number of Takahashi's participation in contest, N, is 10 or more, or not.

If N is more than or equal to 10

In such case, the Inner Rating and the Displayed Rating is the same, so you can directly output the value of Takahashi's displayed rating you received as an input as the value of Takahashi's Inner Rating as an output.

If N is less than 10

In such case, a relational expression of

$$\text{Displayed rating} = \text{Inner rating} - 100 \times (10 - N)$$

holds. Therefore, it appears that the value of Takahashi's Inner rating is equal to the value of the Displayed Rating, added by $100 \times (10 - N)$.

We will show an implementation example of the logic mentioned above in C++ in Listing 6.

Also, like the code in Listing 7, you can also write a code without if statement by using min function.

Listing 6 Sample Code in C++

```
#include<bits/stdc++.h>
using namespace std;

int main() {
    int N, R;
    cin >> N >> R;
    if(N >= 10) cout << R << endl;
    else cout << R + 100 * (10 - N) << endl;
    return 0;
}
```

Listing 7 Sample Code in C++ 2

```
#include<bits/stdc++.h>
using namespace std;

int main() {
    int N, R;
```

```
6       cin >> N >> R;
7       cout << R + 100 * (10 - min(10, N)) << endl;
8       return 0;
9   }
```

B: Digits

(draft, preparation, editorial: gazelle)

Assume that the number of digits of integer N in base K is D.

Then, by the definition of positional notation, there exists a sequence of integer $a_0, a_1, ..., a_{D-1}$ $(0 \leq a_i < K, a_{D-1} \neq 0)$ such that

$$N = a_{D-1}K^{D-1} + a_{D-2}K^{D-2} + ... + a_0K^0.$$

Since it can be deformed to

$$N = (a_{D-1}K^{D-2} + a_{D-2}K^{D-3} + ... + a_1K^0) \times K + a_0,$$

given that $0 \leq a_0 < K$, it appears that the quotient of N divided by K is

$$a_{D-1}K^{D-2} + a_{D-2}K^{D-3} + ... + a_1K^0.$$

Since $a_{D-1} \neq 0$, it has $D - 1$ digits in base K.

By applying this discussion repeatedly, we can find the number of digits of N in base K. Therefore, when a operation of replacing an integer with its quotient divided by K is performed repeatedly until N becomes 0, The number of operations applied will be the number of digits of N in base K (strictly, it can be show by induction).

When performed naively, this calculation takes time proportional to the number of digits. If the number of digits is very large, it seems that it may exceed the time limit, but in fact under the input constraints of this problem, N has at most several dozen of digits. For example, if $N = 10^9$, $K = 2$, since $2^{30} = 1073741824 > 10^9$, N has at most 30 digits in base 2. When N is smaller or K is larger, the number of digits is less than that. Generally, the number of digits of N in base K is proportional to logarithm of N, that is far less than N.

C++ による解法の実装例を Listing 1 に示します。We will show an implementation example of solution in C++ in Listing 8.

Listing 8 Sample code in C++

```
#include<bits/stdc++.h>
using namespace std;

int main() {
    int N, K;
    cin >> N >> K;
    int ans = 0;
    while(N > 0) {
        N /= K; // replace N with its quotient divided by K
```

```
10          ans++;
11      }
12      cout << ans << endl;
13      return 0;
14  }
```

C: Rally

(draft: namonakiacc, preparation, editorial: gazelle)

Let L be the minimum coordinate, and R be the maximum coordinate, where a person lives. Then, we do not have to consider holding a meeting at coordinate x such that $x < L$ or $R < x$. That is, in such case the total points of stamina is no less than when holding a meeting at coordinates L or R.

Strictly speaking, when holding a meeting at coordinate x which can be represented as $x = R + d$ $(d > 0)$ for example, the points of stamina that i-th person will spend is:

$$(X_i - (R + d))^2 = ((R + d) - X_i)^2 = (R - X_i)^2 + d(R - X_i) + d^2,$$

and since the second term of the rightmost hand side is non-negative and the third term is positive, the points of stamina spent is strictly greater than when holding a meeting at coordinate R. It is the same when holding a meeting at coordinate x such that $x < L$.

Based on this idea, we can decrease the number of coordinates we have to consider as candidates of meeting to at most 100 places. When a place of holding the meeting is fixed, the sum of points of stamina can be calculated in a total of O(N) operations. Therefore, the algorithm of looking up all the candidates left and finding the minimum sum of points of stamina consumed is fast enough.

We will show an implementation example in C++ in Listing 9.

Also, considering the given input constraints, though it may be more or less slower, it can be rather simply like Listing 10.

Listing 9 Sample Code in C++

```
#include<bits/stdc++.h>
using namespace std;
#define rep(i, c) for(int i = 0; i < (int)c; i++)
const int inf = 1000000000; // 10^9

int main() {
    int N;
    cin >> N;
    vector<int> X(N);
    rep(i, N) cin >> X[i];
    int L = X[0], R = X[0];
    rep(i, N) {
        L = min(L, X[i]);
        R = max(R, X[i]);
    }
```

```
    int ans = inf;
    for(int i = L; i <= R; i++) {
        int cost = 0;
        rep(j, N) cost += (X[j] - i) * (X[j] - i);
        ans = min(ans, cost);
    }
    cout << ans << endl;
    return 0;
}
```

Listing 10 Sample Code in C++

```
#include<bits/stdc++.h>
using namespace std;
#define rep(i, c) for(int i = 0; i < (int)c; i++)
const int inf = 1000000000; // 10^9

int main() {
    int N;
    cin >> N;
    vector<int> X(N);
    rep(i, N) cin >> X[i];
    int ans = inf;
    for(int i = 1; i <= 100; i++) { // No need of calculations of L and R
        int cost = 0;
        rep(j, N) cost += (X[j] - i) * (X[j] - i);
        ans = min(ans, cost);
    }
    cout << ans << endl;
    return 0;
}
```

D: Bouquet

(draft, preparation, editorial: gazelle)

First, we will solve the problem ignoring the condition that the number of flowers in the bouquet cannot be a or b. In such case, the answer will be $2^n - 1$. This can be understood by considering a tree diagram of choosing / not choosing flower 1, choosing / not choosing flower 2, $\cdots$, choosing / not choosing flower n (since a bouquet with no flowers cannot be made, it is subtracted by 1). $2^n \bmod (10^9 + 7)$ can be calculated rapidly by a technique called fast exponentiation algorithm.

By subtracting from it the number of bouquet with a flowers and the number of bouquet with b flowers, the original problem can be solved.

The number of bouquet with a flowers mod $(10^9 + 7)$ can be calculated by

$$ {}_n\mathrm{C}_a \bmod (10^9 + 7) = \frac{n \times (n-1) \times ... \times (n-a+1)}{a \times (a-1) \times ... \times 1} \bmod (10^9 + 7). $$

We denote the numerator of the left hand side by X, and the denominator by Y. How can we calculate

$$ \frac{X}{Y} \bmod (10^9 + 7)? $$

Let's try dividing by Y, specifically let's find Y^{-1}, such integer that when multiplied by Y it will become 1 in a world of mod $(10^9 + 7)$. Here, since $10^9 + 7$ is a prime, by Fermat's little theorem, it holds that

$$ Y \times Y^{(10^9+7)-2} \bmod (10^9 + 7) = 1. $$

multiplying Y^{-1} to the both sides, we obtain

$$ Y^{(10^9+7)-2} \bmod (10^9 + 7) = Y^{-1}. $$

Therefore, in a world of mod (10^9+7), dividing by Y and multiplying by $Y^{(10^9+7)-2}$ are equivalent (for more strict discussion, please search by "competitive programming modulo" or something). Therefore, it holds that

$$ \frac{X}{Y} \bmod (10^9 + 7) = X \times Y^{(10^9+7)-2} \bmod (10^9 + 7). $$

The value of $Y^{(10^9+7)-2}$ can be calculated by fast exponentiation just like 2^n, so this value can be calculated fast.

We can similarly calculate the number of bouquets with b flowers mod $(10^9 + 7)$, So by subtracting them from $2^n - 1$, the problem could be solved.

E: Roaming

(draft, preparation, editorial: gazelle)

Let c_i be the number of people in room i currently, and let us consider the condition that the sequence of non-negative integers $c_1, c_2, ..., c_n$ should satisfy.

First, obviously $\sum_{i=1}^{n} c_i = n$ should hold. Also, the number of i such that $c_i = 0$ should be no more than k. That is, in one move, such i increases by at most 1.

Actually, conversely, for any sequence $c_1, c_2, ..., c_n$ such that satisfies the condition mentioned above, there exists a sequence of move that gives such results. We will prove that.

If no i satisfies $c_i = 0$

> If the person who initially was in room 1 first moves to room 2, then moves back and forth between room 2 and 3 for $k - 2$ times, then at last goes back to room 1, then the final state satisfies the conditions.

If there exists exactly one i such that $c_i = 0$

> Without loss of generality, we assume that $c_1 = 0,\ c_2 = 2$. In such case, if the person who initially was in room i moves back and forth between room 1 and 3 for $k - 1$ times, and then at last goes to room 2, then the final state satisfies the conditions.

If there exist more than one i such that $c_i = 0$

> Assume that the number of i such that $c_i = 0$ is m $(2 \leq m < n)$, and without loss of generality, we assume that $c_1 = 0,\ c_2 = 0,\ ...,\ c_m = 0$. In such case, if the person who initially was in room 1 moves back and forth between room 1 and 2, then each of the people who were initially in between room 1 to m can move to the room with index more than m so that the final state satisfies the conditions.

Therefore, the problem has been attributed to a problem of counting the number of sequence of non-negative integers $c_1, c_2, ..., c_n$ such that satisfies the condition mentioned above. Next, we will consider how to count such sequences.

The number of sequence such that the number of i such that $c_i = 0$ is m can be represented as:

$$ {}_n\mathrm{C}_m \times {}_{n-m}\mathrm{H}_m, $$

where ${}_x\mathrm{H}_y$ denotes the number of combination of choosing y items from x types of items, possibly with duplicates of types (multiset coefficient). In general, it holds that ${}_x\mathrm{H}_y = {}_{x+y-1}C_{x-1}$. This can be understood from the fact that it can be associated to the arrangement of $x - 1$ bars and y balls in a row, where the number of balls left to the leftmost bar represents the number of the first type, the number of balls between the leftmost and the second leftmost bar

represents the number of the second type, $\cdots$ and so on.

With proper precauculations, all the binomial coefficients needed to calculate this expression can be calculated in a $O(1)$ time each. Therefore, an algorithm of calculating the expression for each $m = 0, 1, ..., n-1$, and then finding the sum of them works fast enough.

F: Modularness

(draft, preparation, editorial: gazelle)

We aim at answering each query in $O(k)$ time. Consider answering i-th query, and we denote $n = n_i,\ x = x_i,\ m = m_i$. Replacing each element of d with its remainder by m does not change the answer for the query. Hereinafter we assume that such applied such replacement.

We will consider how many i $(0 \leq i < n-1)$ do not satisfy $(a_i \bmod m) < (a_{i+1} \bmod m)$, in other words, such i that $(a_i \bmod m) = (a_{i+1} \bmod m)$ or $(a_i \bmod m) > (a_{i+1} \bmod m)$.

If $(a_i \bmod m) = (a_{i+1} \bmod m)$

In such case, $d_{i \bmod k} = 0$. The number of such i can be calculated in $O(K)$ time by considering how many times each of elements of d of value 0 is added to a.

If $(a_i \bmod m) > (a_{i+1} \bmod m)$

In such case, given that the elements of d are non-negative, and also considering the replacement done beforehand, it appears that the quotient of a_{i+1} divided by m is exactly larger by 1 than the quotient of a_i divided by m. Also, it can be seen that the quotient increases only in such cases. The number of such i can be calculated by subtracting the quotient of a_0 divided by m from the quotient of a_0 divided by m. The value of a_{n-1} can be calculated in $O(k)$ time by considering how many times each element of d are added to a.

Therefore, we can count the number of such i that does not satisfy $(a_i \bmod m) < (a_{i+1} \bmod m)$ By subtracting this from $n-1$, we could answer the query in a total of $O(k)$ time.

ABC 157 解説

writer: kort0n

2020 年 3 月 1 日

For International Readers: English editorial starts on page 8.

A: Duplex Printng

答えは floor $\left(\frac{N}{2}\right)$ です (floor (x) は x より小さくない最大の整数を表す)。
例えば、C++ では N を int 型の変数とすると、これは $(N+1)/2$ として求めることが出来ます。
以下は C++ での実装例です。

```
#include <bits/stdc++.h>
using namespace std;
int main() {
  int N;
  cin >> N;
  int ans = (N + 1) / 2;
  cout << ans << endl;
  return 0;
}
```

B: Bingo

問題文の指示通りに各マスにマークが付いているか否かを管理し、ビンゴが成立している列が存在するかを確かめます。
ビンゴカードの管理には二次元配列が便利です。
以下は C++ での実装例です。

```
#include <bits/stdc++.h>
using namespace std;
int N, A[3][3], b[10];
bool appear[3][3];
int main() {
    for(int i = 0; i < 3; i++) {
        for(int j = 0; j < 3; j++) cin >> A[i][j];
    }
    cin >> N;
    for(int i = 0; i < N; i++) cin >> b[i];
    for(int i = 0; i < N; i++) {
        for(int j = 0; j < N; j++) {
            appear[i][j] = false;
            for(int k = 0; k < N; k++) {
                if(A[i][j] == b[k]) appear[i][j] = true;
            }
        }
    }
    string ANS = "No";
    for(int i = 0; i < 3; i++) {
        if(appear[i][0] and appear[i][1] and appear[i][2]) ANS = "Yes";
    }
    for(int i = 0; i < 3; i++) {
        if(appear[0][i] and appear[1][i] and appear[2][i]) ANS = "Yes";
    }
    if(appear[0][0] and appear[1][1] and appear[2][2]) ANS = "Yes";
    if(appear[0][2] and appear[1][1] and appear[2][0]) ANS = "Yes";
    cout << ANS << endl;
    return 0;
}
```

C: Guess the Number

解法 1

0 以上 10^N 未満の整数を全て調べます。条件の成立チェックを行う上では、整数を一旦文字列に変換すると楽です。
時間計算量は $O\left(10^N M\right)$ です。

解法 2

M 個の条件について、矛盾が存在しないかを前から順に確かめつつ解の候補を絞り、最後にそれらの最小値を出力します。
ただし、$N \geq 2$ **の場合に限り**左から 1 桁目は 0 が許されないことに注意してください。
時間計算量は $O\left(N + M\right)$ です。

D: Friend Suggestions

人を頂点、友達関係を辺とした無向グラフを考えると、人 i に対する答えは、

(頂点 i の連結成分のサイズ)
−(頂点 i と頂点 j が同じ連結成分に含まれて、人 i と人 j が友達関係もしくはブロック関係にあるような j の数)
−1

です (最後の 1 は自分自身)。
各頂点が属する連結成分を求め、各連結成分のサイズを求めておけば、各人ごとの上記の値を高速に計算することが出来ます。
これは、Union Find のようなデータ構造を用いるか、もしくはグラフ上で DFS を行うこと等により実現出来ます。
時間計算量は、例えば DFS の場合は $O(N+M+K)$ です。

E: Simple String Queries

各文字 (a, b, ..., z) 毎に文字列中に現れる場所の集合を管理することを考えます。
例えば、set(平衡二分探索木) を使えば、type 1 のクエリは erase 関数と insert 関数を用いて $O(\log N)$ で処理可能であり、type 2 のクエリは各文字ごとに lower_bound 関数で l_q 文字目以降初めて存在する場所を求め、その値と r_q を比較することにより、$O(\log N)$ で処理可能です。以上より、全体の時間計算量 $O((\text{Alphabet Size})\, Q \log N)$ で処理可能です。
他にも、Segment Tree や平方分割により各文字が各区間に表れる回数を管理したり、各区間に表れる文字の集合を管理したりすることによって、クエリを高速に処理することも可能です。

F: Yakiniku Optimization Problem

解法 1

「時刻 T の時点で K 枚以上の肉が焼けるような熱源の配置場所は存在するか？」という判定問題を考えると、この判定問題には単調性がありますから、答えを二分探索で求めることが出来ます。
この判定問題を数学的に定式化すると、以下の通りになります。
「N 枚の円板がある。i 枚目の円板の中心は (x_i, y_i) であり、半径は $\frac{T}{c_i}$ である。K 枚以上の円板に含まれるような点が存在するか判定せよ。」
この問題は、以下のように言い換えることが出来ます。
「N 枚の円板がある。i 枚目の円板の中心は (x_i, y_i) であり、半径は $\frac{T}{c_i}$ である。これらの N 枚の円板から K 枚の円板を選ぶ選び方であって、K 枚の円板の積集合が非空となるような選び方があるか判定せよ。」
ここで, K 枚の円板の積集合 X が非空であったとき、その集合は 1 枚の円板と一致するか、或いは互いに包含関係に無い複数枚の円板の積集合と一致します。
前者の場合はその円板の中心の座標が X に含まれ、後者の場合はある 2 枚の円板が存在し、それらの境界を成す円の交点が X に含まれます。
これより、N 個の円の中心及び N 個の円のうち 2 個の円の交点全てについて、その点を含む円板が K 枚以上存在するかを調べることにより、前述の判定問題を $O\left(N^3\right)$ で解くことが出来ます。
以上より、許容誤差を eps として、元の問題を $O\left(\frac{c_{\max}(|x_{\max}|+|y_{\max}|)}{eps}N^3\right)$ で解くことが出来ます。

解法 2

$K = 1$ のときは答えは明らかに 0 です。以下では $K \geq 2$ とします。$K \geq 2$ の元、答えは明らかに 0 より大きくなります。
以下では点 $P\,(X,Y)$ と肉 i の”距離”を、$c_i\sqrt{(X-x_i)^2+(Y-y_i)^2}$ で定めます.
肉 i と肉 j からの距離が等しい点の集合は、$c_i = c_j$ のときは直線であり、$c_i \neq c_j$ のときは円です。
$K \geq 2$ のもと、最適な熱源位置は、異なる 2 枚の肉からの距離が等しく、かつその値が最小となる点か、或いは異なる 3 枚の肉からの距離が等しい点です。
これを背理法により示します。前述の条件を満たさない点 P が最適な熱源配置であると仮定します。このときの答えを T と置きます。丁度時刻 T に焼ける肉を tight な肉と呼びます。仮定より、tight な肉は高々 2 枚です。
tight な肉が 0 枚であるとき、熱源を P に置くと、時刻 T より僅かに前でも K 枚以上の肉が焼けています。
tight な肉が 1 枚であるとき、点 P からその肉の方向へ僅かに進んだ点では、時刻 T より僅かに前でも K 枚以上の肉が焼けています。

tight な肉が 2 枚であるとき、それらを肉 i、肉 j と置くと、肉 i を中心とする半径 $\frac{T}{c_i}$ の円板及び肉 j を中心とする半径 $\frac{T}{c_j}$ の円板に含まれ、更に肉 i からの距離と肉 j からの距離が等しい点の集合は、仮定より一点集合ではなく、有限の長さを持つ線分或いは曲線となります。この線の内部の点に熱源を配置すると、時刻 T より僅かに前でも K 枚以上の肉が焼けています。
以上より、最適な熱源配置の候補は、前述の条件を満たす点に限られます。
これより、前述の点を全列挙して各点に熱源を配置した際に各肉が焼けるまでの時間を求め、K 枚目の肉が焼けるまでに掛かる時間を求めることにより、解を得ることが出来ます。
時間計算量は、K 枚目の肉が焼けるまでに掛かる時間を求める際にソートを行えば $O\left(N^4 \log N\right)$ であり、選択アルゴリズムを用いれば $O\left(N^4\right)$ です。

A: Duplex Printng

The answer is floor $\left(\frac{N}{2}\right)$ (where floor (x) denotes the maximum integer that is not less than x)。

例えば、C++ では N を int 型の変数とすると、これは $(N+1)/2$ として求めることが出来ます。

For example in C++, when N is a variable of type int, this can be calculated by $(N+1)/2$.

The following is a sample code in C++.

```
#include <bits/stdc++.h>
using namespace std;
int main() {
  int N;
  cin >> N;
  int ans = (N + 1) / 2;
  cout << ans << endl;
  return 0;
}
```

B: Bingo

Follow the problem statement's instruction and manage if each square is marked or not, and check if there is a row that forms a bingo.
When managing the bingo card, two-dimensional array is useful.
The following is a sample code in C++.

```
#include <bits/stdc++.h>
using namespace std;
int N, A[3][3], b[10];
bool appear[3][3];
int main() {
    for(int i = 0; i < 3; i++) {
        for(int j = 0; j < 3; j++) cin >> A[i][j];
    }
    cin >> N;
    for(int i = 0; i < N; i++) cin >> b[i];
    for(int i = 0; i < N; i++) {
        for(int j = 0; j < N; j++) {
            appear[i][j] = false;
            for(int k = 0; k < N; k++) {
                if(A[i][j] == b[k]) appear[i][j] = true;
            }
        }
    }
    string ANS = "No";
    for(int i = 0; i < 3; i++) {
        if(appear[i][0] and appear[i][1] and appear[i][2]) ANS = "Yes";
    }
    for(int i = 0; i < 3; i++) {
        if(appear[0][i] and appear[1][i] and appear[2][i]) ANS = "Yes";
    }
    if(appear[0][0] and appear[1][1] and appear[2][2]) ANS = "Yes";
    if(appear[0][2] and appear[1][1] and appear[2][0]) ANS = "Yes";
    cout << ANS << endl;
    return 0;
}
```

C: Guess the Number

Solution 1

Check for all the integers more than or equal to 0, and less than 10^N. When checking the conditions, temporal conversion from integer to string will make the implementation easier. The total time complexity is $O\left(10^N M\right)$.

Solution 2

For each of M conditions, narrow down the candidates while checking for contradictions, and at last output the minimum of them.
Note that the 1-st digit should not be 0 **only if** $N \geq 2$.
The total time complexity is $O\left(N + M\right)$.

D: Friend Suggestions

Consider a graph, whose vertices correspond the users and whose edges correspond to the friendship. Then the answer for user i is

(The size of the connected component of vertex i)
$-$(The number of j, such that vertex i and vertex j belong to the same connected component, and user i and user j is in a friendship or in a blockship)
-1.

(The last 1 is the user him/herself.)
By finding the connected component to which each vertex belongs, and finding the size of each connected component, the value mentioned above for each user can be calculated fast.
This can be achieved by using data structures like Union Find, or performing a DFS on the graph. The time complexity is, $O(N + M + K)$ in case of DFS for example.

E: Simple String Queries

Consider managing the set of positions such that each alphabets (a, b, ..., z) appears in the string.

For example, by using a set (self-balancing binary search tree), a query of type 1 can be processed in a $O(\log N)$ time using erase function and insert function, and a query of type 2 can be processed in a $O(\log N)$ time by find the position of the first appearance after l_q-th letter, and then comparing it with r_q, using lower_bound function. Therefore, the overall queries can be processed in a total of $O((\text{Alphabet Size})\, Q \log N)$ time.

Otherwise, the query can be processed fast by managing the number of appearances of each alphabet in each segment, or the set of alphabets that appear in each segment, using Segment Tree or square-root splitting.

F: Yakiniku Optimization Problem

Solution 1

By considering a decision problem of "does there exist a place to put the heat source such that K or more pieces of meat is ready at time T?", then this problem has monotonicity, so the answer can be found in a binary search.

A mathematical formalization of this decision problem is as follows:

"You are given N disks. The center of i-th disk is (x_i, y_i) and its radius is $\frac{T}{c_i}$. Determine whether there exists a point such that is included in K or more disks."

This problem can be reworded as follows.

"You are given N disks. The center of i-th disk is (x_i, y_i) and its radius is $\frac{T}{c_i}$. Determine if there exists a set of K disks out of those N disks, such that the union set of those K disks is non-empty."

Here, if a union of K disks X is non-empty, then it corresponds to a disk, or a union of some disjoint multiple disks.

In the former case, the center coordinate of the disk is included in X, and in the latter case, there exist two disks such that the intersection of their boundary circles is included in X.

Hence, by checking if a point is included in K or more disks for all the N centers and all the intersections of pairs of circles, the aforementioned decision problem can be solved in a total of $O\left(N^3\right)$ time.

Therefore, denoting by *eps* the maximum permissible error, the original problem can be solved in a total of $O\left(\frac{c_{\max}(|x_{\max}|+|y_{\max}|)}{eps}N^3\right)$ time.

Solution 2

If $K = 1$, then obviously the answer is 0. Hereinafter assume that $K \geq 2$. Under $K \geq 2$, the answer is obviously more than 0.

Hereinafter we define a "distance" between point $P\left(X, Y\right)$ and meat i by $c_i\sqrt{\left(X - x_i\right)^2 + \left(Y - y_i\right)^2}$.

The set of points such that the distances from meat i and from meat j are equal is a line if $c_i = c_j$, and a circle if $c_i \neq c_j$.

Under $K \geq 2$, the optimal position of heat source is a point such that the distances from distinct two pieces of meat are the equal and its value is minimum, or a point such that the distances from distinct three pieces of meat are the equal.

We will show this by contradiction. Suppose that a point P such that does not satisfy the aforementioned conditions is the optimal heat source position. Let T be the answer in such

case. We will call a piece of meet that becomes ready at time T. By assumption, there exist at most two pieces of meat is tight.

If 0 pieces of meat are tight, by placing a heat source at P, K or more pieces of meat is ready even before time T.

If 1 piece of meat is tight, at the point a little nearer to the meat than point P, K or more pieces of meat is ready at the time a little before time T.

If 2 pieces of meat are tight, denoting those meat by i and j, a set of points, such that is included in a disk whose center is meat i and whose radius is $\frac{T}{c_i}$, is included in a disk whose center is meat j and whose radius is $\frac{T}{c_j}$, and the distances from meat i and from meat j is equal, is not a singleton, but a segment or a curve of finite length, by assumption. By placing a heat source at the interior point of this curve, K or more pieces of meat is ready at the time a little before time T.

Therefore, the candidates of optimal heat source positions are narrowed down into such points that satisfy the aforementioned conditions.

Hence, by iterating all the points mentioned above, finding the time needed for each meet to be ready when the heat source is placed at each point, and by finding the time needed until the K-th meat is ready, the solution can be found.

The time complexity is $O\left(N^4 \log N\right)$ if a sort is performed when finding the time needed until the K-th meat is ready, and is $O\left(N^4\right)$ if a selection algorithm is used.

ABC 158 解説

writer: satashun evima, tempura0224

2020 年 3 月 7 日

For International Readers: English editorial starts on page 7.

A: Station and Bus

S 内に A, B の両方が存在する場合は Yes、どちらかしかない場合は No です。

以下は Python での実装例です。

```
S = input()
if S[0] == S[1] and S[1] == S[2]:
  print('No')
else:
  print('Yes')
```

B: Count Balls

1 回の操作をセットで考えると考えやすいです。
N 個のうち、完全に含まれる操作の回数は $\lfloor \frac{N}{A+B} \rfloor$ 回です。
中途半端に含まれる操作は高々 1 回分ですが、N を $A+B$ で割った余りと A の大小関係が重要で、N を $A+B$ で割った余りを R とすると、$min(R, A)$ だけ含まれます。

以下は Python での実装例です。

```
N, A, B = map(int, input().split())
ans = N // (A + B) * A
rem = N % (A + B)
ans += min(rem, A)
print(ans)
```

C: Tax Increase

消費税が 8% のとき A 円、消費税が 10% のとき B 円となる金額を直接求めなくても、$1 \leq A \leq B \leq 100$ の条件から、最大でも考える必要のある金額は 1009 円であるため、小さい方から計算していけばよいです。

課題：$O(1)$ で答えを求めてください。

D: String Formation

問題文の通りに直接シミュレーションするだけでは反転クエリが高速に処理できず、$O((N+Q)^2)$ となり遅いです。

$T_i = 1$ のとき実際に文字列を操作するのではなく、現在どちら側が先頭かという情報を持っておけば、$T_i = 2$ のクエリでもその情報を考慮することで、位置関係を保ったままシミュレーションできます。最後に全体を反転する必要がある場合に気をつけてください。

ここで、$C++$ などで普通の文字列型の先頭に追加する操作は計算量が悪いため、dequeue で文字列を管理するか、左右で分けて文字列を保持し最後に適切に連結するといった方法で対処できます。

E: Divisible Substring

まず、左端を固定して実際に P で割った余りを計算しながら右側を伸ばしていくことで、$O(N^2)$ で答えを求めることができます。

例えば文字列の左右を固定した場合に P で割った余りを計算できないか、定式化を考えてみましょう。

$U_i := S[i:N]$ (S の i 文字目以降) を評価した整数 と置きます。便宜上、$U_{N+1} = 0$ とします。(この時点ではまだ mod P で考えていません)

このとき、S の i 文字目から j 文字目を評価した整数は、$\frac{U_i - U_{j+1}}{10^{j+1-i}}$ と表せます。$(1 \leq i \leq j \leq N)$

ここで、P と 10 が互いに素かどうかが重要です。$P = 2, 5$ の場合は右端が P で割り切れるかどうかのみで決まるので、$O(N)$ で解けます。

$P \neq 2, 5$ とします。このとき、$\frac{U_i - U_{j+1}}{10^{j+1-i}}$ が P で割り切れることは、$U_i - U_{j+1}$ が P で割り切れることと同値です。よって、右側から U_i mod P を計算し、現在 x mod P であるような j が何個あるかを配列や map で管理しておくことで、$O(N + P)$ でこの問題が解けました。(今回は P が小さいですが、P が大きい場合は map を使いましょう)

F: Removing Robots

まず、高橋君が操作を行う順番は重要ではありません。

$f_i :=$ ロボット i が一連の操作の中で起動されたかどうかの bool 値、$g_i :=$ ロボット i が高橋君によって直接起動されたかどうかの bool 値と置きます。このとき、$f_i := g_i \,\mathrm{OR}\, f_j(j \neq i, X_j < X_i < X_j + D_j)$ と表せることがわかります。

全てのロボットが残っている場合に、ロボット i を起動した場合に連動して起動することになる最大の j を R_i で表します。(他のロボットに関与しない場合は、$R_i = i$ とします) このとき、

$R_i = \max(i, R_j(i < j, X_i + D_i > X_j))$

が成り立つので、segment tree などを用いて R_i を X の大きい方から求めていけます。一部のロボットが取り除かれている場合も、起動した場合に同じ範囲のロボットが取り除かれた状態になることがわかります。

$dp_i :=$ ロボット i 以降のみを考慮した場合の組み合わせの個数と置きます。このとき、ロボット i を起動しない場合... dp_{i+1}、ロボット i を起動する場合 ... dp_{R_i+1} となり、X の大きい方から求まります。

以上の計算量は $O(NlogN)$ です。

A: Station and Bus

If S has both A and B, then the answer is Yes, and if it has only either of them, then the answer is No.

The following is a sample code in Python.

```
S = input()
if S[0] == S[1] and S[1] == S[2]:
  print('No')
else:
  print('Yes')
```

B: Count Balls

It is easy to think when one operation is considered as a set.
Among N balls, the number of operations that is completely included in is $\lfloor \frac{N}{A+B} \rfloor$.
The number of operations that is included halfway is at most one, but the remainder of N divided by $A+B$ is only important, and let R be the remainder of $A+B$ divided by N, then there are $min(R, A)$ balls.

The following is a sample code in Python.

```
N, A, B = map(int, input().split())
ans = N // (A + B) * A
rem = N % (A + B)
ans += min(rem, A)
print(ans)
```

C: Tax Increase

Instead of directly finding the price such that the consumption tax is A and B yen when tax rate is 8% and 10%, by the condition of $1 \leq A \leq B \leq 100$, the maximum price that has to be considered is 1009 yen, so it is sufficient to calculate in the increasing order.

Task: Find the answer in a total of $O(1)$ time.

D: String Formation

By only performing simulation in accordance with the problem statement, the reverse query cannot be processed fast, and it will cost a total of $O((N+Q)^2)$ time, which is slow.

When $T_i = 1$, instead of actually manipulating the string, you can hold the state of which side is the beginning of the string so that, by considering the state also in the query of $T_i = 2$, the simulation can be done while maintaining the positional relation. Note that you may need to reverse the whole string at last.

Here, in $C++$ or some other programming languages, operation of appending to the ordinal string type has bad time complexity, so you can handle with it by managing the string with dequeue, or storing the string with two parts, left and right, and at last concatenating properly.

E: Divisible Substring

First, we can find the answer in a total of $O(N^2)$ time, by fixing the left end and actually calculating the remainder by P while extending the right end.

Let's consider a formularization that enables to calculate the remainder by P when, for example, the both ends of the string is fixed.

Let $U_i := S[i : N]$ (the i-th and later character of S), evaluated as an integer. For convenience, let $U_{N+1} = 0$. (At this point we are not considering mod P yet.)

Then, substring of S from i-th to j-th, evaluated as an integer, can be denoted as $\frac{U_i - U_{j+1}}{10^{j+1-i}}$. $(1 \leq i \leq j \leq N)$

Here, it is important whether P and 10 are coprime or not. If $P = 2, 5$, then it only depends on whether the right end is divisible by P, so it can be solved in a total of $O(N)$ time.

Assume that $P \neq 2, 5$. Then, $\frac{U_i - U_{j+1}}{10^{j+1-i}}$ is divisible by P if and only if $U_i - U_{j+1}$ is divisible by P. Therefore, by calculating U_i mod P from right to left, and managing the number of j such that x mod P with arrays or maps, the problem could be solved in a total of $O(N + P)$ time. (This time P is small, but when P is large, use map)

F: Removing Robots

First, the order of Takahashi's operations are not important.

Let $f_i :=$ a boolean value whether robot i was activated in the series of operations, and $g_i :=$ a boolean value whether robot i was directly activated by Takahashi. Then, it can be seen that $f_i := g_i \,\mathrm{OR}\, f_j (j \neq i, X_j < X_i < X_j + D_j)$.

Let R_i be the maximum j such that, when robot i is activated when all the robots are remaining, robot j is accordingly activated. (If it does not contribute to other robots, then let $R_i = i$.) Then, it holds that

$R_i = max(i, R_j (i < j, X_i + D_i > X_j))$,

so, by using data structures like segment tree, R_i can be calculated in the decreasing order of X. Even when some robots are removed, the robots in the same range will be removed when it's activated.

Let $dp_i :=$ the number of possible sets of robots, when considering only robot i and later. Then, if robot i is not activated... dp_{i+1}, and if robot i is activated... dp_{R_i+1}, and these can be calculated in the decreasing order of X.

The time complexity above is $O(NlogN)$ in total.

ABC 159 解説

writer: DEGwer, kyopro_friends, latte0119, namonakiacc,
satashun, tempura0224, tozangezan, yokozuna57

2020 年 3 月 22 日

For International Readers: English editorial starts on page 7.

A: the number of even pairs

選んだ 2 つのボールに書かれている数の和が偶数になるのは、 2 つのボールに書かれている数が共に偶数であるか、共に奇数である場合に限ります。したがって、そのようなボールの選び方の数を求めればよいです。これは、 $\binom{N}{2} + \binom{M}{2}$ 通りです。以下は C++ における実装例です。

```
#include<iostream>
using namespace std;

int main(){
    int N,M;
    cin>>N>>M;

    cout<<N*(N-1)/2+M*(M-1)/2<<endl;
    return 0;
}
```

B: Strong Palindrome

文字列 S の 長さを N とします。
S, S の 1 文字目から $\frac{N-1}{2}$ 文字目, S の $\frac{N+3}{2}$ 文字目から N 文字目 がすべて回文であるかどうかを調べ、すべてが回文であるときに "Yes" , そうでないときに "No" を出力すればよいです。
S が回文であるかどうかを調べる方法としては例えば以下の方法などがあります。

1. すべての i $(1 \leq i \leq N)$ について、S の i 文字目と S の $(N-i-1)$ 文字目が等しいことを確認する。
2. S を反転した文字列と S が等しいかどうかを調べる。

C++ での実装例：https://atcoder.jp/contests/abc159/submissions/11091525

C: Maximum Volume

縦、横、高さをそれぞれ a, b, c とおくと、$a + b + c = L$ をみたす中で abc を最大化する問題となります。

相加相乗平均不等式より、$(abc)^{\frac{1}{3}} \leq \frac{a+b+c}{3}$、すなわち $abc \leq \left(\frac{a+b+c}{3}\right)^3$ です。等号成立条件は $a = b = c$ です。

よって、$a = b = c = \frac{L}{3}$ のとき体積最大となり、その時の体積は $\frac{L^3}{27}$ となります。

Listing 1　C++ での実装例

```
#include <stdio.h>
int main(){
        int L;
        scanf("%d",&L);
        printf("%.12f\n",(double)L*L*L/27);
}
```

D: Banned K

$i = 1, 2, ..., N$ に対して、 $c_i := \#\{1 \leq j \leq N | A_j = i\}$ と定義します。
$k (1 \leq k \leq N)$ を 1 つ固定して考えます。 k に対する問題の答えは、

- N 個のボールから、書かれている整数が等しいような異なる 2 つのボールを選び出す方法の数
- k 番目のボールを除いた $N-1$ 個のボールから、 k 番目のボールと同じ整数が書かれたボールを選び出す方法の数

をそれぞれ求めて、前者から後者を引けば求まります。

前者は、 $\sum_{i=1}^{N} \binom{c_i}{2}$, 後者は $c_{A_k} - 1$ となります。　前者の値は k によらないので、事前に $O(N)$ で計算しておけばよいです。後者は、各 k に対して $O(1)$ で求められます。

以上より、この問題を $O(N)$ で解くことができました。

E: Dividing Chocolate

もし「縦方向にしか割れない」という条件があれば、左端から順に貪欲にとることで問題を解くことができます。よって、「横方向にどう割るか」を 2^{H-1} 通り全探索し、それぞれの場合について「縦方向にしか割れない」問題を解けばよいです。計算量は $O(2^H HW)$ です。

F: Knapsack for All Segments

求めたい合計値は、

$a_{x_1} + a_{x_2} + \cdots + a_{x_k} = S$, $L \leq x_1 < x_2 < \cdots < x_k \leq R$ をみたすような $(L, x_1, x_2, \ldots, x_k, R)$ の組の個数

と言い換えることができます。

これは

$dp[i][s][t] =$ i 番目までみて、選んだ要素の和が s で、
L をまだ決めていない状態の場合の数 ($t = 0$ のとき)
L は決めたが R はまだ決めていない状態の場合の数 ($t = 1$ のとき)
L も R も決まっている状態の場合の数 ($t = 2$ のとき)

と定めて動的計画法を用いると、時間計算量、空間計算量ともに $O(NS)$ で計算することができます。

求めるものは $dp[N][S][2]$ です。

遷移等については実装例を参考にしてください。https://atcoder.jp/contests/abc159/submissions/11096062

A: the number of even pairs

The sum of integers on the chosen two balls is an even number only if the integers on the two balls are both even numbers, or both odd numbers. Therefore, it is enough to find the number of such ways of choosing. This is equal to $\binom{N}{2} + \binom{M}{2}$. The following is a sample code in C++.

```
#include<iostream>
using namespace std;

int main(){
    int N,M;
    cin>>N>>M;

    cout<<N*(N-1)/2+M*(M-1)/2<<endl;
    return 0;
}
```

B: Strong Palindrome

Let N be the length of the string S.
It is enough to check if S, the 1-st to the $\frac{N-1}{2}$-th letters of S and the $\frac{N+3}{2}$-th to the N-th letters are all palindromes, and output "Yes" if all of them are palindromes and "No" otherwise.
To check if S is a palindrome, for example the followings method is available.

1. For all i $(1 \leq i \leq N)$, check if the i-th letter of S and $(N-i-1)$-th letter of S are the same.
2. Check if S and the reversed string of S are the same.

Sample Code in C++ : https://atcoder.jp/contests/abc159/submissions/11091525

C: Maximum Volume

Let a, b, c be the depth, width and height respectively, then this problem asks to maximize abc subject to $a + b + c = L$.

By the inequality of arithmetic and geometric means, $(abc)^{\frac{1}{3}} \leq \frac{a+b+c}{3}$, hence $abc \leq \left(\frac{a+b+c}{3}\right)^3$. The equality holds when $a = b = c$.

Therefore, the volume is maximum when $a = b = c = \frac{L}{3}$, in which case the volume is $\frac{L^3}{27}$.

Listing 2 Sample Code in C++

```
#include <stdio.h>
int main(){
        int L;
        scanf("%d",&L);
        printf("%.12f\n",(double)L*L*L/27);
}
```

D: Banned K

For $i = 1, 2, \ldots, N$, we define ....

Fix a $k (1 \leq k \leq N)$. Then, to find the answer of the problem for k, find the following two values:

- The number of ways to choose two distinct balls from the N balls so that the integers written on them are equal
- The number of ways to choose s ball from the $N - 1$ balls other than the k-th ball so that the integer written on it is same to that on the k-th ball

Then, subtract the latter from the former, which will be the answer.

The former is $\sum_{i=1}^{N} \binom{c_i}{2}$ and the latter is $c_{A_k} - 1$. Since the former is independent of k, it can be precalculated in an $O(N)$ time. The latter can be calculated in a $O(1)$ time each for each k.

Therefore, the problem could be solved in a total of $O(N)$ time.

E: Dividing Chocolate

If there is a constraints that "divisions can be done only vertically," then the problem can be solved by greedily taking from left to right. Therefore, it is enough to solve the "only-vertical" problem for all 2^{H-1} combinations of ways of dividing vertically. The time complexity is $O(2^H HW)$.

F: Knapsack for All Segments

The desired sum can be rephrased as:
the number of tuple $(L, x_1, x_2, \ldots, x_k, R)$ such that $a_{x_1} + a_{x_2} + \cdots + a_{x_k} = S$ and $L \leq x_1 < x_2 < \cdots < x_k \leq R$.
This can be calculated with a dynamic programming with the following relational equations:

$dp[i][s][t] =$ The number of combinations chosen from the first i elements, such that the sum of elements is s and
L has not been added to the tuple yet (When $t = 0$)
L has been added but R has not been added yet(When $t = 1$)
both L and R has been added to the tuple(When $t = 2$)

which would be in a total of $O(NS)$ time and memory.
The desired sum is $dp[N][S][2]$.
For more details such as transitions, please refer to the following sample code.
https://atcoder.jp/contests/abc159/submissions/11096062

ABC 160 解説

writer: DEGwer, kyopro_friends, latte0119, satashun, ynymxiaolongbao

2020 年 3 月 28 日

For International Readers: English editorial starts on page 7.

A: Coffee

長さ 6 の英小文字からなる文字列に対して、3 文字目と 4 文字目が等しく、5 文字目と 6 文字目も等しいか判定する問題です。

以下は C++ における実装例です。

```
#include <iostream>
using namespace std;
int main(){
  string s;
  cin>>s;
  if(s[2]==s[3]&&s[4]==s[5]){
    cout<<"Yes"<<endl;
  }
  else{
    cout<<"No"<<endl;
  }
}
```

B:Golden Coins

500 円以上持っているのであれば、500 円硬貨 1 枚を手に入れた方が、5 円硬貨 100 枚を手に入れるより嬉しさは大きくなります。よって、できる限り 500 円硬貨を手に入れた後、端数で 5 円硬貨を手に入れるのが最善です。

C 言語での実装例は次のとおりです。

```
#include<stdio.h>
int main(){
    int x;
    scanf("%d",&x);
    int c500 = x/500; // 500 円硬貨の数
    int r500 = x%500; // 端数
    int c5 = r500/5; // 5 円硬貨の数
    printf("%d\n",c500*1000+c5*5);
}
```

C:Traveling Salesman around Lake

湖の周りを家によって N 個の区間に分けます。セールスマンが全ての家を訪れるとき、N 個の区間のうち通らないものは高々 1 つです。また、N 個の区間のうち最も長いものを通らず、かつ、それ以外の区間をちょうど 1 度だけ通るようにしてすべての家を訪ねることができます。したがってそのような移動方法をとると最短距離となります。最長の区間は $O(N)$ で求めることができるのでこの問題が解けました。実装上は真北 (座標 0) をまたいだ区間の扱いに気を付けてください。

D: Line++

整数 $i, j (1 \leq i < j \leq N)$ を固定します。グラフ G における、頂点 i と頂点 j の最短距離を効率的に求めることを考えます。 頂点 X と頂点 Y を結ぶ辺を使う場合と使わない場合に分けて考えると、求める最短距離は $min\{|j-i|, |X-i|+1+|j-Y|, |Y-i|+1+|j-X|\}$ となります。この式に従って計算することで、 頂点 i と頂点 j の最短距離を $O(1)$ で求めることができます。後は、$1 \leq i < j \leq N$ を満たすすべての i, j について、上記の方法で最短距離を求めれば、 $O(N^2)$ でこの問題を解くことができます。

E: Red and Green Apples

元から赤色のリンゴを X 個以上食べることはないので、美味しさの大きい方から X 個以外は捨てます。元から緑色のリンゴを Y 個以上食べることもないので、美味しさの大きい方から Y 個以外は捨てます。すると残ったリンゴからどのように $X+Y$ 個のリンゴを選んでも、無色のリンゴを適切に着色することで赤色のリンゴ X 個と緑色のリンゴ Y 個にすることができます。よって、残ったリンゴを美味しさの大きい方から $X+Y$ 個食べれば良いです。

F: Distributing Integers

頂点 1 を根とする根付き木として考えます。以下のような値を定義します、

dp_v = 頂点 v を根とする部分木のみを考えたときの、頂点 v に 1 が書かれるような整数の書き方の数

これは、以下の漸化式に従って、ボトムアップに計算することができます。

$$dp_v = (size_v - 1)! \prod_{u \in ch(v)} \frac{dp_u}{size_u!}$$

後は、上記の木 dp を元に全方位木 dp を行うことで、$O(N)$ でこの問題を解くことができます。

A: Coffee

This problems asks to check if a string of length 6 consisting from English lowercase alphabets has the properties that the third and the forth letters are the same and that the fifth and the sixth letters are the same.

The following is a sample code in C++.

```
#include <iostream>
using namespace std;
int main(){
  string s;
  cin>>s;
  if(s[2]==s[3]&&s[4]==s[5]){
    cout<<"Yes"<<endl;
  }
  else{
    cout<<"No"<<endl;
  }
}
```

B:Golden Coins

If he has more than 500 yen, then the happiness will be larger when he has a 500-yen coin than when he has 100 5-yen coins. Therefore, it is optimal to obtain 500-yen coins as much as possible, then obtain 5-yen coins with the fraction.

The following is a sample code in C.

```
#include<stdio.h>
int main(){
    int x;
    scanf("%d",&x);
    int c500 = x/500; //The number of 500-yen coins
    int r500 = x%500; //Fraction
    int c5 = r500/5; //The number of 5-yen coins
    printf("%d\n",c500*1000+c5*5);
}
```

C:Traveling Salesman around Lake

Split the circumference of the pond into the N segments by the houses When the salesman visits all the house, there are at most 1 segment out of the N segments which the salesman does not pass. Also, it is possible to visit all the houses so that the salesman does not pass the longest segment of the N segments while passing the other segments exactly once. The longest segment can be found in a total of $O(N)$ time, so the problem could be solved. When implementing, be careful of the treatment of the segment that strides over the due north.

D: Line++

Fix two integers $i, j (1 \leq i < j \leq N)$. Let us consider an efficient way of finding the shortest distance between vertex i and vertex j. By considering whether or not use the edge between vertex X and vertex Y, the desired shortest distance will be $min\{|j-i|, |X-i|+1+|j-Y|, |Y-i|+1+|j-X|\}$. By following this equation, the shortest distance between vertex i and vertex j can be calculated in an $O(1)$ time. Finally, by finding the shortest distance for all i, j such that $1 \leq i < j \leq N$ with the method above, the problem can be solved in a total of $O(N^2)$ time.

E: Red and Green Apples

Since no more than X red apples will be eaten, we can abandon the apples other than X apples with the largest deliciousnesses. Also, since no more than Y green apples will be eaten, we can abandon the apples other than Y apples with the largest deliciousnesses. Then, no matter how we choose $X + Y$ apples out of the apples left, by properly painting the colorless apples, we can obtain X red apples and Y green apples Therefore, it is optimal to eat the $X + Y$ apples with the largest deliciousness out of the apples left.

F: Distributing Integers

Consider it as a rooted tree with the root being vertex 1. We define the following value:

dp_v =The number of the ways of writing integers, in which integers are only to the subtree with the root being vertex v, such that 1 is written on vertex v.

This value can be calculated bottom-up with the following recurrence equation:

$$dp_v = (size_v - 1)! \prod_{u \in ch(v)} \frac{dp_u}{size_u!}$$

By performing rerooting based on the tree DP above, the problem can be solved in a total of $O(N)$ time.

ABC 161 解説

chokudai, evima, kyopro_friends, latte0119,
namonakiacc, sheyasutaka, tozangezan, ynymxiaolongbao

2020 年 4 月 4 日

For International Readers: English editorial starts on page 7.

A: ABC swap

実際にシミュレーションを行えばよいです。以下は C++ における実装例です。

```
#include<iostream>
using namespace std;

int main(){
        int a,b,c;
        cin>>a>>b>>c;
        swap(a,b);
        swap(a,c);
        cout<<a<<"␣"<<b<<"␣"<<c<<endl;
        return 0;
}
```

B: Popular Vote

M 位の商品が総得票数の $\frac{1}{4M}$ 以上の票を獲得していれば YES、そうでなければ NO です。これは得票数順にソートすることで判定できます。

```
N, M = map(int,input().split())
A = list(map(int,input().split()))
A.sort(reverse=True)
S = sum(A)
if A[M-1] >= S / (4*M):
    print("Yes")
else:
    print("No")
```

その他、「総得票数の $\frac{1}{4M}$ 以上の票を獲得している商品が M 個以上あれば YES、そうでなければ NO」として解くこともできます。この場合ソートは不要です。

```
N, M = map(int,input().split())
A = list(map(int,input().split()))
S = sum(A)
cnt = 0
for a in A:
    if a >= S / (4*M):
        cnt += 1
if cnt >= M:
    print("Yes")
else:
    print("No")
```

なお、C++,Java などの言語を利用している場合、除算の挙動に注意してください (整数型変数同士の除算では、結果は整数に切り捨てられます)。

C: Replacing Integer

x が K 以上のとき、操作を行うことで $x-K$ となります。すなわち、N から N/K 回操作を行うことで、整数は N を K で割った余りとなります。

N を K で割った余りを t とします。t に操作を行うと、 $K-t$ となります。$K-t$ に操作を行うと、t に戻るだけです。すなわち K 以下の値として取りうるものは t と $K-t$ のいずれかのみとなります。

よって答えは t と $K-t$ のうち小さい方、すなわち N を K で割った余りと、$K-$ (NをKで割った余り) のうち小さい方です。

D: Lunlun Number

この問題は、Queue というデータ構造を用いることで効率的に解くことができます。まず、空の Queue を 1 つ用意し、$1, 2, ..., 9$ を順に Enqueue します。それから、以下の操作を K 回行います。

- Queue に対して Dequeue を行う。取り出した要素を x とする。
- $x \, mod \, 10 \neq 0$ なら、 $10x + (x \, mod \, 10) - 1$ を Enqueue する。
- $10x + (x \, mod \, 10)$ を Enqueue する。
- $x \, mod \, 10 \neq 9$ なら、 $10x + (x \, mod \, 10) + 1$ を Enqueue する。

K 回目の操作において取り出した数が、 K 番目の Lunlun Number となっています。

E:Yutori

ある期間内に働く日数を最大化するためには、前から貪欲に働く日を決めるのが最適です。したがって前から貪欲に働く日を決めた場合を考えることで「x 回目に働く日は $L[x]$ 日目以降」という配列 L を求めることができます。同様に後ろから貪欲に働く日を決めた場合を考えることで「x 回目に働く日は $R[x]$ 日目以前」という配列 R を求めることができます。i 日目に必ず働くのは、$L[x] = R[x] = i$ となる x が存在するときそのときに限るので、この問題は $O(N)$ で解けました。

F:Division or Substraction

割り算の操作を 1 度もしない場合、操作によって $N \bmod K$ は変化しません。したがって、最終的に 1 になるためには $N \bmod K = 1$ であることが必要十分です。そのような K は $N-1$ の約数のうち、1 でないもの全てであり、その個数は $O(\sqrt{N})$ で求めることができます。

割り算の操作を 1 度以上する場合、そのような K は N の約数です。実際に K で割り切れなくなるまで操作をしたあと、$N \mod K = 1$ になっているかを確かめることにより、各約数毎に $O(\log N)$ で判定することができます。

よって以上により問題が解けました。N および $N-1$ の約数を求める部分がボトルネックとなり、計算量は $O(\sqrt{N})$ となります。

A: ABC swap

You can actually perform the simulation. The following is a sample code in C++.

```
#include<iostream>
using namespace std;

int main(){
        int a,b,c;
        cin>>a>>b>>c;
        swap(a,b);
        swap(a,c);
        cout<<a<<"␣"<<b<<"␣"<<c<<endl;
        return 0;
}
```

B: Popular Vote

If the M-th popular item is with more than or equal to $\frac{1}{4M}$ of the total number of votes, then the answer is YES, otherwise the answer is NO. This can be checked by sorting in the order of votes received.

```
N, M = map(int,input().split())
A = list(map(int,input().split()))
A.sort(reverse=True)
S = sum(A)
if A[M-1] >= S / (4*M):
    print("Yes")
else:
    print("No")
```

Otherwise, it can also be solved by "if there are more than or equal to M items with more than or equal to $\frac{1}{4M}$ of the total number of votes, then the answer is YES, otherwise the answer is NO." In such case, there is no need of sorting.

```
N, M = map(int,input().split())
A = list(map(int,input().split()))
S = sum(A)
cnt = 0
for a in A:
    if a >= S / (4*M):
        cnt += 1
if cnt >= M:
    print("Yes")
else:
    print("No")
```

Be careful of the behavior of division if you are using languages like C++ or Java (quotient between the integers are rounded down).

C: Replacing Integer

If x is more than or equal to K, then by performing the operation it becomes $x-K$. Therefore, by applying the operation N/K times to N, the integer becomes the remainder of N divided by K.

Let t be the remainder of t. When the operation is performed to t, it becomes $K-t$. When the operation is performed to $K-t$, it only goes back to t. Therefore, the possible value less than or equal to K are only either t or $K-t$.

Therefore, the answer is the smaller of t and $K-t$, that is, the smaller of the remainder of N divided by K and $K-$ (*thedivisorof* N *dividedby* K).

D: Lunlun Number

This problem can be solved by utilizing a data structure called Queue. First, prepare an empty queue, and Enqueue $1, 2, ..., 9$ in this order. Then perform the following operations K times.

- Perform Dequeue from the Queue. Let x be the dequeued element.
- If $x \, mod \, 10 \neq 0$, then Enqueue $10x + (x \, mod \, 10) - 1$.
- Enqueue $10x + (x \, mod \, 10)$.
- If $x \, mod \, 10 \neq 9$, then Enqueue $10x + (x \, mod \, 10) + 1$.

The dequeued number in the K-th operation is the K-th Lunlun Number.

E:Yutori

To maximize the number of workdays in a certain period, it is optimal to greedily determining the workdays. Therefore, by considering determining the workdays greedily from the beginning to the end, one can obtain an array K such that "the x-th workday is no earlier than Day $L[x]$." Similarly, by considering determining the workdays greedily from the end to the beginning, one can obtain an array R such that "the x-th workday is no later than Day $R[x]$." He is bound to work on i-th day if and only if there exists a x such that $L[x] = R[x] = i$, so the problem can be solved in a total of $O(N)$ time.

F:Division or Substraction

If the operation of division is never performed, then N mod K stays constant by the operation. So, it will become 1 in the end if and only if N mod $K = 1$. Such K are all the divisors of $N - 1$ except for 1, and the number of them can be counted in a total of $O(\sqrt{N})$ time.

If the operation of division is performed more than once, then such K is a divisor of N. Perform the operation until it is indivisible by K and check whether $N \mod K = 1$, and it can be check in an $O(\log N)$ time for each divisor.

Therefore, here the problem has been solved. The bottleneck part is finding the divisors of N and $N - 1$, and the total time complexity is $O(\sqrt{N})$.

ABC 162 解説

kyopro_friends, gazelle, camypaper, tempura0224, ynymxiaolongbao

2020 年 4 月 12 日

For International Readers: English editorial starts on page 7.

A:Lucky 7

N を文字列として受け取り、7 を含むかどうかで判定できます。

```
int main(){
    char s[4];
    scanf("%s",s);
    if(s[0]=='7'||s[1]=='7'||s[2]=='7')puts("Yes");
    else puts("No");
}
```

B:FizzBuzz Sum

実際に各項が数かどうかを判定し、数ならば足すことで答えを求めることができます。

```
int main(){
    int n;
    scanf("%d",&n);
    long long ans=0;
    for(int i=1;i<=n;i++){
        if(i%3!=0 && i%5!=0)ans+=i;
    }
    printf("%lld\n",ans);
}
```

なお、この問題は $O(1)$ で解くこともできます。

```
long long sum(long long n){return n*(n+1)/2;}
int main(){
    int n;
    scanf("%d",&n);
    long long ans;
    ans=sum(n)-sum(n/3)*3-sum(n/5)*5+sum(n/15)*15;
    printf("%lld\n",ans);
}
```

C: sum of gcd of tuples (easy)

$\gcd(a,b,c) = \gcd(\gcd(a,b),c)$ が成立します。

K 以下の 2 つの数の最大公約数は、ユークリッドの互除法を用いることで $O(\log K)$ で求めることが出来ます。したがって、実際に全ての (a,b,c) の組に対して最大公約数を計算することで、この問題は $O(K^3 \log K)$ で解けました。

C 言語でのユークリッドの互除法の実装例は次のとおりです。

再帰版

```
int gcd(int p, int q){
    if(p % q == 0)return q;
    return gcd(q, p % q);
}
```

非再帰版

```
int gcd(int p, int q){
    while(q != 0){
        int r = p % q;
        p = q;
        q = r;
    }
    return p;
}
```

D: RGB Triplets

1 つ目の条件を満たす組の数は、S に含まれる R, G, B の数をそれぞれ r, g, b としたとき rgb です。

このうち 2 つ目の条件を満たさない組がいくつあるかを考えます。$j - i = k - j$ を満たすような組 $(i,\ j,\ k)$ の個数は $O(N^2)$ です。よって、例えば i, j を固定するといった方法で、この全てを調べて、それが 1 つ目の条件を満たしているかを確認すればいいです。このアルゴリズムの計算量は、後半の全探索がボトルネックになり $O(N^2)$ です。

E: sum of gcd of tuples (hard)

各数列 $\{A_i\}$ に対して最大公約数を計算していては間に合いません。そこで、$1 \leq X \leq K$ に対して「$\gcd(A_1, ..., A_N) = X$ となる数列 $\{A_i\}$ がいくつあるか？」という問題を考えます。これが解ければ元の問題にも答えることが出来ます。

最大公約数が X の倍数であるための必要十分条件は、$A_1, ..., A_N$ が全て X の倍数であることです。そのような数列は $\lfloor \frac{K}{X} \rfloor^N$ 個あります。

ぴったり X であるための必要十分条件は、「X の倍数であり、かつ、$2X, 3X, ...$ ではない」です。X が大きい方から順に計算していくことによって、$2X, 3X, ...$ の個数を引いて求めることができます。

計算量は $O(K \log K + K \log N)$ です。

F:Select Half

この問題は、もし N が十分小さければ次のような DP で解くことができます。

$DP[i][j] = \{i$ 番目までのうちどの 2 個も連続しない j 個を選んだ時の和の最大値 $\}$

この DP には無駄が多いので、ここから状態数を減らします。連続する要素を選んではいけないので、i 番目までの数のうち選べるのは最大で $\lfloor \frac{i+1}{2} \rfloor$ 個です。同様に、残りの $N-i$ 個から選べるのは最大で $\lfloor \frac{N-i+1}{2} \rfloor$ 個なので、最終的に $\lfloor \frac{N}{2} \rfloor$ 個を選ぶためには、i 番目までに $\lfloor \frac{N}{2} \rfloor - \lfloor \frac{N-i+1}{2} \rfloor$ 個以上選んでいる必要があります。

$\lfloor \frac{N}{2} \rfloor - \lfloor \frac{N-i+1}{2} \rfloor \geq \lfloor \frac{i}{2} \rfloor - 1$ であるので、冒頭の DP で考慮すべき j の値は、各 i について $\lfloor \frac{i}{2} \rfloor - 1$ 以上 $\lfloor \frac{i+1}{2} \rfloor$ 以下の高々 3 通りであることがわかり、状態数及び計算量が $O(N)$ となって解けました。

A:Lucky 7

You can judge by receiving N as a string and check if it contains 7.

```
int main(){
    char s[4];
    scanf("%s",s);
    if(s[0]=='7'||s[1]=='7'||s[2]=='7')puts("Yes");
    else puts("No");
}
```

B:FizzBuzz Sum

You can find the answer by actually checking if each term is a number or not, and adding up if it is a number.

```
int main(){
    int n;
    scanf("%d",&n);
    long long ans=0;
    for(int i=1;i<=n;i++){
        if(i%3!=0 && i%5!=0)ans+=i;
    }
    printf("%lld\n",ans);
}
```

This problem can also be solved in a total of $O(1)$ time.

```
long long sum(long long n){return n*(n+1)/2;}
int main(){
    int n;
    scanf("%d",&n);
    long long ans;
    ans=sum(n)-sum(n/3)*3-sum(n/5)*5+sum(n/15)*15;
    printf("%lld\n",ans);
}
```

C: sum of gcd of tuples (easy)

It holds that $\gcd(a, b, c) = \gcd(\gcd(a, b), c)$.

The greatest common divisor between two numbers less than or equal to K can be found in a total $O(\log K)$ time using the Euclidean Algorithm.

Therefore, by actually calculating the greatest common divisors for all (a, b, c), this problem could be solved in a total of $O(K^3 \log K)$ time.

The following are implementation examples of Euclidean Algorithm in C Language.

Recurrent version

```
int gcd(int p, int q){
    if(p % q == 0)return q;
    return gcd(q, p % q);
}
```

Non-recurrent version

```
int gcd(int p, int q){
    while(q != 0){
        int r = p % q;
        p = q;
        q = r;
    }
    return p;
}
```

D: RGB Triplets

The number of pairs that satisfies the first condition is rgb, where r, g and b denotes the number of appearances of R, G, B in S, respectively. Now let's consider how many of them do not satisfies the second condition. The number of tuples $(i,\ j,\ k)$ such that $j - i = k - j$ is $O(N^2)$. Therefore, you can iterate through all the of them by, for example, fixing i and j, and then check if it satisfies the first condition. このアルゴリズムの計算量は、後半の全探索がボトルネックになり $O(N^2)$ です。The time complexity of this algorithm is $O(N^2)$, where the latter brute force is the bottle neck.

E: sum of gcd of tuples (hard)

It won't finish in time if you calculate the greatest common divisor for each sequence $\{A_i\}$. Then, for each $1 \leq X \leq K$, let us consider the problem "how many sequences $\{A_i\}$ are there such that $\gcd(A_1, ..., A_N) = X$?". If this problem was solved, then the original problem can also be answered.

The necessary and sufficient condition of the greatest common divisor being a multiple of X is that all the $A_1, ..., A_N$ are multiples of X. The number of such sequences is $\lfloor \frac{K}{X} \rfloor^N$.

The necessary and sufficient condition of it being exactly X is that "it is a multiple of X, but not $2X, 3X$...." If they are calculated in the decreasing order of X, it can be found by subtracting the number of those of $2X, 3X, ...$

The time complexity is $O(K \log K + K \log N)$.

F:Select Half

If N is small enough, this problem can be solved by the following DP.

$DP[i][j] = \{$The maximum sum of j elements chosen from the first i elements where no two of them are adjacent$\}$

This DP is inefficient a lot, so we will decrease the number of state from this. Since we cannot choose adjacent elements, we can choose at most $\lfloor \frac{i+1}{2} \rfloor$ elements out of the first i elements. Similarly, we can choose at most $\lfloor \frac{N-i+1}{2} \rfloor$ out of the remaining $N-i$ elements, so in order to choose $\lfloor \frac{N}{2} \rfloor$ elements in the end, at least $\lfloor \frac{N}{2} \rfloor - \lfloor \frac{N-i+1}{2} \rfloor$ elements has to be chosen from the first i elements.

Since $\lfloor \frac{N}{2} \rfloor - \lfloor \frac{N-i+1}{2} \rfloor \geq \lfloor \frac{i}{2} \rfloor - 1$, it appears that at most three kinds of j, which is greater than or equal to $\lfloor \frac{i}{2} \rfloor - 1$ and less than or equal to $\lfloor \frac{i+1}{2} \rfloor$, has to be considered, and now the number of states and the time complexity is $O(N)$, so the problem could be solved.

ABC 163 解説

gazelle, kyopro_friends, nuip, latte0119, ynymxiaolongbao, yuma000

2020 年 4 月 19 日

For International Readers: English editorial starts on page 8.

A. Circle Pond

(円の周長) = (円の半径) * 2 * (円周率)
の式より、答えを求めることができます。以下に、今回の問題の注意点をあげておきます。

円周率の求め方

あらかじめ定数として宣言しておいたり、言語によっては標準ライブラリから求めることもできます。(今回は誤差の許容範囲が広いので、実は 3.14 まで求めておけば十分です)。

浮動小数点数の出力

これは言語によって仕様が異なるため一概には言えませんが、整数型への自動的な丸めの発生等に注意しながら実装してください。
以下が、c++ のサンプルコードです。

```
#include <iostream>
#include <math.h>
#include <iomanip>

int main(){
    int R;
    std::cin>>R;

    const double pi=acos(-1.0);

    double answer=2*R*pi;

    std::cout<<std::setprecision(20)<<std::fixed<<answer<<std::endl;
```

```
14
15      return 0;
16  }
```

以下が、Python のサンプルコードです。

```
1  print(int(input())*6.3)
```

B:homework

全ての宿題をするためにかかる日数は $A_1+\ldots+A_M$ です。したがって、最大で $N-(A_1+\ldots+A_M)$ 日間遊ぶことができます。

宿題を終わらせることが不可能な場合に注意してください。

```
int a[10010];
int main(){
    int n,m;
    scanf("%d%d",&n,&m);
    for(int i=0;i<m;i++)scanf("%d",&a[i]);
    int s=0;
    for(int i=0;i<m;i++)s+=a[i];
    if(s>n){
        printf("-1\n");
    }else{
        printf("%d\n",n-s);
    }
}
```

C:management

社員番号 x の社員の直属の部下は $A_i = x$ を満たします。つまり、各数が A に何度登場するかを求めればよいです。

```
int a[200010];
int ans[200010];
int main(){
    int n;
    scanf("%d",&n);
    for(int i=2;i<=n;i++)scanf("%d",&a[i]);
    for(int i=2;i<=n;i++)ans[a[i]]++;
    for(int i=1;i<=n;i++)printf("%d\n",ans[i]);
}
```

D:Sum of Large Numbers

10^{100} は非常に巨大な数であるため、M 個の数の和はほぼ $M \times 10^{100}$ になります。”端数”の影響はわずかであることから、選んだ数の個数が異なると、和が等しくなることはありません。

よって、選んだ数の個数が $K, K+1, ..., N+1$ の各場合に答えを求め、その和を取ればよいです。

M 個の数を選ぶ場合、和としてあり得る最小値は、小さい方から M 個とった場合であり、最大値は大きい方から M 個取った場合です。実は最小値と最大値の間の値は全て作ることができます。（最小値から始めて、選ぶ数を少しずつ大きくしていくことを想像するとわかります）

あとは最大値－最小値を求めることができればよいですが、この計算において $M \times 10^{100}$ は相殺されるので、初めから 10^{100} の部分を無視して計算してよいです。和を求める公式を利用する、あらかじめ累積和を求めておく、尺取法のように和を逐次更新していく、などの方法により、この値は O(1) で求めることができます。

以上より、$O(N)$ 個の場合に対しそれぞれ $O(1)$ の時間で答えを求めることができるので、全体では $O(N)$ でこの問題が解けました。

なお、この問題は $O(1)$ で解くこともできます。

E: Active Infants

はじめ左から i 番目にいる幼児の移動先を p_i とすると、求めるものは $A_i * |i - p_i|$ を足し合わせた値の最大値になります。$|x - y| = max(x - y, y - x)$ であることから、" $A_i * |i - p_i|$ をスコアに足す" と言う代わりに " $A_i * (i - p_i)$ と $A_i * (p_i - i)$ の好きな方を選んでスコアに足す" と言い換えても同値な問題になります。

すると、$A_i * (i - p_i)$ の方を選んだ添字 i の集合に対しては A_i の値の大きい方から p_i を $1, 2, 3, \ldots$ とし、$A_i * (p_i - i)$ の方を選んだ添字 i の集合に対しては A_i の大きい方から p_i を $N, N-1, N-2, \ldots$ とするのが最適だということが言えます。

以上のことを踏まえれば、原始的な動的計画法を用いることで $O(N^2)$ の計算量で答えを求めることができます。具体的には、$DP[x][y]$ を活発度の高い方から $x + y$ 人を既に配置しそのうち x 人では $A_i * (i - p_i)$ を選択し y 人では $A_i * (p_i - i)$ を選択したときのスコアの最大値と定義し、$x + y$ の昇順に計算していけば良いです。

F: path pass i

各色について、その色を一度も通らないようなパスを数え上げることができればよいです。以降、頂点 v と頂点 w を結ぶ単純パスを、$P_{v,w}$ と表します。

適当な頂点を根として DFS を行います。頂点 v では、それぞれの子 u に対して以下を計算します。

- u を根とする部分木の中で、u から色が c_v であるような頂点を一度も通らずに移動できる頂点の数

頂点 v を根とする部分木に含まれる頂点の集合を S_v とすると、上記の値は以下の式で表すことができます。(2020/4/20 修正)

$$|S_u| - \sum_{w \in S_u, c_w = c_v, f(P_{v,w})=2} |S_w|$$

ただし、$f(P_{v,w}) = |\{x \mid x \in P_{v,w}, c_x = c_v\}|$ とします。

各色について $\sum_{w \in S_u, c_w = c_v, f(P_{v,w})=2} |S_w|$ の値を計算することを考えます。

これは、頂点 v に入った時点での値 X_v を保持しておき、出るときに $X_v + |S_v|$ で更新すればよいことがわかります。

詳しくは実装例を確認してください。計算量は $O(N)$ です。

実装例: `https://atcoder.jp/contests/abc163/submissions/12125379`

A. Circle Pond

You can find the answer by the following equations:
(The circumference of a circle) = (the radius of the circle) * 2 * (Pi)
Here are some points of this problem.

The way to find Pi

You can define as a constant beforehand, or it can be also found with the standard library depending on the language. (This time, the accepted range of floating point error is wide, so in fact it is sufficient to use 3.14.)

Outputting floating point numbers

It depends on the specification of the language, but be careful of some points like automatic rounding to the integer type when implementing.

The following is a sample code in C.

```
#include <iostream>
#include <math.h>
#include <iomanip>

int main(){
    int R;
    std::cin>>R;

    const double pi=acos(-1.0);

    double answer=2*R*pi;

    std::cout<<std::setprecision(20)<<std::fixed<<answer<<std::endl;

    return 0;
}
```

The following is a sample code in Python.

```
print(int(input())*6.3)
```

B:homework

The total days needed to finish all the assignments are $A_1 + \ldots + A_M$. Therefore, he can hang out $N - (A_1 + \ldots + A_M)$ days at most.

Be careful of the cases where it is impossible to finish the assignments.

```
int a[10010];
int main(){
    int n,m;
    scanf("%d%d",&n,&m);
    for(int i=0;i<m;i++)scanf("%d",&a[i]);
    int s=0;
    for(int i=0;i<m;i++)s+=a[i];
    if(s>n){
        printf("-1\n");
    }else{
        printf("%d\n",n-s);
    }
}
```

C:management

The immediate subordinates of the member numbered x satisfies $A_i = x$. Therefore, it is enough to find how many times each number appears in A.

```
int a[200010];
int ans[200010];
int main(){
    int n;
    scanf("%d",&n);
    for(int i=2;i<=n;i++)scanf("%d",&a[i]);
    for(int i=2;i<=n;i++)ans[a[i]]++;
    for(int i=1;i<=n;i++)printf("%d\n",ans[i]);
}
```

D:Sum of Large Numbers

Since 10^{100} is a very large number, the sum of M numbers is almost $M \times 10^{100}$. Since the contribution of the "fraction" is small enough, two sets with different numbers of items have not the same sum in common.

Therefore, it is sufficient to find the answer where the number of chosen numbers are $K, K+1, ..., N+1$ and sum them up.

When choosing M numbers, the sum is minimum when the smallest M numbers are chosen, and the sum is maximum when the largest M numbers are chosen. Actually every value between the minimum and the maximum can be constructed. (You can see that by imagining increasing the chosen numbers little by little, starting from the minimum)

All that left is finding maximum - minimum. In this calculation $M \times 10^{100}$ is cancelled out, so you can ignore 10^{100} in the first place. This value can be calculated in an $O(1)$ time with the methods like using the formula of calculating sums, precalculate the cumulative sums, or successively updating the sums like sliding windows.

Hence, the answer can be found in a total of $O(1)$ time for each of $O(N)$ cases, so the original problem could be solved in a total of $O(N)$ time.

Bonus: this problem can also be solved in a total of $O(1)$ time.

E: Active Infants

Let p_i be the destination of the child who are initially at the i-th position from the left, then the desired answer is the maximum sum of $A_i * |i - p_i|$. Since $|x - y| = max(x - y, y - x)$, instead of "adding $A_i * |i - p_i|$ to score," one can "choose either $A_i * (i - p_i)$ or $A_i * (p_i - i)$ arbitrarily and add to the score" without changing the answer of the problem.

Then, it appears that it is optimal to, for the set of indices i where $A_i * (i - p_i)$ was chosen, assign $1, 2, 3, \ldots$ to p_i in the decreasing order of A_i, and for the set of indices i where $A_i * (p_i - i)$ are chosen, assign $N, N-1, N-2, \ldots$ to p_i in the decreasing order of A_i.

From the above, you can find the answer in a total of $O(N^2)$ time using primitive Dynamic Programming. Specifically, let $DP[x][y]$ be the maximum score when children with the $x + y$ highest activity is configured, where $A_i * (i - p_i)$ are chosen for the x children of them, and where $A_i * (p_i - i)$ are chosen for the y children of them; then calculate in the increasing order of $x + y$.

F: path pass i

It is sufficient to, for each color, count the number of paths that does not visit the vertices with the color at all. Hereinafter we denote the simple path connecting vertex v and vertex w by $P_{v,w}$. Take an arbitrary root and perform a DFS from it.

At the vertex v, calculate the value below for each children u.

- The number of vertices in the subtree whose root is u which can be reached from u without visiting the vertices of color c_v

Let S_v be the set of vertices in the subtree whose root is v, then the value above can represented by the following equations:

$$|S_u| - \sum_{w \in S_u, c_w = c_v, f(P_{v,w})=2} |S_w|$$

In the above equation, we define $f(P_{v,w}) = |\{x \mid x \in P_{v,w}, c_x = c_v\}|$.

Consider calculating the value $\sum_{w \in S_u, c_w = c_v, f(P_{v,w})=2} |S_w|$ for each color.

This can be calculated by the following steps: hold the value X_v when entering the vertex v, and update to $X_v + |S_v|$ when exiting.

For details, please check the sample code. The total time complexity is $O(N)$.

Sample Code: `https://atcoder.jp/contests/abc163/submissions/12125379`

ABC 164 解説

kyopro_friends, Kmcode, latte0119, tozangezan, ynymxiaolongbao, wo01

2020 年 4 月 26 日

For International Readers: English editorial starts on page 7.

A:Sheep and Wolves

if 文などにより条件分岐を行うことで、それぞれの場合に対応した出力を行います。

C 言語での実装例

```
int main(){
    int s,w;
    scanf("%d%d",&s,&w);
    if(w>=s)printf("unsafe");
    else printf("safe");
}
```

Python での実装例

```
s, w = map(int, input().split())
if w >= s:
    print("unsafe")
else:
    print("safe")
```

B. Battle

仮にどちらかのモンスターの体力が 0 以下になっても攻撃を続けたとき、高橋君のモンスターの体力が 0 以下になるのは青木君の $\lceil \frac{A}{D} \rceil$ 回目の攻撃のとき、青木君のモンスターの体力が 0 以下になるのは高橋君の $\lceil \frac{B}{C} \rceil$ 回目の攻撃のときです。したがって、$\lceil \frac{A}{D} \rceil$ を X、$\lceil \frac{B}{C} \rceil$ を Y として、$X \geq Y$ なら高橋君の勝ち、そうでなならば青木君の勝ちです。高橋君が先攻であるため $X = Y$ のとき高橋君の勝ちであることに注意してください。

以下が、c++ のサンプルコードです。

```
#include<iostream>
using namespace std;
int main(){
    int A,B,C,D;
    cin>>A>>B>>C>>D;
    int X=(A+D-1)/D;
    int Y=(C+B-1)/B;
    cout<<(X>=Y?"Yes":"No")<<"\n";
}
```

C: gacha

S_i がすでに手に入れたものと重複しているかどうかを毎回確認していると TLE になります。

文字列を辞書順などでソートしてから重複を確認したり、set や map,dict などのデータ構造を用いて管理することで高速に求めることができます。

D: Multiple of 2019

(4 月 27 日: 誤りを訂正しました。)

S の長さを n、S の左から k 番目の数字を a_k とします。

左から $k+1$ 文字目以降の数字列を整数とみなした時の値 $a_n + 10a_{n-1} + ... + 10^{n-k-1}a_{k+1}$ を T_k とします ($T_n = 0$)。 すると、(i, j) が条件をみたすのは $T_{i-1} == T_j (mod 2019)$ のときです。

各 $T_i mod 2019$ を計算し、mod の値が同じごとに個数を計算して足したものが答えです。$T_i = T_{i+1} + 10^{n-i-1}a_i$ より、これは DP で計算でき、mod ごとに $m(m-1)/2$ (m は $T_i mod 2019$ がある値になる $0 \leq i \leq n$ の個数) を足せば良いです。

E: Two Currencies

$A_{max} = max\{A_i \mid i = 1, 2, ..., M\}$ とします。銀貨を $A_{max}(N-1)$ 枚以上所持している場合の戦略を考えてみましょう。これは明らかに、最短経路に沿って目的地に向かうのが最適です (このとき消費する銀貨の枚数は高々 $A_{max}(N-1)$ 枚なので、銀貨が足りなくなることはありません)。従って、以下の制約を追加しても、問題の答えは変わりません。

追加の制約

移動の途中で所持している銀貨の枚数が $A_{max}(N-1)$ 枚を超えた場合、$A_{max}(N-1)$ 枚になるまで銀貨を捨てる

上記の制約を追加した問題に対するアルゴリズムを考えましょう。(現在の頂点, 所持している銀貨の枚数) を状態として dijkstra 法を適用すると、$O(A_{max}NMlog(A_{max}N))$ でこの問題を解くことができます。$A_{max} \leq 50, N \leq 50, M \leq 100$ なので、十分高速です。

F: I hate Matrix Construction

まず、2 進数表現したときの各ビットの値は独立に決められるので、それぞれのビットで独立に考えます。そうすると $N \times N$ の行列に条件を満たすような 0, 1 を当てはめてる問題に落ちます。

さて、各要素について、値を確定してよい部分を先に確定させましょう。
各行及び各列について、「論理積が 1 」というのが条件だったとき、その行・列のすべての要素は 1 である必要があります。
同様に、「論理和が 0 」というのが条件だったとき、その行・列のすべての要素は必ず 0 である必要があります。
さらに、確定していない各要素について、その要素が所属している行・列の条件の組み合わせが論理和、論理積にかかわらず同じ値だったとき、その要素をその値に確定しても問題ありません。
ここまでの操作を「基本的な操作」とします。

ここで、まだ確定していない要素をすべて 0 で埋めることにすると、各行・各列について「論理和が 1 」であるという条件が満たせない場合があります。
まず、行の「論理和が 1 」という条件を満たすことを考えましょう。条件を満たすには対応する行に所属する要素 1 つを適切に選んで 1 にすればよいです。
そもそも、一連の基本的な操作で条件を満たすことができなかった場合、列は「論理積が 0 」と「論理和が 0 」の場合で構成されているときのみであることがわかります。
「論理和が 0 」の場合その列の要素はすべて 0 である必要がありますので、「論理積が 0 」の列と現在見ている行に所属する要素 1 つを適切に選んで 1 にかえる必要があります。
具体的には、「論理積が 0 」の列 1 つを選んで、その列に 0 が 1 個以上残るならば、対応する要素 1 つを 1 に変えます。

列の「論理和が 1 」という条件を満たすことは同様の操作で可能です。

一連の操作に矛盾が発生した場合、すべての条件を満たす行列は構築できないことがわかります。

実装例: `https://atcoder.jp/contests/abc164/submissions/12391952`

A:Sheep and Wolves

Perform conditional branch using statements like if statements, and output the string corresponding to each case.

Implementation example in C:

```c
int main(){
    int s,w;
    scanf("%d%d",&s,&w);
    if(w>=s)printf("unsafe");
    else printf("safe");
}
```

Implementation example in Python:

```python
s, w = map(int, input().split())
if w >= s:
    print("unsafe")
else:
    print("safe")
```

B. Battle

If they continue to attack each other even after the health of either monster becomes 0 or below, then the health of Takahashi's monster becomes 0 or less in the $\lceil \frac{A}{D} \rceil$-th Aoki's attack, and the health of Aoki's monster becomes 0 or less in the $\lceil \frac{B}{C} \rceil$-th Aoki's attack. Therefore, let X be $\lceil \frac{A}{D} \rceil$ and Y be $\lceil \frac{B}{C} \rceil$, and Takahashi will win if $X \geq Y$, and Aoki will win otherwise. Note that Takahashi will win when $X = Y$ because Takahashi takes turn first.

The following is a sample code in c++.

```
#include<iostream>
using namespace std;
int main(){
    int A,B,C,D;
    cin>>A>>B>>C>>D;
    int X=(A+D-1)/D;
    int Y=(C+B-1)/B;
    cout<<(X>=Y?"Yes":"No")<<"\n";
}
```

C: gacha

Checking if S_i duplicates to one of the obtained elements so far leads to TLE.

You can calculate fast by first sorting the strings in the lexicographical order and checking for duplicates, or managing them with the data structures like set, map or dict.

D: Multiple of 2019

(Corrected on April 27.)

Let n be the length of S, and a_k be the k-th leftmost digit of S.

(i, j) satisfies the condition when $a_j + 10a_{j-1} + 100a_{j-2} + ... + 10^{j-i}a_i$ is a multiple of 2019.

By the way, 2019 is not divisible by 2 or 5, so there exists the inverse of 10 in $mod 2019$. So $10n$ is a multiple of 2019 if and only if n is a multiple of n. (2020 does not satisfy such conditions, so 2019 is used for this problem.)

Let T_k be the digits after the $k+1$-th digit, represented as an integer, that is, $a_n + 10a_{n-1} + ...+10^{n-k-1}a_{k+1}$ ($T_n = 0$). Then, (i, j) satisfies the condition when $T_{i-1} == T_j (mod 2019)$ (Here the discussion above was used).

Ultimately, it appears that it is sufficient to calculate each $T_i mod 2019$, and count the number for the elements with the same mod and sum them up. Since, $T_i = T_{i+1} + 10^{n-i-1}a_i$, this can be calculated by DP, and it is sufficient to add $m(m-1)/2$ for each mod (where m is the number of i such that $T_i mod 2019$ is equal to that remainder).

E: Two Currencies

Let $A_{max} = max\{A_i \mid i = 1, 2, ..., M\}$. Consider the strategy when you have more than $A_{max}(N-1)$ silver coins. Obviously, it is optimal to go to the destination along the shortest path (during that, at most $A_{max}(N-1)$ coins are consumed, so you never run out the silver coins). Therefore, adding the constraints below does not change the answer of the problem.

Additional constraints

If the number of silver coins exceed $A_{max}(N-1)$ during the travel, discard them until it becomes $A_{max}(N-1)$

Let us consider the algorithm to the problem under the constraints above. By applying Dijkstra method while holding the state of (current vertex, the number of silver coins), the problem can be solved in a total of $O(A_{max}NMlog(A_{max}N))$ time. Since $A_{max} \leq 50, N \leq 50, M \leq 100$, it is fast enough.

F: I hate Matrix Construction

First, when binary notated, the value of each bit can be determined independently, so we consider each bit independently. Then this problem falls into a construction of matrix of size $N \times N$ consisting of 0 and 1 that satisfies the conditions.

Now, let us determine the elements where it can be determined first.
For each row and column, if the condition is that "the logical product is 1," then all the elements of the row/column has to be 1.
Similarly, if the condition is that "the logical sum is 0," then all the elements of the row/column has to be 0.
Moreover, for each undetermined element, if the conditions of the row column that the element belongs was the same, no matter it is of logical sum or product, then the value of the element can be determined to that value.
We will call the operations so far as "basic operations."

Then, if the undetermined elements are all filled by 0, then some conditions of rows/columns that "the logical sum is 1" may not be satisfied.
First, let's consider satisfying the condition that "the logical sum of a row is 1." To satisfy the condition, we can appropriately choose one of the elements belongs to the corresponding row and set it 1.
In the first place, it can be seen that a condition could not be satisfied by the basic operations only if the columns are composed of "the logical product is 0" and "the logical sum is 0."
If "the logical sum is 0," then all the elements of the column has to be 0, so we have to choose such a appropriate element that belongs to the column of "the logical product is 0" and the column we are currently looking at, and set its value 1.
Specifically, choose a column such that "the logical product is 0," and if 0 remains in the column, then pick a corresponding element and set it 1.

Satisfying the conditions of "the logical product of the column is 1" can be satisfied by the similar operations.

If any of those operations leads to contradiction, then it appears that a matrix that satisfies all the conditions cannot be constructed.

Sample Code: `https://atcoder.jp/contests/abc164/submissions/12391952`

ABC 165 解説

beet, DEGwer, kort0n, sheyasutaka, tozangezan, ynymxiaolongbao

2020 年 5 月 2 日

For International Readers: English editorial starts on page 7.

A: We Love Golf

いろいろな方法がありますが，ここでは B 以下の最大の K の倍数を求め，それが A 以上かどうか確かめる方法を採ってみましょう．

実装はたとえば以下のようになります．

Listing 1　C での実装例

```c
#include <stdio.h>

int main(void) {
        int k, a, b;

        scanf("%d", &k);
        scanf("%d%d", &a, &b);

        int largest = (b / k) * k;
        if (a <= largest) {
                puts("OK");
        } else {
                puts("NG");
        }

        return 0;
}
```

B: 1%

今持っている金額を P とすると、$P < X$ のあいだ P に $\left\lfloor \frac{P}{100} \right\rfloor$ を足せばよいです。
入力例 2 からもわかるように、 ループ回数は高々 3760 回となり、十分高速です。
64bit 整数を扱う言語ではオーバーフローに注意してください。
($P' = \left\lfloor \frac{P \times 101}{100} \right\rfloor$ のように計算するとオーバーフローする場合があります)
以下は C++ による回答例です。

```
#include <bits/stdc++.h>
using namespace std;
int main(){
  long long X;
  cin >> X;
  long long P = 100, step = 0;
  while(P < X){
    P += P / 100;
    step++;
  }
  cout << step << endl;
  return 0;
}
```

C: Many Requirements

結論から述べると, 問の条件を満たす数列を全探索します.
問の条件を満たす数列は, N 個の仕切りと $M-1$ 個のボールを並び替えた列と一対一に対応します.
実際, 各列に対して,

$$A_i = i \text{ 番目の仕切りより左側に存在するボールの数} + 1$$

とすれば, 前述の一対一対応が得られます.
N 個の仕切りと $M-1$ 個のボールを並び替える列の数は, ${}_{N+M-1}\mathrm{C}_N$ 通り存在します ($N+M-1$ 個の場所から仕切りを置く N 箇所を選ぶ方法の数え上げに相当).
各数列を深さ優先探索で生成する場合, 各数列の生成に要する時間は $O(N)$ ですから, 全ての数列の生成に要する時間は $O(N\,{}_{N+M-1}\mathrm{C}_N)$ です.
数列を決めればその数列のスコアは $O(Q)$ で計算出来ます.
以上より, この問題は $O((N+Q)\,{}_{N+M-1}\mathrm{C}_N)$ で解けました.

D: Floor Function

$f(x) = floor(Ax/B) - A \times floor(x/B)$ とします。

$f(x+B) = f(x)$ であることは実際に代入することで容易に分かります。なので、$0 \leq x \leq B-1$ の場合のみを考えます。

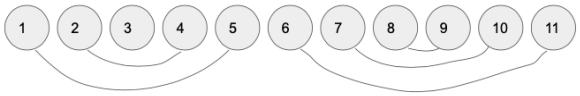
このとき、$f(x) = floor(Ax/B) - A \times floor(x/B) = floor(Ax/B)$ で、$floor(Ax/B)$ は広義単調増加なので、与えられた制約 $(0 \leq x \leq B-1,\ 0 \leq x \leq N)$ の中で最も大きい x について、$f(x)$ が求める最大値です。

よって、答えは $f(min(B-1, N))$ です。

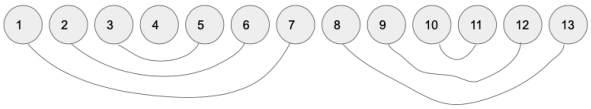
E:Rotation Matching

$N = 2 \times M + 1$ のケースを考えます。すると、以下のような割り振りが条件を満たすことがわかります。ただし、図では、線で結ばれた数字を同じ対戦場に割り振ることを意味します。

M が奇数のとき（具体例は $M = 5$ のとき）



M が偶数のとき（具体例は $M = 6$ のとき）



N が $2 \times M + 1$ より大きい場合でも、このような割り振りが条件を満たします。

F: LIS on Tree

結論から述べると、この問題は最長増加部分列問題を解く動的計画法に巻き戻しのテクニックを適応させることで解くことができます。

はじめに、最長増加部分列問題 (Longest Increasing Subsequence) を解く動的計画法について説明します。LIS には様々なアプローチがありますが、二分探索を用いた動的計画法が最も有名です。dp_i を長さが i である増加部分列における最終要素の最小値（存在しない場合は無限大）として、$dp_1, dp_2, .., dp_N$ を無限大に初期化し、dp_i の値が A_j 以上になる最も小さな i に対して dp_i を A_j の値に更新するということを $j = 1, 2, .., N$ について行っていくといった方法です。

次に、巻き戻しのテクニックについて説明します。ここでの巻き戻しとは、変数の値の更新と質問を次々に処理するという状況の中で、直前の更新をする前の状態に戻すことをいいます。以下のような処理をすることで実現できます。

- ある変数の値を更新したとき、
  - どの変数を更新したか
  - その変数の元の値は何だったか

  という情報を stack に push する。
- 巻き戻しをするとき、stack の top に示された変数をその元の値に変更し、stack を pop する。

最後に、この問題への解法を説明します。頂点 1 を始点として深さ優先探索をし、各頂点に対応する再帰関数内では、はじめに dp の値を更新して答えを求めた後、隣接する頂点に対応する再帰関数を呼び出し、最後に dp の値を巻き戻すことで、すべての頂点までの答えを正しく求めることができます。計算量は $O(NlogN)$ です。

A: We Love Golf

There are many ways to solve this problem, but let us introduce the following way: find the maximum multiple of K which is less than or equal to B, and check if it is equal to A.

The implementation would be as follows for example.

Listing 2 Sample Code in C

```
#include <stdio.h>

int main(void) {
        int k, a, b;

        scanf("%d", &k);
        scanf("%d%d", &a, &b);

        int largest = (b / k) * k;
        if (a <= largest) {
                puts("OK");
        } else {
                puts("NG");
        }

        return 0;
}
```

B: 1%

Let P be the amount of money he has, then it is sufficient to add $\left\lfloor \frac{P}{100} \right\rfloor$ to P while $P < X$.

As it can be seen from Sample Input 2, the maximum number of loops is 3760 times, which is fast enough.

In the language which treats 64-bit integers, be careful of overflows.

(Calculating like $P' = \left\lfloor \frac{P \times 101}{100} \right\rfloor$ may lead to overflow.)

The following is a sample code in a C++.

```
#include <bits/stdc++.h>
using namespace std;
int main(){
  long long X;
  cin >> X;
  long long P = 100, step = 0;
  while(P < X){
    P += P / 100;
    step++;
  }
  cout << step << endl;
  return 0;
}
```

C: Many Requirements

To come to the point, perform a brute-force search through all the sequences that satisfies the conditions given in the problem.

A sequence that satisfies the conditions corresponds one-to-one to a rearranged sequence of N bars and $M-1$ balls. Actually, for each sequence, let

$$A_i = \text{the number of balls that are left to the } i\text{-th bar},$$

then the aforementioned one-to-one correspondence can be obtained. The number of arrangements of N bars and $M-1$ balls is ${}_{N+M-1}\mathrm{C}_N$ (which is equivalent to counting the number of ways of choosing the positions of N bars out of $N+M-1$ places).

When generating each sequence with depth-first searching, each sequence needs a total of $O(N)$ time to be generated, so the total time needed to generate all the sequences is $O(N\,{}_{N+M-1}\mathrm{C}_N)$.

Once a sequence is fixed, its score can be calculated in a total of $O(Q)$ time.

Therefore, this problem could be solved in a total of $O((N+Q)\,{}_{N+M-1}\mathrm{C}_N)$ time.

D: Floor Function

Let us define$f(x) = floor(Ax/B) - A \times floor(x/B)$.

It can be seen easily that $f(x + B) = f(x)$ by actually assigning. So, we consider only $0 \leq x \leq B - 1$.

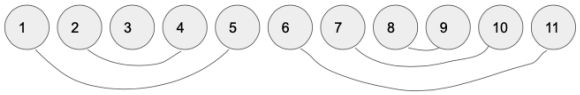
Then, $f(x) = floor(Ax/B) - A \times floor(x/B) = floor(Ax/B)$, and since $floor(Ax/B)$ is monotonically non-decreasing, so for the maximum x within the given constraints ($0 \leq x \leq B-1$, $0 \leq x \leq N$), $f(x)$ is the desired maximum value.

Therefore, the answer is $f(min(B - 1, N))$.

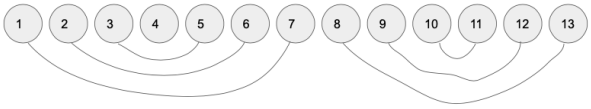
E:Rotation Matching

Let us consider the case where $N = 2 \times M + 1$. Then, it appears that the following assignment satisfies the conditions. Note that in the figure a line that connects two numbers denote that they are assigned to the same arena.

When M is odd (specifically, when $M = 5$)



When M is odd (specifically, when $M = 6$)



Even when N is greater than $2 \times M + 1$, such assignments still satisfies the conditions.

F: LIS on Tree

To come to the point, this problem can be solved by applying rollback technique to the dynamic programming of solving the longest increasing subsequence problem.

First, we will explain about the dynamic programming to solve Longest Increasing Subsequence problem. Among various approaches of LIS, the most famous one is to use binary searching. Specifically, this can be done as follows: let dp_i be the minimum last element of increasing subsequence of length i (or infinity if not exists) initialize $dp_1, dp_2, .., dp_N$ with infinity, then for each $j = 1, 2, .., N$, find the smallest i such that dp_i is greater than or equal to A_j, and then update dp_i to A_j

Next, we will explain about the rollback technique. Here the rollback refers to, when processing updates of values of variables and questions one after another, withdrawing the changes as it was before the latest changes. This can be achieved by the following processes:

- When the value of a variable has been updated, record
  - which variable has been updated and
  - what was the original value of the variable

  and push them to a stack.
- When rolling back, update the variable indicated by the top of the slack to the original value, and pop the stack.

At last, we will explain how to solve this problem. Perform a depth-first searching starting from vertex 1, and in the recurrence function corresponding to each vertex, first update the value of dp and find the answer, then call the recurrence functions corresponding to the adjacent vertices, and finally roll back the value of dp; this way, you can find the answer for all the vertices. The time complexity is $O(NlogN)$.

ABC 166 解説

gazelle, kort0n, kyopro_friends, sheyasutaka, ynymxiaolongbao

2020 年 5 月 3 日

For International Readers: English editorial starts on page 8.

A: A?C

与えられた文字列が ABC なら ARC を、ARC なら ABC を出力します。

```
#include <iostream>
using namespace std;
int main(){
    string s;
    cin >> s;
    if(s == "ABC"){
        cout << "ARC" << endl;
    }else{
        cout << "ABC" << endl;
    }
}
```

B: Trick or Treat

各すぬけ君について, お菓子を持っているか否かを調べます.
以下は C++ での実装例です.

```
#include <bits/stdc++.h>
using namespace std;

int N, K;
vector<int> d;
vector<vector<int>> A;

void input() {
    cin >> N >> K;
    d.resize(K);
    A.resize(K, vector<int>());
    for(int i = 0; i < K; i++) {
        cin >> d[i];
        A[i].resize(d[i]);
        for(int j = 0; j < d[i]; j++) {
            cin >> A[i][j];
        }
    }
}

void solve() {
    int ans = 0;
    for(int i = 1; i <= N; i++) {
        bool have = false;
        for(auto v : A) {
            for(auto p : v) {
                if(p == i) have = true;
            }
        }
        if(!have) ans++;
    }
    cout << ans << endl;
}

int main() {
    input();
    solve();
```

```
38      return 0;
39  }
```

C. Peaks

グラフの問題であるので隣接リストでグラフを表現するといった実装もありますが、今回はそれを用いない方法を紹介します。「展望台 i から一本の道を使って辿り着けるどんな展望台よりも展望台 i の方が標高が高い」ことは、「展望台 i から一本の道を使って辿り着ける展望台の標高の最大値よりも展望台 i の標高が高い」ことと等しいです。このことを利用すると、展望台 i から一本の道を使って辿り着ける展望台の標高の最大値を求めることで答えを求めることができます。

以下が、c++ のサンプルコードです。

```
#include<iostream>
#include<algorithm>
using namespace std;
#define N 100010
int main(){
    int n,m,h[N],ma[N];
    cin>>n>>m;
    for(int i=1;i<=n;i++){
        cin>>h[i];
        ma[i]=0;
    }
    for(int i=0;i<m;i++){
        int a,b;
        cin>>a>>b;
        ma[a]=max(ma[a],h[b]);
        ma[b]=max(ma[b],h[a]);
    }
    int ans=0;
    for(int i=1;i<=n;i++){
        ans+=h[i]>ma[i];
    }
    cout<<ans<<endl;
}
```

D. I hate Factorization

添字が負でもよい数列 $a_n = n^5 - (n-1)^5$ すなわち $...781, 211, 31, 1, 1, 31, 211, 781...$ を考えます。この数列は、a_0 から左に、a_1 から右に向けて値が大きくなっていくことがわかります。数列の値がはじめて 10^9 を超えるのは、$a_{120} = 120^5 - 119^5 = 1019663401$ のときと、$a_{-119} = (-119)^5 - (-120)^5 = 1019663401$ のときになります。よって、$A - B = 1$ を仮定すると、$-118 \leq A \leq 119$ のケースだけ考えれば良いことがわかります。

そして、n^5 が n に対して単調増加であることに注意すると、$A - B > 1$ の場合でも $-118 \leq A \leq 119$ のケースだけ見ていれば十分だとわかります。同じ A の値に対して、$A - B$ の値が大きくなって B の値が小さくなれば、B^5 の値も小さくなって、$A^5 - B^5$ の値は大きくなるためです。$A - B = 1$ のときと同様に数列 $a_n = n^5 - (n-2)^5$ や $a_n = n^5 - (n-3)^5$ などについて考えることで更に A の範囲を狭くすることができますが、今回は計算時間に余裕があるためその必要はありません。

A と B が整数かつ $1 \leq A^5 - B^5$ より $A - B \geq 1$ なので、これですべての場合が網羅されています。B に対しても同じ考察をすると $-118 \leq A \leq 119$ かつ $-119 \leq B \leq 118$ なる整数の組 (A, B) に候補を絞ることができ、これらの候補が実際に式を満たすかどうか判定していくことで解くことができます。

E: This Message Will Self-Destruct in Five Seconds

参加者 i, j $(i < j)$ について，条件は以下のように書けます:

$$j - i = A_i + A_j$$

これを式変形すると，以下のようになります:

$$i + A_i = j - A_j$$

そこで，$L_i = i + A_i, R_i = i - A_i$ と定義すると，条件は $L_i = R_j$ と言い換えられます．したがって，とり得る値 X を全て試し，

$$\begin{cases} L_i = X \\ R_j = X \end{cases}$$

を満たす (i, j) の組を求めればいいですが，上と下の条件は独立なので，たとえば連想配列を使えば簡単に求まります．

計算量は $\mathcal{O}(N \log N)$ 時間 (X を見る範囲を少し工夫すれば $\Theta(N)$ 時間) です．

F:Three Variables Game

- $A+B+C=0$ のとき
  N は 1 以上なので、答えは No です。
- $A+B+C=1$ のとき
  いつどの二つの変数を見ても少なくとも一方は零なので、それぞれのターンでの行動が一意に定まります。
- $A+B+C>=2$ のとき
  一回目のターンで足し引きする変数が両方零であれば答えは No、そうでなければ答えは Yes です。後者の場合、以下のような戦略をそれぞれのターンで取ることで足し引きする変数のいずれかが正である状態を保つことができます。
  - 足し引きする変数の片方が零でもう一方が正のとき、零である方に 1 を足し、もう一方から 1 を引く。
  - $A+B+C=2$ であり、足し引きする変数が両方 1 であり、最後のターンでなく、次のターンの文字列と現在のターンの文字列が異なるとき、次のターンで足し引きする予定がある方の変数に 1 を足し、もう一方から 1 を引く。
  - それ以外のとき、適当に選択する

A: A?C

If the given string is ABC, then print ARC; if it is ARC, then print ABC.

```
#include <iostream>
using namespace std;
int main(){
    string s;
    cin >> s;
    if(s == "ABC"){
        cout << "ARC" << endl;
    }else{
        cout << "ABC" << endl;
    }
}
```

B: Trick or Treat

For each Snuke, judge if he has snacks or not.

The following is a sample code in C++.

```
#include <bits/stdc++.h>
using namespace std;

int N, K;
vector<int> d;
vector<vector<int>> A;

void input() {
    cin >> N >> K;
    d.resize(K);
    A.resize(K, vector<int>());
    for(int i = 0; i < K; i++) {
        cin >> d[i];
        A[i].resize(d[i]);
        for(int j = 0; j < d[i]; j++) {
            cin >> A[i][j];
        }
    }
}

void solve() {
    int ans = 0;
    for(int i = 1; i <= N; i++) {
        bool have = false;
        for(auto v : A) {
            for(auto p : v) {
                if(p == i) have = true;
            }
        }
        if(!have) ans++;
    }
    cout << ans << endl;
}

int main() {
    input();
    solve();
```

```
38      return 0;
39  }
```

C. Peaks

Since this is a problem of a graph, an implementation of representing the graph with adjacency list, but this time we will introduce a way without them. That "the elevation of Obs. i is higher than those of all observatories that can be reached from Obs. i using just one road" is equivalent to that "the elevation of Obs. i is higher than the maximum elevation of observatories that can be reached from Obs. i using just one road." Using this fact, you can find the answer by calculating the maximum elevations of observatories that can be reached from Obs. i using just one road.

The following is a sample code in C++.

```
#include<iostream>
#include<algorithm>
using namespace std;
#define N 100010
int main(){
    int n,m,h[N],ma[N];
    cin>>n>>m;
    for(int i=1;i<=n;i++){
        cin>>h[i];
        ma[i]=0;
    }
    for(int i=0;i<m;i++){
        int a,b;
        cin>>a>>b;
        ma[a]=max(ma[a],h[b]);
        ma[b]=max(ma[b],h[a]);
    }
    int ans=0;
    for(int i=1;i<=n;i++){
        ans+=h[i]>ma[i];
    }
    cout<<ans<<endl;
}
```

D. I hate Factorization

Consider a sequence that allows negative index $a_n = n^5 - (n-1)^5$, i.e. $...781, 211, 31, 1, 1, 31, 211, 781....$ This sequence is increasing toward left from a_0, and toward right from a_1. The elements exceed 10^9 for the first time when $a_{120} = 120^5 - 119^5 = 1019663401$ and when $a_{-119} = (-119)^5 - (-120)^5 = 1019663401$. Therefore, assuming that $A - B = 1$, it appears that it is sufficient to consider only the range of $-118 \leq A \leq 119$.

Moreover, considering the increasing monotonicity of n^5 to n, it appears that it is sufficient to consider only the range of $-118 \leq A \leq 119$ even when $A - B > 1$. When A is fixed, if $A - B$ increases and B decreases, then B^5 decreases too and $A^5 - B^5$ increases. As with when $A - B = 1$, you can restrict the range of A even narrower by considering sequences $a_n = n^5 - (n-2)^5$ or $a_n = n^5 - (n-3)^5$, but this time there is a plenty of execution time, so there is no need to do so.

Since A and B are integers and $1 \leq A^5 - B^5$, it holds that $A - B \geq 1$, so all the cases are covered by them. By applying the same observations for B, you can narrow down the candidates of ranges to the pairs of integers (A, B) such that $-118 \leq A \leq 119$ and $-119 \leq B \leq 118$, And check if those candidates actually satisfies the equation.

E: This Message Will Self-Destruct in Five Seconds

For participants i, j $(i < j)$, the condition can be written as follows:

$$j - i = A_i + A_j$$

By transforming this equation, the following can be obtained:

$$i + A_i = j - A_j$$

So, let $L_i = i + A_i, R_i = i - A_i$, then the condition can be written as $L_i = R_j$. Therefore, it is sufficient to search exhaustively through all the possible X and count the number of (i, j) such that

$$\begin{cases} L_i = X \\ R_j = X \end{cases}$$

. Since the former and the latter are independent, it can be obtained easily by using associative arrays for example.

The total time complexity is $\mathcal{O}(N \log N)$ (or $\Theta(N)$ if you properly narrowed the range of X).

F:Three Variables Game

- When $A + B + C = 0$
  Since N is at least one, the answer is No.
- When $A + B + C = 1$
  Whenever you see whatever two variables, at least one of them is zero, so the action for each step is uniquely determined.
- When $A + B + C >= 2$
  If the two variables which you modify in the first turn are both zero, the answer is No; otherwise the answer is Yes. In the latter case, you can always keep one of the two variables modified for each turn to be positive by the following strategy:
  - If one of the variables that will be modified is zero and the other is positive, then add one to the variable whose value is zero, and subtract one from the other.
  - If $A + B + C = 2$, the variables that will modified is both one, it is not the last turn, and the string for this turn and the next turn differs, then add one to the variables that will be modified for the next time, and subtract one from the other.
  - Otherwise, choose arbitrarily

ABC 167 解説

gazelle, kort0n, kyopro_friends, potetisensei, sheyasutaka, tempura0224

2020 年 5 月 10 日

For International Readers: English editorial starts on page 7.

A: Registration

T の最後の 1 文字を消去して得られる文字列が S と一致していれば答えは Yes であり, そうでなければ No です.
以下は C++ での実装例です.

```
#include <bits/stdc++.h>
using namespace std;

string S, T;
void input() {
    cin >> S >> T;
}

void solve() {
    T.pop_back();
    if(S == T) cout << "Yes" << endl;
    else cout << "No" << endl;
}

int main() {
    input();
    solve();
    return 0;
}
```

B: Easy Linear Programming

大きい数が書かれたカードから貪欲に選んでいくのが最適です。

ナイーブにシミュレートすると時間がかかってしまうので、以下のコードのように場合分けをするとよいです。

Listing 1　C++ による実装例

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
using P = pair<ll, ll>;
#define rep(i, n) for(ll i = 0; i < (ll)n; i++)

int main() {
    int a, b, c, k;
    cin >> a >> b >> c >> k;
    if(k <= a) cout << k << endl;
    else if(k <= a + b) cout << a << endl;
    else cout << a - (k - a - b) << endl;
    return 0;
}
```

C: Skill Up

どの参考書を購入するかを決めると、それらを読むことによってすべての理解度を X 以上にできるかどうかの判定は難しくありません。
そこで、購入する参考書の組み合わせ全てについて上記の判定をし、条件を満たすもののなかでの合計金額の最小値を求めることにしましょう。
購入する参考書の組み合わせは、各参考書について 買う or 買わない を選ぶ場合の数なので全部で 2^N 通りです。1 回あたりの判定は $O(NM)$ でできるので計算量は全体で $O(2^N NM)$ であり、今回の制約においては十分高速です。
購入する参考書の組を全探索する部分の実装は、「深さ優先探索」あるいは 集合を整数にエンコードする (いわゆる「bit 全探索」) などを用いるとよいでしょう。

以下は C++ における bit 全探索での実装例です。https://atcoder.jp/contests/abc167/submissions/13086283

D: Teleporter

町 1 から愚直に移動をシミュレートしていくと，N 回以内に既に来た町にもう一度帰りつきます．そこでその町を「香り高い町」として，香り高い町から移動を続けて何度目に香り高い町に帰りつくかを (これまた愚直に) 求めます．あとは，残りの移動回数をこれで割ったあまりは N 未満なので，愚直にシミュレートすればよいです．

E: Colorful Blocks

「隣り合うブロックの組であって同じ色で塗られている組が、丁度 k 組である」場合の色の塗り方を考えます.
条件を満たす為の必要十分条件は, 左端を除いた $N-1$ 個のブロックのうち, k 個が左隣のブロックと同じ色で塗られており, $N-1-k$ 個が左隣と異なる色で塗られていることです.
このような色の塗り方は, 左端のブロックの色の選び方及び前述の k 個のブロックの選び方も考慮すると,

$$M \times {}_{N-1}\mathrm{C}_k \times 1^k \times (M-1)^{N-1-k}$$

で表されます. この値を $k = 0, 1, \cdots, K$ について足し合わせることで, 答えを求めることが出来ます.

F: Bracket Sequencing

概要

文字列のうち '(' と ')' が連続している箇所があれば、その 2 文字を予め取り除いても結果に影響しません。この操作を繰り返すことで、各 S_i は「')' が 0 個以上連続したあと、'(' が 0 個以上連続するような文字列 (空文字列含む)」となります。このときの')' の個数を A_i, '(' の個数を B_i とします。

S_i たちを次の順にソートして連結したものが括弧列になっていれば Yes、そうでなければ No が答えです。

前半： $B_i - A_i \geq 0$ であるような S_i たちを、A_i が小さい順に並べたもの

後半： $B_i - A_i < 0$ であるような S_i たちを、B_i が大きい順に並べたもの

正当性は、「もしこのような順になっていない隣接 2 項があれば、その 2 項の順序を入れ替えても損をしない」という方針で示すことができます。

気持ち

括弧列は、「全ての括弧がうまくペアになっている文字列」です。文字列を前から順に見て「すでに全ての括弧がペアになっているにも関わらず')' が登場した」とき、そのときに限り、その文字列は括弧列ではありません。なので、'(' が増える文字列を先に使うのが良さそうです。すでにある'(' の貯金分以上の')' をつなげることは出来ないので、A_i の小さい順に使うのが良さそうです。

また、文字列が括弧列であることと、文字列全体を反転し、'(' を')' に、')' を'(' に置き換えた文字列が括弧列であることは同値です。したがって後半も同様の条件を得ることができます。

実装

A_i, B_i は括弧を取り除く操作を実際に行うことなく、次のようにして文字列長に対して線形な時間で求めることができます。

$$A_i = S_i\text{の prefix についての、')' の個数} - \text{'(' の個数 の最大値}$$
$$B_i = S_i\text{の suffix についての、'(' の個数} - \text{')' の個数 の最大値}$$

A: Registration

If the string obtained by removing the last letter from T is equal to S, then the answer is Yes; otherwise the answer is No.
The following is a sample code in C++.

B: Easy Linear Programming

It is optimal to greedily choose the cards with larger numbers.

Simulating naively takes time, so it is good to split into cases like as follows.

Listing 2 Sample C++ Implementation

```
#include<bits/stdc++.h>
using namespace std;
using ll = long long;
using P = pair<ll, ll>;
#define rep(i, n) for(ll i = 0; i < (ll)n; i++)

int main() {
    int a, b, c, k;
    cin >> a >> b >> c >> k;
    if(k <= a) cout << k << endl;
    else if(k <= a + b) cout << a << endl;
    else cout << a - (k - a - b) << endl;
    return 0;
}
```

C: Skill Up

If the books to buy are determined, then it is not difficult to check if it is available to make all his understanding levels X or higher.
So, let's perform the check for all the combination of books and find the minimum sum of money among the satisfying combinations.
The number of combinations of books is 2^N, since you have choices of whether buying or not buying for each book. Each judgement can be performed in a total of $O(NM)$ time, so the total time complexity is $O(2^N NM)$, which is fast enough for this problem.
For the implementation of the iteration of combinations of books to buy, one can use "Depth-First Search" or encode sets to integers (in other words, list the subsets by bitmasking).

The following is a sample code of subset listing in C++. https://atcoder.jp/contests/abc167/submissions/13086283

D: Teleporter

If you naively simulate the movement starting from town 1, you will come back to the town you have already visited before using the teleporter more than N times. Let us call the town the "fragrant town." Then, starting from the fragrant town, how many times the teleporter is needed to be used in order to go back to the fragrant town, naively again. Lastly, the remainder of the remaining number of moves divided by it is less than N, so it can be naively simulated.

E: Colorful Blocks

Consider the ways of painting where "there are exactly k pairs of adjacent blocks that are painted in the same color."

The necessary and sufficient condition is that, out of $N-1$ blocks except for the leftmost block, k blocks are painted in the same color of the blocks in their left, and $N-1-k$ blocks are painted in the different color of the blocks in their left.

Such ways of painting is, considering the choice of the color of the leftmost block and the choice of aforementioned k blocks as well, represented by following equation:

$$M \times {}_{N-1}\mathrm{C}_k \times 1^k \times (M-1)^{N-1-k}$$

By accumulating them for $k = 0, 1, \cdots, K$, the answer can be calculated.

F: Bracket Sequencing

Summary

If a string contains a consecutive pair of letters '(' and ')', then removing those two letters does not affect the result. By repeating this operation, each S_i becomes "a (possibly empty) string consisting of 0 or more repetition of ')', followed by 0 or more repetition of '('." Then let A_i be the number of ')', and B_i be the number of '('.

The answer is Yes if the concatenation of S_i in the following order forms a bracket sequence; otherwise the answer is No.

The former part: S_i such that $B_i - A_i \geq 0$, in the increasing order of A_i

The latter part: S_i such that $B_i - A_i \leq 0$, in the decreasing order of B_i

It can be justified mainly by the fact that "if adjacent two elements are not in that order, then swapping them does not make the situation worse."

Intuition

A bracket sequence is "a string such that every bracket forms proper pairs." A string is not a bracket sequence if and only if "despite all the brackets so far are already paired, another ')' has appeared" at some point in the string. So, it seems to be better to use strings that increase the number of '('. Since a string in which the number of ')' exceeds the deposit of '('s so far cannot be connected, it seems to be better to use in the increasing order of A_i.

Also, if a string is a bracket sequence, then the reversed string, '(' replaced with ')' and ')' replaced with '(', is also a bracket sequence. Therefore, we can obtain the conditions for the latter part akin to the former part.

Implementation

A_i, B_i can be obtained in a linear time as follows, without actually removing the brackets:

$$A_i = \text{maximum of (the number of ')') } - \text{ (the number of '(') for all prefixes of } S_i$$

$$B_i = \text{minimum of (the number of '(') } - \text{ (the number of ')') for all suffixes of } S_i$$

ABC 168 解説

sheyasutaka

2020 年 5 月 17 日

For International Readers: English editorial starts on page 8.

A: ∴ (Therefore)

1 の位ごとに場合分けをしても解けますが，より簡潔に書けないでしょうか？

C/C++ や C#, Java などには switch 文というものがあり，これを使うことで問題文通りの記述を直感的に書けます (C での実装例を以下に示します).

余談ですが，Ruby では case 文がこれにあたり，もっと高機能です.

Listing 1　C での実装例

```c
#include <stdio.h>

int main(void) {
        int n;
        scanf("%d", &n);

        switch (n % 10) {
        case 2:
        case 4:
        case 5:
        case 7:
        case 9:
                puts("hon");
                break;
        case 0:
        case 1:
        case 6:
        case 8:
                puts("pon");
                break;
        case 3:
```

```
22                  puts("bon");
23                  break;
24          }
25
26          return 0;
27  }
```

B: ... (Triples Dots)

長さが K 以下かどうか判定し，上回っていれば問題文通りの文字列操作を行ってから出力すればいいです．

PHP には truncate というそのものずばりな機能があり，これを使うのも手です．

Listing 2　C++ での実装例

```
#include <iostream>
using std::cin;
using std::cout;

#include <string>
using std::string;

int main(void) {
        int k;
        string s;

        cin >> k;
        cin >> s;

        if (s.size() > k) {
                s = s.substr(0, k) + "...";
        }

        cout << s << '\n';

        return 0;
}
```

Listing 3　Python3 での実装例

```
k = int(input())
s = input()

if len(s) > k:
        s = s[:k] + "..."

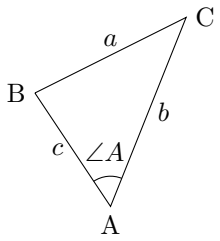
print(s)
```

C: : (Colon)

2 端点の座標を求めてからその距離を計算するのは面倒です．これを使ってみましょう:

余弦定理

三角形 ABC について, $a^2 = b^2 + c^2 - 2bc\cos\angle A$ が成り立つ．ここで, a, b, c は $|BC|, |CA|, |AB|$ のことを指す.



時針・分針の長さを b, c, 時針・分針の間の角度を $\angle A$ として上の式に代入すれば，求める長さ (の 2 乗) が求まります．角度については，「時針が $H + \frac{M}{60}$ 時間で動いた角度」から「分針が M 分で動いた角度」を引けばよいです (C++ 標準の cos 関数の場合，mod をとる必要はありません).

Listing 4　C での実装例

```
#include <stdio.h>
#include <math.h>

#define PI 3.14159265358979323846264338327950L

int main(void) {
        int a, b, h, m;
        scanf("%d%d%d%d", &a, &b, &h, &m);

        long double rad = PI * 2 * (
                (long double)h / 12.0 + ((long double)m / 60.0) / 12.0 - (long
                     double)m / 60.0);

        long double rsq = (long double)(a * a + b * b) - (long double)(2 * a *
             b) * cosl(rad);

        printf("%20.20Lf\n", sqrtl(rsq));

        return 0;
}
```

D: .. (Double Dots)

ある部屋から部屋 1 に移動するために廊下を通る最小の回数のことを，その部屋の「深さ」と呼ぶことにします．特に，部屋 1 の深さだけが 0 です．

すると，以下の重要な事実が成り立ちます．

＜！＞＜！＞＜！＞＜！＞＜！＞＜！＞

深さ $d+1$ の部屋には，深さ d の部屋が少なくとも 1 つ繋がっている．

証明は簡単です (そうでない部屋があると，直感的に嫌な気分になります)．このことから，深さ $d+1$ の部屋の道しるべが深さ d の部屋を指すようにすれば，常に目標を達成できることがわかります．

実装の際には，幅優先探索で各部屋の深さを求めてから，辺を 1 つずつ見ていって随時更新すれば楽です．

E: · (Bullet)

$(A_i, B_i) = (0, 0)$ のイワシは他のどの個体とも仲が悪いので，「$(A_i, B_i) = (0, 0)$ なイワシをどれか 1 匹だけ選ぶ」「$(A_i, B_i) = (0, 0)$ なイワシをどれも選ばない」のいずれかです．勿論，前者の選び方はそのようなイワシの個数に等しいので，以下では $(A_i, B_i) = (0, 0)$ なイワシを除外して考えます．

イワシの「傾き」を $\frac{A_i}{B_i}$ として，これを既約分数で表すことを考えます．具体的には，次のように決めます．

- $B_i = 0$ のとき，傾きは $1/0$.
- $B_i > 0$ のとき，傾きは $\frac{A_i}{\gcd(A_i,B_i)} / \frac{B_i}{\gcd(A_i,B_i)}$.
- $B_i < 0$ のとき，傾きは $-\frac{A_i}{\gcd(A_i,B_i)} / -\frac{B_i}{\gcd(A_i,B_i)}$.

ちょっとした場合分けから，傾きと仲の悪さの関係について次のことが言えます．

- 傾き $1/0$ のイワシと仲が悪いのは，傾き $0/1$ のイワシ全てのみ．逆もまた然り．
- 傾き a/b $(a, b \neq 0)$ のイワシと仲が悪いのは，傾き $-b/a$ (を a の符号で通分したもの) のイワシ全てのみ．逆もまた然り．

たとえば以下の傾きが，互いに ”仲の悪いつがい” です．

$$
\begin{aligned}
1/0 &\longleftrightarrow 0/1 \\
5/3 &\longleftrightarrow -3/5 \\
-2/1 &\longleftrightarrow 1/2
\end{aligned}
$$

したがって，”仲の悪いつがい” になる傾きのペア (高々 N 通り) 全てについて，連想配列などを使って各傾きになるイワシの個数が求まれば，その後は基礎的な数え上げの範疇です．オーバーフローには気を付けてください．

A_i, B_i の制約が大きいので，傾きを有理数の代わりに実数で表現しようとすると (long double でも) 精度が足りないおそれがあります．

F: . (Single Dot)

x 座標・y 座標それぞれを重複を除いてソートし，十分なサイズの 2 次元グリッド上に各線分を刻み込んでから BFS すれば，$\mathcal{O}(NM)$ 時間となって十分間に合います．

A: ∴ (Therefore)

It can be solved by dividing cases depending on the 1's digit, but aren't there any better ways?

Languages like C/C++, C# and Java have switch switch statements, and by using them the description in the problem statement can be expressed intuitively (sample code in C is shown below).

By the way, Ruby has case statements, which is more functional.

Listing 5 Sample Code in C

```c
#include <stdio.h>

int main(void) {
        int n;
        scanf("%d", &n);

        switch (n % 10) {
        case 2:
        case 4:
        case 5:
        case 7:
        case 9:
                puts("hon");
                break;
        case 0:
        case 1:
        case 6:
        case 8:
                puts("pon");
                break;
        case 3:
                puts("bon");
                break;
        }

        return 0;
}
```

B: ... (Triples Dots)

Check if the length is more than K letters, perform the operations described in the problem statement if needed, and print it.

PHP has a straightforward function called truncate, so that will be an option as well.

Listing 6 Sample Code in C++

```
#include <iostream>
using std::cin;
using std::cout;

#include <string>
using std::string;

int main(void) {
        int k;
        string s;

        cin >> k;
        cin >> s;

        if (s.size() > k) {
                s = s.substr(0, k) + "...";
        }

        cout << s << '\n';

        return 0;
}
```

Listing 7 Sample Code in Python3

```
k = int(input())
s = input()

if len(s) > k:
        s = s[:k] + "..."

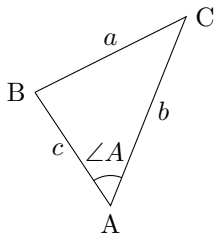
print(s)
```

C: : (Colon)

Computing the coordinates of two endpoints and then calculating the distance between them is troublesome. Let's use the following:

Laws of Cosines

For any triangle ABC, it holds that $a^2 = b^2 + c^2 - 2bc\cos\angle A$. Here, a, b and c denotes $|BC|, |CA|$ and $|AB|$, respectively.



By assigning the length of the hour and minute hands to b and c, and the angle between the hour and minute hands to $\angle A$ in the formula above, the (squared) desired length can be calculated. The angle can be obtained by subtracting "the amount of angle the minute hand moved in M minutes" from "the amount of angle the hour hand moved in $H + \frac{M}{60}$ hours" (in C++, you don't have to take mod for the cos function).

Listing 8 Sample Code in C

```
#include <stdio.h>
#include <math.h>

#define PI 3.14159265358979323846264338327950L

int main(void) {
        int a, b, h, m;
        scanf("%d%d%d%d", &a, &b, &h, &m);

        long double rad = PI * 2 * (
                (long double)h / 12.0 + ((long double)m / 60.0) / 12.0 - (long
                     double)m / 60.0);

        long double rsq = (long double)(a * a + b * b) - (long double)(2 * a *
             b) * cosl(rad);

        printf("%20.20Lf\n", sqrtl(rsq));

```

```
17          return 0;
18  }
```

D: .. (Double Dots)

We define the "depth" of room by the minimum number of passages from the room to the room 1. In particular, only the room 1 is of depth 0.

Then, the following important fact holds:

<! >< ! >< ! >< ! >< ! >< ! >

For any room of depth $d+1$, at least one room of depth d is connected to it.

The proof is easy (if there is no such room, it is intuitively unpleasant). Therefore, if each the signpost of depth $d+1$ indicates a room of depth d, then the purpose can be always achieved.

When implementing, you can compute the depths of the rooms first, and then look at each edge and update it one by one.

E: · (Bullet)

Since a sardine of $(A_i, B_i) = (0, 0)$ is on bad terms with every other individuals, so either "choose one sardine such that $(A_i, B_i) = (0, 0)$" or "never choose sardines such that $(A_i, B_i) = (0, 0)$" is possible. Of course, the number of the former ways of choosing is equal to the number of such sardines, in the further discussion the sardines such that $(A_i, B_i) = (0, 0)$ will be excluded.

Define the "slope" of a sardine by $\frac{A_i}{B_i}$ and consider representing them by irreducible fractions. Specifically, we will define them by:

- When $B_i = 0$, the slope is $1/0$.
- When $B_i > 0$, the slope is $\frac{A_i}{\gcd(A_i,B_i)} / \frac{B_i}{\gcd(A_i,B_i)}$.
- When $B_i < 0$, the slope is $-\frac{A_i}{\gcd(A_i,B_i)} / -\frac{B_i}{\gcd(A_i,B_i)}$.

By a little case division, the following dependency of the relationships on the slope can be found:

- The sardines of slope $1/0$ is on bad terms with, and only with, all the sardines of slope $0/1$, and vice versa.
- The sardines of slope a/b $(a, b \neq 0)$ is on bad terms with, and only with, all the sardines of slope $-b/a$ (normalized by the sign of a), and vice versa.

For example, the following slopes are "the pairs on bad terms with each other."

$$\begin{aligned} 1/0 &\longleftrightarrow 0/1 \\ 5/3 &\longleftrightarrow -3/5 \\ -2/1 &\longleftrightarrow 1/2 \end{aligned}$$

Therefore, for all pairs of slopes on bad terms with each other, count the number of sardines of such slopes using data structures such as associative arrays, and all the left is a basic combinatorics. Be careful of overflows.

Since the constraints of A_i, B_i is large, representing the slope with real numbers instead of rationals may not hold enough precision.

F: . (Single Dot)

Sort the x and y coordinates without duplicates, inscribe each segment into a two-dimensional grid of sufficient size and perform the BFS, then it will be in a total of $\mathcal{O}(NM)$ time, which is fast enough.

ABC 169 解説

kyopro_friends, latte0119, tempura0224, ynymxiaolongbao, yuma000

2020 年 5 月 31 日

For International Readers: English editorial starts on page 8.

A: Multiplication 1

入力を読み込み、計算結果を出力します。

C 言語での実装例

```c
#include<stdio.h>
int main(){
    int a,b;
    scanf("%d%d",&a,&b);
    printf("%d\n",a*b);
}
```

python の実装例

```python
a,b = map(int,input().split())
print(a*b)
```

B: Multiplication 2

A_i に 0 が含まれる場合、答えは 0 です。そうでない場合を考えます。

64bit 整数でそのまま掛け算を行うとオーバーフローします。倍精度浮動小数点数で計算を行うと精度が足りません。多倍長整数で最後まで計算すると TLE します。したがって何らかの工夫が必要です。

Python などの多倍長整数が使える言語や、128bit 整数が使える環境であれば「積が 10^{18} が超えた時点で -1 を出力する」とすることで解くことができます。

Python での実装例

```python
def main():
    N = int(input())
    A = list(map(int,input().split()))

    if 0 in A:
        print(0)
        return

    prod = 1
    for a in A:
        prod *= a
        if prod > 1000000000000000000:
            print(-1)
            return

    print(prod)

main()
```

そうでない場合も、10^{18} を超えるかどうかの判定を乗算ではなく除算で行う必要がありますが、同様の方針で解くことができます。

C 言語での実装例

```c
#include<stdio.h>
long long a[100010];
int main(){
    int n;
    scanf("%d",&n);
    for(int i=0;i<n;i++)scanf("%lld",&a[i]);

    int zero=0;
```

```
    for(int i=0;i<n;i++)if(a[i]==0)zero++;
    if(zero>0){
        // a[i] contains 0
        printf("0\n");
        return 0;
    }

    long long prod=1;
    for(int i=0;i<n;i++){
        if(a[i]<=1000000000000000000/prod){
            // This condition is equivalent to prod*a[i]<=10^18
            prod*=a[i];
        }else{
            printf("-1\n");
            return 0;
        }
    }
    printf("%lld\n",prod);
}
```

C: Multiplication 3

浮動小数点数として掛け算を行うと精度が足りません。B を 100 倍して、 $(A \times (B \times 100))/100$ とすることで、整数の範囲で誤差なく計算することができます。B を 100 倍して整数に変換する際にも誤差に注意してください。

D: Div Game

$N = p_1^{e_1} \times ... \times p_k^{e_k}$ と素因数分解されるとします。このとき、各 p_i に対して、 $z = p_i^1, p_i^2, ...$ と順番に選んでいくのが最適です。したがって、 N を素因数分解することができれば十分です。これは $O(\sqrt{N})$ で実装することができ、十分高速です。

E. Count Median

N が奇数のとき A の中央値以上 B の中央値以下のすべての整数、N が偶数のとき A の中央値以上 B の中央値以下のすべての $\frac{1}{2}$ の倍数が X の中央値になり得ます。X の値のうちひとつを 1 増やしたときの中央値の変化は、N が奇数なら 0 か 1 、偶数なら 0 か $\frac{1}{2}$ であり、X の値のうちひとつを 1 増やす操作の繰り返しで X の中央値を A から B へ変化させることができることを踏まえると、その経路上に先に述べた実数のすべてが中央値として現れることがわかります。それ以外の実数は中央値として現れないため、これで十分です。

F: Knapsack for All Subsets

問題は以下のように言い換えることができます。

> $\{1, 2, \ldots, N\}$ の空でない部分集合 (T, U) の組であって以下の条件を満たすものは何通りありますか？
>
> 1. U は T の部分集合である
> 2. $U = \{x_1, x_2, \ldots, x_k\}$ とすると $a_{x_1} + a_{x_2} + \cdots + a_{x_k} = S$

このとき、各 i $(1 \leq i \leq N)$ について、

1. U にも T にも入れる
2. T には入れるが U には入れない
3. U にも T にも入れない

の 3 つの選択肢があり、1 つめの選択肢を選んだときのみ和が a_i だけ増えます。
以上の考察により、

$dp[i][j] =$ i 番目までの選択をして、1 つめの選択肢を選んだ k に対する a_k の和が j であるような場合の数

と定め、動的計画法を用いるとよいということがわかります。
求めたい答えは $dp[N][S]$ であり、計算量は $O(NS)$ です。
実装例: https://atcoder.jp/contests/abc169/submissions/13772481

A: Multiplication 1

Read the input and output the calculation result.

Sample code in C

```c
#include<stdio.h>
int main(){
    int a,b;
    scanf("%d%d",&a,&b);
    printf("%d\n",a*b);
}
```

Sample code in python

```python
a,b = map(int,input().split())
print(a*b)
```

B: Multiplication 2

If A_i contains 0, then the answer is 0. We will consider otherwise.

If you directly perform multiplication on 64-bit integers, it causes overflow. Double-sized floating point numbers do not have enough precisions. Multiplying entirely using multiple-precision integers leads to TLE. Therefore, we need to devise some good way.

In the programming language where multiple-precision integers like Python, or in the environment where 128-bit integers are available, it can be solved by "outputting -1 immediately after the product exceeded 10^{18}."

Sample code in Python

```python
def main():
    N = int(input())
    A = list(map(int,input().split()))

    if 0 in A:
        print(0)
        return

    prod = 1
    for a in A:
        prod *= a
        if prod > 1000000000000000000:
            print(-1)
            return

    print(prod)

main()
```

Otherwise, you can still solve it in a similar way, although you have to check if it exceed 10^{18} by performing division instead of multiplication.

Sample code in C

```c
#include<stdio.h>
long long a[100010];
int main(){
    int n;
    scanf("%d",&n);
    for(int i=0;i<n;i++)scanf("%lld",&a[i]);

    int zero=0;
```

```
    for(int i=0;i<n;i++)if(a[i]==0)zero++;
    if(zero>0){
        // a[i] contains 0
        printf("0\n");
        return 0;
    }

    long long prod=1;
    for(int i=0;i<n;i++){
        if(a[i]<=1000000000000000000/prod){
            // This condition is equivalent to prod*a[i]<=10^18
            prod*=a[i];
        }else{
            printf("-1\n");
            return 0;
        }
    }
    printf("%lld\n",prod);
}
```

C: Multiplication 3

Multiplying as floating point numbers does not provide sufficient precision. By multiplying B by 100 and calculating $(A \times (B \times 100))/100$, it can be calculated within the range of integer without errors. Note that you have to be careful also when converting B to an integer by multiplying by 100.

D: Div Game

Assume that it can be factorized as $N = p_1^{e_1} \times \ldots \times p_k^{e_k}$. Then, for each p_i, it is optimal to choose in the order of $p_i^1, p_i^2, \ldots$. Therefore, it is sufficient to factorize N. This can be implemented in a total of $O(\sqrt{N})$ time, which is fast enough.

E. Count Median

If N is odd, then the median of X can be every integer between the median of A and the median of B, and if N is even, then the median of X can be every multiple of $\frac{1}{2}$ between the median of A and the median of B, inclusive. The variation of the median of X when one of its elements is increased by one is either 0 or 1 when N is odd, and either 0 or $\frac{1}{2}$ when N is even. Considered that the median of X can be increased from A to B by repeating the operation of increasing one of the elements of X by 1, it can be seen that all of the real numbers mentioned above appears on that path as the median. The other real number cannot appear as the median, so it is sufficient.

F: Knapsack for All Subsets

The problem can be paraphrased as follows:

How many pairs of sets (T, U) such that both are non-empty subsets of $\{1, 2, \ldots, N\}$ and also satisfies the conditions below?

1. U is a subset of T
2. Let $U = \{x_1, x_2, \ldots, x_k\}$, then $a_{x_1} + a_{x_2} + \cdots + a_{x_k} = S$

Then, for each $i\,(1 \leq i \leq n)$, there are three options:

1. put to both U and T
2. put to T, but do not put to U
3. do not put to both U and T

and the sum increases by a_i only if the first option was chosen.

By the observation above, it can be seen that performing Dynamic Programming with the following recurrence relations will do:

$dp[i][j]$ = When the choices for the first i options is already determined, the number of combination such that the sum of a_k for each k that the first option was chosen is equal to j

The desired answer is $dp[N][S]$, and the total time complexity is $O(NS)$.

Sample Code: https://atcoder.jp/contests/abc169/submissions/13772481

ABC 170 解説

evima, gazelle, kort0n, tozangezan, ynymxiaolongbao, yuma000

2020 年 6 月 14 日

For International Readers: English editorial starts on page 7.

A: Five Variables

$x_1, \ldots, x_5$ を標準入力から読み込んだ後は、if 文を 5 つ連ねて 5 つの変数のうちどれが 0 であるかを直接確かめることも、$15 - x_1 - x_2 - x_3 - x_4 - x_5$ が答えとなることを利用することもできます。

それぞれの方針における Python での実装例を示します。

```
X = list(map(int, input().split()))

if X[0] == 0:
    print(1)
elif X[1] == 0:
    print(2)
elif X[2] == 0:
    print(3)
elif X[3] == 0:
    print(4)
else:
    print(5)
```

```
print(15 - sum(list(map(int, input().split()))))
```

B: Crain and Turtle

(原案: yuma000, 準備・解説: evima)

方針 1

発言の前半「庭の動物の総数は X 匹」が正しいと仮定すると、考えられる鶴と亀の数の組合せは「鶴 0 匹、亀 X 匹」「鶴 1 匹、亀 $X-1$ 匹」...「鶴 X 匹、亀 0 匹」の $X+1$ 通りです。これらのそれぞれにおける足の総数を計算し、その結果が一度でも Y 本となれば答えは Yes、一度も Y 本とならなければ答えは No です。素直に実装するなら、以下の Python のコードのように for 文を使うことになるでしょう。

```
X, Y = map(int, input().split())
ans = 'No'
for a in range(X + 1):
    b = X - a
    if 2 * a + 4 * b == Y:
        ans = 'Yes'
print(ans)
```

方針 2

答えが Yes であることの必要十分条件は、Y が $2X$ 以上 $4X$ 以下の偶数であることです。

必要性の証明: a 匹の鶴と b 匹の亀について発言が正しいとすると、$Y = 2a + 4b = 2(a + 2b)$ であり、a, b が整数であることから Y は偶数でなければなりません。また、$a + b = X, 2a + 4b = Y$ から $a = (4X - Y)/2, b = (Y - 2X)/2$ が得られ、a, b が非負であることから Y は $2X$ 以上 $4X$ 以下でなければなりません。よって示されました。

十分性の証明: Y が $2X$ 以上 $4X$ 以下の偶数であれば、$(4X - Y)/2, (Y - 2X)/2$ はともに非負整数となり、発言が正しいような鶴と亀の数の組合せとして「鶴 $(4X - Y)/2$ 匹、亀 $(Y - 2X)/2$ 匹」が存在します。よって示されました。

C: Forbidden List

(原案: tozangezan, 準備・解説: evima)

$X, p_1, \ldots, p_N$ はいずれも 1 以上 100 以下ですが、答えは 0 または 101 となりうる点に注意を要します。

この点にさえ注意すれば、与えられる値の小ささのため計算を高速化する工夫などは必要なく、「書かれていることを実装しなさい」という通常の Beginner Contest 問題 B 相当の問題です。本問が問題 B として出題されていたなら以下に具体的な実装方法を述べるところですが、問題 C を解かれる方にそのような説明は不要と考え、省略します。

追記 (6/18): 以下に Python での実装例を示します。

```
X, N = map(int, input().split())
p = list(map(int, input().split()))
for d in range(X + 1):
    for s in [-1, +1]:
        a = X + s * d
        if p.count(a) == 0:
            print(a)
            exit(0)
```

答えの候補を $X, X-1, X+1, X-2, X+2, \ldots$ の順に調べています。`count` を何度も呼び出している部分に高速化の余地がありますが、この問題の制約下ではその必要はありません。

D: Not Divisible

エラトステネスの篩と同様の処理を, 数列 A の要素についてのみ行います.
サイズ A_{max} の bool 配列 dp を用意し, true で初期化します.$dp[i]$ =true のとき, i より小さい i の約数が A に存在しないことを表します. 数列の要素を昇順に見ます.x が数列 A に含まれているとき, x より大きい x の倍数 y について, $dp[y]$ を false に変更します.
同じ値について処理を複数回行わないように実装すると, 調和級数を用いた解析により, 時間計算量が $O(A_{max} \log A_{max} + N \log N)$ であることが分かります.
同じ値が複数存在する場合に, それらを答えに含めないことに注意してください.

E. Smart Infants

ある幼稚園の中で最もレートの高い幼児を、その幼稚園の最強園児と呼ぶことにします。
以下のようなデータを管理することで時系列順に答えを求めてゆくことができます。

- それぞれの園児が所属する幼稚園の番号
- それぞれの幼稚園に所属する幼児のレートの順序付き多重集合（幼稚園一つに対して一つ）
- それぞれの幼稚園の最強園児のレートの順序付き多重集合（全体に対して一つ）

順序付き多重集合は C++ の multiset などを用いることで高速に処理することができます。
それぞれのクエリでは、以下のような変更操作を行います。

- 最強園児のレートの集合から、転園する園児の元の幼稚園の元の最強園児のレートを、削除する
- 最強園児のレートの集合に、転園する園児の元の幼稚園の新しい最強園児のレートを、挿入する（園児が一人もいない場合何もしない）
- 最強園児のレートの集合から、転園する園児の新しい幼稚園の元の最強園児のレートを、削除する（園児が一人もいない場合何もしない）
- 最強園児のレートの集合に、転園する園児の新しい幼稚園の新しい最強園児のレートを、挿入する
- 転園する園児の元の幼稚園のレートの集合から、転園する園児のレートを、削除する
- 転園する園児の新しい幼稚園のレートの集合に、転園する園児のレートを、挿入する
- 転園する園児の所属する幼稚園の番号を更新する

以上の変更操作の後、最強園児のレートの集合の中の最も小さい値を出力すれば良いです。計算量は $O(NlogN)$ です。

F: Pond Skater

Dijkstra 法を用います。以下のようなグラフを考えます。
次のような状態量をグラフの頂点と見做します。

- 現在いるマス
- すぬけ君が東西南北のどの方向を向いているか

次のように遷移します。

- 向いている方向に 1 つ進んだマスに蓮がないなら、「向いている方向に 1 進んだマスにいて、向きはそのまま」という状態量に、今の状態量までの最短コストに $1/K$ を足したコストで到達可能とする。
- 「同じマスにいて、向きが 90 度回転した」という状態量に、今の状態量までの最短コストの小数点以下を切り上げたコストで到達可能とする。

計算量は $O(HWlog(HW))$ です。

A: Five Variables

After reading $x_1, \ldots, x_5$ from the standard input, you can write if statements 5 times and check which of 5 variables is 0, or use the fact that $15 - x_1 - x_2 - x_3 - x_4 - x_5$ is the answer.

The following are the sample code in Python for each way.

```
X = list(map(int, input().split()))

if X[0] == 0:
    print(1)
elif X[1] == 0:
    print(2)
elif X[2] == 0:
    print(3)
elif X[3] == 0:
    print(4)
else:
    print(5)
```

```
print(15 - sum(list(map(int, input().split()))))
```

B: Crain and Turtle

(Original writer: yuma000, preparation and editorials: evima)

Course 1

Assuming that the first half description that "there are X animals in total" is true, then there are $X+1$ possible combinations of the numbers of cranes and turtles: "0 cranes and X turtles", "1 crane and $X-1$ turtles", ..., and "X cranes and 0 turtles". Calculate the total number of legs for each situation, and if at least one of the sum is equal to Y, then the answer is Yes, otherwise the answer is No. Naively, for statement can be used in the implementation, like the following Python code.

```python
X, Y = map(int, input().split())
ans = 'No'
for a in range(X + 1):
    b = X - a
    if 2 * a + 4 * b == Y:
        ans = 'Yes'
print(ans)
```

Course 2

The necessary and sufficient conditions for the answer to be Yes is that Y is an even number between $2X$ and $4X$, inclusive.

Proof of necessity: if the statement is true for a cranes and b turtles, then $Y = 2a + 4b = 2(a + 2b)$, and since a, b is an integer, Y must be an even number. Also, since $a + b = X, 2a + 4b = Y$, we obtain $a = (4X - Y)/2, b = (Y - 2X)/2$, and since a, b are non-negative, Y should be between $2X$ and $4X$, inclusive. Here the proof ends.

Proof of sufficiency: if Y is an even number between $2X$ and $4X$, inclusive, then $(4X - Y)/2, (Y - 2X)/2$ are both a non-negative integer, so there exists a combination of numbers of cranes and turtles: "$(4X - Y)/2$ cranes and $(Y - 2X)/2$ turtles." Here the proof ends.

C: Forbidden List

(Original writer: tozangezan, preparation and editorials: evima)

Note that although $X, p_1, \ldots, p_n$ are all between 1 in 100, the answer may be 0 or 101¿

If you just take care of that point, you don't have to do any optimization because of the small constraints, so it's just as easy as Beginner Contest's Problem B, in which you are only asked to "implement just as described." If this problem was set in Problem B, we would describe the specific way to implement it, but we will omit it since we think such description is unnecessary for those who tries solving Problem C.

P.S. (June 18): Sample implementation in Python follows:

```
X, N = map(int, input().split())
p = list(map(int, input().split()))
for d in range(X + 1):
    for s in [-1, +1]:
        a = X + s * d
        if p.count(a) == 0:
            print(a)
            exit(0)
```

It checks candidates for the answer in the order $X, X-1, X+1, X-2, X+2, \ldots$ We can optimize the part where it calls `count` many times, but we have no need to do so under the constraints of this problem.

D: Not Divisible

Perform the same operations to Sieve of Eratosthenes only for the elements of the sequence A.
Prepare a bool array dp of size A_{max}, and initialize them with true. $dp[i]$ =true means that A contains none of the divisors of i less than i. Inspect each element in the increasing order. When x is contained in A, for each multiple y of x larger than x, update $dp[y]$ to false.
If you avoid performing operations multiple times for the same value, the total time complexity appears to be $O(A_{max} \log A_{max} + N \log N)$ by the analysis using harmonic series. Be careful not to count the values that appears multiple times.

E. Smart Infants

We will refer to the infant with the highest rating in a kindergarten as the “strongest infant” in the kindergarten.

By managing the following data, you can find the answer in the chronological order.

- The index of Kindergarten to which each infant belongs
- Ordered multiset of ratings of infants who belong to each kindergarten (one multiset for one kindergarten each)
- A multiset of ratings of the strongest infants of all the kindergartens (one common multiset for all of them)

Ordered multisets can be processed fast by using, for instance, multiset in C++.

For each query, perform the following update operations.

- Erase the rating of the former strongest infant in the former kindergarten of the infant who will transfer from the set of ratings of the strongest infants
- Insert the rating of the new strongest infant in the former kindergarten of the infant who will transfer to the set of ratings of the strongest infants (or do nothing if no infants are there)
- Erase the rating of the former strongest infant in the new kindergarten of the infant who will transfer from the set of ratings of the strongest infants (or do nothing if no infants are there)
- Insert the rating of the new strongest infant in the new kindergarten of the infant who will transfer to the set of the ratings of the strongest infants
- Erase the rating of the infant who will transfer from the set of ratings of the former kindergarten
- Insert the rating of the infant who will transfer to the set of ratings of the new kindergarten
- Update the index of kindergarten that the infant who will transfer belongs to

After the update operations above, print the lowest rating of the set of ratings of the strongest infants. The total time complexity is $O(NlogN)$.

F: Pond Skater

We will use Dijkstra's algorithm. Consider the following graph.
We will regard the states determined by the following as vertices.

- The current square
- The direction in which Snuke is facing

It will be updated as follows.

- If there is no leaf in the square one unit ahead, then it can advance to the state where "the current square is the square one unit ahead and the direction remains the same" by consuming $1/K$ cost.
- It can advance to the state where "the current square remains the same and the direction rotates by 90 degrees" by updating the cost to the current cost, rounded up.

The total time complexity is $O(HWlog(HW))$.

ABC 171 解説

tozangezan, kyopro_friends, sheyasutaka, gazelle, evima, Kmcode

2020 年 6 月 21 日

For International Readers: English editorial starts on page 10.

A: αlphabet

(解説: evima)

プログラミングの学習を始めたばかりで何から手をつけるべきかわからない方は、まずは「practice contest」(`https://atcoder.jp/contests/practice/`) の問題 A「はじめてのあっとこーだー」をお試しください。言語ごとに解答例が掲載されています。

今回の問題を解くためには、以下の 4 つの要素が必要です。

1. 標準入力からの文字の読み込み
2. 文字 α が大文字か小文字かの判定
3. 上記の判定結果に基づく処理の分岐
4. 標準出力への文字の出力

このうち要素 1, 4 は「はじめての〜」でカバーされたとして (その問題では正確には文字ではなく文字列の入出力が扱われていますが、文字 1 個も長さ 1 の文字列と考えられます)、残りの要素 2, 3 について述べます。

2. 現代の多くの言語には、この判定を行う機能が `isUpperCase`, `islower` などといった名前であらかじめ実装されています。検索エンジンで「[言語名] 大文字 小文字 判定」などと検索することでそのような機能、またはその代わりとなる手法を発見できるはずです。(AtCoder では数十の言語が利用可能であり、ここでそれらについて網羅的に述べるにはこの PDF は狭すぎます。)

3. ほとんどの言語で「if 文」または類似の構文が利用可能であり、また多くの言語に「条件演算子」または類似の要素が存在します。上記と同様に、「[言語名] if」「[言語名] 条件」などと検索することで、その言語で利用可能な構文などの情報が手に入るはずです。

Python での実装例:

```
a = input()
print('A' if a.islower() else 'a')
```

Ruby での実装例 (2. で述べた `isUpperCase` のような標準機能が存在しない言語の例として):

```
a = gets.chomp
if a == a.upcase
    puts 'A'
else
    puts 'a'
end
```

B: Mix Juice

(解説: evima)

まず、入力されるデータの量が文字 1 個で固定であった問題 A と異なり、今回は価格が入力される果物の数 N が 1 以上 1000 以下と一定ではありません。これに対応するための一般的なキーワードは「for 文」「配列」であり、検索エンジンで「[言語名] for」などと検索することで言語ごとの情報が得られるはずです。ただし、言語によってはこれらのキーワードからややかけ離れた構文や要素も有用であることがあります。

さて、この問題で N 種類の果物から購入する K 種類を選ぶ際は、もちろん価格が低い方から K 種類を選ぶべきです。すなわち、N 個の整数 $p_1, \ldots, p_N$ のうち最も小さいもの、2 番目に小さいもの、...、K 番目に小さいものの和がこの問題の答えで、これをプログラムにどのように求めさせるかが課題となります。結論としては、多くの言語に「ソート」、すなわち複数の値を小さい順に並べ替える機能があらかじめ実装されており、これを用いて N 個の整数を小さい順に並べ替えてから先頭の K 個を選択するのが最も簡単です。やはり「[言語名] ソート」などで情報が手に入ります。

C++ での実装例:

```cpp
#include <algorithm>    // sort
#include <iostream>
using namespace std;

int N, K, p[1010];

int main(){
    cin >> N >> K;
    for(int i = 0; i < N; ++i) cin >> p[i];
    sort(p, p + N);
    int ans = 0;
    for(int i = 0; i < K; ++i) ans += p[i];
    cout << ans << endl;
}
```

Python での実装例 (「for 文」からややかけ離れた要素が有用であるような言語の例として):

```
N, K = map(int, input().split())
p = list(map(int, input().split()))
print(sum(sorted(p)[:K]))
```

C: One Quadrillion and One Dalmatians

(解説: evima)

まず、10^{15} という数は現代の我々の計算機にとっても大きく、何らかの処理をこの程度の回数行って実行時間を 2 秒以内に収めることは不可能です。

素直な方針は、問題文で「(以下省略)」となっている部分を計算し、まず番号 N の犬の名前の長さを求めることでしょう。以下、「番号 N の犬の名前」を「名前 N」と書きます。$i = 1, 2, \ldots$ のそれぞれに対し、長さが i であるような名前は 26^i 個存在します。よって、名前 $1, \ldots, 26$ の長さは 1、名前 $26+1, \ldots, 26+26^2$ の長さは 2、名前 $26+26^2+1, \ldots, 26+26^2+26^3$ の長さは 3、$\ldots$ です。$26+26^2+26^3+\ldots+26^{11} = 3817158266467286 > 10^{15}+1$ より、長さ 11 での計算までに名前 N の長さが判明します。これにより、名前 N の長さが l であると判明したとします。

すると、長さが l であるような 26^l 個の名前の中で、名前 N が辞書順で $N-(26+26^2+\ldots+26^{l-1})$ (この値を k とします) 番目の名前であることもわかります。残る問題は「長さが l であるような文字列の中で、辞書順で k 番目のものを求めよ」というものです。

この問題の解き方がよくわからなければ、アルファベットが $\mathtt{a}, \mathtt{b}, \ldots, \mathtt{j}$ の 10 文字であったとして考えるとわかりやすいでしょう。例えば、この状況では長さ 3 の名前は $\mathtt{aaa}, \mathtt{aab}, \ldots, \mathtt{aaj}, \mathtt{aba}, \ldots, \mathtt{jjj}$ の $10^3 = 1000$ 個存在します。このうち、例えば辞書順で 246 番目のものは $\mathtt{bde}$ です。このように、$k-1$ の十進表記での一の位が辞書順で k 番目の名前の最後の文字に、十の位が名前の最後から 2 番目の文字に、$\ldots$、10^i の位が名前の最後から $i+1$ 番目の文字に対応します ($\mathtt{a}, \mathtt{b}, \ldots, \mathtt{j}$ が $0, 1, \ldots, 9$ に対応しています)。

アルファベットが 26 文字の場合も同様に、$k-1$ の「二十六進表記」における 26^i の位が名前の最後から i 番目の文字に対応します。これらの位の値の求め方の例は次の通りです。(この手法に関しても、動作原理がよくわからなければ十進表記の場合で考えるとわかりやすいでしょう。)

- 変数 x を $k-1$ で初期化し、x が 0 となるまで以下の操作を繰り返す。
- x を 26 で割った商が q、余りが r であるとする。この操作が i 回目に実行されたときの r が $k-1$ の下から i 番目の桁である。x を q で置き換える。

Python での実装例:

```python
N = int(input())
ans = ''
for i in range(1, 99):
    if N <= 26 ** i:
        N -= 1
        for j in range(i):
            ans += chr(ord('a') + N % 26)
            N //= 26
        break
    else:
        N -= 26 ** i
print(ans[::-1])    # reversed
```

(付録) より簡潔な実装例:

```python
N = int(input())
ans = ''
while N > 0:
    N -= 1
    ans += chr(ord('a') + N % 26)
    N //= 26
print(ans[::-1])
```

D: Replacing

(解説: evima)

問題文に書かれている指示にそのまま従うと、最悪の場合にのべ 100 億個の要素を書き換えることになります。この場合、C++ などの高速に動作する言語を使うとして少なくとも 10 秒程度の実行時間が欲しいところですが、この問題の実行制限時間は 2 秒であり、間に合いません。

そこで、この問題で出力することを要求されているものが全要素の和のみであることに着目します。「値が B であるような要素をすべて C に置き換える」という操作 (これを操作 X とします) を行うと、全要素の和は $(C - B) \times$ (操作が行われる直前に値が B であった要素の個数) だけ増加します。よって、数列 A そのものを操作する代わりに、A に値 $1, \ldots, 100000$ が出現する個数 $num_1, \ldots, num_{100000}$ を管理するのが合理的です。操作 X により、全要素の和のみならず各要素の値そのものも変化しますが、この変化も num_B と num_C を操作することにより表現することができます。よって、操作を行い始める前に一度だけ時間をかけて全要素の和と $num_1, \ldots, num_{100000}$ を求めておけば、各操作を定数時間で処理することができます。

E: Red Scarf

(解説: evima)

以下、演算子としての XOR を $\oplus$ と表します。

問題文中の XOR の定義から、3 つ以上の整数の XOR の計算は自由な順序で行えること、例えば任意の 3 つの整数 a, b, c に対して $a \oplus b \oplus c = b \oplus c \oplus a = (a \oplus b) \oplus c = c \oplus (a \oplus b)$ であることがわかります。また、任意の整数 a に対して $a \oplus a = 0$ であることもわかります。以上を組み合わせると、例えば任意の 2 つの整数 a, b に対して $a \oplus b \oplus a = (a \oplus a) \oplus b = 0 \oplus b = b$ であることがいえます。

では、今回の問題の内容に移ります。i 番の猫の整数を b_i とします。このとき、$a_i = b_1 \oplus \ldots \oplus b_{i-1} \oplus b_{i+1} \oplus \ldots \oplus b_N$ です。やや唐突ですが、与えられた N 個の値 $a_1, \ldots, a_N$ の XOR を計算することにします。この値は、上の段落で述べた性質を用いて、次のように整理することができます。

$$
\begin{aligned}
&a_1 \oplus a_2 \oplus \ldots \oplus a_N = (b_2 \oplus \ldots \oplus b_N) \oplus (b_1 \oplus b_3 \oplus \ldots \oplus b_N) \oplus \ldots \oplus (b_1 \oplus \ldots \oplus b_{N-1}) \\
&= (\underbrace{b_1 \oplus \ldots \oplus b_1}_{N-1 \text{ 個}}) \oplus (\underbrace{b_2 \oplus \ldots \oplus b_2}_{N-1 \text{ 個}}) \oplus \ldots \oplus (\underbrace{b_N \oplus \ldots \oplus b_N}_{N-1 \text{ 個}}) \\
&= b_1 \oplus b_2 \oplus \ldots \oplus b_N \ (\because N \text{ は偶数})
\end{aligned}
$$

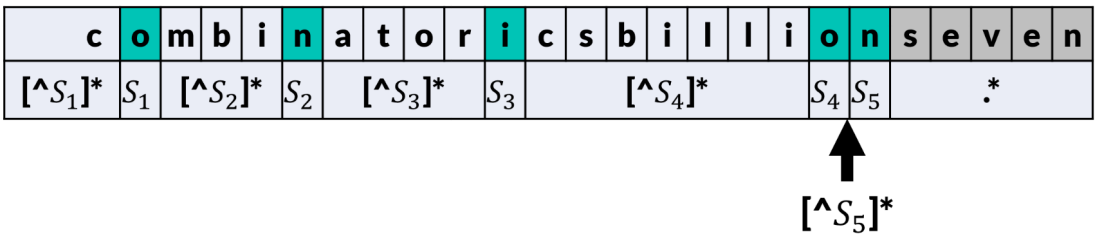
したがって、$a_1 \oplus \ldots \oplus a_N = b_1 \oplus \ldots \oplus b_N$ であることがわかりました。さらに、この値を S とすると、任意の i に対して次が成り立ちます。

$$
\begin{aligned}
&S \oplus a_i = (b_1 \oplus \ldots \oplus b_N) \oplus (b_1 \oplus \ldots \oplus b_{i-1} \oplus b_{i+1} \oplus \ldots \oplus b_N) \\
&= (b_1 \oplus b_1) \oplus \ldots \oplus (b_{i-1} \oplus b_{i-1}) \oplus b_i \oplus (b_{i+1} \oplus b_{i+1}) \oplus \ldots \oplus (b_N \oplus b_N) = b_i
\end{aligned}
$$

よって、任意の i に対して $b_i = S \oplus a_i$ であることがわかり、問題が解けました。問題の制約に「与えられた情報と整合するようなスカーフ上の整数の組合せが存在する」、「出力」セクションに「解が複数存在する場合、どれを出力しても構わない」とありましたが、この解法より、実際には任意の N 個の整数 $a_1, \ldots, a_N$ の組合せに対してそれに整合するようなスカーフ上の整数 $b_1, \ldots, b_N$ の組合せがちょうど一通り存在することがわかります。

F: Strivore

条件を満たす文字列は長さ $|S|+K$ であり，次のような形をしています．



ここで，"[^S_i]*" は S_i でない英小文字からなる長さ 0 以上の文字列を，".*" は任意の英小文字からなる長さ 0 以上の文字列を指します．英小文字の数は 26 であることに注意してください．

$|S|+K$ 文字のうちの $S_1 \sim S_N$ の位置を固定してみたときの，S_i 以外の位置の文字を選ぶ通り数を考えると，文字は互いに影響を及ぼさず独立に選べるので単純に各位置の候補数を掛け合わせればよく，通り数は $25^{K-(S_N\text{以降の文字数})} \times 26^{(S_N\text{以降の文字数})}$ となります．

また，S_N の位置を固定したとき，$S_1 \sim S_{N-1}$ の位置を選ぶ通り数は ${}_{|S|+K-(S_N\text{以降の文字数})-1}C_{N-1}$ となります (「二項係数」で調べるとこの式の意味が分かります)．

したがって，S_N の位置を全て試し，各々における [$S_1 \sim S_{N-1}$ の位置の選び方] × [S_i 以外の文字の選び方] を足し合わせることで，求める値が得られます．

適切に前処理をして，時間・空間ともに $\Theta(|S|+K)$ で解けます．

A: αlphabet

(Editorial: evima)

If you are a beginner of studying programming and have no idea what to start with, then try solving "Welcome to AtCoder", Problem A from the "practice contest" (`https://atcoder.jp/contests/practice/`). There you can find sample codes for each language.

To solve this problem, the following four elements are required.

1. Read a character for standard input
2. Check if character α is uppercase or lowercase
3. Conditional branch according to the result above
4. Output a character to standard output

As elements 1 and 4 are covered by the "Welcome to —" (in which, precisely speaking, it is explained the way of input and output of string, but a single character can considered to be a string of length 1), we will explain the rest, elements 2 and 3.

2. In most modern languages, the functionality of such judgement is implemented out-of-the-box, whose name is such like `isUpperCase`, `islower`. You can find such functionality or alternative methods by searching like "[language name] check case of letter". (Tens of languages are available in AtCoder, so this PDF is too narrow to explain comprehensively about them.)

3. In most languages, "if statement" or similar syntax is available, and also most languages have "conditional operators" or similar one. Just like above, you can obtain some information about the syntax available in the language by searching "[language name] if" or "[language name] conditional branch".

Sample Code in Python:

```python
a = input()
print('A' if a.islower() else 'a')
```

Sample Code in Ruby (as an example of languages which do not have standard functions like `isUpperCase` as mentioned in 2.):

```ruby
a = gets.chomp
if a == a.upcase
    puts 'A'
else
    puts 'a'
end
```

B: Mix Juice

(Editorial: evima)

First, unlike Problem A, in which the amount of input data is fixed to a single character, this time, N, the number of fruits whose prices are provided, is not constant. General keywords to deal with them are "for statements" and "arrays," and you will obtain the information for each language by searching like "[language name] for". Note that, however, rather different syntax or elements might be useful sometimes.

In this problem, when choosing K kinds of fruits to buy out of N kinds of fruits, choosing K kinds of cheapest fruits is optimal of course. That is, the answer of this problem is the sum of the smallest, the second smallest, ..., and the K-th smallest ones out of N integers $p_1, \ldots, p_n$, and the task is how to instruct the program to find them. In conclusion, most languages have functionalities of "sorting," that is, the function of arranging multiples values in the ascending order, so the easiest way is to use this function to sort N integers in the ascending order and choosing the first K elements. Still you can obtain information by "[language name] sort".

Sample code in C++:

```cpp
#include <algorithm>    // sort
#include <iostream>
using namespace std;

int N, K, p[1010];

int main(){
    cin >> N >> K;
    for(int i = 0; i < N; ++i) cin >> p[i];
    sort(p, p + N);
    int ans = 0;
    for(int i = 0; i < K; ++i) ans += p[i];
    cout << ans << endl;
}
```

Sample code in Python (as an example of languages in which rather different elements to “for statements” are useful):

```
N, K = map(int, input().split())
p = list(map(int, input().split()))
print(sum(sorted(p)[:K]))
```

C: One Quadrillion and One Dalmatians

(Editorial: evima)

First, the number 10^{15} is so large even for our modern machine that it is impossible to perform some operation this number of times within two seconds.

A simple way is to find the length of the name of the dog numbered N by calculating the "and so on"-part found in the problem statement. By "Name X", we denote the name of the dog numbered N. For each $i = 1, 2, \ldots$, there are 26^i names of length i. Therefore, Names $1, \ldots, 26$ have lengths 1, Names $26 + 1, \ldots, 26 + 26^2$ have lengths 2, Names $26 + 26^2 + 1, \ldots, 26 + 26^2 + 26^3$ have lengths 3, 3, ... and so on. Since $26 + 26^2 + 26^3 + \ldots + 26^{11} = 3817158266467286 > 10^{15} + 1$, by the time we calculate the length 11, we can find the length of Name N. We assume that Name N appeared to have length l.

Then we can see that, Name N is the $N - (26 + 26^2 + \ldots + 26^{l-1})$ (let the value be k)-th name among 26^l names of lengths l. Now we want to find "the k-th among the strings of length l."

If yo are not sure how to answer this question, you can simplify it by considering the situation where only 10 alphabets,`a`, `b`, ... and `j`, are available. For example, under that assumption, the number of the name of length 3 is $10^3 = 1000$: `aaa`, `aab`, ..., `aaj`, `aba`, ... and `jjj`. Among them, for instance, the 246-th one is `bde`. Likewise, in the decimal representation of $k - 1$, the ones' place corresponds to the last letter, the tens' place corresponds to the second last letter, and the 10^is' place corresponds to the $i + 1$-th last letter (`a`, `b`, ..., `j` correspond to $0, 1, \ldots, 9$).

Similarly, when 26 kinds of alphabets are used, in the notation of base-26, the 26^is' place corresponds to the $i + 1$-th last letter. You can find each digit by the following procedures for example:

- Initialize variable x with $k - 1$, and repeat until x becomes 0.
- Let q and r be the quotient and remainder of x divided by 26, respectively. The i-th result of r in this operation is the i-th last digit of $k - 1$. Replace x with q.

Sample code in Python:

```python
N = int(input())
ans = ''
for i in range(1, 99):
    if N <= 26 ** i:
        N -= 1
        for j in range(i):
            ans += chr(ord('a') + N % 26)
            N //= 26
        break
    else:
        N -= 26 ** i
print(ans[::-1])    # reversed
```

(Appendix) a simpler implementation example:

```python
N = int(input())
ans = ''
while N > 0:
    N -= 1
    ans += chr(ord('a') + N % 26)
    N //= 26
print(ans[::-1])
```

D: Replacing

(Editorial: evima)

If you naively follow the directions in the problem statement, you will have to replace as many as 10 billion elements in total. In this case, even if we use fast languages like C++, we want about 10-second execution time, but the time limit for this problem is 2 seconds, so it won't finish in time.

Note that we are only asked to output the sum of all elements. After performing the operation of "replacing every element whose value is B with C" (we denote this operation by X), the total sum of elements increases by $(C - B) \times$ (the number of elements whose values are B before the opration). Therefore, instead of manipulating sequence A itself, it is more reasonable to manage the number of appearances of $1, \ldots, 100000$ in A. By the operation X, not only the sum of all elements but also the value of each element itself also changes, but this changes can also be represented by operating num_B and num_C. Therefore, by taking time for calculating the sum of all the elements and $num_1, \ldots, num_{100000}$ only once before start to perform the operations, each operation can be performed in a constant time.

E: Red Scarf

(Editorial: evima)

Hereinafter $\oplus$ denotes XOR as an operator.

By the definition of XOR in the problem statement, we can see that calculation of XOR of 3 or more integers can be done in an arbitrary order; for example, for any three integers a, b, c, it holds that $a \oplus b \oplus c = b \oplus c \oplus a = (a \oplus b) \oplus c = c \oplus (a \oplus b)$. We can also see that, $a \oplus a = 0$ for all integer a. Together, we can claim that $a \oplus b \oplus a = (a \oplus a) \oplus b = 0 \oplus b = b$.

Now let's move on to the original problem. Let b_i b the integer of i-th cat. Here, $a_i = b_1 \oplus \ldots \oplus b_{i-1} \oplus b_{i+1} \oplus \ldots \oplus b_N$. It's a bit kind of sudden, but we will calculate the XOR of the given N values $a_1, \ldots, a_N$. This value can be transformed as followed:

$$
\begin{aligned}
&a_1 \oplus a_2 \oplus \ldots \oplus a_N = (b_2 \oplus \ldots \oplus b_N) \oplus (b_1 \oplus b_3 \oplus \ldots \oplus b_N) \oplus \ldots \oplus (b_1 \oplus \ldots \oplus b_{N-1}) \\
&= (\underbrace{b_1 \oplus \ldots \oplus b_1}_{N-1\text{elements}}) \oplus (\underbrace{b_2 \oplus \ldots \oplus b_2}_{N-1\text{elements}}) \oplus \ldots \oplus (\underbrace{b_N \oplus \ldots \oplus b_N}_{N-1\text{elements}}) \\
&= b_1 \oplus b_2 \oplus \ldots \oplus b_N \ (\because N\text{is even})
\end{aligned}
$$

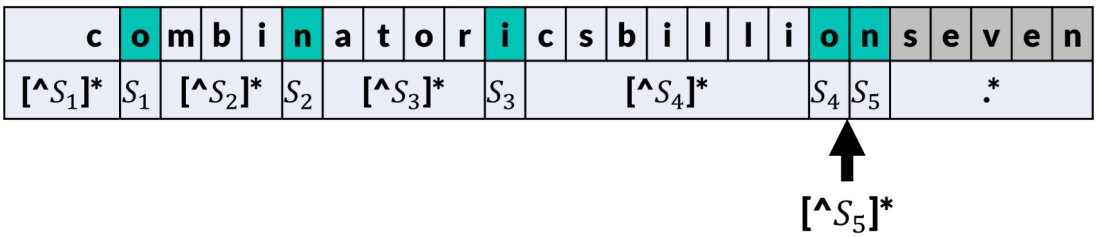
Therefore, we could see that $a_1 \oplus \ldots \oplus a_N = b_1 \oplus \ldots \oplus b_N$. Moreover, let this value be B, then for all i the following equations hold:

$$
\begin{aligned}
&S \oplus a_i = (b_1 \oplus \ldots \oplus b_N) \oplus (b_1 \oplus \ldots \oplus b_{i-1} \oplus b_{i+1} \oplus \ldots \oplus b_N) \\
&= (b_1 \oplus b_1) \oplus \ldots \oplus (b_{i-1} \oplus b_{i-1}) \oplus b_i \oplus (b_{i+1} \oplus b_{i+1}) \oplus \ldots \oplus (b_N \oplus b_N) = b_i
\end{aligned}
$$

Therefore, for each i it appears that $b_i = S \oplus a_i$, so the problem has been solved. There was a constraints in the problem statement that "there exists a combination of integers on the scarfs that is consistent with the given information," and the "Output" section states that "if there are multiple possible solutions, you may print any of them;" however, by this solution, for any combination of N integers $a_1, \ldots, a_N$, there always exists a unique combination of integers on the scarfs $b_1, \ldots, b_N$ that satisfies it.

F: Strivore

Each string that satisfies the given conditions is of length $|S| + K$, which has the following form:



Here, "[^S_i]*" denotes a string consisting of English alphabets other than S_i of 0 or more length, and ".*" denotes a string consisting of any English alphabets of 0 or more length. Note that there are 26 kinds of English alphabets.

Assume that the positions of $S_1 \sim S_N$ among $|S| + K$ letters are fixed. Then, the coices of alphabets at the positions other than those of S_I are independent of each other, so the number of such combinations can be calculated by simple multiplications: there are $25^{K-(\text{the number of letters before } S_N)} \times 26^{(\text{the number of letters after } S_N)}$ combinations.

Also, when the positions S_N is fixed, the number of combinations of positions of $S_1 \sim S_{N-1}$ is ${}_{|S|+K-(\text{The number of letters after } S_N)-1}C_{N-1}$ (search for "binomial coefficient" for the meaning of this formula.

There, try all possible positions of S_N, and for each of them, calculate [the choices of positions of $S_1 \sim S_{N-1}$] $\times$ [the choices of alphabets other than S_i], then the sum is the desired answer.

After a proper precalculation, this problem can be solved in a total of $\Theta(|S| + K)$.

ABC 172 解説

beet, evima, kyopro_friends, sheyasutaka, tozangezan, ynymxiaolongbao

2020 年 6 月 27 日

For International Readers: English editorial starts on page 9.

A: Calc

(解説: evima)

プログラミングの学習を始めたばかりで何から手をつけるべきかわからない方は、まずは「practice contest」(`https://atcoder.jp/contests/practice/`) の問題 A「はじめてのあっとこーだー」をお試しください。言語ごとに解答例が掲載されています。

今回の問題に含まれる要素であって「はじめての〜」に含まれないものは、a^2, a^3 といった値の計算のみです。言語を問わずに使える確実な方法は、`a * a * a` などとして計算してしまうことです。言語に `**`, `^` といった累乗演算子が用意されていればそれを使うこともできますが、結果が実数ではなく整数として計算されることを確認してからの方が安全かもしれません。

`pow()` といった名前の標準ライブラリも多くの言語に用意されていますが、それらは実数の実数乗を計算するものであることが多く、整数の整数乗の計算に用いるには適さない可能性があります。$2^{53}-1$ を超える整数は 64 ビット実数型では正しく表現できないため、計算結果がその範囲にあると正しい結果が得られません[*1]。また、正しい計算結果が得られても、`cout << a + pow(a, 2) + pow(a, 3)` などと実数のまま出力して `1.0101e+06` などとなってしまうと不正解と判定されます (この問題に限らず、AtCoder では、整数での出力が期待される問題で実数形式の数値を出力すると不正解と判定されます)。ただし、今回の $1 \leq a \leq 10$ という制約下では計算結果が誤っていることはないはずで、出力形式に関しても都合よく処理する言語が多いようです。

[*1] 今回の制約には反しますが、例えば $a = 1000000$ の場合、$a + a^2 + a^3$ を 64 ビット実数を用いて計算した結果を整数にキャストすると 1000001000000999936 となります

C++ での実装例:

```cpp
#include <iostream>
using namespace std;
int main(){
    int a;
    cin >> a;
    cout << a + a * a + a * a * a << endl;
}
```

B: Minor Change

(解説: evima)

問題文は厳密を期してやや堅苦しく書かれていますが、平たくいえば「S と T は何か所が異なるか？」という内容に過ぎません。

主な課題は、二つの文字列の長さが一定でない上に大きいことです。「if 文」の扱いは Beginner Contest の問題 A でも要求されますが、今回の問題も

`if length(S) >= 100 and S[99] != T[99] then ans += 1`

といった行を 20 万行並べて理論上は解くことができます。しかし、ソースコードが数百万文字以上の長さになり、AtCoder に提出できません (提出可能な最大長は 52 万文字程度です)。

20 万行の if 文を不要にするには、「for 文」と呼ばれるループ構造を使うのが最も素直です。`if S[i] != T[i] then ans += 1` という処理を for ループ内で $i = 0, i = 1, \ldots, i = (S \text{ の長さ}) - 1$ のそれぞれに対して実行させれば、一個の if 文で済みます。「for 文」の言語ごとの詳細は検索エンジンで「[言語名] for」などと検索することで得られるはずです。

C++, Python での実装例 (後者は「for 文」以外のアプローチの例として):

```cpp
#include <iostream>
using namespace std;

int main(){
    string S, T;
    cin >> S >> T;
    int N = S.size(), ans = 0;
    for(int i = 0; i < N; ++i){
        if(S[i] != T[i]) ++ans;
    }
    cout << ans << endl;
}
```

```python
S, T = input(), input()
print(len(list(filter(lambda i: S[i] != T[i], range(len(S))))))
```

C: Tsundoku

(解説: evima)

結局のところ、机 A, B からそれぞれ何冊の本を読むか以外に行うべき選択はありません。$a_0 = 0, a_1 = A_1, a_2 = A_1+A_2, a_3 = A_1+A_2+A_3, \ldots, a_N = A_1+\cdots+A_N$ とします。$b_0, b_1, \ldots, b_M$ も同様に定義します。これらの値をプログラムで求める際は、$a_1 = a_0+A_1, a_2 = a_1+A_2, a_3 = a_2+A_3, \ldots$ などとします (毎回 A_1 から足していくと時間切れになります)。すると、机 A から i 冊、机 B から j 冊読むことを考えれば、問題は次のように言い換えられます。

> 整数 i, j $(0 \leq i \leq N, 0 \leq j \leq M)$ を $a_i + b_j \leq K$ となるように選ぶとき、$i + j$ がとりうる最大の値を求めよ。

N, M がある程度小さければ、$(i, j) = (0, 0), (0, 1), \ldots, (0, M), (1, 0), \ldots, (N, M)$ という $(N + 1) \times (M + 1)$ 通りすべての選択を実際に試すことでこの問題を解くことができます。しかし、$N = M = 200000$ の場合には少なくとも数十秒の実行時間が必要で、間に合いません。

そこで、i についてのみ $i = 0, 1, \ldots, N$ というすべての選択を試し、このそれぞれについて選択可能な j の最大値、つまり $b_j \leq K - a_i$ であるような j の最大値を求めることにします。

まず、$i = 0$ のときに選択可能な j の最大値を求めます。これは、$j = M, M - 1, \ldots$ と降順に走査すると、$b_j \leq K - a_0$ が最初に成立した j として求まります。この j の値を $best_0$ とします。

次に、$i = 1$ のときに選択可能な j の最大値を求めます。ここで、本一冊を読むのにかかる時間が正であること (a, b がともに単調増加数列であること) から、この最大値は $best_0$ 以下です。よって、降順の走査を $j = M$ から始める必要はなく、$j = best_0$ から始めることができます。

この要領で $i = N$ まで計算を続ける (ただし $a_i > K$ となったら打ち切る) ことで、計算量を大幅に削減することができます。次ページの実装例 (Python のコードではありますが、主要部である 11 行目から 18 行目まではほぼ疑似コードといえるでしょう) で考えると、15, 16 行目の while ループの内部は合計で M 回以下しか実行されないため、時間計算量は $\mathrm{O}(N + M)$ となります。

Python での実装例:

```python
N, M, K = map(int, input().split())
A = list(map(int, input().split()))
B = list(map(int, input().split()))

a, b = [0], [0]
for i in range(N):
    a.append(a[i] + A[i])
for i in range(M):
    b.append(b[i] + B[i])

ans, j = 0, M
for i in range(N + 1):
    if a[i] > K:
        break
    while b[j] > K - a[i]:
        j -= 1
    ans = max(ans, i + j)
print(ans)
```

D: Sum of Divisors

(解説: evima)

一般に、正整数 X に対して、$f(X)$ つまり X の約数の個数は $\mathrm{O}(\sqrt{X})$ 時間で求められます[*2]。しかし、$X = 1, \ldots, 10^7$ のすべてに対してこれを行う時間はありません。

そこで、$1, \ldots, N$ の約数の個数を求めることなく問題で要求された値を計算することを考えます。要求された値は、以下の疑似コードが出力する値であると考えられます。

```
1 ans = 0
2 for i = 1, ..., N:
3     for j = 1, ..., N:
4         if i % j == 0 then ans += i
5 print ans
```

ここで、4 行目は結局のところ $(i, j) = (1, 1), \ldots, (1, N), (2, 1), \ldots, (N, N)$ の N^2 通りのペアに対して実行されるため、次のように 2, 3 行目の for 文を入れ替えても結果は変わりません。

```
1 ans = 0
2 for j = 1, ..., N:
3     for i = 1, ..., N:
4         if i % j == 0 then ans += i
5 print ans
```

このコードの挙動を考えると、問題は次のように言い換えられます。

> 正整数 j に対し、j の倍数であって N 以下であるものの総和を $g(j)$ とする。$\sum_{j=1}^{N} g(j)$ を求めよ。

この $g(X)$ は等差数列の和であり、$f(X)$ より高速に求めることができます。具体的には、$Y = \lfloor N/X \rfloor$ (N/X 以下の最大の整数) とすると $g(X) = X + 2X + \ldots + YX = (1 + 2 + \ldots + Y)X = Y(Y+1)X/2$ であり、この式を用いれば $\mathrm{O}(N)$ 時間で問題が解けます。

(一部の言語では、上の式を用いても実行制限時間に間に合わない可能性があります。その場合は、心苦しいですが、他の言語を用いてくださいというほかありません。制約における N の上限を小さくしてしまうと、$X = 1, \ldots, N$ のすべてに対して $\mathrm{O}(\sqrt{X})$ 時間で約数を列挙する方針を C++ などの言語で実装すれば間に合いかねないため、このような制約となりました。)

[*2] $i = 1, \ldots, \lfloor \sqrt{X} \rfloor$ ($\sqrt{X}$ 以下の最大の整数) のそれぞれについて X を割り切るか確かめ、割り切るなら i と X/i は X の約数である、とすることで X のすべての約数が得られます

E. NEQ

(解説: ynymxiaolongbao)

$1 \leq i \leq N$ なる任意の i について $A_i \neq B_i$ であるという条件を一旦無視します。すなわち、1 以上 M 以下の整数からなる長さ N の数列 A と B の組であって、$1 \leq i < j \leq N$ なる任意の (i, j) について $A_i \neq A_j$ かつ $B_i \neq B_j$ であるようなもの全体を考えます。

各要素が 1 以上 N 以下の整数である集合 S について、「$i \in S$ なるすべての i について $A_i = B_i$ である」という条件を満たす A と B の組の個数は ${}_MP_{|S|} \times ({}_{M-|S|}P_{N-|S|})^2$ です。この式は、1 以上 M 以下の整数から $|S|$ 個を並べて $i \in S$ なる i の $A_i (= B_i)$ の値とし、残りの $M - |S|$ 個の整数から $N - |S|$ 個を並べて $i \notin S$ なる i の A_i の値とし、同様に $M - |S|$ 個の整数から $N - |S|$ 個を並べて $i \notin S$ なる i の B_i の値とすることを考えることで得られます。

包除原理より、すべての 1 以上 N 以下の整数の集合 S について、「$i \in S$ なるすべての i について $A_i = B_i$ である」という条件を満たす A と B の組の個数を求め、それに $(-1)^{|S|}$ を掛けた値の総和が答えになります。すなわち $(-1)^{|S|} \times {}_MP_{|S|} \times ({}_{M-|S|}P_{N-|S|})^2$ の総和が答えになります。

ここで、$(-1)^{|S|} \times {}_MP_{|S|} \times ({}_{M-|S|}P_{N-|S|})^2$ の値を 2^N 通りすべての S について計算していては間に合いませんが、この値が $|S|$ のみに依存することに注目すると、$|S|$ に対する値に $|S|$ がその値になるような S の個数、すなわち ${}_NC_{|S|}$ を掛けたものを足し合わせることでも答えを求めることができます。つまり、答えは $\sum_{K=0}^{N}({}_NC_K \times (-1)^K \times {}_MP_K \times ({}_{M-K}P_{N-K})^2)$ と表現できます。

コンビネーションや順列は、フェルマーの小定理を用いる方法や、拡張ユークリッドの互除法を用いる方法で高速に求めることができます。前者を用いれば、計算量は $O(M + N)$ になります。

F: Unfair Nim

(解説: evima)

タイトルで示唆されているように、このゲームは Nim と呼ばれる有名なもので、$A_1 \oplus A_2 \oplus \ldots \oplus A_N = 0$ ($\oplus$ は XOR: ビットごとの排他的論理和) であれば後手が必勝法を持ち、そうでなければ先手が必勝法を持つことが知られています。この問題を解く上では、Nim についてはこの知識を持ってさえいれば十分であり、詳細には立ち入りません。

上記より、$A_1 + A_2$ を S、$A_3 \oplus A_4 \oplus \ldots \oplus A_N$ を X として、問題は次のように言い換えられます。

> $a + b = S, a \oplus b = X, a \leq A_1$ であるような非負整数のペア (a, b) が存在するか判定し、存在する場合はそのようなペアで a がとりうる最大の値を求めよ。

(元の問題で第 1 の山からすべての石を移すことが禁じられていることが反映されていませんが、答えが 0 であった場合に代わりに「存在せず」と報告することで対処できます)

解法の一つは、問題をさらに言い換えて「A_1 以下の非負整数 a であって $a + (a \oplus X) = S$ を満たす最大のものを求めよ」として、動的計画法により実質的に a の値の全探索を行うことです。

しかし、$a + b = (a \oplus b) + 2 \times (a \ \& \ b)$ (`&` は AND: ビットごとの論理積。この式が成り立つことは、XOR が「繰り上がりを無視した足し算」であることを考えればわかります) であることを利用すると、探索の必要はなくなります。条件を満たすような (a, b) について、先ほどの式より $a \ \& \ b = (S - X)/2$ (この値を D とします) が成り立ちます。D が非負整数でなかったり、D と X で共通して立っているビットが一か所でもある場合、答えは「存在せず」です。

そうでなければ、$a \leq A_1$ という条件を一旦無視すると、X で立っているビットの集合を二分割して得られる整数を Y, Z としたとき ($Y \ \& \ Z = 0, Y \oplus Z = X$)、$a = D \oplus Y, b = D \oplus Z$ というペアは残りの条件を満たします。また、この方法で得られるペアの他に残りの条件を満たすペアはありません。このようなペアにおける a の値のうち、$a \leq A_1$ を満たす最大のものが求める答えです。

まず、このようなペアにおける a の最小の値は $Y = 0$ としたときの $a = D$ で、この値が $a \leq A_1$ を満たさなければ答えは「存在せず」です。そうでなければ、$a \leq A_1$ が満たされる範囲で $a = D \oplus Y$ をできるだけ大きくするように Y を選択する必要があります。結論から述べると、まず $Y = 0$ として、X で立っているビットを一つずつ降順に取り上げ、「このビットを Y に入れても $a = D \oplus Y \leq A_1$ は成立するか？」と問い、答えが Yes なら入れる、という貪欲な戦略が最適です。これは、数の大小の定義 (任意の桁は、それより下のすべての桁を合わせたものより重要) から示せます。よって、言い換え後の問題は $O(\log X)$ 時間で解けます。

A: Calc

(Editorial: evima)

If you are a beginner of studying programming and have no idea what to start with, then try solving "Welcome to AtCoder", Problem A from the "practice contest" (`https://atcoder.jp/contests/practice/`). There you can find sample codes for each language.

The only element in this problem that is not covered by "Welcome to —" is the calculation of a^2, a^3 and so on. The most certain way that is available regardless of languages is calculating like `a * a * a`. If the language has operators like `**` or `^`, then you can also use them, but it might be safer to ascertain that the result is calculated as integers, not as real numbers.

Most languages provides with standard libraries named `pow()` or similar, but they tend to aim at calculating a real-number power of real number, and not be suitable for calculating an integer-power of integer. Since integers exceeding $2^{53} - 1$ cannot be represented accurately by 64-bit real numbers, so if the result is not within the range, you cannot obtain accurate result[*3]. Also, even if you obtained an accurate result, if you output as real number like `cout << a + pow(a, 2) + pow(a, 3)`, you will obtain outputs like `1.0101e+06`, which will be judges as wrong answer (not only in this problem; in any AtCoder problems which expects integer outputs, if you output in a real-number style, it will be judged as being wrong answer). However, under the constraints $1 \leq a \leq 10$ this time, the calculation result must be accurate, and also output format seems to be processed suitably in many languages.

[*3] If $a = 1000000$, though this is not within the constraints this time, calculating $a + a^2 + a^3$ with 64-bit real numbers results in 1000001000000999936 when casted to integer

Sample code in C++:

```cpp
#include <iostream>
using namespace std;
int main(){
    int a;
    cin >> a;
    cout << a + a * a + a * a * a << endl;
}
```

B: Minor Change

(Editorial: evima)

The problem statement is written rather formally for the sake of strictness, but in fact the problem simply asks "how many letters are different between S and T?"

The main issue is that the lengths of the strings are not constant, and large. The treatment of "if statements" is required in Problem A of Beginner Contests, and this problem can be theoretically solved by repeating 200 thousands lines of if statements like

```
if length(S) >= 100 and S[99] != T[99] then ans += 1
```

However, the source code would have millions of characters, which is not submittable to AtCoder (The possible maximum length is about 520 thousands).

To avoid 200 thousand if statements, most straightforward way is to use the structure called "for statements." If you execute the operation of `if S[i] != T[i] then ans += 1` for each $i = 0, i = 1, \ldots, i =$ (`length of` S) $- 1$ in the for statement, you just have to write if statement once. You can obtain more details of for statement by searching like "[language name] for".

Sample code in C++ and Python (the latter of which is as an example of an approach other than "for statement")

```cpp
#include <iostream>
using namespace std;

int main(){
    string S, T;
    cin >> S >> T;
    int N = S.size(), ans = 0;
    for(int i = 0; i < N; ++i){
        if(S[i] != T[i]) ++ans;
    }
    cout << ans << endl;
}
```

```python
S, T = input(), input()
print(len(list(filter(lambda i: S[i] != T[i], range(len(S))))))
```

C: Tsundoku

(Editorial: evima)

After all, the only choice you can make is how many books to read from desk A and B. Let $a_0 = 0, a_1 = A_1, a_2 = A_1 + A_2, a_3 = A_1 + A_2 + A_3, \ldots, a_N = A_1 + \cdots + A_N$. We also define $b_0, b_1, \ldots, b_M$ similarly. When calculating those value in programs, we calculating in the way like $a_1 = a_0 + A_1, a_2 = a_1 + A_2, a_3 = a_2 + A_3, \ldots$ (if you sum up from A_1 every time, it will exceed time limit). Then, considering reading i books from desk A and j books from desk B, the problem can be rephrased as follows:

> For integers i, j $(0 \leq i \leq N, 0 \leq j \leq M)$ such that $a_i + b_j \leq K$, find the maximum possible value of $i + j$.

If N and M are fairly small, we can solve this problem by trying all possible $(N+1) \times (N+1)$ combinations of $(i, j) = (0, 0), (0, 1), \ldots, (0, M), (1, 0), \ldots, (N, M)$ However, when $N = M = 200000$, we need at least tens of seconds of execution time, which will exceed the time limit.

Then, let us try exhaustive search for only $i = 0, 1, \ldots, N$, and for each i, find the maximum choosable j, that is, maximum j such that $b_j \leq K - a_i$.

First, let us find the maximum choosable j when $i = 0$. This can be found as the first j such that $b_j \leq K - a_0$ when scanning in the order of $j = M, M - 1, \ldots$. Let such j be $best_0$.

Next, we will find the maximum choosable j when $i = 1$. Since the time needed to read a book is positive (therefore a and b are both monotonic increasing sequence), the maximum value no more than $best_0$. Therefore, you don't have to start scanning in the decreasing order from $j = M$; instead, you can start it from $j = best_0$.

By continuing calculation until $i = N$ in the similar way (stopping when $a_i > K$ though), you can dramatically decrease the execution time. In the sample code on the next page (although it is written in Python, the cardinal part of line 11 to 18 is readable as pseudocode), the while loop on line 15 and 16 is at most executed M times, so the total time complexity is $O(N + M)$.

Sample code in Python:

```python
N, M, K = map(int, input().split())
A = list(map(int, input().split()))
B = list(map(int, input().split()))

a, b = [0], [0]
for i in range(N):
    a.append(a[i] + A[i])
for i in range(M):
    b.append(b[i] + B[i])

ans, j = 0, M
for i in range(N + 1):
    if a[i] > K:
        break
    while b[j] > K - a[i]:
        j -= 1
    ans = max(ans, i + j)
print(ans)
```

D: Sum of Divisors

(Editorial: evima)

Generally, for a positive integer X, the number of divisors of X, $f(X)$, can be calculated in a total of $\mathrm{O}(\sqrt{X})$ time. [*4]. However, the time limit is not long enough to execute this for all $K = 1, \ldots, 10^7$.

Then, let us consider finding the value desired in the problem statement without finding the number of divisors of $1, \ldots, N$. The desired answer can be considered as the value that the following pseudocode outputs.

```
1 ans = 0
2 for i = 1, ..., N:
3     for j = 1, ..., N:
4         if i % j == 0 then ans += i
5 print ans
```

Here, line 4 is executed for all N^2 pairs of $(i, j) = (1, 1), \ldots, (1, N), (2, 1), \ldots, (N, N)$, so you can swap the for statements in the lines 2 and 3 without changing the result.

```
1 ans = 0
2 for j = 1, ..., N:
3     for i = 1, ..., N:
4         if i % j == 0 then ans += i
5 print ans
```

Considering the behavior of the code, the problem can be reworded as follows:

> For a positive integer j, we define $g(j)$ as the sum of multiples of j that does not exceed N. Find $\sum_{j=1}^{N} g(j)$,

This $g(X)$ is a sum of an arithmetic progression, and can be calculated faster than $f(X)$. Specifically, let $Y = \lfloor N/X \rfloor$ (the maximum integer that does not exceed N/X), then $g(X) = X + 2X + \ldots + YX = (1 + 2 + \ldots + Y)X = Y(Y+1)X/2$, so by using this formula, the problem can be solved in a total of $\mathrm{O}(N)$ time.

(In some language, even if you use the equations above, your program may not finish in the time limit. In such case, regretfully, there is no way but using other languages. If the upper bound of N is smaller, iterating all the divisors of all $X = 1, \ldots, N$ in $\mathrm{O}(\sqrt{X})$ time each might

[*4] For each $i = 1, \ldots, \lfloor \sqrt{X} \rfloor$ (maximum integer not exceeding $\sqrt{X}$), check if it divides X, and if it does, then i and X/i are the divisors of X; by doing so you can obtain all the divisors of X

be finish in the time limit when implemented in languages like C++, and this is the reason for such constraints.)

E. NEQ

(Editorial: ynymxiaolongbao)

We first ignore the conditions that $A_i \neq B_i$ for any i such that $1 \leq i \leq N$. In other words, we consider all the pairs A and B of sequences of length N consisting of integers between 1 and M (inclusive; the same applies hereinafter) such that, for all (i, j) such that $1 \leq i < j \leq N$, $A_i \neq A_j$ and $B_i \neq B_j$.

For a set of integers whose each element is between between 1 and N, the number of pairs A and B of sequences such that "for all $i \in S$, $A_i = B_i$" is ${}_MP_{|S|} \times ({}_{M-|S|}P_{N-|S|})^2$. This expression can be obtained by considering the $|S|$-permutation of integers between 1 and M assigned to $A_i (= B_i)$ for each i such that $i \in S$, $N - |S|$-permutation of the remaining $M - |S|$ integers assigned to A_i for each i such that $i \notin S$, and similarly $N - |S|$ permutation of the $M - |S|$ integers assigned B_i for each i such that $i \notin S$.

By inclusion-exclusion principle, the answer is the sum of the number of pairs of sets A and B such that "for all $i \in S$, $A_i = B_i$", multiplied by $(-1)^{|S|}$, for all sets of integers between 1 and N. In other words, the answer is the sum of $(-1)^{|S|} \times {}_MP_{|S|} \times ({}_{M-|S|}P_{N-|S|})^2$.

Here, calculating $(-1)^{|S|} \times {}_MP_{|S|} \times ({}_{M-|S|}P_{N-|S|})^2$ for all 2^N possibilities of S, but since this value only depends on $|S|$, you can also calculate the sum by finding the value for $|S|$ multiplied by the number of S such that $|S|$ is that value, that is, multiplied by ${}_NC_{|S|}$. Therefore, the answer is can be represented as $\sum_{K=0}^{N} ({}_NC_K \times (-1)^K \times {}_MP_K \times ({}_{M-K}P_{N-K})^2)$.

Combinations and permutations can be calculated by making use of the little theorem of Fermat or extended Euclidean algorithm. By using the former, the total time complexity is $O(M + N)$.

F: Unfair Nim

(Editorial: evima)

As the title implies, this game is a famous one called Nim; it is known that the player who moves second has a strategy to always win if and only if $A_1 \oplus A_2 \oplus \ldots \oplus A_N = 0$ (where $\oplus$ denotes XOR: bitwise logical exclusive sum), and otherwise the player who moves first has a strategy to always win. When solving this problem, what you only have to know about Nim is this knowledge, so we won't explain in detail.

By above, let S be $A_1 + A_2$ and X be $A_3 \oplus A_4 \oplus \ldots \oplus A_N$, then the problem can be rephrased as follows:

> Judge if there exists a pair of non-negative integers (a, b) such that $a + b = S, a \oplus b = X, a \leq A_1$, and if so, find the maximum value of a.

(In the problem above, you are allowed to move all the stones for the first pile, which you are not in the original problem, but you can handle with it by answering "impossible" if the answer is 0.

One of the solutions is to rephrase the problem further as "find the maximum non-negative integer a such that $a+(a \oplus X) = S$, and use dynamic programming to virtually perform exhaustive search for all a.

However, since $a + b = (a \oplus b) + 2 \times (a \ \texttt{\&} \ b)$ (where `&` denotes AND: bitwise logical product. This formula can be justified by considering that XOR is operation of "calculating the sum while ignoring carryovers), you don't have to perform searching. For each (a, b) that satisfies the conditions, by the equation above, it holds that $a \ \texttt{\&} \ b = (S - X)/2$ (let this value be D). If D is not a non-negative integer, or D and X has one or more bit in common, then the answer is "impossible".

Otherwise, if we ignore the condition that $a \leq A_1$, then for all pairs of integers Y and Z which can be obtained by partitioning the set of bits 1 in X ($Y \ \texttt{\&} \ Z = 0, Y \oplus Z = X$), the pair $a = D \oplus Y, b = D \oplus Z$ satisfies the remaining constraints. Also, no other pair satisfies such conditions. Among each a of such pairs, the maximum a such that $a \leq A_1$ is the desired answer.

First, the minimum a is $a = D$, which can be obtained when $Y = 0$, and if this value does not satisfy $a \leq A_1$, then the answer is "impossible." Otherwise, we want to choose as large Y

as possible, subject to $a \leq A_1$, so that $a = D \oplus Y$ is maximum possible. Come to the point, the optimal is a greedy strategy of, first let $Y = 0$, then iterate each bit of X whose value is 1, checking if "after adding this bit to Y, does $a = D \oplus Y \leq A_1$ still holds?", and if the answer is Yes, then add it. This can be proven by definition of ordering of numbers (that any digit is significant that all lower digits combined). Therefore, the rephrased problem can be solved in a total of $\mathrm{O}(\log X)$ time.

ABC 173 解説

evima, gazelle, kyopro_friends, satashun, sheyasutaka, ynymxiaolongbao

2020 年 7 月 5 日

For International Readers: English editorial starts on page 10.

A: Payment

(解説: evima)

プログラミングの学習を始めたばかりで何から手をつけるべきかわからない方は、まずは「practice contest」(`https://atcoder.jp/contests/practice/`) の問題 A「はじめてのあっとこーだー」をお試しください。言語ごとに解答例が掲載されています。

「はじめての〜」に正解するソースコードがあれば、今回の問題でも入出力は行えるはずです。以下では、お釣りの金額を求めることに集中します。

支払う千円札の枚数を計算する方針も考えられますが、ここでは価格 N の下三桁に注目します。千円札のみで支払いを行う以上、N の下三桁のみがお釣りの金額に関係します。この下三桁に対応する数値は、多くの言語に存在する剰余演算子 `%` を用いて `N % 1000` と求められます。

この値が 0 であればお釣りは 0 円、そうでなければ 1000 からその値を引いたものがお釣りの金額です。ここでは if 文を使うこともできますが、`1000 - N % 1000` をさらに 1000 で割った余りを計算することでも対処できます。以下は後者を C++ で実装した例です。(説明は次頁へ続きます)

```cpp
#include <iostream>
using namespace std;
int main(){
    int N;
    cin >> N;
    cout << (1000 - N % 1000) % 1000 << endl;
}
```

言語によっては、「負の整数 % 正の整数」が負になりません。そのような言語では、以下の Ruby での実装例のように楽をすることもできます。

```
puts -gets.to_i % 1000
```

B: Judge Status

(解説: evima)

仮にテストケース数 N が 3 などの値に固定されていれば、if 文のみでも処理することができます。しかし、現実には N は 1 以上 10 万以下と一定でない上に大きく、for 文の出番です。

最も単純な方針は、以下の疑似コードの内容を実装することです。

```
N := input
C0, C1, C2, C3 := 0, 0, 0, 0
for i = 0, ..., N-1:
    S := input
    if S == "AC": C0 = C0 + 1
    else if S == "WA": C1 = C1 + 1
    else if S == "TLE": C2 = C2 + 1
    else: C3 = C3 + 1
print C0, C1, C2, C3
```

多くの言語で、この疑似コードを言語に合わせて修正することでこの問題を解くことができます。ただし、文字列を比較する際に言語によっては特別な注意を要します。言語ごとの詳細な情報は、検索エンジンで「[言語名] for 文」「[言語名] 文字列 比較」などと検索すれば手に入るはずです。以下に、疑似コードを Java のソースコードに変換したものを挙げます。

```java
import java.util.*;
public class Main {
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        int N = sc.nextInt();
        int C0 = 0, C1 = 0, C2 = 0, C3 = 0;
        for(int i = 0; i < N; ++i){
            String S = sc.nextLine();
            if(S.equals("AC")) ++C0;
            else if(S.equals("WA")) ++C1;
            else if(S.equals("TLE")) ++C2;
            else ++C3;
```

```
        }
        System.out.println("AC x " + C0);
        System.out.println("WA x " + C1);
        System.out.println("TLE x " + C2);
        System.out.println("RE x " + C3);
    }
}
```

もちろん、上記の疑似コードと見た目が異なるコードを書くことも可能です。その一例として、以下に Python での実装例を挙げます。

```
N = int(input())
s = [input() for i in range(N)]
for v in ['AC', 'WA', 'TLE', 'RE']:
    print('{0} x {1}'.format(v, s.count(v)))
```

C: H and V

(解説: evima)

要するに、「1 行目を赤く塗るか？ 2 行目を塗るか？ $\cdots$ H 行目を塗るか？ 1 列目を塗るか？ 2 列目を塗るか？ $\cdots$ W 列目を塗るか？」という $H+W$ 個の二者択一の問いに「Yes, No, Yes, Yes, $\cdots$」などと答えることが求められています。このような Yes/No 列は 2^{H+W} 通り考えられ、今回の制約 $H, W \leq 6$ のもとでは最大でも $2^{12} = 4096$ 通りです。この程度であれば、全通りの Yes/No 列を試して実際に黒いマスが K 個残るようなものの数を数えても時間切れの心配はありません。

あとは Yes/No 列を列挙するというプログラミングの課題です。再帰関数などを用いる方針も考えられますが、ここでは Yes/No 列を二進数と対応づけて列挙することにします。以下、`0b` で始まる数は二進数です (例: `0b0101` $= 4 + 1 = 5$)。

しばらく行についてのみ考えます。例えば $H = 3$ のとき、3 個の列に対して $2^3 = 8$ 通りの Yes/No 列が考えられます。そして、「No, No, No」を `0b000` $= 0$、「Yes, No, No」を `0b001` $= 1$、「No, Yes, No」を `0b010` $= 2$、「Yes, Yes, No」を `0b011` $= 3$、$\cdots$、「Yes, Yes, Yes」を `0b111` $= 7$ に対応させれば、2^3 通りの Yes/No 列と 0 から $2^3 - 1 = 7$ までの整数が一対一に対応します (左右が反転していますが、整数の表記では大きい桁ほど左に置かれるためです)。

同様に、行の数がいくつであっても、Yes/No 列の前から i 番目の要素を二進数の 2^i の位の桁に紐付けることで、2^H 通りの Yes/No 列と 0 から $2^H - 1$ までの整数が一対一に対応します。列に関しても 2^W 通りの Yes/No 列を 0 から $2^W - 1$ までの整数と一対一に対応させれば、以下の疑似コードのように二重ループで 2^{H+W} 通りすべてのシナリオを列挙することができます。

```
H, W, K, c := input
ans := 0
for maskR = 0, ..., (1 << H) - 1:
    for maskC = 0, ..., (1 << W) - 1:
        black := 0
        for i = 0, ..., H-1:
            for j = 0, ..., W-1:
                if ((maskR >> i) & 1) == 0 and ((maskC >> j) & 1) == 0
                and c[i][j] == '#': black = black + 1
        if black == K: ans = ans + 1
print ans
```

D: Chat in a Circle

A_i をソートして昇順にします．このとき，$\sum_{k=1}^{N-1} A_{N-\lfloor k/2 \rfloor}$ が答えになることを以下に示します (ちなみに，$\sum$ は総和を表す記号です．たとえば上の式であれば，$k = 1, 2, \ldots, N-1$ のときの $A_{N-\lfloor k/2 \rfloor}$ を全て足し合わせた値を表します．便利な記法ですので，ぜひ知っておいてください)．

最大値を証明するには，「1. この値にできる」「2. これより良い値にはできない」をそれぞれ示すのが定石です．ここでもこの方針を採用します．

どうすれば「この値にできる」？

フレンドリーさが高い順に移動させます．$i+1$ 番目に輪に加わった人の両隣に $2i+1, 2i+2$ 番目の人を入れるようにすると，「まだ両隣挿入の対象になっていない人 A の両隣には，A より早く来た人がいる」状態を常に保てることから，上の値を達成できることが言えます．

どうして「これより良い値にはできない」？

まず，フレンドリーさが高い順に移動させる状況のみを考えればよいことを示します．i 番目に移動する人のフレンドリーさを a_i として $a_1 \geq a_2 \geq \cdots \geq a_k < a_{k+1}$ なる k があったとき，

- $k+1$ 人目が割り込む位置が k 人目の隣でないとき，入れ替えてもお互い知ったこっちゃありません．
- $k+1$ 人目が割り込む位置が k 人目の隣であるとき，その時点で x, a_k, a_{k+1}, y と並んでいたとすると，$\min(x, y) + \min(a_k, y) \leq \min(x, y) + \min(x, a_{k+1})$ なので，割り込む順番を入れ替えても損しません．

したがってバブルソートの要領で，移動の順番をフレンドリーさが高い順に並べ替えても損しません．ここからはそのような状況だけを考えることにします．

A_N が心地よさに寄与するのは高々 1 回です．それ以外の人はどうでしょう？ A_k の左隣に割り込みが起きたら，それ以降は A_k の左隣に A_k よりも後に来た人がいる状態が保たれます．右隣も同様のことが言えるので，結局 A_k の寄与は 2 回までです．

A_N (最大値) が 1 つ，$A_1 \sim A_{N-1}$ が 2 つずつあり，そのうち自由に $N-1$ 個を選んで足し合わせるとき，最大値は $\sum_{k=1}^{N-1} A_{N-\lfloor k/2 \rfloor}$ (冒頭に挙げた答え) です．したがって，本問の答えはこれより良くなりません．

以上より，この解法の正当性が示せました．ソートがボトルネックとなり，$\Theta(N \log N)$ 時間で解けます．Median of Medians というテクニックで $\Theta(N)$ 時間を達成できますが，発展的内容なのでここでは軽く触れるにとどめます．

E: Multiplication4

(解説：kyopro_friends)

まず $K = N$ の場合は明らかです。また、A_i の全ての数が負かつ K が奇数のとき、答えは負になります。このときは絶対値が小さい方から K 個の積を取れば良いです。それ以外の場合、積は必ず非負にすることができます。

(証明：値の大きい方から K 個の積を考える。これが非負なら OK。負なら、選んだ要素のうち最大の数 (仮定より正) を取り除き、かわりに選ばなかった要素のうち最小の数 (仮定より負) を加えることで正になる)

したがって、符号に気を付けながら絶対値の最大化をすればよいです。

解法 1: 最後に符号を合わせる

$\{A_i\}$ を絶対値の降順にソートし、最初の K 個の積を P とします。P が非負なら答えは P です。P が負の場合、非負にするためには次の 2 種類の操作のどちらかをする必要があります。

(1) 負の数を 1 つ取り除き、非負の数を 1 つ加える
(2) 正の数を 1 つ取り除き、負の数を 1 つ加える

操作後の積の絶対値を最大化するには、取り除く要素の絶対値はなるべく小さく、加える要素の絶対値はなるべく大きくするのがよいので、それぞれの操作で選ぶべき数は一意に決まります。比較の方法に気を付けながら[*1]、より積が大きくなる方の操作を選べば良いです。そもそも一方の操作しか出来ない場合に注意してください。

解法 2: 非負の状態を保ちながら積を最大化する

S を空集合とします。「S の数の積が非負である」という条件を保ちながら、K 個の数を S に加えて積を最大化することを考えます。S の積が非負である状態を保ちながら要素を加える操作は全て、「非負の数を 1 つ加える」「負の数を 2 つ加える」という 2 種類の操作の繰り返しに分解できます。このことに注意すると、次のようなアルゴリズムで答えを得ることが出来ます。

次の操作を S の要素が K 個になるまで繰り返す。
操作：「まだ選んでいない非負の要素のうち絶対値が大きい順の 2 つ x_1, x_2 の積」と「まだ選んでいない負の要素のうち絶対値が大きい順の 2 つ y_1, y_2 の積」を比較する。前者の方が大きいか $|S| = K-1$ ならば x_1 を S に加える。そうでなければ y_1, y_2 を S に加える。

[*1] 「x_i を取り除いて y_i を加える」$(i = 1, 2)$ という 2 つの操作のどちらで積が大きくなるかは、$|P/x_1 * y_1| \leq |P/x_2 * y_2|$ かどうかがわかればよく、整理すると $|x_2 * y_1| \leq |x_1 * y_2|$ となり、今回の制約の下では 64bit 整数の範囲で計算できます。

これは、あらかじめ $\{A_i\}$ を非負の数と負の数に分け、絶対値の降順にソートしておくことで高速に行えます。「まだ選んでいない非負の要素・負の要素」が 1 つ以下となる場合に注意してください。

F: Intervals on Tree

実際にシミュレーションをする方法としては、L を固定して R を順に $+1$ していきながら Union Find 木などで連結成分を管理することで、連結成分の個数を実際に全ての L, R について計算することができますが、この時間計算量は $O(N^2\alpha(N))$ となり遅いです。

元のグラフが木であることを利用するために、まず固定された L, R に対して $f(L, R)$ の値がどうなるか考えてみましょう。$S = \{x | L \leq x \leq R\}$ が誘導する部分グラフは、各連結成分が木であるようなグラフ (森) になります。木の場合、頂点数は辺の本数 $+1$ であるので、森の場合には 頂点数 $=$ 辺の本数 $+$ 連結成分 (木) の個数 となります。

よって、連結成分数 ($=$ 頂点数 $-$ 辺数) に各頂点と辺が寄与していると見なすことで、連結成分数の和を求める代わりに、各頂点・辺が何回足し引きされるかを考えればいいことがわかります。これは、頂点についてはその頂点を含む S の個数、辺についてはその両端点を含む S の個数をそれぞれ計算すれば良いので、実は答えを入力を読みながら直接計算することができ、時間計算量は $O(N)$ です。

A: Payment

(Editorial: evima)

If you are a beginner of studying programming and have no idea what to start with, then try solving "Welcome to AtCoder", Problem A from the "practice contest" (`https://atcoder.jp/contests/practice/`). There you can find sample codes for each language.

If you have a code which is accepted for "Welcome to —", then you will be able to perform input and output for this problem to. We will now focus on finding the amount of change.

We may consider the number of 1000-yen bills, but this time, let us focus on the lower three digits of N. Since we use only 1000-yen bills for the payment, the charge is only dependent on the last three digits. The number that corresponds to the last three digits can be calculated as `N % 1000`, where `%` denotes modulus operator, which is available in many languages.

If this value is 0, then the charge is 0 yen, and otherwise the amount of charge is 1000 subtracted by that value. Here we can use if statements, but we can also handle with it by calculating the modulo of `1000 - N % 1000` divided by 1000 once again. The following is sample code in C++, implemented in the latter way. (The explanation goes on to the next page.)

```cpp
#include <iostream>
using namespace std;
int main(){
    int N;
    cin >> N;
    cout << (1000 - N % 1000) % 1000 << endl;
}
```

In some languages, "negative integer `%` positive integer" does not result in a negative number. In such languages, the implementation can be simplified as the following Ruby sample code.

```ruby
puts -gets.to_i % 1000
```

B: Judge Status

(Editorial: evima)

If the number of test case N is a constant, for example, 3, then we only have to use if statement. However, in reality N is a large non-constant number between 1 and 100 thousands, so it's time to use for statements.

The most simple way is to implement the content of the following pseudocode.

```
N := input
C0, C1, C2, C3 := 0, 0, 0, 0
for i = 0, ..., N-1:
    S := input
    if S == "AC": C0 = C0 + 1
    else if S == "WA": C1 = C1 + 1
    else if S == "TLE": C2 = C2 + 1
    else: C3 = C3 + 1
print C0, C1, C2, C3
```

In many languages, this problem can be solved by adjusting this pseudocode to those languages. Note that extra attention may be needed to be paid when comparing strings in some languages. For detailed information for each language, use searching engine with keywords "[language name] for statement" or "[language name] string comparison". The following is the pseudocode transpiled into Java.

```
import java.util.*;
public class Main {
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        int N = sc.nextInt();
        int C0 = 0, C1 = 0, C2 = 0, C3 = 0;
        for(int i = 0; i < N; ++i){
            String S = sc.nextLine();
            if(S.equals("AC")) ++C0;
            else if(S.equals("WA")) ++C1;
            else if(S.equals("TLE")) ++C2;
```

```
12              else ++C3;
13          }
14          System.out.println("AC x " + C0);
15          System.out.println("WA x " + C1);
16          System.out.println("TLE x " + C2);
17          System.out.println("RE x " + C3);
18      }
19  }
```

Of course you can write a code that looks different to the pseudocode above. As an example, we will show a sample code in Python.

```
1  N = int(input())
2  s = [input() for i in range(N)]
3  for v in ['AC', 'WA', 'TLE', 'RE']:
4      print('{0} x {1}'.format(v, s.count(v)))
```

C: H and V

(Editorial: evima)

In short, you are asked to answer $H + W$ number of Yes-No questions, each of which is like "Will we paint red the first row? Will we paint the second row? $\cdots$ Will we paint the H-th row? Will we paint the first column? Will we paint the second column? $\cdots$ Will we paint the W-th row?", the answers of which will be like "Yes, No, Yes, Yes, $\cdots$" and so on. There are 2^{H+W} number of such Yes-No sequences, and under the constraint $H, W \leq 6$ for this problem, there are at most $2^{12} = 4096$ of them. So we do not have to care about time limit even if we try all the possible Yes-No sequences, whose number is fairly small, and actually check if K black squares remain.

All the left is a programming task to enumerate Yes-No sequences. We may use recursive functions, but here we consider mapping Yes-No sequences to binary integers. Hereinafter the number starting with `0b` is a binary integer (e.g. `0b0101` $= 4 + 1 = 5$).

Let us consider only about rows for a while. For example, when $H = 3$, there are $2^3 = 8$ number of Yes-No sequences in total that have to be considered. By associating "No, No, No" with `0b000` $= 0$, "Yes, No, No" with `0b001` $= 1$ "No, Yes, No" with `0b010` $= 2$, "Yes, Yes, No" with `0b011` $= 3 \cdots$ and "Yes, Yes, Yes" with `0b111` $= 7$, 2^3 number of Yes-No sequences and integers from 0 to $2^3 - 1 = 7$ are mapped one-by-one. (The order is reversed because, in binary notation, the larger digits are written to the left).

Similarly, no matter how many rows are there, there is a one-to-one correspondence between 2^H number of Yes-No sequence and integers from 0 to $2^H - 1$, by associating the i-th element of a Yes-No sequence to the 2^i's digit of binary integer. Similar correspondence can be formed between 2^W number of Yes-No sequence and integers from 0 to $2^W - 1$. Therefore we can enumerate all the possible 2^{H+W} scenarios with a dual loop like the following pseudocode.

```
H, W, K, c := input
ans := 0
for maskR = 0, ..., (1 << H) - 1:
    for maskC = 0, ..., (1 << W) - 1:
        black := 0
        for i = 0, ..., H-1:
            for j = 0, ..., W-1:
```

```
                if ((maskR >> i) & 1) == 0 and ((maskC >> j) & 1) == 0
                and c[i][j] == '#': black = black + 1
        if black == K: ans = ans + 1
print ans
```

D: Chat in a Circle

Sort A_i in the increasing order. Then, we will show that $\sum_{k=1}^{N-1} A_{N-\lfloor k/2 \rfloor}$ is the answer (here, $\sum$ is a symbol that denotes summation. For example, in the expression above, it denotes the sum of all $A_{N-\lfloor k/2 \rfloor}$ for $k = 1, 2, \ldots, N-1$. It is a useful notation, so learn it now!).

One of the common strategy to prove the maximum value is to show that "1. the value can be achieved" and that "2. it cannot be exceeded", which will be used here too.

How can "the value can be achieved"?

Move them in the decreasing order of friendliness. Put the $2i+1$-st and $2i+2$-nd person in each side of $i+1$-th person, then it always hold that "if both sides of a person A is not occupied with the people who will be put there, then then there are people who came earlier than person A in his both sides," so it can be said that the value above can be achieved.

Why can "it not be exceeded"?

First, we will show that we can always assume that the players are moved in the decreasing order of friendliness. Suppose that there exists a k such that $a_1 \geq a_2 \geq \cdots \geq a_k < a_{k+1}$. Then,

- IF the $k+1$-st person does not cut in to next to k-th person, then whether or not swapping them is none of their business.
- IF the $k+1$-st person cuts in to next to k-th person, let us assume that they are aligned in the order of x, a_k, a_{k+1}, y. Then, $\min(x, y) + \min(a_k, y) \leq \min(x, y) + \min(x, a_{k+1})$, so swapping them does not decrease the total sum.

Therefore, we can always move them in the decreasing order of friendliness, by swapping them in manner of bubble sort. Hereinafter we will only consider such situation.

A_N contribute to the total comfort at most once. How about the other people? If a person cut into left to A_k, then after that there is always a person who came later than A_k to the left of A_k. The right side is the same, so ultimately A_k contribute to it at most twice.

Suppose that you are asked to choose $N-1$ elements from a multiset consisting of one A_N (maximum value) and two $A_1 \sim A_{N-1}$ each. Then, the maximum some of the chosen value is $\sum_{k=1}^{N-1} A_{N-\lfloor k/2 \rfloor}$ (which is written in the beginning). Therefore, the answer for this problem cannot exceed this.

Thus, this solution has been justified. The problem can be solved in a total of $\Theta(N \log N)$ time,

where the sort is the bottleneck. We can also achieve a total of $\Theta(N)$ time with the technique called Median of Medians, but it is beyond the editorial, so we will just introduce the name.

E: Multiplication4

(Editorial : kyopro_friends)

First, it is obvious when $K = N$. Also, when all A_i are negative and K is an odd number, then the answer is also negative. In such case, it is optimal to take K elements with the smallest absolute value. Otherwise, we can always choose such elements that their product is non-negative.

(Proof: consider the product of K largest elements. If it is non-negative, it's OK. If negative, remove the largest element from the chosen element (which is positive by assumption), and instead add the smallest element that was not chosen (which is negative by assumption); then the product will be positive)

Therefore, all that left is to maximize the absolute value, while paying attention to the sign.

Solution 1: Adjust the sign in the end

Sort $\{A_i\}$ in the decreasing order of absolute values, and let P be the product of the first K element. If P is non-negative, then that is the answer. Otherwise, to make the product non-negative, you have to perform either of the following two operations:

(1) Remove a negative element, and add a non-negative element
(2) Remove a positive element, and add a negative element

In order to maximize the absolute value of product after the operation, it is optimal to remove the element with the smallest absolute value as possible, and add the element with the largest absolute value as possible, so it can be uniquely determined which element to choose. Paying attention to the way of comparison [*1] choose the operation which results in larger product. Note that one of those operations may not be available.

Solution 2: Maximize the product while keeping it non-negative

Let S be an empty set. Let us consider adding K elements to S and maximize the product, while keeping the condition that "the product of S is non-negative." Any operation of adding elements while keeping the product of S non-negative can be decomposed to a repetition of two kinds of operations: "add one non-negative element" or "add to negative elements". Considering this, we can find the answer by the following algorithm.

[*1] Consider two operations of "removing x_i and adding y_i" ($i = 1, 2$); which of them results in a larger product? We can decide by checking if $|P/x_1 * y_1| \leq |P/x_2 * y_2|$, or equivalently, $|x_2 * y_1| \leq |x_1 * y_2|$, which can be calculated in the range of 64-bit integer under the constraints in this problem.

Repeat the following operations until the number of elements of S becomes K.
Operation: Compare "the product of two non-negative elements x_1, x_2 with the largest absolute value which are not chosen yet" and "the product of two elements negative y_1, y_2 with the largest absolute value which are not chosen yet". If the former is the larger, or if $|S| = K - 1$, then add x_1 to S. Otherwise, add y_1, y_2 to S.

This can be performed fast by splitting $\{A_i\}$ into non-negative and negative numbers, and sorting them in the decreasing order of their absolute value respectively beforehand. Note that the number of "non-negative / negative elements which are not chosen yet" may be less than two.

F: Intervals on Tree

To actually perform simulation, fix L and increment R while managing the connected components with Union Find tree etc., then the number of connected components can actually be counted for all L, R. However it needs a total of $O(N^2\alpha(N))$ time, which is slow.

To make use of the fact that the original graph is a tree, let's first consider the value $f(L, R)$ for some fixed L, R. Each connected component of subgraph induced by $S = \{x | L \leq x \leq R\}$ is a tree (i.e. the induced subgraph is a forest). In a tree, the number of vertices is the number of edges $+1$, so in a forest, the number of vertices = the number of edges + the number of connected components (trees).

Therefore, by regarding that each vertex and edge contributes to the number of connected components (= the number of vertices − the number of edges), instead of counting the sum of the numbers of connected components, we can consider how many times each vertex and edge is added or subtracted. It can be calculated by, for each vertex, counting the number of S that contains the vertex, and for each edge, counting the number of S that contains both endpoints, so actually the value can be directly calculated while reading the input, where the time complexity is $O(N)$.

ABC 174 解説

evima, kyopro_friends, ynymxiaolongbao

2020 年 8 月 2 日

For International Readers: English editorial starts on page 8.

A: Air Conditioner

(解説: evima)

「何をすればいいか全く分からない！」という場合、まずは「practice contest」(`https://atcoder.jp/contests/practice/`) の問題 A「はじめてのあっとこーだー」をお試しください。言語ごとに解答例が掲載されています。

今回の問題に移ります。突然ですが、架空の言語で正解例を示します。[*1]

```
read X as int
if X >= 30:
    write "Yes"
else:
    write "No"
```

これをお使いの言語に合わせて書き換えれば問題を解けます。以下、各行について述べます。

1 行目: 入力された気温 X を整数として読み込みます。「はじめての〜」で整数 a を読み込むのと同様に行えるはずです。

3, 5 行目: 文字列 `Yes` または `No` を出力します。「はじめての〜」で入力された文字列 s を出力するソースコードの `s` を `"Yes"` と書き換えればほとんどの言語で動作するはずです。もし動作しなければ、検索エンジンで「(言語名) 文字列 出力」などと検索してください。(次頁へ続く)

[*1] なぜそのようなことを？というと、AtCoder では数十の言語が使用可能で、それらをできるだけ平等に扱うためです。

2, 4 行目: 読み込んだ整数 X が 30 以上であれば 3 行目、そうでなければ 5 行目を実行させます。言語ごとの if 文の書き方については、検索エンジンで「(言語名) if 文」などと検索してください。整数の大小比較については、`X >= 30` と書けば多くの言語で動作するはずですが、もし動作しなければ検索エンジンで「(言語名) 大小比較」などと検索してください。

B: Distance

(解説: evima)

前問と同様に架空の言語で正解例を示し、各行について述べます。

```
1 read N, D as int
2 ans = 0
3 for i = 1, ..., N:
4     read X, Y as int
5     if X * X + Y * Y <= D * D:
6         ans += 1
7 write ans
```

1 行目: 入力された値 N, D を整数として読み込みます。

2 行目: カウンターとして用いる変数 ans を宣言し、0 で初期化します。

3 行目: 4 行目から 6 行目までを N 回繰り返し実行させます。言語ごとの for 文の書き方については、検索エンジンで「(言語名) for 文」などと検索してください。

4 行目: i 個目の点の座標 X_i, Y_i を整数変数 X, Y に読み込みます。N 個すべての点の座標を配列に保存する手もありますが、この解答例では処理した点の座標は「忘れる」ことにしています。

5, 6 行目: 距離 $\sqrt{X^2 + Y^2}$ が D 以下であるか判定し、そうであれば ans に 1 を加算します。ただし、これら二つの値を直接比較する代わりに両者の二乗同士を比較しています (入力される D の値は 0 以上であることが保証されているため、単に二乗同士を比較しても結果は変わりません)。一般に、計算機上での実数の計算は浮動小数による近似計算であり、その結果には誤差が含まれる[*2]ため、可能な限り避けるべきです。

7 行目: カウンターとして用いた変数 ans の最終的な値を出力します。

[*2] 例えば、コードテストで `print(2**0.5 + 2**0.5 == 18**0.5)` (Python) を実行してみてください

C: Repsept

(解説: evima)

与えられた数列の i 項目は $7 \times (1 + 10 + 10^2 + \ldots + 10^{i-1}) = \frac{7(10^i-1)}{9}$ と書ける (等比数列の和) ため、求めるべきものは $7(10^i - 1)$ が $9K$ で割り切れるような最小の正の整数 i です。

K が 7 の倍数である場合は、代わりに $10^i - 1$ が $\frac{9K}{7}$ で割り切れるかを考えても同じことで、そうでない場合は、代わりに $10^i - 1$ が $9K$ で割り切れるかを考えても同じことです。したがって、整数 L を、K が 7 の倍数であれば $L = \frac{9K}{7}$、そうでなければ $L = 9K$ と定義し、$10^i - 1$ が L で割り切れるような最小の i、すなわち 10^i を L で割った余りが 1 であるような最小の正の整数 i を求めればよいことになります。

L が 2 の倍数であるなら、どの正の整数 i に対しても 10^i は 2 の倍数であるため、L で割った余りが 1 となることはありません。L が 5 の倍数である場合も同様です。このいずれにも該当しない場合、すなわち 10 と L が互いに素である場合、オイラーの定理より $10^{\varphi(L)} \equiv 1 \pmod{L}$ [*3] が成立します。すなわち、$i = 1, 2, \ldots,$ と順に検討していけば遅くとも $i = L(\leq 9000000)$ までには求めるべき整数が見つかる (例えば $i = 10^{100}$ でようやく見つかるといったことはない) ことが保証されているため、あとは実際に $10^1, 10^2, \ldots$ を実際に L で割った余りを計算していけば十分速く解を求められます。

ただし、10^{999982} などといった巨大な値を直接 L で割ろうとするべきではありません。その代わりに、10^i を L で割った余りを 10 倍して L で割れば、10^{i+1} を L で割った余りが求まります。

なお、以上の数学的考察をせずとも、「答えが -1 でないならどうせある程度小さいだろう、でなければこの “枠” には難しすぎる」と予想してしまい、$7, 77, \ldots$ を K で割った余りを上の段落で述べたような方針で計算していくことで答えを求めることも可能ではあります。

[*3] $\varphi(n)$ はオイラーのφ関数、すなわち n と互いに素であるような 1 以上 n 以下の整数の個数です

D: Alter Altar

(解説: evima)

「赤い石の左隣に置かれた白い石」がない状態では、赤い石 A の左隣に石 B があれば B は必ず赤で、B の左隣に石 C があれば C もまた赤で、同様にして A より左にある石は全て赤です。このことから、目標が満たされた状態は次の $N+1$ 通りしか存在しません: `WWW...WWW` (すべて白)、`RWW...WWW` (左端の 1 個のみ赤)、`RRW...WWW` (左端の 2 個のみ赤)、$\cdots$、`RRR...RRW` (左端の $N-1$ 個のみ赤)、`RRR...RRR` (すべて赤)。これをもとに、目標を次のように言い換えます。

新たな目標

操作を行い始める前に、祭壇上のいずれか一箇所を選んで仕切りを置く (つまり、仕切りは石と石の間か、どの石よりも左側か、どの石よりも右側のいずれかに置かれる)。目標は、仕切りより左側の石をすべて赤に、仕切りより右側の石をすべて白にすることである。

仕切りの位置がすでに決まっているとします。すると、操作の性質上、もはや仕切りの左右に赤い石と白い石がそれぞれ何個あるか以外は目標達成と関係しません。仕切りより左側の白い石の個数を W、仕切りより右側の赤い石の個数を R とすると、目標は W と R をともに 0 とすることです。一回の操作で W が 1 より多く減ることはないため、目標達成には少なくとも W 回の操作が必要です。同様に少なくとも R 回の操作が必要であるため、目標達成には少なくとも $\max(W,R)$ [*4] 回の操作が必要です。逆に、次のようにすれば $\max(W,R)$ 回の操作で目標を達成することができます。

- $W \leq R$ の場合: 仕切りより左側の白い石と右側の赤い石の入れ替えを W 回行い、右側に残った $R-W$ 個の赤い石を一個ずつ白くする。合計操作回数: R。
- $W > R$ の場合: 仕切りより左側の白い石と右側の赤い石の入れ替えを R 回行い、左側に残った $W-R$ 個の白い石を一個ずつ赤くする。合計操作回数: W。

よって、仕切りの位置が固定された場合の必要な最小操作回数が $\max(W,R)$ であることがわかりました。あとは、仕切りの位置の候補 $N+1$ 通りをすべて試し、それぞれについて $\max(W,R)$ の値を求めれば、それらのうち最小のものが求めるべき最小操作回数です。

ただし、入力される N の上限はやや大きく、N 個の石すべてを $N+1$ 回見直す時間はありません。これに対処する方法の一つは、あらかじめ白い石の総数と赤い石の総数を数えておき、仕切りを最も左の位置から一つずつ右にずらしていくことです。仕切りが一つ右にずれた際に起こることは、W が 1 増えるか、R が 1 減るかのいずれかであることを用いれば、線形時間で処理が完了します。

[*4] $\max(a,b)$ は a と b のうち小さくない方を表します。例: $\max(3,5)=\max(5,3)=\max(5,5)=5$

E: Logs

（解説：ynymxiaolongbao）

答えが X 以下であるか？という問題を考えます。この問題は言い換えれば、 K 回以内のカットですべての丸太の長さを X 以下にすることができるか？という問題になります。はじめ長さ A_i の丸太を切り分けて長さ X 以下にするためには、$\lceil \frac{A_i}{X} \rceil - 1$ 回切る必要があります。そして、この回数の合計が K 以下であれば答えは Yes、そうでなければ答えは No です。

二分探索を用いて、答えが X 以下であるか？という問題の答えが Yes になる最も小さな整数 X を求め、それを出力すれば良いです。計算量は $O(Nlog(maxA))$ になります。

F: Range Set Query

区間に対するクエリに答えるというと、Range Minimum Query などに用いられる SegmentTree が知られていますが、集合のマージには時間がかかってしまいます。

全体で左から k 番目かそれより左にある玉だけに注目したとき、それぞれの色で最も右にある玉を k についての良い玉と呼ぶことにします。すると、i 番目のクエリに対する答えは $[l_i, r_i]$ にある r_i についての良い玉の個数になります。

クエリを r_i の昇順にソートし、良い玉の集合を管理しながらクエリに答えて行きます。良い玉の集合は以下の二つのデータ構造を用いて管理すると良いです。

- 色 i の良い玉が存在するか、存在するならその場所はどこかを管理し、集合の更新をするための配列
- 良い玉のある場所が 1 、それ以外が 0 であるような数列を管理し、区間和を取ることでクエリに答えるための Fenwick Tree

計算量は $O(N + QlogN)$ です。
解答例
https://atcoder.jp/contests/abc174/submissions/15644133

A: Air Conditioner

(Editorial: evima)

If you are like "I have no idea where to start¡', then first try Problem A "Welcome to AtCoder" in the "practice contest" (`https://atcoder.jp/contests/practice/`). There you can find sample codes for each language.

Let's move on to the problem in this contest. It's a bit abrupt, but here is a sample answer in an imaginary language.[*5]

```
1 read X as int
2 if X >= 30:
3     write "Yes"
4 else:
5     write "No"
```

If you rewrite this to the language you use, you will be able to solve the problem. The following is explanation for each line.

The 1st line: read the input temperature X as an integer. It should be done in the same way as reading integer a in "Welcome to—".

The 3rd and 5th lines: output string `Yes` or `No`. You can achieve it in most language by replacing `s` to `"Yes"` in the source code which outputs the input string s in the "Welcome to—". If it doesn't work, use a search engine with the keywords like "(language name) string output". (Continued on the next page)

The second and the fourth line: If the input integer X is more than or equal to 30, then the third line will be executed; otherwise the fifth line will be. For the syntax of if statements in each language, use search engine with keywords "(language name) if statement". As for integer comparisons, `X >= 30` should work in the most languages, but if it doesn't, use a search engine with the keywords like "(language name) comparing".

[*5] Some of you may wonder why do we do such thing, but actually this is to treat tens of languages available in AtCoder equally.

B: Distance

(Editorial: evima)

As in the previous problem, we will show a sample answer in an imaginary language, and explain each line.

```
1 read N, D as int
2 ans = 0
3 for i = 1, ..., N:
4     read X, Y as int
5     if X * X + Y * Y <= D * D:
6         ans += 1
7 write ans
```

1st line: read the input values N, D as an integer.

2nd line: Declare a variable *ans* which will be used as a counter, and initialize it with 0.

3rd line: repeat the 4th to 6th line for N times. For the syntax of for statements in each language, use search engine with keywords "(language name) for statement".

4th line: input the coordinates X_i, Y_i of the i-th point to the variables X, Y. You may store all the coordinates of points to an array, but in this sample code, we would rather "forget" the coordinates that has been processed.

5th and 6th line: check if distance $\sqrt{X^2 + Y^2}$ is less than or equal to D, and add 1 to *ans* in such case. However, instead of directly comparing those two values, the squares of those values are compared (Since it is guaranteed that input D is more than or equal to 0, we can simply compare the squares without changing the result). In general, calculations of real numbers on computers are approximation by floating point numbers, and they may contain precision errors[*6], so it has to be avoided as much as possible.

7th line: output the ultimate value of the counter variable *ans*.

[*6] for example, try `print(2**0.5 + 2**0.5 + 2**0.5 == 18**0.5)`(Python) in the Code Test

C: Repsept

(Editorial: evima)

Since the i-th term of the given sequence can be written as $7 \times (1 + 10 + 10^2 + \ldots + 10^{i-1}) = \frac{7(10^i-1)}{9}$ (sum of geometric series), so the desired answer is the minimum positive integer i such that $7(10^i - 1)$ is divisible by $9K$.

If K is a multiple of 7, you can consider whether $10^i - 1$ is divisible by $\frac{9K}{7}$ instead, and otherwise, you can consider whether $10^i - 1$ is divisible by $9K$. Therefore, let us define an integer L in such way that $L = \frac{9K}{7}$ if K is a multiple of 7, or $L = 9K$ otherwise, then it is sufficient to find the minimum positive integer i such that $10^i - 1$ is divisible by L, that is, such that the remainder of 10^i divided by L is 1.

If L is a multiple of 2, then 10^i is a multiple of 2 for any positive integer i, so its remainder divided by L will never be 1. The same applies when L is a multiple of 5. If neither of them is satisfied, then by Euler's theorem, it holds that $10^{\varphi(L)} \equiv 1 \pmod{L}$[*7]. Therefore, if you examine for each $i = 1, 2, \ldots$, then you will be able to find the desired integer until $i = L(\leq 9000000)$ at the latest (that is, you will never find it at, for example, $i = 10^{100}$ for the first time), so by actually dividing $10^1, 10^2, \ldots$ by L in this order, the answer can be found fast enough.

However, you should not try to divide 10^{999982} by L directly. Instead, multiply the remainder of 10^i divided by L by 10 and then divide it by L, and you will find the remainder of 10^{i+1} divided by L.

Note that, without trying the mathematical observation above, you might assume that "if the answer is not -1, then the answer will be fairly small; otherwise it is too difficult for this 'position'," and calculate the remainder of $7, 77, \ldots$ by K as described above. This way you can find the answer too.

[*7] $\varphi(n)$ is Euler's φ-function, that is, the number of integers between 1 and n, inclusive, such that is coprime with n

D: Alter Altar

(Editorial: evima)

When there are no "white stone placed to the immediate left of a red stone," if there is a red stone A and another stone B in the immediate left to stone A, then stone B is always B, and if stone C is in the immediate left to stone B, then stone C is also red, and similarly all the stones left to stone A are red. Therefore, there are only $N + 1$ ways to achieve the goal: `WWW...WWW` (all stones are white), `RWW...WWW` (only the leftmost stone is red), `RRW...WWW` (only the leftmost two stones are red), $\cdots$, `RRR...RRW` (only the leftmost $N - 1$ stones are red) or `RRR...RRR` (all stones are red). Based on this, we define a new goal as follows:

New goal

Before start performing operations, choose one position to place a partition (that is, the partition will be placed either between two stones, or in the left of any stones, or in the right of any stones). Th goal is to make all the stones left to the partition to be red and all the stones right to the partition to be white.

Assume that the position of the partition is fixed. Then, by the property of the operations, it doesn't matter anymore other than how many red and white stones are there. Let W be the number of white stones in the left of the partition, and R be the number of red stones in the right of the partition, then the target is to make both W and R zero. Since W does not decrease by more than one, it requires at least W operations to achieve the goal. Likewise, it requires at least R operations, so it requires at least $\max(W, R)$ [*8] times of operations. Conversely, the following procedure enables to achieve the goal in $\max(W, R)$ operations:

- if $W \leq R$: swap each white stone in the left of the partition and each red stone in the right of the partition for W times, and then change the remaining $R - W$ red stones to white, one by one. Total number of operations: R.
- if $W \leq R$: swap each white stone in the left of the partition and each red stone in the right of the partition for R times, and then change the remaining $W - R$ white stones to red, one by one. Total number of operations: W.

Therefore, now we can see that the minimum number of operations when the partition is fixed is $\max(W, R)$. All that left is to try all the $N + 1$ candidates of partition's position and find

[*8] $\max(a, b)$ denotes the biggest of a and b. e.g. $\max(3, 5) = \max(5, 3) = \max(5, 5) = 5$

$\max(W, R)$ for each of them; then the desired minimum number of operations is the smallest of them.

However, the input N is somewhat large, so there is no time to inspect all the N stones for $N+1$ times. One way to handle this is to count the total number of white and red stones beforehand, and shift the partition from the leftmost position one by one. Every time the partition shift to the right, either W increases by one or R decreases by one. Therefore, the overall process finishes in a linear time.

E: Logs

(Editorial : ynymxiaolongbao)

Consider the problem whether the answer is less than or equal to X. This problem can be rephrased as whether all the logs can be cut down to the length less than or equal to X within at most K cuts. In order to cut a log of initial length A_i into those of length less than or equal to X, at least $\lceil \frac{A_i}{X} \rceil - 1$ cuts are required. If the sum of them does not exceed K, then the answer is Yes, otherwise the answer is No.

It is sufficient to find the minimum integer X such that the answer to the question "is the answer less than or equal to X?" is Yes. The total computation time is $O(Nlog(maxA))$.

F: Range Set Query

When it comes to answering queries for segments, well known is SegmentTree that is used for problems like Range Minimum Query, but merging sets requires long time.

For each k, let us call a ball "good ball for k" if it is one of the leftmost k balls such that no other ball in the right of the ball out of those k balls has the same color to that ball. Then, the answer for the i-th query is the number of good ball for r_i that is in $[l_i, r_i]$.

Sort the queries in the increasing order of r_i, and answer to the query while managing the set of good balls. Set of good balls can be managed efficiently by the following two data structures:

- an array that manages whether a good ball of color i exists, and its position (if exists), which is used to update the set
- a Fenwick tree that manage an array whose element is 1 if there is a good ball at that position, or otherwise 0, that enables to answer the queries by taking segment sums

The total time complexity is $O(N + QlogN)$.

Sample code

https://atcoder.jp/contests/abc174/submissions/15644133